Firecast SDK

SUMÁRIO

Introdução
Primeiros passos
Histórico de Versões
Tutoriais
- Criar um modelo de ficha à partir de um PDF
  - Observações
- Criar campos calculados em fichas
  - Tutorial 1 - Campo calculado simples
  - Tutorial 2 - Criar um campo calculado que depende do valor de outros 3 campos
- Criar abas para a ficha
- Permitir o usuário escolher uma imagem dentro da ficha
- Criar uma lista dinâmica na ficha
- Fazer uma ficha enviar rolagens para a mesa
  - O que o código acima faz?
  - Mudanças que você pode querer fazer no código acima
- Criar uma lista dinâmica + painel de detalhes
  - Dois arquivos Lua Form (extensão ".LFM")
  - Sobre a implementação
  - Lista Dinâmica + Painel de Detalhes alternativo
  - Incrementando o código: Selecionar o primeiro item ao carregar os dados
- Criar um botão que esconde itens de uma lista dinâmica
  - Começando....
  - Próximos passos
  - Resultado final
  - Código fonte final (sem os comentários)
- Criar uma janela acoplável de mesa
- Criando um plug-in para vários idiomas

Referências
- Projetos de Plug-in
  - Criando um novo projeto de plug-in
  - Testando o projeto de plug-in
  - Disponibilizando um plug-in
- Arquivos em um projeto de plug-in
  - O arquivo "module.xml"
  - Embedded font files
- Ferramentas do SDK 3

- Ferramenta rdk
- Arrastando e Soltando Informações / Drag And Drop
  - Fluxo de acontecimento
  - Informações e Ações em uma operação drag-and-drop
  - Exemplos
- Chat messaging structures
  - Entity table
  - Impersonation table
  - LogRec table
  - Medium table
  - MsgContent table
  - MsgUpdate table
  - MsgVisibility table
  - QueryLogRecsParams table
  - QueryLogRecsResponse table
  - SendMessageParameters table
  - TalemarkBlockStyle table
  - TalemarkOptions table
  - TalemarkTag table
  - Timestamp table
- HD Virtual
  - Principais características do HD Virtual
  - Sobre o sistema de arquivos no HD Virtual
  - Manipulando o HD Virtual:
- Modelos de Fichas
  - Criando um novo modelo de ficha
  - Criando um novo modelo de ficha - Exemplo passo a passo
- NodeDatabase
  - Conteúdo:
- Sandboxes de Plug-ins
  - Veja também:
- Interfaces Visuais (Lua Forms)
  - Principais características do Lua Form
  - Exemplo 1 – Olá mundo
  - Exemplo 2 – Um layout, um label, um edit e um botão
  - Guias
  - Tags suportadas pelo Lua Form
  - Frames
- Bibliotecas LUA

        Async library
        Audio library
        Biblioteca Dialogs
        Biblioteca FireDrive
        Biblioteca GUI
        Biblioteca Locale
        Log library
        Biblioteca Internet
        Biblioteca NDB
        Biblioteca Firecast
        Biblioteca Firecast.Messaging
        Biblioteca Firecast.Plugins
        Biblioteca Firecast.Requests
        Biblioteca SceneLib
        Biblioteca System
        Biblioteca Utils
        Biblioteca VHD
Macros
    Criando e editando suas macros
    Tipos de macros
    Macros em Lua
    Biblioteca para Macros em Lua
        Variáveis
        Funções
Apêndice
    A linguagem de programação LUA
        Veja também:
    Documentos XML
        Lidando com caracteres reservados do XML
    FAQ - Perguntas Frequentes

INTRODUÇÃO

(Conteúdo desta página não consta no documento original.)

PRIMEIROS PASSOS

(Conteúdo desta página não consta no documento original.)

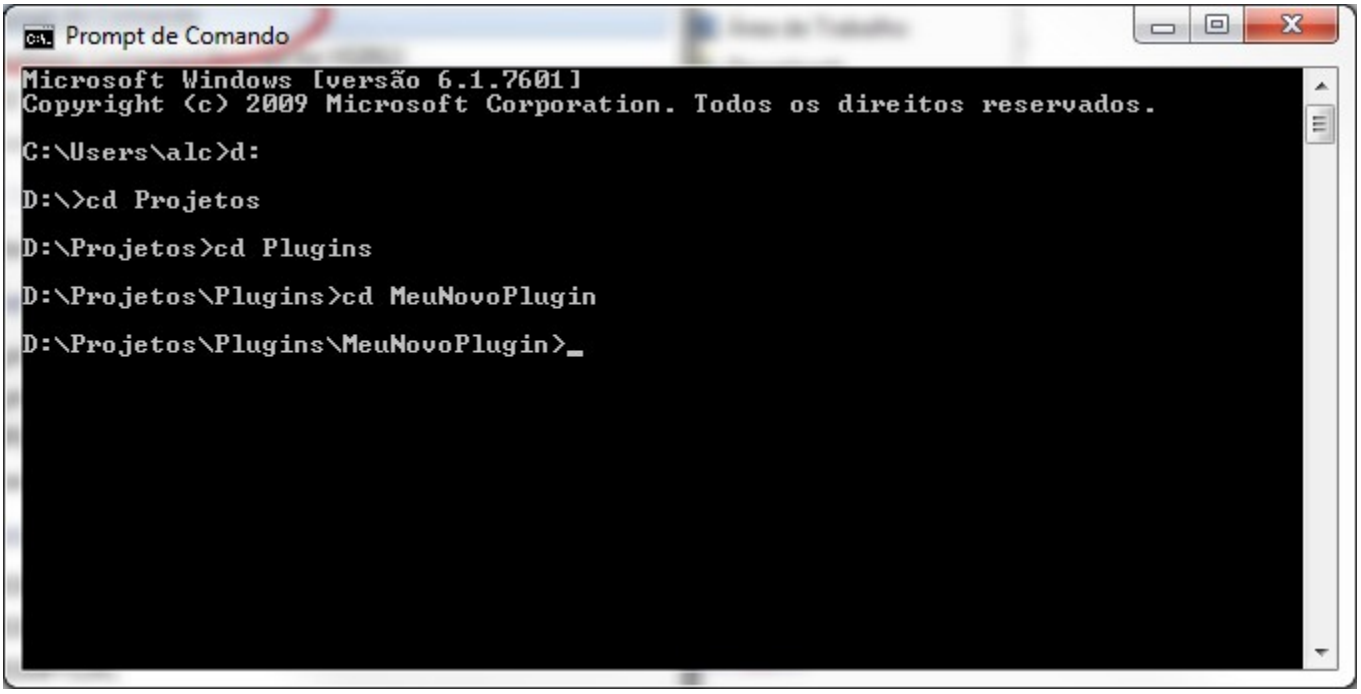
HISTÓRICO DE VERSÕES

(Conteúdo desta página não consta no documento original.)

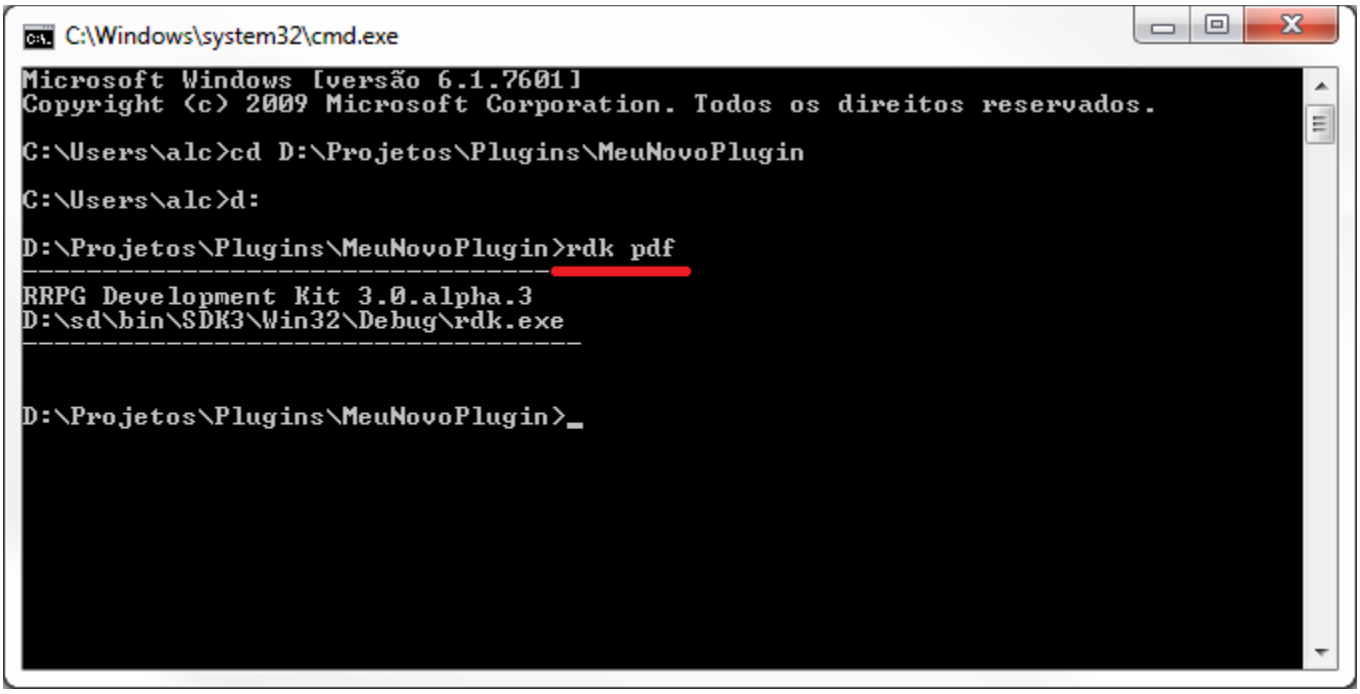
TUTORIAIS

Criar um modelo de ficha à partir de um PDF

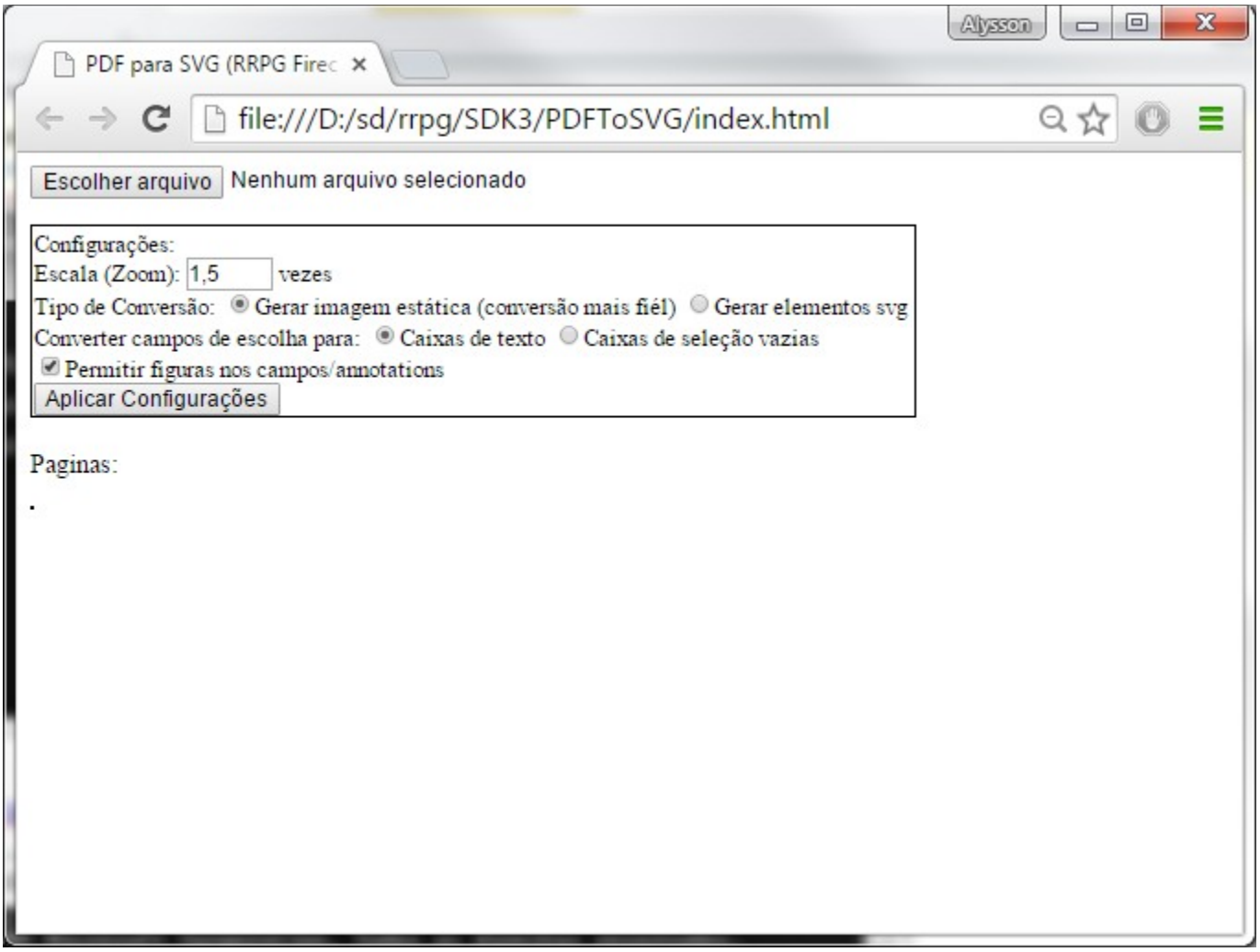
1. Você deve ter um projeto de plug-in já criado. Se você ainda não tem um projeto de plug-in, leia "Criando um novo projeto de plug-in" e crie o seu antes (é bem fácil =])
2. Abra o prompt de comando e navegue até a pasta de seu projeto de plug-in.



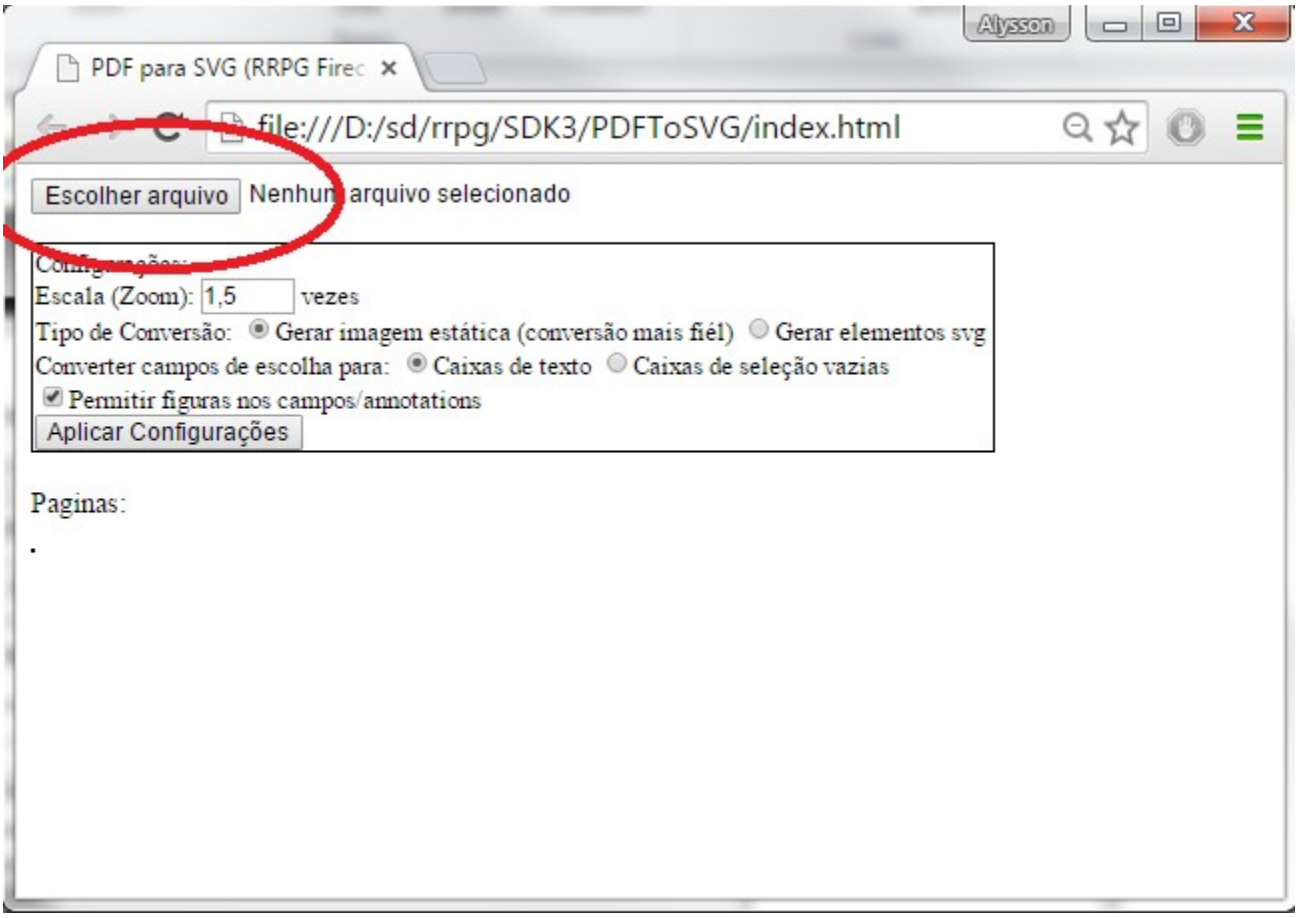
3. Execute o comando "rdk pdf" sem aspas no prompt de comando



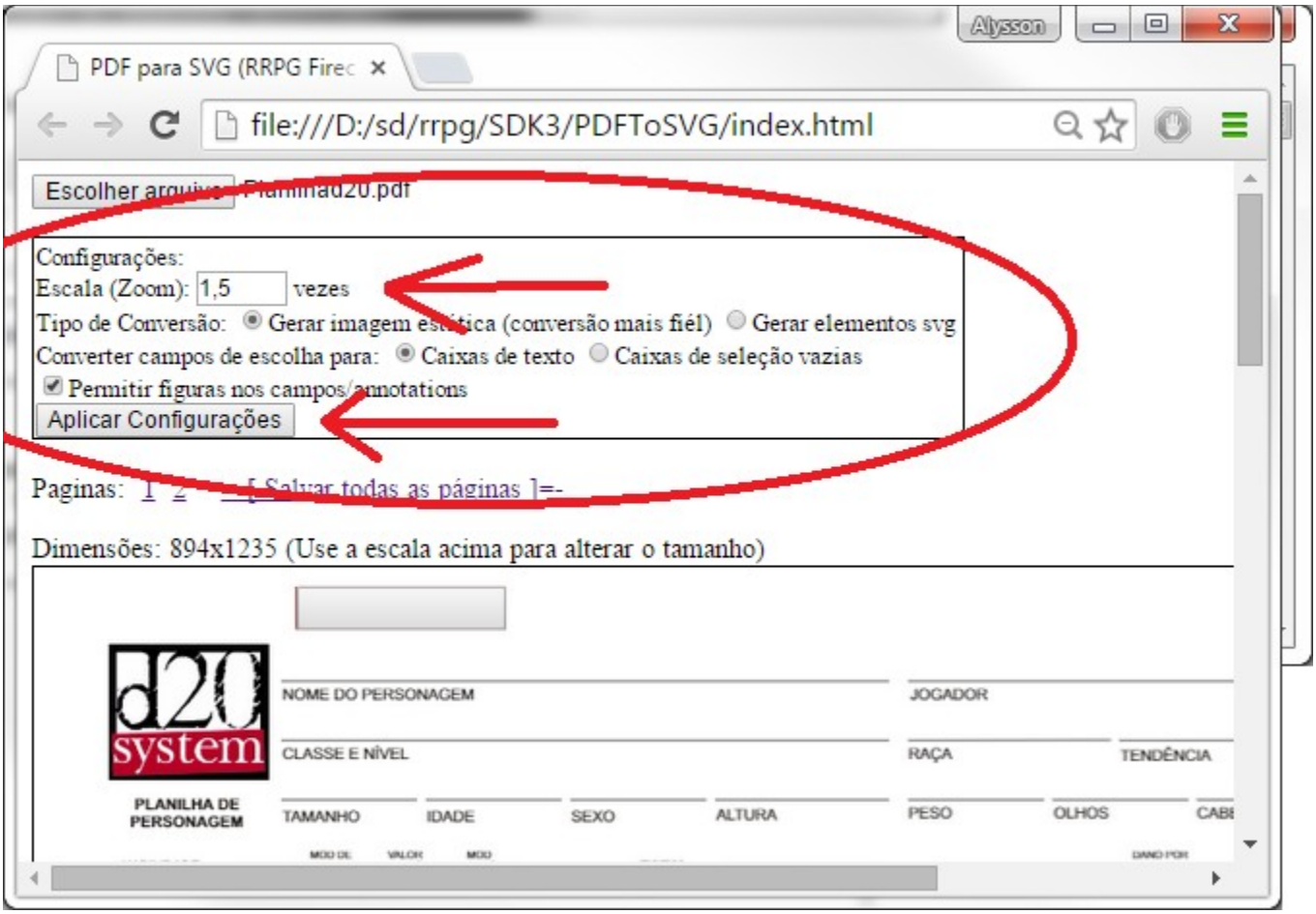
Uma página como esta se abrirá no navegador de internet (Sempre utilize um browser mais atualizado. Experimente o Google Chrome):



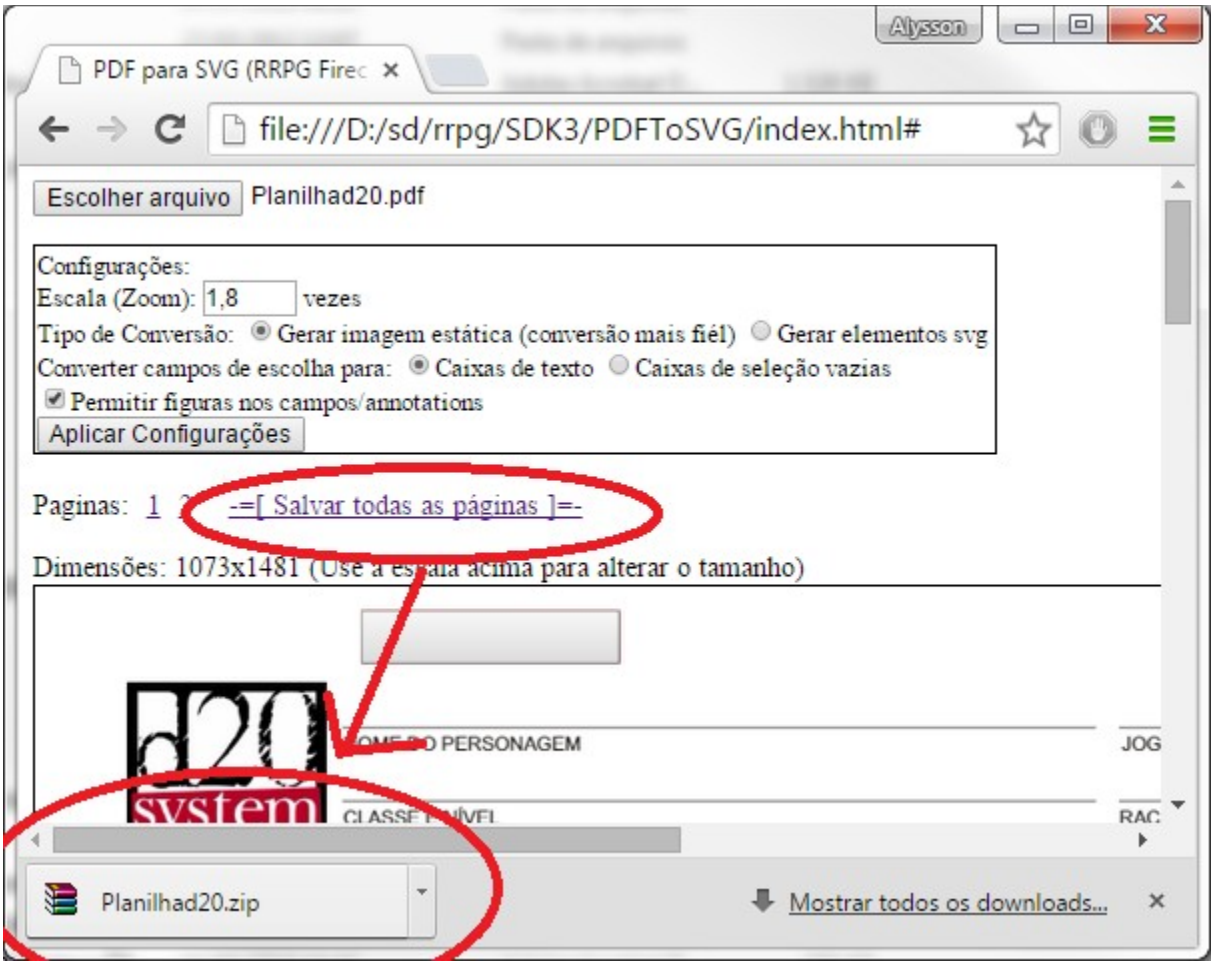
4. Clique em "escolher arquivo" e selecione o PDF que deseja converter.



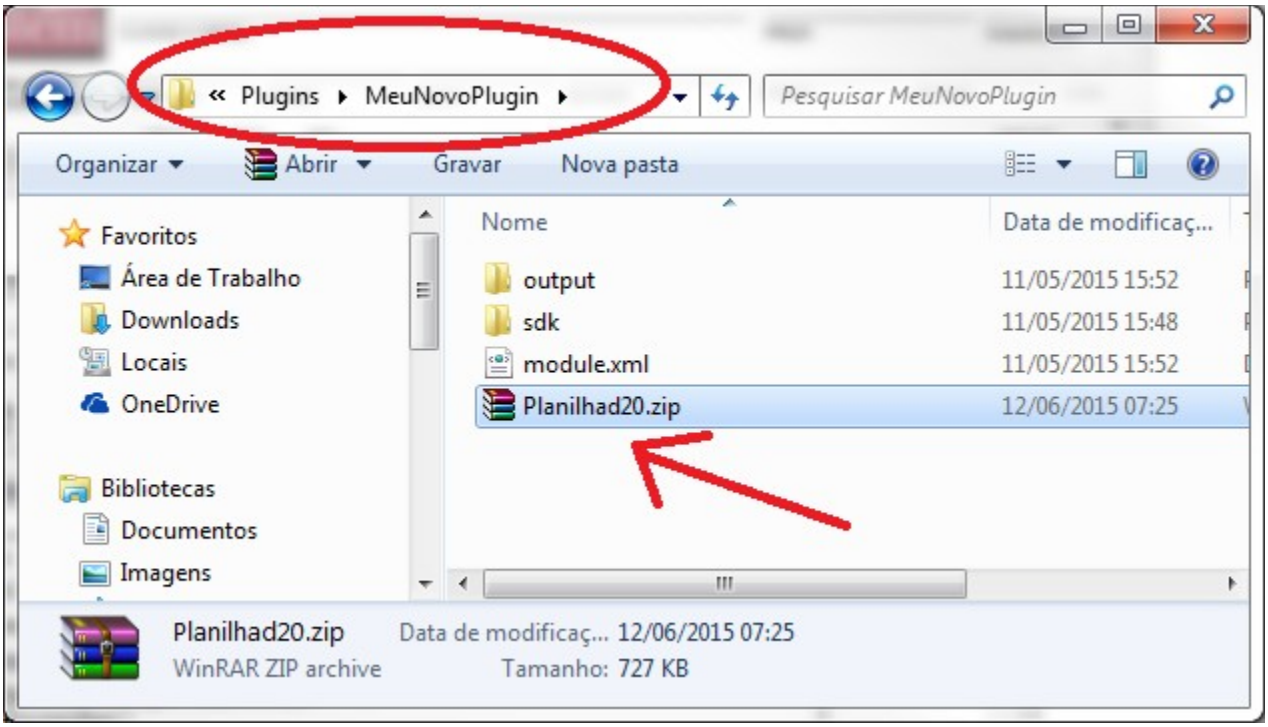
5. Se o Zoom padrão de 1,5x deixar o conteúdo muito pequeno, escolha outro valor de escala que melhor se encaixar em seu PDF e clique em "Aplicar Configurações". Experimente outras configurações também!



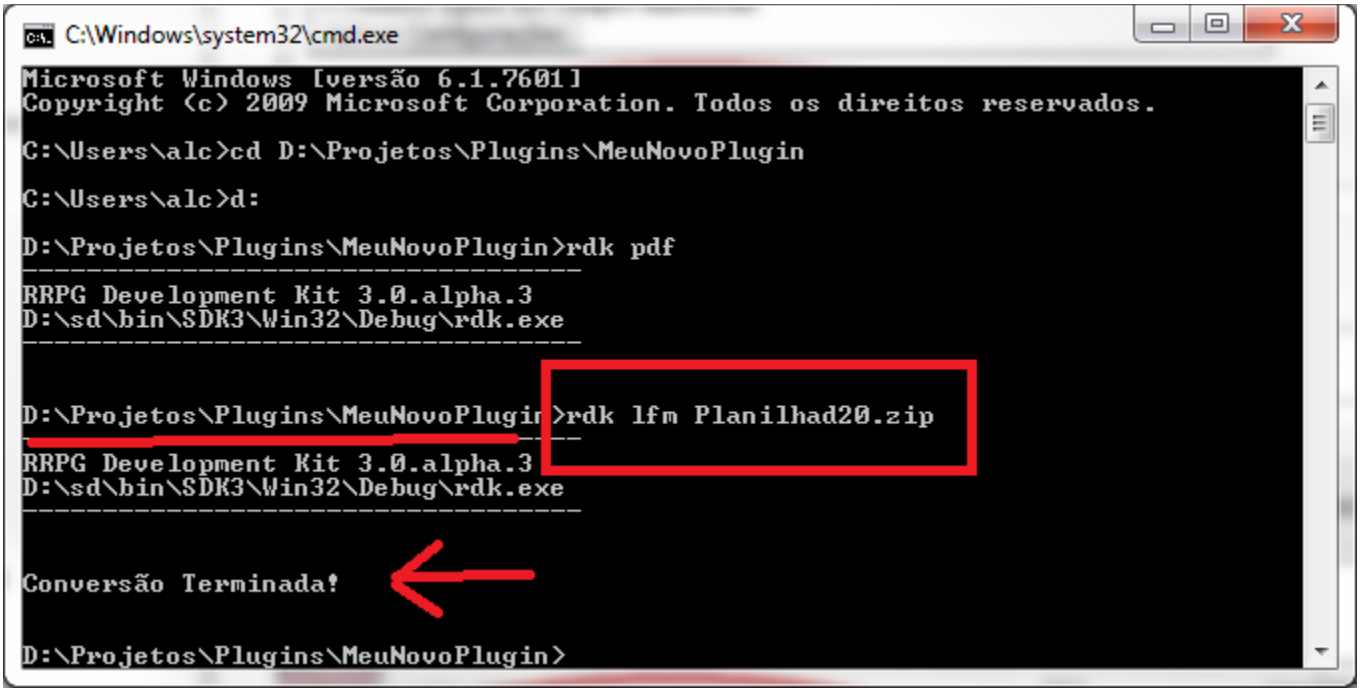
6. Clique em "salvar todas as páginas". Um arquivo ".zip" que contém o PDF em formato .SVG será gerado!



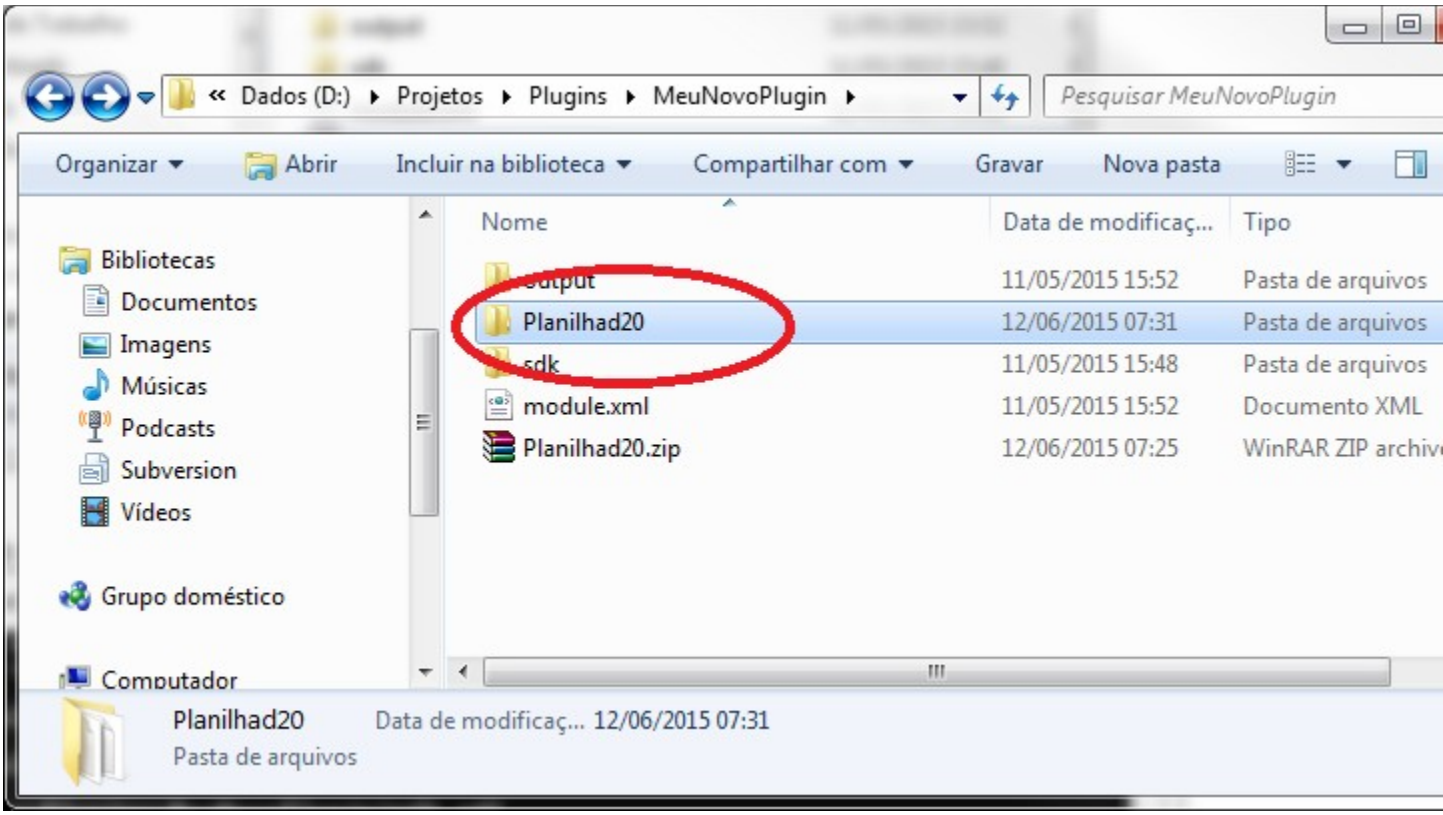
7. Copie o arquivo .zip (gerado no passo anterior) para a pasta do projeto do plug-in.



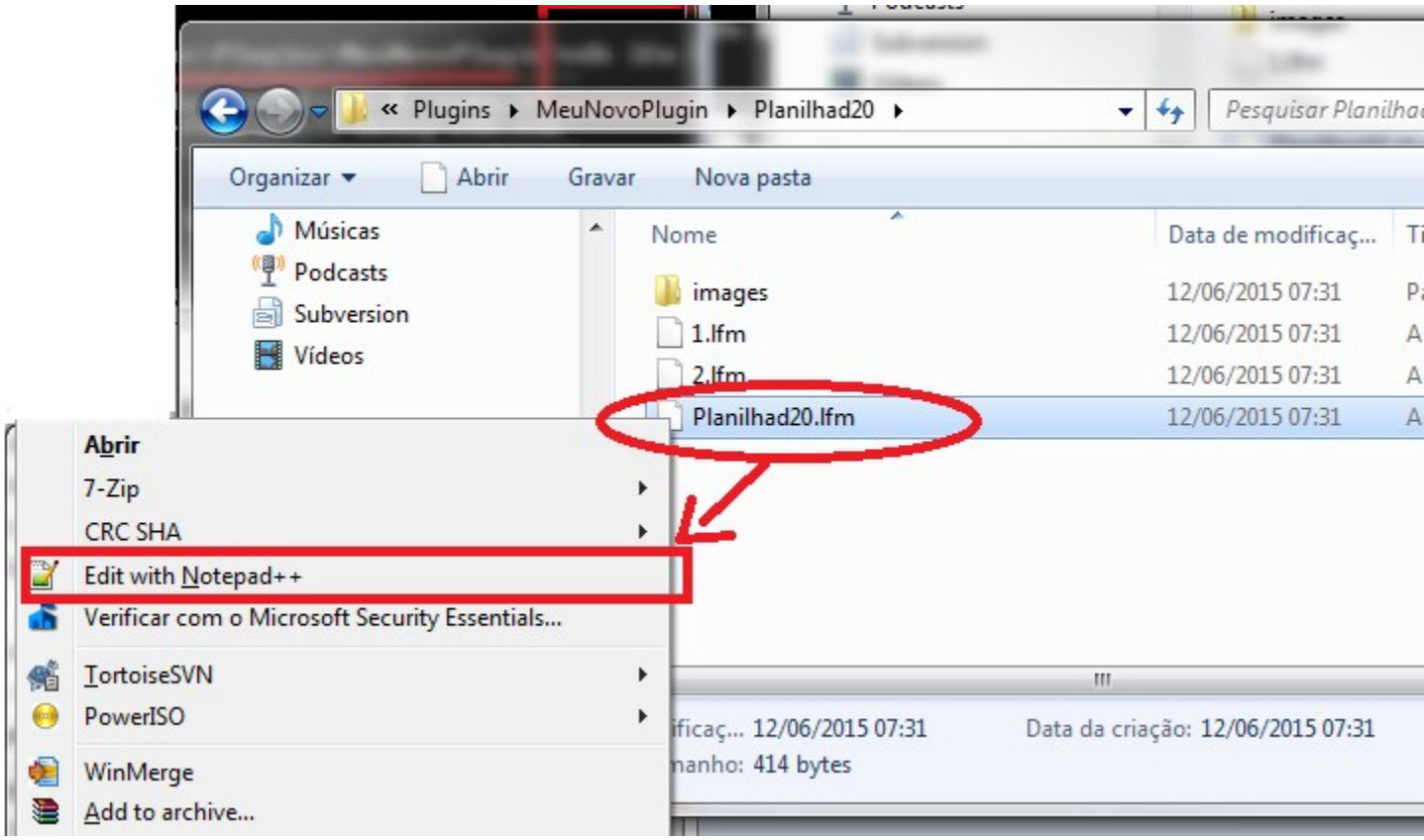
8. Volte no prompt de comando e execute "rdk lfm <ArquivoZip.zip>" sem as áspas e substituindo <ArquivoZip.zip> pelo nome do arquivo gerado no passo 6.



9. Volte na pasta projeto do plug-in e entre no sub-diretório criado pelo comando no passo anterior (Local onde contém os arquivos .lfm convertidos)



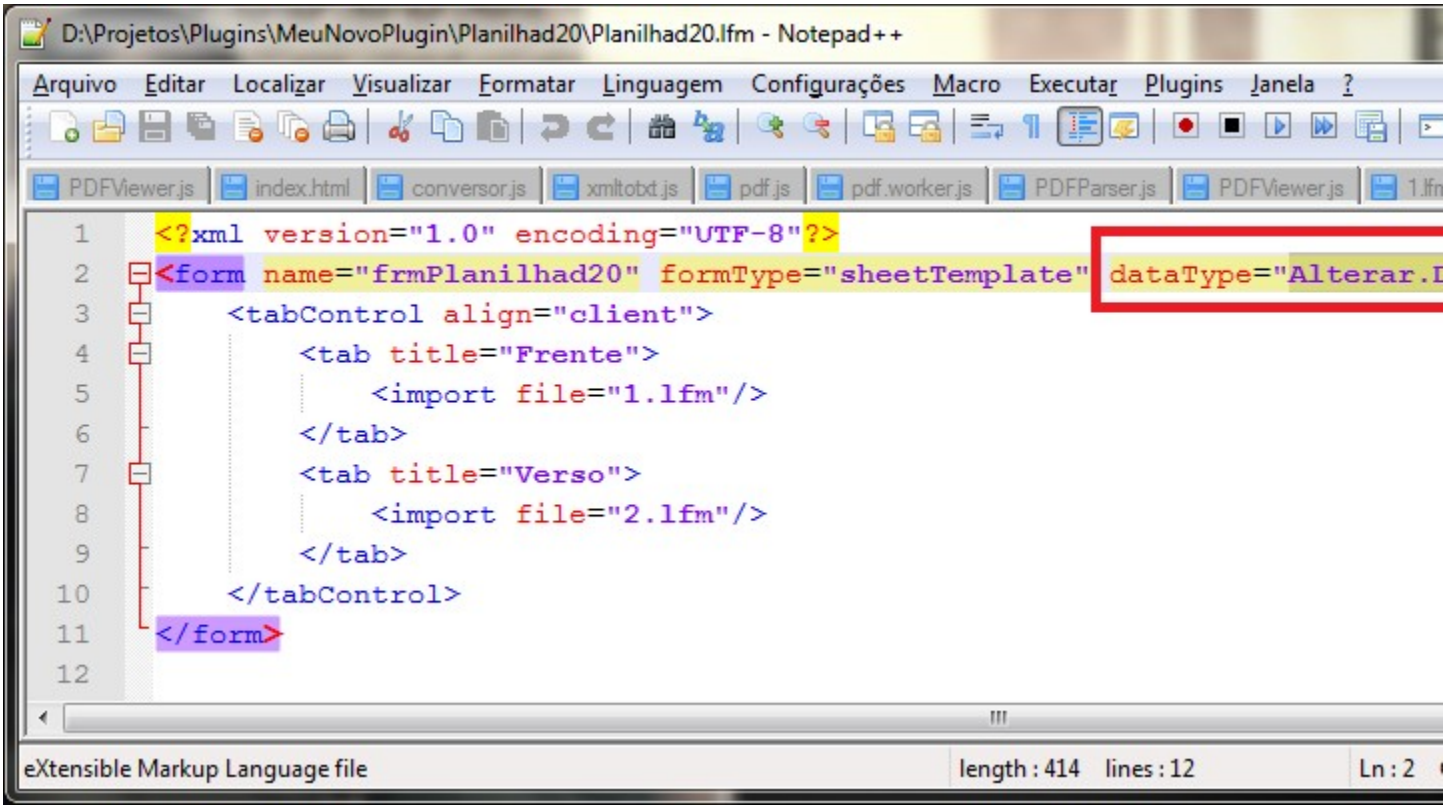
10. Edite o arquivo .lfm principal (aquele que contém o mesmo nome do arquivo .PDF):



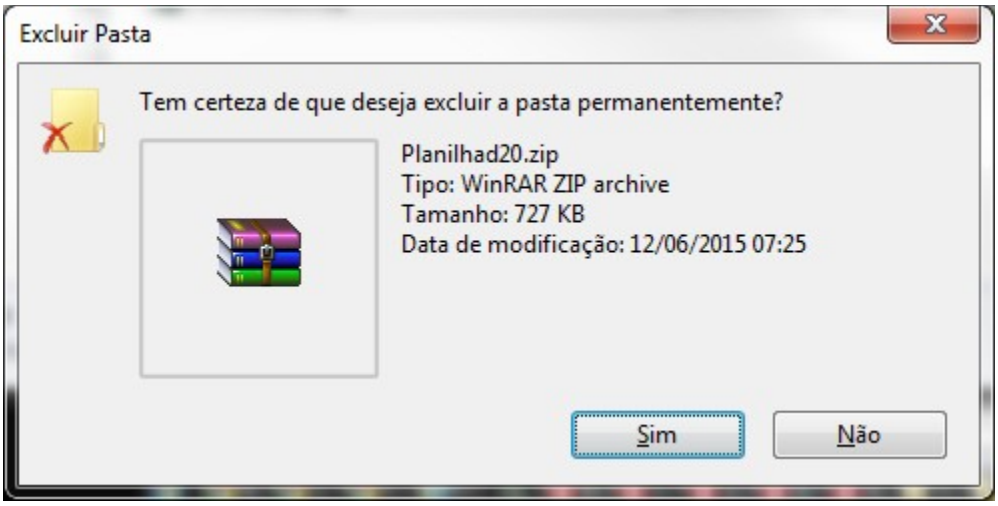
11. Altere os atributos "dataType" e "title" do form para valores adequados:

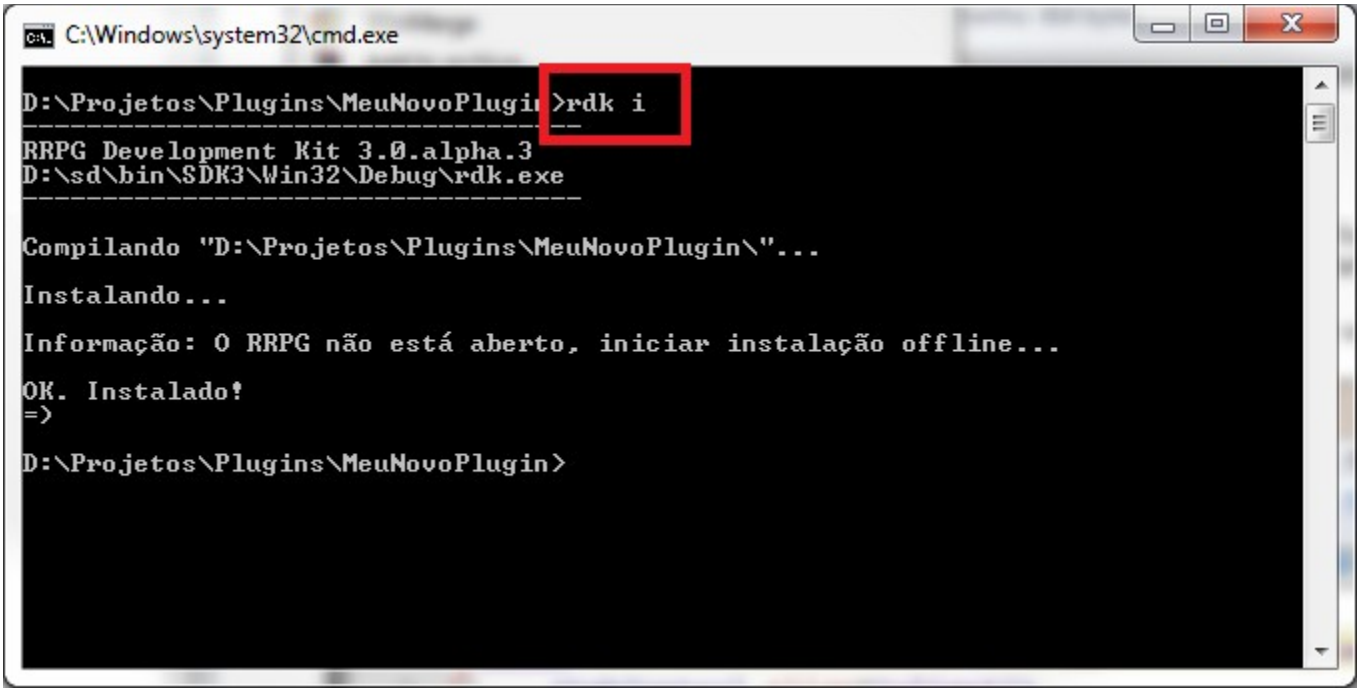
    dataType – Um texto que identifica qual é o tipo de conteúdo do seu modelo de ficha. Você **deve inventar** um Data Type único que possua ao menos cinco caracteres alfanuméricos, "_" ou "." e que não comece com dígitos. Use a criatividade! Exemplos: “br.com.rrpg.DnD5_S3”, “br.com.rrpg.VampiroAMascara”, “br.com.meusite.MeuModeloDeFicha”, “meuplugin.FichaX”, etc..

    title – O título do seu modelo de ficha. Este texto aparecerá na lista de fichas quando o usuário criar um personagem. Exemplos: “DnD 5th”, “Vampiro a Máscara”, “Meu modelo de ficha”, etc..



12. Pronto! Agora você já pode excluir o arquivo ".zip" que colocou na pasta projeto do plug-in e executar o comando "rdk i" para testar o modelo de ficha no RRPG =)


Excluir Pasta
Tem certeza de que deseja excluir a pasta permanentemente?
Planilhad20.zip
Tipo: WinRAR ZIP archive
Tamanho: 727 KB
Data de modificação: 12/06/2015 07:25
Sim
Não


C:\Windows\system32\cmd.exe
D:\Projetos\Plugins\MeuNovoPlugin>rdk i
RRPG Development Kit 3.0.alpha.3
D:\sd\bin\SDK3\Win32\Debug\rdk.exe
Compilando "D:\Projetos\Plugins\MeuNovoPlugin\"...
Instalando...
Informação: O RRPG não está aberto, iniciar instalação offline...
OK. Instalado!
=>
D:\Projetos\Plugins\MeuNovoPlugin>



13. Se quiser, edite os arquivos ".lfm" gerados em um editor de texto para complementar a ficha gerada!

Observações

- Os cálculos e scripts existentes no PDF não são convertidos automaticamente. Você deve recriá-los manualmente, se quiser. Leia o tutorial Criar campos calculados em fichas.

Criar campos calculados em fichas

Os campos calculados são feitos usando a tag dataLink para definir cálculos que devem ser executados quando houver mudança nos valores de alguns campos da ficha (NodeDatabase).

Tutorial 1 - Campo calculado simples

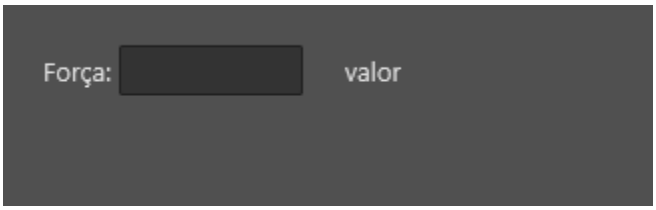
Neste tutorial, iremos fazer com que o label da interface abaixo sempre mostre o valor do cálculo "((campo forca) / 2 - 5) arredondado para baixo".

Importante: Note que o edit já está ligado ao campo "forca" da ficha/NodeDatabase

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

        <layout left="20" top="20" height="25" width="200">
                <!-- Atributo -->
                <label align="left" text="Força: " autoSize="true"/>
                <edit align="client" horzTextAlign="center" field="forca"/>


                <!-- Modificador -->
                <label align="right" text="valor" width="70"
horzTextAlign="center"/>
        </layout>
</form>
```



Passo 1 - Associar o label a um campo do NodeDatabase/Ficha

Vamos ligar o label a um campo chamado "modificadorDeForca":

```
<label field="modificadorDeForca" align="right" text="valor" width="70"
horzTextAlign="center"/>
```

Passo 2 - Criar uma tag dataLink

Vamos criar uma tag dataLink e ligá-la ao campo "forca" para que possamos associar um código LUA que será executado sempre que o campo "forca" for alterado.

```
<dataLink field="forca">
</dataLink>
```

Passo 3 - Criar uma tag event para tratar o evento "onChange" do dataLink que criamos.

```
<dataLink field="forca">
        <event name="onChange">


        </event>
</dataLink>
```

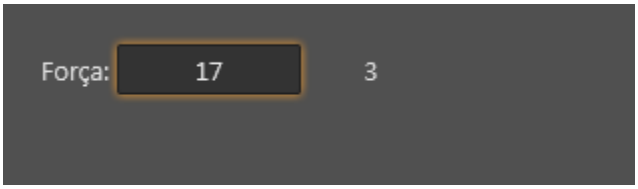
Veja também: Tratando eventos do Lua Form

Passo 4 - Escrever o código LUA do cálculo

```
<dataLink field="forca">
        <event name="onChange">
                sheet.modificadorDeForca = math.floor(((sheet.forca or 10) / 2)
- 5);
        </event>
</dataLink>
```

Aqui realmente colocamos o código do cálculo. O que este código faz:

- Toda vez que o valor do campo "forca" mudar, atribui ao campo "modificadorDeForca" (àquele que o label foi conectado) o resultado de:
  - o valor do campo "forca" (ou 10, caso ainda não tenha sido preenchido) dividido por 2, subtraído em 5 e arredondado para baixo (math.floor).



Passo 5 - Aprimorar o código do cálculo

```
<dataLink field="forca">
        <event name="onChange">
                local mod = math.floor(((sheet.forca or 10) / 2) - 5);

                if (mod >= 0) then
                        mod = "+" .. mod;
                end;

                sheet.modificadorDeForca = mod;
        </event>
```

```
</dataLink>
```

Colocamos uma condição (um "if") para adicionar o caracter "+" na frente do resultado se for positivo ou zero.



Código final do tutorial

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

	<!-- Aqui começa o layout da ficha -->
	<layout left="20" top="20" height="25" width="200">
		<!-- Atributo -->
		<label align="left" text="Força: " autoSize="true"/>
		<edit align="client" horzTextAlign="center" field="forca"/>
		<!-- Modificador -->
		<label field="modificadorDeForca" align="right" text="valor"
			width="70" horzTextAlign="center"/>
	</layout>

	<!-- Começa aqui os cálculos automáticos da ficha -->
	<dataLink field="forca">
		<event name="onChange">
			local mod = math.floor(((sheet.forca or 10) / 2) - 5);

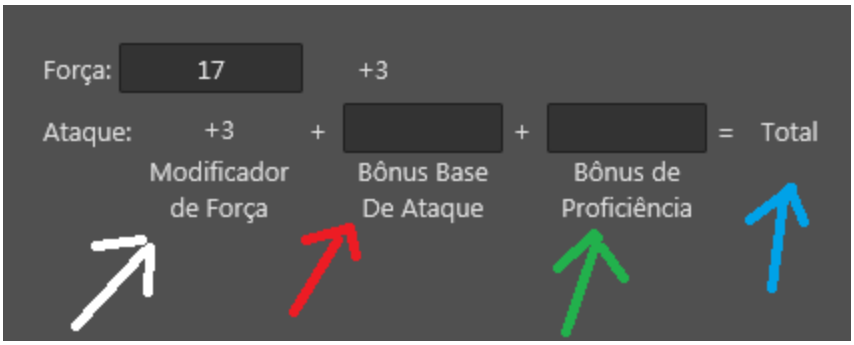

			if (mod >= 0) then
				mod = "+" .. mod;
			end;

			sheet.modificadorDeForca = mod;
		</event>
	</dataLink>
</form>
```

Tutorial 2 - Criar um campo calculado que depende do valor de outros 3 campos

Neste tutorial, vamos completar o tutorial 1 acima para adicionar um campo calculado chamado "Bônus de Ataque Corpo-A-Corpo", que será igual à soma dos seguintes campos:

- "Bônus Base de Ataque" (campo editável)
- "Bônus de Proficiência da Arma" (campo editável)
- Modificador de força (campo calculado).



Considere que:

- O texto (label) apontado pela seta branca está ligado ao field "modificadorDeForca"
- A caixa de texto (edit) apontada pela seta vermelha está ligada ao field "bonusBaseAtaque"
- A caixa de texto apontada pela seta verde está ligada ao field "bonusProficiencia"
- O texto apontado pela seta azul está ligado ao field "ataque"

Passo 1 - Criar uma tag dataLink que se conectará, ao mesmo tempo, aos 3 campos necessários para o cálculo:

```
<dataLink fields="{'modificadorDeForca', 'bonusBaseAtaque',
'bonusProficiencia'}">
</dataLink>
```

ATENÇÃO: Para ligar um dataLink à mais de 1 campo ao mesmo tempo, você deve usar o atributo "fields" do DataLink ao invés de "field" (Um está no plural, enquanto outro está no singular).

Passo 2 - Criar uma tag event para tratar o evento "onChange" do dataLink que criamos.

```
<dataLink fields="{'modificadorDeForca', 'bonusBaseAtaque',
'bonusProficiencia'}">            <event name="onChange">


        </event>
```

```
</dataLink>
```

Este evento será chamado sempre que qualquer um dos campos informados em "fields" sofrer alteração.

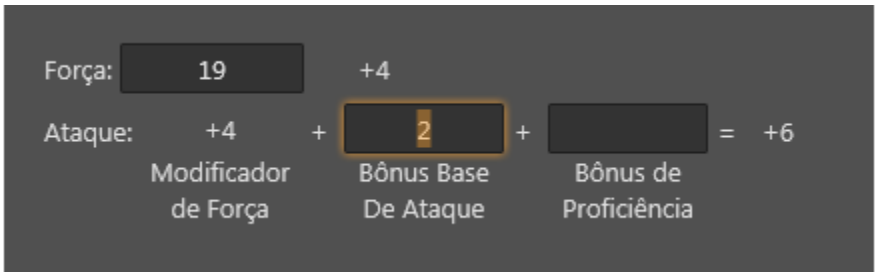
Passo 3 - Escrever o código LUA do cálculo

```
<dataLink fields="{'modificadorDeForca', 'bonusBaseAtaque',
'bonusProficiencia'}">            <event name="onChange">
              sheet.ataque = (tonumber(sheet.modificadorDeForca) or 0) +
                             (tonumber(sheet.bonusBaseAtaque) or 0) +
                               (tonumber(sheet.bonusProficiencia) or 0);

              if sheet.ataque >= 0 then
                      sheet.ataque = "+" .. sheet.ataque;
              end;
       </event>
</dataLink>
```

Sempre que o valor de algum dos campos ligados mudar ("modificadorDeForca", "bonusBaseAtaque" ou "bonusProficiencia"):

- o campo "ataque" receberá a soma de:
  - Valor do campo "modificadorForca" interpretado como número (o valor campo pode não ser um número válido, exemplo: "ABC") ou 0 caso não consiga interpretar como número.
  - Valor do campo "bonusBaseAtaque" interpretado como numero ou 0 caso não consiga interpretar como número.
  - Valor do campo "bonusProficiencia" interpretado como número ou 0 caso não consiga interpretar como número.
- Se o valor do campo "ataque" for maior ou igual a zero:
  - Adiciona o caracter "+" na frente do valor.



Criar abas para a ficha

Para criar abas na ficha, você deve utilizar as tags tabControl e tab.

Você cria 1 tag "tabControl" e adiciona, dentro, quantas tags "tab" quiser, uma para cada aba. Os controles de cada aba devem ser adicionados dentro de sua respectiva tag "tab".

Para entender melhor, veja exemplos de tab e tabControl.

Permitir o usuário escolher uma imagem dentro da ficha

É necessário o usar a tag image, colocar sua propriedade "editable" para "true" e definir um "field" para ela.

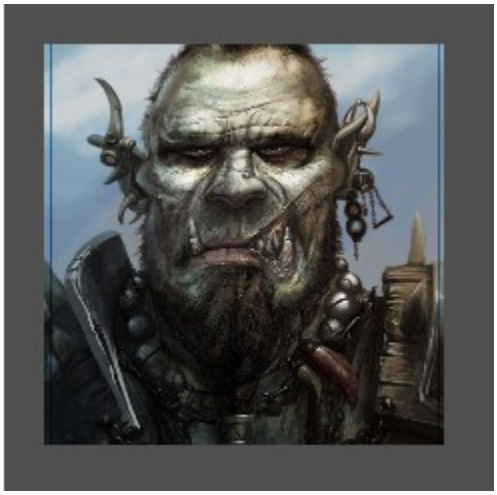
Adicionalmente, você poderá definir a propriedade "style" da tag como "autoFit" para que o a imagem ocupe todo o espaço definido para ela.

Exemplo:

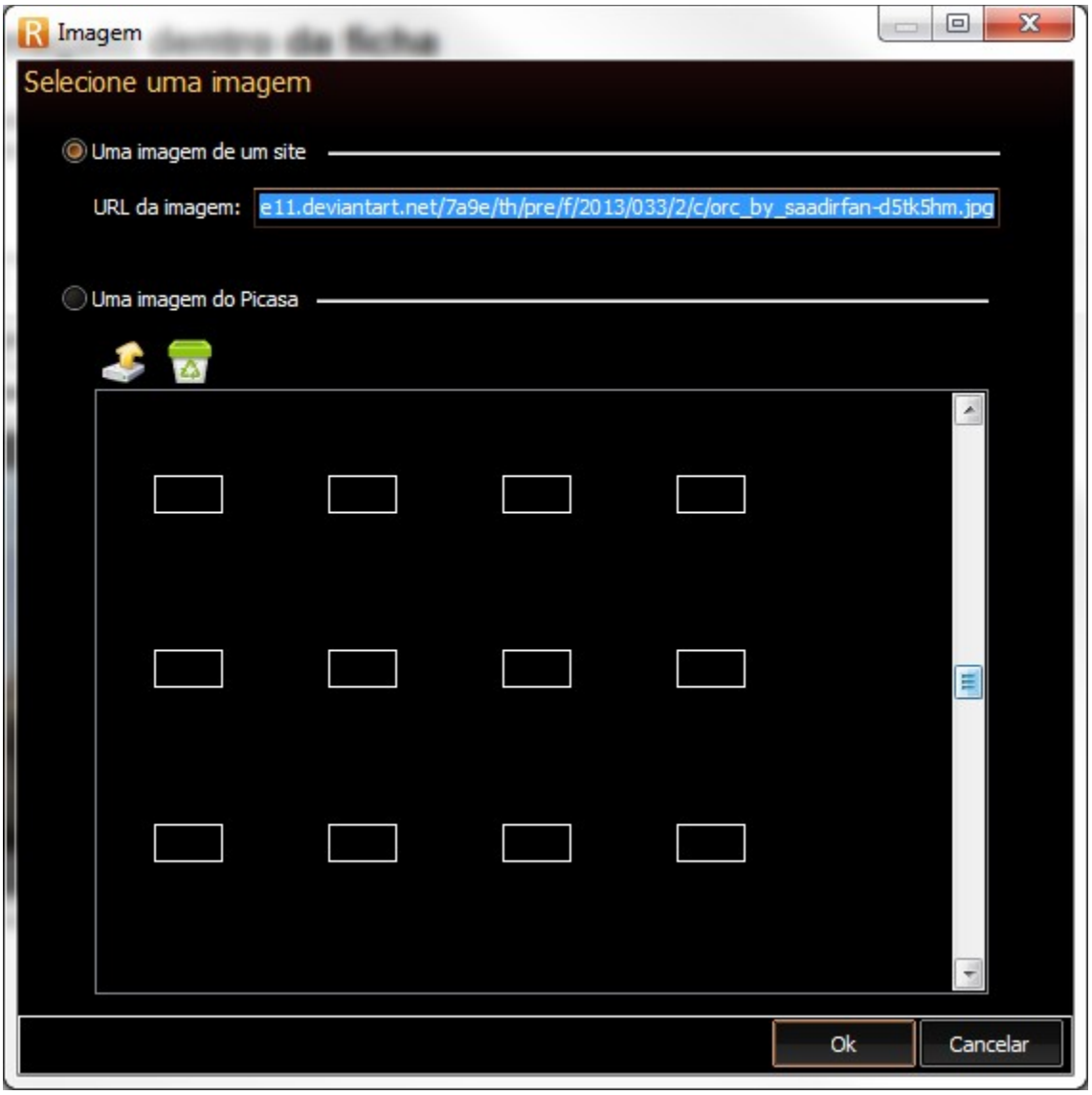
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

        <image field="imagemDoPersonagem" editable="true" style="autoFit"
               left="20" top="20" width="200" height="200" />

</form>
```



O usuário poderá alterar a imagem quando clicar nela:



Criar uma lista dinâmica na ficha

Se você deseja adicionar uma uma lista dinâmica de algo (com quantidade personalizada pelo usuário) em sua ficha, você deve utilizar a tag recordList.

Exemplos:

- Uma lista de equipamentos
- Uma lista de ataques
- Uma lista de magias

Você precisará ter, no mínimo, dois arquivos ".lfm", um representando a ficha que contém a lista (a tag recordList) e um que descreve o painel que deve "repetir" para cada item desta lista.

Para melhor compreender, veja este exemplo.

Fazer uma ficha enviar rolagens para a mesa

1. Adicione uma tag button ao Lua Form
2. Manipule o evento "onClick" da tag button
3. Utilize o código abaixo:

```
       <button>
                <event name="onClick">
                        local rolagem =
rrpg.interpretarRolagem(sheet.ataqueDoPersonagem);
                        if not rolagem.possuiAlgumDado then
                                -- se o usuario não tiver preenchido qual dado
rolar,
                                -- vamos adicionar um 1d20 + na "fórmula da
rolagem"
                                rolagem =
rrpg.interpretarRolagem("1d20"):concatenar(rolagem);
                        end;
                        local mesaDoPersonagem = rrpg.getMesaDe(sheet);
                        if mesaDoPersonagem ~= nil then
                               mesaDoPersonagem.chat:rolarDados(rolagem, "Ataque
do personagem");
                        end;
                </event>
        </button>
```

1. Altere o código conforme a necessidade

O que o código acima faz?

1. Utiliza a função interpretarRolagem da biblioteca "rrpg" para interpretar o campo "ataqueDoPersonagem" como uma rolagem de dados.
2. Verifica se a rolagem utiliza algum dado (d4, d6, d8, etc..)... Se não possuir dado na rolagem (Exemplos: "+5", ou "-2"), faz:
    1. Interpreta uma nova rolagem "1d20" e a concatena/junta ao inicio (Para ficar, por exemplo, "1d20 + 5" ou "1d20 - 2").

3. Usando a ficha atual (sheet), tenta localizar o objeto mesa à qual a ficha pertence. É usado a função getMesaDe da biblioteca rrpg para isso.
4. Se conseguiu localizar o objeto mesa à qual a ficha atual pertence:
    1. Invoca o método rolarDados do chat da mesa, passando a rolagem que queremos e um texto adicional "Ataque do personagem".

Mudanças que você pode querer fazer no código acima

- O código acima usa o texto da rolagem contido no campo chamado "ataqueDoPersonagem". Talvez a rolagem de sua ficha esteja em outro campo.
- Quando a rolagem não usar nenhum dado (d4, d6, d8, etc..) o código adiciona "1d20" ao inicio dela. Talvez o sistema da sua ficha não utiliza 1d20 nesta rolagem!
- Que tal alterar o texto "Ataque do personagem"?
- Talvez você queira que somente o mestre e o jogador dono da ficha possam rolar dados desta ficha.
    - Você pode utilizar a função getPersonagemDe da biblioteca rrpg e verificar se sua propriedade dono é igual à propriedade meuJogador da variável mesaDoPersonagem. Se for igual, é porque a ficha pertence ao usuário.
    - Você pode utilizar a propriedade isMestre do objeto meuJogador para verificar se o usuário é mestre da mesa.
    - O código que faz esta verificação seria parecido com:

```
local personagem = rrpg.getPersonagemDe(sheet);

if (personagem ~= nil) then
	local mesa = personagem.mesa;

	if ((personagem.dono == mesa.meuJogador) or
(mesa.meuJogador.isMestre)) then		-- permitir
enviar a rolagem
	end;
end;
```

Criar uma lista dinâmica + painel de detalhes

Este tutorial vai ensinar a criar uma lista dinâmica de itens que, quando você selecionar um dos itens desta lista, exibe um painel detalhando as informações sobre o item selecionado.

É recomendado você ler Criar uma lista dinâmica na ficha antes, pois este tutorial utiliza a mesma ideia.

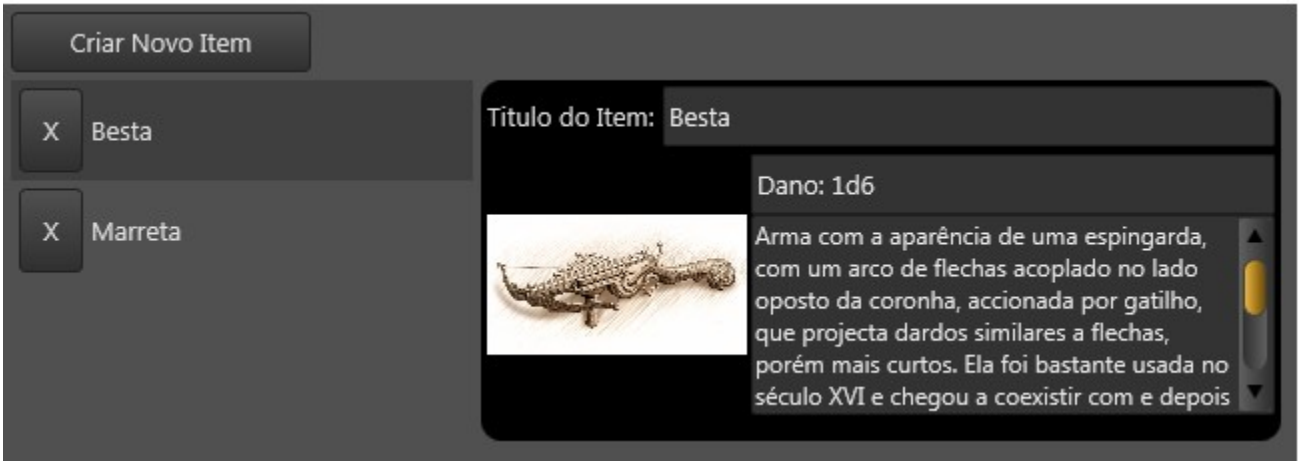
Dois arquivos Lua Form (extensão ".LFM")

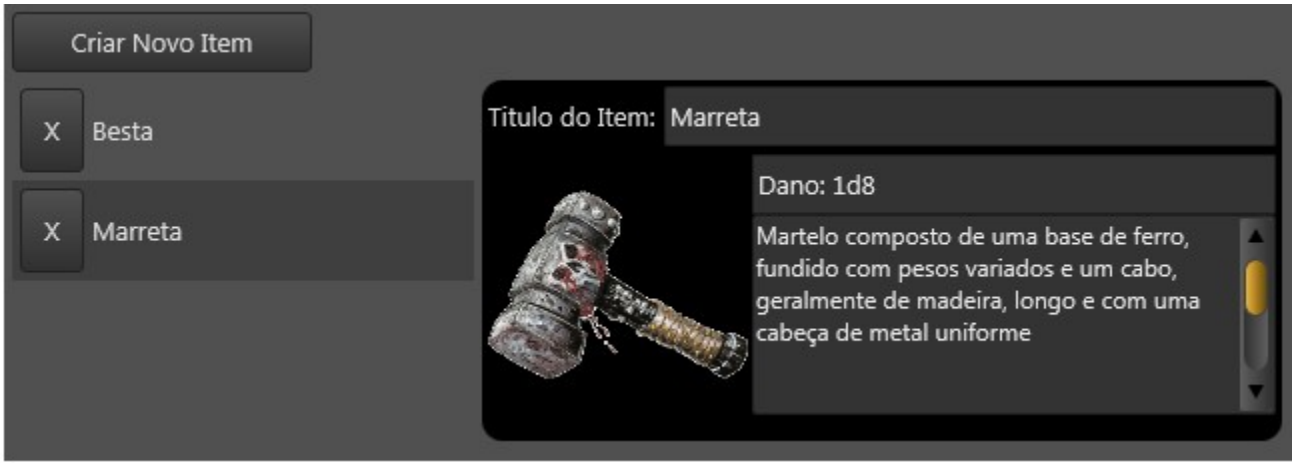
Utilizaremos a tag recordList neste tutorial, portanto, você deve possuir dois arquivos ".lfm":

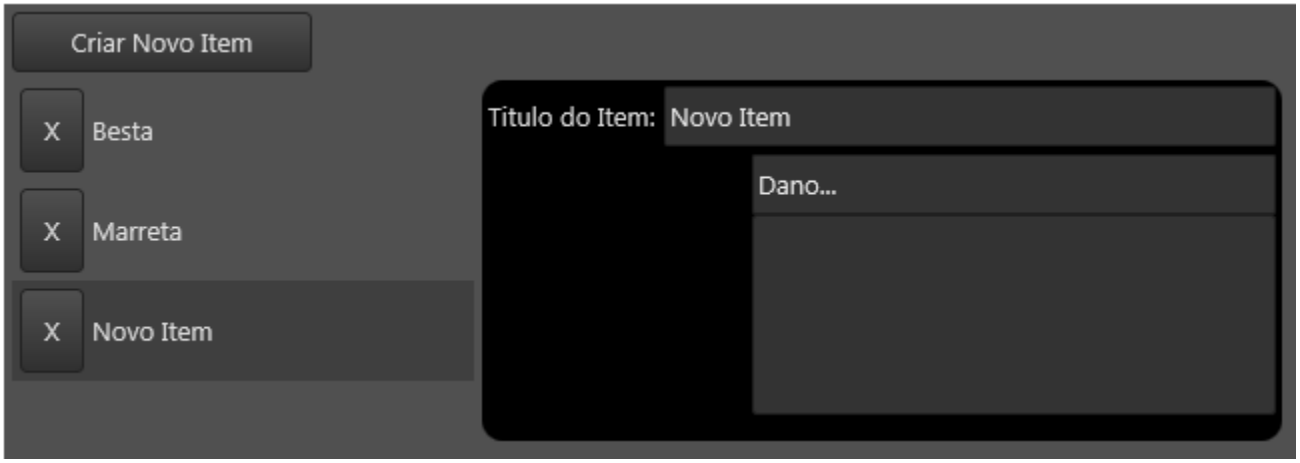
- O arquivo Lua Form principal que contém a tag recordList
- O arquivo Lua Form Template dos Itens do Record List: Este form vai ser instanciado 1 vez para cada item na lista.

Neste tutorial, chamaremos o arquivo Lua Form Principal de "MinhaFicha.lfm" e o arquivo Lua Form Template dos Itens de "ItemDaLista.lfm"

Imagens







Sobre a implementação

A ideia para criar esta lista dinâmica com painel de detalhes é bem simples. Existem dois componentes chaves nesta ideia:

- Você precisa de um recordList que funcionará como uma lista onde o usuário possa escolher o item.
- Você precisará de um dataScopeBox que conterá as tags que são apresentadas no painel de detalhes.

Quando o usuário escolher um item no recordList (evento "onSelect" da tag recordList), fazemos com que os componentes que estão dentro do dataScopeBox passem a salvar/carregar dados neste item ao alterar o escopo de dados das tags que estão dentro. Caso tenha dúvida sobre escopo de dados, veja Escopo de Dados no Lua Form





Código Fonte

Arquivo "MinhaFicha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste" theme="dark" padding="{left=4, top=4, right=4, bottom=4}">

	<!-- Botão de inserir novo item à lista -->
	<layout align="top" height="30" margins="{bottom=4}">
		<button text="Criar Novo Item" width="150" align="left">
			<event name="onClick">
				-- Usuário clicou no botão de criar novo item.
				-- Vamos inserir um novo item no nosso recordList
				self.rclListaDosItens:append();
			</event>
		</button>
	</layout>

	<!-- recordList, representando a lista -->
	<recordList name="rclListaDosItens" field="campoDosItens" templateForm="frmItemDaLista"
				align="client" selectable="true">

		<event name="onSelect">
			--[[
			Este evento é chamado quando o usuário selecionar/deselecionar itens da lis
Quando o usuário selecionar, vamos fazer nosso dataScopeBox (e todas as tag dentro dele) salvar e c
dados no   objeto Nodo (NodeDatabase) do item selecionado.

1.			--]]
2.			local node = self.rclListaDosItens.selectedNode;
				self.boxDetalhesDoItem.node = node;
```

3.
```
                    -- a caixa de detalhe só ficará visível se houver item selecionado
                        self.boxDetalhesDoItem.visible = (node ~= nil);
                </event>
            </recordList>

        <!-- dataScopeBox representando a nossa caixa de detalhes -->
        <dataScopeBox name="boxDetalhesDoItem" visible="false" align="right"
                      width="400" margins="{left=4, right=4}">
            <!-- Faça um layout "bonito" para sua caixa de detalhes =). Utilize a propriedade "
nas tags aqui dentro   -->
            <rectangle align="top" color="black" xradius="10" yradius="10"
                           height="180" padding="{top=3, left=3, right=3, bottom=3}">


                <layout align="top" height="30" margins="{bottom=4}">
                    <label align="left" text="Titulo do Item:" autoSize="true"/>
                    <edit align="client" field="campoTitulo"/>
                </layout>

                <layout align="top" height="130">
                    <image align="left" editable="true" field="campoURLImagem" width="1

                    <layout align="client" margins="{left=2}">
                        <edit align="top" field="campoSubTitulo" height="30"/>
                        <textEditor align="top" field="campoTextoGrande" height="10
                    </layout>
                </layout>
            </rectangle>
        </dataScopeBox>
</form>
```

Arquivo “ItemDaLista.lfm”

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmItemDaLista" height="50" theme="dark">

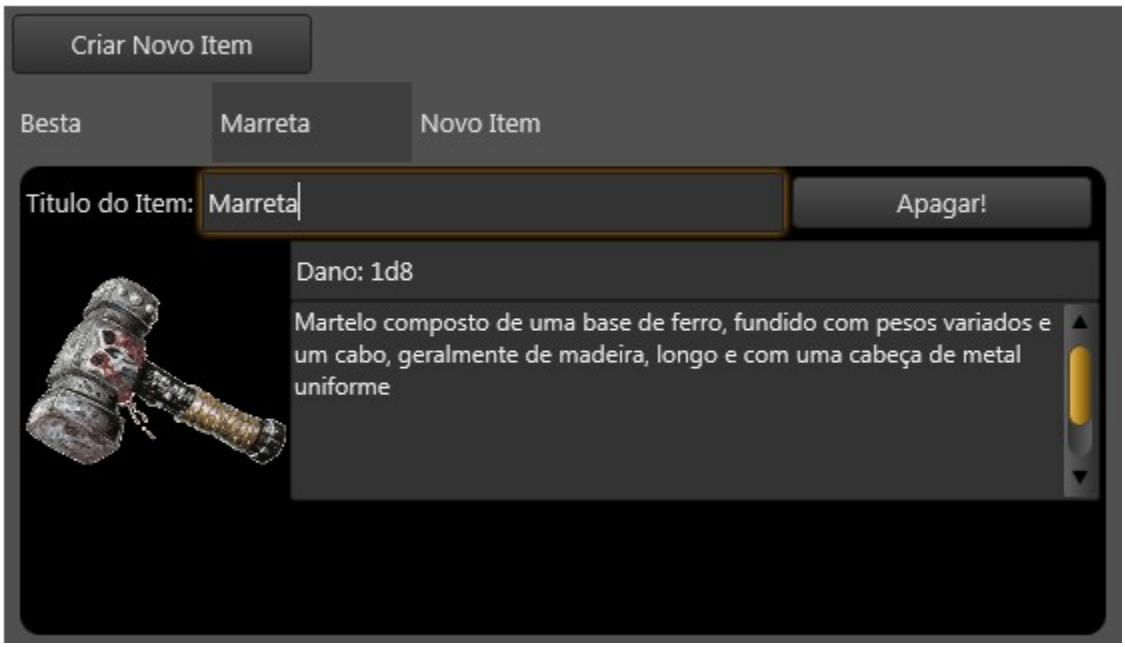
        <layout align="client" margins="{left=4, right=4, top=4, bottom=4}">
            <!-- Botão para apagar o item da lista -->
            <button align="left" width="32" text="X" onClick="ndb.deleteNode(sheet);"/>

            <!-- O título do item que aparece na lista -->
            <label align="client" text="Teste de label" field="campoTitulo" margins="{left=4}"/
        </layout>
```

```
</form>
```

Lista Dinâmica + Painel de Detalhes alternativo





Código Fonte Alternativo

Arquivo “MinhaFicha.lfm”

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste" theme="dark" padding="{left=4, top=4, right=4, bottom=4}">
```

```
        <!-- Botão de inserir novo item à lista -->
        <layout align="top" height="30" margins="{bottom=4}">
                <button text="Criar Novo Item" width="150" align="left">
                        <event name="onClick">
                                -- Usuário clicou no botão de criar novo item.
                                -- Vamos inserir um novo item no nosso recordList
                                self.rclListaDosItens:append();
                        </event>
                </button>
        </layout>
        <!-- recordList, representando a lista -->
        <recordList name="rclListaDosItens" field="campoDosItens" templateForm="frmItemDaLista" ali
layout="horizontal" height="40">
                <event name="onSelect">
                        --[[
                        Este evento é chamado quando o usuário selecionar/deselecionar itens da lis
fazer nosso dataScopeBox (e todas as tag dentro dele) salvar e carregar dados no   objeto Nodo (Noc
                        --]]
                        local node = self.rclListaDosItens.selectedNode;
                                self.boxDetalhesDoItem.node = node;
                        -- a caixa de detalhe só ficará visível se houver item selecionado
                                self.boxDetalhesDoItem.visible = (node ~= nil);
                </event>
        </recordList>
        <!-- dataScopeBox representando a nossa caixa de detalhes -->
        <dataScopeBox name="boxDetalhesDoItem" visible="false" align="client"
                      margins="{left=4, right=4, top=2}">
                <!-- Faça um layout "bonito" para sua caixa de detalhes =). Utilize a propriedade "
                <rectangle align="client" color="black" xradius="10" yradius="10"
                           padding="{top=3, left=3, right=3, bottom=3}">
                         <layout align="top" height="30" margins="{bottom=4}">
                                <label align="left" text="Titulo do Item:" autoSize="true"/>
                                <edit align="client" field="campoTitulo"/>
                                <!-- Botão de apagar o item -->
```

```
                                <button align="right" text="Apagar!"
                                        margins="{left=4, right=4, top=2, bottom=2}" width="150"
                                        onClick="ndb.deleteNode(self.boxDetalhesDoItem.node);" />

                        </layout>

                        <layout align="top" height="130">
                                <image align="left" editable="true" field="campoURLImagem" width="1

                                <layout align="client" margins="{left=2}">
                                        <edit align="top" field="campoSubTitulo" height="30"/>
                                        <textEditor align="top" field="campoTextoGrande" height="10
                                </layout>
                        </layout>
                </rectangle>
        </dataScopeBox>
</form>
```

Arquivo "ItemDaLista.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmItemDaLista" height="50" width="100">
        <!-- O título do item que aparece na lista -->
        <label align="client" text="Teste de label" field="campoTitulo"
                margins="{left=4, right=4, top=4, bottom=4}"/>


        <!-- Adicionar um valor padrão para o campoTitulo caso esteja vazio.
                Isto é especialmente útil em novos itens -->
        <dataLink field="campoTitulo" defaultValue="Titulo Aqui"/>
</form>
```

Incrementando o código: Selecionar o primeiro item ao carregar os dados

Trate o evento "onEndEnumeration" da tag recordList com o seguinte código:

```
  <event name="onEndEnumeration">
          if self.rclListaDosItens.selectedNode == nil and sheet ~= nil then
                  local nodes = ndb.getChildNodes(sheet.campoDosItens);

                  if #nodes > 0 then
                          self.rclListaDosItens.selectedNode = nodes[1];
                  end;
          end;
  </event>
```

Criar um botão que esconde itens de uma lista dinâmica

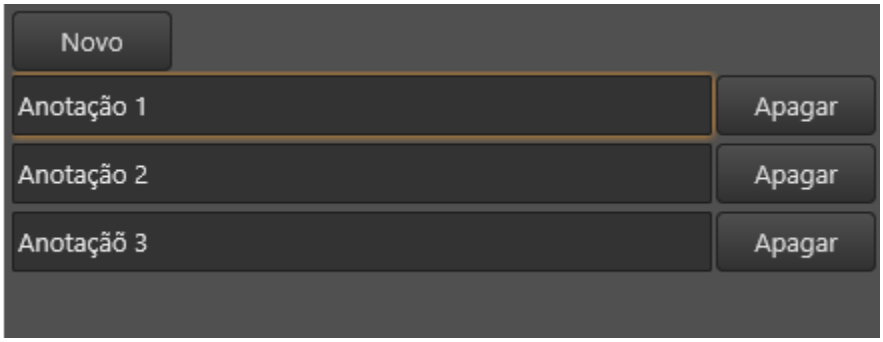
Atenção: Este é um tópico avançado e requer o conhecimento de como criar uma lista dinâmica.

Este tipo de funcionalidade é útil quando em uma ficha ou em uma janela acoplável é necessário esconder certos itens de certos usuários, por exemplo:

- NPCs - Talvez o mestre queira esconder determinado NPC dos jogadores
- Equipamentos - Talvez um jogador de um grupo possua um equipamento que o Mestre deseja que os demais jogadores não saibam de sua existência.
- Uma anotação secreta, que somente o mestre pode visualizar.

Começando....

Vamos supor que você já tenha implementado em seu código uma lista dinâmica com a seguinte aparência:



e com o seguinte código:

Arquivo Principal "ficha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">


        <layout align="top" height="30">
                <button align="left" text="Novo"
onClick="self.rclLista:append();"/>
        </layout>


        <recordList name="rclLista" align="client" field="itens"
                    templateForm="frmItem"/>
</form>
```

Arquivo Secundário "item.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmItem" height="30" margins="{top=2,bottom=2}">

    <edit align="client" field="descricao" margins="{right=2}"/>


    <button align="right" text="Apagar" width="80"
            onClick="ndb.deleteNode(sheet);"/>
</form>
```

Próximos passos

Para adicionar a funcionalidade de esconder item, você deverá fazer as seguintes alterações em seu arquivo equivalente ao "item.lfm" (aquele que repete para cada item na lista):

- Criar um botão novo ou outra funcionalidade que permita o usuário clicar e realizar uma ação. Neste tutorial usaremos a tag imageCheckBox pelo simples fato de eu achá-la bacana =).

```
<imageCheckBox name="cbxInvisivel" align="right" width="25"
margins="{left=2, right=2}"
checkedImage="images/invisivel.png"
uncheckedImage="images/visivel.png" autoChange="false"
onClick="self:alternarVisibilidade();" />
```

Esta é a imagem "images/invisivel.png": 
Esta é a imagem "images/visivel.png":
Note que a propriedade autoChange foi setada para false para que a tag não mude automaticamente sua propriedade "checked".

Note também que ao clicar no imageCheckBox, será invocada a função "self:alternarVisibilidade"

- Criar o código da função "self:alternarVisibilidade" que utiliza a função ndb.setPermission

```
<script>
        function self:alternarVisibilidade()
                if self.cbxInvisivel.checked then
                        -- o imageCheckbox está marcando como
invisivel..
                        -- Vamos mudar as permissões para
tornar visivel então..

                        -- resetar as permissões de read dos
jogadores e espectadores
```

```
                        ndb.setPermission(sheet, "group",
"jogadores", "read", nil);
                        ndb.setPermission(sheet, "group",
"espectadores", "read", nil;
                else
                        -- o imageCheckbox está marcando como
visível
                        -- Vamos mudar as permissões para
tornar invisível então...

                        -- negar as permissões de read dos
jogadores e espectadores
                        ndb.setPermission(sheet, "group",
"jogadores", "read", "deny");
                        ndb.setPermission(sheet, "group",
"espectadores", "read", "deny");
                end;
        end;
</script>
```

- Criar um código que fica responsável por atualizar o estado do imageCheckBox conforme as permissões existentes e alteradas:

```
<script>
        function self:atualizarCbxInvisivel()
                -- Esta função é chamada em vários pontos para atualizar o estado do
checkbox de invisibilidade

                -- Marcar o checkbox se existir uma permissão "read deny" para jogadores
ou para espectadores
                self.cbxInvisivel.checked = ndb.getPermission(sheet, "group",
"espectadores", "read") == "deny" or
                                                ndb.getPermission(sheet, "group",
"jogadores", "read") == "deny";
                -- Permitir o click no checkbox apenas se o usuário atual possuir a
permissão "writePermissions"
                self.cbxInvisivel.enabled = ndb.testPermission(sheet,
"writePermissions");
```

```
        end;
</script>
```

Veja também:

- Função ndb.getPermission
- Função ndb.testPermission
    - Criar um NodeObserver que será responsável por invocar a função acima "self:atualizarCbxInvisivel" quando ocorrer alguma mudança nas permissões do NodeDatabase. O NodeObserver será criado no evento onScopeNodeChanged da tag Form

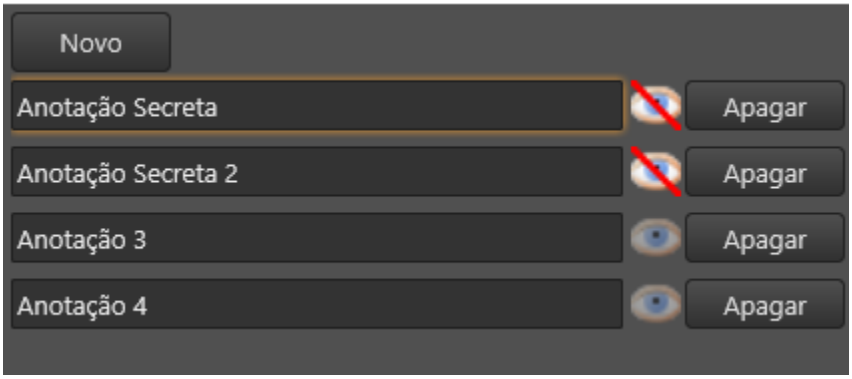
```
<event name="onScopeNodeChanged">
	if self.observer ~= nil then
		-- Já tinhamos criado um observer, vamos desativá-lo.
		self.observer.enabled = false;
		self.observer = nil;
	end;
	if sheet ~= nil then
		-- O form foi ligado a um NodeDatabase!
		-- Vamos criar um NodeObserver para este nodo.
		self.observer = ndb.newObserver(sheet);
		self.observer.onPermissionListChanged =
			function(node)
				-- Esta função será chamada quando
				-- as definições de permissões de sheet mudarem

				self:atualizarCbxInvisivel();
			end;
		self.observer.onFinalPermissionsCouldBeChanged =
			function(node)
				-- Esta função será chamada quando
				-- minhas permissões em sheet puderem
				-- ter sido afetadas por alterações de permissões
				-- no NodeDatabase ou porque me tornei
				-- mestre/jogador/espectador

				self:atualizarCbxInvisivel();
			end;
		-- Devemos atualizar o checkbox agora também
		self:atualizarCbxInvisivel();
	end;
</event>
```

Resultado final

O que aparece para os Mestres da mesa:



O que os Jogadores e Espectadores da mesa veem:



Código fonte final (sem os comentários)

Arquivo Principal "ficha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">


        <layout align="top" height="30">
                <button align="left" text="Novo"
onClick="self.rclLista:append();"/>
        </layout>


        <recordList name="rclLista" align="client" field="itens"
                templateForm="frmItem"/>
</form>
```

Arquivo Secundário "item.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmItem" height="30" margins="{top=2,bottom=2}">

    <script>
         function self:alternarVisibilidade()
             if self.cbxInvisivel.checked then
                  ndb.setPermission(sheet, "group", "jogadores", "read", nil);
                  ndb.setPermission(sheet, "group", "espectadores", "read",
nil);
             else
                  ndb.setPermission(sheet, "group", "jogadores", "read",
"deny");
                  ndb.setPermission(sheet, "group", "espectadores", "read",
"deny");
             end;
         end;

         function self:atualizarCbxInvisivel()
             self.cbxInvisivel.checked = ndb.getPermission(sheet, "group",
"espectadores", "read") == "deny" or
                                         ndb.getPermission(sheet, "group",
"jogadores", "read") == "deny"

              self.cbxInvisivel.enabled = ndb.testPermission(sheet,
"writePermissions");
         end;
    </script>

    <edit align="client" field="descricao" margins="{right=2}"/>

    <imageCheckBox name="cbxInvisivel" align="right" width="25"
margins="{left=2, right=2}"
                   checkedImage="images/invisivel.png"
                   uncheckedImage="images/visivel.png"
                   autoChange="false"
                   onClick="self:alternarVisibilidade();" />

    <button align="right" text="Apagar" width="80"
onClick="ndb.deleteNode(sheet);"/>

    <event name="onScopeNodeChanged">
            if self.observer ~= nil then
                    self.observer.enabled = false;
                    self.observer = nil;
```

```
            end;


            if sheet ~= nil then
                    self.observer = ndb.newObserver(sheet);
                    self.observer.onPermissionListChanged =
                            function(node)
                                    self:atualizarCbxInvisivel();
                            end;
                    self.observer.onFinalPermissionsCouldBeChanged =
                            function(node)
                                    self:atualizarCbxInvisivel();
                            end;
                    self:atualizarCbxInvisivel();
            end;
    </event>
</form>
```

Criar uma janela acoplável de mesa

Uma janela acoplável de mesa segue os mesmos princípios de um modelo de ficha. A única diferença é que você deve usar "tablesDock" na propriedade "formType" da tag Form.

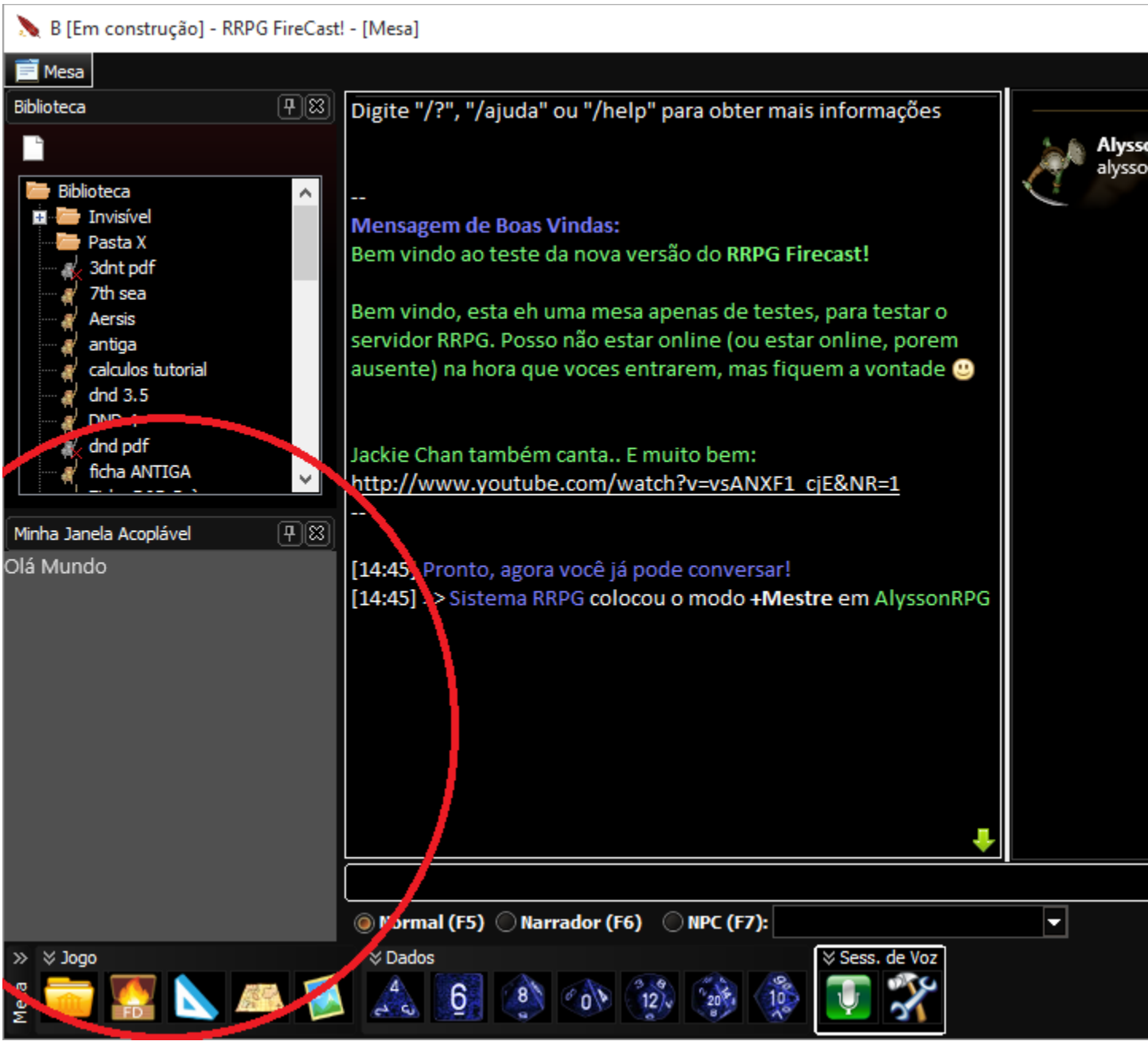
Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmMinhaJanelaAcoplavel" formType="tablesDock"
dataType="RRPG.DataTypeUnico.DaMinhaJanelaAcoplavel"
      title="Minha Janela Acoplável">


    <label text="Olá Mundo"/>


    ... Outras tags LFM e códigos lua.....
</form>
```

Agora basta compilar seu plug-in que o RRPG fará o resto =)



Como dito anteriormente, janelas acopláveis seguem os mesmos princípios de modelos de fichas e salvam/carregam dados em um NodeDatabase da mesa. A propriedade "dataType" da tag Form serve para informar ao RRPG onde os dados devem ser salvos, e, portanto, uma vez definido, é recomendado que não seja alterado.

Todos da mesa que estiverem com esta janela acoplável aberta lerão e salvarão dados em um único mesmo lugar: As mudanças no NodeDatabase serão sincronizadas com todos da mesa!

Criando um plug-in para vários idiomas

(Conteúdo desta página não consta no documento original.)

REFERÊNCIAS

Projetos de Plug-in

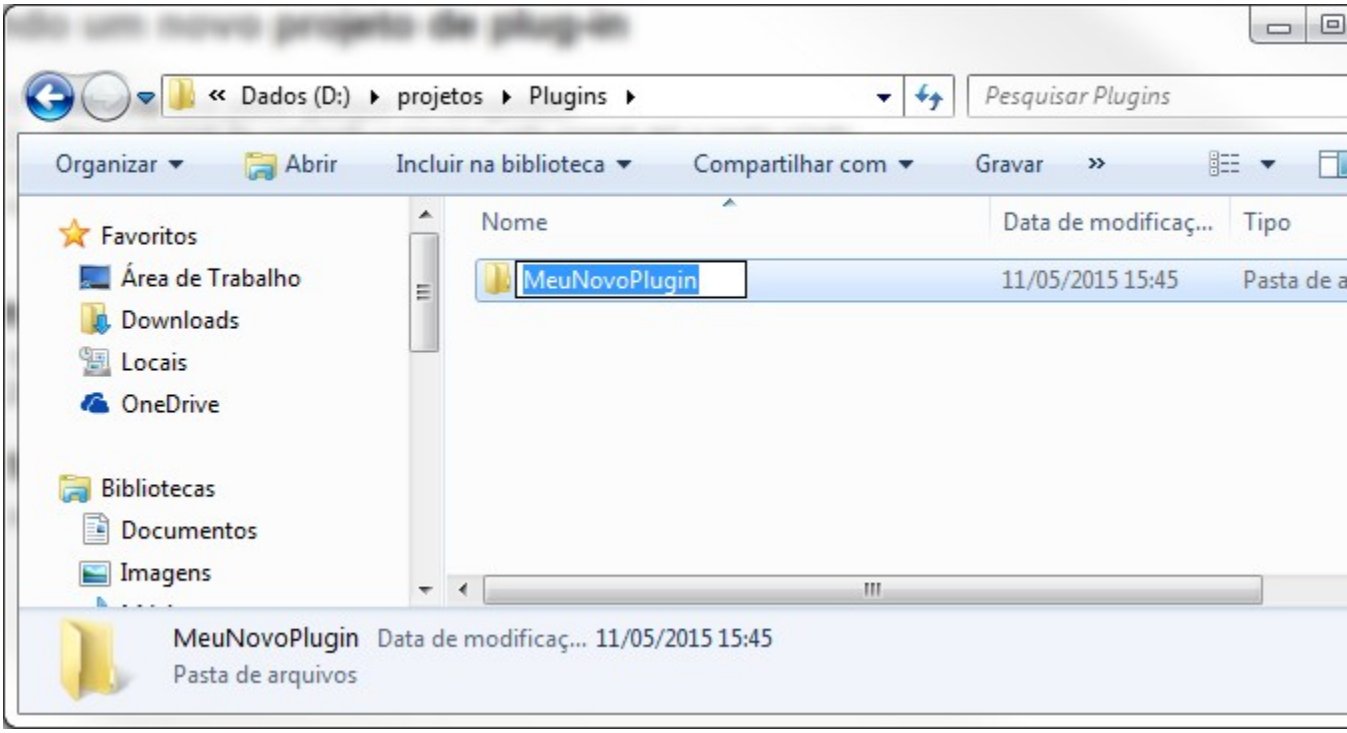
Um projeto de plug-in é um diretório no computador que contém todo o código fonte e arquivos auxiliares que são compilados e empacotados sob a forma de um único arquivo de extensão '.rpk', o plug-in.

Criando um novo projeto de plug-in

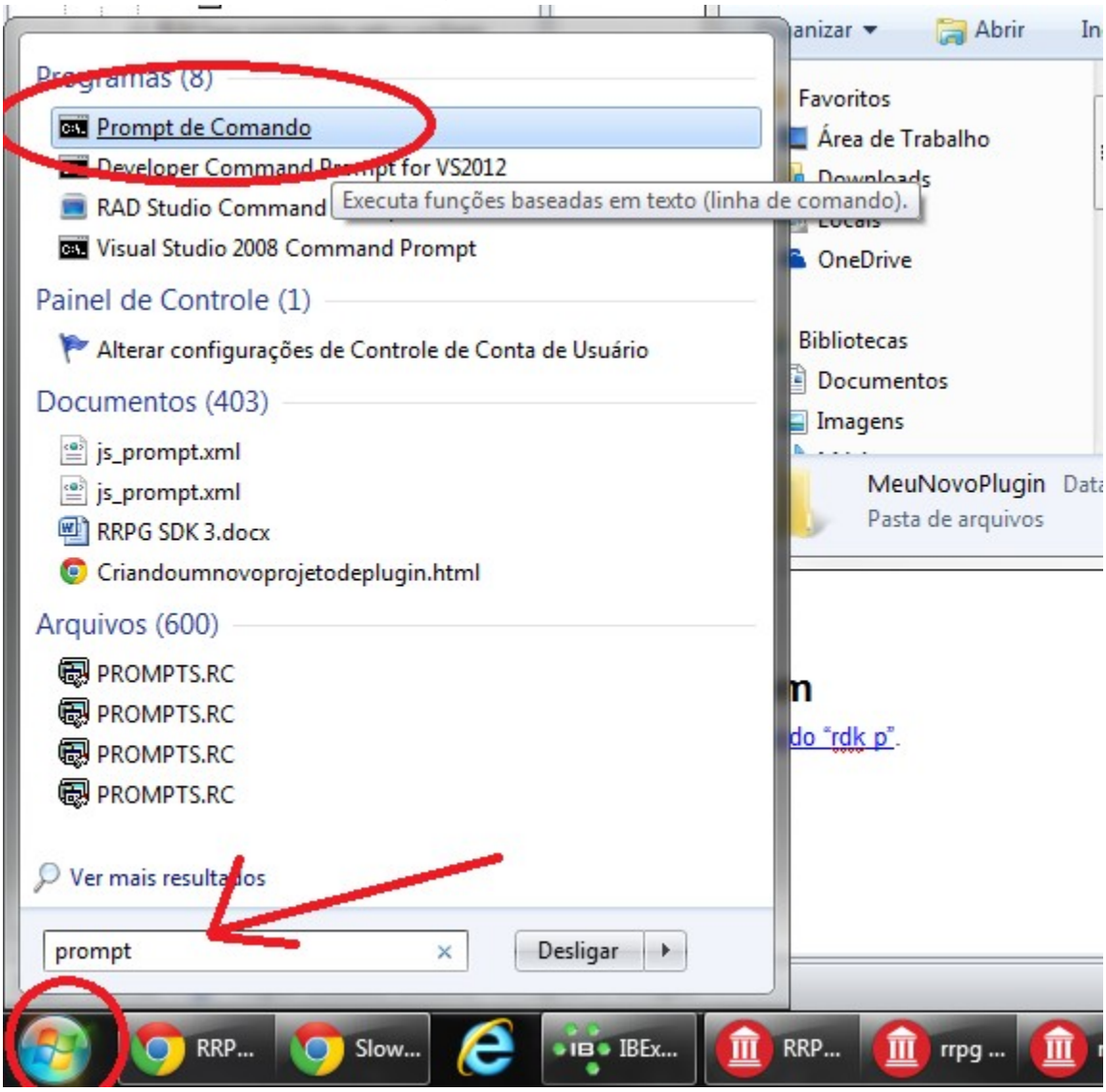
1. Crie uma nova pasta qualquer no computador.
2. Abra o prompt de comando e navegue pelo prompt até a pasta criada.
3. Execute o comando "rdk p", sem as aspas, dentro da pasta para transformar o diretório em um projeto de plug-in.
4. Vá até a pasta criada, abra, em um editor de texto, o arquivo "module.xml" que foi criado pelo comando acima e o preencha . Saiba mais sobre este arquivo lendo O arquivo "module.xml"

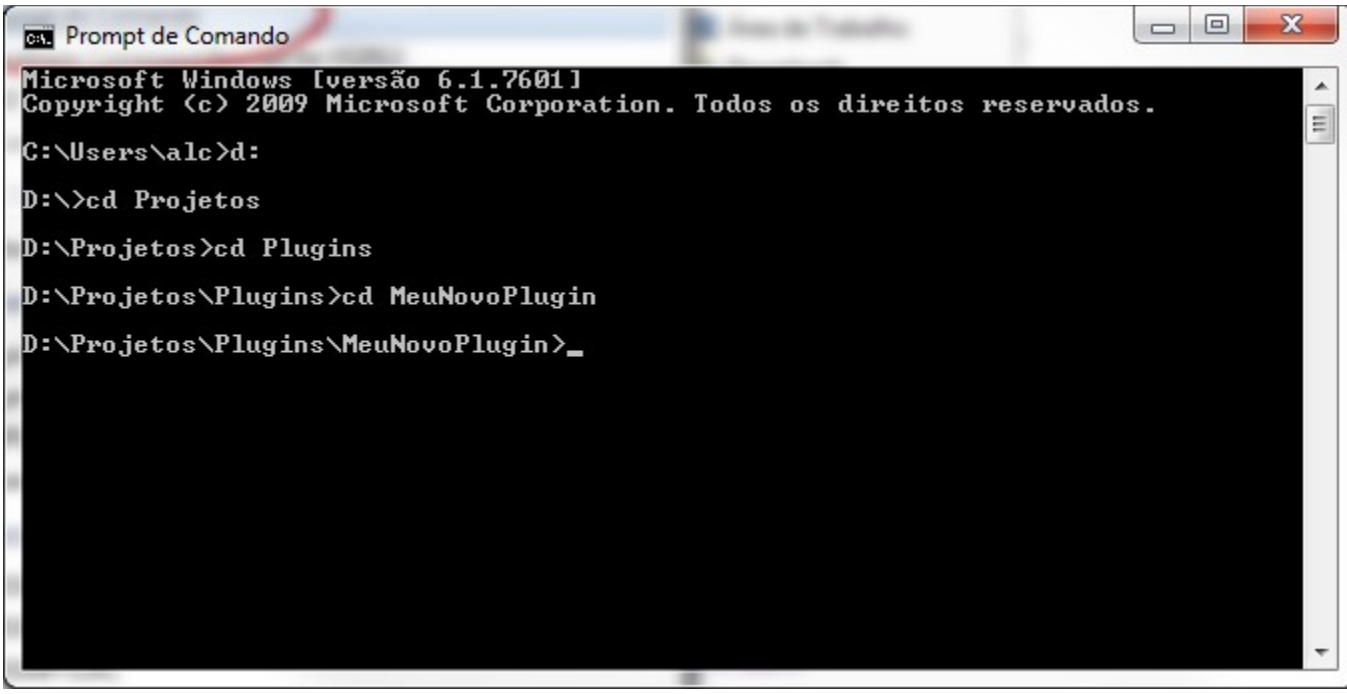
Exemplo passo a passo

1. Criando uma nova pasta no computador:

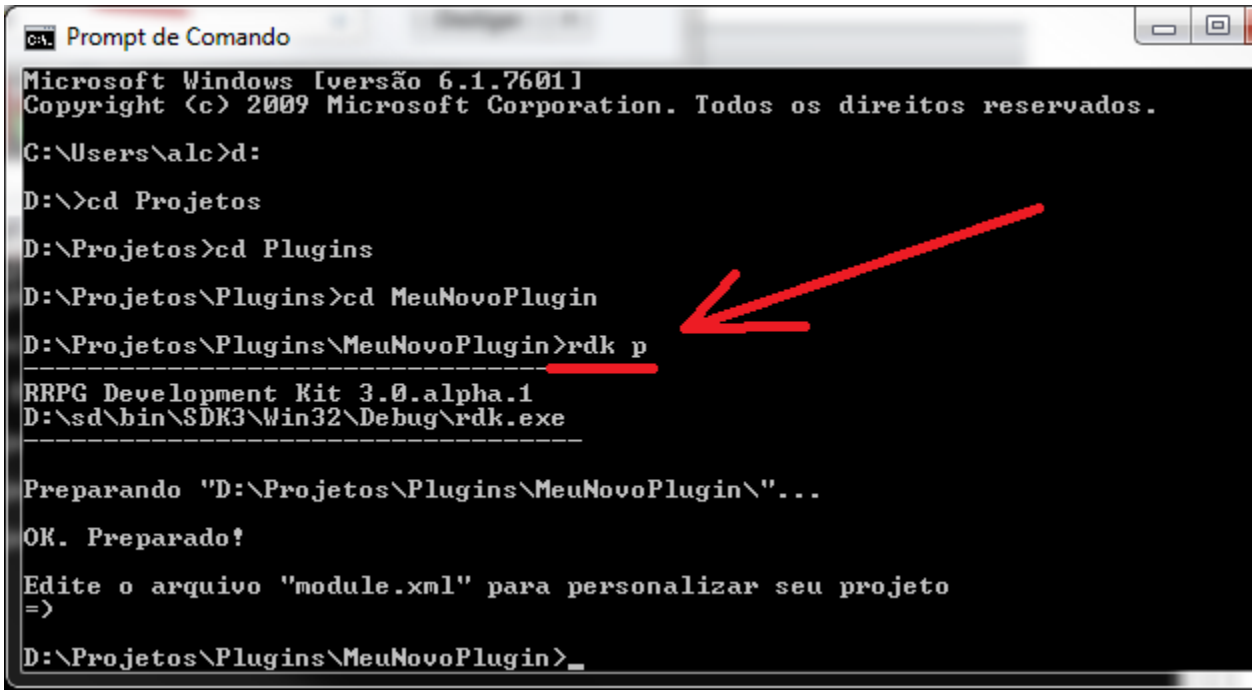


2. Abrindo o prompt de comando e navegando até a pasta criada...

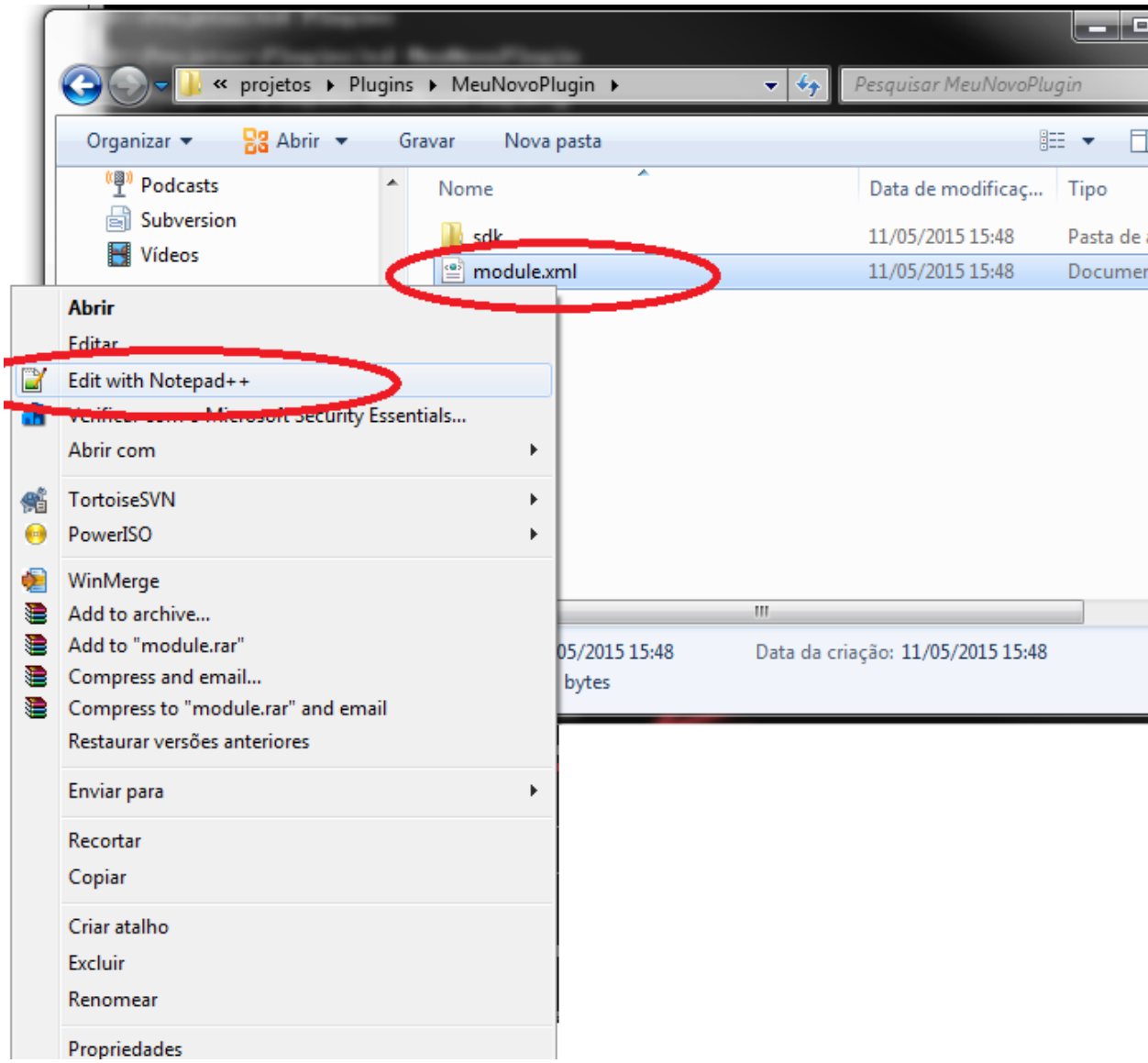


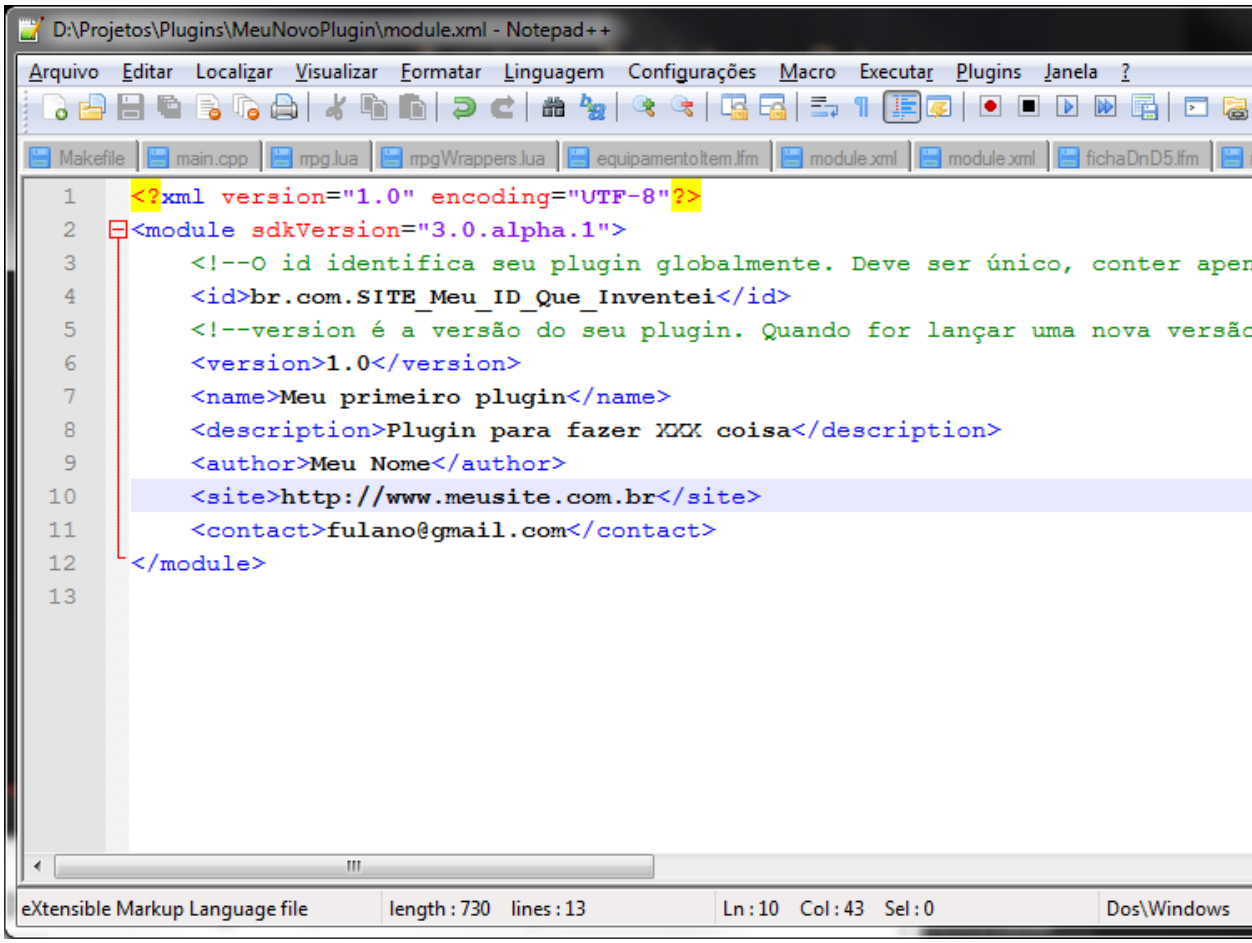


1. Executando o comando "rdk p"



2. Editando o arquivo "module.xml":


« projetos ▸ Plugins ▸ MeuNovoPlugin ▸
Pesquisar MeuNovoPlugin
Organizar
Abrir
Gravar
Nova pasta
Podcasts
Subversion
Vídeos
Nome
Data de modificaç...
Tipo
sdk
11/05/2015 15:48
Pasta de
module.xml
11/05/2015 15:48
Abrir
Editar
Edit with Notepad++
Abrir com
TortoiseSVN
PowerISO
WinMerge
Add to archive...
Add to "module.rar"
Compress and email...
Compress to "module.rar" and email
Restaurar versões anteriores
Enviar para
Recortar
Copiar
Criar atalho
Excluir
Renomear
Propriedades
05/2015 15:48
Data da criação: 11/05/2015 15:48
bytes



Testando o projeto de plug-in

Para testar seu projeto de plug-in, ele deve ser compilado e instalado no RRPG. Siga estes passos:

1. Abra o prompt de comando.
2. Navegue pelo prompt até a pasta do projeto de plug-in.
3. Execute o comando “rdk i”, sem as aspas, dentro da pasta para compilar e instalar o projeto de plug-in.
4. Teste seu plug-in no RRPG Firecast.

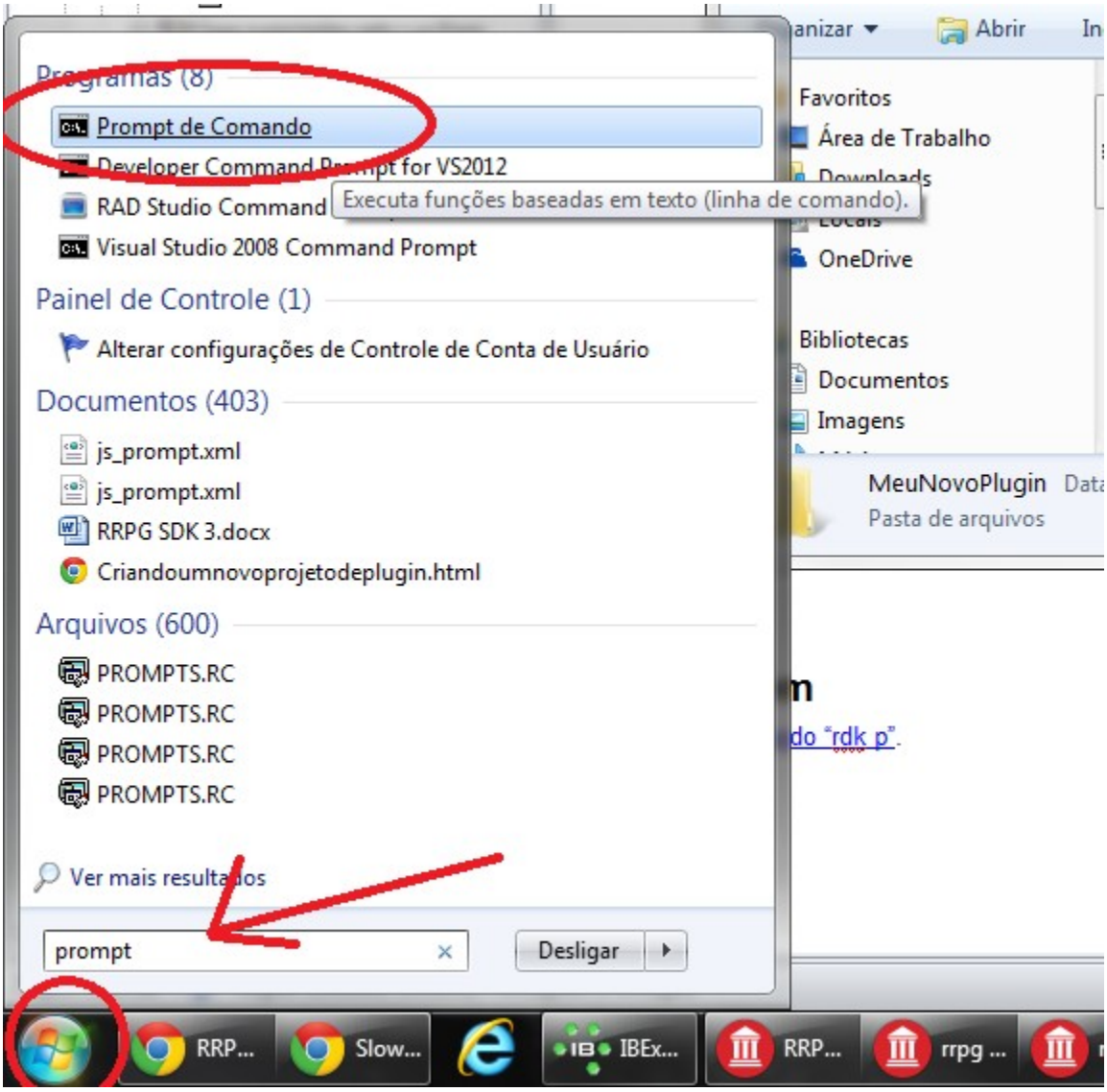
Observações:

- O comando “rdk i” não instala o plug-in caso encontre algum erro no projeto.
- Se o RRPG Firecast estiver em execução, o comando “rdk i” automaticamente descarrega o plug-in, instala a nova versão e recarrega o plug-in.

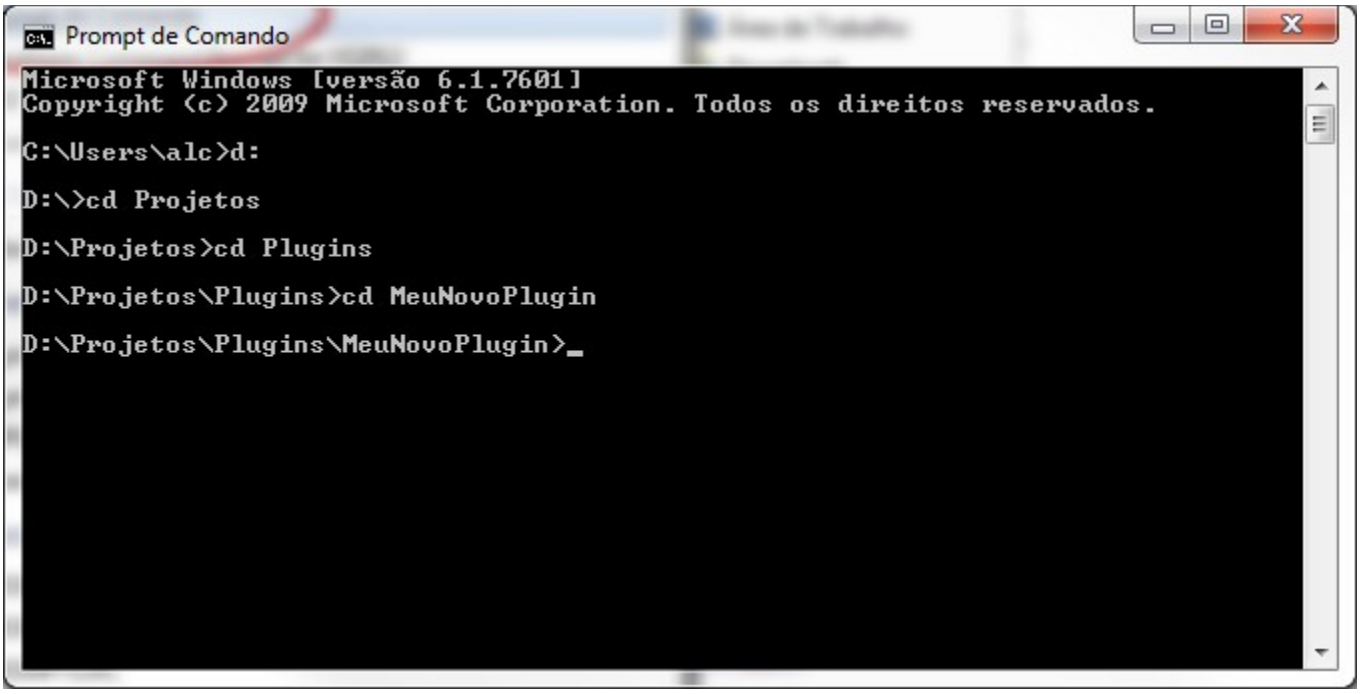
- Se o RRPG Firecast estiver fechado, o comando “rdk i” realiza uma instalação “off-line” e a nova versão será carregada quando o RRPG Firecast executar.

Exemplo passo a passo:

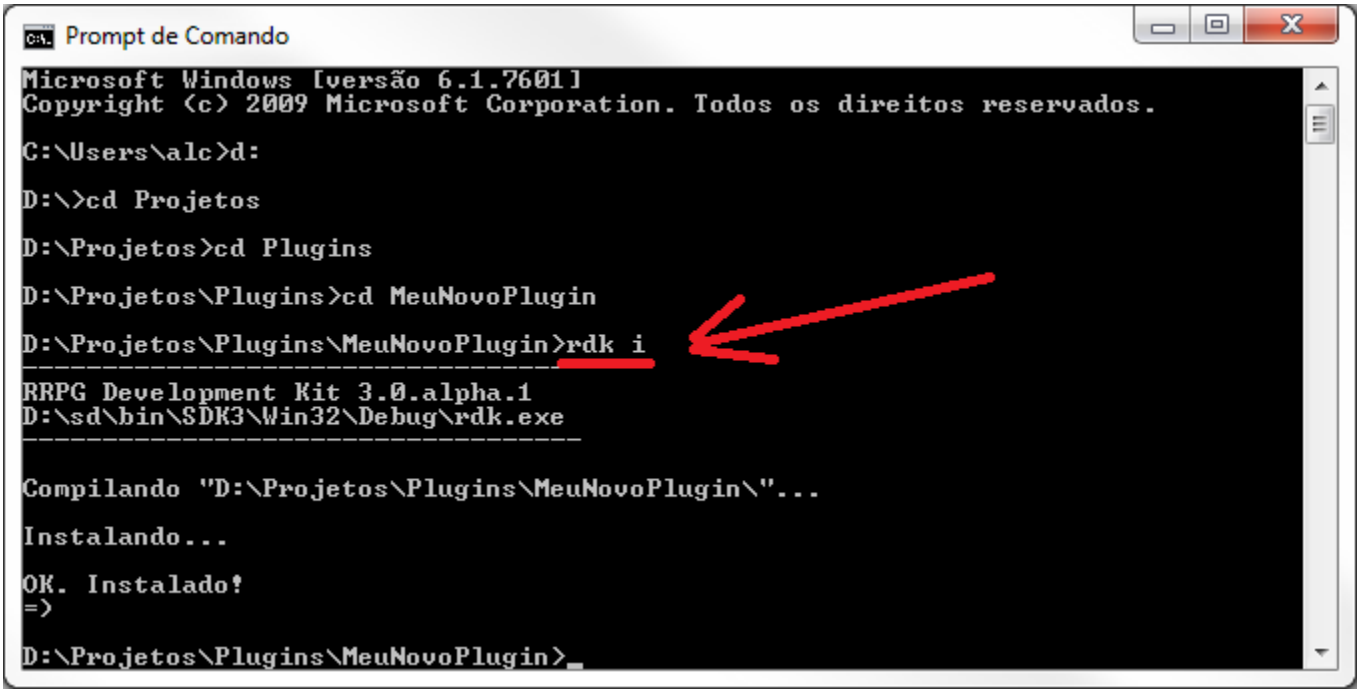
1. Abrindo o prompt de comando:



2. Navegando pelo prompt até a pasta do projeto de plug-in:



3. Executando o comando "rdk i"



Disponibilizando um plug-in

Agora que você já fez e testou seu plug-in, está na hora de o tornar disponível a outras pessoas!

Onde está o plug-in que criei?

Ao compilar seu projeto (comando "rdk i" ou comando "rdk c"), o arquivo final de seu plug-in é colocado dentro sub-pasta "output" do projeto.

Distribuição manual

Envie para os outros o arquivo que está dentro da sub-pasta "output"! (Um arquivo de extensão ".rpk")

Instalação manual do plug-in

É bem simples: Se a pessoa já possuir o RRPG Firecast instalado, ela só precisa "dar 2 cliques" no arquivo baixado e o processo de instalação se iniciará.

Uma maneira alternativa de instalar o plug-in é ir ao menu de plug-ins do RRPG, clicar no botão "instalar" e escolher o arquivo ".rpk" baixado.

Após a instalação manual, o arquivo ".rpk" pode ser excluído pois não interferirá no funcionamento do plug-in!

Arquivos em um projeto de plug-in

O arquivo "module.xml"

Todo projeto de plugin deve conter um arquivo de nome “module.xml” em sua raiz que contém as informações sobre o mesmo. A estrutura das tags/marcações deste documento é:

- **module**
    - **id** – Uma identificação global e única do plug-in. É um texto que contém apenas caracteres alfanuméricos, underlines, pontos e deve conter ao menos 5 caracteres e um máximo de 40. Você deve inventar um valor único para este campo, e, uma vez definido, o ID não deve mudar. Exemplos: “br.com.rrpg.DnD5_SDK3”, “br.com.outrosite.storytellerX”.
    - **version** – Um texto que identifica a versão do seu plug-in. Não se esqueça de atualizar este campo quando for lançar uma nova versão do seu plug-in =). Exemplos: “1.0”, “1.1”, “1.2b”
    - **info** – Tag que reúne as informações do puglin traduzidas em uma determinada língua. Pode aparecer mais de uma vez, uma para cada tradução. Ela contém as seguintes tags filhas:
        - **name** – O um texto curto que nomeia seu plug-in. Exemplos: “DnD 5”, “Plugin de Mighty Blade”, “Plugin X”, etc..
        - **description** – Um texto breve que descreve o que seu plug-in faz. Exemplos: “Adiciona suporte a fichas de DnD 5”.
        - **author** – Seu nome/nickname
        - **site** – Você possui um site? Coloq-ue aqui o endereço dele se tiver

        - **contact** – Como se faz contato com você? Adicione endereço de e-mail, facebook, twitter, etc..
    - **settings** – A tag to configure the behaviors of your plugin. Available settings are:
        - **automaticStringLocalization** – When true, it allows the Firecast to automatically translate attributes of LFM files and strings passed to the API calls. It's a very cool feature to make your plugin multi-language but may produce inconsistent behaviors. Default: false. See Criando um plug-in para vários idiomas

Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<module sdkVersion="3.6">
    <!--O id identifica seu módulo globalmente. Deve ser único,
conter apenas caracteres alfanuméricos, underlines, pontos e deve
conter ao menos 5 caracteres e um máximo de 40. Uma vez definido
o ID, você não deve alterar ele ;) -->
    <id>br.com.rrpg.DnD5_SDK3</id>

    <!--version é a versão do seu módulo. Quando for lançar uma
nova versão do seu módulo, altere aqui =) -->
    <version>1.0</version>

    <settings>

<automaticStringLocalization>true</automaticStringLocalization>
    </settings>

    <info lang="pt-BR">
        <!--Informações do plugin na língua "pt-BR". Você pode
ter várias tags "info", uma para cada tradução que quiser.-->
        <name>D&amp;D 5e</name>
        <description>Modelo de ficha de D&amp;D 5ª
edição</description>
        <author>Firecast</author>
        <site>https://firecast.app/</site>
        <contact>https://www.facebook.com/AlyssonRPG</contact>
    </info>

    <info lang="en">
        <!--Informações do plugin na língua "en". Você pode ter
várias tags "info", uma para cada tradução que quiser.-->
        <name>D&amp;D 5th Edition</name>
        <description>D&amp;D 5th edition character sheet for
firecast</description>
    </info>
</module>
```

Embedded font files

(Conteúdo desta página não consta no documento original.)

Ferramentas do SDK 3

Ferramenta rdk

“rdk” é a principal ferramenta do SDK3 e é usada como linha de comando no prompt.

comando "rdk p"

Este comando prepara o diretório corrente para o uso da ferramenta “rdk” mais nova.

Observações:

- Quando baixar uma nova versão do SDK 3, você deve executar este comando em cada um do seus projetos de plug-ins a fim de atualizá-los.
- Você pode preparar um diretório mais de uma vez sem causar nenhum problema.
- Este comando cria/atualiza o arquivo “module.xml” (O arquivo “module.xml”) .

comando "rdk c"

Este comando compila o diretório corrente e empacota todo o conteúdo em um único arquivo de extensão ‘.rpk’ em “<DiretorioCorrente>\output\<NomeDoModulo>.rpk”

Observações:

- O comando percorre todos os arquivos de extensão “lfm” e gera arquivos de extensão “lua” equivalentes.
- O comando realiza validações antes de empacotar o conteúdo do diretório, como:
  - O arquivo “module.xml” deve existir e estar preenchido corretamente (O arquivo “module.xml”) .
  - Os arquivos de extensão “lua” devem estar sintaticamente corretos.
  - Os arquivos de extensão “lfm” devem estar sintaticamente corretos, e possuir tags e atributos válidos.
- Os arquivos e subdiretórios de nomes iniciados com dois underlines (“__”) e o diretório “output” são ignorados por este comando.

comando "rdk i"

Este comando compila o diretório corrente (ver Comando “rdk c”) e instala o plugin gerado.

Observações:

- Se for encontrado algum erro de validação durante a compilação, o plug-in não é instalado.
- Se o RRPG Firecast estiver em execução, o comando “rdk i” automaticamente descarrega o plug-in, instala a nova versão e recarrega o plug-in no RRPG Firecast.
- Se o RRPG Firecast estiver fechado, o comando “rdk i” realiza uma instalação “off-line” e a nova versão será carregada quando o RRPG Firecast for executado

comando "rdk pdf"

Este comando abre uma ferramenta que converte arquivos no formato .PDF para o formato .SVG. Esta ferramenta funciona no navegador de internet.

Após a conversão, você poderá utilizar o comando "rdk lfm" para converter o conteúdo do .pdf para um Lua Form

comando "rdk lfm"

rdk lfm <arquivo>

Converte o arquivo informado para um Lua Form .

Formatos aceitos:

- Arquivos .svg
- Arquivos .zip que contém arquivos .svg

Se você deseja converter um .PDF para Lua Form, utilize a ferramenta "rdk pdf" para extrair o conteúdo do PDF como SVG antes =)

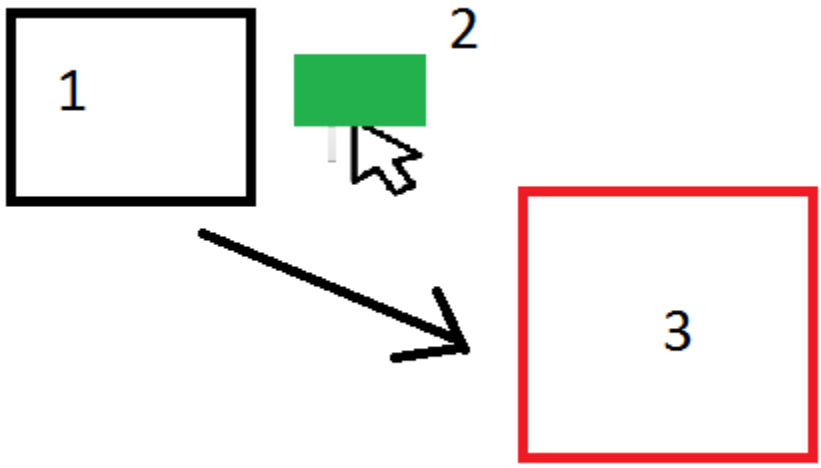
Arrastando e Soltando Informações / Drag And Drop

Arrastar e Soltar, do inglês "Drag and Drop", é a ação de clicar em um objeto virtual e "arrastá-lo" a uma posição diferente ou sobre um outro objeto virtual. De maneira geral, ele pode ser usado para invocar diversos tipos de ações, ou criar vários tipos de associações entre dois objetos abstratos.
A sequência básica de ações relacionadas ao drag-and-drop é:

- Clicar, e manter o botão do mouse apertado para "agarrar" o objeto,
- "Arrastar" (drag) o objeto/cursor à posição ou localização desejada,
- "Soltar" (drop) o objeto, soltando o botão pressionado.

Fluxo de acontecimento

No exemplo a seguir, o usuário iniciou o processo drag-and-drop agarrando o objeto preto (1 na figura), arrastando o mouse até o objeto vermelho (3 na figura) e soltando o botão.



O fluxo segue na seguinte ordem:

- O Usuário clica e mantêm o botão do mouse pressionado sobre o objeto 1 para iniciar um processo de drag-and-drop.
- O Firecast, através do evento onStartDrag do objeto 1, pede para que o objeto 1 informe as informações que o usuário pode arrastar dele.
- O Plugin SDK3, dentro da chamada do evento onStartDrag, informa ao Firecast o que o usuário pode arrastar do objeto 1. Os dados são adicionados a um objeto Drag (2 na figura) que o Firecast cria automaticamente para esta operação de drag-and-drop específica.
- O Firecast interage com o usuário até que ele passe o mouse sobre uma área de Drop, isto é, de um objeto que aceite que operações drag-and-drop sejam "largadas" sobre ele. O objeto 3 possui uma área de drop.
- O Firecast, através do evento onStartDrop do objeto 3, pede para que o objeto 3 informe o quais informações o usuário pode "largar" sobre ele.
- O Plugin SDK3, dentro da chamada do evento onStartDrop, informa ao Firecast o que o usuário pode largar sobre o objeto 3. Os dados são adicionados a um objeto Drop que o Firecast cria automaticamente para receber estas informações.
- O Firecast, com base nas informações do objeto Drag preenchidas pelo objeto 1 e nas informações do objeto Drop preenchidas pelo objeto 3, decide se a operação drag-and-drop pode acontecer. Se um não conseguir lidar com as informações do outro, a operação drag-and-drop não poderá ser concluída.
- Caso a operação Drag-and-Drop puder ser finalizada, o Firecast chama a ação do objeto 3 ou objeto 1 que melhor puder lidar com esta operação.

Informações e Ações em uma operação drag-and-drop

Para que uma operação Drag-And-Drop aconteça, é necessário haver ao menos uma informação e ao menos uma ação capaz de lidar com ela.
Informações e ações podem existir tanto no objeto Drag quanto no objeto Drop:

- Informações existentes no objeto Drag são combinadas com ações existentes no objeto Drop
- Informações existentes no objeto Drop são combinadas com ações existentes no objeto Drag

Informações e ações estão atreladas a um DataType, isto é, a uma cadeia de caracteres que define o tipo de dados que é arrastado.

DataTypes padrões do Firecast

O programador pode inventar qualquer DataType, porém existem os seguintes que são padrões do Firecast:

DataType padrão	Valor transportado
text	Uma cadeia de caracteres, um texto qualquer.
imageURL	Uma cadeia de caracteres que contém uma URL de uma imagem.
player	Um objeto Jogador
players	Array de objeto Jogador
character	Um objeto Personagem
characters	Array de objeto Personagem
libraryItem	Um objeto BibliotecaItem
libraryItens	Array de objeto BibliotecaItem
sceneUnitClass	Um objeto SceneUnitClass
sceneUnitClasses	Array de objeto SceneUnitClass
imageFile	A table containing the following attributes: ● name - String with the name of the file. ● mimeType - String with the MIME type of the image (examples: image/jpeg, image/png) ● stream - A read-only Stream object containing the image file bytes.
imageFiles	An array of tables with the same structure as the "imageFile".

Observação: Ao utilizar os dataTypes padrões, o RRPG valida e exige que sejam transportados valores adequados conforme a tabela acima.

Exemplos

Exemplo 1 - Arrastando um texto de um Label

Experimente arrastar o label para a caixa de digitação de mensagens e comandos do RRPG.

```
<label text="Me arraste!" top="150" hitTest="true">
    <event name="OnStartDrag">
        drag:addData("text", "Texto que será arrastado deste
label!");
    </event>
</label>
```

Exemplo 2 - Arrastando uma imagem e um texto de um Label

Experimente arrastar o label para o log do chat do RRPG!

```
        <label text="Me arraste!" top="150" hitTest="true">
            <event name="OnStartDrag">
                drag:addData("text", "Texto que será arrastado deste
label!");
                drag:addData("imageURL", "http://www.rrpg.com.br/wp-
content/uploads/2013/05/cropped-Untitled-2.png");
            </event>
        </label>
```

Exemplo 3 - Um controle que captura arraste de imagens.

Arraste uma imagem do browser Chrome ou do jogador para este controle.

```
        <rectangle left="150" top="150"
                   width="150" height="150"
                   strokeColor="yellow" strokeSize="1"
                   color="none">

            <image name="imgImagem" align="client" hitTest="true">
                <event name="OnStartDrop">
                    drop:addAction("imageURL",
                        function(value)
                            self.imgImagem.src = value;
                        end);
                </event>
            </image>

        </rectangle>
```

Exemplo 4 - Um controle que captura arraste de Jogadores.

Arraste um jogador da mesa para este controle!

```
        <rectangle left="150" top="150"
                   width="150" height="150"
                   strokeColor="yellow" strokeSize="1"
                   color="none"
                   hitTest="true">

            <event name="OnStartDrop">
                drop:addAction("player",
                    function(value)
                        showMessage("O login deste jogador é: " ..
value.login);
                    end);
            </event>
        </rectangle>
```

Exemplo 5 - Arrastando uma ação à partir de um botão

Arraste este botão para um jogador da mesa!

```
            <button left="150" top="150" text="meArraste">
                <event name="OnStartDrag">
                    drag:addAction("player",
                        function(destValue)
                            showMessage("Você me arrastou para o jogador: "
    .. destValue.login);
                        end);
                </event>
            </button>
```

Chat messaging structures

This section describes common structures and Lua tables used to handle chat messaging jobs.

Entity table

The Entity table contains information about a user. It is a Lua table with the following fields:

Field	Type	Description
id	Integer	Contains the internal ID of the user.
login	String	Contains the username of the user. If the value is equal to "@Firecast", the entity is the Firecast server itself.
nick	String	*(Optional)* Contains the nickname of the user. If this field is missing, the value of the field login is assumed as nick.
gender	Enumerated/String: ● "undefined" ● "masculine" ● "feminine" ● "neuter"	*(Optional)* Defines the grammatical gender to be used when referring to the user. ● "undefined" - Gender is not specified. ● "masculine" - Masculine gender. ● "feminine" - Feminine gender. ● "neuter" - Neuter gender. If this field is missing, the grammatical gender "undefined" is assumed.

Impersonation table

The Impersonation table is a Lua table utilized to specify how a message is impersonated. It contains the following fields:

Field	Type	Description

mode	Enumerated/String: ● "none" ● "narrator" ● "character"	Indicates the mode of impersonation: ● "none" - No impersonation is used. ● "narrator" - Impersonates the narrator/GM. ● "character" - Impersonates a character, perharps an NPC. If this field is missing, the mode "none" is assumed.
avatar	String	*(Optional)* A URL string pointing to an image used as the avatar of the impersonation. This field is optional and applicable to "narrator" and "character" impersonation modes.
gender	Enumerated/String: ● "undefined" ● "masculine" ● "feminine" ● "neuter"	*(Optional)* Defines the grammatical gender to be used when referring to the impersonated entity. ● "undefined" - Gender is not specified. ● "masculine" - Masculine gender. ● "feminine" - Feminine gender. ● "neuter" - Neuter gender. If this field is missing, the grammatical gender "undefined" is assumed.
name	String	*(Optional)* Defines the name and/or nickname of the character being impersonated. This field is optional and utilized with the "character" impersonation mode.

LogRec table

The LogRec table describes a message in a chat. It is a Lua table with the following fields:

Field	Type	Description
id	String	Identification of the message, generated by Firecast Client. The message is uniquely identified by combining the fields **id** and **tid**.

tid	String	Identification of the message, generated by Firecast Server. The message is uniquely identified by combining the fields **id** and **tid**. The name **tid** stands for "timestamp id" because its value includes a server timestamp of when it was generated. Firecast uses **tid** to sort messages and perform sequential scans.
timestamp	Timestamp table	When the message was sent in UTC.
medium	Medium table	Identifies the chat in which the message was sent.
entity	Entity table	This field identifies who sent the message. In the case of a kick, this field contains the user who was kicked, and "msg.responsible" identifies who performed the kick.
msg	MsgContent table	The content of the message.
visibility	MsgVisibility table	Describes the visibility of the message.
updates	Array of MsgUpdate table	*(Optional)* An array containing the update history for this message, sorted in edition order. This message has not been updated if this field is missing or its length is zero.
isDeleted	Boolean	*(Optional)* Indicates if the message was deleted from the chat. If missing or **nil**, the message was not deleted.
deleter	Entity table	*(Optional)* Valid if isDeleted == true, identifies who deleted the message.
deletionTimestamp	Timestamp table	*(Optional)* Valid if isDeleted == true, contains the timestamp in UTC of when the message was deleted.
_visible	Boolean	An auxiliary flag, which is not part of the message itself. Indicates if this message can be visible to the user.
_storable	Boolean	An auxiliary flag, which is not part of the message itself. Indicates if this message can be both visible to the user and can be stored in the Firecast server log.

Medium table

The Medium table identifies a communication medium in Firecast. It is a Lua table with the following fields:

Field	Type	Description
kind	Enumerated/String: ● "undefined" ● "direct" ● "room" ● "pvtOnRoom" ● "groupPvtOnRoom" ● "roomChannel"	Defines the kind of the communication medium an can be one of the following values: ● "undefined" - Should not be used. ● "direct" - A contact list chat. ● "room" - A room chat. ● "pvtOnRoom" - A private chat whitin a room. ● "groupPvtOnRoom" - A private group chat within a room. ● "roomChannel" - A room channel chat (future implementation).
user1 **user2**	Integer	*(Optional)* These two fields are valid when kind is "direct" or "pvtOnRoom" and contain the internal identification of the users involved in the chat. Remarks: ● The value stored in the field "user1" must be less than or equal to the value stored in the field "user2".
roomId	Integer	*(Optional)* Valid when kind is "room," "pvtOnRoom," "groupPvtOnRoom," or "roomChannel." Contains the internal identification of the room.
groupId	String	*(Optional)* Valid when kind is "groupPvtOnRoom" and contains the unique identification of the private group chat inside a room.
channelId	String	*(Optional)* Valid when kind is "roomChannel" and contains the unique identification of the channel inside a room (future implementation).

MsgContent table

The MsgContent table describes the content of a message. It is a Lua table with the following fields:

Field	Type	Description
msgType	String/Enumerated:	dentifies the type of message and

	• "standard" • "action" • "dice" • "laugh" • "file" • "startTypingStatus" • "stopTypingStatus" • "signal" • "sys_notification" • "sys_mediumCreation" • "sys_mediumDestruction" • "sys_mediumPropertyChange" • "sys_userJoin" • "sys_userLeave" • "sys_userPresencePropertyChange" • "sys_genericLogMessage"	can be one of the following values: • "standard" - A standard chat message • "action" - An action message, when the /me command is used • "dice" - A dice roll message. • "laugh" - A laugh message, when the /rir command is used • "file" - A file upload message. • "startTypingStatus" - A message used to signal when a user is typing (not persisted, not shown) • "stopTypingStatus" - A message used to signal when a user stops typing (not persisted, not shown). • "signal" - A custom signal message (not persisted, not shown). • "sys_notification" - A notification generated by the Server. • "sys_mediumCreation" - A message about when the chat medium was created in the server. • "sys_mediumDestruction" - A message about when the chat medium was destroyed in the server. • "sys_mediumPropertyChange" - A message that describes a change made to a property of the chat medium. • "sys_userJoin" - A message informing that some user joined the chat medium. • "sys_userLeave" - A message informing that some user left the chat medium.

		● "sys_userPresencePropertyChange" - A message that describes a change made to a user's presence in the chat medium. ● "sys_genericLogMessage" - A client generic log message (not persisted and not sent to the server). Remarks: ● If this field is missing or empty, the message is not handled by the client because, probably, the Firecast client needs to be updated.
content	String	The string message content.
roll	Rollagem object	*(Optional)* Valid when msgType == "dice". Describes the dice roll and its result.
impersonation	Impersonation table	Defines how this message is impersonated.
talemarkOptions	TalemarkOptions table	The TalemarkOptions of this message, describing how Firecast should color, format, and parse the message.
responsible	Entity table	*(Optional)* If present, this field identifies the responsible of the event, like who kicked a user, who invited a user to a PVT group chat, who changed a property in the chat, and so on.
propertyId	Any type	*(Optional)* Valid for msgType "sys_mediumPropertyChange" and "sys_userPresencePropertyChange". Contains the identification of what property was changed. Some of the known values of this field are listed below: ● "modes" - User modes (gm, mute, voice, blind, player, juggernaut) ● "nick" - User nick ● "isModerated" - The moderation flag of the chat medium

		● "password" - The password required to join the chat medium. ● "welcomeMsg" - The welcome message of the chat medium. ● "name" - The name of the chat medium ● "statusMsg" - A status string of the chat medium. ● "tags" - The tags of the chat medium
oldValue	Any type	*(Optional)* Valid for msgType "sys_mediumPropertyChange" and "sys_userPresencePropertyChange". Contains the previous value of the property before its change.
newValue	Any type	*(Optional)* Valid for msgType "sys_mediumPropertyChange" and "sys_userPresencePropertyChange". Contains the new value of the property after its change.
fileName	String	*(Optional)* Valid when msgType == "file". Contains the name of the file.
fileURL	String	*(Optional)* Valid when msgType == "file". Contains the URL address to download the file.
fileMimeType	String	*(Optional)* Valid when msgType == "file". Contains the Mime Type of the file, like 'image/png', 'text/html', 'video/mp4', and so on.
fileSize	Integer	*(Optional)* Valid when msgType == "file". Contains the file size in byte count.
joinType	String/Enumerated: ● "conscious" ● "added"	*(Optional)* Valid when msgType == "sys_userJoin". Defines how the user joined the chat medium and can be one of the following values: ● "conscious" - The join was conscious; the user itself joined the chat medium.

		● "added" - he join occurred because someone added him/her/them to the medium chat.
leaveType	String/Enumerated: ● "standard" ● "kicked"	*(Optional)* Valid when msgType == "sys_userLeave". Defines how the user left the chat medium and can be one of the following values: ● "standard" - The user left the chat medium, or some disconnection happens. ● "kicked" - The user left the chat because someone, identified by the field "responsible," has kicked him/her/them.
extra	A custom Lua table	*(Optional)* If defined, contains custom extra details of the message. Its content is not defined and can be anything. Used in msgType == "signal".

MsgUpdate table

The MsgUpdate table describes a single update made to the message. It is a Lua table with the following fields:

Field	Type	Description
timestamp	Timestamp table	The timestamp in UTC of when this update was made to the message.
updater	Entity table	This field identifies who changed the message.
contentChanged	Boolean	*(Optional)* Indicates if the message content was changed in this update. Assumes **false** If this field is missing or **nil.**
oldContent	String	*(Optional)* Valid when contentChanged == true, contains the message content string before the update.

impersonationChanged	Boolean	*(Optional)* Indicates if the message impersonation was changed in this update. Assumes **false** If this field is missing or **nil.**
oldImpersonation	Impersonation table	*(Optional)* Valid when impersonationChanged == true, contains the message impersonation before the update.
talemarkOptionsChanged	Boolean	*(Optional)* Indicates if the message TalemarkOptions was changed in this update. Assumes **false** If this field is missing or **nil.**
oldTalemarkOptions	TalemarkOptions table	*(Optional)* Valid when talemarkOptionsChanged == true, contains the message TalemarkOptions before the update.

MsgVisibility table

The MsgVisibility table describes the visibility of a message. It is a Lua table with the following optional fields:

Field	Type	Description
mode	Enumerated/String: ● "standard" ● "recipients"	Defines the message visibility mode: ● "standard" - The standard mode, where the message is visible to all. ● "recipients" - Targeted mode, where the field "recipients" defines who should be able to view the message If this field is missing, the mode "standard" is assumed.

recipients	Array of Integer	*(Optional)* A Lua array containing the internal IDs of Firecast users that defines the message recipients. When mode == "standard", this field is ignored when sending messages, but Firecast Server uses it to describe who was in the chat at the moment the message was sent and could see it. When mode == "recipients", only those who are in this array can see the message. Internal IDs can be obtained using: ● Firecast.getCurrentUser().id ● Jogador Object's id property
blacklist	Array of Integer	(Optional) A Lua array containing the internal IDs of Firecast users who **can't** see the message. It can be used when mode == "standard". Internal IDs can be obtained using: ● Firecast.getCurrentUser().id ● Jogador Object's id property

QueryLogRecsParams table

The QueryLogRecsParams table is utilized to specify how to retrieve messages stored in the Firecast server. It is a Lua table with the following optional fields:

Field	Type	Description
audit	Boolean	*(Optional)* Indicates whether the server should include deleted messages in the response. If this field is missing, it assumes a default value of "false".
limit	Integer	*(Optional)* Specifies the number of messages to fetch. Valid values range from 1 to 50. If absent, the default value assumed is 50.
timestamp	Timestamp table	*(Optional)* If defined, queries for the latest messages with a timestamp less than or equal to this value. Remarks: ● It is not possible to use both "tid" and "timestamp" fields simultaneously. ● If both the "tid" and "timestamp" fields are missing, the latest messages of the chat

		are fetched.
tid	String	(Optional) If defined, queries for the latest messages with **tid** less than this **tid**. Remarks: ● **tid** stands for timestamp id, representing a unique identifier for each message, used for sorting and sequential scanning of messages in the log. ● The message with **tid** equal to this **tid** is not fetched; only the preceding ones. ● It is not possible to use both "tid" and "timestamp" fields simultaneously. ● If both the "tid" and "timestamp" fields are missing, the latest messages of the chat are fetched.

QueryLogRecsResponse table

The QueryLogRecsResponse table contains the response of a query for messages stored in the Firecast server. It is a Lua table with the following fields:

Field	Type	Description
bof	Boolean	This field indicates whether the query reached the beginning of the file. If **true**, no more messages exist with the same query parameters. If **false**, you can fetch more messages using the **"tid"** of the first message in the "logRecs" field. Remarks: ● The "logRecs" field may contain messages even when "bof == true". In this case, the messages in the "logRecs" field are the last messages that could be fetched with the same query parameters.
logRecs	Array of LogRec table	The messages fetched from the server, sorted by their **tid** (timestamp id).

SendMessageParameters table

The SendMessageParameters table is employed to define additional parameters when sending messages to chats. It is a Lua table with the following optional fields:

Field	Type	Description

impersonation	Impersonation table	*(Optional)* Describes how to impersonate the message. If missing, the message will not be impersonated.
talemarkOptions	TalemarkOptions table	*(Optional)* Describes how to color, format, and parse the message. If missing, the user's current talemark options will be used.
visibility	MsgVisibility table	(Optional) Describes who will receive the message. If missing, the standard visibility will be used.

TalemarkBlockStyle table

The TalemarkBlockStyle table specifies the text style of a message block. It is a Lua table with the following optional fields

Field	Type	Description
color	String	*(Optional)* A color string or a number with the pallete index defining the text font color. If missing, a default color will be assumed.
bkgColor	String	*(Optional)* A color string or a number with the pallete index defining the text background color. If missing, a default background color will be assumed.
bold	Boolean	*(Optional)* Should the text use bold style? Default: false
italic	Boolean	*(Optional)* Should the text use italic style? Default: false
strikeout	Boolean	*(Optional)* Should the text be struck out? Default: false
underline	Boolean	*(Optional)* Should the text be underlined?? Default: false

TalemarkOptions table

The TalemarkOptions table specifies how Firecast should color, format, and parse a message. It is a Lua table with the following optional fields:

Field	Type	Description

defaultTextStyle	TalemarkBlockStyle table	*(Optional)* Describes the default text style. If missing, a default style will be assumed.
charActionTextStyle	TalemarkBlockStyle table	*(Optional)* Describes the text block style for character actions. If missing, a default style will be assumed.
charEmDashSpeechTextStyle	TalemarkBlockStyle table	*(Optional)* Describes the text block style for character speeches marked by "--". If missing, a default style will be assumed.
charQuotedSpeechTextStyle	TalemarkBlockStyle table	*(Optional)* Describes the text block style for character speeches marked by quotes. If missing, a default style will be assumed.
outOfCharTextStyle	TalemarkBlockStyle table	*(Optional)* Describes the text block style for out-of-character interlocutions. If missing, a default style will be assumed.
parseCharActions	Boolean	*(Optional)* Should parse character actions marked with * * delimiters? Default: true
parseCharEmDashSpeech	Boolean	*(Optional)* Should parse character speeches marked with -- delimiters? Default: true
parseCharQuotedSpeech	Boolean	*(Optional)* Should parse character speeches marked by quotes? Default: true
parseCommonMarkStrongEmphasis	Boolean	*(Optional)* Should parse strong/bold emphasis marked by ** ** and/or __ __ delimiters? Default: true
parseHeadings	Boolean	*(Optional)* Should parse headings marked by #,

		===, and/or ---- delimiters? Default: true
parseHorzLines	Boolean	*(Optional)* Should parse horizontal lines marked by ----- and/or ****? Default: true
parseInitialCaps	Boolean	*(Optional)* Should parse initial caps marked by {} delimiters? Default: true
parseOutOfChar	Boolean	*(Optional)* Should parse out-of-character interlocution marked by (( )) delimiters? Default: true
parseSmileys	Boolean	*(Optional)* Should parse smileys and memes? Default: true
parseURL	Boolean	*(Optional)* Should parse URLs and links? Default: true
trimTexts	Boolean	*(Optional)* Should excess spaces in the text be trimmed? Default: true

TalemarkTag table

The TalemarkTag table contains a single parsed tag of a talemark markup text. It is a Lua table that forms a tree structure with the following optional fields:

Field	Type	Description
type	String/Enumerated: ● "root" ● "paragraph" ● "heading" ● "horzLine" ● "quotedSpeech" ● "emDashSpeech" ● "charAction" ● "outOfCha	Present in all tags, contains the identification of what tag is this, and can be one of the following values: ● "root" - The root tag, used to group all other tags inside. ● "paragraph" - A standard paragraph. ● "heading" - A heading paragraph. ● "horzLine" - A horizontal line paragraph. ● "quotedSpeech" - A span to delimit quoted speech.

	r" ● "initialCaps" ● "emphasis" ● "strong" ● "fontStyle" ● "fontColor" ● "bkgColor" ● "text" ● "breakline" ● "url" ● "image" ● "smiley"	● "emDashSpeech" - A span to delimit em-dash speech. ● "charAction" - A span to delimit character action. ● "outOfChar" - A span to delimit out-of-character interlocution. ● "initialCaps" - A style-span whose content must be presented as an initial capital letter. ● "emphasis" - A style-span whose content is emphasized, usually in italic. ● "strong" - A style-span whose content is strongly emphasized, usually in bold. ● "fontStyle" - A style-span whose content must be presented according to the "style" field. ● "fontColor" - A style-span whose text must be presented with the font color defined by the fields "mode" and "color". ● "bkgColor" - A style-span whose text must be presented with the background color defined by the fields "mode" and "color". ● "text" - A text element whose content is stored in the field called "text". ● "breakline" - A break-line element. ● "url" - A link text element whose content is stored in the field "text" and the URL address stored in the "url" field. ● "image" - An image element, whose URL is stored in the "url" field. ● "smiley" - A smiley element identified by the "id" field.
children	Array of TalemarkTag table	*(Optional)* As TalemarkTag tables represent a tree structure, this field contains the children nodes and/or leaves of this tag.

		Remarks: ● If this field is missing or **nil**, the tag does not contain children tags. Always test if this field is missing or **nil**.
altText	String	*(Optional)* Valid when type == "smiley". If present, contains a Unicode string/emoji that can be used as an alternative way to present the smiley.
aspect	String/Enumerated: ● "stretch" ● "proportional" ● "autoFit"	*(Optional)* Valid when type == "image". Determines how the image should be presented, regarding its aspect ratio: ● "stretch" - The image should be resized so that it occupies all the space, being able to flatten or lengthen. ● "proportional" - The image must be resized proportionally to the available size, without the need to occupy all the available space. ● "autoFit" - The image must be displayed in such a way that it always occupies all the available space in the most appropriate way, that is, enlarging/decreasing the image, centering it, and cropping excesses.
color	String or Integer	*(Optional)* Valid for types "fontColor" and "bkgColor": Contains the color identification. The meaning of this value is stored in the field "mode".
descent	Number	*(Optional)* Valid when type == "image". If present, contains how much, in logical pixels, the image should descend the text baseline when presenting it.

float	String/Enumerated: ● "left" ● "right"	*(Optional)* Valid when type == "image". If present, indicates how the image should float in the text: ● "left" - The image should float on the left, and the text should surround it on the right. ● "right" - The image should float on the right, and the text should surround it on the left.
height	Number	*(Optional)* Valid when type == "image". If present, contains the height, in logical pixels, in which the image should be presented.
id	String	*(Optional)* Valid when type == "smiley". Contains the identification of what smiley is this.
isATX	Boolean	*(Optional)* Valid when type == "heading". Indicates if the heading was marked up in the text as an ATX-Style. False means the text was marked up as Setext-Style. ATX-Style example: ## My Heading Text Example Setext-Style example: My Heading Text Example =======
level	Integer	*(Optional)* Valid when type == "heading". Contains the depth level of the heading. Starts with the value 1.

mode	String/Enumerated: ● "default" ● "palette" ● "rgb"	*(Optional)* Valid for types "fontColor" and "bkgColor": ● "default" - A default color. The field "color" does not contain any information. ● "pallete" - A color indexed by the standard palette. The field "color" contains an integer with the palette index. ● "rgb" - A custom specific color defined as a Color String in the the field "color".
shortcut	String	*(Optional)* Valid when type == "smiley". Contains the text in the parsed text that generated this smiley.
style	String/Enumerated: ● "bold" ● "underline" ● "italic" ● "strikeOut"	*(Optional)* Valid when type == "fontStyle". Contains which font style the span should be presented with.
text	String	*(Optional)* Valid for types "text" and "url". The string text content of the tag.
url	String	*(Optional)* Valid for types "url" and "image". The URL address of the link or image.
width	Number	*(Optional)* Valid when type == "image". If present, it contains the width, in logical pixels, in which the image should be presented.

Timestamp table

The Timestamp table specifies a date and time. It is a Lua table with the following fields:

Field	Type	Description
year	Integer	The year component
month	Integer	The month component, with values in the range from 1 to 12.
day	Integer	The day component, with values in the range from 1 to 31.

hour	Integer	*(Optional)* The hour component, with values in the range from 0 to 23.
min	Integer	*(Optional)* The minute component, with values in the range from 0 to 59.
sec	Integer	*(Optional)* The seconds component, with values in the range from 0 to 59.
msec	Integer	*(Optional)* The milliseconds component, with values in the range from 0 to 999.

HD Virtual

Cada plug-in possui seu próprio HD Virtual

Principais características do HD Virtual

- Cada plugin possui um HD Virtual. Um plug-in **não** acessa o conteúdo do HD virtual de outro plug-in.
- Os plug-ins **não** conseguem acessar arquivos que estão fora de seu "HD Virtual" diretamente. Isto significa que o plugin não enxerga o drive de armazenamento real do dispositivo em que está rodando. A única forma do plug-in conseguir acessar arquivos no HD do usuário é através das funlções dialogs.openFile e dialogs.saveFile
- Os arquivos do "HD virtual" são persistentes, isto é, ficam salvos mesmo quando o dispositivo é desligado.
- O conteúdo do "HD virtual" é perdido quando seu plug-in é desinstalado.
- O VHD (HD Virtual) do plugin é composto pela união lógica do conteúdo do arquivo .rpk + arquivos que foram criados durante a execução. O Firecast faz essa união lógica automaticamente de forma transparente para que o programador não precise se preocupar.
- Todos os arquivos originais do plug-in (aqueles que estão contidos na pasta projeto do plug-in) ficam no modo somente-leitura. Não é possível apagar e nem alterá-los.
- Ao instalar uma nova versão de plugin, os arquivos que foram criados durante execução do módulo são preservados.

Sobre o sistema de arquivos no HD Virtual

- O caractere separador de diretórios é "/"
- A raiz do HD virtual é o diretório "/"
- Os nomes de arquivos e pastas são case-sensitives, isto é, letras maiúsculas são diferentes das mesmas letras em sua forma minúscula. (Exemplo: "/arquivo.lua" e "/Arquivo.lua" são dois arquivos distintos)

Manipulando o HD Virtual:

Para manipular o HD Virtual, o SDK3 provê a biblioteca "VHD" .

Modelos de Fichas

Cada modelo de ficha do SDK3 é um documento Lua Form (extensão '.lfm') e toda vez que o usuário abre um personagem, o RRPG Firecast localiza o Lua Form adequado, cria uma nova instância e a exibe na tela.

Veja:

- Interfaces Visuais (Lua Forms)
- Lua Form e Instâncias

Criando um novo modelo de ficha

Para criar um novo modelo de ficha:

1. Crie um arquivo texto vazio de extensão '.lfm" dentro da pasta do projeto do plug-in.
2. Abra o arquivo em um editor de texto, copie e cole o seguinte documento:

```
<?xml version="1.0" encoding="UTF-8"?>

<form formType="sheetTemplate" dataType="DATA_TYPE_AQUI"
      title="TITULO_AQUI" name="NOME_AQUI">

</form>
```

1. Preencha os campos "dataType", "title" e "name":
    a. ***dataType*** – Um texto que identifica qual é o tipo de conteúdo do seu modelo de ficha. Você **deve inventar** um Data Type único que possua ao menos cinco caracteres alfanuméricos, "_" ou "." e que não comece com dígitos. Use a criatividade! Exemplos: "br.com.rrpg.DnD5_S3", "br.com.rrpg.VampiroAMascara", "br.com.meusite.MeuModeloDeFicha", "meuplugin.FichaX", etc..

    b. ***title*** – O título do seu modelo de ficha. Este texto aparecerá na lista de fichas quando o usuário criar um personagem. Exemplos: "DnD 5th", "Vampiro a Máscara", "Meu modelo de ficha", etc..

    c. ***name*** – Nome interno do Lua Form. Deve conter apenas caracteres alfanuméricos ou "_" e não deve começar com dígitos. Exemplos: "frmDnD5", "frmVampiroAMascara", "frmMeuModeloDeFicha", etc.. Apesar de não ser exigida, a prática de colocar "frm" no início do name é recomendada.
1. **Pronto!** Você acaba de criar um modelo de ficha vazio.

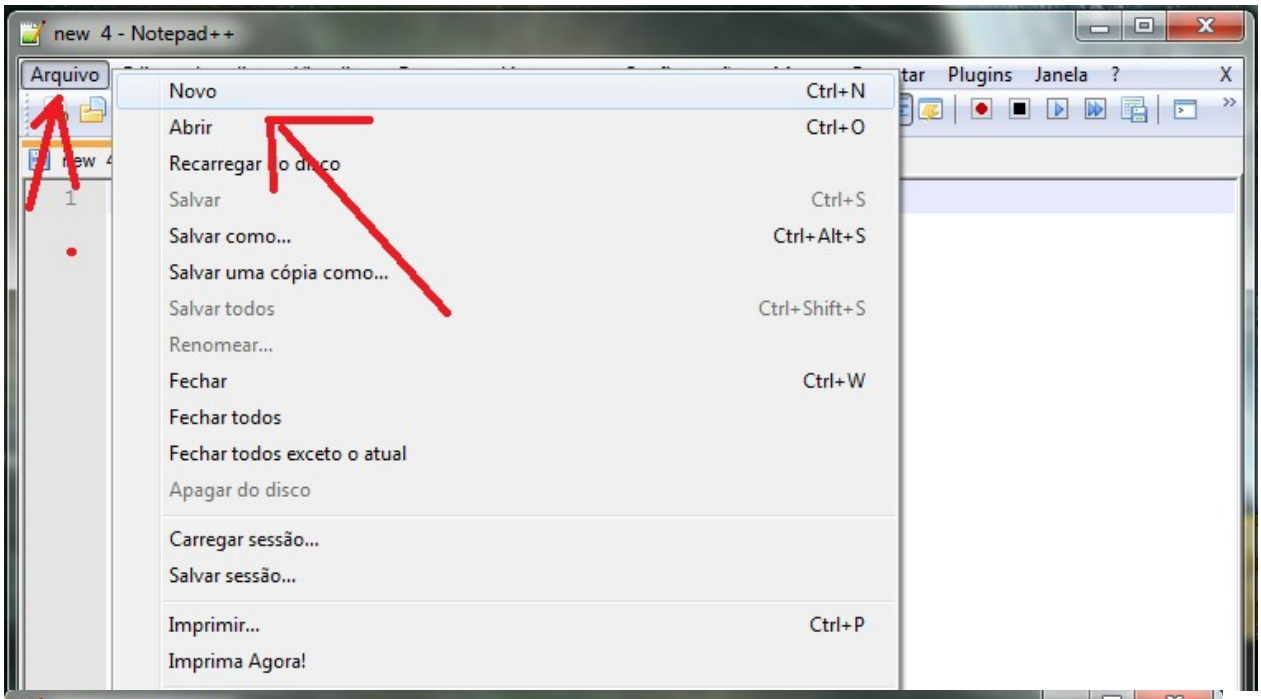
Veja agora:

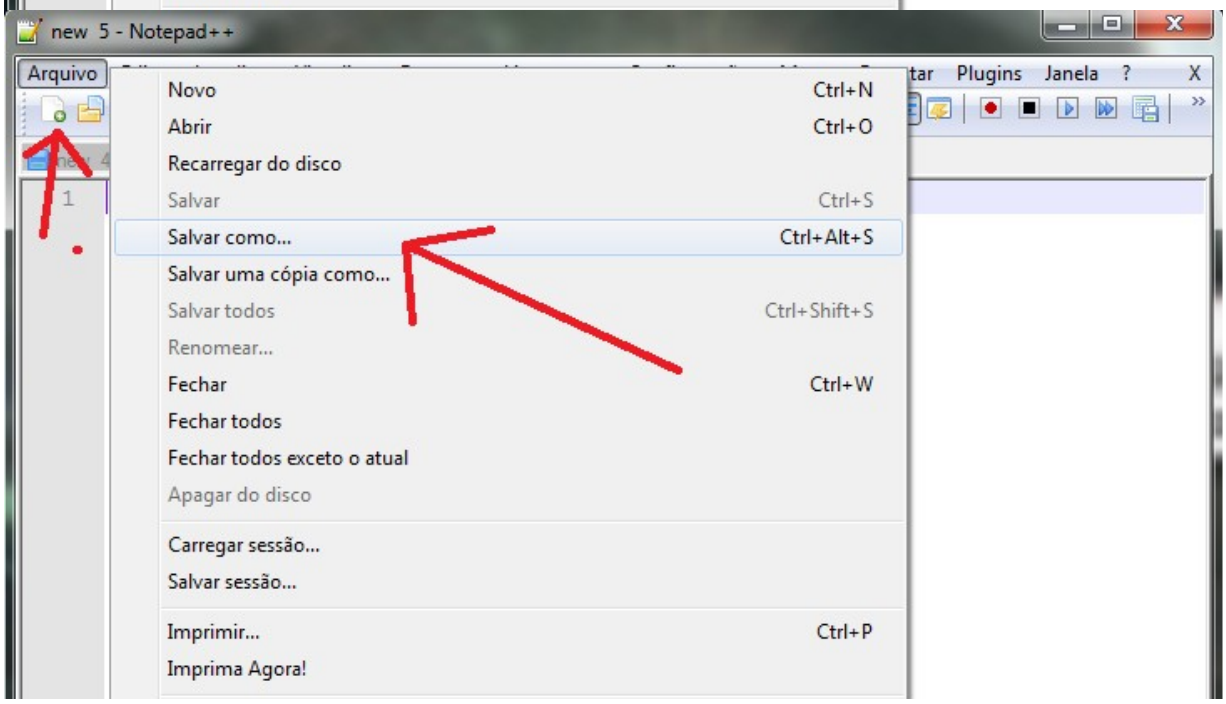
1. Interfaces Visuais (Lua Forms) para saber mais sobre o arquivo .lfm e completar seu plug-in.
2. Tutoriais para ter mais ideias para seu modelo de ficha =).

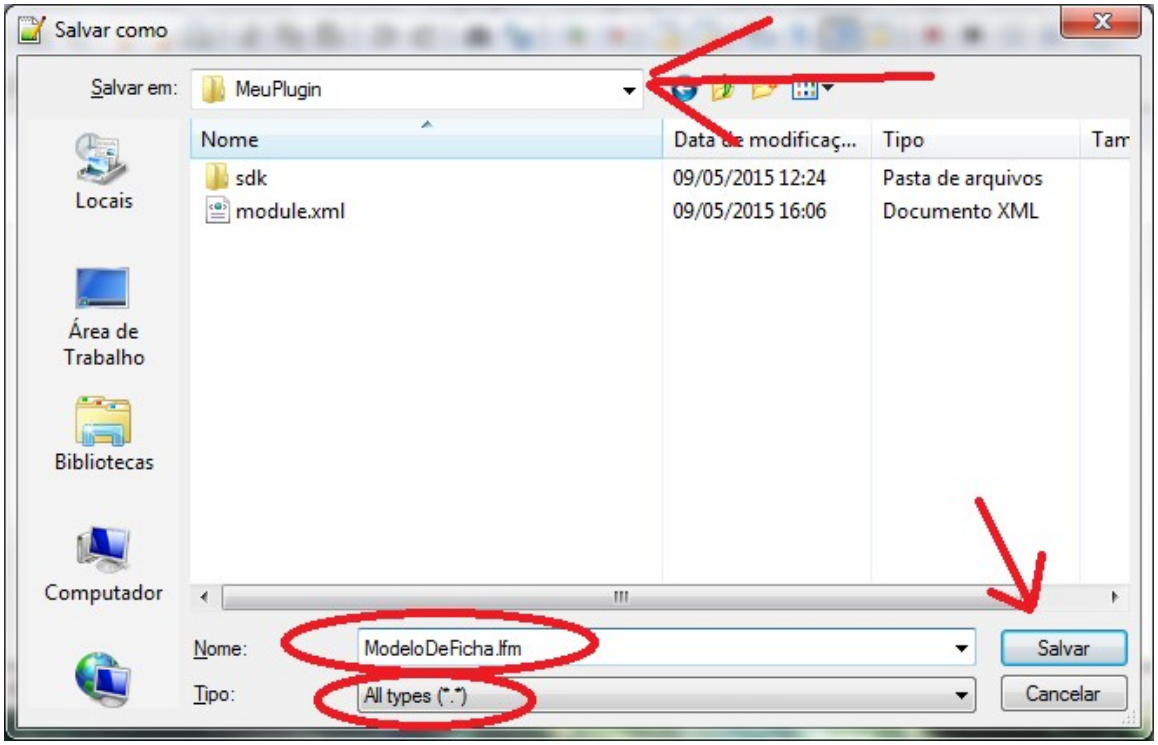
Criando um novo modelo de ficha - Exemplo passo a passo

O exemplo a seguir utiliza o editor de texto Notepad++.

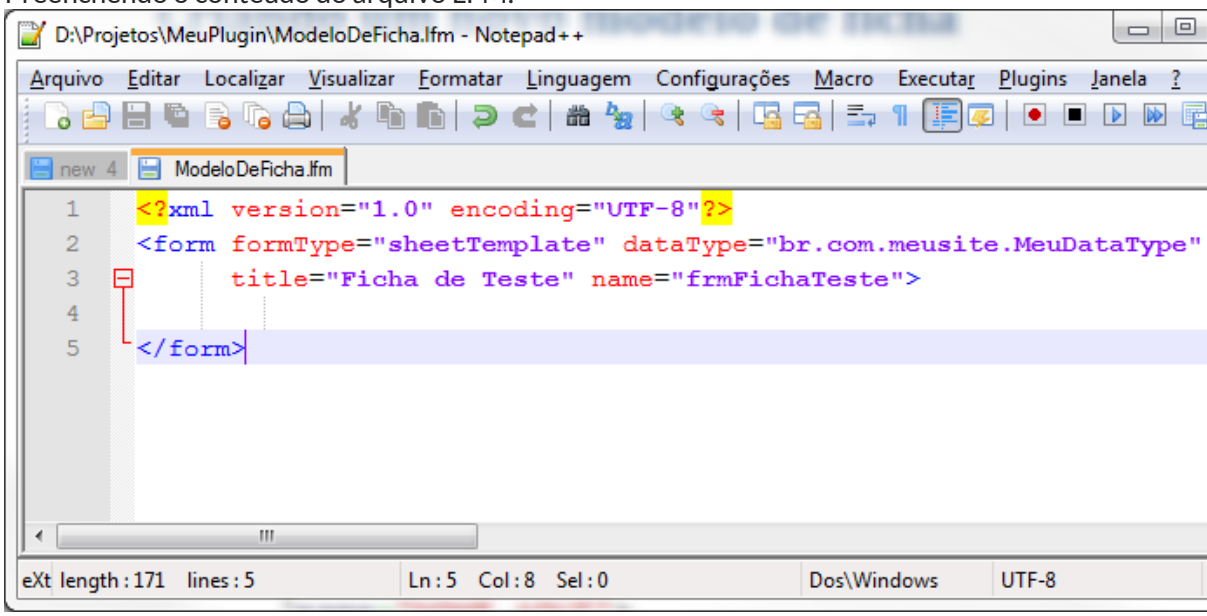
1. Criando o arquivo texto vazio de extensão '.lfm' dentro da pasta do projeto de plug-in:



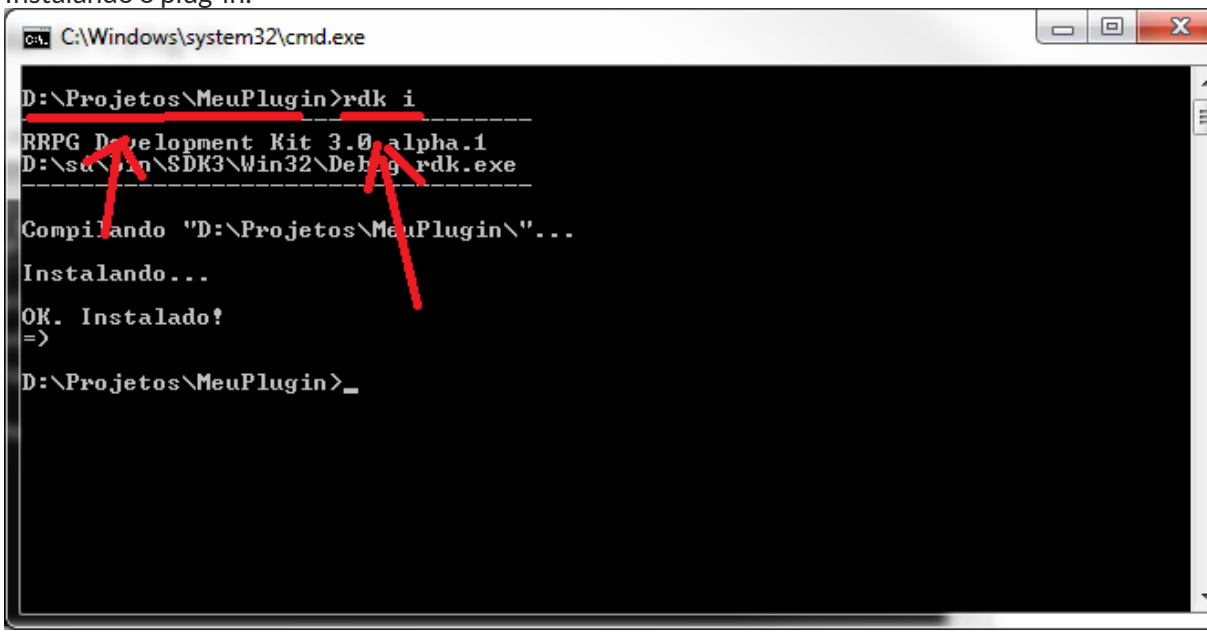




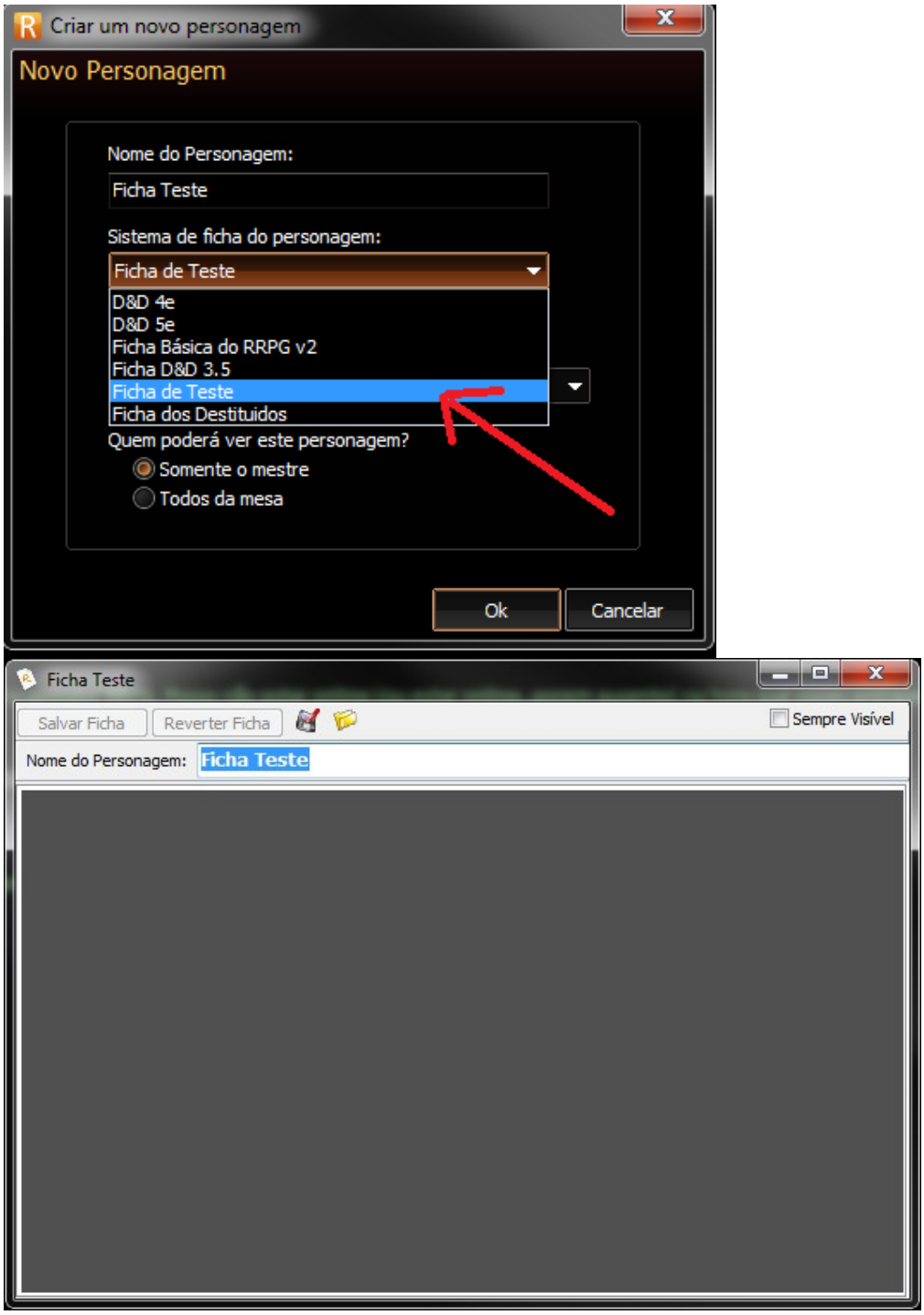
1. Preenchendo o conteúdo do arquivo LFM:



1. Instalando o plug-in:



1. Testando a ficha no RRPG:



NodeDatabase

Um NodeDatabase é um documento de fácil leitura, alteração e compartilhamento de dados usado para salvar fichas de personagens, configurações e outras informações.

Se chama assim pois é estruturado como uma árvore, isto é, existe um nó raiz e cada nó pode possuir outros nós de forma recursiva. Cada nó possui um nome, um nó pai (com exceção do nó raiz), nós filhos e atributos.
Todas as alterações feitas em um nó de um NodeDatabase são automaticamente salvas no documento.

Conteúdo:

- Objeto Nodo
- Biblioteca NDB

Sandboxes de Plug-ins

O SDK3 utiliza sandboxes para a execução de plug-ins. Suas principais características são:

- Um sandbox é um ambiente virtual que representa um plug-in em execução. São instanciados/criados quando o Firecast se inicia (e na instalação de plug-ins) e são destruídos quando o Firecast é fechado (ou na desinstalação de plug-ins).
- Cada plug-in roda em sua própria sandbox.
- Os sandboxes restringem o código dos plug-ins a fim de proteger o computador/dispositivo do usuário de acessos indevidos.
- Cada plug-in está associado com seu próprio "HD virtual".
    - Cada sandbox enxerga APENAS o "HD virtual" do plugin que está em execução.
    - **Não** é possível acessar arquivos que estão fora do "HD virtual"
    - Os arquivos do "HD virtual" são persistentes, isto é, ficam salvos mesmo quando o sandbox é destruído.
    - Um sandbox **não** consegue acessar o "HD virtual" de outro sandbox.
    - Um sandbox **não** consegue acessar o "HD virtual" de outro plugin.
    - O conteúdo do "HD virtual" é perdido quando o plugin é desinstalado.
    - Saiba mais lendo o tópico HD Virtual.
- Os objetos, tabelas e variáveis vivem isolados. Isto é, um sandbox não enxerga as variáveis e tabelas Lua de outro sandbox.
- Os objetos, tabelas e variáveis de cada sandbox **não** são persistentes. Isto é, estas informações são perdidas quando o sandbox é destruído.
- Durante o carregamento de uma sandbox, todos os arquivos de extensão "lua" do plug-in são automaticamente carregados.
    - Se existir o arquivo "/init.lua" na raiz do plugin, ele é carregado primeiro.
    - A ordem de carregamento dos demais arquivos respeita o uso da função "require".
    - Observação: A tabela global Lua é compartilhada com todos os arquivos Lua do plug-in. Logo, um arquivo Lua

enxerga as variáveis e tabelas de outro arquivo LUA (se o modificador “local” não for usado).
- Apenas arquivos “lua” que estão na pasta projeto do plug-in são carregados automaticamente.

Por segurança, o sandbox oferece **APENAS** as seguintes funções nativas LUA:

- Variáveis:
  - *_G*
  - *_VERSION*
- Funções:
  - *next*
  - *pairs*
  - *ipairs*
  - *pcall*
  - *select*
  - *tonumber*
  - *tostring*
  - *type*
  - *unpack*
  - *xpcall*
  - *collectgarbage*
  - *assert*
  - *error*
  - *load*
  - *rawget*
  - *rawset*
  - *rawequal*
  - *setmetatable*
  - *getmetatable*
  - *require*
- Biblioteca String
  - *string.byte*
  - *string.char*
  - *string.find*
  - *string.format*
  - *string.gmatch*
  - *string.gsub*
  - *string.len*
  - *string.lower*
  - *string.match*
  - *string.pack*
  - *string.packsize*
  - *string.rep*
  - *string.reverse*
  - *string.sub*
  - *string.upper*
  - *string.unpack*
- Biblioteca Table
  - *table.insert*
  - *table.maxn*
  - *table.remove*

        - *table.sort*
        - *table.concat*
        - *table.pack*
        - *table.unpack*
    - Biblioteca Math
        - *math.abs*
        - *math.acos*
        - *math.asin*
        - *math.atan*
        - *math.ceil*
        - *math.cos*
        - *math.deg*
        - *math.exp*
        - *math.floor*
        - *math.fmod*
        - *math.huge*
        - *math.log*
        - *math.log10*
        - *math.max*
        - *math.min*
        - *math.maxinteger*
        - *math.mininteger*
        - *math.modf*
        - *math.pi*
        - *math.rad*
        - *math.random*
        - *math.randomseed*
        - *math.sin*
        - *math.sqrt*
        - *math.tan*
        - *math.tointeger*
        - *math.type*
        - *math.ult*
    - Biblioteca debug
        - *debug.getmetatable*
        - *debug.setmetatable*
        - *debug.traceback*
    - Biblioteca OS
        - *os.clock*
        - *os.difftime*
        - *os.time*
        - *os.date*
    - Biblioteca Package
        - *package.loaded*
        - *package.config*
    - *Biblioteca coroutine*
        - *coroutine.create*
        - *coroutine.close*
        - *coroutine.resume*
        - *coroutine.running*
        - *coroutine.status*

        - *coroutine.wrap*
        - *coroutine.yield*
    - *Biblioteca utf8*
        - *utf8.char*
        - *utf8.charpattern*
        - *utf8.codes*
        - *utf8.codepoint*
        - *utf8.len*
        - *utf8.offset*

Veja também:

O SDK3 já provê algumas bibliotecas e funções LUA para você utilizar em seus plug-ins! Saiba mais em Bibliotecas LUA.

Interfaces Visuais (Lua Forms)

As interfaces visuais do SDK3, incluindo os modelos de fichas, são feitas usando Lua Forms (arquivos de extensões '.lfm').

Principais características do Lua Form

- Sua linguagem segue o padrão XML. Saiba mais sobre em Documentos XML.
- Ao compilar um projeto de plugin (Comando "rdk c") todos os documentos '.lfm' são validados e compilados em arquivos ".lua".
- Possui um conjunto próprio de tags e atributos. O plug-in não compila se alguma tag for utilizada incorretamente no arquivo '.lfm'. Ver Tags suportadas pelo Lua Form.
- A tag raiz (a mais externa) deve ser "form" ou "popupForm". Ver Tag FORM e Tag popupForm .
- A posição de cada controle é relativa à posição de sua tag pai/seu controle pai. Saiba mais em Posição e Tamanho dos Controles.

Exemplo 1 – Olá mundo

```
<?xml version="1.0" encoding="UTF-8"?>
<form formType="sheetTemplate" dataType="br.com.meusite.MeuDataType"
      title="Ficha de Teste" name="frmFichaTeste">

        <label text="Olá mundo"/>
```

```
</form>
```

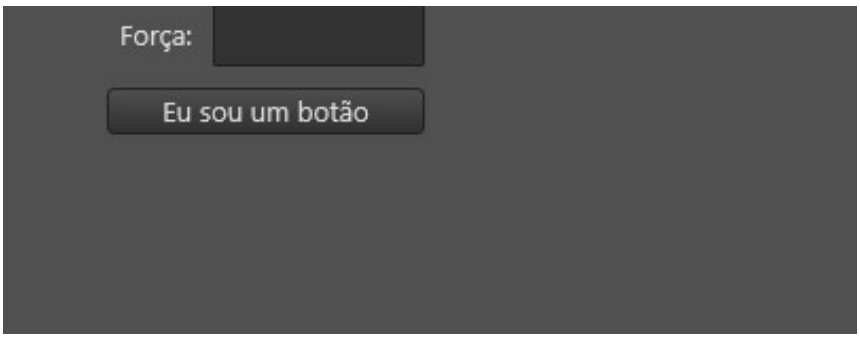


Exemplo 2 – Um layout, um label, um edit e um botão

```
<?xml version="1.0" encoding="UTF-8"?>
<form formType="sheetTemplate" dataType="br.com.meusite.MeuDataType"
          title="Ficha de Teste" name="frmFichaTeste">

        <layout height="30" width="200">
                <label text="Força:" align="client" horzTextAlign="trailing"/>
                <edit align="right" margins="{left=10}" width="100"/>
        </layout>

        <button text="Eu sou um botão" top="40" left="50" width="150"/>
</form>
```



Guias

Posição e Tamanho dos Controles

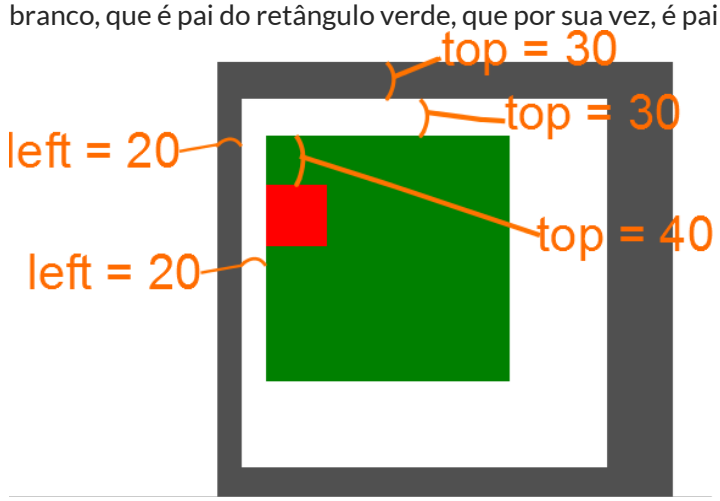
Toda tag/controle visual possui uma localização expressa com um par de coordenadas *left* e *top*, e duas dimensões, expressas como *width* e *height* (largura e altura), uma geometria que lembra um retângulo. A unidade destes atributos é o "ponto lógico" (Veja Ponto Lógico vs Pixel).
A posição de cada controle é relativa à posição de seus controles/tags pais.

Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <rectangle left="20" top="30" color="white" width="300" height="300">
                <rectangle  left="20" top="30" color="green" width="200"
height="200">
                        <rectangle left="0" top="40" color="red" width="50"/>
                </rectangle>
        </rectangle>
</form>
```

O exemplo contém 3 retângulos: Um vermelho, um verde e um branco. O form é pai do retângulo branco, que é pai do retângulo verde, que por sua vez, é pai do vermelho.



A posição top final e absoluta do quadrado vermelho é 100, pois é a soma das posições top relativas (40 + 30 + 30).

Ponto Lógico vs Pixel

Todas as dimensões ligadas à interface do Lua Form são medidas em pontos lógicos ao invés de pixels.

O que é um ponto lógico?

Ponto lógico é um "pixel virtual" que você deve usar quando estiver definindo as interfaces visuais para expressar tamanho ou posição de forma independente da **densidade da tela** dos dispositivos.

O que é densidade da tela?

Representa a quantidade de pixels dentro de uma área física da tela de um dispositivo. A densidade é comumente medida em ppi (points per inch - pontos por polegada).
Telas de alta densidade possuem mais pixels em uma área (uma polegada² ou centímetro², por exemplo) do que uma tela de menor densidade em uma mesma área. Quanto maior a densidade de pixels, mais definida e "bonita" é a tela.
Um monitor LED padrão de computador desktop possui por volta de 100ppi, um celular Motorolla Moto G possui 326ppi

A quanto equivale um ponto lógico?

O valor real de um ponto lógico varia de dispositivo para dispositivo, conforme sua densidade de pixels.

Densidade de Pixels	Equivalência	Exemplos de dispositivos
Interface de baixa densidade de pixels.	1 ponto lógico = 0,75 pixels	Celulares Androids de baixa densidade de pixels.
Interface de média densidade de pixels.	1 ponto lógico = 1 pixel	Computadores padrões, dispositivos iOS sem retina.
Interface de alta densidade de pixels.	1 ponto lógico = 1,5 pixels	Computadores configurados para textos ampliados, celulares Androids de alta resolução (Moto G ou Zenfone, por exemplo).
Interface de altíssima densidade de pixels.	1 ponto lógico = 2 pixels	Google Nexus 7, dispositivos iOS com retina.
Interface de super densidade de pixels.	1 ponto lógico = 3 pixels	Google Nexus 5, Galaxy S4.

O que isso tudo significa?

Que o SDK 3 lida automaticamente com as diferentes densidades de telas automaticamente para você.
Normalmente você pode definir sua interface sem preocupações como se 1 ponto lógico equivalesse a 1 pixel.

Arquivo de imagens e pontos lógicos.

Os arquivos de imagens (.png, jpg, bmp, etc..) possuem dimensões medidas em pixels. Ao usar estas imagens em uma interface medida em pontos lógicos, o SDK3 poderá automaticamente fazer conversões de tamanho para você.
Exemplo: O comportamento de uma imagem de 64x64 pixels usada em uma tag image de largura 64 e altura 64, será diferente para cada densidade de pixels:

Equivalência	Comportamento

1 ponto lógico = 0,75 pixels	A imagem será reduzida para 48x48 pixels antes de ser desenhada.
1 ponto lógico = 1 pixel	A imagem será desenhada como é, sem escala.
1 ponto lógico = 1,5 pixels	A imagem será ampliada para 96x96 pixels antes de ser desenhada.
1 ponto lógico = 2 pixels	A imagem será ampliada para 128x128 pixels antes de ser desenhada.
1 ponto lógico = 3 pixels	A imagem será ampliada para 192x192 pixels antes de ser desenhada.

Orientações ao usar código LUA em um Lua Form

Você pode usar códigos LUA (veja A linguagem de programação LUA) em seus Lua Forms de 2 maneiras:

- Manipulando eventos das tags - Saiba mais como em Tratando eventos do Lua Form.
- Usando a Tag script.

Mas para usar de forma adequada é importante você saber as características e comportamento do código LUA em um Lua Form.

1 - Instâncias

Um Lua Form pode ser instanciado várias vezes (veja Lua Form e Instâncias), isto é, é perfeitamente possível existir mais de 1 "cópia" de um Lua Form rodando ao mesmo tempo. Seu código deve estar apto a lidar com esta situação!

A variável "self" representa a instância do Lua Form em que o código está executado.

2 - Acessando as tags/controles via programação.

Cada instância do Lua Form possui sua própria "cópia viva" das tags e, se você quiser acessar uma tag/controle via programação, você precisará identificar de qual Lua Form é o controle que deseja ter acesso.

Para identificar um controle do próprio Lua Form, você deve usar "self.".
Em 99% dos casos que precisar acessar as tags via programação, você provavelmente usará "self.".

Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <label name="labMeuLabel" left="20" top="20" text="Meu Texto!" />
```

```
        <button text="Clique" left="20" top="50"
                onClick="self.labMeuLabel.text = 'Cliquei no Botão'"/>
</form>
```

Importante: Um erro ocorrerá se tentar acessar uma tag/controle sem usar "self." (ou outra forma de identificar o Lua Form).

3 - Declarando funções

Você pode usar a tag script para declarar funções, mas você deve usar o modificador "local" nestas funções para que não haja confusão entre as diversas instâncias do Lua Form.

Não usar o modificador "local" nas declarações de funções causa confusão entre as instâncias do form, já que se trataria de uma função global a todas as instâncias.

Exemplo de declaração:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <script>
                local function exibirMensagem()
                        local texto = "";

                        for i = 1, 5, 1 do
                                texto = texto .. "Linha " .. i .. "\n";
                        end;

                        showMessage(texto);
                end;
        </script>

        <button name="btnMeuBotao" text="Meu Botão" onClick="exibirMensagem()"/>
</form>
```

Você também pode declarar funções ligadas à instância para evitar os mesmos problemas:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <script>
                function self.exibirMensagem()
                        showMessage('Olá Mundo');
                end;
        </script>

        <button onClick="self.exibirMensagem();"/>
</form>
```

4 - Acessando o Nodo do NodeDatabase associado à instância do LuaForm

Cada instância de LuaForm pode estar associada a um objeto nodo de um NodeDatabase (ver NodeDatabase e Lua Form e NodeDatabase).

A variável "sheet" contém este objeto nodo associado e você pode usá-lo via programação.

Exemplo de uso:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <button onClick="sheet.campoDeForca = 8;"/>

</form>
```

Tratando eventos do Lua Form

Um evento é um acontecimento específico de alguma tag. É possível definir códigos LUA (veja A linguagem de programação LUA) que serão executados toda vez que determinado evento ocorrer em uma tag!

A maioria das tags/controles do Lua Form disparam eventos, mas cada uma possui seu próprio conjunto de eventos. Você deve consultar a documentação!

Manipulando eventos

Existem duas maneiras de manipular eventos do Lua Form:

- Associando uma única linha de código ao evento;
- Usando a tag event para associar várias linhas de códigos ao evento.

Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

Modo 1 - Associando uma única linha de código ao evento.

Esta é a forma mais simples de manipular eventos, porém não é possível associar um código de mais de 1 linha aos eventos por este modo.

Dado um evento de nome NOME_EVENTO de uma tag de nome NOME_TAG, você pode associar um código Lua assim:

```
<NOME_TAG NOME_EVENTO="<CODIGO_LUA_DE_UMA_LINHA>"/>
```

Modo 1, Exemplo 1 - OnClick de button

A tag button possui o evento "onClick".

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste"">

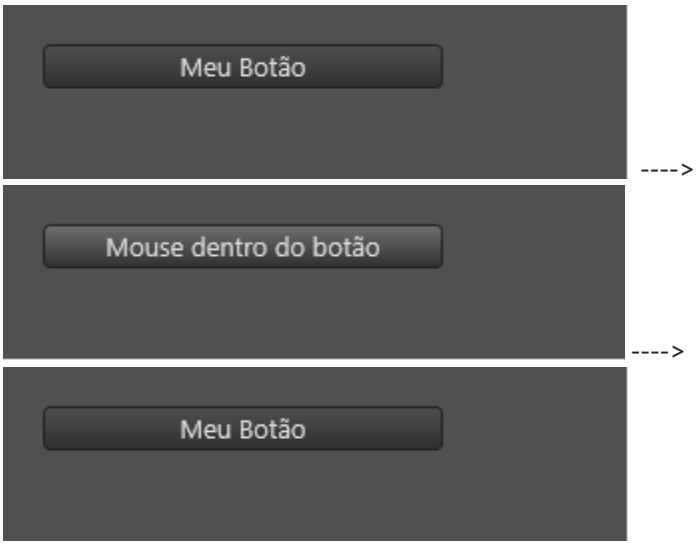
        <button text="Meu Botão" onClick="showMessage('Cliquei no botão');"/>
</form>
```



Modo 1, Exemplo 2 - OnMouseEnter e OnMouseLeave de button

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <button name="btnMeuBotao" text="Meu Botão" left="20" top="20" width="200"
                        onMouseEnter="self.btnMeuBotao.text = 'Mouse dentro do botão'"
                        onMouseLeave="self.btnMeuBotao.text = 'Meu Botão'"/>
</form>
```



Modo 1, Exemplo 3 - Definindo função e a chamando no evento OnClick de button

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<script>
		local function exibirMensagem()
			local texto = "";

			for i = 1, 5, 1 do
				texto = texto .. "Linha " .. i .. "\n";
			end;

			showMessage(texto);
		end;
	</script>

	<button name="btnMeuBotao" text="Meu Botão" onClick="exibirMensagem()"/>
</form>
```



Foi usada a Tag script neste exemplo.

Modo 2 - Usando a tag event para manipular eventos.

Saiba como usar a tag event e exemplos em: Tag event.

String de cores no Lua Form

As cores usadas nas tags do Lua Form são definidas por um string, e podem ter 2 formatos:

- Formato hexadecimal
- Formato literal

Formato hexadecimal

As cores no formato hexadecimal são uma mistura de RED, GREEN, BLUE e opcionalmente ALPHA no seguintes formatos:

Formato hexadecimal 1 — dois caracteres por canal de cor

#RRGGBB ou 0xRRGGBB

onde:

- RR - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção vermelha
- GG - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção verde
- BB - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção azul

Formato hexadecimal 2 — dois caracteres por canal de cor + opacidade

Também possível especificar a opacidade da cor (ALPHA), seguindo o padrão:

#RRGGBBAA ou 0xRRGGBBAA
onde:

- RR - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção vermelha
- GG - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção verde
- BB - código hexadecimal em 2 caracteres (valor entre 00 e FF) da porção azul

- AA - código hexadecimal em 2 caracteres (valor entre 00 e FF) do valor alpha/opacidade, onde 00 = 100% transparente e FF = 100% opaco.

Formato hexadecimal 3 — um caracter por canal de cor (também conhecido como shorthand hex color)

#RGB ou 0xRGB

onde:

- R - código hexadecimal em 1 caracter (valor entre 0 e F) da porção vermelha
- G - código hexadecimal em 1 caracter (valor entre 0 e F) da porção verde
- B - código hexadecimal em 1 caracter (valor entre 0 e F) da porção azul

Formato hexadecimal 4 — um caracter por canal de cor + Opacidade (também conhecido como shorthand hex color)

#RGBA ou 0xRGBA

onde:

- R - código hexadecimal em 1 caracter (valor entre 0 e F) da porção vermelha
- G - código hexadecimal em 1 caracter (valor entre 0 e F) da porção verde
- B - código hexadecimal em 1 caracter (valor entre 0 e F) da porção azul
- A - código hexadecimal em 1 caracter (valor entre 0 e F) do valor alpha/opacidade, onde 0 = 100% transparente e F = 100% opaco.

Exemplos:

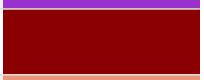
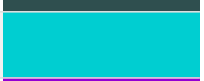
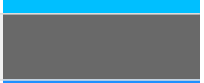
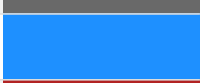
- #FF0000 - cor vermelha (Formato 1 — dois caracteres por canal de cor)
- #F00 - cor vemelha (Formato 3 — um caracter por canal de cor)
- #00FF00 - cor verde (Formato 1 — dois caracteres por canal de cor)
- #0F0 - cor verde (Formato 3 — um caracter por canal de cor)
- #0000FF - cor azul (Formato 1 — dois caracteres por canal de cor)
- #00F - cor azul (Formato 3 — um caracter por canal de cor)
- #FF00007F - cor vermelha 50% transparente (Formato 2 — dois caracteres por canal de cor + opacidade)
- #F007 - cor vermelha 47% transparente (Formato 4 — um caracter por canal de cor + opacidade)

Formato literal

O SDK3 também provê algumas constantes que identificam facilmente algumas cores! São elas:

Constante	Hexadecimal equivalente	Cor
AliceBlue	#F0F8FF	
AntiqueWhite	#FAEBD7	
Aqua	#00FFFF	

Aquamarine	#7FFFD4	
Azure	#F0FFFF	
Beige	#F5F5DC	
Bisque	#FFE4C4	
Black	#000000	
BlanchedAlmond	#FFEBCD	
Blue	#0000FF	
BlueViolet	#8A2BE2	
Brown	#A52A2A	
BurlyWood	#DEB887	
CadetBlue	#5F9EA0	
Chartreuse	#7FFF00	
Chocolate	#D2691E	
Coral	#FF7F50	
CornflowerBlue	#6495ED	
Cornsilk	#FFF8DC	
Crimson	#DC143C	
Cyan	#00FFFF	
DarkBlue	#00008B	
DarkCyan	#008B8B	
DarkGoldenRod	#B8860B	
DarkGray	#A9A9A9	
DarkGreen	#006400	
DarkKhaki	#BDB76B	
DarkMagenta	#8B008B	

DarkOliveGreen	#556B2F	
DarkOrange	#FF8C00	
DarkOrchid	#9932CC	
DarkRed	#8B0000	
DarkSalmon	#E9967A	
DarkSeaGreen	#8FBC8F	
DarkSlateBlue	#483D8B	
DarkSlateGray	#2F4F4F	
DarkTurquoise	#00CED1	
DarkViolet	#9400D3	
DeepPink	#FF1493	
DeepSkyBlue	#00BFFF	
DimGray	#696969	
DodgerBlue	#1E90FF	
FireBrick	#B22222	
FloralWhite	#FFFAF0	
ForestGreen	#228B22	
Fuchsia	#FF00FF	
Gainsboro	#DCDCDC	
GhostWhite	#F8F8FF	
Gold	#FFD700	
GoldenRod	#DAA520	
Gray	#808080	
Green	#008000	
GreenYellow	#ADFF2F	
HoneyDew	#F0FFF0	

HotPink	#FF69B4	
IndianRed	#CD5C5C	
Indigo	#4B0082	
Ivory	#FFFFF0	
Khaki	#F0E68C	
Lavender	#E6E6FA	
LavenderBlush	#FFF0F5	
LawnGreen	#7CFC00	
LemonChiffon	#FFFACD	
LightBlue	#ADD8E6	
LightCoral	#F08080	
LightCyan	#E0FFFF	
LightGoldenRodYellow	#FAFAD2	
LightGray	#D3D3D3	
LightGreen	#90EE90	
LightPink	#FFB6C1	
LightSalmon	#FFA07A	
LightSeaGreen	#20B2AA	
LightSkyBlue	#87CEFA	
LightSlateGray	#778899	
LightSteelBlue	#B0C4DE	
LightYellow	#FFFFE0	
Lime	#00FF00	
LimeGreen	#32CD32	
Linen	#FAF0E6	
Magenta	#FF00FF	

Maroon	#800000	
MediumAquaMarine	#66CDAA	
MediumBlue	#0000CD	
MediumOrchid	#BA55D3	
MediumPurple	#9370DB	
MediumSeaGreen	#3CB371	
MediumSlateBlue	#7B68EE	
MediumSpringGreen	#00FA9A	
MediumTurquoise	#48D1CC	
MediumVioletRed	#C71585	
MidnightBlue	#191970	
MintCream	#F5FFFA	
MistyRose	#FFE4E1	
Moccasin	#FFE4B5	
NavajoWhite	#FFDEAD	
Navy	#000080	
OldLace	#FDF5E6	
Olive	#808000	
OliveDrab	#6B8E23	
Orange	#FFA500	
OrangeRed	#FF4500	
Orchid	#DA70D6	
PaleGoldenRod	#EEE8AA	
PaleGreen	#98FB98	
PaleTurquoise	#AFEEEE	
PaleVioletRed	#DB7093	

PapayaWhip	#FFEFD5	
PeachPuff	#FFDAB9	
Peru	#CD853F	
Pink	#FFC0CB	
Plum	#DDA0DD	
PowderBlue	#B0E0E6	
Purple	#800080	
RebeccaPurple	#663399	
Red	#FF0000	
RosyBrown	#BC8F8F	
RoyalBlue	#4169E1	
SaddleBrown	#8B4513	
Salmon	#FA8072	
SandyBrown	#F4A460	
SeaGreen	#2E8B57	
SeaShell	#FFF5EE	
Sienna	#A0522D	
Silver	#C0C0C0	
SkyBlue	#87CEEB	
SlateBlue	#6A5ACD	
SlateGray	#708090	
Snow	#FFFAFA	
SpringGreen	#00FF7F	
SteelBlue	#4682B4	
Tan	#D2B48C	
Teal	#008080	

Thistle	#D8BFD8	
Tomato	#FF6347	
Turquoise	#40E0D0	
Violet	#EE82EE	
Wheat	#F5DEB3	
White	#FFFFFF	
WhiteSmoke	#F5F5F5	
Yellow	#FFFF00	
YellowGreen	#9ACD32	
None	#00000000	
Transparent	#00000000	
Null	#00000000	

Lua Form e NodeDatabase

O Lua Form foi desenvolvido de forma a se conectar facilmente ao NodeDatabase para criar interfaces que salvam dados.
Para integrar um lua form ao Node Database é preciso duas coisas:

- Preencher o atributo "field" das tags que suportam NodeDatabase
- Definir um escopo de dados do Lua Form, isto é, informar em qual nodo de qual NodeDatabase o LuaForm salvará os dados.
  - Observação importante: O SDK3 faz isto automaticamente para você se for um modelo de ficha, se estiver usando a tag recordList ou se for uma janela acoplável.

Tags que suportam NodeDatabase

As tags/controles que se conectam ao NodeDatabase são:

- label
- edit
- checkBox
- comboBox
- colorComboBox
- image
- imageCheckBox
- textEditor
- recordList
- dataLink

- dataScopeBox
- progressBar

Escopo de dados

Toda tag/controle que suporta NodeDatabase precisa de um escopo de dados, isto é, um objeto nodo de um nodedatabase onde os dados serão salvos.
As tags form, popup e dataScopeBox possuem o método "controle:setNodeObject" que permitem definir o escopo de dados e, após definido um objeto nodo como escopo, todas as tags que estão dentro passarão a salvar/carregar dados neste obeto nodo.
Exemplo:

Suponha a seguinte interface:

```
<form
name="minhaTagForm">
        <edit
field="campoNome"/>
</form>
```

Se o seguinte trecho de código Lua executar:

```
minhaTagForm:setNodeObjec
t(NODO_A)
```

o edit passará a ler e salvar dados em NODO_A.campoNome...

Agora, se o seguinte trecho de código Lua executar:

```
minhaTagForm:setNodeObject(OUTRO_NODO
)
```

o edit agora passará a ler e salvar dados em OUTRO_NODO.campoNome...

E, por fim, se o seguinte trecho de código Lua executar:

```
minhaTagForm:setNodeObject(nil)
```

o edit agora ficará desabilitado, pois utilizamos o atributo "field" mas não há nodo escopo de dados associado.

Importante: O SDK3 define o escopo de dados automaticamente para os modelos de fichas, ao usar a tag recordList e para as janelas acopláveis:

- Quando o usuário abre uma ficha no RRPG, o SDK3 carrega o NodeDatabase do personagem e define seu objeto nodo raiz como escopo de dados do form.
- No caso da tag recordList, o SDK3 automaticamente define o escopo de dados de cada painel de item.
- Quando o usuário ativa um janela acoplável, o SDK 3 carrega o NodeDatabase do form (se ele tiver a propriedade dataType preenchida) e define o objeto nodo raiz como escopo de dados do form.

Preenchendo o atributo "field"

Nas tags que suportam NodeDatabase, o atributo "field" contém o nome do campo de um objeto nodo a qual o controle deve se conectar.

Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <edit field="nomeESobrenome"/>
        <edit field="caracteristicas"/>

        <label field="forca"/>
        <label field="destreza"/>
</form>
```

No exemplo acima, quando a interface estiver rodando, o primeiro edit irá se ligará ao campo "nomeESobrenome" do objeto nodo, isto é:

- Exibirá na interface o conteúdo do campo "nomeESobrenome" do objeto nodo escopo.
- Quando alguém alterar o valor do campo "nomeESobrenome" do objeto nodo escopo, o edit captará a mudança e automaticamente mostrará o novo valor na interface.
  - Desde que estejam acessando o mesmo nodedatabase, este comportamento acontece mesmo se a pessoa que alterar o valor estiver em outro computador.
- Se o usuário alterar o texto do edit, o novo valor será salvo no campo "nomeESobrenome" do objeto nodo escopo.

Idem para as demais tags que tem o atributo "field" definido.

Acessando/salvando dados que estão em um nodo filho.

É possível utilizar o caractere "." para informar que o campo na verdade está em um nodo filho do nodo escopo.
Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <edit field="atributos.modificadores.forca"/>

        <edit field="nodoFilho.nodoNeto.campo1"/>
        <edit field="nodoFilho.campo2"/>
</form>
```

No exemplo acima, o primeiro edit se ligará ao campo "forca" do nodo de nome "modificadores" que está dentro do nodo de nome "atributos" que, por sua vez, está dentro do nodo escopo.
Se o usuário alterar o conteúdo do edit e o não existir um nodo de nome "atributos" no nodo escopo, o SDK3 automaticamente criará este nodo antes de salvar os dados. Idem para o nodo "modificadores" dentro do nodo "atributos".

Variável sheet

Os códigos que estão dentro de um Lua Form tem acesso à uma variável chamada "sheet" que representa o objeto nodo escopo do form.
Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

    <edit field="campoDeForca"/>


    <button onClick="sheet.campoDeForca = 8;"/>

</form>
```

No exemplo acima, ao clicar no botão, a propriedade "campoDeForca" será alterado para "8" e a mudança será refletida no edit já que está ligado ao mesmo campo que sofreu alteração.
Veja também:

- Tratando eventos do Lua Form
- Orientações ao usar código LUA em um Lua Form

Exemplos

Exemplo 1 - Pequeno cadastro que informa se a pessoa é maior de idade

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

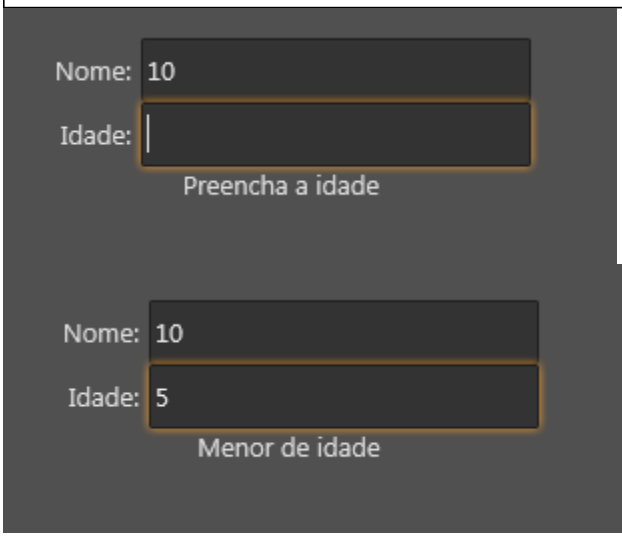
        <layout left="20" top="20" height="500" width="250">
                <layout align="top" height="32">
                        <label align="left" text="Nome: " width="50"
horzTextAlign="trailing"/>
                        <edit align="client" field="nome" margins="{left=5}"/>
                </layout>

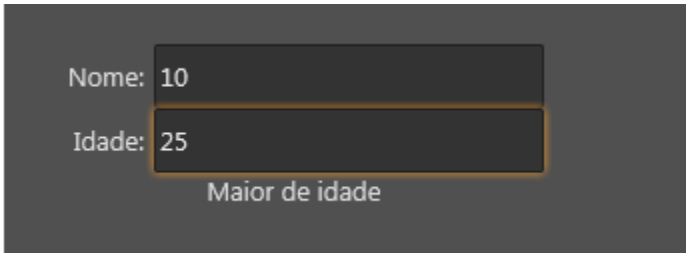
                <layout align="top" height="32">
                        <label align="left" text="Idade: " width="50"
horzTextAlign="trailing"/>
                        <edit align="client" field="idade" margins="{left=5}"/>
                </layout>

                <label name="labMaiorDeIdade" text="Preencha a idade"
align="top" horzTextAlign="center"/>
        </layout>

        <dataLink field="idade">
                <event name="onChange">
                        local idade = tonumber(sheet.idade);
```

```
                        if idade ~= nil then
                                if idade >= 18 then
                                        self.labMaiorDeIdade.text = "Maior de
idade";
                                else
                                        self.labMaiorDeIdade.text = "Menor de
idade";
                                end;
                        else
                                self.labMaiorDeIdade.text = "Preencha a idade";
                        end;
                </event>
        </dataLink>
</form>
```





Neste exemplo foram usados:

- Tag layout
- Tag edit
- Tag label
- Tag dataLink
- Tag event

Veja também:

- Orientações ao usar código LUA em um Lua Form

Lua Form e Instâncias

O documento Lua Form (arquivo '.lfm') contém apenas o molde/a receita de como a interface deve ser e se comportar.

Uma instância é uma interface concreta que foi criada a partir do molde, sendo assim, podem existir uma ou mais instâncias do mesmo LFM ao mesmo tempo. A instância é destruída quando o usuário não precisar mais da interface (por exemplo, ao fechar a janela do personagem), mas o molde nunca é destruído.

Estilos em Cascata

É possível aplicar estilos em cascata semelhante ao CSS do HTML no LuaForm.

Saiba mais em Tag style

Dividindo o arquivo Lua Form em documentos menores

Se seu arquivo .lfm cresceu e está muito grande, saiba que é possível organizá-lo em documentos XML menores!

Saiba mais lendo Tag import.

The Grid System

The Grid System is a tool designed to organize the tags/controls of your character sheet (and other Lua forms) in a way that adapts to different screen sizes. It is inspired by the grid system in the Bootstrap library, offering a flexible and powerful layout solution.

Links:

- Key concepts of the grid system: https://sdk3.firecast.app/grid/concepts/

Grid system key concepts

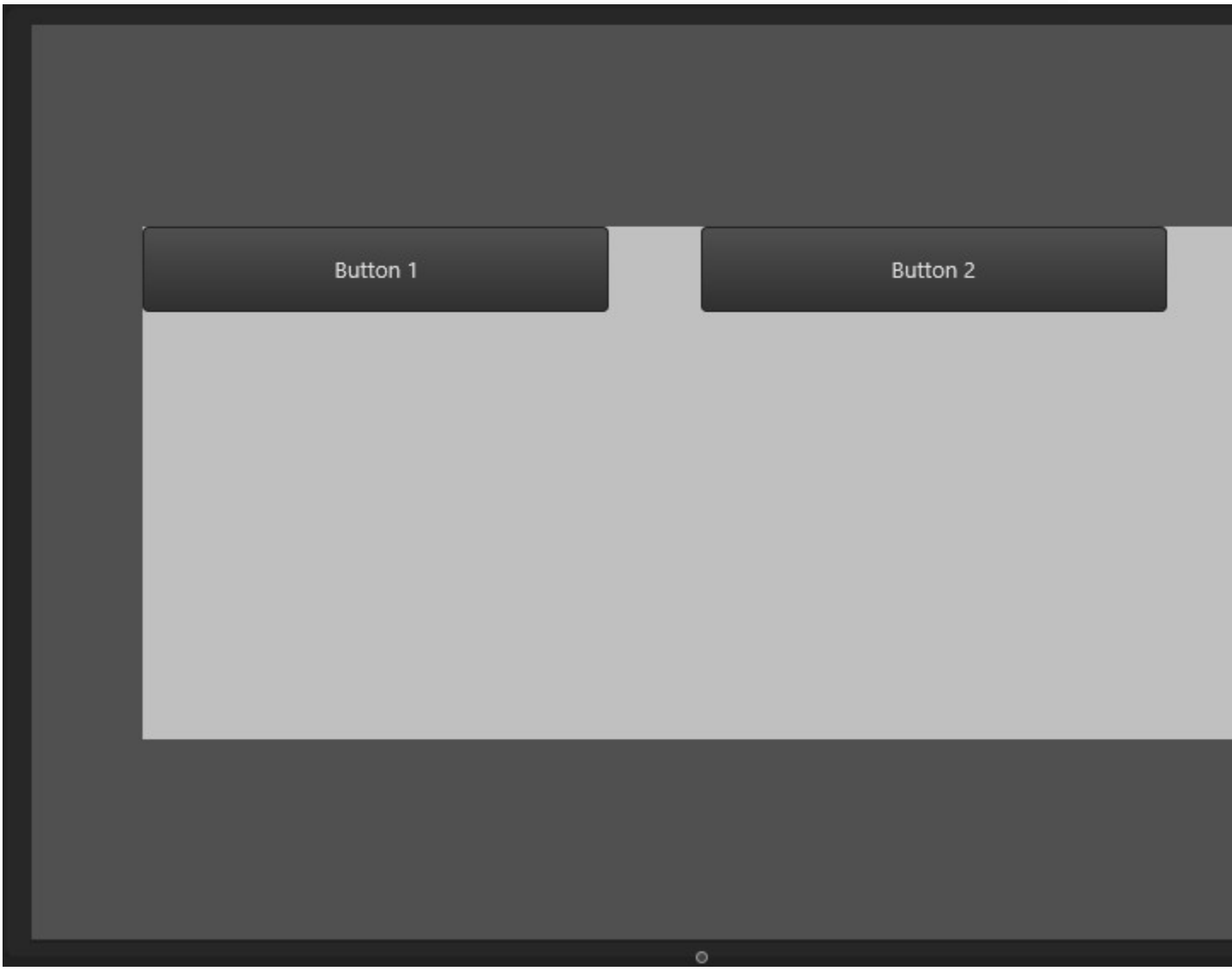
The Grid System is a tool designed to organize the tags/controls of your character sheet (and other Lua forms) in a way that adapts to different screen sizes. It is inspired by the grid system in the Bootstrap library, offering a flexible and powerful layout solution.

Key concept: The 12-column division

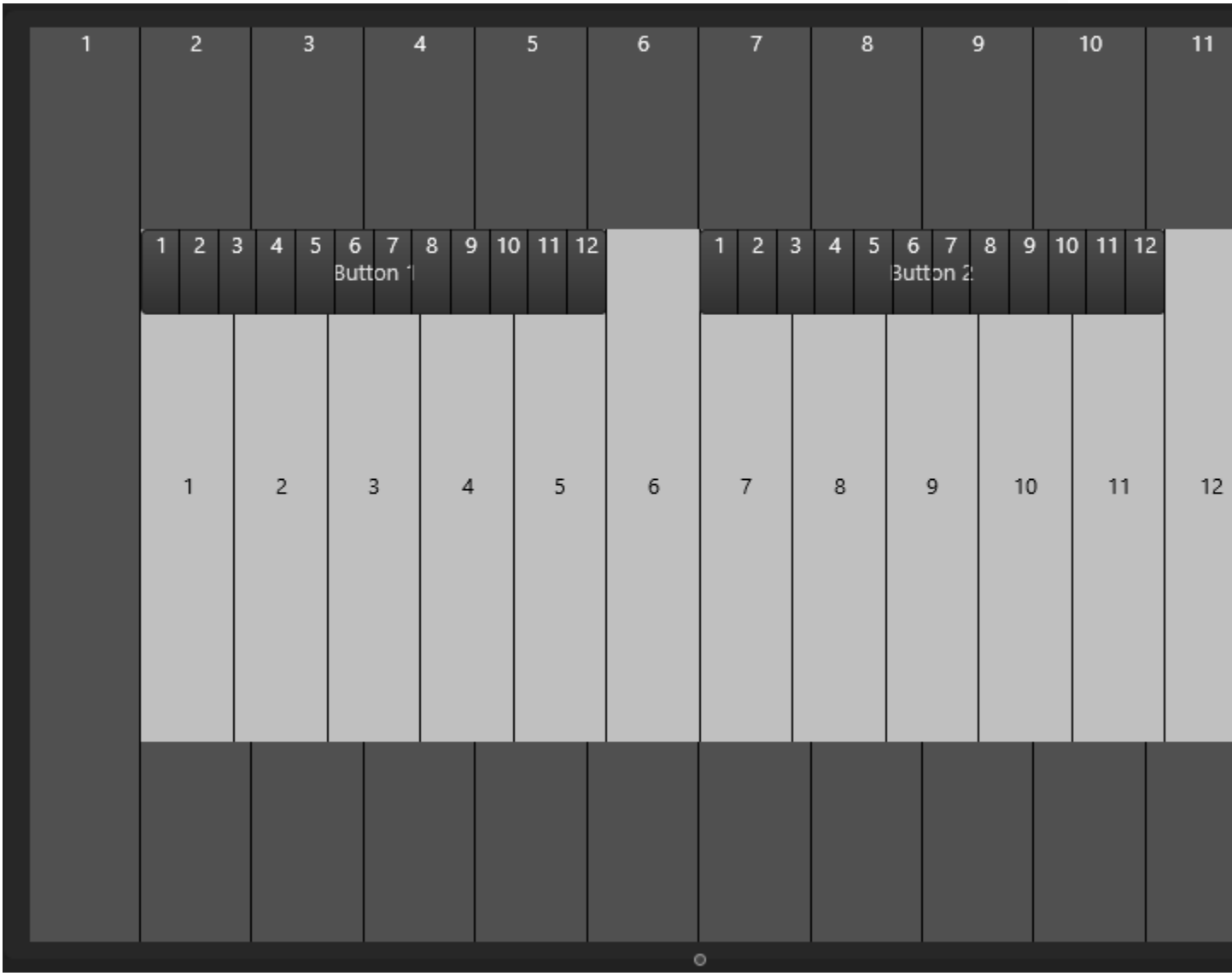
Forms, layouts, and all other controls are structured around a 12-column grid system, which works as follows:

- **12 virtual columns**: The content area of controls is divided into 12 equal virtual columns. These columns are used for alignment and sizing.
- **Automatic row breaks**: When the 12 virtual columns of a row are filled, the next control automatically starts on a new row.

Here's how it looks:



And here's how the grid system divides its content into 12 virtual columns:



Key concept: Roles in the grid system

Controls can assume one of 5 roles in the grid system:

- `none`: A control that does not align using the grid system. This is the default.
- `row`: A control that takes up an entire line in the 12-column division.
- `col`: A control whose width is defined by how many of the 12 columns it occupies.
- `block`: A control whose width is defined in pixels instead of 12-column spaces.
- `container`: A control that contains the entire interface layout while keeping the content width within a readable and manageable range.

Note

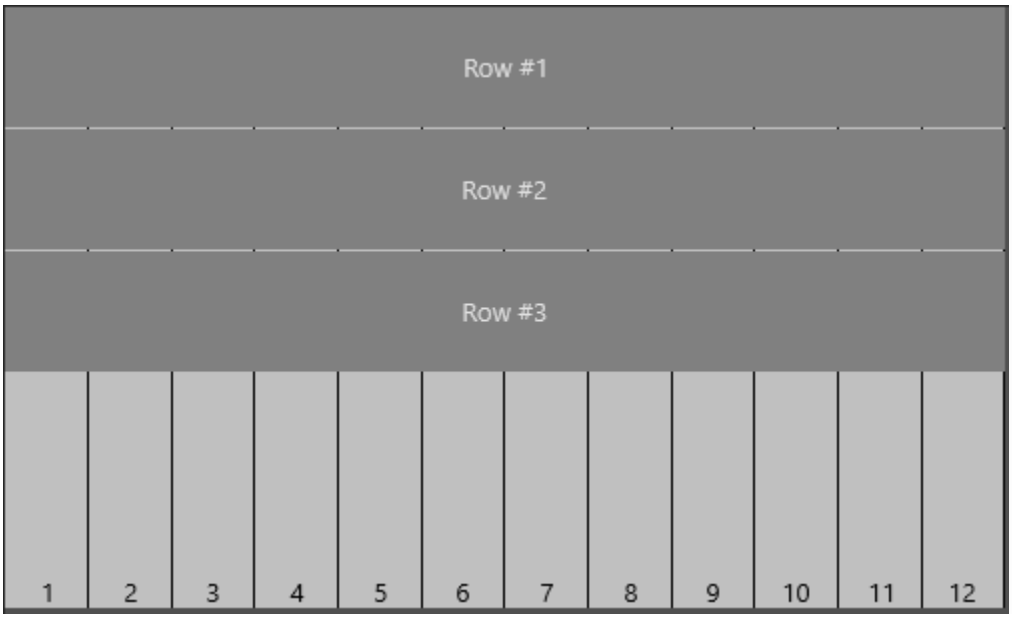
All controls can be aligned using the grid system by assigning them a role with the `g` or `g-role` attributes.

"row" role

Controls with the `row` role take up an entire line in the layout.

Example:

```
<layout name="r1" g="row" g-min-height="60"/>'
<layout name="r2" g="row" g-min-height="60"/>
<layout name="r3" g="row" g-min-height="60"/>
```



"col" role

Controls with the `col` role have their width defined by how many of the 12 columns they occupy. This is set with the `g-width` attribute.

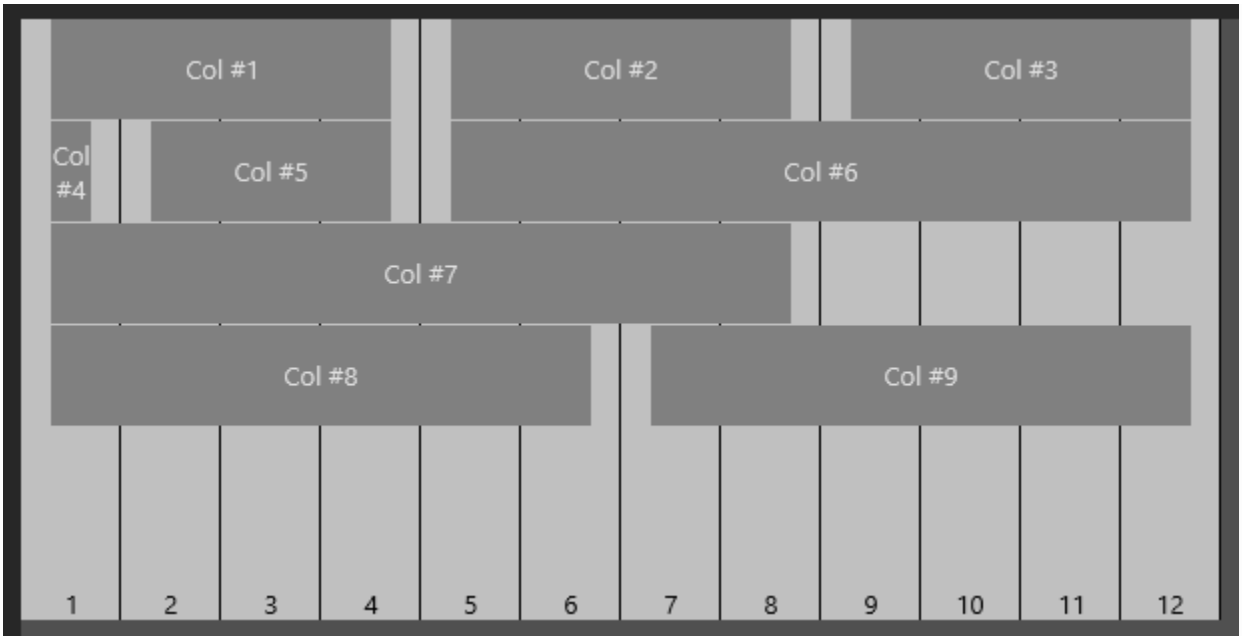
Example:

```
<!-- 1st line -->
<layout name="c1" g="col" g-width="3"/>
<layout name="c2" g="col" g-width="3"/>
<layout name="c3" g="col" g-width="3"/>

<!-- 2nd line -->
<layout name="c4" g="col" g-width="1"/>
<layout name="c5" g="col" g-width="3"/>
<layout name="c6" g="col" g-width="8"/>

<!-- 3rd line -->
<layout name="c7" g="col" g-width="8"/>

<!-- 4th line -->
<layout name="c8" g="col" g-width="6"/>
<layout name="c9" g="col" g-width="6"/>
```



Note

The horizontal spaces between the aligned controls are called gutter. The default gutter value is 30 and can be customized by using the `g-gutter` and/or `g-cnt-gutter` attributes

Note

In the 3rd line, a single `col` with `g-width=8` is used. Although there's a free space of 4 columns in that line, the next control does not fit into it as it occupies 6 columns. This behavior ensures that controls are aligned properly and do not overlap.

"block" role

Controls with the `block` role are placed in lines like `col` controls, but they don't use the 12-column spaces. Instead, they define their width in pixels, allowing precise control over their size in the layout.

Example:

```
<layout name="b1" g="block" width="100"/>
<layout name="b2" g="block" width="80"/>
<layout name="b3" g="block" width="450"/>
<layout name="b4" g="block" width="300"/>
```



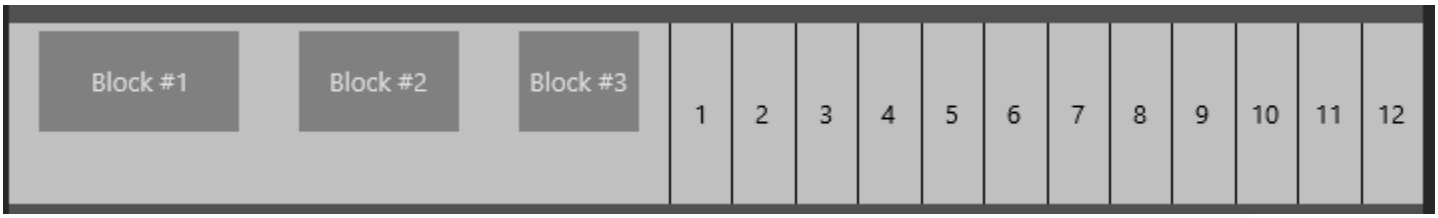
Blocks and the 12-column spaces

Controls with the `block` role don't use the available 12-column spaces. Instead, they shrink the size of the 12-column spaces available in the layout.

Example:

```
<layout name="b1" g="block" width="100"/>
<layout name="b2" g="block" width="80"/>
```

```
<layout name="b3" g="block" width="60"/>
```



If we add two `col` controls to the above layout:

```
<layout name="c1" g="col" g-width="8"/>
<layout name="c2" g="col" g-width="4"/>
```



"container" role

Controls with the `container` role act as a wrapper for all other controls in the interface. They ensure that the content width remains within a readable and manageable range by limiting the width of the entire layout.

Example:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmMyForm" >

    <rectangle g="row">
        <label text="Name: " g="col" g-width="1"/>
        <edit g="col" g-width="2"/>

        <label text="Age: " g="col" g-width="1"/>
        <edit g="col" g-width="2" height="30"/>
```

```
        <label text="Race: " g="col" g-width="1"/>
        <edit g="col" g-width="2"/>

        <label text="Field 4: " g="col" g-width="1"/>
        <edit g="col" g-width="2"/>
    </rectangle>


    <style>
        form { g-padding-top: 8; g-cnt-line-spacing: 4; theme: dark}
        rectangle { color: gray; g-padding-top: 16; g-padding-bottom: 16; }
        label { horzTextAlign: trailing; }
        edit, label { height: 30; }
    </style>
</form>
```

The code above produces the following layout on an extra-large screen (width=1920):



The labels have `g-width` set to 1 and yet are too large. On larger screens, the issue would get even worse. By setting the rectangle role to `container`, we will then have the following layout on the same screen:



Note

The maximum width for a container is 1378px.

Note

Using a `container` control is optional. It helps maintain a consistent layout and is particularly useful when you want to prevent content from becoming too wide.

Key concept: Screen size breakdown

Forms are given a screen size class based on their width in pixels, which helps define different layouts for different screen sizes.

Screen Size Class	Description	Form Width (px)
xs	Extra Small	< 576
sm	Small	>= 576
md	Medium	>= 768

lg	Large	>= 992
xl	Extra Large	>= 1200

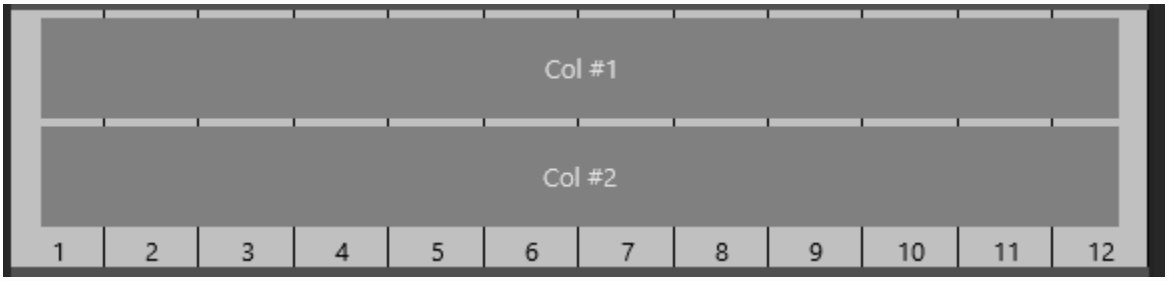
To specify different values for different screen sizes, append `"-xs"`, `"-sm"`, `"-md"`, `"-lg"`, or `"-xl"` to the grid attribute names. The grid system then selects the most appropriate suffix based on the screen size.

Example:

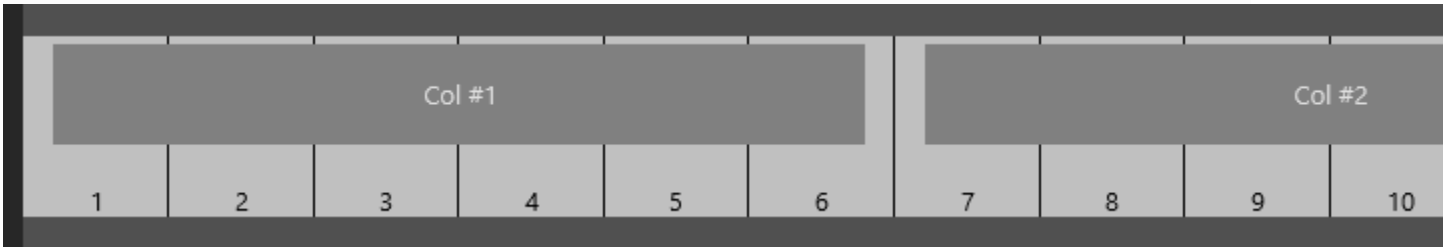
```
<layout g="col" g-width="12" g-width-md="6"/>
<layout g="col" g-width="12" g-width-md="6"/>
```

In this example, on screen sizes `xs` and `sm`, the width of the `col` elements will be 12, taking up the full width of the row. But on screen sizes `md` and **larger**, the width will be 6, allowing two `col` elements to be displayed side by side.

Running on `xs` and `sm` screen sizes:



Running on `md` and larger screen sizes:



- Tags attributes for the grid system: https://sdk3.firecast.app/grid/attributes/

Grid attributes

Note

All tags/controls have all attributes described here in addition to their own attributes.

Base attributes

- `g` or `g-role`: The role of the control in a grid system aligned layout.
- `g-auto-height`: Specifies whether the control's height should adjust automatically to fit its grid-aligned children controls vertically.

Attributes for controls with col role

- `g-width`: How many 12-columns count the the control occupies.
- `g-offset`: Layout offset
- `g-push`: Push offset
- `g-pull`: Pull offset

Attributes for controls with container role

- `g-horz-align`: Specifies horizontal alignment for the container.

Margins and padding

Margins are defined individually by side, in pixel count:

- `g-margin-left`
- `g-margin-top`
- `g-margin-right`
- `g-margin-bottom`

Padding is also defined individually by side, in pixel count:

- `g-padding-left`
- `g-padding-top`
- `g-padding-right`
- `g-padding-bottom`

Note

When a `g-margin-*`/`g-padding-*` value is not defined, the grid system will use the corresponding value defined in the control's `margins`/`padding` attribute.

Gutters

There are two ways to define gutter for controls, measured in pixels:

- `g-cnt-gutter`: Sets the gutter size for all content controls, i.e., for all controls that are inside.
- `g-gutter`: Sets the gutter size for the control only.

Learn more about the gutter here;

Vertical line spacing

- `g-cnt-line-spacing`: Define an amount of pixels to insert between lines inside the control.

Alignment attributes

For children alignment:

- `g-cnt-vert-align`: Specifies vertical alignment for the children of the control.
- `g-cnt-horz-align`: Specifies horizontal alignment for the children of the control.

For alignment of the control itself:

- `g-vert-align`: Specifies vertical alignment for the control itself.

Displacement attributes

Use the following attributes to displace the control by a specified amount of pixels:

- **`g-final-offset-x`**
- **`g-final-offset-y`**

Note

The `g-final-offset-*` attributes affect the control only and do not affect the positioning of following controls.

Breakline attributes

- **`g-break-line-before`**: Set to true to force a line break before the control.
- **`g-break-line-after`**: Set to true to force a line break after the control.

Sorting attributes

- **`g-order`**: An integer that defines the index of the control for ascending order (lower numbers appear first in the interface than greater numbers). Default: 0.
- **`g-group`**: An integer that defines which group the control belongs to. The groups are sorted in ascending order. Default: 0.

Controls are sorted first by their group (ascending `g-group` number), and then controls within each group are sorted by their `g-index` value.

Visibility attributes

- **`g-invisible-reserved`**: Set to true to force the grid system to reserve space and consider this control in the layout even if it is invisible (its `visible` attribute is `false`).

Dynamic size attributes based on text content

- **`g-dyn-width-txt`**: Specifies whether the Grid System should dynamically calculate the width for the control based on its text content.
- **`g-dyn-height-txt`**: Specifies whether the Grid System should dynamically calculate the height for the control based on its text content.

Tiling attributes

- `g-vert-tile`: Specifies whether the Grid System should calculate the control's height to fill the available vertical area.
- `g-vert-tile-weight`: A real number that determines the weight this control has when filling the available vertical area. It is used when multiple lines want to fill a content height. Lines with a higher average `g-vert-tile-weight` will "grow more" than lines with a lower average `g-vert-tile-weight`. Default: 1.0
- `g-horz-tile`: Specifies whether the Grid System should calculate the control's width to fill the available horizontal area. This attribute functions similarly to `g-vert-tile` but applies to horizontal spaces.

Size constraint attributes

For width:

- `g-min-width`: Specifies the minimum width the control should have, measured in pixels. The grid system will ensure that the control width is not less than this value.
- `g-max-width`: Specifies the maximum width the control should have, measured in pixels. The grid system will ensure that the control width is not more than this value.

For height:

- `g-min-height`: Specifies the minimum height the control should have, measured in pixels. The grid system will ensure that the control height is not less than this value.
- `g-max-height`: Specifies the maximum height the control should have, measured in pixels. The grid system will ensure that the control height is not more than this value.

Attributes used when mixing col and block roles

- `g-cnt-min-grid-width`: Specifies the minimum width, measured in pixels, that the entire 12 columns of the content should have.
- `g-cnt-min-grid-width-ft`: Specifies the minimum width, measured as a factor of the control width, that the entire 12 columns of the content should have.

Special built-in attributes

- `g-dyn-width-single-line`: If set to true, the grid system will calculate the width for the control so it can fit horizontally all of its children `block` controls into a single line, without the need for breaklines. It replaces the `g-on-get-width` event.

- `g-dyn-width-each-line`: If set to true, the grid system will calculate the width for the control so it can fit horizontally each of its children `block` controls into separate lines. It replaces the `g-on-get-width` event.

- `g-dyn-width-match-height`: If set to true, the grid system will calculate the width for the control so it matches the control height. It replaces the `g-on-aligned` event.

Tags suportadas pelo Lua Form

Tags de componentes

Tag	Descrição
form	Representa um formulário/uma janela genérica. A **tag raiz** do arquivo LFM **deve ser** “form” ou "popupForm". Pode haver mais de um form no arquivo LFM. Leia mais em: Tag form.
popupForm	Representa um formulário/uma janela cuja finalidade é ser exibida na interface como um popup. Leia mais em: Tag popupForm.
layout	Componente visual **invisível**. Excelente para agrupar ou organizar um conjunto de outros componentes visuais. Leia mais em: Tag layout
button	Um botão. Excelente para executar uma tarefa quando o usuário optar por ela! Leia mais em: Tag button
label	Um componente texto não editável. Usado apenas para mostrar informações. Leia mais em Tag label

image	Uma imagem. Leia mais em Tag image
edit	Uma caixa de texto. Componente que permite a edição de texto ou números. Leia mais em Tag edit
textEditor	Uma caixa de texto que permite várias linhas. Leia mais em Tag textEditor
tabControl	Um componente que permite adicionar abas à interface. Deve ser usado em conjunto com "tab" Leia mais em Tag tabControl
tab	Uma aba. Deve ser usado em conjunto com "tabControl" Leia mais em Tag tab
scrollBox	Controle que adiciona barras de rolagem, permitindo realizar "scroll" nos componentes que estão dentro da caixa. Em dispositivos que possuem touch screen, permite também o gesto de "pinça" para efetuar zoom do conteúdo. Leia mais em Tag scrollBox
rectangle	Controle para desenhar um retângulo de cor personalizada na interface. Leia mais em tag rectangle.
richEdit	Caixa de texto de múltiplas linhas com possibilidade de adicionar formatação, cores, alinhamentos, imagens, etc Leia mais em tag richEdit
horzLine	Uma linha horizontal. Leia mais em Tag horzLine
path	Uma forma geométrica/shape personalizada. Leia mais em Tag path.
checkBox	Uma caixa de marcação. Leia mais em Tag checkBox

imageCheckBox	Uma caixa de marcação onde é possível personalizar a imagem apresentada quando está selecionada e quando não está selecionada. Leia mais em Tag imageCheckBox
radioButton	Uma caixa de marcação que, quando usada em grupo, permite apenas uma esteja marcada por vez. Leia mais em Tag radioButton
comboBox	Uma caixa de seleção que permite o usuário selecionar um item entre vários. Leia mais em Tag comboBox
colorComboBox	Uma caixa de seleção que permite o usuário escolher uma cor qualquer. Leia mais em Tag colorComboBox
popup	Controle que permite exibir popups na interface. Leia mais em Tag popup
dataLink	Controle invisível que monitora mudanças de valores nos campos do NodeDatabase Leia mais em Tag dataLink
flowLayout	Controle que organiza controles na interface de forma dinâmica. Excelente para criar interfaces que se adaptam para diferentes tamanhos de tela. Leia mais em Tag flowLayout
flowPart	Usado em conjunto com “flowLayout”, define uma parte dinâmica do layout dinâmico. Leia mais em Tag flowPart
flowLineBreak	Usado em conjunto com “flowLayout”, realiza uma quebra de linha no layout dinâmico. Leia mais em Tag flowLineBreak
recordList	Um controle que permite adicionar uma lista de registros à interface. Leia mais em Tag recordList
dataScopeBox	Um controle que permite definir em qual objeto Nodo seus controles salvarão e lerão os dados (NodeDatabase). Leia mais em tag dataScopeBox

timer	Controle invisível que executa determinado código LUA de tempo em tempo. Leia mais em Tag timer
activityIndicator	Controle que exibe uma animação na interface para indicar ao usuário que alguma atividade está sendo feita, como um carregamento ou uma espera. Leia mais em Tag activityIndicator
progressBar	Controle que exibe uma barra de progresso/estado na interface. Leia mais em Tag progressBar

Tags especiais

Tag	Descrição
event	Tag que associa um código lua a um evento de algum componente/tag. Leia mais em Tag event
style	Tag que permite a entrada de código de estilo em cascata, muito similar ao CSS em HTML. *Estude um pouco esta tag! Irá ajudar **muito**!* Leia mais em Tag style
template	Tag avançada que permite a criação de novas tags para serem usadas no arquivo LFM! *Estude um pouco esta tag! Irá ajudar **muito**!* Leia mais em Tag template
import	Importa um arquivo XML. Leia mais em Tag import
group	Tag simples cujo único propósito é organizar o XML do arquivo Lua Form. Leia mais em Tag group
script	Define um script/bloco de código em Lua. Leia mais em Tag script

Características de todas as tags visuais

O SDK3 utiliza o conceito de orientação a objetos nos componentes visuais e todos os componentes visuais são objetos do tipo "controle" (herança, em orientação a objetos). Isto significa que todos os componentes e tags visuais de arquivo LFM (edit, label, button, layout, etc..) possuem as características, propriedades e métodos descritos nesta seção.

Propriedades e atributos de todas as tags visuais

Propriedade	Tipo	Valor Padrão	Descrição
visible	Boolean	true	Define se o controle está visível ou invisível. Um controle pode existir invisível e ser exibido quando alguma condição ou ocasião acontecer.
enabled	Boolean	true	Define se o controle está ativo ou não. Um controle inativo é exibido na interface, mas não é possível interagir com ele, como se tivesse bloqueado.
name	String	<string vazio>	Define um nome para o controle. O nome deve ser único, isto é, dentro de um **form**, não é possível existir 2 controles com o mesmo nome. Se não for definido um nome para o controle no arquivo LFM, um nome único será gerado para ele em tempo de compilação. Nomear controles é especialmente útil quando se quer trabalhar com códigos LUA.

align	Enumerado: ● “none” ● “top” ● “left” ● “bottom” ● “right” ● “client”	“none”	Define o alinhamento visual do controle na interface. Com esta propriedade é possível definir um comportamento padrão de posicionamento e tamanho para os controles. none – O controle não sofrerá nenhum tipo de posicionamento e redimensionamento automático. top – O controle será posicionado no topo de seu pai e a largura será calculada para preencher o espaço. Os valores de left, top e width são calculados automaticamente, mas é possível definir a altura (height). left – O controle será posicionado à esquerda de seu pai e a altura será calculada para preencher o espaço. Os valores de “left”, “top” e “height” são calculados automaticamente, mas é possível definir a largura (width). bottom - O controle será posicionado no rodapé de seu pai e a largura é calculada para preencher o espaço. Os valores de left, top e width são calculados automaticamente, mas é possível definir a altura (height). right – O controle será posicionado à direita de seu pai e a altura será calculada para preencher o espaço. Os valores de “left”, “top” e “height” são calculados automaticamente, mas é possível definir a largura (width). client – O controle será posicionado de forma que ocupe toda a área de seu pai. Os valores de left, top, width e height são automaticamente calculados.
left	Float	0.0	Define a posição X do controle. A posição é relativa a seu pai. Veja Posição e Tamanho dos Controles.
top	Float	0.0	Define a posição Y do controle. A posição é relativa a seu pai. Veja Posição e Tamanho dos Controles.

width	Float	Depende do controle	Define a largura do controle. Veja Posição e Tamanho dos Controles.
height	Float	Depende do controle	Define a altura do controle. Veja Posição e Tamanho dos Controles.
margins	Bounds (Objeto Lua)	{}	Define o tamanho das margens do controle, isto é, o tamanho da "borda invisível" que existe ao redor do controle que o separa do outros. Especialmente útil quando se usa a propriedade "align". O valor desta propriedade é uma tabela lua com os campos "left", "right", "top" e "bottom", onde a omissão de algum campo representa o valor 0.0. Exemplos de valores: ● {} - left, right, top e bottom 0.0 ● {left = 5} - left 5, demais valores 0 ● {left = 5, top = 5} – left 5, top 5, right 0, bottom 0 ● {left = 5, right = 5, top = 5, bottom = 5} – todos os valores 5 ●
padding	Bounds (Objeto Lua)	{}	Define o tamanho do preenchimento do controle, isto é, o tamanho da "borda invisível" que existe no interior do controle entre sua periferia e seu conteúdo. O valor desta propriedade é uma tabela lua com os campos "left", "right", "top" e "bottom", onde a omissão de algum campo representa o valor 0.0.
anchors	Conjunto de: ● left ● top ● right ● bottom	"left top"	Define como o controle é ancorado em seu pai. Quando o pai do controle é redimensionado, o controle mantém sua posição relativa baseado em sua âncora. Exemplos: ● "left" **presente**, "right" **ausente**: O controle manterá sempre a

			mesma posição X. ● “left” **ausente**, “right” **presente**: A posição X será calculada automaticamente para manter sempre a mesma distância entre sua borda direita e a borda direita de seu pai. ● “left” e “right” **presentes**: O controle manterá tanto a distância entre (1) sua borda esquerda com a borda esquerda de seu pai; (2) como sua borda direita com a borda direita de se pai. Para isso, a largura do componente será alterada conforme o pai é redimensionado. ● “left” e “right” **ausentes**: O controle não está ancorado e ocorrerá um efeito similar ao de centralização. O comportamento das âncoras “top” e “bottom” é o mesmo das âncoras “left” e “right”, porém agora no eixo Y.
hitTest	Boolean	Depende do controle	Define se o clique e movimento do mouse e o touch são capturados por este controle.
cursor	Enumerado: ● "default" ● "arrow" ● "handPoint" ● "hourGlass" ● "IBeam" ● "size" ● "sizeNESW" ● "sizeNS" ● "sizeNWSE" ● "sizeWE" ● "upArrow" ● "drag" ● "noDrop" ● "hSplit"	"default"	Define qual cursor de mouse deve ser apresentado quando o mouse estiver em em cima do controle. OBS: hitTest deve ser "true" para esta propriedade ter efeito. ● "default" - Cursor padrão, uma seta padrão. ● "arrow" - Uma seta padrão. ● "handPoint" - Uma mão com um dedo apontando para cima. Normalmente é usado para identificar um item selecionável/clicável ● "hourGlass" - Uma ampulheta para indicar um

	● "vSplit" ● "multiDrag" ● "sqlWait" ● "no" ● "appStart" ● "help" ● "cross"		processo em andamento. ● "IBeam" - Cursor que sinaliza a possibilidade de entrar com texto no controle. ● "size" - Indicação de possibilidade de redimensionamento. ● "sizeNESW" - Indicação de possibilidade de redimensionamento no sentido Nordeste-Sudoeste ● "sizeNS" - Indicação de possibilidade de redimensionamento no sentido Norte-Sul ● "sizeNWSE" - Indicação de possibilidade de redimensionamento no sentido Noroeste-Sudeste ● "sizeWE" - Indicação de possibilidade de redimensionamento no sentido Leste-Oeste ● "upArrow" - Uma seta apontando para cima. ● "drag" - Um cursor indicando que itens estão sendo arrastados. ● "noDrop" - Indicação de que não é possível arrastar para este alvo. ● "hSplit" - Indicação de splitter horizontal ● "vSplit" - Indicação de splitter vertical. ● "multiDrag" - Indicação de que vários itens estão sendo arrastados. ● "sqlWait" - Ampulheta indicando um processo de banco de dados em andamento. ● "no" - Uma indicação de negação, de que não é possível fazer algo. ● "appStart" - Combinação de "default" e "hourGlass", normalmente usado para indicar que um aplicativo

			está sendo aberto. ● "help" - Uma seta com um ponto de interrogação. ● "cross" - Um cursor em formato de cruz, usado para indicar um posicionamento preciso.
hint	String	""	Define um texto de ajuda que é exibido quando o usuário deixar o mouse por um certo tempo em cima do controle. Esta propriedade só tem efeito se o atributo "hitTest" do controle for true.
canFocus	Boolean	Depende do controle	Define se o controle pode receber o foco do teclado.
tabOrder	Integer	<automático>	Define um número que representa a ordem de tabulação do controle, isto é, quando o usuário apertar TAB no teclado para pular para o próximo controle de edição.
opacity	Float	1.0	Define a transparência do controle. É um número real que varia de 0.0 a 1.0, onde: ● 0.0 – totalmente transparente ● 0.25 ● 0.5 – parcialmente transparente ● 0.75 ● 1.0 – totalmente opaco. ●
rotationAngle	Float	0.0	Define o ângulo de rotação do controle. O ângulo é medido em graus.
rotationCenterX	Float	<centro X do controle>	Define o valor X da coordenada (X, Y) do pivô de rotação do controle. Usado em conjunto com “rotationAngle”
rotationCenterY	Float	<centro Y do controle>	Define o valor Y da coordenada (X, Y) do pivô de rotação do controle. Usado em conjunto com “rotationAngle”
scaleX	Float	1.0	Define o fator de escala que deve ser aplicado à largura do controle.
scaleY	Float	1.0	Define o fator de escala que deve ser aplicado à altura do controle.
scale	Float	1.0	Define o fator de escala que deve ser aplicado à largura e altura do controle.

parent	Um objeto	nil	Define ou retorna o objeto parent deste controle. Não há necessidade de utilizar este atributo em tags de arquivos ".lfm", pois o parent de cada tag já é setada automaticamente conforme seu aninhamento. Veja também: ● Posição e Tamanho dos Controles

Métodos de todas as tags visuais

É possível invocar métodos dos controles usando código LUA.

Método	Descrição
controle:destroy();	Destrói o controle, liberando a memória usada por ele. Após a chamada "destroy", não é mais possível utilizar o controle. Normalmente você não precisa se preocupar em destruir os controles, pois isto é feito automaticamente.
controle:setFocus();	Coloca o foco do teclado neste controle.
controle:beginUpdate();	Coloca o controle num estado de "em alteração". Neste estado, as alterações feitas nas propriedades não serão apresentadas na interface. É útil quando se pretende alterar várias propriedades ou várias vezes, pois o processo fica mais rápido, já que as mudanças não serão apresentadas na tela de imediato. Para cada chamada de "beginUpdate", deve haver uma chamada de "endUpdate".
controle:endUpdate();	Tira o controle do estado "em alteração", voltando a seu comportamento normal. Para cada chamada de "beginUpdate", deve haver uma chamada de "endUpdate".
controle:needRepaint();	Sinaliza o SDK3 que o controle precisa ser redesenhado na tela. Normalmente você não precisa se preocupar em chamar este método!

controle:getChildren();	Retorna um array (tabela lua com chaves de 1 a QUANTIDADE) contendo os controles que são filhos destes controles.
	Exemplo:
	
	`local filhos = self.panel1:getChildren();`
	`local i;`
	
	`for i = 1, #filhos, 1 do`
	`  filhos[i]:algumaCoisa();`
	`end;`

controle:getParent();	Retorna o controle pai deste controle ou **nil** se não houver.
controle:setParent(parent);	Define o controle pai deste controle. Parâmetros: ● parent – O objeto controle que será pai deste controle ou **nil.**
controle:isChildFocused();	Retorna um boolean (true ou false) indicando se este controle ou algum dos controles filhos está com o foco do teclado.
controle:isFocused();	Retorna um boolean (true ou false) indicando se este controle está com o foco do teclado.
controle:isMouseOver();	Retorna um boolean (true ou false) indicando se o mouse está por cima do controle (não necessariamenteclicado).
controle:isVisible();	Retorna um boolean (true ou false) indicando se o controle está visível.
controle:findControlByName(controlName);	Procura outro controle através do nome passado pelo parâmetro “controlName” em toda a hierarquia de controles onde este está. Retorna o controle ou **nil** caso não encontre.
controle:getClassName();	Retorna o nome da classe/tipo da tag deste controle. Exemplos: "edit", "label", "layout", etc..

Eventos de todas as tags visuais

Nome do evento	Descrição
onClick	Este evento é invocado quando ocorre um click com o botão esquerdo do mouse no controle, ou quando ocorre um “tap touch” no controle. Parâmetros: ● event - uma tabela Lua que contém os seguintes

	atributos: ○ x – posição X (relativa a este controle) indicando onde o click ocorreu. ○ y – posição Y (relativa a este controle) indicando onde o click ocorreu. ○ shiftKey - true se a tecla shift estiver pressionada ○ ctrlKey - true se a tecla control estiver pressionada ○ altKey - true se a tecla alt estiver pressionada ○ cmdKey - true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen.
onDblClick	Este evento é invocado quando ocorre um clique duplo com o mouse no controle. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X (relativa a este controle) indicando onde o click duplo ocorreu. ○ y – posição Y (relativa a este controle) indicando onde o click duplo ocorreu. ○ shiftKey - true se a tecla shift estiver pressionada ○ ctrlKey - true se a tecla control estiver pressionada ○ altKey - true se a tecla alt estiver pressionada ○ cmdKey - true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque

	em um display touchscreen.
onMouseDown	Este evento é invocado quando um botão do mouse é pressionado sobre o controle ou quando o dedo toca a tela (touchscreen). Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ button – indica qual botão do mouse foi pressionado, e pode conter um dos seguintes valores: “left”, “middle” ou “right”... Quando o dedo é pressionado (touchscreen), este valor é “left”. ○ x – posição X (relativa a este controle) indicando onde o mouse foi pressionado. ○ y – posição Y (relativa a este controle) indicando onde o mouse foi pressionado. ○ shiftKey - true se a tecla shift estiver pressionada também ○ ctrlKey - true se a tecla control estiver pressionada também ○ altKey - true se a tecla alt estiver pressionada também. ○ cmdKey - true se a tecla command (macOS) estiver pressionada também. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen. Observações: ○ Este evento só é disparado se a propriedade hitTest do controle for "true"
onMouseMove	Este evento é invocado quando o mouse se movimenta pelo controle ou quando o dedo (touchscreen) é arrastado. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X (relativa a este controle) indicando para onde o mouse moveu. ○ y – posição Y (relativa a este controle) indicando para onde o mouse moveu.

	○ shiftKey - true se a tecla shift estiver pressionada ○ ctrlKey - true se a tecla control estiver pressionada ○ altKey - true se a tecla alt estiver pressionada ○ cmdKey - true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen. Observações: ○ Este evento só é disparado se a propriedade hitTest do controle for "true"
onMouseUp	Este evento é invocado quando um botão do mouse é liberado sobre o controle ou quando o dedo solta da tela (touchscreen). Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ button – indica qual botão do mouse foi liberado, e pode conter um dos seguintes valores: “left”, “middle” ou “right”... Quando o dedo é levantado (touchscreen), este valor é “left”. ○ x – posição X (relativa a este controle) indicando onde o mouse foi liberado. ○ y – posição Y (relativa a este controle) indicando onde o mouse foi liberado. ○ shiftKey - true se a tecla shift estiver pressionada ○ ctrlKey - true se a tecla control estiver pressionada ○ altKey - true se a tecla alt estiver pressionada ○ cmdKey - true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla

	control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen. Observações: ○ Este evento só é disparado se a propriedade hitTest do controle for "true"
onMouseEnter	Este evento é invocado quando o mouse entra na área do controle. Observações: ○ Este evento só é disparado se a propriedade hitTest do controle for "true"
onMouseLeave	Este evento é invocado quando o mouse sai da área do controle. Observações: ○ Este evento só é disparado se a propriedade hitTest do controle for "true"'
onResize	Este evento é invocado quando o controle é redimensionado
onEnter	Este evento é invocado quando o controle recebe o foco do teclado
onExit	Este evento é invocado quando o controle perde o foco do teclado.
onKeyDown	Este evento é invocado quando uma tecla (teclado) é pressionada. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos ○ key - Cadeia de caracteres contendo a letra do teclado que foi pressionada ○ keyCode – Código numérico da tecla que foi pressionada (algumas teclas, como F7, por exemplo, não representam uma letra, daí a necessidade do keyCode) ○ shiftKey - true se a tecla shift estiver pressionada também ○ ctrlKey - true se a tecla control estiver pressionada também ○ altKey - true se a tecla alt estiver pressionada também. ○ cmdKey - true se a tecla command (macOS) estiver pressionada também. ○ ctrlOrCmdKey - No macOS: true

	se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. Observações: ● Se você tratou a tecla e nenhum outro componente deveria tentar tratá-la, altere o valor de event.key para "" (string vazia) e event.keyCode para 0. ● Se você alterar o valor de key e/ou keyCode, a mudança é incorporada. Caso queira suprimir/ignorar a tecla que acabou de ser pressionada, atribua event.key = "" e event.keyCode = 0. ● Se a combinação de teclas pressionada puder ser um atalho, então event.key virá com valor "" (string vazio) e event.keyCode com valor diferente de 0. ● Se a combinação de teclas pressionada puder ser tratada como um caractere exibível, event.key vira com uma string contendo apenas este caracter e event.keyCode virá com valor igual a 0.
onKeyUp	Este evento é invocado quando uma tecla (teclado) é liberada. Parâmetros: ● event - uma tabela Lua que contém as mesmas propriedades descritas no evento onKeyDown mas com informações sobre a tecla liberada.
onMenu	Este evento é invocado quando o usuário deseja exibir o Menu do controle. Pode ser que o usuário clicou com botão direito do mouse, pressionou a tecla menu ou segurou por um longo tempo o dedo na tela (LongTouch). Parâmetros: ○ x - Número contendo onde, em pixels, no eixo X da tela do usuário, o menu deve ser exibido. ○ y - Número contendo onde, em pixels, no eixo Y da tela do usuário, o menu deve ser exibido. ○ event - uma tabela Lua que contém os seguintes atributos: ■ x – Número contendo onde, em pixels, no eixo X da tela do usuário, o menu deve ser exibido. ■ y – Número contendo onde, em pixels, no eixo Y da tela do usuário, o menu deve ser

	exibido. ■ shiftKey – true se a tecla shift estiver pressionada ■ ctrlKey – true se a tecla control estiver pressionada ■ altKey – true se a tecla alt estiver pressionada ■ cmdKey – true se a tecla command (macOS) estiver pressionada ■ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada.
onStartDrag	Evento que é chamado quando o usuário deseja iniciar o processo de arrastar-e-soltar informações partindo deste controle e você deseja informar quais ações ou dados existem em um determinado ponto do controle. Parâmetros: ● drag - Objeto Drag o qual você deve preencher as informações para o RRPG acasalar o drag com o drop. ● x - posição X (relativa a este controle) indicando onde o drag começou. ● y - posição Y (relativa a este controle) indicando onde o drag começou. ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X (relativa a este controle) indicando onde o drag começou. ○ y – posição Y (relativa a este controle) indicando onde o drag começou. ○ shiftKey – true se a tecla shift estiver pressionada ○ ctrlKey – true se a tecla control estiver pressionada ○ altKey – true se a tecla alt estiver pressionada ○ cmdKey – true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a

	tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. Por favor, consulte Arrastando-e-Soltando Informações / Drag and Drop para conhecer melhor sobre o processo drag and drop. Observações: ● hitTest deve ser "true" para este evento poder ser chamado.
onStartDrop	Evento que é chamado quando alguém arrasta uma informação em cima do controle e você deseja informar quais ações ou dados existem em um determinado ponto do controle. Parâmetros: ● drop - Objeto Drop o qual você deve preencher as informações para o RRPG acasalar o drag com o drop. ● x - posição X (relativa a este controle) indicando onde o drop está tentando ser feito. ● y - posição Y (relativa a este controle) indicando onde o drop está tentando ser feito ● drag - Objeto Drag representando a informação que está sendo arrastada. Você não deve alterar as propriedades deste objeto. ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X (relativa a este controle) indicando onde o drop está tentando ser feito. ○ y – posição Y (relativa a este controle) indicando onde o drop está tentando ser feito ○ shiftKey – true se a tecla shift estiver pressionada ○ ctrlKey – true se a tecla control estiver pressionada ○ altKey – true se a tecla alt estiver pressionada ○ cmdKey – true se a tecla command (macOS) estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. Por favor, consulte Arrastando-e-Soltando Informações / Drag and Drop para conhecer melhor sobre o processo drag and drop. Observações: ● hitTest deve ser "true" para este evento poder ser chamado.

Características de todas as tags de textos

O SDK3 utiliza o conceito de orientação a objetos nos componentes visuais e alguns controles são controles de texto (Exemplos: **button**, **label**, **edit**).

Herança

As tags de textos possuem **todas** as características de um controle qualquer. Veja Características de todas as tags visuais.

Características

Além das características herdadas, todas as tags textos também possuem as seguintes características:

Propriedades e Atributos

Propriedade	Tipo	Valor Padrão	Descrição
text	String	<string vazio>	Define o texto que será exibido no controle.
fontColor	String contendo uma cor	Em temas escuros, uma cor clara. Em temas claros, uma cor escura.	Define a cor do texto. Exemplos: ● Black ● White ● #FDFDFD Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso.
fontFamily	String	<Varia de Plataforma para Plataforma>	Define a fonte que será usada nos textos. As fontes disponíveis variam de plataforma para plataforma (Android, iOS, Windows, etc..)
fontSize	Float	13.0	Define o tamanho da fonte.
fontStyle	Conjunto de: ● “bold” ● “italic” ● “underline” ● “strikeout”	“”	Define o estilo da fonte... Exemplos: ● “bold” ● “bold italic” ● “italic underline bold” ● “”
horzTextAlign	Enumerado: ● “leading” ● “center” ● “trailing”	“leading”	Define o alinhamento horizontal do texto, onde: ● “leading” – alinhado à esquerda ● “center”- centralizado ● “trailing” –

			alinhado à direita
vertTextAlign	Enumerado: ● “leading” ● “center” ● “trailing”	“center”	Define o alinhamento vertical do texto, onde: ● “leading” – alinhado ao topo. ● “center”- centralizado ● “trailing” – alinhado ao rodapé.
wordWrap	Boolean	false	True = Quebra de linha automática quando o tamanho do texto ultrapassar a largura do controle False = Limitar o texto em apenas uma linha, mesmo que ultrapasse a largura do controle.
textTrimming	Enumerado: ● “none” ● “character” ● “word”	<Varia de controle para controle>	Quando o tamanho do texto ultrapassa a largura do controle, o que acontece com o texto? ● none - Nenhum tratamento especial, o desenho do texto será truncado de forma “cara-crachá” ● character – o caractere mais perto da linha de corte será substituído pelo símbolo “...” ● word – a palavra que estiver mais perto da linha de corte será substituída pelo símbolo “...”

Características de tags shapes

O SDK3 utiliza o conceito de orientação a objetos nos componentes visuais e alguns controles são controles shape (Exemplos: **rectangle**, **horzLine**, **etc..**).

Herança

As tags shapes possuem **todas** as características de um controle qualquer. Veja Características de todas as tags visuais.

Características

Além das características herdadas, todas as tags shapes também possuem as seguintes características:

Propriedades e Atributos

Propriedade	Tipo	Valor Padrão	Descrição
color	String contendo uma cor	"silver"	Define a cor do conteúdo da forma. Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso.
strokeColor	String contendo uma cor	"black"	Define a cor das linhas da borda. Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso.
strokeSize	Float	0.0	Define a espessura das linhas da borda.
strokeCap	Enumerado: ● "flat" ● "round"	"flat"	Define o estilo gráfico usado ao desenhar o fim das linhas. "flat" - linhas com pontas retangulares "round" - linhas com pontas arredondadas.

strokeJoin	Enumerado: ● "miter" ● "round" ● "bevel"	"miter"	Define o estilo gráfico usado ao juntar segmentos de linha em uma forma. "miter" - As quinas/junções são "quadradas". "round" - As quinas/junções são arredondadas. "bevel" - As quintas/junções são ligadas de forma diagonal.
strokeDash	Enumerado: ● "solid" ● "dash" ● "dot" ● "dashDot" ● "dashDotDot"	"solid"	Define o estilo gráfico da linha. "solid" - A linha é sólida "dash" - A linha é tracejada "dot" - A linha é feita de pontos. "dashDot" - A linha é alternada entre traços e pontos. "dashDotDot" - A linha é alternada entre traço, ponto e ponto.

Tag activityIndicator

A tag/componente **activityIndicator** representa um controle que exibe uma animação na interface com o intuito de indicar ao usuário que alguma atividade está sendo feita, como um carregamento ou uma outra espera qualquer!

Herança

O **activityIndicator** possui todas as características de uma tag visual qualquer. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **activityIndicator** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição

enabled	Boolean	false	Este atributo controla a animação do indicador de atividade. Quando true, o indicador apresenta uma animação na interface.

Exemplos:

Exemplo 1 - Um indicador de atividade simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

        <activityIndicator left="20" top="20" enabled="true"/>
</form>
```

Tag button

A tag/componente **button** representa um botão na interface. Excelente para executar uma tarefa quando o usuário optar por ela!

Herança

O **button possui** todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **button** também possui as seguintes características:

Eventos

Nome do evento	Descrição
onClick	Este evento é invocado quando ocorre um click com o botão esquerdo do mouse no controle, ou quando ocorre um "tap touch" no controle.

Veja Tratando eventos do Lua Form

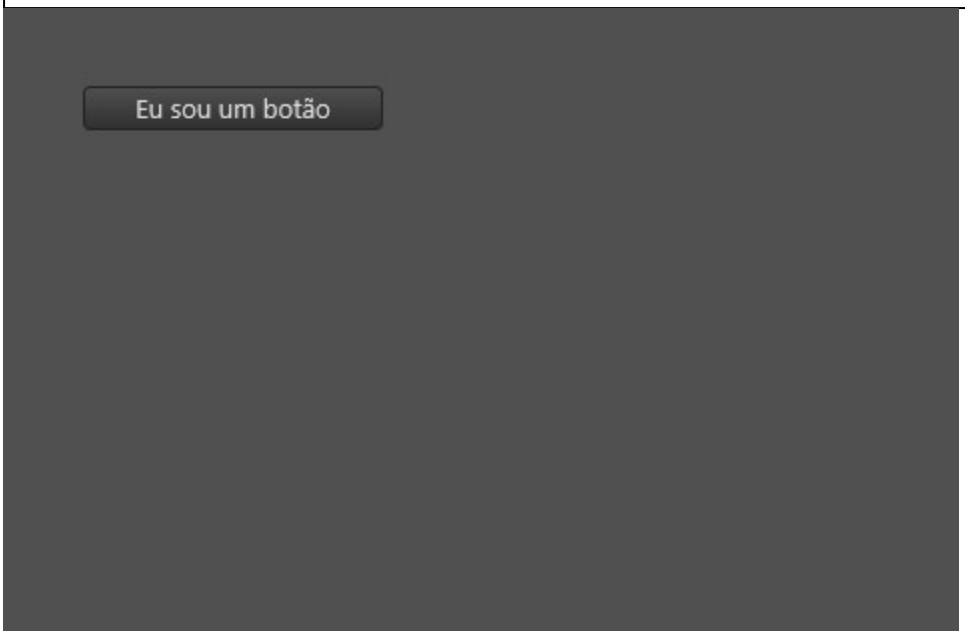
Exemplos:

Exemplo 1 - Um botão simples

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<form fame="frmFichaTeste">

       <button left="40" top="40" text="Eu sou um botão" width="150"/>
</form>
```

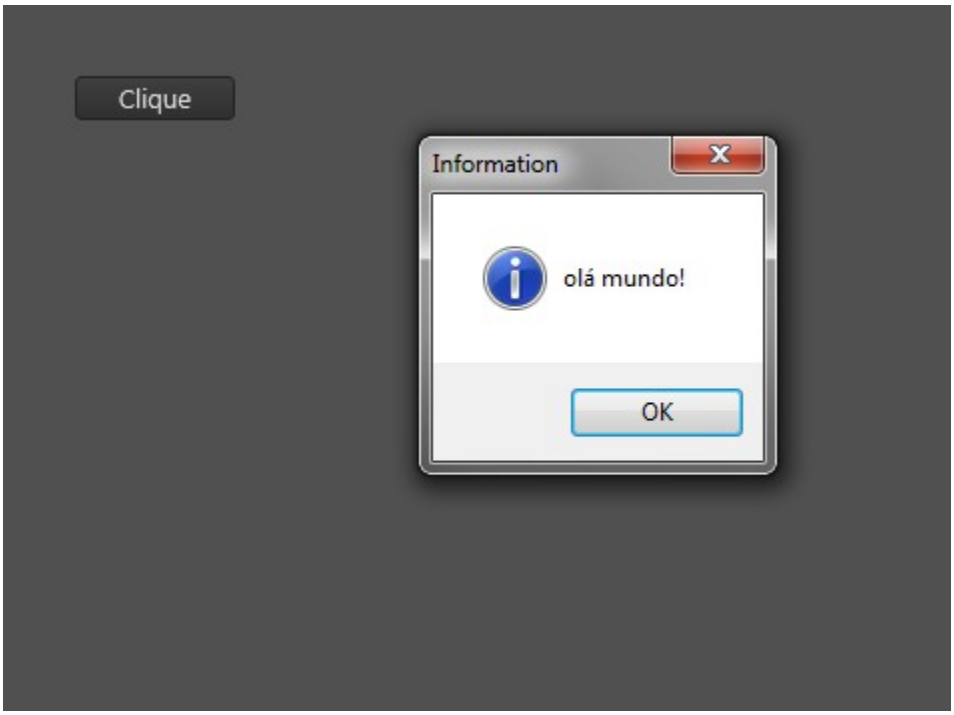


Exemplo 2 - Tratando o evento onClick (estilo 1)

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <button left="40" top="40" text="Clique" onClick="showMessage('olá
mundo!');"/>

</form>
```



Veja também [Tratando eventos do Lua Form.]

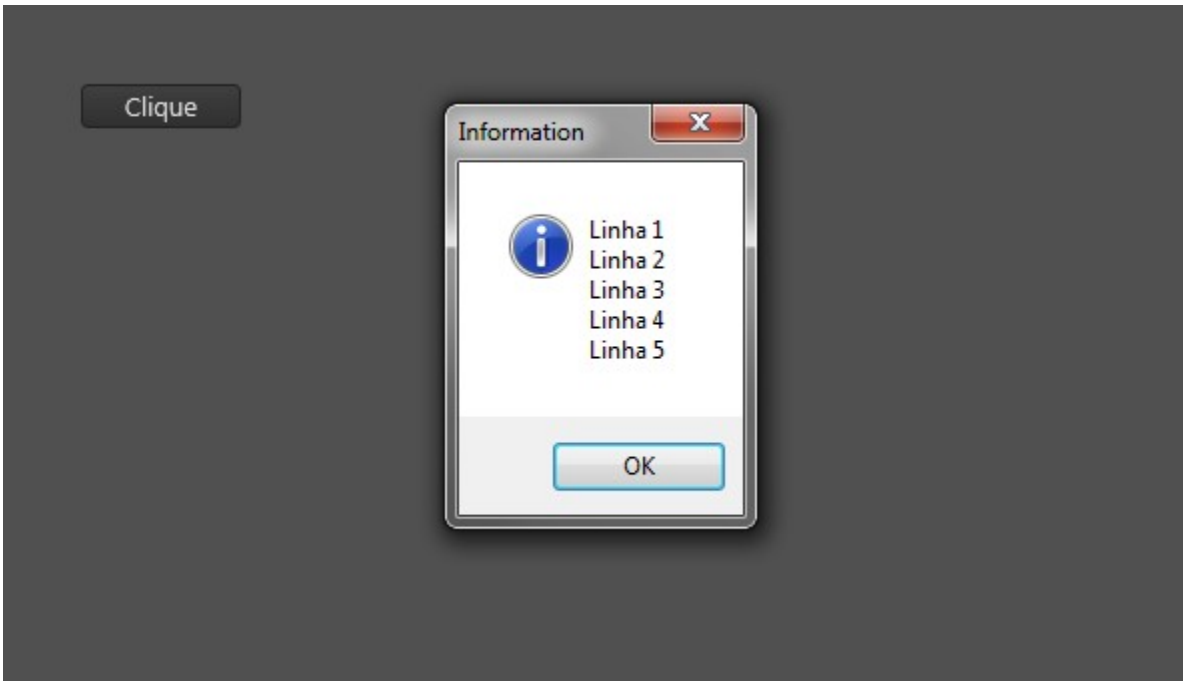
Exemplo 3 - Tratando o evento onClick (estilo 2)

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <button left="40" top="40" text="Clique">
                <event name="onClick">
                        local texto = "";

                        for i = 1, 5, 1 do
                                texto = texto .. "Linha " .. i .. "\n";
                        end;

                        showMessage(texto);
                </event>
        </button>
</form>
```



Tag col

The "col" tag/component represents a column in a grid-aligned interface. It is a shorthand for the following equivalent declarations:

```
<layout g="col"/>
<layout g-role="col"/>
```

Inheritage

The "col" tag inherits **all** the characteristics of the "layout" tag.

See:

- layout tag
- Characteristics of all visual tags

Tag colorComboBox

A tag/componente **colorCmboBox** representa uma caixa de seleção que permite o usuário selecionar uma cor qualquer.

Herança

O **colorComboBox** possui todas as características de uma tag visual. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **colorComboBox** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
color	String de Cor	"Null"	A cor escolhida pelo usuário. Exemplos: "Red", "White", "#005623"
useAlpha	Boolean	True	Se true, permite o usuário escolher uma cor com transparência.
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o colorComboBox passa a apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
frameRegion	String	<string vazio>	Considerando que o componente/tag pai possui um frame (ver Frames), ao definir o valor desta propriedade, a posição e tamanho deste controle é automaticamente calculada para se encaixar na Região do Frame de mesmo nome. Veja Frames e Regiões

Métodos

Método	Descrição
colorComboBox:dropDown()	"Abre" o colorComboBox. Exibe a caixa de seleção de cores na interface.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando a seleção do colorComboBox é alterada.

onUserChange	Este evento é disparado quando o valor do campo do objeto Nodo associado (ver atributo "field") for alterado porque o usuário alterou o conteúdo do colorComboBox. Importante: Este evento é disparado APENAS se o atributo "field" estiver preenchido corretamente e se a alteração de valor ocorrer neste colorComboBox mediante ação do usuário.

Veja Tratando eventos do Lua Form

Exemplos

Exemplo 1 - colorComboBox simples

Tag comboBox

A tag/componente **comboBox** representa uma caixa de seleção que permite o usuário selecionar um item entre vários..

Herança

O **comboBox** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **comboBox** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
text	String	<string vazio>	O texto escolhido pelo usuário e apresentado na interface pelo comboBox. Observação: Este campo retorna e aceita somente valores que estão na propriedade "items"

value	String	<string vazio>>	O valor associado ao item escolhido e apresentado na interface. Observação: Este campo retorna e aceita somente valores que estão na propriedade "values".
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o comboBox passa a apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase ●
frameRegion	String	<string vazio>	Considerando que o componente/tag pai possui um frame (ver Frames), ao definir o valor desta propriedade, a posição e tamanho deste controle é automaticamente calculada para se encaixar na Região do Frame de mesmo nome. Veja Frames e Regiões
transparent	Boolean	false	Quando True, as bordas e o fundo do comboBox não serão exibidas, mostrando apenas o texto conteúdo.
items	Arranjo de String	{}	Lista de itens do qual o usuário poderá escolher um. Exemplo: {'Lança', 'Espada', 'Adaga'}

values	Arranjo de String	{}	Lista de valores associados aos itens da propriedade "items". Cada posição deste arranjo é "casado" com a mesma posição do arranjo "items". Observação 1: Não é obrigatório o uso desta propriedade; Observação 2: Quando esta propriedade e o atributo "field" estiverem definidos, este são os valores que serão salvos no NodeDatabase. Exemplo: {'L', 'E', 'A'}

Métodos

Método	Descrição
comboBox:dropDown()	"Abre" o comboBox. Exibe a lista de seleção na interface.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando a seleção do comboBox é alterada.

Veja Tratando eventos do Lua Form

Exemplos

Exemplo 1 - comboBox simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <comboBox left="20" top="20" items="{'Lança', 'Espada', 'Adaga'}"/>
</form>
```





Exemplo 2 - comboBox usando a propriedade "values" para escolha do item padrão.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <comboBox left="20" top="20" items="{'Lança', 'Espada', 'Adaga'}"
                  values="{'L', 'E', 'A'}" value="E"/>
</form>
```



Neste exemplo, devido as propriedades "items" e "values", houve a associação dos seguintes valores e itens:

- L = Lança
- E = Espada
- A = Adaga

Tag container

The "container" tag/component represents a container in a grid-aligned interface. It is a shorthand for the following equivalent declarations:

```
<layout g="container"/>
<layout g-role="container"/>
```

Inheritage

The "container" tag inherits **all** the characteristics of the "layout" tag.

See:

- layout tag
- Characteristics of all visual tags

Tag checkBox

A tag/componente **checkBox** representa uma caixa de marcação na interface.

Herança

O **checkBox** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **checkBox** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
checked	Boolean	false	Indica se a caixa de marcação está marcada (true) ou desmarcada (false)
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o checkBox passa a apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando a marcação do checkBox sofre alteração.

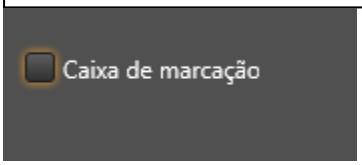
Veja Tratando eventos do Lua Form

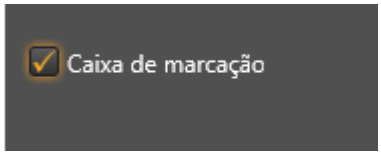
Exemplos:

Exemplo 1 - CheckBox simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <checkBox left="10" top="20" text="Caixa de marcação"/>
</form>
```





Tag dataLink

A tag **dataLink** representa um componente não visual (não é exibido na interface) que monitora mudanças em um NodeDatabase.
Veja também:

- NodeDatabase

- Lua Form e NodeDatabase

Características

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
name	String	<string vazio>	Define um nome para a componente. O nome deve ser único, isto é, dentro de um **form**, não é possível existir 2 controles com o mesmo nome. Se não for definido um nome para o controle no arquivo LFM, um nome único será gerado para ele em tempo de compilação. Nomear controles é especialmente útil quando se quer trabalhar com códigos LUA.
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o dataLink passa a monitorar mudanças no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
fields	Arranjo de String	{} // <arranjo vazio>	Idem à propriedade "field", porém permite monitorar mudanças em mais de um campo com um único dataLink. Observação: Não é possível utilizar as propriedades "fields" e "field" ao mesmo tempo, elas são mutualmente exclusivas.
defaultValue	String	<String Vazio>	Quando definido, se o campo identificado pelo atributo "field" estiver vazio, ele será inicializado com este valor informado.

defaultValues	Arranjo de String	{} // <arranjo vazio>	Idem à propriedade "defaultValue", porém permite definir valores padrões para cada um dos campos informado na propriedade "fields" Observação: Não é possível utilizar as propriedades "defaultValue" e "defaultValues" ao mesmo tempo pois elas são mutualmente exclusivas.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando ocorre uma mudança de valor no campo definido pela propriedade "field" ou quando ocorrer mudança de valor em algum dos campos definidos pela propriedade "fields" Parâmetros/Informações: ● field - string informando qual campo mudou ● oldValue - Antigo valor do campo ● newValue - Novo valor do campo Observações: ● Este evento pode ser invocado em qualquer uma destas ocasiões: ○ Os dados do nodedatabase da ficha/form acabaram de ser carregados ○ O usuário local alterou o valor de algum campo monitorado pelo dataLink ○ Algum outro usuário remoto alterou algum dos campos monitrados pelo dataLink.
onPersistedChange	Este evento é invocado quando ocorre uma mudança de valor no campo definido pela propriedade "field" ou quando ocorrer mudança de valor em algum dos campos definidos pela propriedade "fields" no **servidor**. Parâmetros/Informações: ● field - string informando qual campo mudou ● oldValue - Antigo valor conhecido do campo no servidor. ● newValue - Novo valor do campo no servidor. Observações: ● Devido a existência de transações no NodeDatabase e também devido à característica assíncrona do NodeDatabase, o valor dos atributos do NodeDatabases podem, temporariamente, não

	refletir ao valor salvo no servidor e nem ao valor que os outros usuários remotos tem acesso. ● Este evento é chamado quando alguma alteração é confirmada no lado do Servidor, persistida e disponibilizada para todos os outros usuários remotos. ● O valor persistido pode ser obtido através da função NDB.getPersistedAttributeValue()
onUserChange	Este evento é invocado quando ocorre uma mudança de valor no campo definido pela propriedade "field" ou quando ocorrer mudança de valor em algum dos campos definidos pela propriedade "fields" **porque** o **usuário local** alterou o conteúdo do do campo, seja editando ele visualmente ou através de código Lua. Parâmetros/Informações: ● field - string informando qual campo mudou ● oldValue - Antigo valor do campo ● newValue - Novo valor do campo. Observações: ● Lembre-se, este evento será disparado apenas quando o usuário local alterar o valor de algum dos campos monitorados pelo dataLink.
onChildAdded	Este evento é invocado quando um objeto nodo filho é criado dentro do field que está sendo monitorado (se o campo for um nodo). Parâmetros: ○ node - O objeto nodo filho que foi criado.
onChildRemoved	Este evento é invocado quando um objeto nodo filho é removido de dentro do field que está sendo monitorado (se o campo for um nodo). Parâmetros: ○ node - O objeto nodo filho que foi removido.

Veja Tratando eventos do Lua Form

Exemplos:

Exemplo 1 - Calculando um modificador de atributo de D&D

```xml
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<!-- Layout da interface -->
	<layout left="20" top="20" height="25" width="125">
		<label align="left" text="Força: " autoSize="true"/>
		<edit align="client" horzTextAlign="center" field="atributoForca"/>

		<!-- Modificador -->
		<label align="right" width="30" field="modificadorForca" horzTextAlign="center"/>
	</layout>

	<!-- Cálculo de campos / uso de dataLink -->
	<dataLink field="atributoForca">
		<event name="onChange">
			local valorForca = tonumber(sheet.atributoForca);

			if (valorForca ~= nil) then
				sheet.modificadorForca = math.floor(valorForca / 2) - 5;

				if sheet.modificadorForca > 0 then
					sheet.modificadorForca = "+" .. sheet.modificadorForca;
				end;
			else
				sheet.modificadorForca = nil;
			end
		</event>
	</dataLink>
</form>
```







Neste exemplo, ligamos um edit ao campo "atributoForca" e um label ao campo "modificadorForca"... Usamos um dataLink para monitorar mudanças em "atributoForca" para calcularmos e salvamos o modificador no campo "modificadorForca" (para que o label possa exibir o valor).
Veja também:

- Tratando eventos do Lua Form
- Lua Form e NodeDatabase

Exemplo 2 - Somando dois campos na interface

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<!-- Layout da interface -->
	<layout left="20" top="20" height="25" width="200">
		<edit field="parcela1" align="left" horzTextAlign="center" width="50"/>
		<label align="left" text="  +  " autoSize="true"/>

		<edit field="parcela2" align="left" horzTextAlign="center" width="50"/>
		<label align="left" text="  =  " autoSize="true"/>

		<!-- Resultado -->
		<label align="client" field="resultadoSoma" horzTextAlign="center"/>
	</layout>

	<!-- Cálculo de campos / uso de dataLink -->
	<dataLink fields="{'parcela1', 'parcela2'}">
		<event name="onChange">
			sheet.resultadoSoma = (tonumber(sheet.parcela1) or 0) +
					      (tonumber(sheet.parcela2) or 0);
		</event>
	</dataLink>
</form>
```





Neste exemplo, usamos o dataLink para monitorar dois campos ao mesmo tempo: "parcela1" e "parcela2"
Veja também:

- Tratando eventos do Lua Form
- Lua Form e NodeDatabase

Tag dataScopeBox

A tag/componente **dataScopeBox** funciona como a tag layout, porém permite definir em qual objeto Nodo seus controles salvarão/lerão os dados (NodeDatabase);
Após definir o nodo escopo, todos os componentes visuais que estão dentro desta tag passarão a ler e salvar os dados neste nodo.
Saiba mais sobre escopo de dados do NodeDatabase em Lua Form e NodeDatabase.
Atenção: A tag popup não é capturada pelo escopo de dados definido pelo dataScopeBox.

Herança

O **dataScopeBox** possui **todas** as características de uma tag layout.
Veja:

- Tag layout
- Características de todas as tags visuais.

Características

Além das características herdadas, o **dataScopeBox** possui também as seguintes características:

Propriedades

Propriedade	Tipo	Valor Padrão	Descrição
scopeNode, nodeObject ou node	Objeto Nodo	**nil**	Define em qual objeto nodo de um NodeDatabase os controles de edição deste dataScopeBox devem salvar os dados. Observações: ● Se for **nil**, os controles dentro desta tag que salvam/carregam dados no NodeDatabase ficam desabilitados. ● Só é possível alterar esta propriedade utilizando código Lua. Veja também: ● NodeDatabase ● Escopo de Dados

Métodos

É possível invocar métodos dos controles usando código LUA.

Método	Descrição
dataScopeBox:setNodeObject(nodeObject)	Método alternativo para alterar a propriedade "scopeNode"
dataScopeBox:getNodeObject();	Método alternativo que retorna o valor da propriedade "scopeNode"

Eventos da tag dataScopeBox

Nome do evento	Descrição
onNodeReady	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este dataScopeBox está pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a propriedade "node" do dataScopeBox está diferente de nil e possui uma referência válida para um objeto nodo. ● O processo de carregamento do nodeDatabase associado já chegou ao fim. Você já consegue acessar os dados armazenados nele normalmente.
onNodeUnready	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este dataScopeBox **deixa de estar** pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a propriedade "node" do dataScopeBox não contém mais uma referência válida para um objeto nodo.
onNodeChanged	Este evento é invocado um objeto nodo de um NodeDatabase é associado ou desassociado a este dataScopeBox. Ver NodeDatabase

Veja também Tratando eventos do Lua Form.

Exemplos:

Exemplo 1 - Lista com Painel de Detalhes

Veja o tutorial Criar uma lista dinâmica + painel de detalhes para ver o exemplo completo .

Tag edit

A tag/componente **edit** representa uma caixa de texto na interface. Um ótimo controle para permitir a entrada de dados!

Herança

O **edit** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **EDIT** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
textPrompt	String	<string vazio>	Um texto que é exibido quando o edit está vazio. Normalmente é usado para orientar o usuário qual informação ele deve preencher. Exemplo: "Preencha com seu nome"
transparent	Boolean	false	Quando True, as bordas e o fundo do edit não serão exibidas, mostrando apenas o texto conteúdo e o cursor de digitação.
isPasswordEdit	Boolean	false	Quando true, o edit conterá uma senha e os caracteres do texto serão exibidos como um caractere coringa.
readOnly	Boolean	false	Quando true, o edit não permite o usuário alterar seu conteúdo (somente leitura).
type	Enumerado: ● "text" ● "number" ● "float"	"text"	Define que tipo de dados o usuário pode digitar neste edit. "text" - Não há restrições e o usuário pode digitar qualquer coisa. "number" - O usuário pode digitar um valor numérico inteiro. Veja também as propriedades min e max do edit. A propriedade "keyboardType" é automaticamente alterada para "numberPad" se ela estiver como "default" ou "alphabet". "float" - O usuário pode digitar um valor numérico de ponto flutuante (números reais, "com vírgula")

min	Double	-2147483648	Quando o atributo "type" for igual a "number" ou "float", define qual o menor valor numérico que este Edit deve aceitar.
max	Double	2147483647	Quando o atributo "type" for igual a "number" ou "float", define qual o maior valor numérico que este Edit deve aceitar.
decimalPlaces	Integer	2	Quando o atributo "type" for igual a "float", define quantas casas decimais pode ter o número digitado pelo usuário.
keyboardType	Enumerado: ● "default" ● "numbersAndPunctuation" ● "numberPad" ● "phonePad" ● "alphabet" ● "url" ● "email"	"default"	Quando um teclado virtual for exibido (como aquele nos smartphones e tablets), qual é o layout preferencial de teclado para este conteúdo? "default" e "alphabet": Um teclado alfanumérico padrão. "numbersAndPunctuation": Um teclado para entrada de números de símbolos de pontuação. "numberPad": Um teclado com apenas números. "phonePad": Um teclado para entrada de número telefônico. "url": Um teclado para preencher uma URL da internet. "email": Um teclado para preencher endereços de e-mail.

enterKeyType	Enumerado: ● "default" ● "done" ● "go" ● "next" ● "search" ● "send"	"default"	Quando um teclado virtual for exibido (como aqueles nos smartphones e tablets), este atributo define o ícone que aparece na tecla enter. "default" - sem tratamento especial. Observação importante: Esta propriedade **apenas** muda a aparência do botão ENTER. Caso queira dar uma funcionalidade ao enter, trate o evento onKeyDown verificando se event.keyCode == 13
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o edit passa a apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
frameRegion	String	<string vazio>	Considerando que o componente/tag pai possui um frame (ver Frames), ao definir o valor desta propriedade, a posição e tamanho deste controle é automaticamente calculada para se encaixar na Região do Frame de mesmo nome. Veja Frames e Regiões
asNumber	Double	0.0	Contém o texto do edit como um número de ponto flutuante (número real). Se o texto do edit não for um número real válido, a propriedade contém 0.

Eventos

Nome do evento	Descrição

onChange	Este evento é invocado quando o conteúdo do edit é alterado.
onUserChange	Este evento é disparado quando o valor do campo do objeto Nodo associado (ver atributo "field") for alterado porque o usuário alterou o conteúdo do edit. Importante: Este evento é disparado APENAS se o atributo "field" estiver preenchido corretamente e se a alteração de valor ocorrer neste edit mediante ação do usuário.

Veja Tratando eventos do Lua Form

Exemplos:

Exemplo 1 - Edit simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <edit left="10" top="10"/>
</form>
```



Exemplo 2 - Edit transparente

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

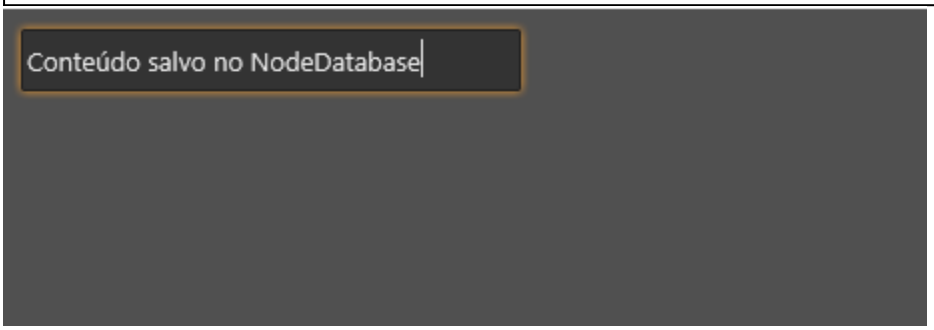
        <edit left="10" top="10" transparent="true"/>
</form>
```



Exemplo 3 - Edit que lê e salva dados no NodeDatabase

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <edit left="10" top="10" field="Destreza" width="250"/>
</form>
```



Veja também:

- Lua Form e NodeDatabase
- NodeDatabase

Exemplo 4 - (Avançado) Uma mini ficha, usando edits, labels, layouts e styles para evitar retrabalho.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
         <style>
                layout.LayoutCampo {
                        width: 64;
                        align: left;
                        margins: {left=2, right=2};
                }

                layout.LayoutCampo edit {
                        height:30;
                        horzTextAlign: center;
                        fontSize: 20;
                        align: top;
                }

                layout.LayoutCampo label {
                        align: top;
                        horzTextAlign: center;
                        vertTextAlign: leading;
                        autoSize: true;

                }
```

```
        </style>
        <layout left="10" top="10" width="200" height="64">
                <layout class="LayoutCampo">
                        <edit field="combate.armadura"/>
                        <label text="Armadura"/>
                </layout>

                <layout class="LayoutCampo">
                        <edit field="saude.pontosDeVida"/>
                        <label text="Pontos de Vida"/>
                </layout>

                <layout class="LayoutCampo">
                        <edit field="equipamentos.carga"/>
                        <label text="Carga (kg)"/>
                </layout>
        </layout>
</form>
```



Neste exemplo, os atributos dos edits, labels e layouts foram definidos por Estilo em Cascata pela tag **style**.
Foram definidos 3 blocos de estilos em cascata:

- Um com seletor "layout.LayoutCampo" (aplicado a todas as tags **layout** de class "**LayoutCampo**")
- Um com seletor "layout.LayoutCampo edit" (aplicado a todas as tags **edit** que estão dentro de uma tag **layout** de class "**LayoutCampo**")
- Um com seletor "layout.LayoutCampo label" (aplicado a todas as tags **label** que estão dentro de uma tag **layout** de class "**LayoutCampo**")

Veja também:
Tag STYLE
Tag LABEL
Tag LAYOUT

Exemplo 5 - (Avançado) O mesmo do exemplo 4, porém feito com tags TEMPLATES

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
```

```
        <template name="LayoutUmCampo">
                <layout width="64" align="left" margins="{left=2, right=2}">
                        <edit field="$(campo)" height="30" horzTextAlign="center" fontSize="20"
align="top"/>
                        <label text="$(titulo)" align="top" horzTextAlign="center"
                                vertTextAlign="leading" autoSize="true"/>
                </layout>
        </template>

        <layout left="10" top="10" width="200" height="64">
                <LayoutUmCampo campo="combate.armadura" titulo="Armadura"/>
                <LayoutUmCampo campo="saude.pontosDeVida" titulo="Pontos de Vida"/>
                <LayoutUmCampo campo="equipamentos.carga" titulo="Carga (kg)"/>
        </layout>
</form>
```



Veja também:
Tag TEMPLATE
Tag LABEL
Tag LAYOUT

Tag event

Esta tag especial serve para tratar um evento de uma tag/controle usando a linguagem de programação LUA (veja A linguagem de programação LUA).
Observações:

- O corpo da tag contém código de programação LUA
- Esta tag não cria nenhum controle.
- Você deve colocar a tag **event** dentro da tag que deseja tratar o evento.

Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

Características

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
name	String	<Não há valor padrão>	Identifica o nome do evento que deseja manipular. Cada tag/controle possui seu próprio conjunto de eventos que podem ser tratados. Consulte a documentação! O preenchimento desta propriedade é obrigatório.

Exemplos

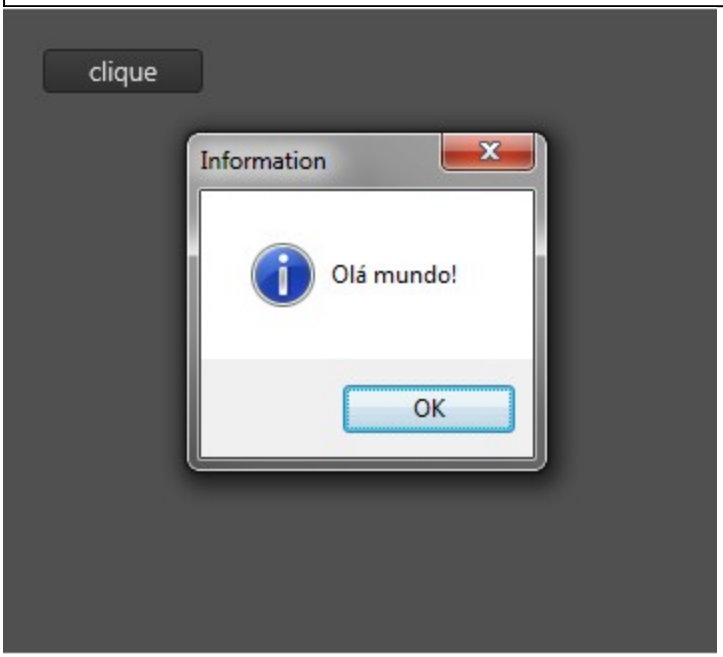
Exemplo 1 - Olá mundo com um botão.

A tag **event** está dentro da tag **button**, portanto, a tag está manipulando o evento "onClick" do **button**

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <button left="20" top="20" text="clique">
                <event name="onClick">
                        showMessage("Olá mundo!");
                </event>
        </button>

</form>
```



Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

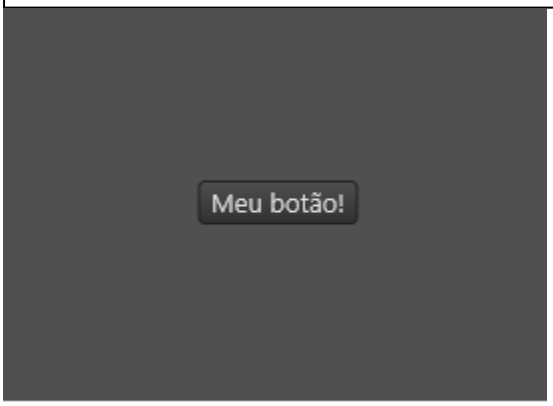
Exemplo 2 - Centralizando um controle na interface visual

A tag **event** está dentro da tag **form**, portanto ela está manipulando o evento "onResize" do **form**.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <event name="onResize">
                self.btnMeuBotao.left = self.width / 2 - self.btnMeuBotao.width / 2;
                self.btnMeuBotao.top = self.height / 2 - self.btnMeuBotao.height / 2;
        </event>

        <button name="btnMeuBotao" text="Meu botão!"/>
</form>
```





Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

Exemplo 3 - Definindo funções e as chamando em eventos

```xml
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

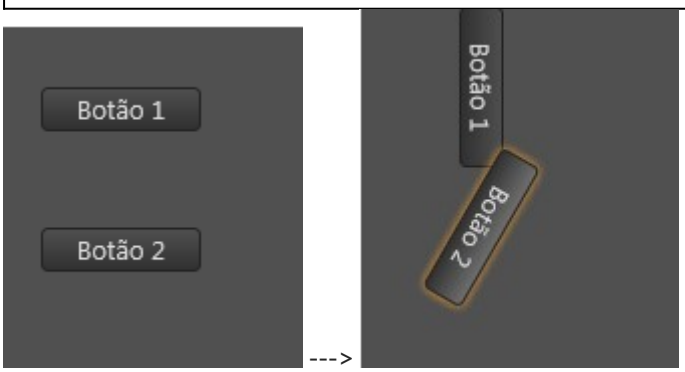
	<script><![CDATA[
		local function rotacionarUmPouco(controle)
			local angulo = controle.rotationAngle;
			angulo = angulo + 30;

			while angulo >= 360 do
				angulo = angulo - 360;
			end;

			controle.rotationAngle = angulo;
		end;
	]]>
	</script>

	<button name="btnMeuBotao" left="20" top="30" text="Botão 1">
		<event name="onClick">
			rotacionarUmPouco(self.btnMeuBotao);
		</event>
	</button>

	<button name="btnMeuBotao2" left="20" top="100" text="Botão 2">
		<event name="onClick">
			rotacionarUmPouco(self.btnMeuBotao2);
		</event>
	</button>
</form>
```



Neste exemplo foi usada a Tag script para definir uma função e foi usado um bloco CDATA para lidar com caracteres reservados do XML.
Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

Tag flowLayout

A tag/componente **flowLayout** organiza controles na interface de forma dinâmica. Excelente para criar interfaces que se adaptam a diferentes tamanhos de tela.
Os componentes que estão dentro de um flowLayout são automaticamente posicionados de forma sequencial e arranjados em linhas quando não houver espaço na horizontal.
Observação: Não é possível usar a propriedade "align" nos controles filhos desta tag.
IMPORTANTE: Esta tag foi descontinuada. Prefira utilizar o sistema de alinhamento em grid em vez desta tag.

Herança

O **flowLayout** possui **todas** as características da tag flowPart.
Veja:

- Tag flowPart
- Tag layout
- Características de todas as tags visuais.

Características

Além das características herdadas, o **flowLayout** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
autoHeight	Boolean	false	Quando true, o layout automaticamente se expande/contraí na vertical para caber seu conteúdo .
horzAlign	Enumerado: ● "leading" ● "center" ● "trailing" ● "justify"	"leading"	Define como os controles serão alinhados horizontalmente no layout dinâmico. "leading": Alinhados à esquerda "center": Alinhados ao centro "trailing": Alinhados à direita "justify": Alinhados de forma justificada
maxControlsPerLine	Integer	0	Quando definido definido (valor maior que zero), o layout dinâmico automaticamente realiza quebra de linhas quando a quantidade máxima de controles por linha for atingido.

orientation	Enumerado: ● "horizontal" ● "vertical"	"horizontal"	Define a orientação do layout. "horizontal": Os controles são dispostos em linhas. "vertical": Os controles são dispostos em colunas
maxColumns	Integer	0	Quando definido (valor maior que zero) e quando "orientation" for vertical, este valor define a quantidade máxima de colunas que o layout dinâmico deve ter.
lineSpacing	Float	0.0	Define o espaçamento entre as linhas do layout dinâmico.
contentWidth	Double	N/D	(Somente Leitura) Após o cálculo do layout, contém a largura do conteúdo do layout. Note que a largura do conteúdo pode ser diferente da largura do flowLayout.
contentHeight	Double	N/D	(Somente Leitura) Após o cálculo do layout, contém a altura do conteúdo do layout. Note que a altura do conteúdo pode ser diferente da altura do flowLayout.

Eventos

Nome do evento	Descrição
onBeforeLayoutCalc	Este evento é invocado antes do layout dinâmico ser calculado.
onAfterLayoutCalc	Este evento é invocado após o layout dinâmico ser calculado.

Métodos

Método	Descrição
flowLayout:needRealign()	Informa ao flowLayout que ele deve realinhar seu conteúdo.

Veja Tratando eventos do Lua Form

Exemplos

Exemplo 1 - Layout Dinâmico de vários botões

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
```

```
        <flowLayout left="20" top="20" width="250" autoHeight="true" orientation="horizontal">
                <button text="Botão 1"/>
                <button text="Botão 2"/>
                <button text="Botão 3"/>
                <button text="Botão 4"/>
                <button text="Botão 5"/>
                <button text="Botão 6"/>
                <button text="Botão 7"/>
        </flowLayout>
</form>
```



orientation="horizontal"



orientation="vertical"

Exemplo 2 - Layout Dinâmico usando flowPart para se ajustar a vários tamanhos de interfaces

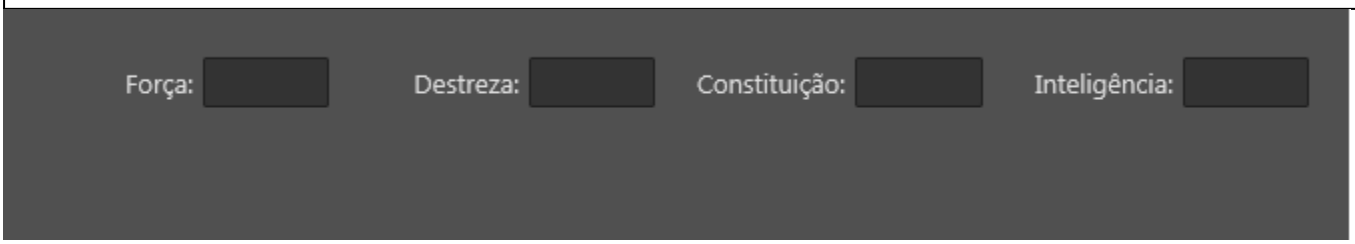
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

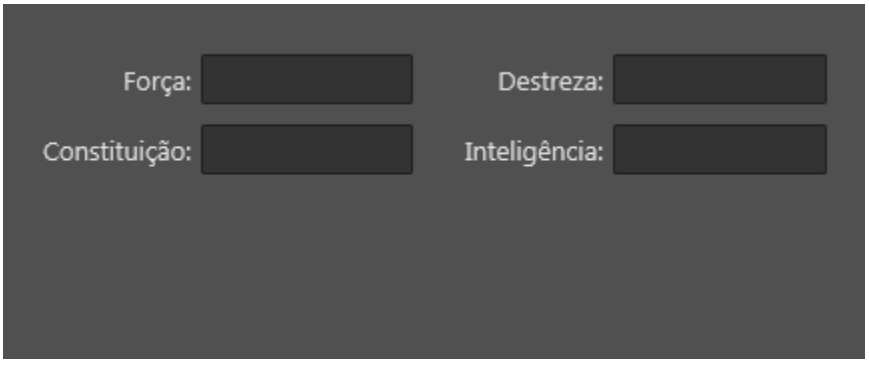
        <style>
                flowPart {
                        margins: {left=5, right=5, top=5, bottom=5};
                }
                label {
                        width: 75;
                        horzTextAlign: trailing;
                        margins: {right=5};
                }
        </style>

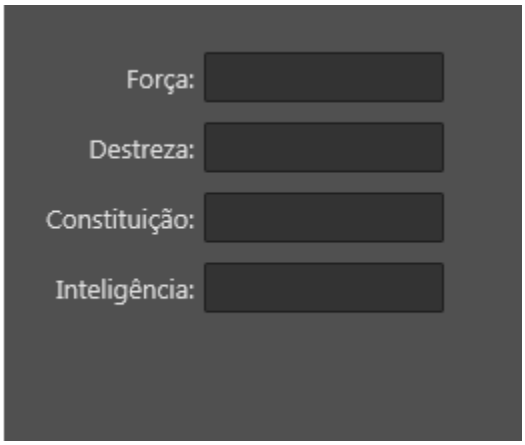
        <flowLayout align="client" orientation="horizontal" horzAlign="justify"
                    margins="{left=20, top=20, bottom=20, right=20}">

                <flowPart minWidth="130" maxWidth="200" height="25">
```

```
                        <label align="left" text="Força: "/>
                        <edit align="client"/>
                </flowPart>
                <flowPart minWidth="130" maxWidth="200" height="25">
                        <label align="left" text="Destreza: "/>
                        <edit align="client"/>
                </flowPart>
                <flowPart minWidth="130" maxWidth="200" height="25">
                        <label align="left" text="Constituição: "/>
                        <edit align="client"/>
                </flowPart>
                <flowPart minWidth="130" maxWidth="200" height="25">
                        <label align="left" text="Inteligência: "/>
                        <edit align="client"/>
                </flowPart>
        </flowLayout>
</form>
```







Neste exemplo foi utilizada a Tag style para aplicar algumas propriedades a vários controles de uma só vez. Cada Tag flowPart usada pode assumir largura entre 130 e 200.

Tag flowLineBreak

A tag/componente **flowLineBreak** é um componente invisível e serve apenas para quebrar linha em um layout dinâmico (Ver Tag flowLayout).
Observações:

- Deve ser usado junto com a Tag flowLayout.
- Se a orientação do flowLayout for "vertical", esta tag cria uma nova coluna no layout.

IMPORTANTE: Esta tag foi descontinuada. Prefira utilizar o sistema de alinhamento em grid em vez desta tag.

Herança

O **flowLineBreak** possui **todas** as características de uma tag visual.
Veja:

- Características de todas as tags visuais.

Características

Além das características herdadas, o **flowLineBreak** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
horzAlign	Enumerado: ● "leading" ● "center" ● "trailing" ● "justify	<não definido>	Se definido, altera as novas linhas do layout dinâmico serão alinhadas horizontalmente: "leading": Alinhados à esquerda "center": Alinhados ao centro "trailing": Alinhados à direita "justify": Alinhados de forma justificada

	"		
lineSpacing	Float	<não definido>	Se definido, altera o espaçamento entre as novas linhas do layout dinâmico.

Exemplos

Exemplo 1 - Quebra de linha

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <flowLayout left="20" top="20" width="250" autoHeight="true">
                <button text="Botão 1"/>
                <button text="Botão 2"/>
                <flowLineBreak/>
                <button text="Botão 3"/>
                <flowLineBreak/>
                <button text="Botão 4"/>
                <button text="Botão 5"/>
                <button text="Botão 6"/>
                <button text="Botão 7"/>
        </flowLayout>
</form>
```



Tag flowPart

A tag/componente **flowPart** representa uma parte dinâmica de um layout dinâmico (Ver Tag flowLayout). É um controle similar à uma tag **layout,** mas com a possibilidade de definir uma largura dinâmica, isto é, um layout que muda sua largura de acordo com a necessidade.
Observação: Deve ser usado junto com a Tag flowLayout.

IMPORTANTE: Esta tag foi descontinuada. Prefira utilizar o sistema de alinhamento em grid em vez desta tag.

Herança

O **flowPart** possui **todas** as características da tag layout.
Veja:

- Tag layout
- Características de todas as tags visuais.

Características

Além das características herdadas, o **flowPart** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
minWidth	Float	<o mesmo que width>	Define a largura mínima que esta parte dinâmica pode assumir. Quando não definido, o valor "width" será usado e, portanto, a parte não mudará sua largura de forma dinâmica.
maxWidth	Float	<o mesmo que width>	Define a largura máxima que esta parte dinâmica pode assumir. Quando não definido, o valor "width" será usado e, portanto, a parte não mudará sua largura de forma dinâmica.
minScaledWidth	Float	<o mesmo que minWidth>	Este valor deve ser menor ou igual à propriedade "minWidth". Quando definido, o layout dinâmico poderá utilizar, para este controle, uma largura menor que "minWidth", porém escalando o conteúdo de forma proporcional até uma largura mínima de "minScaledWidth"
maxScaledWidth	Float	<o mesmo que maxWidth>	Este valor deve ser maior ou igual à propriedade "maxWidth". Quando definido, o layout dinâmico poderá utilizar, para este controle, uma largura maior que "maxWidth", porém escalando o conteúdo de forma proporcional até uma largura máxima de "maxScaledWidth"

avoidScale	Boolean	false	Quando true e quando alguma das propriedades "minScaledWidth" e "maxScaledWidth" estiverem definidas, o layout dinâmico assumirá uma largura menor que "minWidth" ou maior que "maxWidth" **apenas** em último caso (Por exemplo: Quando um controle por si só não caber numa única linha).
vertAlign	Enumerado: ● "leading" ● "center" ● "trailing"	"leading"	Na linha do layout dinâmico que este controle foi alocado, qual será seu alinhamento vertical? ● "leading" - Este controle estará alinhado no topo de sua linha ● "center" - Este controle estará alinhado no centro de sua linha ● "trailing" - Este controle estará alinhado no rodapé de sua linha.
stepSizes	Arranjo de Float	{}	Por padrão, este controle poderá ter qualquer largura entre "minWidth" e "maxWidth" (uma faixa de números reais). Ao usar "stepSizes", este controle assumirá apenas alguma das larguras que estão no arranjo. Exemplos: ● {100, 200, 300} - O controle poderá assumir larguras 100, 200 ou 300.. Não poderá assumir, por exemplo, a largura 150. ● {200, 450} - o controle poderá assumir ou a largura de 200 ou de 450. Observação: Ao usar esta propriedade, o controle passa a desconsiderar as propriedades "minWidth" e "maxWidth" (porém minScaledWidth e

			maxScaledWidth continuam sendo respeitados)
adjustHeightToLine	Boolean	false	Quando true, este controle pode se redimensionar a fim de ocupar todo o espaço vertical da linha em que foi alocado.

Exemplos

Exemplo 1 - Layout Dinâmico usando flowPart para se ajustar a vários tamanhos de interfaces

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<style>
		flowPart {
			margins: {left=5, right=5, top=5, bottom=5};
		}

		label {
			width: 75;
			horzTextAlign: trailing;
			margins: {right=5};
		}
	</style>

	<flowLayout align="client" orientation="horizontal" horzAlign="justify"
	    margins="{left=20, top=20, bottom=20, right=20}">

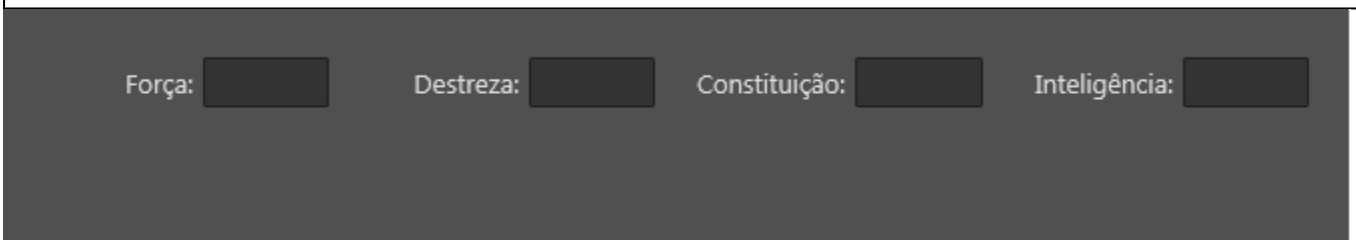
		<flowPart minWidth="130" maxWidth="200" height="25">
			<label align="left" text="Força: "/>
			<edit align="client"/>
		</flowPart>

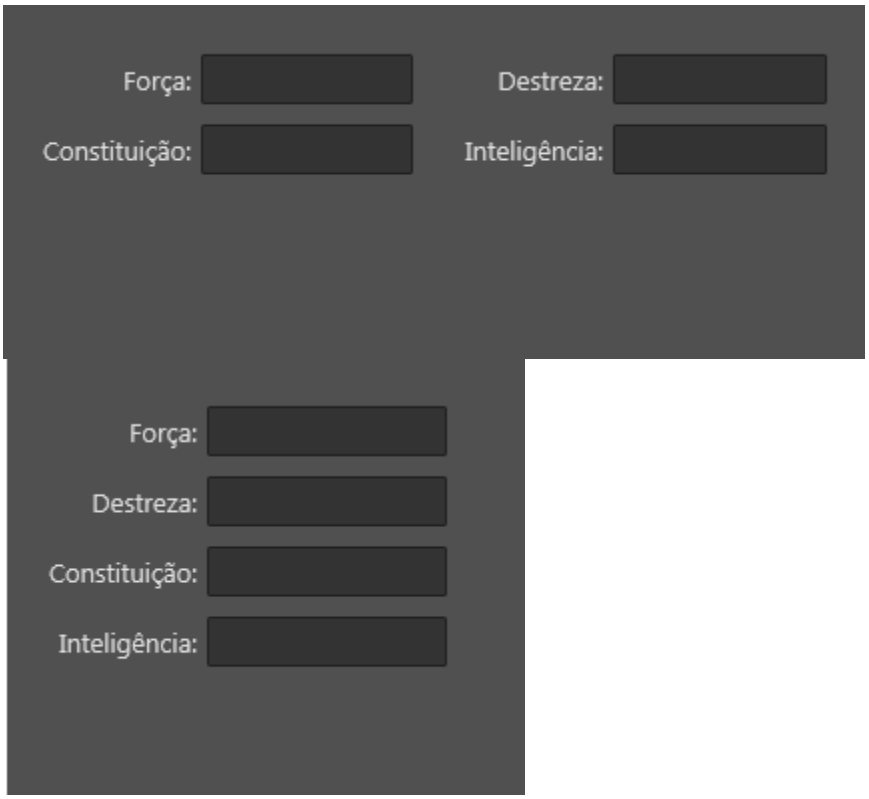
		<flowPart minWidth="130" maxWidth="200" height="25">
			<label align="left" text="Destreza: "/>
			<edit align="client"/>
		</flowPart>

		<flowPart minWidth="130" maxWidth="200" height="25">
			<label align="left" text="Constituição: "/>
			<edit align="client"/>
		</flowPart>

		<flowPart minWidth="130" maxWidth="200" height="25">
```

```
                        <label align="left" text="Inteligência: "/>
                        <edit align="client"/>
                </flowPart>
        </flowLayout>
</form>
```





Neste exemplo foi utilizada a Tag style para aplicar algumas propriedades a vários controles de uma só vez. Cada tag flowPart usada pode assumir largura entre 130 e 200.

Tag form

A tag/componente **form** representa uma ficha, um formulário ou uma janela do plug-in.
As principais características do **form** são:

- Para que alguma interface visual seja apresentada na tela do usuário, é necessário existir ao menos 1 **form**. Sem **form**, não há apresentação da interface.
- Apenas os componentes que estão dentro de um **form** são apresentados na tela.
- É possível existir um **form** dentro de outro **form.**
- É possível definir um tema para um **form** e todos os componentes que estão dentro dele. Exemplo: Você quer que o form (e demais componentes) tenha aparência escura ou clara?
- É possível associar um Nodo de um NodeDatabase (Ver NodeDatabase) a um **form.** Quando você faz isso, todos os componentes de edição (que estão dentro do form) passam a salvar os dados neste nodo.
    - Os modelos de ficha fazem isso! Mas você não precisa se preocupar com isso, pois o SDK 3 faz isso automaticamente para você ;) .

Herança

O **form** possui **todas** as características de um controle qualquer. Veja Características de todas as tags visuais**.**

Características

Além das características herdadas, o **form** possui também as seguintes características:

Propriedades e atributos da tag FORM

Propriedade	Tipo	Valor Padrão	Descrição
title	String	<string vazio>	Define o título do formulário. Este título pode ser apresentado em vários lugares, como identificação da ficha (se for um modelo de ficha) ou como título de janela.
description	String	<string vazio>	Define um texto descritivo do formulário. Se for um modelo de ficha, este valor pode ser apresentado ao usuário durante a criação do personagem.

dataType	String	<string vazio>	Define um tipo de dados que este formulário é capaz de editar. Se você está criando uma ficha nova, você deve inventar este texto. A partir de então, este será o identificador da sua ficha! Este campo deve ser preenchido se você estiver criando uma janela acoplável de mesa que salve dados em um NodeDatabase armazenado no servidor RRPG. Neste caso, o dataType serve para identificar qual NodeDatabase da mesa deverá ser aberto para salvamento de dados. O valor deve possuir ao menos 5 caracteres alfanuméricos, "_" ou "." e não deve começar com um dígito. Exemplos: “RRPG.MeuNome.DnD5” “br.com.Meusite.VampiroAMascara” "br.com.MeuSite.MeuTablesDock"
formType	Enumerado: ● “undefined” ● “sheetTemplate” ● "tablesDock"	“undefined”	Define a finalidade deste formulário. “sheetTemplate” – Este formulário é um modelo de ficha. "tablesDock" - Este formulário é uma janela acoplável de mesa. **Importante**: Se você quiser que sua janela acoplável de mesa salve dados em um NodeDatabase armazenado no servidor RRPG, preencha também a propriedade "dataType". Janelas acopláveis são visíveis apenas para assinantes Gold e/ou em mesas onde seu criador for Gold Plus. “undefined” – Este formulário não tem uma finalidade bem definida e pode ser usado para várias finalidades diferentes. O RRPG não fará nenhum tratamento especial.

theme	Enumerado: ● “default” ● “light” ● “dark”	“default”	Define o tema de cores que os componentes visuais terão neste formulário. “default”: Tema padrão (cores escuras). “light”: Tema de cores claras. “dark”: Tema de cores escuras. Observação: Mesmo que outros temas sejam adicionados futuramente, "default" sempre usará cores escuras.
lockWhileNodeIsLoading	Boolean	true	Quando true, o form apresenta o texto “carregando” e trava a interface enquanto o NodeDatabase associado estiver baixando dados.
minWidth	Float	<o mesmo que width>	Quando dentro de uma tag flowLayout ou quando for um item de uma tag recordList layout "horizontalTiles" ou "verticalTiles", define a largura mínima que este form pode assumir, o tornando adaptável ao "tamanho da tela". Quando não definido, o valor "width" será usado e, portanto, o form não mudará sua largura de forma dinâmica.
maxWidth	Float	<o mesmo que width>	Quando dentro de uma tag flowLayout ou quando for um item de uma tag recordList layout "horizontalTiles" ou "verticalTiles", define a largura máxima que este form pode assumir, o tornando adaptável ao "tamanho da tela". Quando não definido, o valor "width" será usado e, portanto, o form não mudará sua largura de forma dinâmica.
isShowing	Boolean	False	(Somente Leitura) Esta propriedade contém True se o form está sendo exibido na interface do usuário ou false se o form não está visível.

cacheMode	Enumerated: ● “none” ● “time”	"none"	Determines if and how the form can be cached to improve the performance of loading character sheets or showing items on record lists/grid record lists. ● "none" - No caching. When instances of this form are needed, new instances are always freshly allocated. ● "time" - Cached by time. When instances of this form are no longer needed (e.g., because a character is closed or an item is removed from a grid record list), the SDK3 will keep the instance alive for 1 minute. When instances of this form are needed, the SDK3 will reuse cached free instances of this form instead of allocating a new one. Remarks: ● Prior to SDK version 3.7, the SDK always used the "time" cache mode implicitly. ● Caching forms can lead to significant performance gains, but the programmer needs to prepare their code for the reuse behavior: the same instance can be used to display more than one character's data.

Métodos da tag FORM

Método	Descrição

form:setNodeObject(nodeObject)	Define em qual objeto nodo de um NodeDatabase os controles de edição deste formulário devem salvar os dados. Parâmetros: ● nodeObject – Um objeto Nodo (de um NodeDatabase) ou **nil**. Ver: ● NodeDatabase ● Escopo de Dados
form:getNodeObject();	Retorna o objeto nodo (de um NodeDatabase) no qual os controles de edição deste formulário devem salvar os dados. **nil** é retornado quando não há nodo associado. Ver: ● NodeDatabase ● Escopo de Dados
form:lockWithActivity([msg])	Desabilita a interação do usuário com o formulário exibindo um indicador de atividade e, opcionalmente, uma mensagem. Ideal quando você quer fazer o usuário aguardar a conclusão de algum processo qualquer antes de voltar a interagir com a interface. Parâmetros: ● (OPCIONAL) msg - uma cadeia de caracteres contendo a mensagem que será exibida na interface. Observações: ● Este método segue uma ideia de pilha: Você pode invocar este método mais de uma vez. Para cada chamada a este método, deve haver uma chamada do método unlockWithActivity para destravar a interface. A mensagem exibida será sempre a da última chamada ao método.
form:unlockWithActivity()	Volta a habilitar a interação entre o usuário e a interface que foi previamente travada pelo método form:lockWithActivity Retorna true se o form foi realmente liberado, ou false se precisar de mais chamadas de unlockWithActivity para isso.

Eventos da tag Form

Nome do evento	Descrição
onNodeReady	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este formulário está pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a variável "sheet" do LuaForm está diferente de nil e possui uma referência válida para um objeto nodo. ● O processo de carregamento do nodeDatabase associado já chegou ao fim. Você já consegue acessar os dados armazenados nele normalmente.
onNodeUnready	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este formulário **deixa de estar** pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a variável "sheet" do LuaForm não contém mais uma referência válida para um objeto nodo.
onNodeChanged **ou** **onScopeNodeChanged**	Este evento é invocado um objeto nodo de um NodeDatabase é associado ou desassociado a este formulário. Ver NodeDatabase
onShow	Este evento é invocado quando o formulário for ficar visível na interface.
onHide	Este evento é invocado quando o formulário for ser escondido/fechado.

Veja também Tratando eventos do Lua Form.

Tag gridRecordList

The **gridRecordList** tag/component is a list of records in the interface. Used in conjunction with a NodeDatabase, the control presents a panel in the interface for each child node of a field.

Remarks:

- The item panel that is shown for each node is customized by the programmer.
- The item panel must be a form (see Tag form) and is called the Template Form.
- You will need at least two files with the ".lfm" extension: One containing the "gridRecordList", and another representing the item panel.
- Each item has its own node object, and each item panel is automatically bound to its data scope.

Key differences from recordList tag:

- gridRecordList is built to be used with The Grid System. To work properly, both the gridRecordList and the templateForm must use the grid system organization.
- gridRecordList does not have a built-in scrollBox. You need to provide your own scrollBox tag in the lua form containing the `gridRecordList` somehow.
- gridRecordList has features for filtering the items.
- gridRecordList has features for paging the items.
- gridRecordList has features for grouping the items.

Inheritance

The **gridRecordList** has **all** the characteristics of the layout tag.

See also:

- Tag layout
- Characteristics of all visual tags.

Characteristics

In addition to the inherited features, **gridRecordList** also has the following features:

Properties and attributes

Attribute	Type	Default value	Description
field	String	<empty string>	The path of a field in NodeDatabase. When associated, the gridRecordList starts using this field to save and display the items in this list. Comments: ● This field must be informed for the gridRecordList to work. ● The informed field will become a node in the `NodeDatabase`. See also: ● Lua Form and

			NodeDatabase ● NodeDatabase
templateForm	String	<empty string>	Identifies the gridRecordList template form, that is, the panel that will be displayed for each item. It must be equal to the "name" attribute of a form contained in some ".lfm" file of the plugin project. Yes, you will need another ".lfm" file to define to make this tag work.
minQt	Integer	0	Defines the minimum number of items that this gridRecordList must present in the interface. Try setting the value 1 to this property. Sometimes this helps the user =)
selectable	Boolean	False	Determines whether the gridRecordList should visually function as a list of selectable items.
selectedNode	Node object	<< none >>	Contains the Node object that is currently focused/selected in the gridRecordList. If there is no selection, it contains **nil**. When assigning a value, if the informed node object is not a child of the node informed in the attribute "field", the selection will be undone, and "selectedNode" will become **nil**.
selectedForm	Form object	<< none >>	(Read-Only) Contains the Form object of the currently focused/selected item in the

			gridRecordList. If there is no selection, it contains **nil**.
paging	Boolean	False	Define if the paging feature is enabled.
pageSize	Integer	20	When "paging" == true, specifies the maximum item count that can be displayed per page.
pageIndex	Integer	0	When "paging" == true, specified the current page index. 0 is the first page.
pageCount	Integer	<< none >>	(Read-Only) When "paging" == true, contains the number of pages that exist in the gridRecordList. When "paging" == false, it always contains the number 1.
filterFields	Array of string	{}	An array containing the names of fields that item nodes have. The gridRecordList monitors these fields for changes and automatically triggers the filter, group, and sort processes in response to such events.

Methods

Method	Description
gridRecordList:append();	Inserts a new item into the gridRecordList. Retorno: ● Returns the new Node object of the inserted item if the insertion is successful, otherwise returns **nil**.

gridRecordList:reorganize();	Forces the gridRecordList to re-execute the process of filtering, grouping, and sorting its items.
gridRecordList:scrollToNode(node)	Scrolls through the interface's scrollBoxes so that the item's panel is visible on the screen for the user. Arguments: ● node - A Node object identifying the node of the item you want to make visible on the user's screen.

Events

Event	Description
onSelect	This event is triggered when the selection of the recordList changes, that is, when an item is selected or when no item is selected. Remarks: ● To determine which item was selected, use the selectedNode and selectedForm properties. ● This event is fired even when the "selectable" property is false.
onItemAdded	Event triggered when a Node object is added to the gridRecordList. Arguments: ○ node - The Node object that was added. Remarks: ● This event represents an item that has been enumerated in the list and not necessarily that a new item has been created in the saved data. During data loading, for example, this event is triggered for each loaded item.
onItemRemoved	Event triggered when an item is removed from the gridRecordList. Arguments: ○ node - The Node object that was removed.

	Remarks: ● This event represents an item that is no longer enumerated in the list and not necessarily that an item has been deleted from the saved data.
onItemFiltered	Event triggered when an item is filtered and can be displayed in the gridRecordList. Arguments: ○ node - The Node object that was filtered.
onItemUnfiltered	Event triggered when an item is no longer filtered and cannot be displayed in the gridRecordList anymore. Arguments: ○ node - The Node object that is no longer filtered.
onItemShow	Event triggered when an item becomes visible in the gridRecordList. Arguments: ○ node - The Node object that is now visible. ○ form - The Form object of the item that is now visible.
onItemHide	Event triggered when an item is no longer visible in the gridRecordList. Arguments: ○ node - The Node object that is no longer visible. ○ form - The Form object of the item that is no longer visible
onPagingStateChange	This event is triggered when some of the paging fields of the gridRecordList have changed. The paging fields are: "paging", "pageSize", "pageIndex", and "pageCount".

onCompare	This event is triggered at key moments to ensure the sorting of items displayed in the interface. When triggered, the event receives 2 nodes for the programmer to return, between the two, which should appear first. Arguments: ○ left - Node object containing a node to be compared. ○ right - Node object containing the other node to be compared. Return: ○ Return a number **less** than 0 if "left" should be displayed **before** "right" ○ Return a number **greater** than 0 if "left" should be displayed **after** "right" ○ Return 0 if the two are equivalent in terms of order.
onFilter	The event is triggered so the programmer can determine if a node is filtered and can be displayed in the gridRecordList. Arguments: ○ node - The Node object that is being queried. Return: ○ Return **true** if the node can be displayed in the gridRecordList or **false** otherwise.
onEmptyState	This event is triggered when no items can be displayed on the gridRecordList. The programmer can customize the appearance of the gridRecordList when it is in an empty state. Return: ○ A visual control instance that the gridRecordList must use as empty state information.

onGroupId	The event is triggered for each item in the gridRecordList so the programmer can assign groups to them. Arguments: ○ node - The Node object that is being queried. Return: ○ A string containing the identification of the group the node belongs to. Remarks: ● If this event is not handled or the programmer does not return a string as a result, the node will be assigned to the group identified by "" (empty string).
onGroupCompare	This event is triggered when the gridRecordList determines the order of item groups. When triggered, the event receives 2 identification strings for the programmer to return, between the two, which should appear first. Arguments: ○ left - A string, the identification of a group; ○ right - A string, the identification of the other group to be compared. Return: ○ Return a number **less** than 0 if "left" should be displayed **before** "right" ○ Return a number **greater** than 0 if "left" should be displayed **after** "right" ○ Return 0 if the two are equivalent in terms of order. Remarks: ● "left" and "right" are values determined by the onGroupId event.

onGroupHeader	This event is triggered so the programmer can provide a control that must be used as a visual group header for the items in the group. Arguments: ○ id - A string, the identification of the group; Returns: ○ A visual control instance that the gridRecordList must use as the group header.
onGroupShow	Event triggered when a group header becomes visible in the gridRecordList. Arguments: ○ id - A string, the identification of the group that became visible. ○ header - The visual control instance of the header that is now visible.
onGroupHide	Event triggered when a group header is no longer visible in the gridRecordList. Arguments: ○ id - A string, the identification of the group that is no longer visible. ○ header - The visual control instance of the header that is no longer visible
onBeginOrganization	Event triggered when the gridRecordList starts the organization of its items After this event, the gridRecordList may trigger multiple calls to the following events: ● onItemAdded/onItemRemoved ● onItemFiltered/onItemUnfiltered ● onItemShow/onItemHide ● onPagingStateChange ● onCompare ● onFilter ● onEmptyState ● onGroupId

	● onGroupCompare ● onGroupHeader ● onGroupShow/onGroupHide
onEndOrganization	Event triggered when the gridRecordList finishes the organization of its items and it will not trigger more of the events listed in the "onBeginOrganization" event until the next trigger to the event "onBeginOrganization".

See Handling Lua Form events

Examples:

Goto https://sdk3.firecast.app/examples/gridRecordList/ to see examples of the gridRecordList

Tag gridRecordListPgNav

The **gridRecordListPgNav** tag/component is a built-in page navigator for gridRecordList tag.

Inheritance

The **gridRecordList** has **all** the characteristics of the layout tag.

See also:

- Tag layout
- Characteristics of all visual tags.

Characteristics

In addition to the inherited features, **gridRecordListPvNav** also has the following features:

Properties and attributes

Attribute	Type	Default value	Description
target	GridRecordList object	**nil**	The gridRecordList that this page navigator must control. Remarks: ● In a Lua form, specify the name of the gridRecordList.

			• This navigator only works if the gridRecordList paging attribute is **true**.

Examples:

Goto https://sdk3.firecast.app/examples/gridRecordListPgNav/ to see examples of the gridRecordListPgNav.

Tag group

Com a tag **group**, o programador pode agrupar outras tags. Esta tag não faz nada além disso, não afeta a interface visual e nem a interpretação do Lua Form.

Não há propriedades e nem atributos.

É especialmente útil para usar com a tag import. Honestamente, sua única utilidade é essa hehehe =)

Exemplos

Exemplo 1 - Importação de várias tags usando a Tag group

Usando a Tag import conseguimos importar um grupo de tags que está em outro documento XML.

Arquivo “ficha.lfm”

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <import file="OutroArquivo.xml"/>

        <MinhaTag titulo="Meu Botão"/>
</form>
```

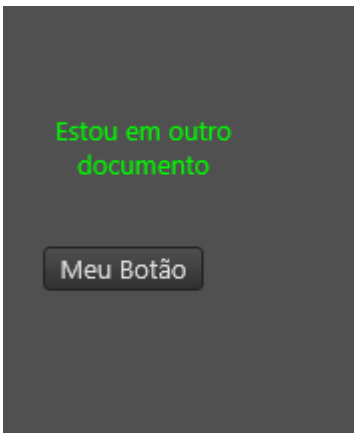
Arquivo “OutroArquivo.xml”

```
<?xml version="1.0" encoding="UTF-8"?>
<group>
        <style>
```

```
                label {
                        fontColor: lime;
                }
        </style>

        <template name="MinhaTag">
                <button left="20" top="120" text="$(titulo)"/>
        </template>

        <layout width="100" height="100" top="20" left="20">
                <label align="client" horzTextAlign="center"
                       text="Estou em outro documento"/>
        </layout>
</group>
```



Tag horzLine

A tag/componente **horzLine** representa uma linha na horizontal com cores e espessura personalizada.

Herança

O **horzLine** possui **todas** as características de um controle shape/forma.

Veja

- Características de tags shapes
- Características de todas as tags visuais.

Exemplos

Exemplo 1 - Uma linha simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <horzLine left="10" top="20" width="200" strokeColor="red"/>
</form>
```



Exemplo 2 - Uma linha mais espessa

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <horzLine left="10" top="20" width="200" strokeColor="red" strokeSize="7"/>
</form>
```



Tag image

A tag/componente **image** representa uma imagem na interface visual.

Herança

O **image** possui **todas** as características de um controle qualquer. Veja Características de todas as tags visuais.

Características

Além das características herdadas, o **image** também possui as seguintes características:

Propriedades e Atributos

Propriedade	Tipo	Valor Padrão	Descrição
animate	Boolean	false	Define se deve executar a animação de imagens que são animadas (exemplo: GIF, GIFV) quando suportadas.
url, src ou imageURL	String	<string vazio>	Define a URL/caminho de onde se encontra a imagem! Este valor: ● Pode ser um endereço de internet (exemplo: "http://xxxx.com.br/a.png") ● Pode ser o caminho de um arquivo contido no pacote do plugin. ● Pode ser um arquivo que se encontra no HD Virtual do plugin instalado.
style ou showStyle	Enumerado: ● "proportional" ● "aut	"proportional"	Define como a imagem será apresentada na interface: "proportional": A imagem será redimensionada, proporcionalmente, para o tamanho do controle, não necessariamente ocupando todo o espaço disponível. "autoFit": A imagem será exibida de forma que SEMPRE ocupe TODO o espaço do controle da forma mais adequada, isto é, aumentando/diminuindo a

	oFit” ● “originalSize” ● “stretch”		imagem, centralizando-a e cortando excessos. “originalSize”: A imagem será apresentada no tamanho original, sem redimensionamento. “stretch”: A imagem será redimensionada de forma que ocupe todo o espaço disponível no controle, podendo achatar ou alongar a imagem.
center	Boolean	true	Quando true, a imagem será exibida de forma centralizada no controle. Esta propriedade é válida apenas quando a propriedade “showStyle” for “proportional” ou “originalSize”
optimizeQuality ou **optimize**	Boolean	false	Quando true, o SDK salvará uma cópia da imagem já redimensionada de acordo com a propriedade “style”. Utilize este atributo **apenas** em controles **image** que não sofrem constante mudança de tamanho.
showProgress	Boolean	true	Quando true, o controle exibe uma barra de carregamento para informar que a imagem está sendo baixada. Quando false, a imagem definida pela propriedade “URLWhileLoading” é exibida enquanto o download é feito.

URLWhileLoading	String	<string vazio>	Define a URL/caminho da imagem que é exibida APENAS enquanto a imagem principal (definida na propriedade “url”) é carregada. Este valor: ● Pode ser um endereço de internet (exemplo: “http://xxxx.com.br/a.png”) ● Pode ser o caminho de um arquivo contido no projeto do plugin ● Pode ser um arquivo que se encontra no HD Virtual do plugin instalado.
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, a imagem passa a apresentar a imagem de URL contida no campo do NodeDatabase informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
editable	Boolean	false	Quando True, a tag permite o usuário alterar a URL da imagem. Se o atributo "field" estiver preenchido, a URL escolhida pelo usuário é salva no campo do NodeDatabase.
naturalAnimat	Boolean	false	(Somente Leitura) Quando a

ed			imagem tiver sido carregada, contém a informação se ela suporta animação.
naturalWidth	Double	0	(Somente Leitura) Quando a imagem tiver sido carregada, contém a largura original da imagem em pixels.
naturalHeight	Double	0	(Somente Leitura) Quando a imagem tiver sido carregada, contém a altura original da imagem em pixels.

Eventos

Nome do evento	Descrição
onLoad	Este evento é invocado quando a imagem tiver sido carregada.
onPictureLoadedChange	Este evento é invocado quando a imagem que será exibida mudou. Este evento é chamado quando a imagem é carregada 100% ou quando uma imagem é descarregada.

Veja Tratando eventos do Lua Form

Exemplos

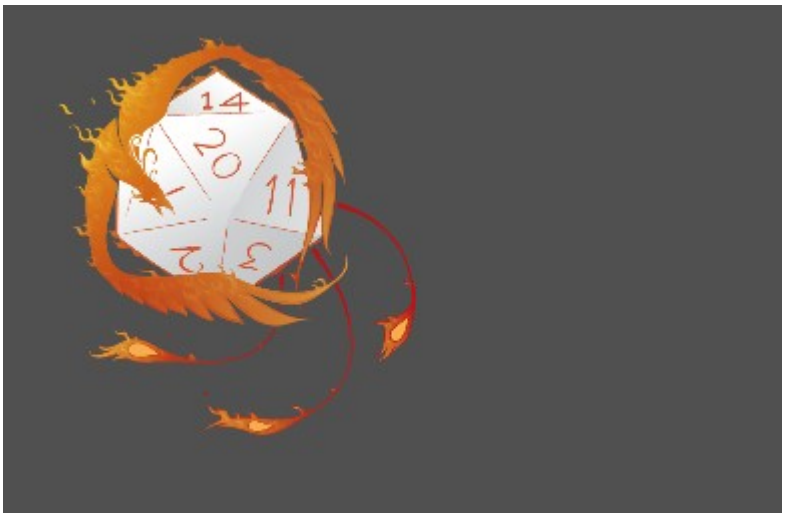
Exemplo 1 - Imagem dentro do projeto de plug-in

Neste exemplo, existe uma pasta chamada "imagens" dentro da pasta projeto do plug-in, e dentro desta pasta existe a imagem "fenix.png".

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <image left="20" top="20" width="200" height="200"
src="/imagens/fenix.png"/>

</form>
```



Exemplo 2 - Imagem baixada por URL e diversos styles

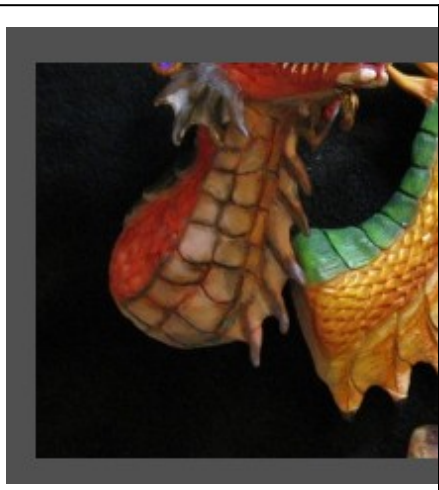
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <image left="20" top="20" width="400" height="200" style="autoFit"

src="http://fc03.deviantart.net/fs70/i/2011/234/5/4/dragon_aspects_by_rattlesnakedefender-d47ii5y.jpg"/>
</form>
```



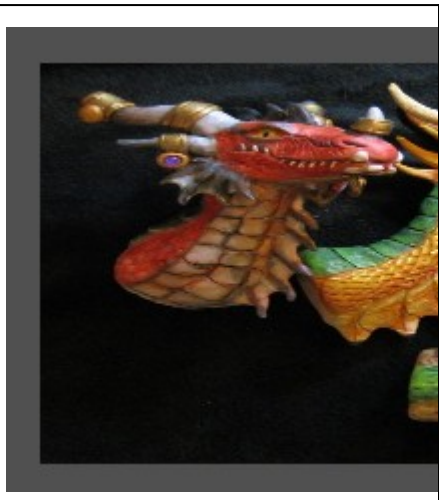
style="proportional"



style="au



style="originalSize"



style="str

Tag imageCheckBox

A tag/componente **imageCheckBox** representa uma caixa de marcação na interface no qual é possível personalizar a imagem apresentada quando a caixa está marcada e quando está desmarcada.

Herança

O **imageCheckBox** possui todas as características de uma tag visual. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **imageCheckBox** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
checked	Boolean	false	Indica se a caixa de marcação está marcada (true) ou desmarcada (false)
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o imageCheckBox passa a

			apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase ●
checkedImage ou imageChecked	String	<string vazio>	Define a URL/caminho de onde se encontra a imagem que será exibida quando a caixa de marcação estiver **marcada**. Este valor: ● Pode ser um endereço de internet (exemplo: "http://xxxx.com.br/a.png") ● Pode ser o caminho de um arquivo contido no pacote do plugin ● Pode ser um arquivo que se encontra no HD Virtual do plugin instalado. ●
uncheckedImage ou imageUnchecked	String	<string vazio>	Similar à propriedade "checkedImage", porém contém o endereço da imagem que será exibida quando a caixa de marcação estiver **desmarcada**.
optimize	Boolean	false	Quando true, o SDK salvará uma cópia das imagens de exibição já redimensionadas para o melhor tamanho. Utilize este atributo **apenas** em controles **image** que não sofrem constante mudança de tamanho.
autoChange	Boolean	true	Define se o imageCheckBox irá

			alternar a propriedade checked automaticamente quando o usuário clicar nele. False significa que você tratará isso por conta própria e alterará a propriedade checked quando for mais propício.

Eventos

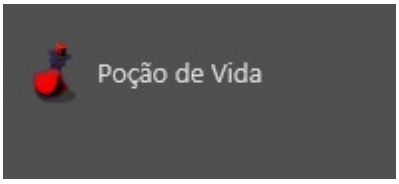
Nome do evento	Descrição
onChange	Este evento é invocado quando a marcação do imageCheckBox sofre alteração.

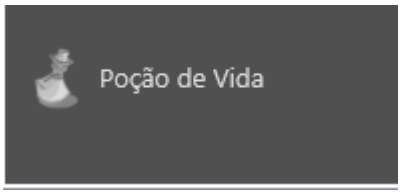
Exemplos:

Exemplo 1 - Usando ícones de poção no imageCheckBox

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <layout left="10" top="20" width="200" height="32">
                <imageCheckBox width="32" align="left"
                               checkedImage="/imagens/potion.png"
                               uncheckedImage="/imagens/potion_off.png"/>
                <label align="client" text="Poção de Vida" vertTextAlign="center" margins="{left=
        </layout>
</form>
```





Tag import

Com a tag **import**, o programador pode importar/incluir um outro documento XML ao Lua Form, acarretando na substituição da tag pelo conteúdo do arquivo especificado.

É especialmente útil para:

- Organizar o documento Lua form. Exemplo: Quando o arquivo ".lfm" ficar grande, é interessante dividir ele em partes menores, cada parte em seu documento XML separado.
- Reaproveitar código. Exemplo: Se você tiver templates ou styles que queira reaproveitar em outros Lua Forms, você pode criar um documento XML que contenha apenas o que queira reaproveitar e o importar nos Lua Forms.

Características

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
file	String	<Não há valor padrão>	Endereço de um arquivo XML. Este endereço é relativo à localização do documento atual. Portanto, se o outro arquivo XML estiver na mesma pasta, você só precisa informar o nome dele. Exemplo: "MeuDocumento.xml"

Exemplos

Exemplo 1 - Importação simples

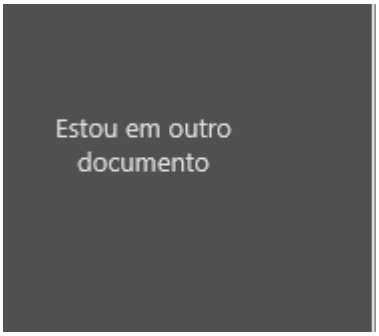
Arquivo "ficha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <import file="OutroArquivo.xml"/>
</form>
```

Arquivo "OutroArquivo.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<layout width="100" height="100" top="20" left="20">
        <label align="client" horzTextAlign="center" text="Estou em outro
documento"/>
</layout>
```



Exemplo 2 - Importação de várias tags usando a Tag group

Usando a Tag group conseguimos importar um grupo de tags que está em outro documento XML.

Arquivo "ficha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <import file="OutroArquivo.xml"/>

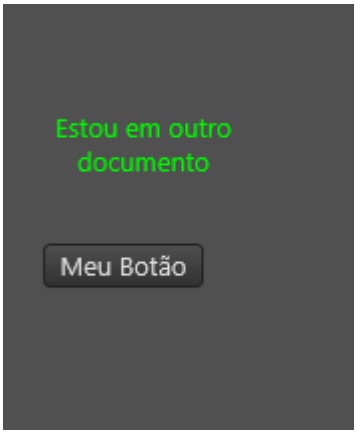
        <MinhaTag titulo="Meu Botão"/>
</form>
```

Arquivo "OutroArquivo.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<group>
        <style>
                label {
                        fontColor: lime;
                }
        </style>

        <template name="MinhaTag">
                <button left="20" top="120" text="$(titulo)"/>
        </template>
```

```
        <layout width="100" height="100" top="20" left="20">
                <label align="client" horzTextAlign="center"
                        text="Estou em outro documento"/>
        </layout>
</group>
```



Tag label

A tag/componente **label** representa um texto não editável na interface. Excelente para mostrar informações.

Herança

O **label** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **label** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
autoSize	Boolean	false	Se for true, o label automaticamente ajusta seu tamanho (crescendo ou encolhendo) para melhor exibir o seu conteúdo.

<table>
<tr><td>field</td><td>String</td><td><string vazio></td><td>Caminho de um campo no NodeDatabase.

Quando associado, o label passa a apresentar o conteúdo do campo na interface.

Veja também:

● Lua Form e NodeDatabase
● NodeDatabase</td></tr>
<tr><td>format</td><td>String</td><td><string vazio></td><td>Define uma formatação automática para o texto do label.

Esta propriedade aceita formatações estilo printf da linguagem C ou Format da linguagem Pascal.

Exemplos:

<table><tr><td>%d</td><td>Número inteiro</td></tr><tr><td>%f</td><td>Número float/"com vírgula"</td></tr><tr><td>%3.3f</td><td>Número float apresentando 3 casas decimais</td></tr><tr><td>%s</td><td>Texto</td></tr><tr><td>%.2f kg</td><td>Número com duas casas decimais seguidas do texto "kg"</td></tr><tr><td>Carga em</td><td>Apresenta um texto</td></tr></table></td></tr>
</table>

<table>
<tr><td></td><td></td><td></td><td>
<table>
<tr><td>%1.1f %%</td><td>como: Carga em 25,3 %</td></tr>
</table>

Para documentação completa, favor procurar a documentação da função Format do Delphi.

Observações:

● Se a propriedade formatFloat estiver preenchida, a propriedade format é ignorada.
● Apenas 1 parâmetro é suportado em format.</td></tr>
<tr><td>formatFloat</td><td>String</td><td><string vazio></td><td>Define uma formatação automática para números no label. Esta permite um controle maior do que a propriedade format.

Esta propriedade aceita formatações da função FormatFloat do Delphi.

Exemplos:
<table>
<tr><td>0.00</td><td>Número com 2 casas decimais</td></tr>
<tr><td>,0.00</td><td>Número com 2 casas decimais e separador de milhares</td></tr>
<tr><td>0.0#</td><td>Número com 1 ou 2</td></tr>
</table></td></tr>
</table>

				casas decimais, dependendo do valor.
			0.### litros	Número com 0 a 3 casas decimais, dependendo do valor e, no final, a palavra "litros"
			0.00## # kg	Número com 2 a 5 casas decimais, dependendo do valor e, no final, a palavra "kg"
			Total: ,0 .00	Palavra "Total: " seguida de número com 2 casas decimais e separador de milhares.

			Receita de 0.00;Prejuízo de 0.00	O caracter ; separa seções para valores positivos, negativos e zero. Se positivo ou maior que zero, usa a formatação "Receita de 0.00", se negativo, utiliza a formatacao "Prejuízo de 0.00"
			+0.00;(0.00);vazio	O caracter ; separa seções para valores positivos, negativos e zero. Se positivo

			<table><tr><td></td><td>: +0.00 Se negativo: (0.00) Se zero: vazio</td></tr></table> Para documentação completa, favor procurar a documentação da função FormatFloat do Delphi. Observações: ● Se a propriedade formatFloat estiver preenchida, a propriedade format é ignorada.

Exemplos:

Exemplo 1 - Olá Mundo

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <label left="20" top="20" text="Olá Mundo"/>

</form>
```



Exemplo 2 - Cores

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <label left="20" top="20" text="Label de cor verde com quebra de
linha" fontColor="lime" autoSize="true"/>

</form>
```



Veja também [String de cores no Lua Form]

Exemplo 3 - Usando Field para integrar com o NodeDatabase

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <label left="20" top="20" field="campoForca" autoSize="true"
wordWrap="false"/>

        <button left="20" top="40" width="150" text="Aumentar Força"
                onClick="sheet.campoForca = (sheet.campoForca or 0) +
1;"/>

</form>
```

Neste exemplo, um botão que incrementa o valor do field "campoForca" enquanto o label, conectado ao mesmo campo, exibe seus valores.



Veja também:

- Tratando eventos do Lua Form
- Lua Form e NodeDatabase

Tag layout

(Conteúdo desta página não consta no documento original.)

Tag path

A tag/componente **path** representa uma forma geométrica/shape totalmente personalizada e vetorial. Todas as formas geométricas básicas podem ser criadas usando esta tag.

Herança

O **path** possui **todas** as características de um controle shape/forma.

Veja

- Características de tags shapes
- Características de todas as tags visuais.

Características

Além das características herdadas, o **rectangle** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
data ou pathData	String	<String	Define a forma vetorial do path. Este texto deve estar

		Vazio>	no mesmo padrão do path do SVG e do XAML.
mode ou wrapMode	Enumerado: ● "fit" ● "original" ● "stretch" ● "tile"	"fit"	Define como a forma será desenhada na interface. "fit" - a forma vai ser redimensionada proporcionalmente para ocupar o tamanho da tag path. "original" - a forma será desenhada sem nenhum tratamento de tamanho e posição. "stretch" - a forma vai ser esticada para ocupar todo o tamanho da tag path. "tile" - a forma será desenhada várias vezes para ocupar o tamanho da tag path.
roundToPixel	Boolean	false	Quando true, as posições do shape serão arredondadas para ocuparem pelo menos 1 pixel da tela. Esta opção pode melhorar o aspecto de alguns paths e pode piorar o aspecto de outros. Teste antes!

Exemplos

Exemplo 1 - Um triângulo laranja

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <path align="client" margins="{left=10, right=10, bottom=10,
top=10}"
              color="orange" data="M 100 100 L 300 100 L 200 300 z"/>
```

```
</form>
```



Exemplo 2 - Outra forma com vários modos diferentes

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <path align="client" margins="{left=10, right=10, bottom=10, top=10}" mode="fit" color="r
data="M133,31.8c-0.6,4.3,0.3,9.7,0.3,9.7c0,0,-3.1,-2.3,-6.5,-4.3c-2,-1.1,-3.8,-3.6,-4.8,-5.5c-0.8
1.3,-1.2,-2.3,-1.2,-2.3c-4.8,5.1,-1.7,11.1,-1.7,11.1c-7.4,-1.6,-12,0.3,-12,0.3c14,9.2,6.6,9.9,-
1.5,13.8c-8.1,3.8,-8.7,8.6,-8.7,8.6c9.6,-3.2,13.5,0.9,13.5,0.9c-5.2,4.9,-0.6,12.7,-
0.6,12.7c5.1,9.7,17.6,13.8,28.8,15.5c11.1,1.7,21,1,21,1l-25.6,8l-22.9,5.8c0,0,-22.8,-2.5,-31.2,-
21.7C71.6,66.1,86.2,50.9,84.7,49.9c-1.4,-1,-3.2,1.1,-3.3,1.4c6.3,-18.2,25.5,-18,26.2,-19.9c0.8,-2
3.8,-2.1,-3.8,-2.1c19.4,-6.4,29.2,2.5,29.2,2.5Z"/>

</form>
```



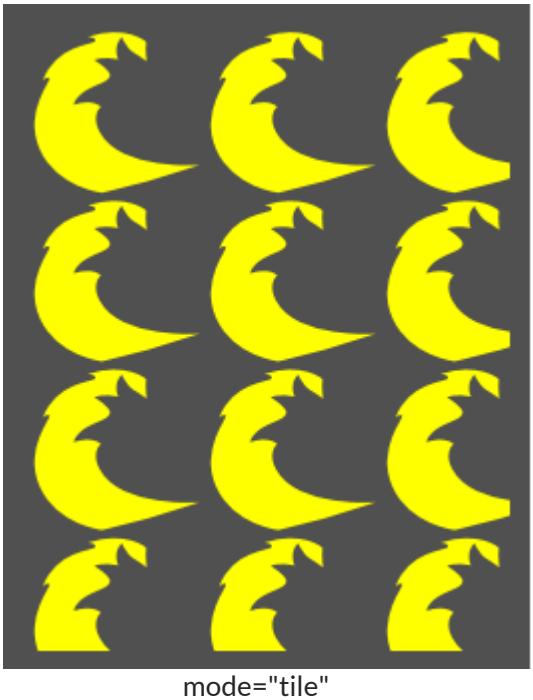
mode="fit"



mode="original"



mode="stretch"



mode="tile"

Tag popup

A tag/componente **popup** representa um componente visual que fica escondido na interface e é posteriormente apresentado, tomando a atenção para si, na forma de um popup.

Herança

O **popup** possui **todas** as características da tag layout.

Veja:

- Tag layout
- Características de todas as tags visuais.

Características

Além das características herdadas, o **popup** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
autoScopeNode	Boolean	true	Define o comportamento dos campos de edição que estão dentro deste popup quando a propriedade "scopeNode" for **nil**. true - se scopeNode for nil, os componentes de edição usarão o scopeNode do formulário. false - se scopeNode for nil, os componentes de edição não funcionarão, pois não haverá um scopeNode. Observação: O valor padrão destee atributo para plugins que forem compilados com o SDK 3.4 ou inferior é false.
cancelable	Boolean	true	Indica se o popup pode ser cancelado pelo usuário. Quando false, a única maneira de fechar o popup é via programação ao invocar o método "close" com o parâmetro "forcar" igual a

			true.
backOpacity	Float	0.85	Indica a opacidade do fundo ao exibir este popup. Um valor entre 0.0 e 1.0, onde 0.0 = Fundo totalmente transparente e 1.0 = Fundo totalmente opaco. Esta opacidade se refere ao fundo que é aplicado a fim de tirar a atenção da interface e chamar a atenção para o popup.
drawContainer	Boolean	true	Define se o popup vai automaticamente desenhar uma borda que delimita o conteúdo do popup.
scopeNode, **nodeObject ou node**	Objeto Nodo	**nil**	Define em qual objeto nodo de um NodeDatabase os controles de edição deste popup devem salvar os dados. Observações: ● Só é possível alterar esta propriedade utilizando código Lua. ● Se **nil**, o nodeObject do form será utilizado para salvar dados se a propriedade autoScopeNode for true. ● Esta propriedade é um "atalho" para os métodos "setNodeObject " e "getNodeObject

			" desta tag. Veja também: ● NodeDatabase ● Escopo de Dados

Eventos

Nome do evento	Descrição
onClose	Este evento é invocado quando o popup for fechado. Parâmetros: ● canceled - Boolean indicando se o popup foi cancelado ou se foi fechado normalmente.
onCanClose	Este evento é invocado quando o popup estiver prestes a ser fechado. Retorne false para impedir o popup de ser fechado ou true para autorizar o fechamento do popup. Parâmetros: ● canceled - Boolean indicando se o popup será cancelado ou se será fechado normalmente.
onCalculateSize	Se você quiser definir a largura e altura do popup de forma dinâmica, trate este evento. Parâmetros: ● dueToResize - Booleano indicando se a interface mudou de tamanho e por isso é necessário recalcular o tamanho do popup. ● width - Largura padrão ● height - Altura padrão Retorne 2 números, o primeiro contendo a largura e o segundo contendo a altura (Exemplo: return 100, 200;)
onNodeReady	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este popup está pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a propriedade "node" do poup está diferente de nil e possui uma referência válida para um objeto nodo. ● O processo de carregamento do nodeDatabase associado já chegou ao fim. Você já consegue

	acessar os dados armazenados nele normalmente.
onNodeUnready	Este evento é invocado quando o objeto nodo de um NodeDatabase associado a este popup **deixa de estar** pronto para ser usado. Quando este evento é chamado, você pode assumir: ● a propriedade "node" do popup não contém mais uma referência válida para um objeto nodo.
onNodeChanged	Este evento é invocado um objeto nodo de um NodeDatabase é associado ou desassociado a este popup. Ver NodeDatabase

Veja Tratando eventos do Lua Form

Métodos

É possível invocar métodos dos controles usando código LUA.

Método	Descrição
popup:show(); **ou** **popup:showPopup();**	Exibe o popup na interface. O popup será exibido no centro a interface.
popup:close(); **ou** **popup:closePopup();**	Fecha o popup.
popup:show(placement, control); **ou** **popup:showPopup(placement, control);** **ou** **popup:showPopupEx(placement, control);**	Exibe o popup na interface permitindo o programador definir onde ela será exibida. Parâmetros: ● placement - String que define onde o popup será exibido em relação ao "control". Pode ser um destes valores: ○ "bottom" - abaixo de control ○ "top" - acima de control ○ "left" - à esquerda de

	control ○ "right" - à direita de control ○ "center" - no centro de control ○ "bottomCenter" - abaixo de control e centralizado horizontalmente ○ "topCenter" - acima de control e centralizado horizontalmente ○ "leftCenter" - à esquerda de control e centralizado verticalmente ○ "rightCenter" - à direita de control e centralizado verticalmente ○ "mouse" - onde o mouse está atualmente ○ "mouseCenter" - centralizado onde o mouse está atualmente. ● control - Um controle/tag lua à qual o parâmetro "placement" se refere. Se nil, o form será utilizado.
popup:close(cancelar, forcar); **ou** **popup:closePopup(cancelar, forcar);**	Fecha o popup. Parâmetros: ● cancelar - Booleano indicando se o popup será cancelado ou se será fechado normalmente. Caso este parâmetro seja omitido, o valor padrão "false" será adotado. ● forcar - Booleano indicando se é para forçar o fechamento do popup mesmo se ele tiver atributo "cancelable" com true. Caso este parâmetro seja omitido, o valor padrão "false" será adotado.
popup:setNodeObject(nodeObject)	Define em qual objeto nodo de um NodeDatabase os controles de edição deste

	popup devem salvar os dados. Parâmetros: ● nodeObject – Um objeto nodo (de um NodeDatabase) ou **nil**. Observação: Se **nil**, o nodeObject do form será utilizado para salvar dados. Veja também: ● NodeDatabase ● Escopo de Dados
popup:getNodeObject();	Retorna o objeto nodo (de um NodeDatabase) no qual os controles de edição deste popup devem salvar os dados. **nil** é retornado quando não há nodo associado. Observação: Se **nil**, o nodeObject do form será utilizado para salvar dados. Veja também: ● NodeDatabase ● Escopo de Dados

Exemplos

Exemplo 1 - Uma imagem pequena que pode ser ampliada através do uso do popup

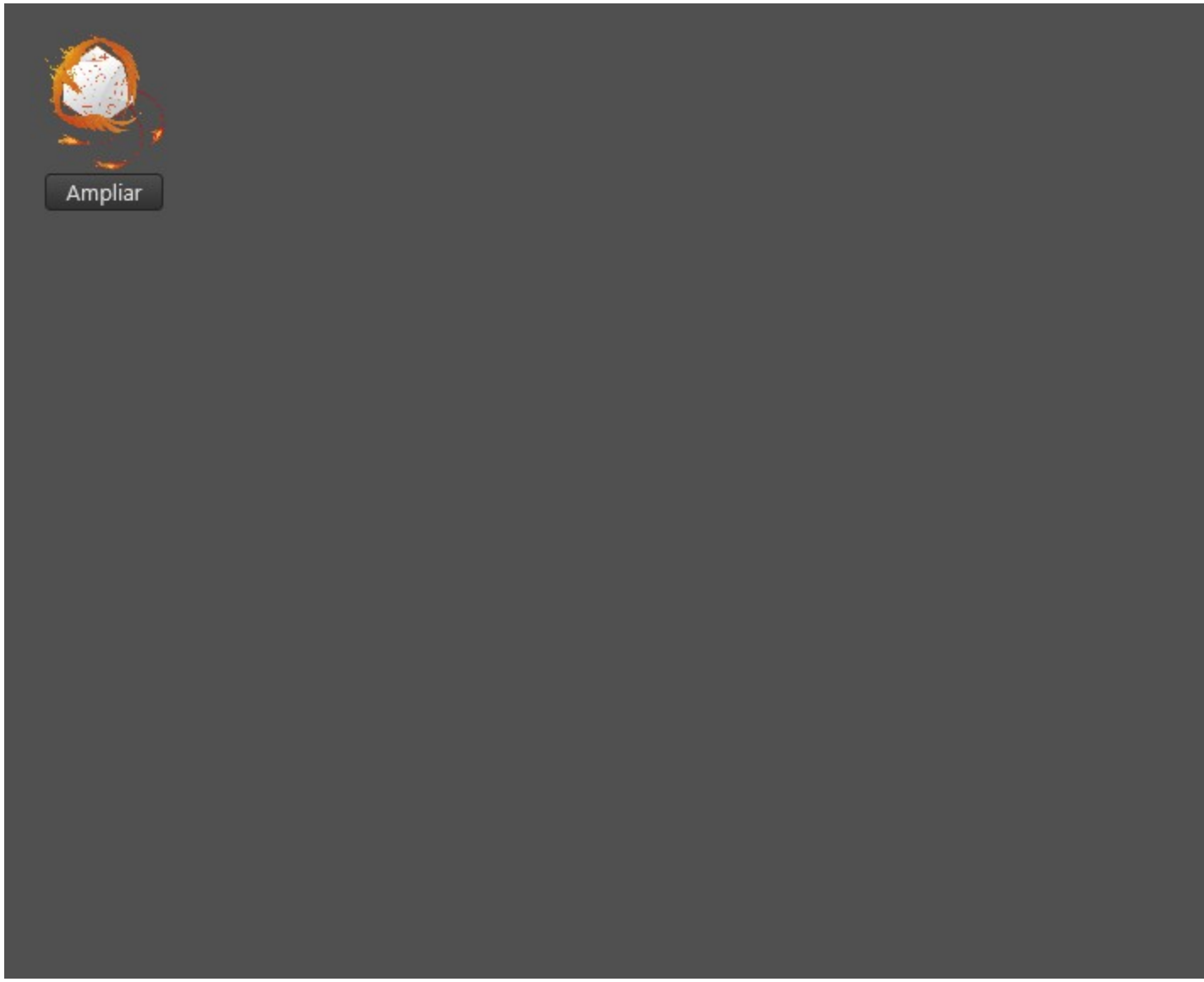
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<!-- Popup -->
	<popup name="popMeuPopup" width="400" height="400" backOpacity="0.5">
		<label align="top" text="Veja só a imagem maior!" autoSize="true"/>
		<image align="client" src="/imagens/fenix.png"/>
	</popup>

	<!-- Invocador do Popup -->
	<image src="/imagens/fenix.png" left="20" top="20" width="80" height="80"/>

	<button text="Ampliar" width="70" left="25" top="102"
		onClick="self.popMeuPopup:show();"/>
```

```
</form>
```





Neste exemplo foram usadas também:

- Tag image
- Tag label
- Tag button

Exemplo 2 - Mostrando o popup abaixo de um componente na interface

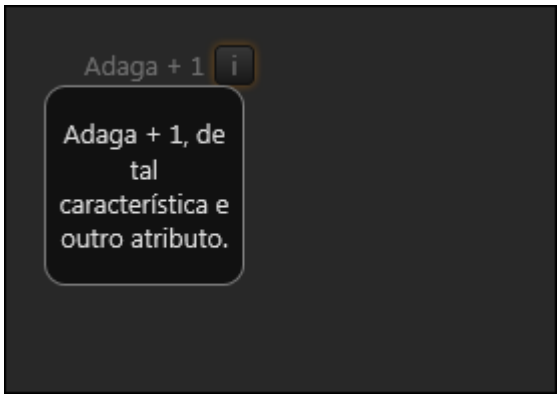
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <!-- Popup -->
        <popup name="popMeuPopup" width="100" height="100" backOpacity="0.5">
                <label align="client" horzTextAlign="center" text="Adaga + 1, de tal caracterísi
```

```
outro atributo." />
        </popup>

        <!-- Invocador do Popup -->
        <label name="labNome" left="20" top="20" width="80" text="Adaga + 1"
               horzTextAlign="trailing" height="20"/>
        <button text="i" width="20" left="105" top="20" height="20"
                onClick="self.popMeuPopup:show('bottom', self.labNome);"/>
</form>
```





Tag popupForm

A tag/componente **popupForm** é uma especialização da tag form que contém propriedades próprias para exibir uma janela popup na interface do usuário .

Você pode usar esta tag como tag raiz de um arquivo LFM.

Herança

O **popupForm** possui **todas** as características da Tag form.

Veja:

- Tag form
- Características de todas as tags visuais.

Características

Além das características herdadas, o **popupForm** possui também as seguintes características:

Propriedades e atributos da tag popupForm

Propriedade	Tipo	Valor	Descrição

		Padrão	
drawContainer	Boolean	true	Define se o popupForm deve desenhar uma borda que delimita seu conteúdo.
cancelable	Boolean	true	Define se o popupForm pode ser cancelado pelo usuário. Quando true, o popupForm fecha automaticamente quando o usuário fizer uma ação de cancelamento de popup (tecla ESC, clique no botão X da janela em desktop, tecla voltar em celulares/tablets, click fora do Popup em dispositivos móveis, etc..). Quando false, o popupForm não fecha automaticamente quando o usuário fizer uma ação padrão de cancelamento do popup. Assim, a única maneira de fechar o popupForm é invocar o método "close" ou "gui.closePopup" via programação. O evento "onCancelRequest" é disparado apenas quando "cancelable" for false.

placement	String enumerado: ● "center" ● "bottom" ● "top" ● "left" ● "right" ● "topLeft" ● "topRight" ● "bottomLeft" ● "bottomRight" ● "mouse" ● "mouseCenter"	"center"	Define onde o popupForm deve ser exibido. "center" - no centro da tela "bottom" - na parte de baixo da tela "top" - na parte de cima da tela "left" - à esquerda na tela "right" - à direita na tela "topLeft" - no canto superior esquerdo da tela "topRight - no canto superior direito da tela "bottomLeft" - no canto inferior esquerdo da tela "bottomRight" - no canto inferior direito da tela "mouse" - onde o mouse está atualmente "mouseCenter" - centralizado onde o mouse está atualmente.
resizable	Boolean	false	Define se o usuário pode redimensionar o popup form manualmente. Funciona somente se a propriedade "drawContainer" for true. Observações: ● Por enquanto, esta propriedade não possui efeito no Firecast Mobile.
movable	Boolean	true	Determines whether the user can move the popup

			form by clicking and dragging it. This property is considered only if the "drawContainer" property is set to "false".

Métodos da tag popupForm

Método	Descrição
popupForm:show();	Exibe o popup na interface do usuário Retorna true se o popupForm não estava visível e passou a ficar visível na interface.
popupForm:close();	Fecha o popup da interface do usuário. Retorna true o popupForm estava visível na interface e passou a não ficar visível.

Eventos da tag popupForm

Nome do evento	Descrição
onCancelRequest	Se o atributo "cancelable" for false, este evento é invocado quando o usuário fizer uma ação sinalizando que deseja fechar o popup. Você pode realizar ações "bacanas" aqui, como, por exemplo: ● Invocar o método "close" apenas se o usuário não tiver preenchido nenhum campo da interface ● Perguntar ao usuário se ele realmente deseja cancelar (usando dialogs.confirmYesNo) e só depois então fechar o popup invocando o método "close".

Veja também Tratando eventos do Lua Form.

Tag progressBar

A tag/componente **progressBar** representa uma barra de progresso/estado, ótimo para dar noção ao usuário do progresso de algo em um formato visual.

Herança

O **progressBar** possui **todas** as características das tags visuais.

Veja:

- Características de todas as tags visuais.

Características

Além das características herdadas, o **progressBar** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
color	Cor	"green"	Define a cor da barra de progresso. Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso.
position **ou** **value**	Float	0.0	Define a posição atual da barra de progresso.
min	Float	0.0	Define o valor mínimo da posição da barra de progresso.
max	Float	100.0	Define o valor máximo da posição da barra de progresso.
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o progressBar passa a ler seu "position/value" a partir deste campo. Veja também: ● Lua Form e NodeDatabase

			• NodeDatabase
fieldMin	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o progressBar passa a ler seu "min" a partir deste campo. Veja também: • Lua Form e NodeDatabase • NodeDatabase
fieldMax	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o progressBar passa a ler seu "max" a partir deste campo. Veja também: • Lua Form e NodeDatabase • NodeDatabase
mouseGlow	Boolean	true	Define se a barra deve brilhar quando o usuário passar o mouse em cima. OBS: Este atributo só tem efeito se o valor do atributo hitTest for "true"
colorMode	Enumerado: • "default" • "hl"	"default"	Define como o progressBar desenhará sua cor na interface. "default" - Modo padrão, efeito degradê sobre a cor escolhida. "hl" - Half Lightness/meia iluminosidade. Aplica efeito degradê em cima do tom da cor escolhida com ilumonisade em 50%. As barrinhas de HP/MP/ETC do RRPG utilizam este modo de coloração. Por fixar a iluminosidade em

			50%, cores com diferentes brilhos serão apresentadas da mesma forma (Exemplo: azul claro e azul escuro)

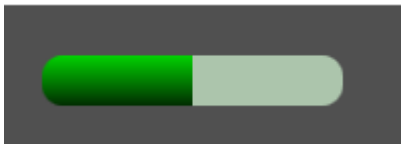
Exemplos

Exemplo 1 - Exemplos de ProgressBars de várias cores

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

        <progressBar width="150" color="green" top="25" left="20"
value="50" max="100"/>

</form>
```

color="green"	
color="red"	
color="black"	
color="darkorchid"	

Tag radioButton

A tag/componente **radioButton** representa uma caixa de marcação na interface que, quando usada em grupo, permite apenas uma esteja marcada por vez.

Herança

O **radioButton** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **radioButton** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
checked	Boolean	false	Indica se a caixa de marcação está marcada (true) ou desmarcada (false)
groupName	String	<string vazio>	Um texto qualquer usado pelo programador programador para especificar grupos de radioButton. Quando um radioButton é marcado, todos os outros radioButtons que possuírem o mesmo valor de groupName são desmarcados.
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o radioButton passa a apresentar e salvar o conteúdo no campo informado. Ao preencher este campo, é recomendado que você também preencha o atributo "fieldValue" abaixo.

			Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
fieldValue	String	<string contendo o mesmo name do controle/tag >	Quando usado com o NodeDatabase através do "field", este atributo contém o valor que é salvo no campo do NodeDatabase quando o radioButton estiver selecionado. Na prática, existirão vários radioButtons ligados ao **mesmo** field, porém cada um com um "fieldValue" diferente.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando a marcação do radioButton sofre alteração.

Veja Tratando eventos do Lua Form

Exemplos:

Exemplo 1 - RadioButtons simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">

        <radioButton text="Força" groupName="grupoDeRadios1" field="atributoBasico"
fieldValue="for"
                     top="10" left="20"/>

        <radioButton text="Destreza" groupName="grupoDeRadios1" field="atributoBasico"
fieldValue="des"
                     top="32" left="20" />
```

```
        <radioButton text="Inteligência" groupName="grupoDeRadios1"
field="atributoBasico" fieldValue="int"
                    top="54" left="20"/>
</form>
```



Tag recordList

A tag/componente **recordList** é uma lista de registros na interface. Usada em conjunto com um NodeDatabase, o controle apresenta um painel na interface para cada item/nodo filho de um campo.

Observações:

- O painel dos itens é customizado pelo programador.
- O painel dos itens deve ser um form (ver Tag form) e é chamado de Template Form.
- Você vai precisar de ao menos dois arquivos de extensão ".lfm": Um onde se encontra o recordList, e outro representando o painel item.
- Cada item possui seu próprio objeto nodo, e cada painel de item é automaticamente ligado a seu escopo de dados.

Considere também a utilização da tag gridRecordList, visto que ela possui mais recursos.

Herança

O **recordList** possui **todas** as características da tag layout.

Veja:

- Tag layout
- Características de todas as tags visuais.

Características

Além das características herdadas, o **recordList** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o recordList passa a usar este campo para salvar e apresentar os itens desta lista. Observações: ● Deve ser preenchido para o recordList funcionar. ● O campo informado se tornará um nodo no NodeDatabase Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
templateForm	String	<string vazio>	Identifica qual é o template form do record list, isto é, o painel que será exibido para cada item. Deve ser igual ao atributo "name" de um form contido em algum arquivo ".lfm" do projeto do plugin.

			Sim, você precisará de outro arquivo ".lfm" para definir fazer esta tag funcionar.
itemHeight	Float	64.0	Altura padrão de cada item do recordList. Este valor é usado quando a propriedade "height" do Template Form não for definida.
minQt	Integer	0	Define qual é a quantidade mínima de itens que este recordList deve apresentar na interface. Experimente colocar o valor 1 nesta propriedade. As vezes isto ajuda o usuário =)
autoHeight	Boolean	False	Quando true, o recordList automaticamente se expande/contrai na vertical para melhor visualização de seus itens. Quando false, o recordList apresenta barras de rolagens quando o conteúdo for maior que sua altura.
layout	Enumerado: ● "vertical" ● "horizontal" ● "horizontalTiles" ● "verticalTiles"	"vertical"	Define como são organizados visualmente os itens do recordList. "vertical' - Os itens são empilhados de forma vertical, 1 item por linha. A altura de cada painel é definido pela propriedade "height" do templateForm e a largura é ajustada para equivaler à largura do recordList. "horizontal" - Os Itens são

			empilhados lado a lado, de forma horizontal em apenas 1 linha. A largura de cada painel é definido pelo atributo "width" do templateForm e a altura é ajustada para equivaler à altura do recordList. A propriedade autoHeight é desativada ao usar este layout. "horizontalTiles" - Os itens são dispostos na horizontal (lado a lado), podendo quebrar de linha quando for preciso mais espaço. Tanto a altura quanto a largura de cada painel são definidos pelas propriedades "width" e "height" do templateForm. "verticalTiles" - Os itens são dispostos na vertical, podendo ocupar mais de 1 coluna quando houver espaço para isso. Tanto a altura quanto a largura de cada painel são definidos pelas propriedades "width" e "height" do templateForm.
selectable	Boolean	False	Indica se o recordList deve funcionar visualmente como uma lista de itens selecionáveis.
selectedNode	Objeto Nodo	<< Não há >>	Contém o Objeto Nodo do item atualmente focado/selecionado no recordList. Caso não haja seleção, contém **nil**. Ao atribuir um valor, o recordList procura qual

			de seus itens está associado ao nodo e o seleciona. Caso não encontre, a seleção será desfeita e selectedNode passará a ser **nil**.
selectedForm	Objeto Form	<< Não há >>	(Somente Leitura) Contém o Objeto Form do item atualmente focado/selecionado no recordList. Caso não haja seleção, contém **nil**.

Métodos

Método	Descrição
recordList:append();	Insere um novo item ao recordList. Retorno: ● Caso consiga inserir um novo item, retorna o novo objeto Nodo do item inserido, senão retorna **nil**.
recordList:sort();	Força o recordList a executar uma reordenação de seus itens. Observação: ● Você deve manipular o evento onCompare para definir como os itens serão ordenados.
recordList:scrollToNode(node)	Se a propriedade "autoHeight" for false, este método movimenta as barras de rolagem do recordList para que o painel do nodo fique visível na tela do usuário. Parâmetros: ○ node - um objeto Nodo identificando o nodo do item que deseja tornar visível na tela do usuário.

Eventos

Nome do evento	Descrição
onSelect	Este evento é invocado quando a seleção do recordList muda, isto é, quando um item for selecionado ou quando nenhum item estiver selecionado. Observações: ● Para descobrir qual item foi selecionado, utilize as propriedades selectedNode e selectedForm ● Este evento é disparado mesmo quando a propriedade "selectable" for false.
onBeginEnumeration	Este evento é invocado quando o recordList começar a enumerar as mudanças de itens, disparando, logo a seguir, 0 ou mais eventos onItemAdded e/ou onItemRemoved.
onItemAdded	Evento disparado quando um item for carregado/adicionado visualmente no recordList Parâmetros: ○ node - Objeto Nodo do item adicionado. ○ form - Objeto Form do item adicionado. Observações: ○ Este evento representa um item que foi enumerado visualmente na lista e não necessariamente que um novo item foi inserido nos dados salvos. Durante o carregamento dos dados, este evento é disparado para cada item carregado.
onItemRemoved	Evento disparado quando um item for descarregado/removido visualmente do recordList Parâmetros: ○ node - Objeto Nodo do item removido. ○ form - Objeto Form do item removido. Observações:

	○ Este evento representa um item que deixou de ser enumerado visualmente da lista e não necessariamente que um item foi deletado dos dados salvos.
onEndEnumeration	Este evento é invocado quando o recordList terminar de enumerar as mudanças de itens e não houver outro evento onItemAdded e/ou oItemRemoved para disparar a seguir.
onCompare	Este evento é invocado em momentos chaves para garantir a ordenação dos itens que são mostrados na interface. Quando invocado, o evento traz 2 nodos para que o programador retorne, entre os dois, qual deve aparecer antes. Parâmetros: ○ nodeA - Objeto Nodo contendo um nodo que deve ser comparado. ○ nodeB - Objeto Nodo contendo o outro nodo que deve ser comparado. Retorno: ○ Retorne um número **menor** que 0 se "nodeA" deve ser exibido **antes** de "nodeB" ○ Retorne um número **maior** que 0 se "nodeA" deve ser exibido **depois** de "nodeB" ○ Retorne 0 se os dois são equivalentes na questão da ordem. Observação: Veja o exemplo 2 para entender melhor como este evento funciona.

Veja Tratando eventos do Lua Form

Exemplos

Exemplo 1 - Lista simples de edits

Arquivo "ficha.lfm"

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<form name="frmFichaTeste">
        <button left="20" top="20" height="25" text="Nova Magia"
width="80"
                onClick="self.rclMagias:append();"/>

        <recordList name="rclMagias" field="magias"
templateForm="frmItemDeMagia"
                    left="20" top="60" width="300" autoHeight="true"/>
</form>
```

Arquivo “itemDeMagia.lfm”

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmItemDeMagia" height="30" margins="{top=2,bottom=2}">

        <edit align="client" field="nome" margins="{right=2}"/>


        <button align="right" text="Apagar" width="80"
                onClick="ndb.deleteNode(sheet);"/>
</form>
```

A tag recordList do arquivo "ficha.lfm" utiliza o form do arquivo"itemDeMagia.lfm" quando é definido sua propriedade "templateForm".



Note que existe uma instância de "frmItemDeMagia" para cada item existente.

Observação: No botão apagar foi utilizada a biblioteca "ndb" para apagar um item. Leia mais sobre em:

- NodeDatabase
- Lua Form e NodeDatabase

Detalhes que valem prestar a atenção:

- Na tag **form** do arquivo "itemDeMagia.lfm", a propriedade "height" foi definida. Esta é a altura do painel que se repete para cada item. Provavelmente você precisará definir um valor "height" apropriado para seu caso =).

- A variável "sheet" que foi referenciada no arquivo "itemDeMagia.lfm" representa o objeto nodo do item ligado ao painel e não da ficha.

Exemplo 2 - Adicionando uma ordenação ao Exemplo 1

Manipulando o evento OnCompare parar ordenar de forma crescente pelo atributo "nome" dos itens.

```
.....
.....

<recordList name="rclMagias" field="magias" templateForm="frmItemDeMagia"
                        left="20" top="60" width="300" autoHeight="true">
        <event name="onCompare">
                return utils.compareStringPtBr(nodeA.nome, nodeB.nome);
        </event>
</recordList>

...
...
```

Ordenando pelo <custo, nome>. Isto é, ordenar pelo custo e considerar o nome em caso de empate.

```
<recordList name="rclMagias" field="magias" templateForm="frmItemDeMagia"
                        left="20" top="60" width="300" autoHeight="true">
        <event name="onCompare"><![CDATA[

                if (nodeA.custo or 0) < (nodeB.custo or 0) then
                        return -1;
                elseif (nodeA.custo or 0) > (nodeB.custo or 0) then
                        return 1;
                else
                        return utils.compareStringPtBr(nodeA.nome,
nodeB.nome);
                end;

                ]]>
        </event>
</recordList>
```

Tag rectangle

A tag/componente **rectangle** representa um retângulo na interface o qual você pode personalizar cores, bordas e outros aspectos da forma.

Herança

O **rectangle** possui **todas** as características de um controle shape/forma.

Veja

- Características de tags shapes
- Características de todas as tags visuais.

Características

Além das características herdadas, o **rectangle** possui também as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
corners	Conjunto de: ● "topLeft" ● "topRight" ● "bottomLeft" ● "bottomRight"	"topLeft topRight bottomLeft bottomRight"	Identifica quais quinas do controle serão customizadas pela propriedade "cornerType" Exemplos: ● Apenas as duas quinas superiores: "topLeft topRight" ● Apenas as duas quinas da esquerda: "topLeft bottomLeft" ● Todas as quinas: "topLeft topRight bottomLeft bottomRight" ● Nenhuma quina: ""

cornerType	Enumerado: ● "round" ● "bevel" ● "innerRound" ● "innerLine"	"round"	Especifica o tipo de formato que as quinas deste retângulo terão. Você deve especificar valores das propriedades "xradius" e "yradius" para o formato das quinas serem visíveis. Veja o Exemplo 3 para mais detalhes.
sides	Conjunto de: ● "top" ● "left" ● "bottom" ● "right"	"top left bottom right"	Define quais bordas serão desenhadas na interface. Exemplos: ● Apenas a borda superior: "top" ● Apenas as bordas laterais: "left right" ● Todas as bordas: "top left bottom right" ● Nenhuma borda: ""
xradius	Float	0.0	Define a "largura" do efeito das quinas (propriedade "cornerType"" Veja o Exemplo 3 para mais detalhes.
yradius	Float	0.0	Define a "altura" do efeito das quinas (propriedade "cornerType"" Veja o Exemplo 3 para

			mais detalhes.

Exemplos

Exemplo 1 - Retângulo simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

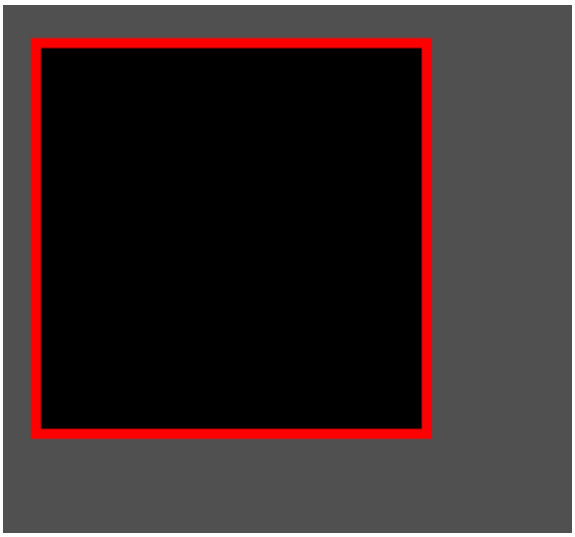
        <rectangle left="20" top="20" width="200" height="200" color="yellow"/>
</form>
```



Exemplo 2 - Retângulo com bordas simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

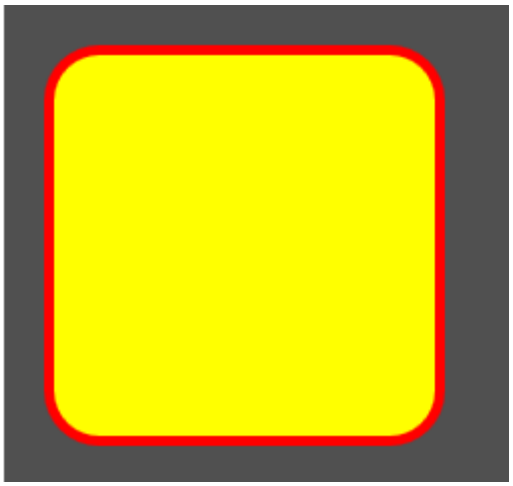
        <rectangle left="20" top="20" width="200" height="200"
                   color="black" strokeColor="red" strokeSize="5"/>
</form>
```



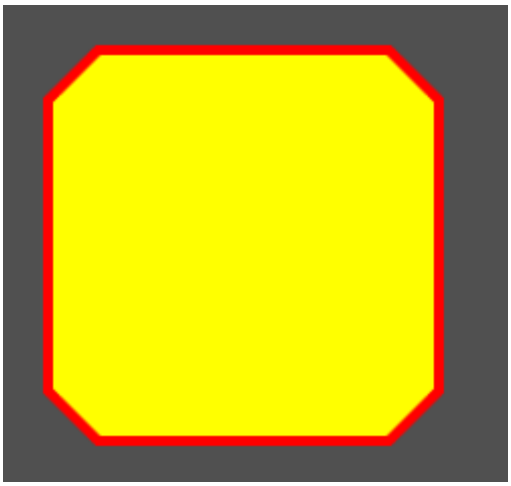
Exemplo 3 - Retângulo com quinas variadas

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

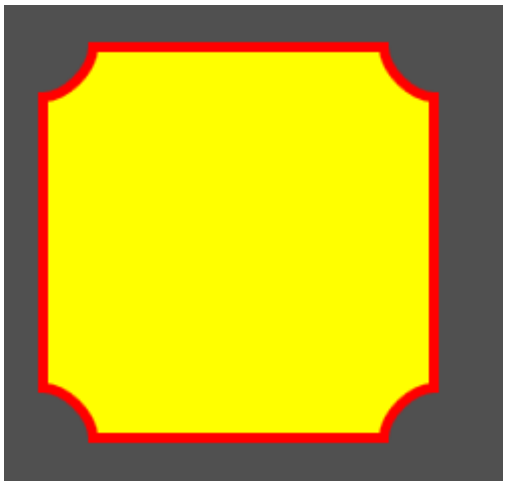
        <rectangle left="20" top="20" width="200" height="200"
                   color="yellow" strokeColor="red" strokeSize="5"
                   xradius="25" yradius="25" cornerType="round"/>
</form>
```



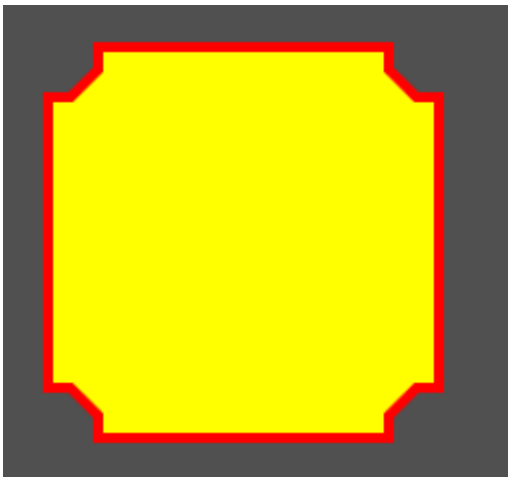
cornerType="round"



cornerType="bevel"



cornerType="innerRound"

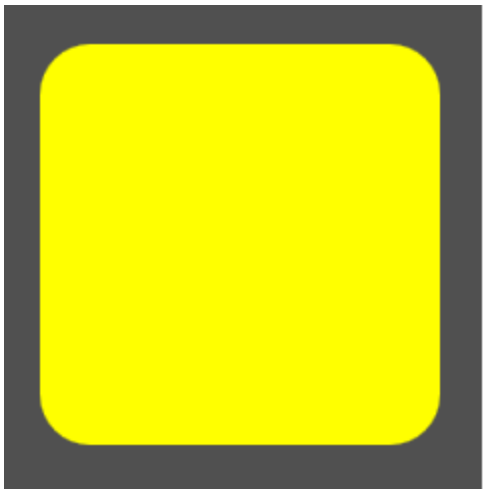


cornerType="innerLine"

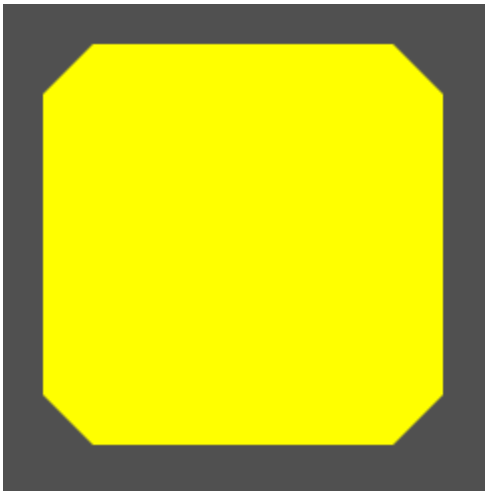
Exemplo 4 - Retângulo com quinas variadas e sem bordas

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

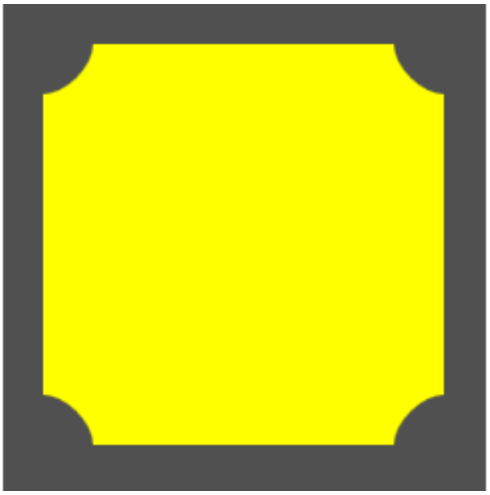
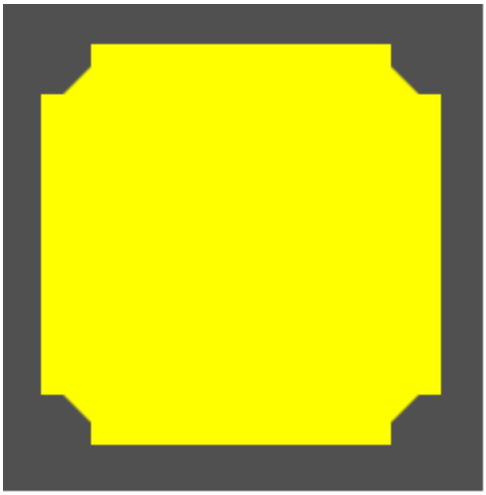
        <rectangle left="20" top="20" width="200" height="200"
                   color="yellow" xradius="25" yradius="25" cornerType="round"/>
</form>
```



cornerType="round"



cornerType="bevel"

 cornerType="innerRound"	 cornerType="innerLine"

Tag richEdit

A tag/componente **richEdit** representa uma caixa de texto de múltiplas linhas com possibilidade de adicionar formatação, cores, alinhamentos, imagens, etc... Um ótimo controle para permitir a entrada de dados "texto ricos".

Herança

O **richEdit** possui todas as características de uma tag visual. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **richEdit** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
animateImages	Boolean	false	Define se o RichEdit deve executar a animação de imagens que são animadas (exemplo: GIF, GIFV) quando suportadas.

readOnly	Boolean	false	Quando true, o richEdit não permite o usuário alterar seu conteúdo
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o richEdit passa a apresentar e salvar o conteúdo no campo informado. Veja também: ● Lua Form e NodeDatabase ● NodeDatabase
backgroundColor	String de Cor	"White"	Define a cor de fundo do RichEdit. Exemplos: ● Black ● White ● #FDFDFD Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso.
defaultFontColor	String de Cor	"Black"	Define a cor padrão para os textos do richEdit Exemplos: ● Black ● White ● #FDFDFD Veja String de cores no Lua Form para obter a completa de cores e outras formas de uso
defaultFontSize	Float	15	Define o tamanho de fonte padrão para os textos do richEdit
showToolbar	Boolean	true	Define se o richEdit deve

			apresentar uma baixa de ferramentas para auxiliar o usuário a formatar seu texto.
hideSelection	Boolean	true	Define se a seleção que o usuário fizer (com o mouse ou teclado) será apresentada quando ele estiver focado em outro controle. True - A seleção continuará existindo, porém não será exibida visualmente se o usuário estiver focado em outro controle False - A seleção continuará existindo e será exibida visualmente mesmo se o usuário estiver focado em outro controle.

Methods

Method	Description
richEdit:beginEdit()	Starts a batch of edit operations. Each call to beginEdit() must be balanced with another call to endEdit()
richEdit:endEdit()	Ends a batch of edit operations. Each call to beginEdit() must be balanced with another call to endEdit()
richEdit:selectAll()	Selects the entire content of the richEdit.
richEdit:unselect()	Cancels all selections made to the richEdit.
richEdit:getLengthInChars()	Returns the length of the richEdit content in UTF-16 char units.
richEdit:getLengthInGlyphs()	Returns the length of the richEdit content in glyph units.

richEdit:getSelectionPosInChars()	Returns two integers containing the current selection start index and selection end index. If both numbers are equal, there is no selection but a single caret at the specified index. The returned indexes are in UTF-16 character units.
richEdit:getSelectionPosInGlyphs()	Returns two integers containing the current selection start index and selection end index. If both numbers are equal, there is no selection but a single caret at the specified index. The returned indexes are in glyph units.
richEdit:getIsSelectionForward()	Returns true if the current selection is forward and false if it is backwards.
richEdit:getSelectionText()	Returns the string content of the current selection.
richEdit:getSelectionURL()	Returns the URL address of the current selection or an empty string if no links are selected.
richEdit:getSelectionFontColor()	Returns a Color String identifying the font color of the selected text.
richEdit:getSelectionFontBkgColor()	Returns a Color String identifying the font background color of the selected text.
richEdit:getSelectionFontSize()	Returns the font size of the selected text.
richEdit:getSelectionFontFamily()	Returns the font family name of the selected text.
richEdit:getSelectionFontStyle()	Returns the font style of the selected text. It is an array that may contain the following strings: "bold", "italic", "underline", "strikeout".

richEdit:getSelectionFloat()	Returns the float mode of the selected text. It can be one of the following values: "none", "left", "right".
richEdit:getSelectionParaAlign()	Returns the paragraph alignment of the selected text. It can be one of the following values: "left", "center", "right", "justify".
richEdit:getText()	Returns the content of the richEdit as plain text.
richEdit:gotoStart([holdSelection])	Moves the caret to the beginning of the content. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoEnd([holdSelection])	Moves the caret to the end of the content. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoNext([holdSelection])	Moves the caret to the next glyph position. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoPrevious([holdSelection])	Moves the caret to the previous glyph position. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.

richEdit:gotoStartOfLine([holdSelection])	Moves the caret to start of the current line. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoEndOfLine([holdSelection])	Moves the caret to end of the current line. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoNextWord([holdSelection])	Moves the caret to the next word. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoPreviousWord([holdSelection])	Moves the caret to the previous word. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoLineUp([holdSelection])	Moves the caret to the line above. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoLineDown([holdSelection])	Moves the caret to the line bellow. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.

richEdit:gotoPageUp([holdSelection])	Moves the caret to the page up. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoPageDown([holdSelection])	Moves the caret to the page down. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoPositionInGlyphs(pos, [holdSelection])	Moves the caret to the specified position measured in glyph count. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:gotoPositionInChars(pos, [holdSelection])	Moves the caret to the specified position measured in UTF-16 char count. Set "holdSelection" to "true" if you want to keep and/or build a selection with the caret movement, simulating the behavior as if the "shift" key were held.
richEdit:copySelectionToClipboard()	Copies the content of the current selection to the clipboard.
richEdit:deleteSelection()	(Edit Operation) Deletes the current selection from the richEdit content.
richEdit:insertText(text)	(Edit Operation) Inserts the specified text into the current caret position, deleting selected content if any, and moving the caret to the end of the inserted text.

richEdit:insertTalemark(talemarkText[, talemarkOptions])	(Edit Operation) Inserts the specified talemark text into the current caret position, deleting selected content if any, and moving the caret to the end of the inserted text. Arguments: ● talemarkText - A string containing the Talemark text to be inserted. ● *(Optional)* talemarkOptions - A TalemarkOptions table parameter to configure the text generation.
richEdit:insertImage(params)	(Edit Operation) Inserts a image into the current caret position, deleting selected content if any, and moving the caret to the end of the inserted image. Arguments: ● params - A lua table containing the following fields: ○ url - A string containing the URL Address of the image ○ width - A number defining the width of the image ○ height - A number defining the height of the image ○ *(Optional)* drawMo

	de - How the image should be rendered. It can be one of the following values: "proportional", "autoFit", "originalSize", "stretch".
richEdit:breakLine()	(Edit Operation) Inserts a line break into the current caret position, deleting selected content if any, and automatically moving the caret to the end of the inserted text.
richEdit:breakParagraph()	(Edit Operation) Inserts a paragraph break into the current caret position, deleting selected content if any, and moving the caret to the end of the inserted text.
richEdit:indent(qty)	(Edit Operation) Changes the indentation of the selected text. The argument "qty" defines how much to indent: - A positive non-zero integer value to indent to the right. - A negative non-zero integer value to indent to the left.
richEdit:pasteFromClipboard()	(Edit Operation) Inserts the content of the clipboard into the current caret position, deleting selected content if any, and moving the caret to the end of the inserted text.
richEdit:cutSelectionToClipboard()	(Edit Operation) Removes the current selection from the richEdit content and places it into the user

	clipboard.
richEdit:setSelectionURL(url)	(Edit Operation) Changes the URL address of the selected text, creating a link if needed. If the argument "url" is empty, then the link is removed.
richEdit:setSelectionFontColor(color)	(Edit Operation) Changes the font color of the selected text to the specified Color String
richEdit:setSelectionFontBkgColor(color)	(Edit Operation) Changes the font background color of the selected text to the specified Color String
richEdit:setSelectionFontSize(size)	(Edit Operation) Changes the font size of the selected text to the specified number.
richEdit:setSelectionFontFamily(family)	(Edit Operation) Changes the font family of the selected text to the specified string.
richEdit:setSelectionFontStyle(style)	(Edit Operation) Changes the font style of the selected text. The argument "style" is an array that may contain the following strings: "bold", "italic", "underline", "strikeout".
richEdit:setSelectionFloat(floatMode)	(Edit Operation) Changes the float mode of the selected text. The argument "floatMode" can be one of the following strings: "none", "left", or "right".
richEdit:setSelectionParaAlign(paraAlign)	(Edit Operation) Changes the paragraph alignment of the selected text. The argument "paraAlign" can be one of the following strings: "left", "center", "right", or "justify".

Eventos

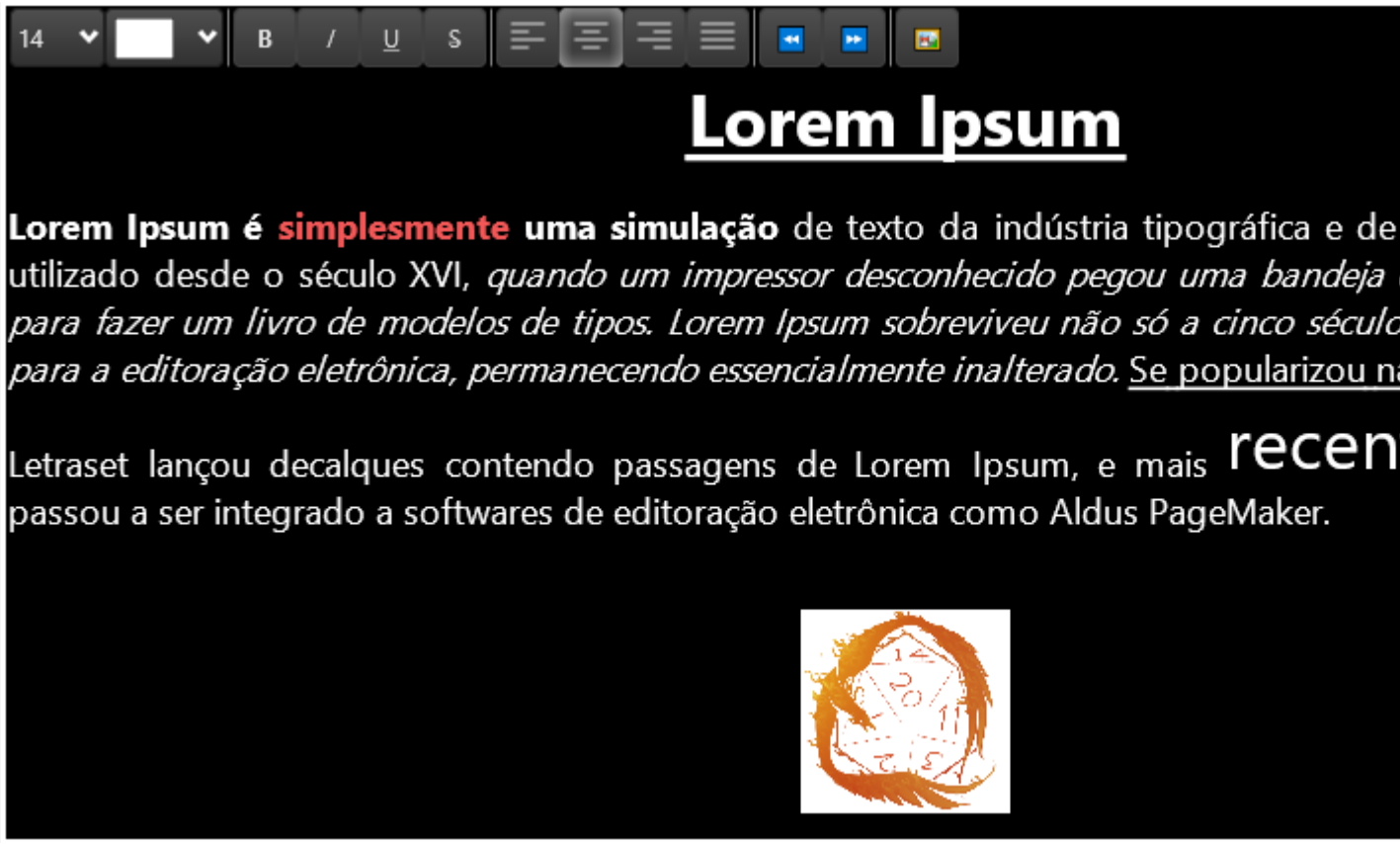
Nome do evento	Descrição

Veja [Tratando eventos do Lua Form]

Exemplos:

Exemplo 1 - Um richEdit simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form ......>
    <richEdit align="client" backgroundColor="black" defaultFontColor="white" field="txt"/>
</form>
```



Tag row

The "row" tag/component represents a container in a [grid-aligned interface]. It is a shorthand for the following equivalent declarations:

```
<layout g="row"/>
<layout g-role="row"/>
```

Inheritage

The "row" tag inherits **all** the characteristics of the "layout" tag.

See:

- layout tag
- Characteristics of all visual tags

Tag script

Esta tag especial serve para definir um bloco de códigos LUA (veja A linguagem de programação LUA) que será executado quando uma nova instância do Lua Form for criada.

Observações:

- O corpo da tag contém código de programação LUA
- Esta tag não cria nenhum controle.
- A ordem que a tag **script** foi definida no documento Lua Form **importa**! O código será executado logo **após** a criação da tag/controle que está acima, porém **antes** da criação da tag/controle que está abaixo. No momento exato em que o script é executado, a variável "sheet" conterá valor **nil** e será preenchido num momento posterior.

Importante: Não deixe de ler as Orientações ao usar código LUA em um Lua Form

Características

Propriedades e atributos

Esta tag não possui propriedades e atributos.

Exemplos

Exemplo 1 - Exibindo um "olá mundo" na hora da criação de uma instância do Lua Form

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<script>
		showMessage("Olá mundo");
	</script>
</form>
```

Ao criar a interface:



Exemplo 2 - Definindo funções para serem invocadas depois

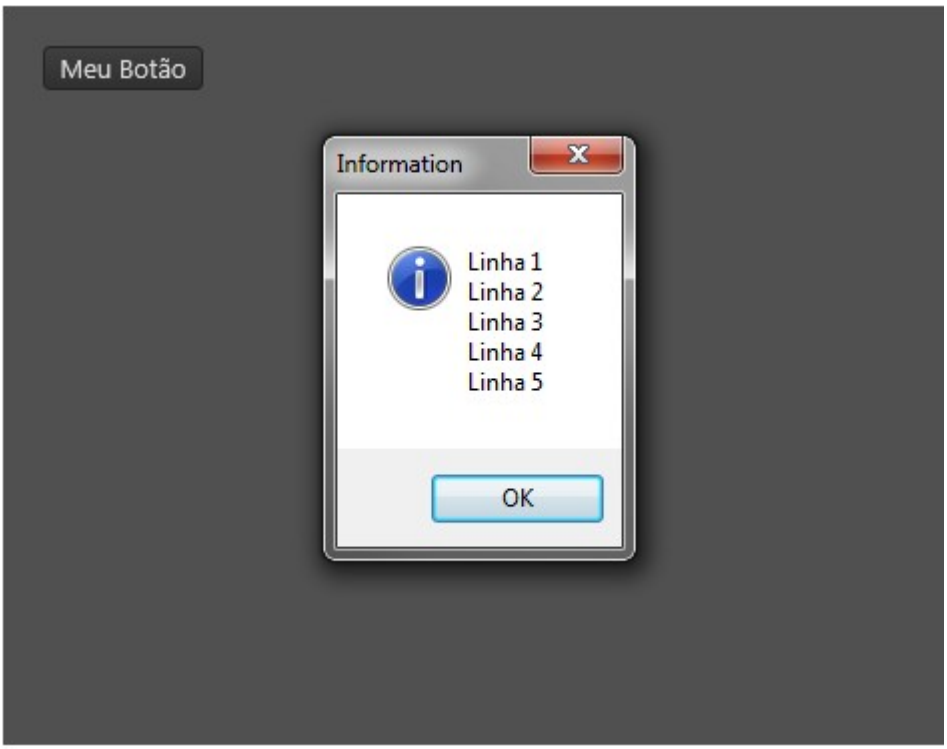
```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<script>
		local function exibirMensagem()
			local msg = "";

			for i = 1, 5, 1 do
				msg = msg .. "Linha " .. i .. "\n";
			end;

			showMessage(msg);
		end;
	</script>

	<button text="Meu Botão" left="20" top="20" onClick="exibirMensagem();"/>
</form>
```



Veja também:

- Tratando eventos do Lua Form

Tag scrollBox

A tag/componente **scrollBox** é um controle que adiciona barras de rolagem ao seu conteúdo, permitindo realizar "scroll" nos componentes que estão dentro da caixa.

Em dispositivos que possuem touch screen, permite também o gesto de "pinça" para efetuar zoom do conteúdo. .

Observação: As barras de rolagem aparecem apenas for necessário, isto é, as dimensões do conteúdo for maior que as dimensões da tag **scrollBox**

Herança

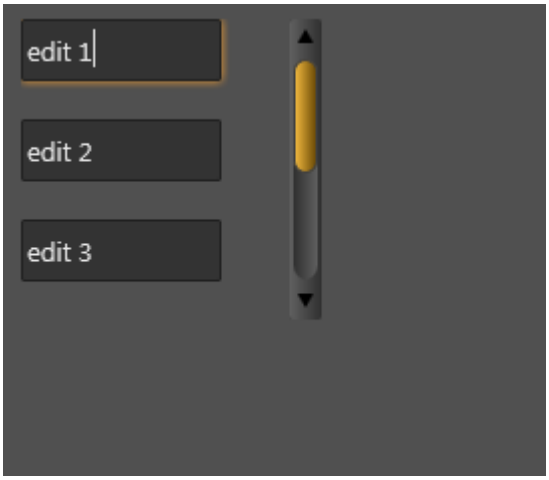
O **scrollBox** possui **todas** as características de um controle qualquer. Veja Características de todas as tags visuais.

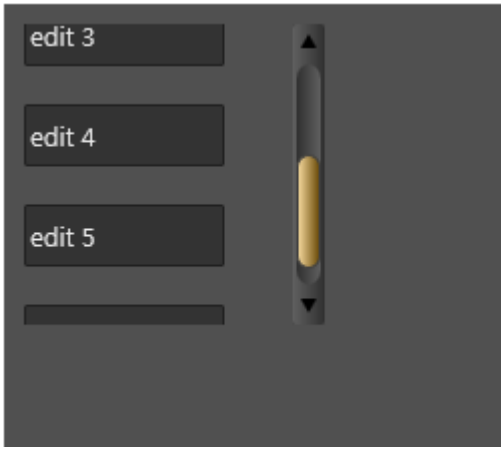
Exemplos

Exemplo 1

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
```

```
        <scrollBox left="10" top="10" width="150" height="150">
                <edit top="0" text="edit 1"/>
                <edit top="50" text="edit 2"/>
                <edit top="100" text="edit 3"/>
                <edit top="150" text="edit 4"/>
                <edit top="200" text="edit 5"/>
                <edit top="250" text="edit 6"/>
        </scrollBox>
</form>
```





Tag style

Esta tag especial serve para definir um estilo em cascata, muito semelhante ao CSS no HTML. Um estilo em cascata é um bloco de códigos especiais que aplica valores à propriedades de 1 ou mais controles de uma vez só, em cascata.

É especialmente útil para evitar retrabalho e para separar o formato do conteúdo principal do Lua Form.

Importante: A sintaxe do estilo em cascata é muito semelhante ao do CSS, porém as propriedades disponíveis são diferentes..

Características

A tag **style** não possui nenhum atributo/propriedade e seu corpo contém o código do estilo em cascata.

Sintaxe

A tag **style** possui uma sintaxe muito parecida com o CSS e seu conteúdo consiste de uma lista de regras. Cada regra ou conjunto de regras consiste de um ou mais seletores e um bloco de declaração. Uma declaração de bloco é composta por uma lista de declarações entre chaves. Cada declaração em si é uma propriedade, dois pontos (:), um valor, então um ponto e vírgula (;).

Exemplo que resume as regras acima:

```
seletor1 [, seletor2, ...] {

  propriedade1: valor;

 [propriedade2: valor2;

  ...]

}
```

Seletores

Um seletor é uma regra genérica que define à quais tags/componentes o estilo será aplicado, e ele pode as seguintes aparências:

Seletor	Exemplo	Descrição
.class	.TituloAtributo	O estilo será aplicado a todas as tags que possuam atributo "class" equivalente a "TituloAtributo". Exemplo de tag que receberia o estilo em cascata: <label text="Olá Mundo" class="TituloAtributo"/>
#nome	#edtNome	O estilo será aplicado à tag cujo atributo "name" seja igual a "edtNome" Exemplo de tag que receberia o estilo em cascata: <edit name="edtNome" text="Campo de text"/>
*	*	O estilo será aplicado a todas as tags.
tagName	label	O estilo é aplicado a todas as tags "tagName", neste exemplo, todos os labels.

		Exemplo de tag que receberia este estilo em cascata: <label text="Olá mundo"/>
tagName.class	layout.PainelAtributo	O estilo é aplicado a todas as tags "layou" cujo atributo "class" seja "PainelAtributo"

Além disso, através da combinação dos seletores acima, é possível definir um seletor composto que analisa onde a tag está. Exemplos:

Seletor	Exemplo	Descrição
.class tagName	.PainelAtributo label	O estilo será aplicado a todos as "label" que estão dentro de alguma tag que contenha o atributo "class" equivalente a "PainelAtributo" Exemplo: <layout class="PainelAtributo"> <label text="Label de dentro"/> </layout> <label text="Label de fora"/> Neste caso, o label de dentro receberá o estilo, porém o label de fora não.
tagName tagName	layout label	O estilo será aplicada a todos "labels" que estão dentro de algum "layout"
.class .class	.PainelAtributo .TituloAtributo	O estilo será aplicado a todas as tags que possuam atributo "class" igual a "TituloAtributo" que estão dentro de alguma tag que possuam "class" igual a "PainelAtributo";
tagName tagName tagName	layout layout label	O estilo será aplicado a todos "labels" que estão dentro de um "layout" que por sua vez estão dentro de outro "layout".
etc....		

É possível definir múltiplos seletores a um único bloco de estilo utilizando vírgula, exemplo:

```
.TituloAtributo, .TituloPericia, .TituloEquipamento {
        fontSize: 18;
        width: 200;
}
```

"fontSize: 18" e "width: 200" serão aplicados a todas as tags cuja propriedade "class" seja "TituloAtributo", "TituloPericia" e "TituloEquipamento".

Propriedades

As propriedades disponíveis são aquelas que a tag disponibiliza. Confira a documentação da TAG.

Importante: O conjunto de propriedades é diferente daquelas que você pode usar em CSS do HTML.

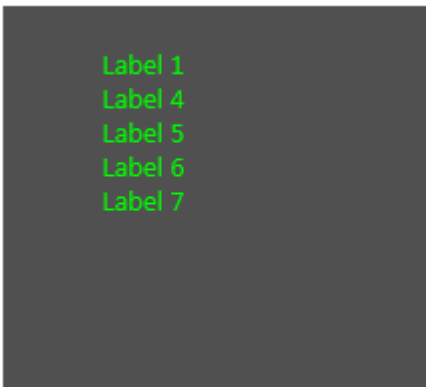
Exemplos

Exemplo 1 - Aplicando uma cor de fonte e alinhamento à todos labels de um lua form.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <style>
                label {
                        fontColor: lime;
                        horzTextAlign: center;
                        vertTextAlign: center;
                }
        </style>

        <layout left="20" top="20" width="100" height="100">
                <label align="top" text="Label 1"/>
                <label align="top" text="Label 4"/>
                <label align="top" text="Label 5"/>
                <label align="top" text="Label 6"/>
                <label align="top" text="Label 7"/>
        </layout>
</form>
```



Exemplo 2 - Aplicando uma cor e negrito à tags de uma determinada class

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <style>
                .Titulo {
                        fontColor: lime;
                        fontStyle: bold;
                }
        </style>

        <layout left="20" top="20" width="100" height="100">
                <label class="Titulo" align="top" text="Label 1"/>
                <label align="top" text="Label 4"/>

                <label class="Titulo" align="top" text="Label 5"/>
                <label align="top" text="Label 6"/>

                <label class="Titulo" align="top" text="Label 7"/>
                <label align="top" text="Label 8"/>
        </layout>
</form>
```



Exemplo 3 - (Avançado) Uma mini ficha, usando edits, labels, layouts e styles.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
	<style>
		layout.LayoutCampo {
			width: 64;
			align: left;
			margins: {left=2, right=2};
		}

		layout.LayoutCampo edit {
			height:30;
			horzTextAlign: center;
			fontSize: 20;
			align: top;
		}

		layout.LayoutCampo label {
			align: top;
			horzTextAlign: center;
			vertTextAlign: leading;
			autoSize: true;

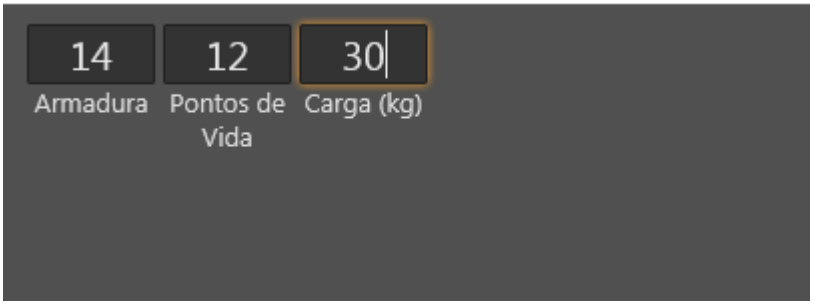
		}
	</style>

	<layout left="10" top="10" width="200" height="64">
		<layout class="LayoutCampo">
			<edit field="combate.armadura"/>
			<label text="Armadura"/>
```

```
                </layout>

                <layout class="LayoutCampo">
                        <edit field="saude.pontosDeVida"/>
                        <label text="Pontos de Vida"/>
                </layout>

                <layout class="LayoutCampo">
                        <edit field="equipamentos.carga"/>
                        <label text="Carga (kg)"/>
                </layout>
        </layout>
</form>
```



Neste exemplo, os atributos dos edits, labels e layouts foram definidos por Estilo em Cascata pela tag **style**.

Foram definidos 3 blocos de estilos em cascata:

- Um com seletor "layout.LayoutCampo" (aplicado a todas as tags **layout** de class "**LayoutCampo**")
- Um com seletor "`layout.LayoutCampo edit`" (aplicado a todas as tags **`edit`** que estão dentro de uma tag **layout** de class "**LayoutCampo**")
- Um com seletor "`layout.LayoutCampo label`" (aplicado a todas as tags **`label`** que estão dentro de uma tag **layout** de class "**LayoutCampo**")

Tag tab

A tag/componente **tab** representa uma aba dentro de um grupo de abas (Ver Tag tabControl). Excelente para dividir controles em duas ou mais abas.

IMPORTANTE: Para usar abas, você **deve** usar uma Tag tabControl.

Herança

O **tab** possui todas as características de uma tag visual. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **tab** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
text ou title	string	<string vazio>	Representa o título da aba.

Métodos

É possível invocar métodos dos controles usando código LUA.

Método	Descrição
tab:activate()	Ativa a aba no tabControl.

Exemplos

Exemplo 1 - Uma aba simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <tabControl left="20" top="20">
                <tab title="Minha aba">
                </tab>
        </tabControl>
</form>
```

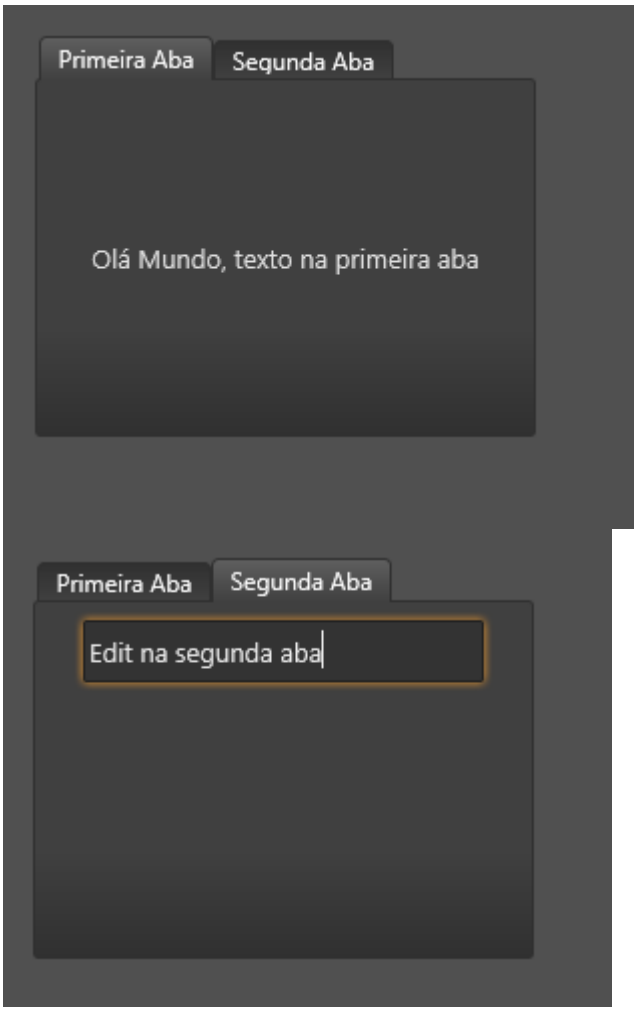


Exemplo 2 - Duas abas com conteúdos distintos

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<tabControl left="20" top="20" width="250">
		<tab title="Primeira Aba">
			<label text="Olá Mundo, texto na primeira aba" align="client"
				vertTextAlign="center" horzTextAlign="center"/>
		</tab>

		<tab title="Segunda Aba">
			<edit text="Edit na segunda aba" width="200" left="25" top="10"/>
		</tab>
	</tabControl>
</form>
```



Observação: Foram usadas neste exemplo a Tag label e a Tag edit

Tag tabControl

A tag/componente **tabControl** representa um controle que agrupa tabs/abas (Ver Tag tab). Excelente para dividir controles em duas ou mais abas.

Herança

O **tabControl** possui todas as características de uma tag visual. Veja:

- Características de todas as tags visuais

Características

Além das características herdadas, a tag **tabControl** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
tabIndex	Integer	0	Indica qual o índice da aba ativa no momento. Este índice começa do 0 e vai até (Quantidade_de_Abas - 1)

Exemplos

Veja exemplos no tópico Tag tab

Tag template

Com esta tag, o programador pode criar seu próprio conjunto de tags do Lua Form que sempre que forem usada, serão substituídas pelo conteúdo do **template**.

Observação: Esta tag **não** representa um componente do lua form e todas as tags que estão dentro do **template** só existirão de fato quando o modelo for invocado.

Características

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
name	String	<Não há valor padrão>	Nome do template. É através deste campo que o programador consegue invocar o modelo posteriormente, usando uma tag de mesmo nome. O preenchimento deste campo é **obrigatório.**

Definindo e invocando templates

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <template name="MinhaTag">
                <label text="Label da Minha Tag" align="top" horzTextAlign="center"/>
```

```
        </template>

</form>
```

Neste exemplo, usando **template,** nós definimos um modelo chamado MinhaTag....

Mas só a definição do template não acarreta em nada! Veja como o exemplo acima fica:



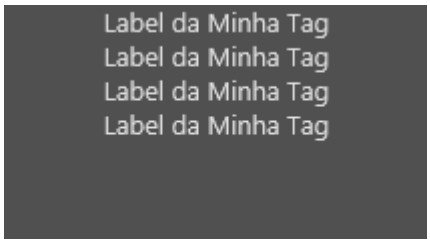
É preciso invocar o modelo....

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <template name="MinhaTag">
                <label text="Label da Minha Tag" align="top" horzTextAlign="center"/>
        </template>

        <MinhaTag/>
        <MinhaTag/>
        <MinhaTag/>
        <MinhaTag/>
</form>
```

Agora invocamos "MinhaTag" 4 vezes... Toda vez que usamos a tag "MinhaTag" no exemplo, ela foi substituída por uma cópia do conteúdo de **template.**

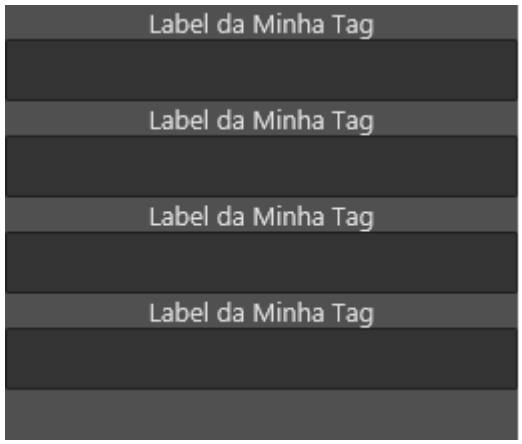


Se adicionarmos um edit ao **template**, o resultado mudaria para:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <template name="MinhaTag">
                <label text="Label da Minha Tag" align="top" horzTextAlign="center"/>
                <edit align="top"/>
        </template>

        <MinhaTag/>
        <MinhaTag/>
        <MinhaTag/>
        <MinhaTag/>
</form>
```



Passando parâmetros para o template

É possível passar parâmetros para o template na hora da substituição! Para passar um parâmetro, basta definir um atributo ao invocar o modelo e para usar o parâmetro no modelo, o programador deve usar **$(nomeDoAtributo)** para a substituição parametrizada.

Exemplo:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<template name="MinhaTag">
		<label text="Atributo $(titulo)" align="top" horzTextAlign="center"/>
		<edit text="$(conteudo)" align="top" horzTextAlign="center"/>
	</template>

	<MinhaTag titulo="Força" conteudo="10"/>
	<MinhaTag titulo="Destreza" conteudo="14"/>
	<MinhaTag titulo="Inteligência" conteudo="17"/>
</form>
```



Observação: Quando a tag template é invocada com um corpo texto (Exemplo: <minhaTag>Corpo Texto</minhaTag>), um parâmetro chamado "body" é criado com este conteúdo.

Templates avançados usando LUA na hora da substituição.

É possível criar templates avançados usando A linguagem de programação LUA na hora da substituição.

Todo comentário de XML dentro de templates são tratados como códigos lua que devem ser executado no momento da substituição!

Exemplo:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<template name="Municao">
		<layout width="$(largura)" height="20" align="top">
			<label text="$(titulo) :" align="left" horzTextAlign="center" width="60"/
```

```
                        <!-- for  i = 1, quantidade, 1 do -->
                        <checkBox align="left" width="20"/>
                        <!-- end; -->
                </layout>
        </template>

        <Municao largura="300" titulo="Flechas" quantidade="20"/>
        <Municao largura="300" titulo="Bombas" quantidade="4"/>
        <Municao largura="300" titulo="Dardos" quantidade="10"/>
</form>
```



Neste exemplo, o parâmetro "quantidade" é utilizado em um código LUA para emitir várias vezes a tag <checkBox> durante o tempo de substituição. Vários checkBox foram emitidos com pouco esforço.

Exemplos

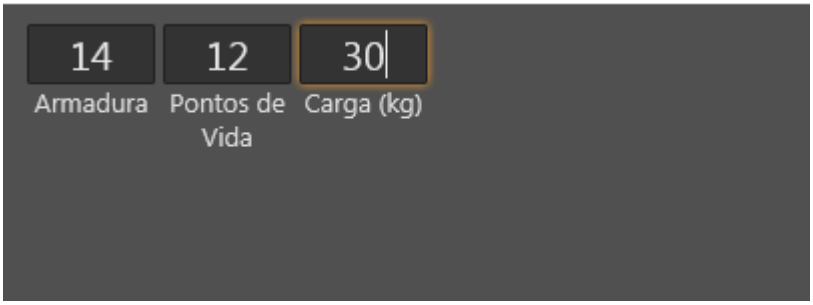
Exemplo 1 - Uma pequena ficha usando templates

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <template name="LayoutUmCampo">
                <layout width="64" align="left" margins="{left=2, right=2}">
                        <edit field="$(campo)" height="30" horzTextAlign="center" fontSize="20"
align="top"/>
                        <label text="$(titulo)" align="top" horzTextAlign="center"
                               vertTextAlign="leading" autoSize="true"/>
                </layout>
        </template>

        <layout left="10" top="10" width="200" height="64">
                <LayoutUmCampo campo="combate.armadura" titulo="Armadura"/>
                <LayoutUmCampo campo="saude.pontosDeVida" titulo="Pontos de Vida"/>
```

```
                <LayoutUmCampo campo="equipamentos.carga" titulo="Carga (kg)"/>
        </layout>
</form>
```



Exemplo 2 - Lista de perícias de D&D 3.5 usando templates

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

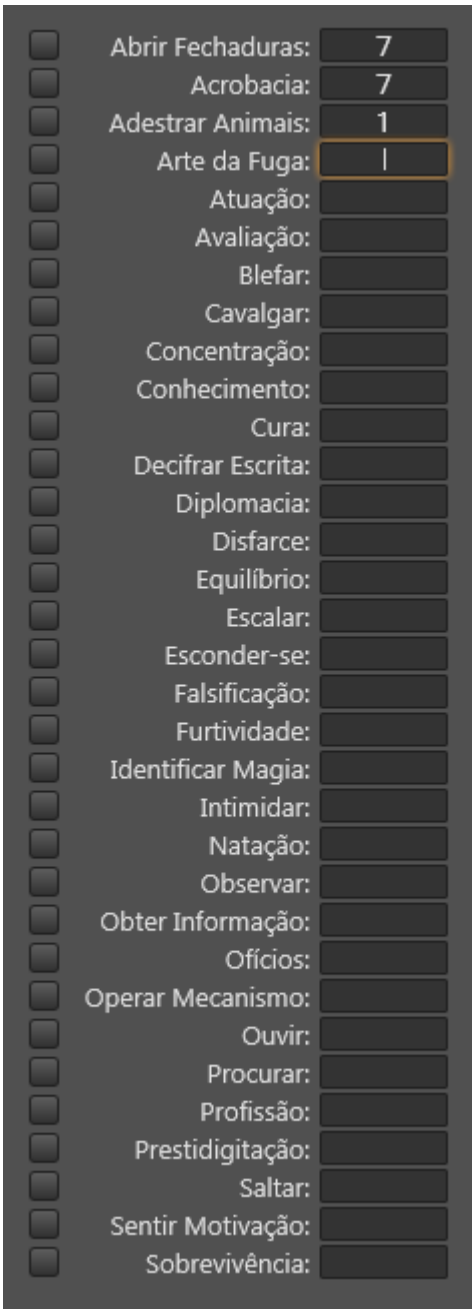
        <template name="CampoEmUmLayoutDeCampos">
                <layout align="top" height="17" margins="{bottom=2}">
                        <checkBox width="17" align="left" margins="{right=5}"/>
                        <label align="left" width="120" text="$(titulo):"
                               horzTextAlign="trailing" margins="{right=5}"/>
                        <edit align="left" horzTextAlign="center" fontSize="16" width="64"/>
                </layout>
        </template>

        <template name="LayoutDeCampos">
                <layout left="20" top="20" width="250" height="800">
                        <!--
                                listaDeCampos = totable(body);

                                for k, v in pairs(listaDeCampos) do
                                        CampoParaGerar = v;
                        -->
                                        <CampoEmUmLayoutDeCampos titulo="$(CampoParaGerar)"/>
                        <!--
                            end;
                        -->
                </layout>
```

```
        </template>

        <LayoutDeCampos>
                {"Abrir Fechaduras", "Acrobacia", "Adestrar Animais",
                 "Arte da Fuga", "Atuação", "Avaliação", "Blefar", "Cavalgar",
                 "Concentração", "Conhecimento", "Cura", "Decifrar Escrita",
                 "Diplomacia", "Disfarce", "Equilíbrio", "Escalar", "Esconder-se",
                 "Falsificação", "Furtividade", "Identificar Magia", "Intimidar", "Natação",
                 "Observar", "Obter Informação", "Ofícios", "Operar Mecanismo",
                 "Ouvir", "Procurar", "Profissão", "Prestidigitação", "Saltar",
                 "Sentir Motivação", "Sobrevivência"}
        </LayoutDeCampos>
</form>
```



Tag textEditor

A tag/componente **textEditor** representa uma caixa de texto de múltiplas linhas. Um ótimo controle para permitir a entrada de dados texto com quebra de linhas.

Herança

O **textEditor** possui todas as características de uma tag de texto. Veja:

- Características de todas as tags de textos
- Características de todas as tags visuais

Características

Além das características herdadas, a tag **textEditor** também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
readOnly	Boolean	false	Quando true, o textEditor não permite o usuário alterar seu conteúdo (somente leitura).
field	String	<string vazio>	Caminho de um campo no NodeDatabase. Quando associado, o textEditor passa a apresentar e salvar o conteúdo no campo informado. Veja também: • Lua Form e NodeDatabase • NodeDatabase
transparent	Boolean	false	Quando True, as bordas e o fundo do textEditor não serão exibidas, mostrando apenas o texto conteúdo e o cursor de digitação.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando o conteúdo do textEditor é alterado.

Veja Tratando eventos do Lua Form

Exemplos:

Exemplo 1 - Um textEditor simples

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <textEditor left="10" top="10" width="150" height="250" text="Este é um textEditor\n\ncom
multiplas linhas"/>
</form>
```



Tag timer

A tag **timer** representa um componente não visual (não é exibido na interface) que executa determinado código LUA de tempo em tempo.

Características

Propriedades e atributos

Propriedade	Tipo	Valor Padrão	Descrição
name	String	<string vazio>	Define um nome para a componente. O nome deve ser único, isto é, dentro

			de um **form**, não é possível existir 2 controles com o mesmo nome. Se não for definido um nome para o controle no arquivo LFM, um nome único será gerado para ele em tempo de compilação. Nomear controles é especialmente útil quando se quer trabalhar com códigos LUA.
interval	Integer	1000	Intervalo, medido em milisegundos, de quando em quando o evento onTimer será disparado. 1000 = 1 segundo 500 = 0,5s 10000 = 10 segundos
enabled	Boolean	true	Quando true, o evento onTimer será disparado de acordo com o atributo interval. Quando false, o evento onTimer não será disparado.

Eventos

Nome do evento	Descrição
onTimer	Este evento é invocado de tempos em tempos, de acordo com o valor da propriedade "interval".

Veja Tratando eventos do Lua Form

Exemplos

Exemplo 1 - Um label "piscando"

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <timer interval="500" onTimer="self.labMeuLabel.visible = not
self.labMeuLabel.visible;"/>

        <label name="labMeuLabel" align="client" horzTextAlign="center"
text="Olá!"/>
</form>
```



Veja Tratando eventos do Lua Form

Frames

Algumas tags/controles do Lua Form permitem o programador definir um frame, isto é, uma moldura para ser exibida na interface visual.

Um frame é uma aparência personalizada que se adapta dinamicamente ao tamanho do controle.

Ainda não entendi o que é um frame.

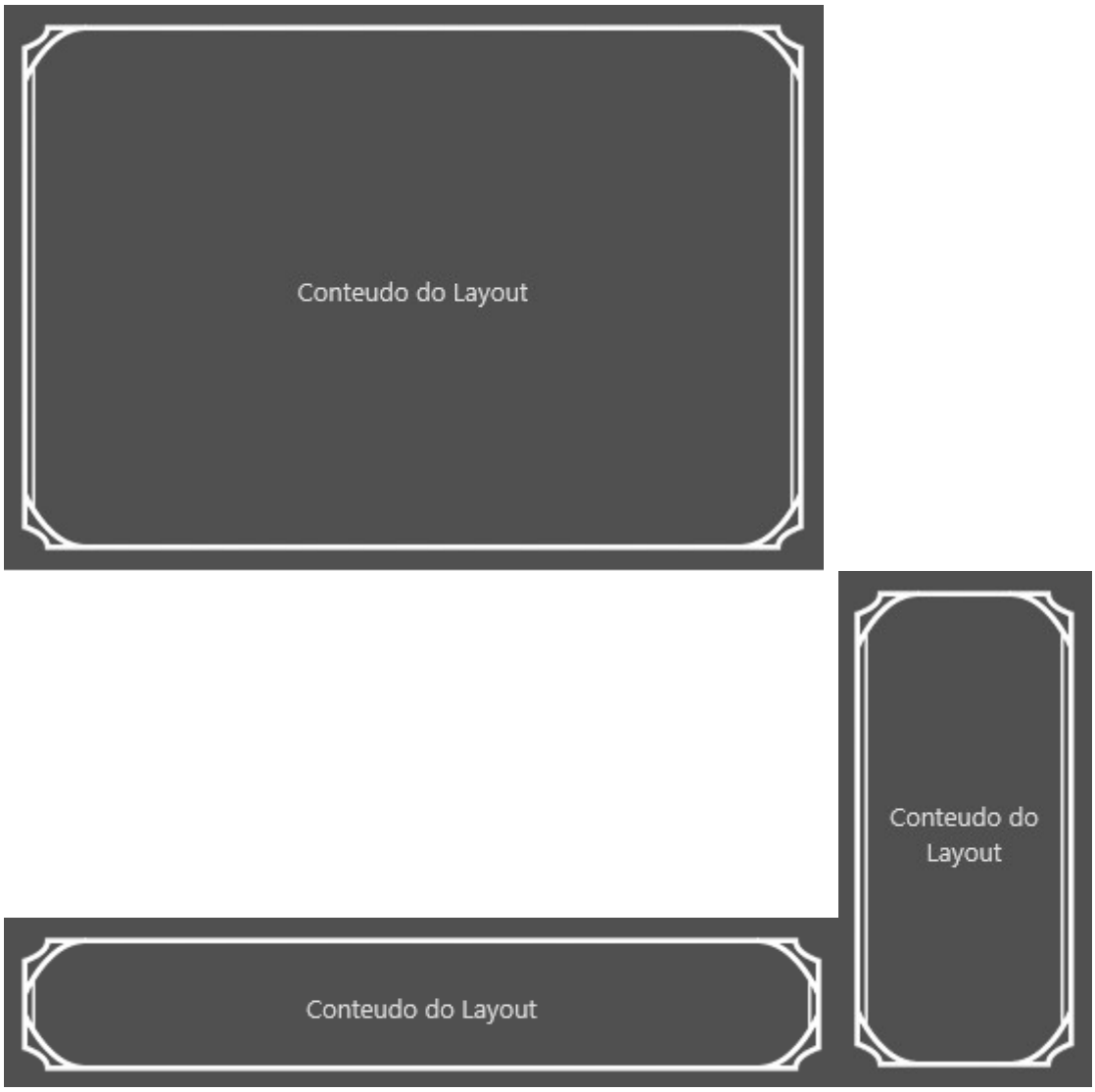
Veja estes exemplos para entender melhor!

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
```

```
        <layout align="client" frameStyle="/frames/frame1.xml"
                margins="{left=10, top=10, right=10, bottom=10}">

                <label align="client" horzTextAlign="center"
text="Conteudo do Layout"/>

        </layout>
</form>
```



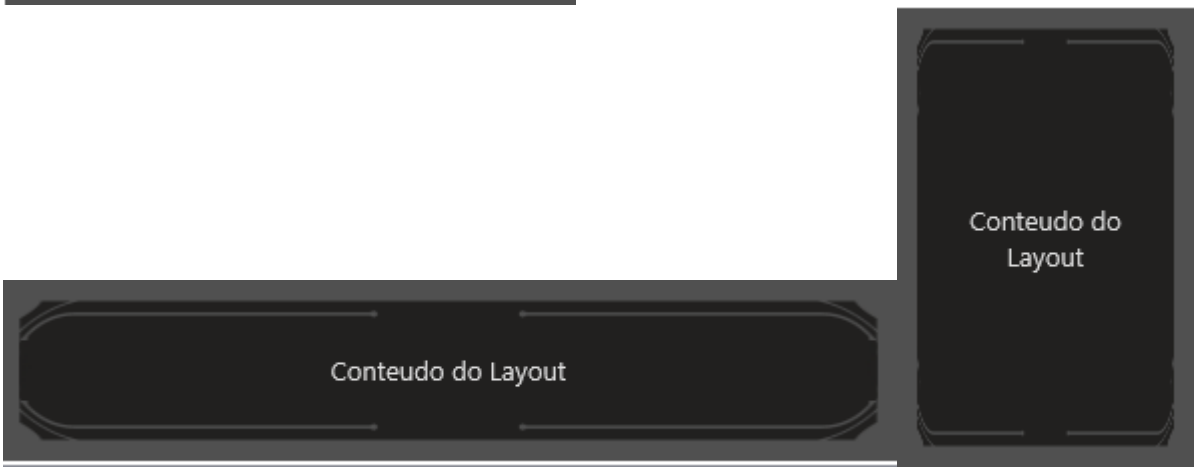

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">
```

```
        <layout align="client" frameStyle="/frames/meuOutroFrame.xml"
                margins="{left=10, top=10, right=10, bottom=10}">

                <label align="client" horzTextAlign="center"
text="Conteudo do Layout"/>

        </layout>
</form>
```





A tag layout por si só não apresenta nada na interface, mas ao aplica um frame à ela, passou a apresentar estas bordas e backgrounds "legais".

O que eu preencho na propriedade "frameStyle"?

Você deve preencher com o caminho de um arquivo XML que contém a definição de um frame. Saiba mais sobre este arquivo lendo Arquivo de Definição de Frame.

Exemplos

Veja alguns exemplos em Exemplos de frames.

Arquivo de Definição de Frame

Todo frame é definido por um documento XML que segue o padrão descrito neste tópico.

Importante: Não é possível utilizar as tags do Lua Form na definição de frames. O documento de definição de frames possui seu próprio conjunto de tags e atributos.

Estrutura do documento XML de definição de frame

Todo documento de XML de definição de frame deve ter a tag frame como tag raiz.

Tag frame

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
width	Float	<Não há valor padrão e deve ser preenchido >	A largura em que o frame foi definido. Todo o restante do frame será interpretado usando esta largura como base.
height	Float	<Não há valor padrão e deve ser preenchido >	A altura em que o frame foi definido. Todo o restante do frame será interpretado usando esta altura como base.
autoScaleX	Boolean	false	Indica como o frame deve se comportar ao ser apresentado em um tamanho diferente do que foi definido. true - O conteúdo do frame será esticado

			ou comprimido no eixo X para equivaler à nova largura. false - O conteúdo do frame será reposicionado no eixo X, respeitando o ancoramento de cada item do conteúdo do frame.
autoScaleY	Boolean	false	Indica como o frame deve se comportar ao ser apresentado em um tamanho diferente do que foi definido. true - O conteúdo do frame será esticado ou comprimido no eixo Y para equivaler à nova altura. false - O conteúdo do frame será reposicionado no eixo Y, respeitando o ancoramento de cada item do conteúdo do frame.

Tags filhas:

Tag filha	Quantidade	Descrição
borders	0 ou 1. (opcional)	Descreve o tamanho das bordas que o frame possui. Leia mais em Tag borders
draw	0 ou 1. (opcional)	Define os itens que serão desenhados no frame. Leia mais em Tag draw. Estes itens são desenhados embaixo dos controles Lua Form de onde o frame for aplicado.

postdraw	0 o ou 1. (opcional)	Define os itens que serão desenhados após o conteúdo do frame ser desenhado. Leia mais em Tag postdraw. Estes itens são desenhados em cima dos controles Lua Form de onde o frame for aplicado.
regions	0 ou 1. (opcional)	Define as regiões que este frame possui. Usado para alinhar controles Lua Form no lugar correto do frame. Leia mais em Tag regions.

Tag borders

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
left	Float	0.0	Define o tamanho da borda esquerda do frame.
top	Float	0.0	Define o tamanho da borda superior do frame.
right	Float	0.0	Define o tamanho da borda da direita do frame.
bottom	Float	0.0	Define o tamanho da borda inferior do frame.

Tag draw

Esta tag contém todos os elementos que são desenhados no frame.

Atributos: Esta tag não possui atributos

Tags filhas:

Tag filha	Quantidade	Descrição
image	0 ou várias vezes. (lista)	Cada tag image representa uma imagem que será desenhada. Leia mais em Tag image.

Tag postdraw

Esta tag contém todos os elementos que são desenhados no frame.

Atributos: Esta tag não possui atributos

Tags filhas:

Tag filha	Quantidade	Descrição
image	0 ou várias vezes. (lista)	Cada tag image representa uma imagem que será desenhada acima do controles Lua Form. Leia mais em Tag image.

Tag image

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
left	Float	<Não há, preenchimento obrigatório>	Define onde, no eixo X, a imagem será desenhada.
top	Float	<Não há, preenchimento obrigatório>	Define onde, no eixo Y, a imagem será desenhada.

right	Float	<Não há, preenchimento obrigatório>	Define onde, no eixo X, a borda direita da imagem será desenhada. A largura da imagem será equivalente a ("right" - "left")
bottom	Float	<Não há, preenchimento obrigatório>	Define onde, no eixo Y, a borda inferior da imagem será desenhada. A altura da imagem será equivalente a ("bottom" - "top")
overflowX	Enumerado: ● "none" ● "tile" ● "stretch" ● "center"	"tile"	Define como a imagem será desenhada no eixo X quando a largura a desenhar for diferente da largura definida. "none": A imagem será desenhada apenas uma vez, a cortando se o tamanho diminuir. "tile": A imagem será desenhada várias vezes para que ocupe o novo tamanho. "stretch": A imagem será encolhida ou espichada para o novo tamanho. "center": A imagem será desenhada apenas uma vez no centro.

overflowY	BoEnumerado: ● "none" ● "tile" ● "stretch" ● "center"	"tile"	Idem ao atributo "overflowX", porém agora analisando a altura e o eixo Y da imagem.
zOrder	Float	0.0	Define a ordem em que esta imagem será desenhada. A image que tiver o menor zOrder será desenhada embaixo das outras, enquanto a que tiver maior zOrder será desenhada acima.

Tags filhas:

Tag filha	Quantidade	Descrição
source	1 vez (preenchimento obrigatório)	A tag source contém informações sobre qual imagem será desenhada. Leia mais em tag source. Observação: Enquanto a tag image possui informações relacionadas ao "onde a imagem será desenhada", a tag source possui informações de qual imagem será desenhada.
anchors	0 ou 1 (opcional).	Define como a imagem será ancorada quando o frame for desenhado com um tamanho maior ou menor do que o definido. Leia mais em tag anchors A ausência desta tag implica no ancoramento

		"left e top".

Tag source

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
url	String	<Não há, preenchimento obrigatório>	Caminho do arquivo da imagem que será desenhada na tela. Contém um caminho relativo à pasta em que se encontra o arquivo ".xml" de definição deste frame.
left	Float	<Não há, preenchimento obrigatório>	Este atributo é usado para definir qual parte da imagem, informada pelo atributo "url", será desenhada na tela. É possível desenhar a imagem inteira, ou apenas um pedaço/retângulo dela (o que é bastante útil). Medido em pixels, informa a posição **esquerda** do pedaço/retângulo que será desenhado na tela.
top	Float	<Não há, preenchimento obrigatório>	Medido em pixels, informa a posição **superior** do pedaço/retângulo que será desenhado na tela.

			Veja a descrição do atributo "left".
right	Float	<Não há, preenchimento obrigatório>	Medido em pixels, informa a posição **da direita** do pedaço/retângulo que será desenhado na tela. Veja a descrição do atributo "left".
bottom	Float	<Não há, preenchimento obrigatório>	Medido em pixels, informa a posição **inferior** do pedaço/retângulo que será desenhado na tela. Veja a descrição do atributo "left".

Tags filhas:

Não há tags filhas.

Tag anchors

Define o ancoramento do componente.
Quando o frame for desenhado com um tamanho diferente do desenhado, o componente mantém sua posição relativa baseada em sua âncora.

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
left	Boolean	false	Quando true, define um âncora à esquerda. Leia as observações abaixo desta tabela.
top	Boolean	false	Quando true, define um âncora acima. Leia as observações abaixo desta tabela.

right	Boolean	false	Quando true, define um âncora à direita. Leia as observações abaixo desta tabela.
bottom	Boolean	false	Quando true, define um âncora abaixo. Leia as observações abaixo desta tabela.

Comportamento de âncora:

- "left" = true, "right" = false: O componente manterá sempre a mesma posição left/X.
- "left" = false, "right" = true: A posição left será calculada automaticamente para manter sempre a mesma distância entre sua borda direita e a borda direita do pai/frame.
- "left" e "right" ambos = true: O controle manterá tanto a distância entre (1) sua borda esquerda com a borda esquerda do pai/frame; (2) como sua borda direita com a borda direita do pai/frame. Para isso, a largura do componente será alterada conforme o tamanho de visualização do frame.
- "left" e "right" ambos = false: O controle não está ancorado e ocorrerá um efeito similar ao de centralização.

O comportamento das âncoras "top" e "bottom" é o mesmo das âncoras "left" e "right", porém agora no eixo Y.

Tags filhas:

Não há tags filhas.

Tag regions

Esta tag contém uma lista de tags region, que define regiões no frame que poderão ser usadas para alinhar tags/controles Lua Form posteriormente. Saiba mais em Frames e Regiões.

Atributos: Esta tag não possui atributos

Tags filhas:

Tag filha	Quantidade	Descrição
region	0 ou várias vezes. (lista)	Cada tag image representa uma região do frame Leia mais em Tag region.

Tag region

Atributos:

Propriedade	Tipo	Valor Padrão	Descrição
name	String	<Não há, preenchimento obrigatório>	Define um nome para esta região para que possa ser usada posteriormente por um controle Lua Form. Saiba mais em Frames e Regiões.
left	Float	<Não há, preenchimento obrigatório>	Define, onde no eixo X, é a posição esquerda desta região no frame.
top	Float	<Não há, preenchimento obrigatório>	Define, onde no eixo Y, é a posição superior desta região no frame.
right	Float	<Não há, preenchimento obrigatório>	Define, onde no eixo X, é a posição da direita desta região no frame. A largura da região será equivalente a ("right" - "left")
bottom	Float	<Não há, preenchimento obrigatório>	Define, onde no eixo Y, é a posição inferior desta região no frame. A altura da região será equivalente a ("bottom" - "top")
autoScaleX	Boolean	false	Indica como as sub-regiões desta região devem se comportar ao ser apresentadas em um tamanho de frame diferente do

			que foi definido. true - As sub-regiões serão esticadas ou comprimidas no eixo X para equivaler à nova largura. false - As sub-regiões serão reposicionadas no eixo X, respeitando seus ancoramentos.
autoScaleY	Boolean	false	Indica como as sub-regiões desta região devem se comportar ao ser apresentadas em um tamanho de frame diferente do que foi definido. true - As sub-regiões serão esticadas ou comprimidas no eixo Y para equivaler à nova altura. false - As sub-regiões serão reposicionadas no eixo Y, respeitando seus ancoramentos

Tags filhas:

Tag filha	Quantidade	Descrição
anchors	0 ou 1 (opcional).	Define como a região será ancorada quando o frame for desenhado com um tamanho maior ou menor do que o definido. Leia mais em tag anchors A ausência desta tag implica no ancoramento "left e top".

regions	0 ou 1 (opcional).	Define as sub-regiões desta região. Leia mais em Tag regions.

Exemplos

Veja os exemplos em Exemplos de frames.

Frames e Regiões

Os frames podem possuir regiões em suas definições que são acessíveis à tags/componentes do LuaForm. Ao associar uma região de frame à algum componente do Lua Form, o controle automaticamente terá sua posição e tamanho ajustados para que ocupe a região escolhida do frame.

Veja o exemplo de frame com regiões.

Exemplos de frames

1 - Background que estica para o novo tamanho

2 - Background que se repete para preencher o tamanho

3 - Frame com 2 partes fixas e 1 parte que se estica

4 - Frame retangular dinâmico, usando vários arquivos de imagens

5 - Frame com regiões

Os exemplos acima são arquivos de definições de frame utilizados em tags layout da seguinte forma em um Lua Form:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <layout align="client" frameStyle="/frames/frame1.xml"
                margins="{left=10, top=10, right=10, bottom=10}">

                <label align="client" horzTextAlign="center"
text="Conteudo do Layout"/>
```

```
        </layout>
</form>
```

1 - Background que estica para o novo tamanho

Arquivo de imagem: background1.png (tamanho 256x223)

```
<?xml version="1.0" encoding="UTF-8"?>
<frame width="256" height="223">
        <draw>
                <!-- Imagem de fundo -->

                <image left="0" top="0" right="256" bottom="223"
overflowX="stretch" overflowY="stretch">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                        <source url="background1.png" left="0" right="256"
top="0" bottom="223"/>
                </image>
        </draw>
</frame>
```


Conteudo do Layout


Conteudo do Layout


Conteudo
do Layout



2 - Background que se repete para preencher o tamanho

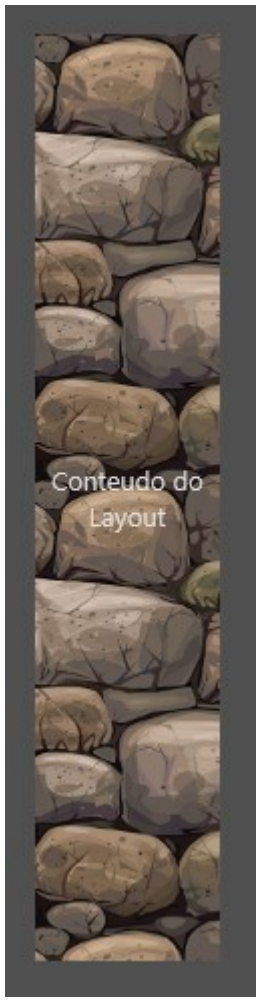
Arquivo de imagem: background1.png (tamanho 256x223)

```
<?xml version="1.0" encoding="UTF-8"?>
<frame width="256" height="223">
        <draw>
                <!-- Imagem de fundo -->

                <image left="0" top="0" right="256" bottom="223"
overflowX="tile" overflowY="tile">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                        <source url="background1.png" left="0" right="256"
```

```
top="0" bottom="223"/>
                </image>
        </draw>
</frame>
```




Conteudo do
Layout



3 - Frame com 2 partes fixas e 1 parte que se estica

Dada a seguinte imagem (header0_smaller.png) de tamanho 468x100:



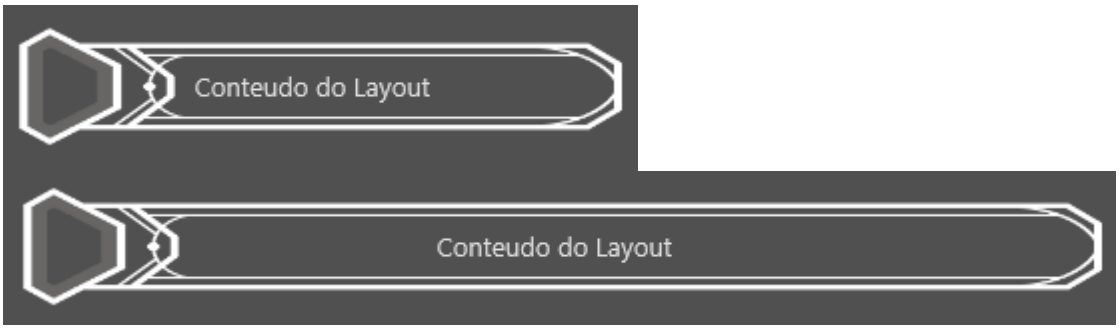
Criamos um frame que se escala na vertical (autoScaleY = true) e que possui 3 partes:

- A parte da esquerda, âncora à esquerda, desenha uma porção da imagem (um retângulo de 84x100)
- A parte da direita, âncora à direita, desenha uma outra porção da imagem (um retângulo 40x100)
- A parte central, âncora à esquerda e à direita, desenha uma outra porção da imagem (um retângulo 341x100) de forma esticada.

```
<?xml version="1.0" encoding="UTF-8"?>
<frame width="468" height="100" autoScaleY="true">
        <borders left="12" top="24" bottom="24" right="20" />
        <draw>
                <!-- Parte Esquerda (FIXA) -->
                <image left="0" top="0" right="84" bottom="100"zOrder="0">
                        <anchors left="true" top="true" bottom="true"/>
                        <source url="header0_smaller.png" left="0"
right="84" top="0" bottom="100"/>
                </image>

                <!-- Parte Direita (FIXA) -->
                <image left="425" top="0" right="468" bottom="100"
zOrder="0">
                        <anchors left="false" top="true" bottom="true"
right="true"/>
                        <source url="header0_smaller.png" left="425"
right="468" top="0" bottom="100"/>
                </image>

                <!-- Parte Central (ESTICA) -->
                <image left="84" top="0" right="425" bottom="100"
                       overflowX="stretch" overflowY="stretch" zOrder="-
1">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                        <source url="header0_smaller.png" left="84"
right="425" top="0" bottom="100"/>
                </image>
        </draw>
</frame>
```





4 - Frame retangular dinâmico, usando vários arquivos de imagens

Deseja-se formar um frame com a seguinte aparência:



Para isso, dividimos a figura em 8 partes

	leftLine.png (12x66)
	rightLine.png (12x66)
	lowerDir.png (29x32)

	lowerEsq.png (29x32)
	upperDir.png (29x32)
	upperEsq.png (29x32)
	lowerLine.png (80x5)
	upperLine.png (80x5)

e usamos o seguinte arquivo de definição de frame:

```
<?xml version="1.0" encoding="UTF-8"?>
<frame width="138" height="130">
        <borders left="12" top="10" bottom="10" right="12"/>
        <draw>
                        <!-- QUINAS -->
                        <!-- quina superior esquerda -->
                        <image left="0" top="0" right="29" bottom="32"
overflowX="stretch">
                                <anchors left="true" top="true"/>
                                <source url="upperEsq.png" left="0" right="29"
top="0" bottom="32"/>
                        </image>

                        <!-- quina superior direita -->
                       <image left="109" top="0" right="138" bottom="32"
overflowX="stretch" overflowY="stretch">
                                <anchors left="false" right="true" top="true"/>
                                <source url="upperDir.png" left="0" right="29"
top="0" bottom="32"/>
                        </image>

                        <!-- quina inferior esquerda -->
                        <image left="0" top="98" right="29" bottom="130">
                                <anchors left="true" top="false" bottom="true"/>
                                <source url="lowerEsq.png" left="0" right="29"
top="0" bottom="32"/>
```

```
                        </image>

                        <!-- quina inferior direita -->
                        <image left="109" top="98" right="138" bottom="130">
                                <anchors left="false" right="true" top="false"
bottom="true"/>
                                <source url="lowerDir.png" left="0" right="29"
top="0" bottom="32"/>
                        </image>

                       <!-- LINHAS -->
                        <!-- linha horizontal superior -->
                        <image left="29" top="0" right="109" bottom="5"
zOrder="-1" overflowX="stretch">
                                <anchors left="true" right="true" top="true"/>
                                <source url="upperLine.png" left="0" right="80"
top="0" bottom="5"/>
                        </image>

                        <!-- linha horizontal inferior -->
                        <image left="29" top="125" right="109" bottom="130"
zOrder="-1" overflowX="stretch">
                                <anchors left="true" right="true" top="false"
bottom="true"/>
                                <source url="lowerLine.png" left="0" right="80"
top="0" bottom="5"/>
                        </image>

                         <!-- linha vertical esquerda -->
                        <image left="0" top="32" right="12"
                               bottom="98" zOrder="-1" overflowY="stretch"
overflowX="stretch">
                                <anchors left="true" right="false" bottom="true"
top="true"/>
                                <source url="leftLine.png" left="0" right="12"
top="0" bottom="66"/>
                        </image>

                        <!-- linha vertical direita -->
```

```
                        <image left="126" top="32" right="138" bottom="98"
                               zOrder="-1" overflowY="stretch"
overflowX="stretch">
                                <anchors left="false" right="true" bottom="true"
top="true"/>
                                <source url="rightLine.png" left="0" right="12"
top="0" bottom="66"/>
                        </image>
        </draw>
</frame>
```

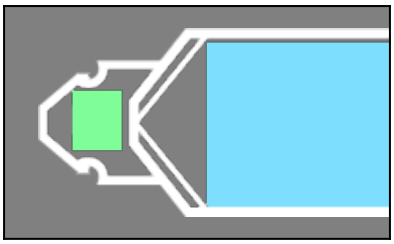






5 - Frame com regiões

Deseja-se montar um frame com 2 regiões com a seguinte aparência:



O exemplo utiliza um arquivo de imagem "pecaCobre.png" de tamanho 228x94

Arquivo de definição de frame:

```
<?xml version="1.0" encoding="UTF-8"?>
<frame width="228" height="94" autoScaleY="true" autoScaleX="true">
        <borders left="6" top="7" bottom="6" right="39" />
        <draw>
                <!-- fundo -->
                <image left="0" top="0" right="228" bottom="94"
                       overflowX="stretch" overflowY="stretch" zOrder="-
1">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                        <source url="pecaCobreBckBlack.png" left="0"
right="228" top="0" bottom="94"/>
                </image>
        </draw>

        <regions>
                <region name="RegiaoSmallTitulo" left="15" top="30"
right="45" bottom="64">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                </region>

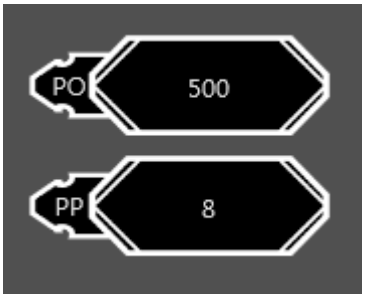
                <region name="RegiaoValor" left="85" top="7" right="187"
bottom="89">
                        <anchors left="true" top="true" bottom="true"
right="true"/>
                </region>
        </regions>
</frame>
```

Uso no Lua Form:

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

        <layout align="top" frameStyle="/frames/meuframe.xml"
                margins="{left=30, top=30, right=30, bottom=5}">
                <label horzTextAlign="center" text="PO"
frameRegion="RegiaoSmallTitulo"/>
                <label horzTextAlign="center" text="500"
frameRegion="RegiaoValor"/>
        </layout>

        <layout align="top" frameStyle="/frames/meuframe.xml"
                margins="{left=30, top=5, right=30, bottom=5}">
                <label horzTextAlign="center" text="PP"
frameRegion="RegiaoSmallTitulo"/>
                <label horzTextAlign="center" text="8"
frameRegion="RegiaoValor"/>
        </layout>
</form>
```



Os labels foram automaticamente alinhados às regiões definidas no frame.

Bibliotecas LUA

Este tópico contém a referência das bibliotecas e funções disponíveis para serem usadas em códigos LUA

Biblioteca	Descrição
NDB	Biblioteca para manipular NodeDatabases, documentos de fácil escrita e leitura para se armazenar informações Leia mais em: Biblioteca ndb.
VHD	Biblioteca do HD Virtual do plug-in, usado para criar, ler e manipular arquivos. Leia mais em: Biblioteca vhd.
Dialogs	Biblioteca que contém funções para exibir mensagens e interagir com o usuário através de diálogos. Leia mais em: Biblioteca dialogs.
GUI	Biblioteca que contém funções relacionadas à interface e Lua Forms. Leia mais em: Biblioteca gui.
Locale	Biblioteca que contém funções para lidar com idiomas e regiões, incluindo funcionalidade para permitir um plug-in em várias línguas. Leia mais em Biblioteca Locale
Firecast	Biblioteca que contém funções relacionadas às mesas, personagens, envio de mensagens, rolagem de dados, etc.. Leia mais em: Biblioteca Firecast.
Firecast.Messaging	Biblioteca que contém funções que tratam as notificações que o Firecast dispara para todos os plug-ins através de mensagens. Leia mais em Biblioteca Firecast.Messaging
Firecast.Plugins	Biblioteca que contém funções relacionadas aos plug-ins instalados no RRPG. Leia mais em: Biblioteca Firecast.Plugins

Firecast.Requests	Biblioteca que contém funções que solicitam determinadas ações ao Firecast, como, por exemplo, criar itens na biblioteca da mesa, obter informações de perfil de um usuário, etc.. Leia mais em Biblioteca Firecast.Requests
Internet	Biblioteca que contém funções relacionadas à internet/intranet, como: downloads, requisições HTTP, etc.. Leia mais em Biblioteca internet
FireDrive	Biblioteca para manipular o FireDrive do usuário. Leia mais em Biblioteca fireDrive
SceneLib	Biblioteca com funções para lidar com Scene 3 do RRPG. Leia mais em Biblioteca scene
System	Biblioteca com funções para lidar com o dispositivo do usuário e outras funcionalidades básicas. Leia mais em Biblioteca system
Utils	Biblioteca que possui funções utilitárias diversas Leia mais em Biblioteca utils.

Async library

The Async library provides functions for handling asynchronous operations, such as loading a remote node database. All of these functions are contained in the "Async" Lua table found in the "async.lua" unit.

Usage example:

```
-- First, you need to require the "async.lua" unit
require("async.lua");

-- Now you can access the functions of the library.
Async.LIBRARY_FUNCTION(Argument1, Argument2, ...);
```

Async library functions

```
function Async.await(promise)
function await(promise)
```

Performs an asynchronous wait for the resolution of a Promise object, suspending the currently running coroutine until the provided Promise is resolved. While suspended, other coroutines and program execution lines can run, preventing the application from blocking or freezing. The coroutine execution is resumed once the given Promise is resolved to either success or failure

Reminder: Coroutines are computer program execution lines that can be suspended and resumed.

Arguments:

- promise - A Promise object identifying the promise for which the asynchronous wait must be performed

Return:

When the wait is complete:

- If the given promise resolves to success, the function returns the promise's success data.
- If the given promise resolves to failure, the function raises an error with the promise's failure message.

Remarks:

- Using this function can significantly simplify your code when waiting for the outcome of promises.
- If the given promise has already been resolved, the await will return, or raise an error, if applicable, immediately, without suspending the execution of the current coroutine.
- Keep in mind that this function works only inside coroutines. All Firecast events run inside coroutines, but you can always start a coroutine with the Async.execute function.
- If you dont want automatic error raising in case the promise resolves to failure, check the similar function pawait.

```
function Async.execute(fn, ...)
```

Starts the execution of a coroutine.

Reminder: Coroutines are computer program execution lines that can be suspended and resumed.

Arguments:

- fn - A function that will be executed by the coroutine.
- ... (variable argument list) - All given arguments will be passed to the function as arguments when it starts the execution.

Return:

- A Promise object that can be used to track the execution of the coroutine. The success data of this promise will be whatever the given function returns during its execution. If an error occurs while executing the function, the promise will be resolved to failure with the failure message describing the execution error.

Remarks:

- You can use the await function inside coroutines!
- The coroutine starts its execution immediately. Calling Async.execute will return when the given function reaches its end, when an execution error occurs, or when it is first suspended (such as by calling `await`), whichever comes first.
- If the given function returns a Promise object at the end of its execution, a delegation will occur, making the promise returned by Async.execute resolve when the mentioned Promise resolves, with the same outcome

```
function Async.pawait(promise)
function pawait(promise)
```

Similar to the await function, pawait performs an asynchronous wait for the resolution of a Promise object, suspending the currently running coroutine until the provided Promise object is resolved. However, unlike await, pawait does **not** automatically raise an error if the promise resolves to failure

Arguments:

- promise - A Promise object identifying the promise for which the asynchronous wait must be performed

Return:

When the wait is complete, the function will return multiple values:

- f the given promise resolves to success, the first returned value will be a boolean with the value **true**, and the second one will be the promise's success data.
- If the given promise resolves to failure, the first returned value will be a boolean with the value **false**, and the second one will be the promise's failure message.

Remarks:

- For more information, please refer to the await function documentation.

Async.Promise library

The Async.Promise library provides functions for handling asynchronous operations, such as loading a remote node database. All of these functions are contained in the “Async.Promise” and/or “Promise” Lua tables found in the “async.lua” unit.

Usage example:

```
-- First, you need to require the “async.lua” unit
require("async.lua");

-- Now you can access the functions of the library.
Async.Promise.LIBRARY_FUNCTION(Argument1, Argument2, ...);
or
Promise.LIBRARY_FUNCTION(Argument1, Argument2, ...);
```

Async.Promise library functions

```
function Async.Promise.all(promise1[, promise2[, promise3[, ...
promiseN]]])
function Async.Promise.all(promisesArray)
function Promise.all(promise1[, promise2[, promise3[, ...promiseN]]])
function Promise.all(promisesArray)
```

This function takes a set of Promise objects and creates a single Promise object that will be resolved when all provided promises are resolved. The set of Promise objects can be passed to this function using one of the following styles:

Arguments (variable number of arguments style):

- Each argument identifies a single Promise object in the set. Examples of valid usages:
  - Promise.all(promise)
  - Promise.all(promise, promise2)
  - Promise.all(promise, promise2, promise3)
  - Promise.all(promise, promise2, promise3, promise4)

Arguments (array style):

- promisesArray - A single array containing all Promise objects of the set.

Return:

- A newly created Promise object that monitors all informed Promise objects.

Remarks:

- The returned Promise will be successfully resolved when all informed promises are successfully resolved. The success data of the returned Promise will be an array containing the set success data, in the order of the promises passed, regardless of completion order.
- If any of the given Promises is resolved to failure, the promise returned by this function will also be resolved to failure, and its associated failure reason will be the same as the first promise that was resolved to failure.
- If the given set of Promises is empty, the function will return an already resolved Promise object.

```
function Async.Promise.isPromise(value)
function Promise.isPromise(value)
```

Check is the provided value is a Promise object

Arguments:

- value - the value that will be verified.

Return:

- A boolean; true means the provided value is a promise object, false otherwise.

```
function Async.Promise.resolved(data)
function Async.Promise.resolve(data)
function Async.Promise.succeeded(data)
function Promise.resolved(data)
function Promise.resolve(data)
function Promise.succeeded(data)
```

Creates and returns a Promise object that has already been **successfully** resolved with the provided "data" argument

Arguments:

- data - the value that was successfully resolved. This can be any information you want to pass as a successful return value

Return:

- The new Promise object

```
function Async.Promise.toHandle(promise, successCallback,[, failureCallback])
```

```
function Promise.toHandle(promise, successCallback[, failureCallback])
```

Creates a Promise object that receives the outcome of another existing Promise object and potentially returns converted data.

Arguments:

- promise - An existing Promise object that will be monitored
- successCallback - A function called when the monitored Promise object resolves to success. The first argument received by this function is the outcome data of the monitored promise. Whatever this function returns will be the success data of the new promise.
- *(Optional)* failureCallback - A function called when the monitored Promise object resolves to failure. The first argument received by this function is the error message of the monitored promise. Whatever this function returns will be the success data of the new promise.

Return:

- A new Promise object

Remarks

- This type of promise is typically used to convert data from an asynchronous operation and/or to provide an alternate outcome if the monitored asynchronous operation fails.
- If the execution of `successCallback` raises an error or exception, the new promise will resolve to failure even if the monitored promise resolved to success.
- If the execution of `successCallback` returns a third promise object, the outcome of the promise created by this function will be delegated to this third promise.
- If the execution of `failureCallback` does not raise an error or exception, the new promise will resolve to success even if the monitored promise resolved to failure.
- If the execution of `failureCallback` returns a third promise object, the outcome of the promise created by this function will be delegated to this third promise.

```
function Async.Promise.toResolve()
function Async.Promise.pending()
function Promise.toResolve()
function Promise.pending()
```

Creates a Promise object that is in a **pending state** and its associated Promise Resolution object.

This function returns 2 values:

- The first return is a new Promise object
- The second return is the PromiseResolution object that is used to resolve the created Promise.

Remarks

- While the Promise object represents an asynchronous process that other pieces of code can track, the PromiseResolution object is used to resolve the promise either to success or failure. Typically, you return or pass along the Promise object as a mechanism to monitor the asynchronous operation while keeping and using the PromiseResolution privately.

```
function Async.Promise.withError(errorMsg)
function Async.Promise.withException(errorMsg)
function Async.Promise.failed(errorMsg)function
Async.Promise.reject(errorMsg)
function Promise.withError(errorMsg)
function Promise.withException(errorMsg)
function Promise.failed(errorMsg)
function Promise.reject(errorMsg)
```

Creates and returns a Promise object that has already been resolved with a **failure** state and with the provided error message

Arguments:

- errorMsg - The error message that describes why the Promise resolved to a failed state.

Return:

- The new Promise object

Promise object

A Promise is an object designed for monitoring asynchronous operations and their eventual outcomes, whether they result in success or failure

A Promise can exist in one of the following states:

- Pending - The asynchronous operation is still in progress
- Success - The asynchronous operation concluded successfully
- Failed - The asynchronous operation concluded with an error or failure.

Once a Promise has resolved to either "Success" or "Failed," its state becomes sealed along with its associated data or error message, and it remains immutable.

Tracking a promise

When dealing with a function that returns a Promise object, you have the flexibility to either monitor or disregard its eventual outcome. If the success or failure of the asynchronous operation doesn't impact your code, you can simply ignore the Promise object.

If you need to capture the result of an asynchronous operation, you can engage with the Promise object using one of the following styles:

"Await" style for tracking promises:

This style simplifies the code, but it is limited to use within coroutines.

Reminder: Coroutines are computer program execution lines that can be suspended and resumed. All Firecast events run inside coroutines, and you can initiate a new one using the Async.execute function.

Example 1 (using a variable to hold the Promise object):

```
local promise = Dialogs.asyncOpenFile("Select a image",
"image/*");

local selectedFiles = await(promise);

showMessage(selectedFiles[1].name);
```

Example 2 (without using a variable to hold the Promise object):

```
local selectedFiles = await(Dialogs.asyncOpenFile("Select a
image", "image/*"));

showMessage(selectedFiles[1].name);
```

Example 3 (using the sugar "await" method that Promises objects have):

```
local selectedFiles = Dialogs.asyncOpenFile("Select a image",
"image/*"):await();

showMessage(selectedFiles[1].name);
```

Please refer to the await function documentation for more details.

"Event" style for tracking promises:

This style works both inside and outside of coroutines but can make your code less readable if used excessively.

Invoke the "thenDo" and/or "onError" methods that all Promise objects have. The functionality of these methods is clarified below.

Example 1 (using a variable to hold the Promise object):

```
local promise = Dialogs.asyncOpenFile("Select a image",
"image/*");


promise:thenDo(

    function(selectedFiles)

        -- the argument "selectedFiles" receives the success
data of the promise

        showMessage(selectedFiles[1].name);

end);
```

Example 2 (without using a variable to hold the Promise object):

```
Dialogs.asyncOpenFile("Select a image", "image/*"):thenDo(

    function(selectedFiles)

        -- the argument "selectedFiles" receives the success
data of the promise

        showMessage(selectedFiles[1].name);

    end);
```

Object characteristics

Methods

Método	Descrição
promise:await()	This method is employed to synchronize with the resolution of the Promise, allowing your code to pause until the asynchronous operation concludes. The **promise:await()** method is functionally equivalent to **await(promise)**. For detailed information, please consult the documentation for the await function.
promise:pawait()	This method is employed to synchronize with the resolution of the Promise, allowing your code to pause until the asynchronous operation concludes. The **promise:pawait()** method is functionally equivalent to **pawait(promise)**

	For detailed information, please consult the documentation for the pawait function.

promise:thenDo(successCallback)
or
promise:onSuccess(successCallback)
	Set a continuation callback function that will be called when the Promise resolves to success.
	
	Arguments:
	
	● successCallback - A function that will be called when the Promise resolves to success. The first argument of the callback function will receive the associated success data.
	
	Return:
	
	● A new Promise object that will be resolved asynchronously with the execution of the successCallback. This new promise can be used to chain promises handling.
	
	Remarks:
	
	● If the Promise is already in a success state, the successCallback will be called as soon as possible
	
	Example:
	
	`local promise = Firecast.asyncOpenUserNDB("name");`
	
	`promise:thenDo(`
	`function(node)`
	`-- This function will be called when the Promise is resolved --`
	`successfully. The first argument, "node", contains the -- Promise`
	`success data, which in this case is an opened -- nodeDatabse`

	`end);`
promise:thenDo(successCallback, failureCallback)	Set continuation callback functions that will be called when the Promise resolves to success or failure. Arguments: ● successCallback - A function that will be called if the Promise resolves to success. The first argument of the callback function will receive the associated success data ● failureCallback - A function that will be called if the Promise resolves to failure. The first argument of the callback function will receive the error message. Return: ● A new Promise object that will be resolved asynchronously with the execution of either the successCallback or failureCallback. This new promise can be used to chain promises handling. Remarks: ● This will monitor the Promise for both success and failure states at the same time ● If the Promise is already in a success state, the successCallback will be called as soon as possible. ● If the Promise is already in a failure state, the failureCallback will be called as soon as possible ● If the "successCallback" is called, "failureCallback" will never be called and vice versa. Example: `local promise = Firecast.asyncOpenUserNDB("name");`

	``` promise:thenDo(function(node) -- This function will be called when/if the -- Promise is resolved successfully. The first argument, -- "node", contains the Promise success data, -- which in this case is an opened nodeDatabase end, function(errorMsg) -- This function will be called when/if the -- Promise is resolved to failure. The first argument, -- "errorMsg", contains the associated error message, -- which in this case is the message explaining why -- it was not possible to open the nodeDatabase end); ```
promise:thenFinally(callback)	Set a continuation callback function that will be called when the Promise resolves, whether to success or failure. Arguments: ● callback - A function that will be called when the Promise resolves, regardless of the outcome. This function receives no parameters and will handle both success and failure cases. Return: ● A new Promise object that

	will be resolved asynchronously with the execution of the callback. This new promise can be used to chain promises handling. Remarks: ● Typically, this method is used to establish a finalization continuation that must be executed regardless of the promise outcome. ● This will monitor the Promise for both success and failure states at the same time ● If the Promise is already in a success state, the callback will be called as soon as possible. ● If the Promise is already in a failure state, the callback will be called as soon as possible ● The callback will be called only once.
promise:thenResolve(promiseResolution)	Set a delegation that will resolve the provided PromiseResolution object with the same outcome (success or failure) as the original promise. Arguments: ● promiseResolution - a PromiseResolution object that will be resolved. Return: ● A new Promise object that will be resolved asynchronously along with the delegation. This new promise can be used to chain promises handling. Remarks: ● If the promise is already resolved, the PromiseResolution will be resolved as soon as possible.

promise:onError(failureCallback)	Set a continuation callback function that will be called when the Promise resolves to failure.
	
	Arguments:
	
	● failureCallback - A function that will be called if the Promise resolves to failure. The first argument of the callback function will receive the error message.
	
	Return:
	
	● A new Promise object that will be resolved asynchronously with the execution of failureCallback. This new promise can be used to chain promises handling.
	
	Remarks:
	
	● If the Promise is already in a failure state, the failureCallback will be called as soon as possible
	
	Example:
	
	`local promise = Firecast.asyncOpenUserNDB("name");`
	
	`promise:onError(`
	`  function(errorMsg)`
	`  -- This function will be called when/if the`
	`  -- Promise is resolved to failure. The first argument,`
	`  -- "errorMsg", contains the associated error message,`
	`  -- which in this case is the message explaining why`
	`  -- it was not possible to open the nodeDatabase`
	`  end);`

PromiseResolution object

While a Promise object represents an asynchronous process that can be tracked by other parts of the code, a PromiseResolution object is responsible for providing a means to resolve its associated promise, either as a success or a failure.

Characteristics

Methods

Método	Descrição
promiseResolution:setSuccess([successData])	Resolves its associated promise as a success with the provided success data. Arguments: ● *(Optional)* successData - a Lua value that will be associated as the success data of the promise. Return: ● A boolean value; true indicates successful resolution of the promise, while false signifies that it was not possible to resolve the promise as a success. This could occur if the promise had already been resolved to either success or failure, and its state cannot be changed further. Remarks ● If "successData" is a Promise object, a delegation will occur: This promiseResolution will monitor the provided promise object and whatever resolution it has, be it failure or success, this promiseResolution will

	also adopt.
promiseResolution:setFailure(errorMsg)	Resolves its associated promise as a failure with the provided error message. Arguments: ● errorMessage - a string containing the reason for the promise failure. Return: ● A boolean value; true indicates successful resolution of the promise, while false signifies that it was not possible to resolve the promise as a failure with the provided error message. This could occur if the promise had already been resolved to either success or failure, and its state cannot be changed further.
promiseResolution:setUserAborted()	Resolves the associated promise as a failure because the user aborted the operation Arguments: None Return: ● A boolean value; true indicates successful resolution of the promise, while false signifies that it was not possible to resolve the promise as a failure. This could occur if the promise had already been resolved to either success or failure, and its state cannot be

	changed further.

Remarks

- When a PromiseResolution object is garbage collected by Lua, and its associated promise has not yet been resolved to either success or failure, the promise will be automatically resolved to failure, signifying an abandoned promise.

Audio library

The *Audio* library provides functions for playing and handling audio. All features are contained in the "Audio" Lua table found in the "audio.lua" unit.

Usage example:

```
-- First, you need to require the "audio.lua" unit
require("audio.lua");

-- Now you can access the functions of the library.
Audio.LIBRARY_FUNCTION(Argument1, Argument2, ...);
```

Properties and attributes

```
Audio.defaultPlayer
```

This attribute contains the Audio Player object used for playing audio to the user's default audio device

Remarks:

- Audio played through this object is not broadcasted to other users. For audio transmission, consider using mesa.audioPlayer instead.

Functions

```
function Audio.asyncLoadAudioStream(source)
```

Asynchronously creates an AudioStream object using the content of an audio file.

Arguments:

- source - The source of the audio file. You can provide:
  - A string containing the external URL of the audio. Example: "https://www.domain.com/my/audioFile.mp3"
  - A string containing the file address in the plugin VHD . Example: "/audios/kick.wav"
  - A Stream object with the audio file content.

Return:

- A promise for an AudioStream object.

Remarks:

- An AudioStream object represents a source of audio signal.
- As a source of audio signal, an AudioStream object does not play audio by itself. To play it, you must also create an AudioTrack object using the "asyncAddTrack" method of the target AudioPlayer object.

```
function Audio.play(source[, volume])
```

Plays an audio file using the user's default audio device.

Arguments:

- source - The source of the audio to be played. You can provide:
  - A string containing the external URL of the audio. Example: "https://www.domain.com/my/audioFile.mp3"
  - A string containing the file address in the plugin VHD . Example: "/audios/kick.wav"
  - A Stream object with the audio file content.
- *(Optional)* volume - A number representing the volume at which the audio should be played. This number functions as a multiplication factor, where 0.0 means total silence, and 2.0 means double the original audio volume. The default value is 1.0.

Return:

- A promise for an AudioTrack object.

Remarks:

- This function is a convenient alias and is equivalent to calling "Audio.defaultPlayer:play(address, volume)"
- Refer to the AudioPlayer:play method for a deeper understanding of how this function operates.

AudioPlayer object

This object represents a target capable of playing audio and offers methods to do so.

You can retrieve instances of AudioPlayer by using "Audio.defaultPlayer" attribute or "room.audioPlayer" attributte.

Characteristics

Methods

Método	Descrição
audioPlayer:asyncAddTrack(audioStream[, params])	Creates an AudioTrack object responsible for playing the audio signal from an AudioStream object. Arguments: ● audioStream - an AudioStream object representing the source of the audio signal to be played. It can be obtained by calling the Audio.asyncLoadAudioStream method. ● *(Optional)* params - A Lua table with the AudioTrack parameters and may contain the following attributes: ○ autoRemoveOnEOF - A boolean; when true, Firecast will automatically remove the track once the provided audioStream reaches its EOF. Default: false; Return: ● A promise for an AudioTrack object. Remarks ● While an AudioStream object represents the source of an audio signal, an AudioTrack handles the actual playback of the audio signal. ● An AudioTrack resides within the AudioPlayer, and while active, it continually retrieves audio signals from the provided AudioStream, playing

	them in real-time. ● Since the AudioTrack retrieves and plays audio in real-time, any changes made to the provided AudioStream object will also be perceived in real-time during playback. This means you can adjust the volume, pause, and seek audio position using the appropriate AudioStream object properties and methods while the AudioTrack is active.
audioPlayer:play(source[, volume])	Plays an audio file. Arguments: ● source - The source of the audio to be played. You can provide: ○ A string containing the external URL of the audio. Example: "https://www.domain.com/my/audioFile.mp3" ○ A string containing the file address in the plugin VHD . Example: "/audios/kick.wav" ○ A Stream object with the audio file content. ● *(Optional)* volume - A number representing the volume at which the audio should be played. This number functions as a multiplication factor, where 0.0 means total silence, and 2.0 means double the original audio volume. The default value is 1.0. Return: ● A promise for an AudioTrack object. Remarks: ● The returned promise will be resolved as soon as the audio starts to play. ● This function will play the audio from start to end only once. ● You can use the AudioTrack object returned by this function to stop the playback if you wish. ● The AudioTrack created by this function will be automatically removed from the player once the audio reaches the end.

	● This function maintains a strong reference to the created AudioTrack object while the audio is playing, preventing Lua from garbage collecting it during playback.

AudioStream object

An AudioStream object represents a source of audio signal and can be created by calling the Audio.asyncLoadAudioStream() function. However, it does not play audio on its own. To play it, you must also create an AudioTrack object using the "asyncAddTrack" method of a target AudioPlayer object.

During playback of an AudioStream through an AudioTrack object, any alterations made to the AudioStream object are immediately reflected in real-time. This allows you to adjust parameters such as volume, pause, and audio position using the pertinent AudioStream object properties and methods while the AudioTrack is actively playing.

Characteristics

Propertys and attributes

Propriedade	Tipo	Descrição
enabled	Boolean	Indicates whether the AudioStream can output audio signal. If the AudioStream is being played by an AudioTrack, setting this attribute to **false** will pause the playback, as no audio signal will be outputted. Setting it back to **true** will resume the playback. Default: True
eof	Boolean	(Read-only) Indicates whether the AudioStream has reached its EOF (end of file).
length	Float	(Read-only) If the duration of the audio is known, this attribute contains its duration in **seconds**; otherwise, it is set to **nil**. Remarks: ● Please be aware that this

		attribute may be **nil** instead of a number, so make sure to handle this possibility in your code.
position	Float	If the current audio stream is seekable and its position is defined, this contains the stream position in **seconds**; otherwise, it is set to **nil**. Remarks: ● When setting a value to this attribute, Firecast will seek the audio stream to the closest keyframe, and its true audio position can be slightly earlier than desired. ● Please be aware that this attribute may be **nil** instead of a number, so make sure to handle this possibility in your code. ● Additionally, when the audio stream is being played by an AudioTrack in real-time, this value will be updated dynamically, allowing you to track the playback progress
volume	Float	A numerical value representing the output volume level of the audio signal from the AudioStream. This value acts as a multiplication factor, with 0.0 indicating complete silence, and 2.0 indicating twice the original audio volume. The default value is 1.0

Methods

Método	Descrição
audioStream:setPosition(newPosition)	Seeks the audio stream to a specified position. Arguments: ● newPosition - A number

	indicating the desired position, in **seconds**, to which Firecast should seek the audio stream. Return: ● A boolean value indicating whether the audio seek was successful. Remarks: ● Firecast will seek the audio stream to the nearest keyframe, which may result in the true audio position being slightly earlier than desired. ● If the audio stream is not seekable, the method will return false. ● Calling this method is equivalent to setting the "position" property, but with the added benefit of being able to check the success of the seek operation.
audioStream:waitEOF();	Sets up a monitoring that waits for the AudioStream to reach its end of file (EOF). Arguments: None Return: ● A promise that will be resolved when the AudioStream reaches its end of file (EOF). Remarks: ● If the AudioStream is already at the EOF, the returned promise will be resolved immediately.

Events

Nome do evento	Descrição
onEOF	This event is triggered by Firecast when the AudioStream reaches its end of file (EOF).

AudioTrack object

An AudioTrack object serves as a real-time agent that continuously fetches audio signals from an AudioStream and plays them. While AudioStream objects provide audio signals, it's the AudioTrack objects residing within an AudioPlayer that actually play them.

Lifecycle

AudioTracks are created using the AudioPlayer's asyncAddTrack method.

While active, AudioTracks continuously retrieve audio signals from their associated AudioStream, playing them in real-time. As AudioTracks retrieve and play audio in real-time, any changes made to their associated AudioStream objects will also be immediately reflected in the playback. This allows you to adjust the volume, pause, and seek audio position using the relevant AudioStream object properties and methods while the AudioTrack is in operation.

AudioTracks can be stopped by calling their "remove" method. If the Lua Garbage Collector collects an AudioTrack object, it will also be automatically stopped.

Additionally, AudioTracks can be forcibly removed by Firecast. For example, when a user leaves the room, all audio transmission to that room must cease. You can use waitRemoval method, removed attribute, and/or onRemove event to monitor when this occurs.

Characteristics

Propertys and attributes

Propriedade	Tipo	Descrição
audioStream	AudioStream object	(Read-only) The associated AudioStream object providing audio signal to the AudioTrack. Remarks: ● As AudioTracks retrieve and play audio in real-time, any changes made to this AudioStream object will also be immediately reflected in

		the playback. This allows you to adjust the volume, pause, and seek audio position using the relevant AudioStream object properties and methods while the AudioTrack is in operation.
removed	Boolean	(Read-only) Indicates whether the track has been removed from its AudioPlayer object. Remarks ● Once a track is removed, it cannot be reactivated. However, its associated AudioStream object can be reused.

Methods

Método	Descrição
audioTrack:remove()	Removes the AudioTrack from its AudioPlayer, halting the playback. Arguments: None Return: ● A promise that can be used to track the asynchronous process of AudioTrack removal. Remarks ● Once a track is removed, it cannot be reactivated. However, its associated AudioStream object can be reused.

audioTrack:waitRemoval()	Sets up a wait that monitors the removal state of the AudioTrack. Arguments: None Return: ● A promise that will be resolved when the AudioTrack is removed from its AudioPlayer. Remarks: ● This function does not remove the AudioTrack; it simply monitors when it happens. ● If the AudioTrack is already removed, the returned promise will be resolved immediately. ● Once a track is removed, it cannot be reactivated. However, its associated AudioStream object can be reused.

Events

Nome do evento	Descrição
onRemoved	This event is triggered by Firecast when the AudioTrack is removed from its AudioPlayer. Remarks: ● Once a track is removed, it cannot be reactivated. However, its associated AudioStream object can be reused.

Biblioteca Dialogs

A biblioteca dialogs provê funções para exibir mensagens e interagir com o usuário através de diálogos.

Todas as funções estão contidas na table/variável "Dialogs" da unidade "dialogs.lua".

Exemplo de uso:

```lua
-- Primeiro, é necessário usar a unidade "dialogs.lua"
require("dialogs.lua");

-- Agora é possível acessar as funções da biblioteca
Dialogs.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Dialogs

```lua
function Dialogs.asyncSelectTalemarkColor([initialColor])
```

Shows a popup allowing the user to select a color suitable for Talemark texts.

Arguments:

- *(Optional)* initialColor – The initial selected color when showing the popup. It can be one of the following:
  - **nil** or an empty string - Undefined color. This is the default initial color.
  - A number - A Firecast palette index.
  - A string - A color string.

Return:

- A Promise for the selected color, which can be in one of the following formats:
  - **nil** - The user selected the "undefined color" option in the popup.
  - A number - A Firecast pallete index
  - A string - A color string.

Remarks:

- If the user cancels the popup, the returned promise will resolve to failure. Because of this behavior, it is recommended to use the pawait() function instead of await() to wait for the promise resolution.

Example:

```lua
local promise = Dialogs.asyncSelectTalemarkColor(12);
local r, color = pawait(promise);

if r then
    showMessage("The selected color is " .. tostring(color));
```

```
else
    showMessage("The operation failed or the user canceled the
popup");
end;
```

```
function Dialogs.showMessage(msg [, callback])
ou
function showMessage(msg [, callback])
```

Exibe uma mensagem de informação para o usuário (um simples diálogo com o botão "OK").

Parâmetros:

- msg – Uma cadeia de caracteres contendo a mensagem que deverá ser exibida na interface.
- (OPCIONAL) callback - uma função que será chamada após o usuário clicar em OK.

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.
- Se quiser executar algo após o usuário confirmar o diálogo, informe o parâmetro "callback"

```
function Dialogs.alert(msg [, callback])
ou
function alert(msg [, callback])
```

Exibe uma mensagem de alerta para o usuário (um simples diálogo com ícone de alerta e botão "OK").

Parâmetros:

- msg – Uma cadeia de caracteres contendo a mensagem que deverá ser exibida na interface.
- (OPCIONAL) callback - uma função que será chamada após o usuário clicar em OK.

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.
- Se quiser executar algo após o usuário confirmar o diálogo, informe o parâmetro "callback"

```
function Dialogs.showErrorMessage(msg [, callback])
```

Displays an error message to the user (a simple dialog with the "OK" button).

Parâmetros:

- msg – A string containing the error message to be displayed on the interface.
- *(Optional)* callback - A function that will be called after the user clicks OK.

Return:
Does not return any value.

Remarks:

- The function does not wait for the user to respond to the dialog to continue. That is, the dialog is presented in the interface and the LUA code continues its normal execution in the meantime.
- If you want to execute something after the user confirms the dialog, provide a "callback" function.

```
function Dialogs.asyncOpenFile(prompt[, accept [, multiple]])
```

Exibe um diálogo para que o usuário escolha um ou mais arquivos do seu computador/dispositivo para abrir.

Parâmetros:

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre qual arquivo ele deve selecionar. Exemplo: "Escolha uma imagem".
- (OPCIONAL) accept - Uma cadeia de caracteres usada para filtrar quais arquivos o usuário pode selecionar. Você pode passar aqui:
  - ".<EXTENSÃO>" para filtrar arquivos de uma determinada extensão. Exemplo: ".txt", ".mp3", ".png", etc..

        - "<MIME_TYPE>" para filtrar arquivos pelo Media/Mime Type. Exemplos: "image/png", "application/xml", "text/plain", etc..
        - "image/*" para filtrar todos os formatos de imagens suportados pelo RRPG.
        - "*.*" ou nil - Para aceitar qualquer tipo de arquivo.
        - Uma lista das opções anteriores separadas por vírgula. Exemplos:
            - ".png, .txt, .bmp" - para aceitar arquivos de extensões PNG, TXT e BMP
            - ".html, text/plain, image/*" - para aceitar arquivos HTML, arquivos de media type text/plain e também qualquer imagem.
    - (OPCIONAL) multiple - true para permitir o usuário selecionar mais de um arquivo ao mesmo tempo ou false para permitir selecionar apenas um arquivo. Se omitido este valor, o padrão é "false".

Retorno:

- Um promise para um array de tabelas Lua que contém cada uma:
    - name - Uma propriedade contendo o nome curto do arquivo selecionado. Exemplos: "Arquivo.txt", "Minha imagem de férias.png", etc..
    - stream - Um objeto stream somente leitura contendo os dados do arquivo selecionado.

Observações:

- Esta é a única maneira que o plug-in tem de obter acesso a arquivos do dispositivo do usuário.
- Se o parâmetro "multiple" for false, a promessa sempre receberá um array contendo 1 tabela lua descrevendo o arquivo selecionado. Se o parâmetro "multiple" for true, o número de tabelas neste array varia conforme a quantidade de arquivos selecionados.
- Para uma forma alternativa que não utiliza promessas, veja a função Dialogs.openFile.
- Se o usuário cancelar o pop-up, a promessa retornada resolverá para falha. Devido a esse comportamento, recomendamos usar a função pawait() em vez de await() para aguardar a resolução da promessa.

Exemplos:

- Selecionando múltiplos arquivos de extensão .txt e .html:

```
local promise = Dialogs.asyncOpenFile("Selecione os arquivos", ".txt, .html", true);


promise:thenDo(
    function(files)
        for i = 1, #files, 1 do
```

```
            local file = files[i];
            -- file.name possui o nome do arquivo
            -- file.stream possui o objeto stream
do conteúdo
        end;
    end);
```

- Selecionando 1 arquivo de imagem:

```
local promise = Dialogs.asyncOpenFile("Selecione o
arquivo de imagem", "image/*");


promise:thenDo(
    function(files)
        local file = files[1];
        -- file.name possui o nome do arquivo
        -- file.stream possui o objeto stream do
conteúdo
    end);
```

```
function Dialogs.confirmOkCancel(question [, callback])
```

Exibe uma informação na interface com os botões OK e CANCEL, permitindo o usuário optar por continuar ou por cancelar alguma ação.

Parâmetros:

- question – Uma cadeia de caracteres contendo a mensagem que deverá ser exibida na interface.
- (OPCIONAL) callback - uma função que será chamada após o usuário se decidir. Esta função receberá no primeiro parâmetro o valor **true** se o usuário tiver optado por "OK" ou **false** caso contrário.

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.
- Se quiser saber qual foi a escolha do usuário, utilize o parâmetro "callback".

Exemplo:

```
Dialogs.confirmOkCancel("Continuar irá acontecer X coisa",
        function (confirmado)
                if confirmado then
                        -- usuario escolheu OK
                else
                        -- usuário escolheu CANCEL
                end;
        end);
```

```
function Dialogs.confirmYesNo(question [, callback])
```

Exibe uma informação na interface com os botões YES e NO, permitindo o usuário responder "sim" ou "não" a alguma pergunta.

Parâmetros:

- question – Uma cadeia de caracteres contendo a pergunta que deverá ser exibida na interface.
- (OPCIONAL) callback - uma função que será chamada após o usuário se decidir. Esta função receberá no primeiro parâmetro o valor **true** se o usuário tiver optado por "SIM" ou **false** caso contrário.

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.
- Se quiser saber qual foi a escolha do usuário, utilize o parâmetro "callback".

Exemplo:

```
Dialogs.confirmYesNo("Deseja realmente apagar este item?",
        function (confirmado)
                if confirmado then
```

```
                                -- usuario escolheu
        SIM
                        else
                                -- usuário escolheu
        NAO
                        end;
                end);
```

```
function Dialogs.showMessageDlg(message, dialogType, dialogButtons [,
callback])
```

Exibe um diálogo customizado pelo programador na interface.

Parâmetros:

- message – Uma cadeia de caracteres contendo a mensagem que será exibida na interface
- dialogType - O tipo de diálogo. Pode ser um dos seguintes valores:
  - dialogs.DT_INFORMATION - uma informação
  - dialogs.DT_ERROR - um erro
  - dialogs.DT_WARNING - um alerta
  - dialogs.DT_CONFIRMATION - uma confirmação
- dialogButtons - Quais botões serão exibidos na interface. É um arranjo/tabela lua contendo a combinação dos seguintes valores:
  - dialogs.DB_YES
  - dialogs.DB_NO
  - dialogs.DB_OK
  - dialogs.DB_CANCEL
  - dialogs.DB_ABORT
  - dialogs.DB_RETRY
  - dialogs.DB_IGNORE
  - dialogs.DB_CLOSE
- (OPCIONAL) callback - uma função que será chamada após o usuário se decidir. Esta função receberá no primeiro parâmetro qual botão foi pressionado. (Este valor é um daqueles informados no parâmetro dialogButtons).

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.

- Se quiser saber qual foi a escolha do usuário, utilize o parâmetro "callback".
- Nem toda combinação de botões são válidas. As combinações de botões válidas são:
  - {dialogs.DB_OK}
  - {dialogs.DB_OK, dialogs.DB_CANCEL}
  - {dialogs.DB_YES, dialogs.DB_NO}
  - {dialogs.DB_YES, dialogs.DB_NO, dialogs.DB_CANCEL}
  - {dialogs.DB_ABORT, dialogs.DB_IGNORE, dialogs.DB_RETRY}
  - {dialogs.DB_ABORT, dialogs.DB_IGNORE}

Exemplo:

```
Dialogs.showMessageDlg("O que fazer?",
dialogs.DT_WARNING,
                       {dialogs.DB_ABORT,
dialogs.DB_IGNORE, dialogs.DB_RETRY},
function (escolha)
                           if escolha ==
dialogs.DB_ABORT then
                               -- usuario
escolheu ABORT
                           elseif escolha ==
dialogs.DB_IGNORE then
                               -- usuário
escolheu IGNORE
                           elseif escolha ==
dialogs.DB_RETRY then
                              -- usuário escolheu
RETRY
                           end;
                        end);
```

```
function Dialogs.inputQuery(caption, prompt[, initialValue [, confirmCallback [, cancelCallback,
[, allowEmptyString]]]])
```

Exibe um diálogo para que o usuário preencha alguma informação.

Parâmetros:

- caption – Cadeia de caracteres contendo o título do diálogo
- prompt - Cadeia de caracteres que contém uma orientação para que o usuário saiba o que deve digitar. Exemplo: "Informe seu nome".
- (OPCIONAL) initialValue - Cadeia de caracteres contendo o valor que já virá preenchido no diálogo.
- (OPCIONAL) confirmCallback - Uma função que será invocada após o usuário confirmar o preenchimento. O valor que foi informado pelo usuário é passado no primeiro parâmetro desta função.

- (OPCIONAL) cancelCallback - Uma função que será invocada se o usuário cancelar o diálogo.
- (OPCIONAL) allowEmptyString - Um booleano indicando se o usuário pode informar uma cadeia de caracteres vazia. Se omitido, este parâmetro é interpretado como **false**.

Retorno:
Não retorna nenhum valor.

Observações:

- A função não espera o usuário responder o diálogo para continuar, Isto é, o diálogo é apresentado na interface e o código LUA continua a sua execução normal enquanto isso.
- Se quiser saber qual foi a informação preenchida pelo usuário, utilize os parâmetros "confirmCallback" e/ou "cancelCallback".

Exemplo:

```
Dialogs.inputQuery("Cadastro", "Informe Seu Nome", "",
        function (valorPreenchido)
                showMessage("O usuario preencheu: " .. valorPreenchido);
        end,

        function()
                showMessage("O usuário cancelou");
        end);
```

```
function Dialogs.openFile(prompt[, accept [, multiple [, callback [, cancelCallback]]]])
```

Exibe um diálogo para que o usuário escolha um ou mais arquivos do seu computador/dispositivo para abrir.

Parâmetros:

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre qual arquivo ele deve selecionar. Exemplo: "Escolha uma imagem".
- (OPCIONAL) accept - Uma cadeia de caracteres usada para filtrar quais arquivos o usuário pode selecionar. Você pode passar aqui:

        - ".<EXTENSÃO>" para filtrar arquivos de uma determinada extensão. Exemplo: ".txt", ".mp3", ".png", etc..
        - "<MIME_TYPE>" para filtrar arquivos pelo Media/Mime Type. Exemplos: "image/png", "application/xml", "text/plain", etc..
        - "image/*" para filtrar todos os formatos de imagens suportados pelo RRPG.
        - "*.*" ou nil - Para aceitar qualquer tipo de arquivo.
        - Uma lista das opções anteriores separadas por vírgula. Exemplos:
            - ".png, .txt, .bmp" - para aceitar arquivos de extensões PNG, TXT e BMP
            - ".html, text/plain, image/*" - para aceitar arquivos HTML, arquivos de media type text/plain e também qualquer imagem.
    - (OPCIONAL) multiple - true para permitir o usuário selecionar mais de um arquivo ao mesmo tempo ou false para permitir selecionar apenas um arquivo. Se omitido este valor, o padrão é "false".
    - (OPCIONAL) callback - Uma função que será invocada quando o usuário confirmar a seleção do(s) arquivo(s). Esta função recebe em seu primeiro parâmetro um array de tabelas luas que contém cada uma:
        - name - Uma propriedade contendo o nome curto do arquivo selecionado. Exemplos: "Arquivo.txt", "Minha imagem de férias.png", etc..
        - stream - Um objeto stream somente leitura contendo os dados do arquivo selecionado.
    - (OPCIONAL) cancelCallback - Uma função que será invocada quando o usuário cancelar a seleção de arquivos.

Observações:

- Esta é a única maneira que o plug-in tem de obter acesso a arquivos do dispositivo do usuário.
- Se o parâmetro "multiple" for false, a função "callback" sempre receberá um array contendo 1 tabela lua descrevendo o arquivo selecionado. Se o parâmetro "multiple" for true, o número de tabelas neste array varia conforme a quantidade de arquivos selecionados.
- Para uma forma alternativa que utiliza promessas, veja a função Dialogs.asyncOpenFile.

Exemplos:

- Selecionando múltiplos arquivos de extensão .txt e .html:

```
Dialogs.openFile("Selecione os arquivos",
".txt, .html", true,
        function(arquivos)
                for i = 1, #arquivos, 1 do
                        local arq = arquivos[i];
```

```
                        -- arq.name possui o nome
do arquivo
                        -- arq.stream possui o
objeto stream do conteúdo
                end;
        end);
```

- Selecionando 1 arquivo de imagem:

```
Dialogs.openFile("Selecione o arquivo de imagem",
"image/*", false,
        function(arquivos)
                local arq = arquivos[1];
                -- arq.name possui o nome do
arquivo
                -- arq.stream possui o objeto
stream do conteúdo
        end);
```

```
function Dialogs.saveFile(prompt, stream, [suggestedFileName [, mimeType [, callback [,
cancelCallback]]]])
```

Exibe um diálogo para que o usuário possa salvar um arquivo em seu dispositivo.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre qual arquivo ele vai salvar. Exemplo: "Salvar imagem do personagem".
- stream - Um Um objeto stream contendo o que será salvo no dispositivo do usuário.
- (OPCIONAL) suggestedFileName - Uma cadeia de caracteres contendo uma **sugestão** de nome de arquivo para o usuário. Exemplo: "novoArquivo.txt"
- (OPCIONAL) mimeType - Uma cadeia de caracteres usada para definir o tipo de conteúdo que "stream" possúi.
  - Exemplos:
    - "text/plain"
    - "image/png"
    - "image/jpeg"

        - "application/octet-stream"
    - (OPCIONAL) callback - Uma função que será invocada quando o usuário confirmar a gravação do arquivo
    - (OPCIONAL) cancelCallback - Uma função que será invocada quando o usuário cancelar a gravação do arquivo.

Alerta:

- Após chamar saveFile, não altere e nem feche o objeto "stream" antes que "callback" ou "cancelCallback" seja invocado. O SDK mantém uma referência ao stream até que o usuário decida gravar o arquivo e não é seguro alterá-lo neste meio tempo.

Observações:

- Esta é a única maneira de salvar um arquivo no dispositivo do usuário.
- Recomenda-se informar os parâmetros "mimeType" e "suggestedFileName" quando possível para que o SDK possa interagir com o usuário da melhor forma possível.
    - Se você **não** informar mimeType, o SDK automaticamente o sugere conforme a extensão de suggestedFileName.
    - Se você **não** informar suggestedFileName, o SDK automaticamente sugere a extensão do arquivo conforme mimeType.
- No Android:
    - Esta função apresenta aquela clássifca interface de "compartilhar" do sistema operacional.
    - A função "callback" é invocada imediatamente, não sendo possível saber se o usuário compartilhou ou não o conteúdo de stream.

```
function Dialogs.selectImageURL(defaultURL, [, callback [, cancelCallback]])
```

Exibe um diálogo para que o usuário possa escolher uma URL de uma imagem.

Parâmetros

- defaultURL - Uma cadeia de caracteres contendo uma URL que será sugerida para o usuário. Pode ser **nil** ou vazia.
- (OPCIONAL) callback - Uma função que será invocada quando o usuário confirmar a seleção de uma imagem. O primeiro parâmetro desta função contém a URL escolhida pelo usuário.
- (OPCIONAL) cancelCallback - Uma função que será invocada quando o usuário cancelar a seleção de imagem.

Observações:

- Esta função abre a tradicional interface de escolha de imagens do RRPG.

- Esta função é assíncrona. Ou seja, o código continua sua execução enquanto o usuário escolhe a imagem. A única maneira de obter a URL escolhida pelo usuário é através do parâmetro "callback".

Exemplo:

```
Dialogs.selectImageURL("http://www.google.com/
imagem_sugerida.png",
    function(url)
        showMessage("Imagem selecionada: " .. url);
    end,

    function()
        showMessage("O usuário cancelou a janela");
    end);
```

```
function Dialogs.choose(prompt, options, callback [, defaultOptionIndex, shortCircuit])
```

Exibe um diálogo para que o usuário possa escolher uma opção dentre uma lista de opções.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- options - Um array/arranjo de cadeias de caracteres, contendo as opções que devem ser apresentadas ao usuário.
- callback - Uma função que será invocada quando o usuário fizer a escolha ou quando ele cancelar a escolha.
    - A função recebe três parâmetros na seguinte ordem:
        - selected - Um valor booleano (true ou false) indicando se o usuário selecionou alguma opção ou se cancelou o diálogo
        - selectedIndex - Se o usuário selecionou alguma opção, este parâmetro recebe um número representando o índice do array "options" que representa qual opção o usuário escolheu. Lembre-se que em Lua o índice de arrays começa do número 1 ao invés de 0. Se o usuário cancelou o diálogo, este parâmetro conterá **nil**.
        - selectedText - Se o usuário selecionou alguma opção, este parâmetro recebe o texto da opção escolhida conforme o parâmetro "options". Se o usuário

cancelou o dialogo, este parâmetro conterá **nil**.

- (OPCIONAL) defaultOptionIndex - Um número contendo um índice informado qual dos itens de "options" deve ser apresentado como a escolha padrão. Se não informado, nenhuma opção será mostrada como padrão.
- (OPCIONAL) shortCircuit - Avaliação Curto-Circuito: Se for "true" e se houver apenas uma opção a ser mostrada ao usuario, a interface não é exibida e esta opçao é automaticamente selecionada. Se não informado, este parâmetro assume o valor false.

Observações:

- Esta função é assíncrona. Ou seja, o código continua sua execução enquanto o usuário escolhe. A única maneira de obter a resposta escolhida pelo usuário é através do parâmetro "callback".
- Se "options" não for válido ou se for um arranjo com 0 itens, a função callback será imediatamente chamada como se o usuário tivesse cancelado a escolha.

Exemplo:

```
Dialogs.choose("Selecione uma das opções", {"Opção 1",
"Opção 2", "Opção 3"},

                function(selected, selectedIndex,
selectedText)

                  if selected then

                    showMessage("O usuário selecionou índice
" .. tostring(selectedIndex) .. ": " .. selectedText);

                  else

                    showMessage("O usuário cancelou");

                  end;

                end)
```

```
function Dialogs.chooseMultiple(prompt, options, callback)
```

Exibe um diálogo para que o usuário possa escolher **uma ou mais** opções dentre uma lista de opções.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- options - Um array/arranjo de cadeias de caracteres, contendo as opções que devem ser apresentadas ao usuário.

- callback - Uma função que será invocada quando o usuário fizer a escolha ou quando ele cancelar a escolha.
  - A função recebe três parâmetros na seguinte ordem:
    - selected - Um valor booleano (true ou false) indicando se o usuário selecionou ao menos uma opção ou se cancelou o diálogo
    - selectedIndexes - Se o usuário selecionou alguma opção, este parâmetro recebe um array de números representando os índices do array "options" que representa quais opções o usuário escolheu. Lembre-se que em Lua o índice de arrays começa do número 1 ao invés de 0. Se o usuário cancelou o diálogo, este parâmetro conterá **nil**.
    - selectedText - Se o usuário selecionou alguma opção, este parâmetro recebe um array de textos das opções escolhida conforme o parâmetro "options". Se o usuário cancelou o dialogo, este parâmetro conterá **nil**.

Observações:

- Esta função é assíncrona. Ou seja, o código continua sua execução enquanto o usuário escolhe. A única maneira de obter a resposta escolhida pelo usuário é através do parâmetro "callback".
- Se "options" não for válido ou se for um arranjo com 0 itens, a função callback será imediatamente chamada como se o usuário tivesse cancelado a escolha.

Exemplo:

```
Dialogs.chooseMultiple("Selecione uma ou mais opções",
{"Opção 1", "Opção 2", "Opção 3"},
                function(selected, selectedIndexes,
selectedTexts)
                  if selected then
                    showMessage("O usuário selecionou os
índices " ..
                                tableToStr(selectedIndexes)
.. ": " ..
                                tableToStr(selectedTexts));
                  else
                    showMessage("O usuário cancelou");
                  end;
```

```
end)
```

Biblioteca FireDrive

A biblioteca fireDrive provê funções para manipular e gerenciar o fireDrive do usuário. O FireDrive é um recurso do Firecast que permite armazenar arquivos na nuvem, compartilhar e utilizar arquivos compartilhados por outros usuários.

Todas as funções estão contidas na table/variável "FireDrive" da unidade "fireDrive.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "fireDrive.lua"
require("fireDrive.lua");

-- Agora é possível acessar as funções da biblioteca
FireDrive.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Estrutura FireDriveItem

Algumas funções desta biblioteca retornam um ou mais FireDriveItem(s). Um FireDriveItem é tabela lua contém detalhes de um arquivo/diretório existente no FireDrive do usuário.

O FireDriveItem é uma tabela Lua que contém as seguintes propriedades/atributos:

- id - Número que identifica unicamente este item.
- name - Cadeia de caracteres contendo o nome do arquivo/diretório. Se o caminho do arquivo for "/diretorio/subpasta/arquivo.txt", esta propriedade conterá "arquivo.txt"
- isDir - Um booleano, onde **true** significa que o item é um diretório e **false** significa que o item é um arquivo.
- kind - Uma cadeia de caracteres que identifica o tipo deste item. Pode conter um dos seguintes valores:
  - "dir" - Um diretório
  - "file" - Um arquivo simples
  - "plugin" - Um arquivo de plug-in
- parentId - Id único que identifica em qual diretório este item se encontra. Se o arquivo estiver na raiz do FireDrive (exemplo: "/arquivo.txt"), este atributo contém **nil**.
- size - Número identificando quantos bytes este arquivo ocupa no FireDrive do usuário.
- isShared - Booleano, onde **true** significa que este item está compartilhado com todos do Firecast, ou **false** significando que o item não está acessível para outros.

- isMine - Booleano, onde **true** significa que o usuário criou o item, e **false** significa que o item foi criado por outra pessoa e foi importado.
- sharedId - Número que identifica este item unicamente e globalmente no Firecast. É utilizado para compartilhar arquivos com outras pessoas do programa.
- keyWords - Cadeia de caracteres contendo as palavras chaves do item
- description - Cadeia de caracteres contendo a descrição do item
- url - Se for um arquivo, contém a URL para baixar seus dados.
- mimeType - Se for um arquivo, contém o MIME/Content-Type do arquivo. (Exemplos: "image/jpeg", "image/png", "text/plain", etc..);
- urlThumb64 - Se preenchido, contém A URL de uma imagem thumbnail do item. Este thumbnail cabe em quadrado de 64x64 de forma a manter o aspecto da imagem.

Funções da biblioteca FireDrive

```
function FireDrive.getFiles(directory[, callback[, failureCallback]])
```

Obtém a lista de arquivos e diretórios que estão dentro de uma pasta no FireDrive do usuário.

Parâmetros:

- directory - Uma cadeia de caracteres contendo um caminho de uma pasta que existe no FireDrive do usuário. O conteúdo desta pasta será enumerada. Exemplos:
  - Se quiser enumerar o conteúdo da raiz do FireDrive, utilize "/"
  - Se quiser enumerar o conteúdo da pasta "imagens", utilize "/imagens".
  - Se quiser enumerar o conteúdo da subpasta "medievais" da pasta "imagens", utilize "/imagens/medievais"
- (OPCIONAL) callback - uma função que será chamada quando a lista de arquivos/diretórios tiver sido obtida com sucesso. Esta função recebe em seu primeiro parâmetro um arranjo de FireDriveItem
- (OPCIONAL) failureCallback - uma função que será chamada quando a lista de arquivos/diretórios não puder ser obtida. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros,

você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.

```
function FireDrive.refresh([, callback[, failureCallback]])
```

Durante a execução, o FireDrive mantém algumas informações em cache na memória, como a lista dos arquivos de cada pasta do FireDrive. Esta função força a atualização destas informações que estão guardas em cache na memória.

Parâmetros:

- (OPCIONAL) callback - uma função que será chamada quando a atualização tiver sido executada com sucesso.
- (OPCIONAL) failureCallback - uma função que será chamada quando a atualização falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.

```
function FireDrive.getFileInfo(fileName[, callback[, failureCallback]])
```

Obtém informações de um arquivo ou diretório existente no FireDrive do usuário.

Parâmetros:

- fileName - Uma cadeia de caracteres contendo o caminho de um arquivo/diretório que existe no FireDrive do usuário. Serão retornadas as informações deste arquivo.
- (OPCIONAL) callback - uma função que será chamada quando as informações tiverem sido obtidas com sucesso. Esta função recebe em seu primeiro parâmetro um FireDriveItem com as informações do arquivo ou **nil** caso não exista.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.

```
function FireDrive.upload(destFileName, stream[, successCallback[,
progressCallback[, failureCallback[, mimeType]]]])
```

Efetua o upload de um conteúdo/arquivo para o FireDrive do usuário.

Parâmetros:

- destFileName - Uma cadeia de caracteres contendo o caminho de um arquivo no FireDrive do usuário onde os dados serão salvos.
- stream - Um objeto Stream contendo os dados que serão enviados e salvos no servidor.
- (OPCIONAL) successCallback - uma função que será chamada quando o upload tiver sido feito com sucesso. Esta função recebe em seu primeiro parâmetro um FireDriveItem com as informações do arquivo no servidor.
- (OPCIONAL) progressCallback - uma função que será chamada de tempos em tempos informando o progresso do envio do arquivo. A função recebe 2 parâmetros: currentBytes e maxBytes, representando, respectivamente, quantos bytes já foram enviados e quantos bytes serão enviados no final das contas.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.
- (OPCIONAL) mimeType - Uma cadeia de caracteres contendo o mimeType/contentType do dados. Exemplos: "image/jpeg", "image/png", "text/plain", etc... Se não informado, o fireDrive automaticamente definirá o mimeType com base na extensão do arquivo destino.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "successCallback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.
- Se o arquivo já existir no FireDrive do usuário, esta função sobrescreve seu conteúdo e não reporta falha.

```
function FireDrive.quickUpload(suggestedFileName, mimeType, stream[,
```

```
successCallback[, progressCallback[, failureCallback]]])
```

Efetua o upload de um conteúdo/arquivo para o FireDrive do Firecast de forma descomplicada que será armazenado **temporariamente**.

Parâmetros:

- suggestedFileName - Uma cadeia de caracteres contendo uma **sugestão** de nome de arquivo para ser armazenado.
  Pode ser **nil**: neste caso, o FireDrive criará um nome aleatório para o upload.
- mimeType - Uma cadeia de caracteres contendo o mimeType/contentType do dados. Exemplos: "image/jpeg", "image/png", "text/plain", etc...
  Pode ser passado **nil** como parâmetro: Neste caso, o fireDrive automaticamente definirá o mimeType com base na extensão de "*suggestedFileName*".
- stream - Um objeto Stream contendo os dados que serão enviados e salvos no servidor.
- (OPCIONAL) successCallback - uma função que será chamada quando o upload tiver sido feito com sucesso. Esta função recebe em seu primeiro parâmetro um FireDriveItem com as informações do arquivo no servidor.
- (OPCIONAL) progressCallback - uma função que será chamada de tempos em tempos informando o progresso do envio do arquivo. A função recebe 2 parâmetros: currentBytes e maxBytes, representando, respectivamente, quantos bytes já foram enviados e quantos bytes serão enviados no final das contas.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "successCallback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.
- O upload é armazenado em um espaço temporário do Firecast. Ele não será visto na pasta FireDrive do usuário.
- Lembre-se: Este upload tem características temporárias. O Firecast poderá excluir o upload, sem aviso prévio, após algum tempo.
- Não é possível fazer o upload se ambos os parâmetros "*suggestedFileName*" e "*mimeType*" não forem informados corretamente. Ao menos um deles deve ser informado corretamente.

```
function FireDrive.createDirectory(directoryName, [, callback[,
```

```
failureCallback]])
```

Cria um diretório no FireDrive do usuário.

Parâmetros:

- directoryName - Uma cadeia de caracteres contendo o caminho do diretório que será criado no FireDrive do usuário.
- (OPCIONAL) callback - uma função que será chamada quando o diretório tiver sido criado com sucesso. Esta função recebe em seu primeiro parâmetro um FireDriveItem com as informações do diretório no servidor.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.
- Se o diretório já existir no FireDrive do usuário, ele é preservado no servidor sem alterações e a função "callback" é invocada sinalizando êxito.

```
function FireDrive.delete(destFileName, [, callback[, failureCallback]])
```

Exclui um arquivo ou diretório do FireDrive do usuário.

Parâmetros:

- destFileName - Uma cadeia de caracteres contendo o caminho do arquivo ou diretório existente no FireDrive do usuário que será removido;
- (OPCIONAL) callback - uma função que será chamada quando a operação tiver sido realizada com sucesso.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a

operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.
- Se for removido um diretório, todo seu conteúdo, pastas e sub-pastas serão excluídos juntos com esta operação.

```
function FireDrive.rename(existingFileName, newFileName [, callback[,
failureCallback]])
```

Renomeia/move um arquivo ou diretório do FireDrive do usuário.

Parâmetros:

- existingFileName - Uma cadeia de caracteres contendo o caminho do arquivo ou diretório existente no FireDrive do usuário que será renomeado/movido;
- newFileName - Uma cadeia de caracteres contendo o novo caminho do arquivo/diretório. Este parâmetro deve conter o novo caminho completo do arquivo/diretório.
- (OPCIONAL) callback - uma função que será chamada quando a operação tiver sido realizada com sucesso.
- (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.
- O valor "newFileName" deve conter o novo caminho completo do arquivo. É possível mover o arquivo através deste método, basta especificar uma outra pasta para o parâmetro "newFileName", exemplo: existingFileName="/pasta1/x.txt" e newFileName="/nova_pasta/x.txt".

```
function FireDrive.getSpaceInfo(callback[, failureCallback]);
```

Retorna as informações de espaço do FireDrive do usuário.

Parâmetros:

- callback - Uma função que será invocada quando as informações de espaço do FireDrive forem obtidas. A função recebe em seu primeiro parâmetro uma tabela Lua com as seguintes propriedades:

        - used - Quantidade de bytes usados no armazenamento do FireDrive do usuário
        - total - Quantidade total de bytes que o usuário pode armazenar no FireDrive.
    - (OPCIONAL) failureCallback - uma função que será chamada se a operação falhar. Ela recebe em seu primeiro parâmetro o motivo da falha.

Observações:

- Esta função é assíncrona, isto é, ela retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Se quiser obter o resultado, você deve informar uma função "callback" que será chamada quando a operação em segundo plano for bem sucedida. Se quiser tratar erros, você deve informar uma função "failureCallback" que será chamada quando alguma falha ocorrer.

Biblioteca GUI

A biblioteca gui provê funções relacionadas às interfaces Lua Form.

Todas as funções estão contidas na table/variável "GUI" da unidade "gui.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "gui.lua"
require("gui.lua");

-- Agora é possível acessar as funções da biblioteca
GUI.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca gui

```
function GUI.asyncOpenFirecastURI(uri[, params])
```

Shows a user interface capable of displaying the contents of the specified Firecast URI

Arguments:

- uri - A string containing the Firecast URI to open.
- *(Optional)* params - A Lua table that may contain the following optional fields:
    - alwaysCreate - A Boolean; This flag is used when the specified URI is already open. If **true**, Firecast should

attempt to open yet another instance of the specified URI, while the value **false** means to reuse the existing interface. Default: **false**;
- newTab - A Boolean; **true** means Firecast should try to open the URI in a new tab. Default: **false**;
- autoFocus - A Boolean; **true** means the Firecast should try to focus the open interface, while the value **false** means to open without focusing it. Default: **false**;
- canReplace - A Boolean; **true** means Firecast can use the behavior of replacing another existing open interface with the interface to open. Default: **false**.

Return:

- A Promise object to track the process.

Remarks:

- If the user cancels the open process, the returned promise will be resolved to failure.
- Only Firecast URIs are accepted by this function. To open a generic URL in a browser, use the GUI.openInBrowser() function instead.
- For example, you can obtain Firecast URIs using the "firecastURI" property of Mesa objects or BibliotecaItem objects.
- Some Firecast URIs may require user attention during the opening process, such as joining a room or entering a password to join a room.

Example:

```
GUI.asyncOpenFirecastURI("https://vtt.firecast.app/artifacts/
rooms/1");


GUI.asyncOpenFirecastURI(character.firecastURI,
{alwaysCreate=true,
                                                newTab=true,


autoFocus=true});
```

```
function GUI.findControlByName(controlName,
referenceControl)
```

Procura outro controle através do nome passado pelo parâmetro “controlName” em toda a hierarquia de controles onde "referenceControl" está.

Parâmetros:

- controlName - uma cadeia de caracteres contendo o nome do controle que deseja procurar.
- referenceControl - um controle para servir de referência durante a busca. O controle será procurado em toda a hierarquia de controles de "referenceControl", isto é, primeiro nos controles que estão dentro dele e depois nos controles pais de "referenceControl".

Retorno:

- o controle, se encontrado
- **nil** caso não encontre.

```
function GUI.getShiftState()
```

Return the current keyboard shift state.

Arguments:

None

Return

- A Lua table with the following fields:
  - shiftKey - A boolean; true if the "shift" key is pressed.
  - ctrlKey - A boolean; true if the "control" key is pressed.
  - altKey - A boolean; true if the "alt" key is pressed (also known as "menu" key).
  - cmdKey - A boolean; true if the "command" key is pressed (macOS device).
  - ctrlOrCmdKey - A boolean; true if the "command" key is pressed in a macOS device; otherwise, true if the "control" key is pressed in a non-macOS device

```
function GUI.newActivityIndicator()
```

Cria e retorna um novo objeto activityIndicator

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto activityIndicator

```
function GUI.newButton()
```

Cria e retorna um novo objeto button.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto button.

```
function GUI.newCheckBox()
```

Cria e retorna um novo objeto checkbox.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto checkbox.

```
function GUI.newCol()
```

Cria e retorna um novo objeto col.

Parâmetros:

Não há parâmetros

Retorno:
Um objeto col.

```
function GUI.newComboBox()
```

Cria e retorna um novo objeto comboBox

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto comboBox

```
function GUI.newContainer()
```

Cria e retorna um novo objeto container.

Parâmetros:

Não há parâmetros

Retorno:
Um objeto container..

```
function GUI.newColorComboBox()
```

Cria e retorna um novo objeto colorComboBox

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto colorComboBox

```
function GUI.newDataLink()
```

Cria e retorna um novo objeto dataLink

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto dataLink

```
function GUI.newDataScopeBox()
```

Cria e retorna um novo objeto dataScopeBox

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto dataScopeBox

```
function GUI.newEdit()
```

Cria e retorna um novo objeto edit

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto edit

```
function GUI.newFlowLayout()
```

Cria e retorna um novo objeto flowLayout.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto flowLayout.

```
function GUI.newFlowLineBreak()
```

Cria e retorna um novo objeto flowLineBreak.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto flowLineBreak.

```
function GUI.newFlowPart()
```

Cria e retorna um novo objeto flowPart.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto flowPart.

```
function GUI.newForm([formName])
```

Cria e retorna um novo objeto form.

Parâmetros:

- (OPCIONAL) formName - Se informado, contém uma cadeia de caracteres identificando o "name" de um dos Lua Forms do plug-in, fazendo com que a função busque em todos os arquivos ".lfm" a tag form ou tag popupForm que possuir o atributo name igual a este valor.

Retorno:

- Se "formName" **não** for informado: Retorna um novo objeto form que não possui nenhum outro componente/tag dentro dele.
- Se "formName" for informado:
  - Se a busca encontrar o LuaForm, retorna um objeto form representando a nova instância do Lua Form.
  - Se a busca não encontrar o LuaForm, retorna **nil**.

```
function GUI.newGridRecordList()
```

Cria e retorna um novo objeto gridRecordList.

Parâmetros:

Não há parâmetros

Retorno:
Um objeto gridRecordList.

```
function GUI.newGridRecordListPgNav()
```

Cria e retorna um novo objeto gridRecordListPgNav.

Parâmetros:

Não há parâmetros

Retorno:
Um objeto gridRecordListPgNav.

```
function GUI.newHorzLine()
```

Cria e retorna um novo objeto horzLine

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto horzLine

```
function GUI.newImage()
```

Cria e retorna um novo objeto image

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto image.

```
function GUI.newImageCheckBox()
```

Cria e retorna um novo objeto imageCheckbox.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto imageCheckbox.

```
function GUI.newLabel()
```

Cria e retorna um novo objeto label.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto label.

```
function GUI.newLayout()
```

Cria e retorna um novo objeto layout.

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto layout.

```
function GUI.newPath()
```

Cria e retorna um novo objeto path

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto path

```
function GUI.newPopup()
```

Cria e retorna um novo objeto popup

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto popup

```
function GUI.newPopupForm([popupFormName])
```

Cria e retorna um novo objeto popupForm.

Parâmetros:

- (OPCIONAL) popupFormName - Se informado, contém uma cadeia de caracteres identificando o "name" de um dos Lua Forms do plug-in, fazendo com que a função busque em todos os arquivos ".lfm" a tag popupForm que possuir o atributo name igual a este valor.

Retorno:

- Se "popupFormName" **não** for informado: Retorna um novo objeto popupForm que não possui nenhum outro componente/tag dentro dele.
- Se "popupFormName" for informado:
  - Se a busca encontrar o LuaForm e se for do tipo popupForm, retorna um objeto popupForm representando a nova instância do Lua Form.
  - Se a busca não encontrar o LuaForm ou se encontrar mas tipo diferente de popupForm, retorna **nil**.

```
function GUI.newRadioButton()
```

Cria e retorna um novo objeto radioButton

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto radioButton

```
function GUI.newRecordList()
```

Cria e retorna um novo objeto recordList

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto recordList

```
function GUI.newRectangle()
```

Cria e retorna um novo objeto rectangle

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto rectangle

```
function GUI.newRow()
```

Cria e retorna um novo objeto row.

Parâmetros:

Não há parâmetros

Retorno:
Um objeto row.

```
function GUI.newScrollBox()
```

Cria e retorna um novo objeto scrollBox

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto scrollBox

```
function GUI.newTab()
```

Cria e retorna um novo objeto tab

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto tab

```
function GUI.newTabControl()
```

Cria e retorna um novo objeto tabControl

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto tabControl

```
function GUI.newTextEditor()
```

Cria e retorna um novo objeto textEditor

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto textEditor

```
function GUI.newTimer()
```

Cria e retorna um novo objeto timer

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto timer

```
function GUI.newProgressBar()
```

Cria e retorna um novo objeto progressBar

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto progressBar

```
function GUI.newInertialMovement()
```

Cria e retorna um novo objeto InertialMovement

Parâmetros:

Não há parâmetros

Retorno:
Retorna um objeto InertialMovement

```
function GUI.showPopup(form [, options])
```

Exibe uma janela na interface do usuário na forma de um Popup

Parâmetros:

- form - O formulário a ser exibido na interface. Pode ser um objeto Lua Form já instanciado ou um string contendo o nome do form a ser exibido.
  - Caso seja passado o nome do formulário em uma cadeia de caracteres, a função percorre todos os "lfm" do plug-in à procura da tag form que possuir name igual ao informado. Se encontrado, uma nova instância é criada para a exibição.
- (OPCIONAL) options - Uma tabela/objeto LUA contendo as opções da exibição do popup. As opções são:
  - onClose - [function, padrão = nil] - Uma função lua que será invocada quando o popup for fechado.
  - placement - [string enumerado] - Onde o popup será exibido.. Os possíveis valores são:
    - "center" - no centro da tela
    - "bottom" - na parte de baixo da tela
    - "top" - na parte de cima da tela
    - "left" - à esquerda na tela
    - "right" - à direita na tela
    - "topLeft" - no canto superior esquerdo da tela
    - "topRight - no canto superior direito da tela
    - "bottomLeft" - no canto inferior esquerdo da tela
    - "bottomRight" - no canto inferior direito da tela
    - "mouse" - onde o mouse está atualmente
    - "mouseCenter" - centralizado onde o mouse está atualmente.

Retorno:
Se conseguir exibir o form como um popup na interface, retorna **true**; Senão retorna **false**.

Observações:

- A função não espera o popup fechar para retornar. Após esta chamada, o código LUA continuará sua execução normal enquanto o form é mostrado.
- Se o parâmetro *form* for uma tag popupForm, os seguintes parâmetros em *options* são ignorados (e são usados os atributos correspondentes da tag no lugar):
  - *options*.placement

```
function GUI.closePopup(form)
```

Fecha uma janela popup que foi previamente aberta pela da função gui.showPopup.

Parâmetros:

- form - O formulário a ser fechado/tirado da interface. Pode ser um objeto Lua Form já instanciado ou um string contendo o nome do form que foi exibido.
  - Caso seja passado o nome do formulário em uma cadeia de caracteres, a função percorre todos os popups abertos e fecha àqueles que possuírem o nome informado.

Retorno:
Se conseguir fechar o form, retorna **true**; Senão retorna **false**.

```
function GUI.openInBrowser(url)
```

Abre um determinado URL/página web no browser do usuário.

Parâmetros:

- url - cadeia de caracteres contendo a URL/caminho da página web que deve ser aberta. Exemplo "http://www.rrpg.com.br".

Observações:

- Se o parâmetro "url" não começar com "http://" e nem com "https://", automaticamente "http://" será adicionado no inicio da URL antes da abertura da página.

```
function GUI.toast(message)
```

Show a quick informative message to the user that does not require user interaction and hides automatically after a short time

Parameters

- message - The message string to be shown

Objeto InertialMovement

Este objeto é responsável por calcular movimentos inerciais em janelas e scrolls de janela. Movimento Inercial é aquele movimento produzido quando você "joga" o dedo numa tela touchscreen e a tela continua "scrollando" por conta própria.

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
active	Boolean	Define se o objeto está ativo ou não. Seu valor padrão é **true** e quando for **false** não invoca eventos.
animated	Boolean	**true** - O objeto irá calcular movimentações inerciais **false** - O objeto não irá calcular movimentações inerciais. Valor padrão: true
decelerationRate	Double	Um número que define a desaceleração do movimento inercial. O valor padrão é 5.75 e quanto maior este número, mais rápido o movimento inercial chega ao fim.

Métodos

Método	Descrição
inertialMov:setPos(x, y)	Define onde se encontra a posição do item/scroll.

	Parâmetros: ● x - um número contendo a posição no eixo X. ● y - um número contendo a posição no eixo Y.
inertialMov:getPos()	Retorna onde está a posição atual do item/scroll. Retorna: ● Dois números, o primeiro contendo a posição no eixo X e o segundo contendo a posição no eixo Y. Exemplo de uso: local posX, posY = inertialMov:getPos();
inertialMov:setBounds(minX, minY, maxX, maxY)	Define os limites de posição do item/scroll. Parâmetros: ● minX - Posição mínima no eixo X ● minY - Posição mínima no eixo Y ● maxX - Posição máxima no eixo X ● maxY - Posição máxima no eixo Y
inertialMov:pointerDown(x, y)	Invoque este método quando o mouse/touch for pressionado em algum lugar. Parâmetros: ● x - um número contendo a posição no eixo X do clique/toque. ● y - um número contendo a posição no eixo Y do clique/toque.
inertialMov:pointerMove(x, y)	Invoque este método quando o mouse/touch se mover pela tela. Parâmetros: ● x - um número contendo a NOVA posição no eixo X do mouse/dedo. ● y - um número contendo a NOVA posição no eixo Y do mouse/dedo.
inertialMov:pointerUp(x, y)	Invoque este método quando o botão do mouse for solto/quando o dedo soltar da tela.

	Parâmetros: ● x - um número contendo a onde no eixo X o mouse/toque foi liberado. ● y - um número contendo a onde no eixo Y o mouse/toque foi liberado.
inertialMov:pointerLeave()	Invoque este método quando o mouse/toque sair dos limites do controle da tela.
inertialMov:pointerWheel(deltaX, deltaY)	Invoque este método quando o usuário rolar a rodinha do mouse. Parâmetros: ● deltaX - um número contendo a variação do eixo X, isto é, o quanto se pretende scrollar no eixo X. ● deltaY - um número contendo a variação do eixo Y, isto é, o quanto se pretende scrollar no eixo Y.

Eventos

Nome do evento	Descrição
onChange	Este evento é invocado quando o objeto possuir novas coordenadas para seu item/scroll. Utilize o método "getPos" para descobrir a nova posição.

Exemplos

Objeto Drag

Este objeto é responsável por conter informações da operação arrastar-e-soltar em operação.

Leia mais sobre a operação Arraste e Solte / Drag And Drop.

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
dataCount	Inteiro	(Somente Leitura) Contém a quantidade de dados associados neste Drag/Arraste.
actionCount	Inteiro	(Somente Leitura) Contém a quantidade de ações associadas neste Drag/Arraste.

Métodos

Método	Descrição
drag:addData(dataType, dataValue)	Adiciona um dado a este Drag/Arraste. Parâmetros: ● dataType - Uma cadeia de caracteres que define o tipo de dados. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType ● dataValue - O valor associado ao dataType
drag:getData(dataType)	Retorna um dado deste Drag/Arraste. Parâmetros: ● dataType - Uma cadeia de caracteres que define o tipo de dados. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType Retorno: ● O valor associado ao dataType ou **nil** se não possuir associação.
drag:addAction(dataType, callback)	Adiciona uma ação a este Drag/Arraste. Parâmetros:

	● dataType - Uma cadeia de caracteres que define o tipo de dados associado a esta ação. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType ● callback - Uma função que será invocada caso o usuário arraste este drag a uma área de drop que contenha informação do tipo "dataType" nela. A função recebe os seguintes 2 parâmetros na ordem: ○ value - O valor da informação provida pela área de drop. ○ drop - Objeto Drop que contém as informações da área de drop combinada.

Eventos

Nome do evento	Descrição

Exemplos

Objeto Drop

Este objeto é responsável por conter informações da área de drop em uma operação arraste e solte / drag and drop.

Leia mais sobre a operação Arraste e Solte / Drag And Drop.

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
dataCount	Inteiro	(Somente Leitura) Contém a

		quantidade de dados associados nesta área de Drop.
actionCount	Inteiro	(Somente Leitura) Contém a quantidade de ações associadas nesta área de Drop.

Métodos

Método	Descrição
drop:addData(dataType, dataValue)	Adiciona um dado a esta área de drop. Parâmetros: ● dataType - Uma cadeia de caracteres que define o tipo de dados. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType ● dataValue - O valor associado ao dataType
drop:getData(dataType)	Retorna um dado desta área de drop. Parâmetros: ● dataType - Uma cadeia de caracteres que define o tipo de dados. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType Retorno: ● O valor associado ao dataType ou **nil** se não possuir associação.

drop:addAction(dataType, callback)	Adiciona uma ação a esta área de drop. Parâmetros: ● dataType - Uma cadeia de caracteres que define o tipo de dados associado a esta ação. Existem alguns dataType que são padrões do RRPG mas você pode inventar um dataType novo, se for o caso. Não deixe de ler "Arrastando e Soltando Informações / Drag And Drop" para mais informações sobre o DataType ● callback - Uma função que será invocada caso o usuário arraste uma informação que contenha "dataType" nela. A função recebe os seguintes 4 parâmetros na ordem: ○ value - O valor da informação que está sendo arrastada ○ x - A posição na coordenada x de onde a informação está sendo arrastada ○ y - A posição na coordenada y de onde a informação está sendo arrastada ○ drag - Objeto Drag que contém a informação arrastada

Eventos

Nome do evento	Descrição

Exemplos

Biblioteca Locale

Biblioteca que contém funções para lidar com idiomas e regiões, incluindo funcionalidade para permitir um plug-in em várias línguas

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "locale.lua"
require("locale.lua");

-- Agora é possível acessar as funções da biblioteca
Locale.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Locale

```
function lang(txtId)
ou
function LANG(txtId)
ou
function Locale.lang(txtId)
```

Percorre todos os arquivos ".lang" do plug-in à procura de um texto cuja identificação seja igual à passada como parâmetro e que combine com o idioma atual do usuário.

Esta função é a principal para você obter strings correto para o idioma do usuário.

Parâmetros:

- txtId - Uma cadeia de caracteres que identifica o texto que deseja obter

Retorno:

- Uma cadeia de caracteres contendo o texto para o idioma do usuário.

Observações:

- A busca ocorre em todos os arquivos de extensão .lang de forma eficiente.
- Caso a busca não encontre o texto para o idioma do usuário, a busca retornará uma cadeia de caracteres de outro idioma. A função emite um erro se "txtId" não for conhecido em nenhum idioma.
- Leia o tutorial de como criar um plug-in multi-línguas para exemplo!

```
function Locale.loadLangTexts(langTexts)
```

Carrega, em tempo de execução, um arquivo ".lang" para que mais textos traduzidos possam ser encontrados pelas funções Locale.lang e Locale.tryLang.

Parâmetros:

- langTexts - Uma cadeia de caracteres com o conteúdo do arquivo .lang que será carregado em tempo de execução.

Observações:

- Leia o tutorial de como criar um plug-in multi-línguas para exemplo!

```
function Locale.newLocalizedTexts()
```

Creates an isolated context to work with translations without affecting the plugin translations.

Arguments:

None

Return:

- A new LocalizedTexts object

```
function tryLang(txtId)
ou
function tryLANG(txtId)
ou
function Locale.tryLang(txtId)
```

Idêntico à função lang, porém retorna **nil** ao invés de um erro quando não encontra a tradução.

LocalizedTexts object

This object represents an isolated context for working with translations without modifying the plugin translations itself.

You can create a new instance of this object by calling Locale.newLocalizedTexts()

Object characteristics

Methods

Método	Descrição
obj:addText(localeId, textId, text)	Add a single translation entry to the LocalizedTexts object. Arguments: - localeId - A string

	containing the identification of the locale, in BCP 47 format, in which this text should be used. Examples: "pt", "pt-BR", "en", "en-US", and so on. **Nil** or empty string means undefined. ○ textId - A string containing the identification of the text. This textId is used when searching translations with translations with obj:lang() and/or obj:tryLang(). ○ text - A string containing the translated text.
obj:lang(textId)	Search for a translation to be used for the current user's language. Arguments: ○ textId - A string containing the identification of the text. Returns: ○ A string containing the translated text. Remarks: ○ If the search does not find text for the user's language, the function will return a string from another language. ○ The function raises an error if "textId" does not exist in any language.
obj:loadTexts(langTexts)	Loads a ".lang" file so that more translated text

	can be found by the obj:lang() and obj:tryLang() methods. Arguments: ○ langTexts - A string with the contents of the .lang file that will be loaded Remarks: ○ Read the tutorial on how to create a multi-language plugin for examples.
obj:tryLang(textId)	This function works similarly to the obj:lang() method, except it does not raise an error if the text entry does not exist. In such a case, a **nil** value is returned.

Log library

The Log library provides functions for emmiting log messages, which can be helpful to developers. All of these functions are contained in the "Log" Lua table found in the "log.lua" unit.

Usage example:

```
-- First, you need to use the "log.lua" unit
require("log.lua");

-- Now you can access the functions of the library.
Log.LIBRARY_FUNCTION(Argument1, Argument2, ...);
```

Where the log messages goes to?

The log messages emmited by this library are sent to:

- The Firecast's console. Use the /console command to reveal the console.
- The Firecast's debug log files (also known as "LogErrors")

Log library functions

```
function Log.e(tag, msg)
function Log.w(tag, msg)
function Log.i(tag, msg)
function Log.d(tag, msg)
function Log.v(tag, msg)
```

Emmit log message

Arguments:

- tag - Used to identify the source of a log message. It usually identifies the location or activity where the log call occurs.
- msg - The message you would like to be logged

Log severity:

Log.e	**Error message** A situation that represents an unhandled error that may interrupt some operation
Log.w	**Warning message** A situation that can be handled but may cause some unintended behavior
Log.i	**Informational/Notice message** A normal but significant condition that may require special handling.
Log.d	**Debug message** A developer debug message
Log.v	**Verbose message** Algorithm steps message, very low priority messages. Remarks: ● Messages of this severity are not emitted in the Firecast's log files ● In the Firecast's console, you need to execute the command "filter add *" to be able to see these messages.

Biblioteca Internet

Biblioteca que contém funções relacionadas à internet/intranet, como: downloads, requisições HTTP, etc..

Todas as funções estão contidas na unidade "internet.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "internet.lua"
require("internet.lua");

-- Agora é possível acessar as funções da biblioteca
Internet.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Internet

```
function Internet.asyncDownload(url[, cacheMode[, progressCallback]])
```

Asynchronously downloads a file from the internet.

Arguments:

- url - String containing the URL address of the file, like "http://www.website.com/directory/file.png".
- *(Optional)* cacheMode - One of the following strings:
  - "alwaysUseCache" - If the file already exists in the cache, Firecast will return the content without even communicating with the internet. This is the **default** mode.
  - "checkForModification" - If the file already exists in the cache, Firecast will check the remote server for changes to the content before deciding to use the local cache or download the content.
  - "alwaysDownload" - Firecast will not use the local cache and will always download content from the remote server.
- *(Optional)* progressCallback - A function that will be called multiple times to report the download progress. This function takes 2 parameters:
  - downloadedBytes - How many bytes have already been downloaded.
  - totalBytes - How many bytes the file has in total.

Return:

- A Promise for two pieces of information in the following order:
  - A Stream object with the downloaded file content.
  - A String containing the content type / mime type of the downloaded file, like 'image/png', 'text/html', and so on

Remarks:

- Internet URL addresses must start with "http://" or "https://".
- File addresses that are located inside the VHD (plugin's virtual HD) are also accepted.
- If you **explicitly** return **false** in a progressCallback call, the download will be aborted.
- The download process is queued in the Firecast download manager and therefore the download may experience a wait before actually starting.

Example:

```
local promise =
Internet.asyncDownload("https://www.address.com/file.txt");

local stream, contentType = await(promise);

showMessage(contentType .. ": " .. tostring(stream.size));
```

```
function Internet.download(url [, finishCallback [, errorCallback [, progressCallback [,
cacheMode]]]])
```

Realiza o download de um arquivo da internet.

Parâmetros:

- url - Cadeia de caracteres contendo a URL do arquivo (Exemplo: http://www.site.com.br/pasta/arquivo.png).
  - URL de arquivos que estão na internet devem começar com "http://" ou "https://"
  - Endereços de arquivos que estão localizados no HD virtual do plug-in também são aceitos.

- (OPCIONAL) finishCallback - Uma função que será invocada quando o download estiver terminado. Esta função recebe os seguintes 2 parâmetros:
  - stream - Um objeto Stream possuindo o conteúdo do arquivo baixado.
  - contentType - Uma cadeia de caracteres identificando o MimeType/tipo do conteúdo.. Exemplos: "application/octet-stream", "image/png", "text/html", etc..

- (OPCIONAL) errorCallback - Uma função que será invocada se ocorrer algum erro no download. A Mensagem de erro é passada no primeiro

parâmetro desta função.

- (OPCIONAL) progressCallback - Uma função que será invocada várias vezes para informar o progresso do download. Esta função recebe 2 parâmetros:
  - downloadedBytes - Quantos bytes já foram baixados.
  - totalBytes - Quantos bytes o arquivo possui no total.

- (OPCIONAL) cacheMode - Pode ser passado um dos seguintes valores e significados:
  - "alwaysUseCache" - Se o arquivo já existir no cache, o Firecast irá retornar o conteúdo sem mesmo se comunicar com a internet. Este é o modo **padrão**.
  - "checkForModification" - Se o arquivo já existir no cache, o Firecast irá verificar no servidor remoto se houve alterações no conteúdo antes de decidir usar o cache local ou baixar o conteúdo.
  - "alwaysDownload" - O Firecast não usará o cache local e sempre baixará o conteúdo do servidor remoto.

Retorno:

- A função retorna um identificador de download. Este valor pode ser utilizado para invocar o método Internet.stopDownload caso queira abortar o download.

Observações:

- A função não espera o download terminar e continua a execução do código enquanto o download é feito em paralelo.
- Para ter acesso ao conteúdo baixado, utilize o parâmetro finishCallback.
- Invocar esta função faz com que a URL seja enfileirada no gerenciador de downloads do Firecast, e, portanto, o download pode passar por uma espera antes de realmente começar.
- Retornar explicitamente "false" na função progressCallback faz com que o download seja abortado.
- Este método utiliza um sistema de cache de arquivos para evitar usar a internet com arquivos que já foram recentemente baixados.
- Se o arquivo estiver pronto no sistema de cache de arquivos, a função "finishCallback" poderá ser invocada imediatamente, antes mesmo da função "Internet.download" retornar.
- Este método, quando possível, faz "resume", isto é, ele consegue fazer download à partir de um ponto que havia parado recentemente.

Exemplo 1:

```
Internet.download("http://www.rrpg.com.br/
arquivo.png",
	function(stream, contentType)
		-- esta função será chamada
quando o download terminar
		-- o conteúdo do arquivo baixado
está em stream.
	end,

	function (errorMsg)
		-- esta função será chamada
quando ocorrer algum erro no download.
		-- errorMsg possui a msg de erro
	end,

	function (downloaded, total)
		-- esta função será chamada
constantemente.
		-- dividir "downloaded" por
"total" lhe dará uma porcentagem do download.
	end);
```

Exemplo 2 - checkForModification

```
Internet.download("http://www.rrpg.com.br/
arquivo.png",
	function(stream, contentType)
		-- esta função será chamada
quando o download terminar
		-- o conteúdo do arquivo baixado
está em stream.
	end,

	function (errorMsg)
		-- esta função será chamada
quando ocorrer algum erro no download.
		-- errorMsg possui a msg de erro
	end,
```

```
                function (downloaded, total)
                        -- esta função será chamada
        constantemente.
                        -- dividir "downloaded" por
        "total" lhe dará uma porcentagem do download.
                end, "checkForModification");
```

```
function Internet.stopDownload(downloadId)
```

Aborta um download que foi iniciado pela função Internet.download

Parâmetros:

- downloadId - a identificação do download a ser abortado. Este valor é retornado pela função Internet.download

```
function Internet.newHTTPRequest([method [, url]])
```

Cria e retorna um objeto HTTPRequest, usado para troca de dados com um servidor através de protocolo HTTP ou HTTPS.

Parâmetros:

- (OPCIONAL) method - Uma cadeia de caracteres identificando o método/verbo HTTP utilizado nesta requisição. Os valores aceitos são:
  - "GET" (valor padrão, se o parâmetro for omitido)
  - "POST"
  - "PUT"
  - "PATCH"
  - "TRACE"
  - "HEAD"
  - "OPTIONS"
  - "DELETE"
- (OPCIONAL) url - A URL da requisição HTTP. Este valor pode ser alterado posteriormente no objeto HTTPRequest.

Retorno:

- Um objeto HTTPRequest representando a requisição.

Observações:

- As requisições HTTP são permitidas se ao menos uma das seguintes afirmativas for verdadeira:
  - O usuário atual é assinante Gold/Platinum/Ruby; ou

        - O usuário atual está em uma mesa cujo criador é assinante Platinum.
    - Esta função apenas cria o objeto HTTPRequest e não a envia. Você deve utilizar o método "send" do objeto para, de fato, enviar a requisição.

```
function Internet.httpEncode(str)
```

Codifica um texto para que possa ser usado como URL em requisições HTTP.

Parâmetros:

- str - A cadeia de caracteres que deseja codificar.

Retorno:

- Uma cadeia de caracteres contendo o texto codificado.

```
function Internet.httpDecode(str)
```

Decodifica os caracteres/códigos de URL contidas no texto passado.

Parâmetros:

- str - A cadeia de caracteres que deseja decodificar.

Retorno:

- Uma cadeia de caracteres contendo o texto decodificado.

Objeto HTTPRequest

O Objeto HTTPRequest é utilizado para a troca de dados com um servidor usando protocolo HTTP ou HTTPS.

É excelente para integrar o plug-in com alguma api REST, baixar ou enviar informações para um servidor da World Wide Web.

Para instanciar um Objeto HTTPRequest, utilize a função internet.newHTTPRequest

Restrições:

- As requisições HTTP são permitidas apenas se o usuário do rrpg for um membro GOLD.

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
url	String	Define a URL para onde a requisição HTTP será direcionada. Exemplo: ● "http://www.google.com/xx/document.txt" ● "https://www.rrpg.com.br/apiexemplo" A URL deve iniciar com "http://" ou "https://"
method	String	Define o método/verbo HTTP que será utilizado na requisição. Os possíveis valores são: ● "GET" (padrão) ● "POST" ● "PUT" ● "PATCH" ● "TRACE" ● "HEAD" ● "OPTIONS" ● "DELETE"
status	Integer	Um número contendo o status da resposta http. (Exemplos: 200 para ok, 404 para não encontrado, etc..) Consulte http://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html para saber quais são os possíveis status.
statusText	String	Uma cadeia de caracteres contendo o texto-status-resposta da requisição HTTP. Diferente da propriedade "status", este valor contém o texto completo da resposta (Exemplos: "200 OK", "404 NOT FOUND").
responseText	String	Contém o corpo da resposta interpretada como texto. O "charset" do cabeçalho de resposta

		"Content-Type" é levado em consideração antes de retornar o texto.
responseStream	Objeto Stream	Contém o corpo da resposta em um objeto stream. Muito útil quando a resposta dada pelo servidor for binária.

Métodos

Método	Descrição
request:send([content])	Inicia o envio da requisição HTTP. Antes de chamar esta função, a URL e todos os cabeçalhos devem ser informados pela função setRequestHeader. Parâmetros: ○ (OPCIONAL) content - Uma cadeia de caracteres ou um objeto stream contendo os dados que serão enviado ao servidor. Se não informado, a requisição é feita com corpo vazio. Observações: ○ A requisição é assíncrona. Isto é, a função send retorna imediatamente e o código LUA continua sua execução normal enquanto a comunicação com o servidor é feita em segundo plano. Para obter a resposta, manipule os eventos onResponse e onError antes de chamar este método. ○ Se o parâmetro content for uma cadeia de caracteres, o "charset" do cabeçalho de requisição "Content-Type" é levado em

	consideração para a codificação do texto.
request:setRequestHeader(header, value)	Altera o valor de um cabeçalho de requisição HTTP. Você deve enviar setRequestHeader antes de invocar o método send. Parâmetros: ○ header - Cadeia de caracteres representando o nome do cabeçalho. Exemplos: "Content-Type" , "Content-Encoding", etc.. ○ value - Cadeia de caracteres com o valor que deve ser associado ao cabeçalho. Observações: ○ Para saber mais sobre os cabeçalhos HTTP, veja http://www.w3.org/Protocols/rfc2616/rfc2616-sec14.html. ○ Se este método for chamado várias vezes alterando o mesmo cabeçalho, o valor da última chamada prevalecerá.
request:getRequestHeader(header)	Obtém o valor de um cabeçalho de requisição HTTP. Parâmetros: ○ header - cadeia de caracteres representando o nome do cabeçalho. Retorno: ○ Uma cadeia de caracteres, contendo o valor do cabeçalho.

request:getResponseHeader(header)	Obtém o valor de um cabeçalho de resposta da requisição HTTP. Parâmetros: ○ header - cadeia de caracteres representando o nome do cabeçalho de resposta. Retorno: ○ Uma cadeia de caracteres, contendo o valor do cabeçalho, ou "" (cadeia de caracteres vazia) se o cabeçalho não existir na resposta. ○ Para saber mais sobre os cabeçalhos HTTP, veja http://www.w3.org/Protocols/rfc2616/rfc2616-sec14.html.
request:getAllResponseHeaders()	Retorna uma cadeia de caracteres contendo todos os cabeçalhos da resposta HTTP de uma só vez.
request:abort();	Aborta a requisição HTTP que está sendo feita.
request:open(method, url);	Inicializa a requisição HTTP Parâmetros: ○ method - Uma cadeia de caracteres identificando o método/verbo HTTP utilizado nesta requisição. Os valores aceitos são: ■ "GET" ■ "POST" ■ "PUT" ■ "PATCH" ■ "TRACE" ■ "HEAD" ■ "OPTIONS"

	■ "DELETE" ○ url - A URL da requisição HTTP Observação: ○ O uso deste método não é obrigatório. É apenas um "doce" para aqueles que estão acostumados com a requisição HTTP do JavaScript.

Eventos

Nome do evento	Descrição
onResponse	Este evento é invocado quando a requisição HTTP concluir sem erros. Após este evento, as propriedades de resposta estarão disponíveis no objeto (como, por exemplo, status, statusText, responseText, etc..). Observação: ○ Se o servidor retornar status 4xx (algum erro de requisição) ou 5xx (algum erro do servidor), o evento "onError" será disparado ao invés de "onResponse"
onError	Este evento é disparado quando ocorrer um erro na requisição HTTP. Parâmetros: ○ errorMsg - Uma cadeia de caracteres contendo informações sobre o erro. Observação: ○ Se o servidor retornar status 4xx (algum erro de requisição) ou 5xx (algum erro do servidor), o evento "onError" será disparado ao invés de "onResponse"

onReceiveProgress	Evento que é disparado frequentemente a fim de informar o progresso de download da requisição HTTP. Parâmetros: ○ currentBytes - Um número indicando quantos bytes já foram baixados ○ maxBytes - Um número indicando quantos bytes serão baixados no final das contas.
onSendProgress	Evento que é disparado frequentemente a fim de informar o progresso de upload da requisição HTTP. Parâmetros: ○ currentBytes - Um número indicando quantos bytes já foram enviados. ○ maxBytes - Um número indicando quantos bytes serão enviados no final das contas.

Exemplos

Exemplo 1 - Baixar o "index.html" de um site.

```
require("internet.lua");

local requisicao = internet.newHTTPRequest("GET",
"http://www.rrpg.com.br/");

requisicao.onResponse =
        function()
                showMessage("Conteúdo: " .. requisicao.responseText);
        end;

requisicao.onError =
        function(errorMsg)
                showMessage("Ops! Ocorreu algum erro: " .. errorMsg);
        end;

requisicao:send();
```

Exemplo 2 - Fazer um upload de um arquivo, receber outro arquivo, rastreando o progresso.

```
require("internet.lua");
require("vhd.lua");

local requisicao = internet.newHTTPRequest("POST",
"http://www.rrpg.com.br/REST_API/URL");

requisicao.onResponse =
        function()
                local streamResposta = requisicao.responseStream;
                local tipoRetornado = requisicao:getResponseHeader("Content-
Type");
                local arquivoDestino;

                if tipoRetornado == "image/png" then
                        arquivoDestino = vhd.openFile("/baixado.png", "w+");
                else
                        arquivoDestino = vhd.openFile("/baixado.outraExtensao",
"w+");
                end;

                arquivoDestino:copyFrom(streamResposta, streamResposta.size);
                arquivoDestino:close();
        end;

requisicao.onError =
        function(errorMsg)
                showMessage("Ops! Ocorreu algum erro: " .. errorMsg);
        end;

requisicao.onSendProgress =
                function(currentBytes, maxBytes)
                        local porcentagem;

                        if maxBytes ~= 0 then
                                porcentagem = (currentBytes / maxBytes) * 100;
                        else
                                porcentagem = 0;
                        end;
```

```
                        --- porcentagem contém quantos % do arquivo já foram
enviados
                end;

requisicao.onReceiveProgress =
                function(currentBytes, maxBytes)
                        local porcentagem;

                        if maxBytes ~= 0 then
                                porcentagem = (currentBytes / maxBytes) * 100;
                        else
                                porcentagem = 0;
                        end;

                        --- porcentagem contém quantos % do arquivo já foi
recebido.
                end;

local arquivoAEnviar = vhd.openFile("/arquivoExistente.png", "r");
requisicao:setRequestHeader("Content-Type", "image/png");
requisicao:send(arquivoAEnviar);
```

Biblioteca NDB

A biblioteca ndb provê funções para manipular NodeDatabases, como abrir um NodeDatabase, retornar o nó pai de um nó, etc..

Todas as funções estão contidas na table/variável "NDB" da unidade "ndb.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "ndb.lua"
require("ndb.lua");

-- Agora é possível acessar as funções da biblioteca
NDB.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca NDB

```
function NDB.load(fileName [, userName])
```

Carrega um NodeDatabase que está localizado no HD Virtual do plug-in e retorna o nodo raiz.

Parâmetros:

- fileName – cadeia de caracteres identificando o nome do arquivo
- [Opcional] userName – Se você desejar fazer controle de permissões em um NodeDatabase local, passe neste parâmetro uma cadeia de caracteres identificando o usuário que está abrindo o database. Normalmente **não** é preciso se preocupar com este parâmetro, e basta omitir seu valor na chamada da função.

Retorno:

- Retorna um Objeto Nodo, representado a raiz do NodeDatabase.
  - Todas as alterações feitas neste nodo (e em seus nodos filhos) serão salvas automaticamente no arquivo passado no parâmetro "fileName".

Observações:

- O parâmetro "fileName" é expandido (através da função "VHD.expandFileName") antes de procurar o arquivo.
- Caso o arquivo não exista, um novo é criado automaticamente.
  - Se foi fornecido um caminho curto no parâmetro "fileName" (não relativo à raiz do HD Virtual), então o nome do arquivo é expandido em relação ao diretório "/" (raiz do HD Virtual) antes da criação do arquivo.
- O formato do arquivo é XML.
- Se o arquivo especificado fizer parte da pasta projeto do plug-in, o NodeDatabase será aberto em modo somente leitura.

```
function NDB.newMemNodeDatabase([initialContent])
```

Cria um novo NodeDatabase temporário que reside apenas na memória. Quando o Firecast fecha, os dados salvos neste NodeDatabase são perdidos.

Parâmetros:

- *(Opcional)* initialContent - Um objeto Stream contendo um arquivo XML. Se informado, o NodeDatabase retornado conterá os dados deste XML. Se não informado, será retornado um NodeDatabase vazio.

Retorno:

- Um Objeto Nodo, representado a raiz do NodeDatabase temporário.

```
function NDB.newObserver(node)
```

Cria um novo objeto NodeObserver para o nodo passado como parâmetro. Um NodeObserver é capaz de monitorar e notificar as mudanças ocorridas em um determinado Nodo de NodeDatabase.

Parâmetros:

- node - Objeto nodo que deseja monitorar mudanças.

Retorno:

- Um novo objeto NodeObserver

```
function NDB.newTransaction(node)
```

Cria uma nova transação para controlar como as mudanças são salvas/compartilhadas em um NodeDatabase

Parâmetros:

- node - um Objeto nodo do NodeDatabase que deseja controlar as mudanças.

Retorno:

- Um objeto NodeTransaction

Observações:

- Apenas criar um NodeTransaction não faz com que as mudanças sejam controladas. É preciso chamar o método NDB.pushTransaction para ativar a captura de mudanças para o controle.

```
function NDB.isNodeObject(value)
```

Verifica se determinado valor é um objeto nodo.

Parâmetros:

- value - O valor que deseja verificar se é um objeto nodo.

Retorno:

- **true** se o valor passado é um objeto nodo válido
- **false** se o valor passado não for um objeto nodo válido.

```
function NDB.getRoot(node)
```

Retorna o nodo raiz de um nodo, isto é, o nodo mais externo do NodeDatabase.

Parâmetros:

- node - Um objeto nodo do qual se deseja obter a raiz.

Retorno:

- Um objeto nodo representando o nodo raiz do NodeDatabase.

```
function NDB.getParent(node)
```

Retorna o nodo pai de um nodo, isto é, o nodo que contém o nodo passado como parâmetro.

Parâmetros:

- node - Um objeto nodo do qual se deseja obter o nodo pai.

Retorno:

- Um objeto nodo representando o nodo pai ou **nil** se o parâmetro node for um nodo raiz.

```
function NDB.getChildNodes(node)
```

Retorna todos os nodos filhos de um determinado nodo.

Parâmetros:

- node - O objeto nodo do qual se deseja obter os nodos filhos.

Retorno:

- Um arranjo (tabela/objeto LUA) onde cada índice, começando do 1, contém um objeto nodo filho do nodo passado como parâmetro.

```
function NDB.getAttributes(node)
```

Retorna uma tabela contendo todos os atributos e valores de um determinado objeto nodo.

Parâmetros:

- node - O objeto nodo do qual se deseja obter todos os atributos.

Retorno:

- Uma tabela/objeto LUA contendo pares de nomeAtributo=valorAtributo.

Observação:

- Esta função não retorna os nodos filhos do objeto nodo. Retorna apenas os atributos.

```
function NDB.getNodeName(node)
```

```

```

Retorna o nome de um objeto nodo.

Parâmetros:

- node - O objeto nodo de qual deseja obter o nome.

Retorno:

- Uma cadeia de caracteres (string) contendo o nome do nodo ou **nil** se o parâmetro node não for um objeto nodo válido.

```
function NDB.getPersistedAttributeValue(node,
attributeName)
```

Retorna o valor que está persistido/salvo no servidor de um atributo de um objeto nodo.

Parâmetros:

- node - O objeto nodo de qual deseja obter o valor persistido.
- attributeName - Cadeia de caracteres contendo o nome do atributo/campo que se deseja obter o valor persistido.

Retorno:

- O valor persistido ou **nil** caso não seja encontrado.

Observações:

- Devido a existência de transações no NodeDatabase e também devido à caracteristica assíncrona do NodeDatabase, o valor dos atributos do NodeDatabases podem, temporariamente, não refletir ao valor salvo no servidor e nem ao valor que os outros usuários remotos tem acesso.
- Esta função retorna o valor que foi aprovado pelo servidor e que é conhecido pelos outros usuários remotos.

```
function NDB.createChildNode(node, childName)
```

Cria um nodo filho dentro de um nodo.

Parâmetros:

- node - O objeto nodo que receberá o novo nodo filho.
- childName - uma cadeia de caracteres (string) contendo o nome do novo nodo.

Retorno:

- Um objeto nodo representando o nodo filho recém criado ou **nil** caso a operação falhe.

```
function NDB.deleteNode(node)
```

Remove um nodo do Node Database.

Parâmetros:

- node - O objeto nodo que deseja remover.

Observações:

- Não é possível remover um nodo raiz do NodeDatabase.

```
function NDB.clearNode(node)
```

Limpa um objeto nodo, isto é, remove todos os atributos e todos os nodos filhos de um determinado nodo.

Parâmetros:

- node - O objeto nodo o qual deseja limpar.

Observações:

- Diferente da função "ndb.deleteNode", esta função não remove o nodo passado como parâmetro do NodeDatabase.

```
function NDB.copy(destinationNode, sourceNode)
```

Copia os dados de um node de NodeDatabase para outro node de NodeDatabase.

Parâmetros:

- destinationNode - um Objeto nodo do NodeDatabase que será o destino da operação de cópia.
- sourceNode - um Objeto nodo do NodeDatabase que será a fonte da operação de cópia.

Observações:

- Os dados de destinationNode serão apagados antes da cópia começar.

```
function NDB.beginUpdate(node)
```

Coloca o Node Database em estado de "em alterações". Enquanto estiver neste estado, as sincronizações de dados e notificações/eventos referentes a alterações no NodeDatabase serão adiadas até que este saia do modo "em alterações".

Alterar o NodeDatabase para o modo de alterações pode melhorar a performance de procedimentos que fazem várias alterações consecutivas no mesmo.

Para tirar o NodeDatabase do estado "em alterações", invoque a função NDB.endUpdate.

Parâmetros:

- node - Um objeto nodo cujo NodeDatabase deseja colocar no modo "em alterações".

Observações:

- Você pode invocar beginUpdate mais de uma vez no mesmo NodeDatabase. Para cada chamada de NDB.beginUpdate, deve haver uma chamada de NDB.endUpdate para fazer com que o NodeDatabase saia do estado "em alterações".
- Apesar de ser passado um objeto nodo no parâmetro, o NodeDatabase inteiro será colocado no estado em alterações.

```
function NDB.endUpdate(node)
```

Faz com que o NodeDatabase saia do modo "em alterações" que havia sido previamente ativado pela função NDB.beginUpdate.

Ao sair do modo "em alterações", as mudanças feitas serão sincronizadas e os devidos eventos/notificações serão invocadas!

Parâmetros:

- node - Um objeto nodo cujo NodeDatabase deseja retirado do modo "em alterações".

Observações:

- Para cada chamada de NDB.beginUpdate, deve haver uma chamada de NDB.endUpdate para fazer com que o NodeDatabase saia do estado "em alterações".
- Apesar de ser passado um objeto nodo no parâmetro, o NodeDatabase inteiro será retirado do modo "em alterações".

```
function NDB.getState(node)
```

Retorna o estado de um NodeDatabase

Parâmetros:

- node - Um objeto nodo cujo NodeDatabase deseja verificar o estado.

Retorno:

- Uma cadeia de caracteres, podendo ser:
  - "open" - O NodeDatabase está aberto e funcional
  - "opening" - O NodeDatabase está abrindo/carregando
  - "closed" - O NodeDatabase está fechado. Algum erro de carregamento pode ter ocorrido ou a conexão com o armazenamento foi perdida.

```
function NDB.onReady(node, callback[, failCallback])
```

Monitora um NodeDatabase e notifica o programador quando ele estiver pronto para ser usado

Parâmetros:

- node - Um objeto nodo cujo NodeDatabase deseja monitorar o estado.
- callback - Um método que será chamado quando o objeto nodo estiver pronto para ser usado, isto é, ele foi completamente carregado e está pronto para ter seus dados acessados.
- (OPCIONAL) failCallback - Um método que será chamado se ocorrer algum erro durante o carregamento do NodeDatabase.

Observações:

- Às vezes, apenas ter a referência ao objeto nodo não significa que ele já está pronto para ser usado. Algumas vezes ocorre um carregamento de dados em segundo plano pela internet e, enquanto ocorre este carregamento, o objeto nodo não aceita operações. Esta é uma boa maneira de saber quando o nodeDatabase está de fato pronto!
- Quando node já estiver carregado, callback será chamado imediatamente após um delay de 1 milisegundo.
- Quando node estiver carregando, callback será chamado quando o processo de carregamento chegar ao fim.

```
function NDB.pushTransaction(node, transaction)
```

Ativa uma transação em um NodeDatabase.

Parâmetros:

- node - um Objeto nodo do NodeDatabase que deseja controlar as mudanças.
- transaction - um objeto NodeTransaction previamente criado pela função NDB.newTransaction que controlará as próximas mudanças no NodeDatabase.

Observações:

- Após ativar uma transação, todas as alterações que ocorrerem no NodeDatabase serão mantidas localmente e não serão compartilhadas automaticamente com os outros usuários. Você deverá controlar o que

acontecerá com estas mudanças ao invocar "transaction:rollback()" se quiser desfazê-las ou "transaction:commit()" para confirmá-las e compartilhar-lhas com os outros usuários.
- Para cada chamada de NDB.pushTransaction(), deve haver uma chamada de NDB.popTransaction().

```
function NDB.popTransaction(node)
```

Desativa a última transação ativada em um NodeDatabase pela função NDB.pushTransaction

Parâmetros:

- node - um Objeto nodo do NodeDatabase que deseja para de controlar as mudanças.

Observações:

- Para cada chamada de NDB.pushTransaction(), deve haver uma chamada de NDB.popTransaction().
- Se antes de chamar pushTransaction o NodeDatabase já estava sendo controlado por alguma transação, esta transação será restaurada.
- Apenas invocar este método não é o suficiente para realizar o controle das mudanças. Você o invocar o método "rollback()" da transaction se quiser desfazer as mudanças capturadas ou invocar o método "commit()" da transaction para confirmar e compartilhar as mudanças capturadas.

```
function NDB.getServerUTCTime(node)
```

Retorna a data e hora do servidor do NodeDatabase em UTC (Coordinated Universal Time)

Parâmetros:

- node - um Objeto nodo do NodeDatabase que deseja consultar o servidor.

Retorno:

- Um número de ponto flutuante contendo quantos dias se passaram desde 30/12/1899 00:00. Exemplo: 2.75 representa a data 01/01/1900 18:00

Observações:

- A função retorna imediatamente, mas o valor retornado é um valor aproximado, podendo ser diferente do horário do servidor de fato em alguns segundos.

Funções de persistência de dados do NodeDatabase

```
function NDB.exportXML(node)
```

Salva o conteúdo de um node de NodeDatabase como XML

Parâmetros:

- node - um Objeto nodo do NodeDatabase que se deseja salvar os dados.

Retorno:

- Uma cadeia de caracteres representando o conteúdo do Node e nodes filhos como um XML (codificado em UTF-8)

```
function NDB.importXML(destinationNode, xmlString)
```

Carrega os dados de um XML em um node de NodeDatabase.

Parâmetros:

- destinationNode - um Objeto nodo do NodeDatabase onde os dados serão carregados.
- xmlString - Uma cadeia de caracteres cujo conteúdo é o XML contendo os dados a serem carregados.

Observações:

- Os dados de destinationNode serão apagados antes da importação.

Funções de gerenciamento de permissões do NodeDatabase

NodeDatabases permitem definir permissões aos usuários, podendo, por exemplo, selecionar quais pessoas poderão alterar um Objeto Nodo e/ou quais pessoas não poderão saber da existência de algum Objeto Nodo.

```
function NDB.setPermission(node, selKind, selId, permission, allowance)
```

Define uma permissão a alguém a um Objeto nodo.

Parâmetros:

- node - Objeto nodo o qual deseja definir a permissão.
- selKind - Selector Kind, uma cadeia de caracteres podendo ser "group" ou "user". A combinação dos parâmetros selKind + selId define a quem esta permissão deve ser aplicada.
- selId - Selector Id, uma cadeia de caracteres contendo a identificação do usuário ou grupo de usuários a quem esta permissão deve ser aplicada.
    - se selKind for "user", selId contém o login do usuário RRPG que deseja conceder/negar a permissão.

        - se selKind for "group", selId pode conter um dos seguintes valores:
            - "todos" - A permissão será aplicada a TODOS do RRPG
            - "mestresAux" - A permissão será a aplicada a todos os mestres auxiliares da mesa.
            - "mestres" - A permissão será aplicada a todos que possuírem o modo +mestre na mesa.
            - "jogadores" - A permissão será aplicada a todos que possuírem o modo +jogador na mesa.
            - "espectadores" - A permissão será aplicada a todos que forem espectadores na mesa.
            - "criadorDaMesa" - A permissão será aplicada em quem tiver criado a mesa.
            - "criador" - A permissão será aplicada em quem criou o NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)
            - "dono" - A permissão será aplicada em quem for dono do NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)
            - "cegos" - A permissão será aplicada em quem tiver o modo +cego na mesa.
            - "mudos" - A permissão será aplicada em quem tiver o modo +mudo na mesa.
            - "voices" - A permissão será aplicada em quem tiver o modo +voz na mesa.
    - permission - Uma cadeia de caracteres identificando qual permissão está sendo definida e pode conter um dos seguintes valores:
        - "read" - Permissão para ler valores de atributos do objeto nodo e permissão para saber que o objeto nodo existe.
        - "write" - Permissão para escrever valores de atributos no objeto nodo.
        - "createChild" - Permissão para criar Objetos Nodos filhos ao objeto nodo passado em parâmetro.
        - "delete" - Permissão para apagar Objetos Nodos.
        - "readPermissions" - Permissão para ler as permissões do objeto nodo (Uma meta-permissão).
        - "writePermissions" - Permissão para definir permissões no objeto nodo (Uma meta-permissão).
        - "writeMetaPermissions" - Permissão para definir meta-permissões no nodo. Quem possui esta permissão é capaz de escolher quem pode gerenciar permissões! (Uma meta-permissão)

- allowance - Define se a permissão é concedida ou negada. Pode ser um dos seguintes valores:
    - "allow" - Esta cadeia de caracteres define que a permissão é concedida.
    - "deny" - Esta cadeia de caracteres define que a permissão é negada.
    - "strongAllow" - Esta cadeia de caracteres define que a permissão é concedida. A diferença entre "allow" e "strongAllow" se dá quando um usuário for afetado por "deny" e "allow/strongAllow" ao mesmo tempo: Se o usuário for afetado por "deny" e "allow" ao mesmo tempo, "deny" prevalece. Se for afetado por "deny" e "strongAllow" ao mesmo tempo, "strongAllow" prevalece.
    - **nil** - Nil significa: Remover, não definir, não conceder e nem negar esta permissão: A permissão será herdada do objeto nodo Pai do objeto passado como parâmetro.

Observações:

- Se selKind for "user" e selId não identificar um usuário válido do RRPG, está função falhará.
- As permissões funcionam de forma herdada: Isto é, todos os nodos filhos do "node" também respeitarão esta permissão se ela não for redefinida no filho.
- Se em um mesmo nodo um usuário for afetado ao mesmo tempo por uma definição de permissão deny e por uma definição de permissão allow, "deny" prevalecerá por ser mais forte que "allow". Se em um mesmo nodo um usuário for afetado ao mesmo tempo por uma definição de permissão deny e por uma definição de permissão strongAllow, "strongAllow" prevalecerá por ser mais forte que "deny".
    - Exemplos de uso em que um usuário pode ser afetado por mais de uma definição de permissão:
        - Você quer que todos os jogadores consigam ver um nodo, exceto José: Adiciona-se uma permissão "group jogadores read allow" e uma permissão "user José read deny". Mesmo fazendo parte do grupo Jogadores, José não terá permissão de leitura por deny ser mais forte que allow.
        - Se você quer que NENHUM jogador consiga ver um nodo, exceto José: Adiciona-se uma permissão "group jogadores read deny" e uma permissão "user José read strongAllow". Mesmo fazendo parte do grupo Jogadores, José terá permissão de leitura por strongAllow ser mais forte que deny.
- Mesmo que você não defina permissões, todo NodeDatabase possui um conjunto de permissões básicos que são herdados em todos os Nodos caso não sejam redefinidos.

- Por segurança:
  - Esta função possui um gatilho que não permite o usuário remover sua própria permissão de "read", "readPermissions", "writePermissions" e "writeMetaPermissions" quando for possuidor de tais permissões.
  - Para conceder/negar permissões "read", "write", "createChild" e "delete", o usuário atual deve possuir a permissão "writePermissions". Para conceder/negar as meta-permissões "readPermissions", "writePermissions" e "writeMetaPermissions", o usuário deve possuir também a permissão "writeMetaPermissions". Por padrão, apenas o criador da sala possui "writeMetaPermissions" e todos os mestres possuem "writePermissions".
  - Um Super Usuário não é afetado por nenhuma definição de permissão de usuário e tem acesso total. Quando um usuário for o criador da mesa e estiver com o modo +mestre nesta mesma mesa, ele é considerado um Super Usuário.

Exemplos:

```
-- Não permitir nem os jogadores e nem os espectadores
verem os valores de Sheet
NDB.setPermission(sheet, "group", "jogadores", "read",
"deny");
NDB.setPermission(sheet, "group", "espectadores", "read",
"deny");

-- Desfazer as permissões acima, voltando para valores
padrões/herdados.
NDB.setPermission(sheet, "group", "jogadores", "read",
nil);
NDB.setPermission(sheet, "group", "espectadores", "read",
nil);

-- Permitir que +jogadores possam alterar valores de
sheet
NDB.setPermission(sheet, "group", "jogadores", "write",
"allow");

-- Permitir que o usuário AlyssonRPG tenha permissões de
mestre:
NDB.setPermission(sheet, "user", "AlyssonRPG", "read",
```

```
        "allow");
        NDB.setPermission(sheet, "user", "AlyssonRPG", "write",
        "allow");
        NDB.setPermission(sheet, "user", "AlyssonRPG",
        "createChild", "allow");
        NDB.setPermission(sheet, "user", "AlyssonRPG", "delete",
        "allow");
        NDB.setPermission(sheet, "user", "AlyssonRPG",
        "readPermissions", "allow");
        NDB.setPermission(sheet, "user", "AlyssonRPG",
        "writePermissions", "allow");
```

```
function NDB.getPermission(node, selKind, selId, permission)
```

Retorna a definição de uma permissão de alguém de um Objeto nodo que foi previamente definida pelo método NDB.setPermission

Parâmetros:

- node - Objeto nodo o qual deseja consultar a permissão.
- selKind - Selector Kind, uma cadeia de caracteres podendo ser "group" ou "user". A combinação dos parâmetros selKind + selId define de quem a permissão deve ser consultada.
- selId - Selector Id, uma cadeia de caracteres contendo a identificação do usuário ou grupo de usuários de quem a permissão deve ser consultada.
  - se selKind for "user", selId contém o login do usuário RRPG que deseja consultar a permissão.
  - se selKind for "group", selId pode conter um dos seguintes valores:
    - "todos" - Permissão que foi aplicada a TODOS do RRPG
    - "mestresAux" - Permissão que foi aplicada a todos os mestres auxiliares da mesa.
    - "mestres" - Permissão que foi aplicada a todos que possuírem o modo +mestre na mesa.
    - "jogadores" - Permissão que foi aplicada a todos que possuírem o modo +jogador na mesa.
    - "espectadores" - Permissão que foi aplicada a todos que forem espectadores na mesa.

- "criadorDaMesa" - Permissão que foi aplicada em quem tiver criado a mesa.
- "criador" - Permissão que foi aplicada em quem criou o NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)
- "dono" - Permissão que foi aplicada em quem for dono do NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)
- "cegos" - Permissão que foi aplicada em quem tiver o modo +cego na mesa.
- "mudos" - Permissão que foi aplicada em quem tiver o modo +mudo na mesa.
- "voices" - Permissão que foi aplicada em quem tiver o modo +voz na mesa.

- permission - Uma cadeia de caracteres identificando qual permissão está sendo consultada e pode conter um dos seguintes valores:
  - "read" - Permissão para ler valores de atributos do objeto nodo e permissão para saber que o objeto nodo existe.
  - "write" - Permissão para escrever valores de atributos no objeto nodo.
  - "createChild" - Permissão para criar Objetos Nodos filhos ao objeto nodo passado em parâmetro.
  - "delete" - Permissão para apagar Objetos Nodos.
  - "readPermissions" - Permissão para ler as permissões do objeto nodo (Uma meta-permissão).
  - "writePermissions" - Permissão para definir permissões no objeto nodo (Uma meta-permissão).
  - "writeMetaPermissions" - Permissão para definir meta-permissões no nodo. Quem possui esta permissão é capaz de escolher quem pode gerenciar permissões! (Uma meta-permissão)

Retorno:

A função pode retornar um dos 4 abaixo:

- "allow" - Cadeia de caracteres que informa que a permissão foi concedida
- "deny" - Cadeia de caracteres que informa que a permissão foi negada.
- "strongAllow" - Cadeia de caracteres que informa que a permissão é concedida. A diferença entre "allow" e "strongAllow" se dá quando um usuário for afetado por "deny" e "allow/strongAllow" ao mesmo tempo: Se o usuário for afetado por "deny" e "allow" ao mesmo tempo, "deny" prevalece. Se for afetado por "deny" e "strongAllow" ao mesmo tempo, "strongAllow" prevalece.
- **nil** - Nil significa que a permissão não foi definida, não foi concedida e nem negada: A permissão está sendo herdada do objeto nodo Pai do objeto passado como parâmetro

Observações:

- Esta função retorna se há uma definição de permissão em determinado nodo mas não analisa a herança de permissões. Caso queira testar se o usuário possui permissão, seja por uma definição local ou herdada, utilize o método NDB.testPermission

```
function NDB.testPermission(node, permission)
```

Realiza um teste de permissão: Verifica se o usuário atual possui determinada permissão em um objeto nodo, seja permissão herdada ou permissão definida localmente.

Parâmetros:

- node - Objeto nodo o qual deseja testar a permissão.
- permission - Uma cadeia de caracteres identificando qual permissão está sendo testada e pode conter um dos seguintes valores:
  - "read" - Permissão para ler valores de atributos do objeto nodo e permissão para saber que o objeto nodo existe.
  - "write" - Permissão para escrever valores de atributos no objeto nodo.
  - "createChild" - Permissão para criar Objetos Nodos filhos ao objeto nodo passado em parâmetro.
  - "delete" - Permissão para apagar Objetos Nodos.
  - "readPermissions" - Permissão para ler as permissões do objeto nodo (Uma meta-permissão).
  - "writePermissions" - Permissão para definir permissões no objeto nodo (Uma meta-permissão).
  - "writeMetaPermissions" - Permissão para definir meta-permissões no nodo. Quem possui esta permissão é capaz de escolher quem pode gerenciar permissões! (Uma meta-permissão)

Retorno:

- Booleano **true** se o usuário possui permissão, senão **false**.

```
function NDB.enumPermissions(node)
```

Retorna todas as permissões que foram definidas em um determinado objeto nodo.

Parâmetros:

- node - Objeto nodo o qual deseja enumerar as permissão.

Retorno:

- Um array (tabela Lua indexada de 1 a #retorno), onde cada item é uma tabela contendo as seguintes propriedades:

- selKind - Selector Kind, uma cadeia de caracteres podendo ser "group" ou "user". A combinação dos parâmetros selKind + selId define a quem a permissão é aplicada.
- selId - Selector ID, uma cadeia de caracteres contendo a identificação do usuário ou grupo de usuários a quem a permissão é aplicada.
    - se selKind for "user", selId contém o login do usuário RRPG a quem a permissão é aplicada.
    - se selKind for "group", selId pode conter um dos seguintes valores:
        - "todos" - Permissão que foi aplicada a TODOS do RRPG
        - "mestresAux" - Permissão que foi aplicada a todos os mestres auxiliares da mesa.
        - "mestres" - Permissão que foi aplicada a todos que possuírem o modo +mestre na mesa.
        - "jogadores" - Permissão que foi aplicada a todos que possuírem o modo +jogador na mesa.
        - "espectadores" - Permissão que foi aplicada a todos que forem espectadores na mesa.
        - "criadorDaMesa" - Permissão que foi aplicada em quem tiver criado a mesa.
        - "criador" - Permissão que foi aplicada em quem criou o NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)
        - "dono" - Permissão que foi aplicada em quem for dono do NodeDatabase (Válido somente se for node for de um NodeDatabase de um personagem)

        - "cegos" - Permissão que foi aplicada em quem tiver o modo +cego na mesa.
        - "mudos" - Permissão que foi aplicada em quem tiver o modo +mudo na mesa.
        - "voices" - Permissão que foi aplicada em quem tiver o modo +voz na mesa.
- permission - Uma cadeia de caracteres identificando qual permissão foi aplicada e pode conter um dos seguintes valores:
    - "read" - Permissão para ler valores de atributos do objeto nodo e permissão para saber que o objeto nodo existe.
    - "write" - Permissão para escrever valores de atributos no objeto nodo.
    - "createChild" - Permissão para criar Objetos Nodos filhos ao objeto nodo passado em parâmetro.
    - "delete" - Permissão para apagar Objetos Nodos.
    - "readPermissions" - Permissão para ler as permissões do objeto nodo (Uma meta-permissão).
    - "writePermissions" - Permissão para definir permissões no objeto nodo (Uma meta-permissão).
    - "writeMetaPermissions" - Permissão para definir meta-permissões no nodo. Quem possui esta permissão é capaz de escolher quem pode gerenciar permissões! (Uma meta-permissão)
- allowance - Define se a permissão foi concedida ou negada. Pode conter um dos seguintes valores:
    - "allow" - Esta cadeia de caracteres define que a permissão foi concedida.
    - "deny" - Esta cadeia de caracteres define que a permissão foi negada.
    - "strongAllow" - Esta cadeia de caracteres define que a permissão é concedida. A diferença entre "allow" e "strongAllow" se dá quando um usuário for afetado por "deny" e "allow/strongAllow" ao mesmo tempo: Se o usuário for afetado por "deny" e "allow" ao mesmo tempo, "deny" prevalece. Se for afetado por "deny" e

"strongAllow" ao mesmo tempo, "strongAllow" prevalece.

```
function NDB.resetPermissions(node)
```

Exclui todas as definições de permissões em um determinado objeto nodo.

Parâmetros:

- node - Objeto nodo que deseja limpar as definições de permissões.

```
function NDB.editPermissions(node)
```

Apresenta uma interface padrão para o usuário editar as permissões de um Node do NodeDatabase

Parâmetros:

- node - Objeto nodo que o usuário editará as permissões.

Funções de comunicação através do NodeDatabase

NodeDatabases permitem o envio e recebimento de mensagens aos usuários que estão acessando o mesmo NodeDatabase.

```
function NDB.broadcastMessage(node, messageId, message[, loopBack])
```

Envia uma mensagem a todos os outros usuários que estão com o nodeDatabase aberto.

Parâmetros:

- node - Objeto nodo do NodeDatabase a qual deseja enviar uma mensagem
- messageId - Uma cadeia de caracteres contendo a identificação da mensagem.
- message - A mensagem a ser transmitida. Aqui são aceitos textos, tabelas lua, números e booleanos.
- (OPICIONAL) loopBack - Informa se quer que esta mensagem seja retransmitida para você mesmo. Valor padrão: false

Observações:

- A mensagem enviada é entregue a todos os outros usuários conectados no NodeDatabase.
- Para receber as mensagens recebidas, o usuário destinatário deve ter previamente invocado a função NDB.newBroadcastListener

```
function NDB.newBroadcastListener(node, messageId, callback)
```

Cria uma escuta de mensagens para receber mensagens postadas pela função NDB.broadcastMessage.

Parâmetros:

- node - Objeto nodo do NodeDatabase do qual deseja receber mensagens.
- messageId - Uma cadeia de caracteres contendo a identificação das mensagens que deseja receber. Deve ser igual ao messageId usado na função NDB.broadcastMessage.
- callback - Uma função que será invocada ao receber mensagens. Esta função recebe três parâmetros na seguinte ordem:
  - sender - Cadeia de caracteres contendo o **login** do usuário que enviou a mensagem
  - messageId - Cadeia de caracteres contendo a identificação da mensagem recebida
  - message - A mensagem que foi postada.

Retorno:

- Retorna um objeto representando o Listener.

Observações:

- Importante: Mantenha referência ao objeto retornado por esta função enquanto quiser que a escuta esteja ativa. Se o coletor de lixo LUA coletar este objeto, a escuta é destruída e a função callback não será mais invocada.
- Se quiser destruir a escuta imediatamente, invoque a função destroy do objeto retornado.... obj:destroy();
- Nos casos das fichas e janelas acopláveis, o RRPG mantém nodeDatabases carregados mesmo após o usuário fechar a janela da ficha. Por este motivo, a função callback pode ser invocada para uma ficha que não possui mais janela. Se fizer uso de mensagens, esteja preparado para este comportamento.

Objeto Nodo

Um Objeto Nodo é uma tabela LUA com um comportamento diferenciado que representa um nó/nodo de um NodeDatabase. Suas principais características são:

- Aparentam ser uma tabela/objeto LUA normal. Isto é, você pode ler e atribuir propriedades como faria com qualquer outra tabela/objeto LUA.

- Todas as alterações feitas no objeto nodo são salvas e sincronizadas automaticamente com as outras pessoas que estão acessando o mesmo NodeDatabase.
- Quando outras pessoas alteram atributos do nodo, o seu objeto nodo recebe e reflete tais alterações automaticamente.
- Não é possível atribuir uma "função" a uma propriedade do nodo.
- Quando se atribui uma tabela LUA a uma propriedade do nodo, uma cópia recursiva de todos os valores é realizada.
- O nome dos atributos são case-insensitive, isto é, não são diferenciadas letras minúsculas das maiúsculas (Ex: "nome" e "NoME" representam o mesmo atributo do nodo).

Alterando atributos de um Objeto Nodo

Para alterar um atributo de um Objeto Nodo basta fazer uma atribuição como você faria em qualquer outra tabela/objeto Lua.

Exemplo:

```
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

nodoRaiz.atributo1 = "valor em string";
nodoRaiz.atributo2 = 2;
nodoRaiz.atributo3 = true;
nodoRaiz.atributoDeOutroNome = 6.7;

nodoRaiz["atributo5"] = "outro valor em string";
nodoRaiz["atributo6"] = 9999;
nodoRaiz["atributo7"] = false;
```

Não tem mistério! Você atribui normalmente.... Só não são aceitos **function** como valores.

As alterações são salvas automaticamente no documento.

Lendo atributos de um Objeto Nodo.

Mais uma vez, não há mistério: Basta acessar normalmente como uma tabela LUA qualquer.

Exemplo:

```
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

 showMessage(nodoRaiz.atributo1);

if nodoRaiz.atributo2 == 2 then
        -- faz alguma coisa....
end;

nodoRaiz.atributo200 = nodoRaiz.atributo3;
nodoRaiz.atributo250 = nodoRaiz.atributo2 + 50;

local valor = nodoRaiz["atributoDeOutroNome"];
valor = nodoRaiz["atributo3"];
```

Removendo um atributo de um Objeto Nodo

Basta atribuir **nil** ao atributo que deseja remover.

Exemplo:

```
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

nodoRaiz.atributoQueDesejoRemover = nil;
nodoRaiz.atributo1 = nil;
nodoRaiz["atributo2"] = nil;
```

As alterações são salvas automaticamente no documento.

Criando um Objeto Nodo filho

Basta atribuir uma tabela a uma propriedade....

Exemplo:

```lua
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

-- Após a linha abaixo, nodoRaiz.nodoFilho será um Objeto Nodo filho de "nodoRaiz" de nome "nodoF
nodoRaiz.nodoFilho = {};

-- Na linha abaixo, nodoRaiz.nodoFilho2 será um Objeto Nodo filho de "nodoRaiz" de nome "nodoFilh
-- Além disso, o novo Objeto Nodo já possuirá os valores nome="Meu Nome" e idade=7.
nodoRaiz.nodoFilho2 = {nome="Meu Nome", idade=7};

 -- Como nodoRaiz.nodoFilho é um Objeto Nodo, você pode criar um nodo filho "nodoNeto" dentro de
 --"nodoRaiz.nodoFilho" facilmente:
nodoRaiz.nodoFilho.nodoNeto = {};

-- E pode atribuir propriedades a ele normalmente, pois "nodoNeto" também é um Objeto Nodo.
nodoRaiz.nodoFilho.nodoNeto.atributoDoNeto = "Valor";
```

As alterações são salvas automaticamente no documento.

Deletando um Objeto Nodo

basta atribuir **nil** ou usar a função "ndb.deleteNode(nodo)"

Exemplo:

```lua
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

-- apagando o nodo "nodoFilho" e todos seus filhos recursivamente do nodo raiz
nodoRaiz.nodoFilho = nil;

-- apgando usando a função ndb.deleteNode(node)
ndb.deleteNode(nodoRaiz.nodoFilho2);
```

As alterações são salvas automaticamente no documento.

Duplicando um Objeto Nodo

Basta fazer uma atribuição simples...

Exemplo:

```
require("ndb.lua");
local nodoRaiz = ndb.load("MeuNodeDatabase.xml");

--[[ nodoRaiz é um Objeto Nodo --]]

-- criando um nodo filho com valores padrões:
nodoRaiz.nodoFilho = {nome="José", idade=75, corFavorita="red", nodoNeto={}};

-- duplicando o nodo filho.
nodoRaiz.nodoClone = nodoRaiz.nodoFilho;
```

As alterações são salvas automaticamente no documento.

Outras operações em um Objeto Nodo

- Obter o nome do objeto nodo: ndb.getNodeName(nodo);
- Limpar o conteúdo do objeto nodo: ndb.clearNode(nodo);
- Obter o nodo pai: ndb.getParent(nodo);

Objeto NodeObserver

Este objeto representa um NodeObserver, um mecanismo capaz de monitorar e notificar mudanças que ocorrerem em um determinado Nodo do NodeDatabase.

Como instanciar um NodeObserver

É possível instanciar um NodeObserver através do método NDB.newObserver

Características

O objeto NodeObserver possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
enabled	Boolean	Determina se o NodeObserve está ativo e se pode disparar eventos

		notificando mudanças. Valor padrão: true
node	objeto Nodo	Somente leitura, contém o objeto Nodo que está sendo monitorado pelo NodeObserver

Eventos

Nome do evento	Descrição
onChildAdded	Este evento é invocado quando um objeto nodo filho é criado no nodo que está sendo monitorado. Parâmetros: ○ node - O objeto nodo filho que foi criado.
onChildRemoved	Este evento é invocado quando um objeto nodo filho é deletado do nodo que está sendo monitorado. Parâmetros: ○ node - O objeto nodo filho que foi removido.
onDeepChildAdded	Este evento é invocado quando um objeto nodo filho é criado no nodo que está sendo monitorado ou em algum de seus filhos (monitoramento recursivo). Parâmetros: ○ node - O objeto nodo filho que foi criado.
onDeepChildRemoved	Este evento é invocado quando um objeto nodo filho é deletado do nodo que está sendo monitorado ou de algum de seus filhos (monitoramento recursivo). Parâmetros:

	○ node - O objeto nodo filho que foi removido.
onChanged	Este evento é invocado quando o valor de algum atributo do nodo que está sendo monitorado é alterado. Parâmetros: ○ node - O objeto nodo onde ocorreu a mudança do do valor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor do atributo.
onDeepChanged	Este evento é invocado quando o valor de algum atributo do nodo ou nodo filho do nodo que está sendo monitorado é alterado (monitoramento recursivo). Parâmetros: ○ node - O objeto nodo onde ocorreu a mudança do do valor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor do atributo.

onPersistedChanged	Este evento é invocado quando ocorre uma mudança de valor no **servidor** de algum atributo do nodo que está sendo monitrado. Parâmetros: ○ node - O objeto nodo onde ocorreu a mudança do do valor no servidor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor de servidor do atributo. Observações: ● Devido a existência de transações no NodeDatabase e também devido à característica assíncrona do NodeDatabase, o valor dos atributos do NodeDatabases podem, temporariamente, não refletir ao valor salvo no servidor e nem ao valor que os outros usuários remotos tem acesso. ● Este evento é chamado quando alguma alteração é confirmada no lado do Servidor, persistida e disponibilizada para todos os outros usuários remotos. ● O valor persistido pode ser obtido através da função NDB.getPersistedAttributeValue()

onDeepPersistedChanged	Este evento é invocado quando ocorre uma mudança de valor no **servidor** de algum atributo do nodoou nodo filho que está sendo monitrado (monitoramento recursivo). Parâmetros: ○ node - O objeto nodo onde ocorreu a mudança do do valor no servidor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor de servidor do atributo. Observações: ● Devido a existência de transações no NodeDatabase e também devido à característica assíncrona do NodeDatabase, o valor dos atributos do NodeDatabases podem, temporariamente, não refletir ao valor salvo no servidor e nem ao valor que os outros usuários remotos tem acesso. ● Este evento é chamado quando alguma alteração é confirmada no lado do Servidor, persistida e disponibilizada para todos os outros usuários remotos. ● O valor persistido pode ser obtido através da função NDB.getPersistedAttributeValue()
onUserChanged	Este evento é invocado quando o valor de algum atributo do nodo que está sendo monitorado é alterado **porque** o **usuário local** alterou o conteúdo do do campo, seja editando ele visualmente ou através de código Lua. Parâmetros:

	○ node - O objeto nodo onde ocorreu a mudança do do valor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor do atributo. Observações: ● Lembre-se, este evento será disparado apenas quando o usuário local alterar o valor de algum dos campos do nodo monitorado.
onDeepUserChanged	Este evento é invocado quando o valor de algum atributo do nodo ou nodo filho que está sendo monitorado é alterado **porque** o **usuário local** alterou o conteúdo do do campo, seja editando ele visualmente ou através de código Lua (monitoramento recursivo). Parâmetros: ○ node - O objeto nodo onde ocorreu a mudança do do valor. ○ attribute - Cadeia de caracteres contendo o nome do atributo que mudou. ○ oldValue - Antigo valor do atributo. Observações: ● Lembre-se, este evento será disparado apenas quando o usuário local alterar o valor de algum dos campos do nodo ou

	nodo filho monitorado.
onDeleted	Este evento é invocado quando o objeto nodo que está sendo monitorado é apagado. Parâmetros: ○ node - O objeto nodo que foi removido.
onPermissionListChanged	Este evento é invocado quando definições de permissões de acesso são inseridas ou removidas do objeto nodo que está sendo monitorado. Parâmetros: ○ node - O objeto nodo que sofreu alteração na lista de definições de permissões.
onDeepPermissionListChanged	Este evento é invocado quando definições de permissões de acesso são inseridas ou removidas do objeto nodo que está sendo monitorado ou em algum de seus nodos filhos (monitoramento recursivo). Parâmetros: ○ node - O objeto nodo que sofreu alteração na lista de definições de permissões.
onFinalPermissionsCouldBeChanged	Este evento é invocado quando as permissões finais de acesso a este nodo podem ter sofrido alterações porque uma permissão em um nodo pai foi removida/inserida e, por consequência, pode ter sido herdada pelo nodo que está sendo monitorado, ou as permissões finais podem ter sofrido alterações porque o usuário mudou de grupo de permissões (Exemplo: O usuário atual virou +mestre). Parâmetros:

	○ node - O objeto nodo que pode ter sofrido alteração nas permissões finais.
onStateChanged	Este evento é invocado quando o estado de carregamento do node (ver ndb.getState) muda. Parâmetros: ○ node - O objeto nodo que sofreu mudança no estado de carregamento.

Observações:

- Se um NodeObserver não for mais acessível pelo código e for destruído pela coleta de lixos do Lua, os eventos não serão mais disparados.

Exemplos

Exemplo 1 - Monitorando um Node "sheet"

```
self.observer = ndb.newObserver(sheet);

self.observer.onChildAdded =
        function(node)
                --  Esta função será chamada quando um novo Nodo Filho for criado em sheet
        end;

self.observer.onChildRemoved =
        function(node)
                --  Esta função será chamada quando um nodo Filho for removido de sheet
        end;

self.observer.onChanged =
        function(node, attribute, oldValue)
                -- Esta função será chamada quando algum atributo de sheet mudar de valor
```

```
                -- attribute possui o nome do atributo que sofreu
alteração
        end;
```

Objeto NodeTransaction

Este objeto representa uma transação em um NodeDatabase, um mecanismo que dá maior controle ao programador sobre as mudanças locais.

Uma Transação é um objeto que armazena localmente todas as mudança realizadas em um NodeDatabase para que você possa decidir, mais tarde, se quer descartá-las ou confirmá-las.

Como instanciar um NodeTransaction

É possível instanciar um NodeTransaction através do método ndb.newTransaction

Observação importante:

Apenas criar um NodeTransaction não faz com que os dados sejam controlados. É preciso também utilizar os métodos ndb.pushTransaction e ndb.popTransaction para isso.

Características

O objeto NodeTransaction possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição

Métodos

Nome do método	Descrição
transaction:commit()	Confirma as mudanças que estavam armazenadas localmente. Ao realizar Commit, os dados serão salvos em arquivo/Servidor RRPG e compartilhados com os demais usuários.

transaction:rollback()	Desfaz as mudanças que estavam armazenadas localmente.
transaction:createUndoData()	Cria um objeto NodeUndoData que contem as instruções necessário para desfazer todas as mudanças que estão armazenadas localmente. Você pode, então, depois, decidir se deseja desfazer as mudanças guardadas utilizando o método transaction:applyUndoData Observações: ● Armazenar o NodeUndoData é uma ótima forma de criar recursos parecidos com o famoso "ctrl+z" ● Esta função deve ser chamada antes de invocar "commit" para ter o efeito esperado.
transaction:applyUndoData(undoData)	Aplica as instruções de um objeto NodeUndoData para desfazer mudanças previamente feitas. Parâmetros: ● undoData - O objeto NodeUndoData que contém o material/instruções para desfazer mudanças, criado previamente pela função transaction:createUndoData() Observações: ● Após invocar este método, ainda é necessário invocar "commit" para que os dados sejam de fato desfeitos ● Materiais de undo devem ser aplicados na ordem reversa que ocorreram e apenas uma vez. Desfazer mudanças fora de ordem ou mais de uma vez leva a um comportamento indefinido

Exemplos

Exemplo 1 - Exemplo de uso do Transaction

```
local tr1 = ndb.newTransaction(sheet);
local tr2 = ndb.newTransaction(sheet);


sheet.nome = "Maria"; -- Mudança 1


ndb.pushTransaction(sheet, tr1);
sheet.idade = 20; -- Mudança 2


ndb.pushTransaction(sheet, tr2);
sheet.nome = "Ana";  -- Mudança 3
ndb.popTransaction(sheet);


sheet.altura = "1.60m";  -- Mudança 4
ndb.popTransaction(sheet);


tr2:rollback();


--[[
    Ao final da execução:
    * Mudança 1 -
        Será salva e compartilhada com todos, pois foi
realizada fora de qualquer transação.


    * Mudanças 2 e 4 -
        Será mantida APENAS localmente e ninguém mais
enxergará as mudanças, pois foram feitas na transação "tr1" e
esta não recebeu nem commit e nem rollback.
        Esta mudança local existirá enquanto o objeto "tr1"
existir. Quando ele for destruído pelo coletor de lixo do Lua,
as mudanças serão desfeitas.


    * Mudança 3 -
        Será desfeita pois foi realizada na transação "tr2" e
ela recebeu uma chamada de "rollback"
]]--
```

Exemplo 2 - Perguntando ao usuário se ele deseja salvar mudanças feitas

```
local tr = ndb.newTransaction(sheet);


ndb.pushTransaction(sheet, tr);
sheet.nome = "maria";
sheet.idade = 20;
sheet.altura = "1.60m"
ndb.popTransaction(sheet);


dialogs.confirmYesNo("Deseja salvar as alterações?",
    function(resposta)
        if resposta then
            tr:commit();
        else
            tr:rollback();
        end;
    end);
```

Objeto NodeUndoData

Este objeto representa um material de "desfazer" de um NodeDatabase, uma estrutura que permite criar mecanismos parecidos com o famoso "ctrl+z"

NodeUndoData armazena internamente as informações necessárias para desfazer alguma mudança no NodeDatabase.

Para aplicar o material de "desfazer", você pode invocar o método transaction:applyUndoData passando o objeto NodeUndoData.

Como instanciar um NodeUndoData

É possível instanciar um NodeUndoData através do método transaction:createUndoData

Características

O objeto NodeUndoData possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição

Métodos

Nome do método	Descrição

Exemplos

Biblioteca Firecast

A biblioteca firecast contém funções relacionadas às mesas, personagens, envio de mensagens, rolagem de dados, etc..

Todas as funções estão contidas na table/variável "Firecast" da unidade "firecast.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "firecast.lua"
require("firecast.lua");

-- Agora é possível acessar as funções da biblioteca
Firecast.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Firecast

```
function Firecast.asyncOpenUserNDB(name[, options])
```

Asynchronously open a per-user NodeDatabase that is physically stored in the Firecast server.

Arguments:

- name - A string identifying which remote user NodeDatabase should be opened
- (OPTIONAL) options - A Lua table describing additional settings that may contain:
  - create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false**
  - skipLoad - a boolean. If **true**, the returned Promise will be resolved before the load process is finished. If **true**, you will receive a "Loading node" instead of a "Loaded node". Default: **false**

Return:

- a Promise for a Node Object. As default behavior, the Promise will be resolved after all data is downloaded and available to be used.

Remarks:

- The content of NodeDatabases opened by this function is stored separately per-user. Each user will have their own copy of the NodeDatabase identified by the "name" argument

Example:

```
 local promise = Firecast.asyncOpenUserNDB("exampleNodeDatabase",
{create=true});

 local node = await(promise);

 -- variable node is ready to be used here
```

```
function Firecast.getRooms()
function Firecast.getMesas()
```

Retorna a lista de objetos mesas representando as mesas em que o usuário está no momento.

Parâmetros:

Não há parâmetros

Retorno:

- Um arranjo de objeto mesa, isto é, uma tabela lua indexada de 1 a N.

Exemplo - Exibindo uma mensagem contendo o nome e o sistema de todas as mesas em que o usuário está:

```
local mesas = Firecast.getRooms();
local msg = "";
```

```
for i = 1, #mesas, 1 do
        local objMesa = mesas[i];

        msg = msg .. objMesa.nome .. " (" ..
objMesa.sistema .. ")\n";
end;

showMessage(msg);
```

```
function Firecast.findRoom(codigoInterno)
function Firecast.findMesa(codigoInterno)
```

Dado o código interno de uma mesa, localiza o objeto mesa de código interno correspondente.

Parâmetros:

Não há parâmetros

Retorno:

- Um objeto mesa ou **nil** caso não encontre.

```
function Firecast.parseRoll(stringDaRolagem)
function Firecast.interpretarRolagem(stringDaRolagem)
```

Dado uma cadeia de caracteres representando uma rolagem de dados (Exemplos: "1d20 + 5", "1d20 + 1d10 - 5", etc..), retorna um objeto rolagem que contém as informações sobre a mesma.

Parâmetros:

- stringDaRolagem - cadeia de caracteres que deseja interpretar como um objeto rolagem.

Retorno:

- Um Objeto Rolagem contendo a interpretação.

Observação:

- Se stringDaRolagem não possuir uma cadeia de caracteres válida, o novo objeto rolagem ainda será retornado, porém vazio.

```
function Firecast.getRoomOf(object)
function Firecast.getMesaDe(object)
```

Dado um objeto qualquer, a função tenta determinar à qual objeto mesa ele está ligado.

Parâmetros:

- object - O objeto que deseja tentar determinar à qual objeto mesa ele pertence ou está ligado.

Retorno:

- Um Objeto Mesa, caso a função consiga determinar
- **nil**, caso a função não consiga determinar

Observações:

- Os objetos que costumam ser passados como parâmetro à esta função, são:
  - objeto Nodo. Se o NodeDatabase for de um personagem ou estiver ligado à uma mesa, a função retorna o objeto mesa.
  - objeto BibliotecaItem. A função retorna à qual mesa o item da biblioteca pertence.
  - objeto Personagem.
  - objeto Jogador.
  - objeto Chat
  - objeto form de formType Table's Dock
  - objeto Scene
  - Outros objetos que possam estar ligados à alguma mesa.
- Também é possível passar um valor numérico para esta função. Se for passado um valor numérico, esta função se comporta como a função rrpg.findMesa.

```
function Firecast.getLibraryItemOf(objeto)
function Firecast.getBibliotecaItemDe(objeto)
```

Dado um objeto qualquer, a função tenta determinar à qual objeto BibliotecaItem ele está ligado.

Parâmetros:

- objeto - O objeto que deseja tentar determinar à qual objeto BibliotecaItem ele pertence ou está ligado.

Retorno:

- Um Objeto BibliotecaItem caso a função consiga determinar
- **nil**, caso a função não consiga determinar

Observações:

- Os objetos que costumam ser passados como parâmetro à esta função, são:
  - objeto Nodo. Se o NodeDatabase for de um item na biblioteca, a função retorna o objeto biblioteca item.
  - Outros objetos que possam estar ligados à biblioteca da mesa.

```
function Firecast.getCharacterOf(object)
function Firecast.getPersonagemDe(object)
```

Dado um objeto qualquer, a função tenta determinar à qual objeto Personagem ele está ligado.

Parâmetros:

- object - O objeto que deseja tentar determinar à qual objeto Personagem ele pertence ou está ligado.

Retorno:

- Um Objeto Personagem caso a função consiga determinar
- **nil**, caso a função não consiga determinar

Observações:

- Os objetos que costumam ser passados como parâmetro à esta função, são:
  - objeto Nodo. Se o NodeDatabase for de um personagem, a função retorna o objeto personagem.
  - Outros objetos que possam estar ligados aos personagens da mesa.

```
function Firecast.getCurrentUser()
```

Retorna uma tabela contendo informações do usuário logado atualmente no Firecast.

Retorno:

- Um tabela LUA contendo as seguintes propriedades/atributos:
  - isLogged - Boleano (true ou false) indicando se possui um usuário logado no momento.

- id - Se o usuário estiver logado, contém o identificador interno do usuário.
- login - Se o usuário tiver logado, contém uma cadeia de caracteres contendo seu login/username.
- isGold - Booleano (true ou false) indicando se o usuário é assinante Firecast Gold
- isGoldPlus - Booleano (true ou false) indicando se o usuário é assinante Firecast Platinum / RRPG Gold Plus
- isRuby - Booleano (true ou false) indicando se o usuário é assinante Firecast Ruby

```
function Firecast.registerChatToolButton(params)
```

Registra um botão-ferramenta que será apresentado nos chats do Firecast.

Parâmetros:

- params - Uma tabela Lua contendo as seguintes propriedades do botão:
  - "hint" - Um string que será apresentado ao usuário quando ele arrastar o mouse em cima do botão.
  - "icon" - Um string contendo o endereço da imagem que será apresentada como ícone do botão.
  - "callback" - Uma função que será chamada quando o usuário clicar no botão. Seu primeiro parâmetro recebe um objeto Chat representado em qual chat o usuário clicou no botão.
  - (OPCIONAL) "priority" - Se quiser sugerir a ordem dos botões, informe um número em priority. Botões com prioridade maior aparecem antes dos com prioridade menor.
  - (OPCIONAL) "group" - Um string. Se quiser vários botões ficarem juntos, você pode especificar o mesmo valor em group para eles.
  - (OPCIONAL) "groupPriority" - Se quiser sugerir a ordem do grupo de botões, informe um número nesta propriedade. Grupos de botões com maior prioridade aparecem antes dos grupos com menor prioridade.

Retorno:

- Um identificador do botão criado. Este valor pode ser usado na função Firecast.unregisterChatToolButton se quiser remover o botão.

Observações:

- O ícone informado poderá sofrer tratamento visual para se encaixar no tema do Firecast. Use ícones com transparência e cuja "sombra" seja capaz de remeter seu significado.
- O botão só será apresentado se:
  - O usuário for assinante Gold.

- Ou o usuário não for pagante e estiver em mesa cujo dono seja assinante Gold+/Platina.

Exemplo:

```
local params = {};
params.hint = "This button do @@localized.text.id";
params.icon = "/icons/myIcon.png";
params.priority = 10;
params.group = "myButtonGroup";

params.callback =
    function(chat)
        -- the param "chat" contains good information
to handle =)
        showMessage("User clicked the button");
    end;

Firecast.registerChatToolButton(params);
```

```
function Firecast.parseTalemark(text[, talemarkOptions])
```

Parse and break down a talemark string, the markup language used by Firecast's chats.

Arguments:

- text - A string, the text to be parsed.
- *(Optional)* talemarkOptions - A TalemarkOptions table containing parameters of how Firecast should color, format, and parse the string

Retorno:

- A TalemarkTag table containing the root tag for the parsed text.

```
function Firecast.unregisterChatToolButton(toolButtonId)
```

Desregistra um botão-ferramenta de chat previamente instalado pela função Firecast.registerChatToolButton.

Parâmetros:

- toolButtonId - O identificador retornado pela função Firecast.registerChatToolButton.

Objeto Mesa

Este objeto representa uma mesa aberta no Firecast.

Herança

O **Objeto Mesa** possui todas as características de um WrappedObject. Veja:

- Objeto WrappedObject

Características

Além das características herdadas, o Objeto Mesa também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
audioPlayer	AudioPlayer object	(Read-only) Contains the AudioPlayer object utilized for playing audio within the room. Audio played through this object will also be broadcasted to all online players. For local playback only, consider using Audio.defaultPlayer instead.
codigoInterno	Integer	Somente leitura, contém o código interno da mesa que é usado para identificá-la unicamente em todo o RRPG Firecast.
firecastURI	String	(Read-only) A string containing the shareable Firecast URI for this room. You can use this URI with the GUI.asyncOpenFirecastURI() function.

nome	String	Somente leitura, contém o nome da mesa.
msgStatus	String	Somente leitura, contém a mensagem de status da mesa.
site	String	Somente leitura, contém o endereço do site da mesa.
logotipo	String	Somente leitura, contém a URL da imagem logotipo da mesa.
sistema	String	Somente leitura, contém o sistema de RPG da mesa.
msgBoasVindas	String	Somente leitura, contém o texto da mensagem de boas vindas da mesa.
isRestrito18Anos	Boolean	Somente leitura, contém true se a mesa estiver marcada como não recomendada para menores de 18 anos.
haVagas	Boolean	Somente leitura, contém true se a mesa estiver marcada como há vagas.
isModerada	Boolean	Somente leitura, contém true se a mesa estiver marcada como Moderada.
podeTablesDock	Boolean	Somente leitura, contém true se a mesa pode exibir Table's Dock do usuário. É true quando o criador da mesa é Gold Plus ou quando o usuário local é assinante Gold.
jogadores	Arranjo de Objeto Jogador	Somente leitura, contém um arranjo de Objeto Jogador representando todos os usuários que estão conectados na mesa no momento. Um arranjo é uma tabela lua indexada de 1 a N, onde N é o comprimento da tabela (operador # no lua)

meuJogador or **me**	Objeto Jogador	Somente leitura, contém o Objeto Jogador do usuário que está usando o Firecast.
library or **biblioteca**	Objeto BibliotecaItem	Somente leitura, contém o objeto BibliotecaItem raiz da biblioteca da mesa. A raiz da biblioteca é aquela pasta que tem no topo da mesma, de nome "Biblioteca".
chat	Objeto Chat	Somente leitura, contém o Objeto Chat da interface de bate papo principal da mesa.
activeChat	Objeto Chat	Somente leitura, contém o Objeto Chat da mesa que está atualmente ativo/focado na interface.

Métodos

Método	Descrição
mesa:isType(typeName)	Retorna true se passado "mesa" como parâmetro.
mesa:findBibliotecaItem(codigoInterno)	Dado um número código interno, localiza o objeto BibliotecaItem que possui "codigoInterno" igual ao valor passado no parâmetro "codigoInterno". Retorna um objeto BibliotecaItem ou **nil** caso não encontre.
mesa:findJogador(loginOuCodigoInterno)	Dado um login de usuário ou seu código interno, localiza o objeto Jogador correspondente representando o jogador atualmente na mesa. Retorna um objeto Jogador ou nil caso não encontre. Observação: A função retorna **nil** se o usuário não estiver na mesa no momento, mesmo quando o parâmetro passado for um login ou um código interno válido.

mesa:requestSetModerada(moderada)	Requisita a ativação/desativação da mesa moderada. Parâmetros: ○ moderada - Booleano, onde true significa "Ativar a mesa moderada" e false "Desativar a mesa moderada". Observações: ○ A alteração do comportamento to "mesa moderada" só ocorre após o servidor Firecast aprovar a mudança.

mesa:abrirNDBDeMesa(nome, callback[, opcoes])	**FUNÇÃO DESCONTINUADA!** Utilize a função mesa:asyncOpenRoomNDB ao invés desta. Abre um NodeDatabase da mesa que está fisicamente armazenado no servidor do Firecast. Parâmetros: ○ nome - Cadeia de caracteres que identifica qual NodeDatabase remoto da mesa deve ser aberto. ○ callback - Uma função que será invocada quando o NodeDatabase for carregado ou quando ocorrer um erro no carregamento. ■ Q ■ Q ○ (OPCIONAL) opcoes - Uma tabela lua contendo as seguintes opções extras de carregamento do nodedatabase: ■ (O ■ (O

	Observações: ○ Esta função é assíncrona, isto é, o código LUA continua sua execução normal enquanto o NDB é carregado em segundo plano. É preciso informar o parâmetro "callback" para obter o NodeDatabase carregado. ○ Cada mesa possui seu próprio conjunto de NodeDatabases que existem independentemente das outras mesas. ○ Alterações feitas no NodeDatabase aberto por esta função são automaticamente sincronizadas com os outros usuários que também abriram o mesmo NodeDatabase. ○ Se o usuário sair da mesa, o NodeDatabas

	e terá sua conexão cortada e não mais sincronizará as alterações. ○ Por padrão, os usuários com o modo +mestre podem ler e alterar o NodeDatabase de mesa enquanto os demais usuários podem apenas ler valores. ○ Para abrir um NodeDatabase de mesa, uma das seguintes afirmativas deve ser verdadeira: ■ O ■ O

mesa:asyncCreateGroupPVT(logins[, params])	Asynchronously creates a group private chat with the specified users within this room.
	
	Arguments:
	
	● logins - An array of strings containing the usernames to be invited to the group private chat.
	
	● *(Optional)* params - A Lua table that may contain the following optional fields:
	○ autoFocus - A Boolean; "true" means Firecast should try to focus the open interface, while "false" means to open without focusing it. Default: "false".
	
	Return:
	
	● A Promise to a Chat object
	
	Remarks:
	
	● The argument "logins" must not be empty and must identify users that are in the room. You don't need to put the current user login in that array.
	● It is not possible to create a private group chat with just the current user in it; there must be at least one other user.
	
	Example:
	
	`local room = Firecast.getRoomOf(sheet);`
	
	`local promise = room:asyncCreateGroupPVT({"Alysso`

	``
	nRPG", "AnotherUser"});
	local chat = await(promise);
	chat:write("Hello!");
	``

mesa:asyncOpenRoomNDB(name[, options])	Asynchronously open a room NodeDatabase which is physically stored on the Firecast server Arguments: ● name - A string identifying the remote NodeDatabase in this room to be opened ● *(Optional)* options - A Lua table describing additional settings that may contain: ○ create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false.** ○ skipLoad - a boolean. If **true**, the returned Promise will be resolved before the load process is finished. If **true**, you will receive a "Loading node" instead of a "Loaded node". Default: **false** Return: ● a Promise for a Node Object. Unless *options.skipLoad* is true, the Promise resolves after all data is downloaded and available for use. Remarks:

	● The content of NodeDatabases opened by this function is stored separately per-room. Each room will have their own copy of the NodeDatabase identified by the “name”.
	● To open a room NodeDatabase, one of the following must be true:
	  ○ The current user is a Gold subscriber; or
	  ○ The room creator is a Platinum subscriber.
	
	Example:
	`local promise = room:asyncOpenRoomNDB('myNDBId', {create=true});`
	`local node = await(promise);`
	`-- variable node is ready to be used here`

mesa:asyncOpenUserRoomNDB(name[, options])	Asynchronously open a per-user NodeDatabase within this room, which is physically stored on the Firecast server Arguments: ● name - A string identifying the remote user NodeDatabase in this room to be opened ● *(Optional)* options - A Lua table describing additional settings that may contain: ○ create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false.** ○ skipLoad - a boolean. If **true**, the returned Promise will be resolved before the load process is finished. If **true**, you will receive a "Loading node" instead of a "Loaded node". Default: **false** Return: ● a Promise for a Node Object. Unless *options.skipLoad* is true, the Promise resolves after all data is downloaded and available for use. Remarks: ● The content of NodeDatabases opened by this function is stored separately for each user and each room. Each user in the

	room will have their own copy of the NodeDatabase identified by the specified "name."
	
	Example:
	
	`local promise = room:asyncOpenUserRoomNDB("myId", {create=true});`
	
	`local node = await(promise);`
	
	`-- variable node is ready to be used here`

mesa:asyncOpenPVT(login[, params])	Asynchronously opens a private chat with the specified user within this room. Arguments: ● login - A string containing the username of the user you want to open the PVT Chat. ● *(Optional)* params - A Lua table that may contain the following optional fields: ○ autoFocus - A Boolean; "true" means Firecast should try to focus the open interface, while "false" means to open without focusing it. Default: "false". ○ mode - One of the following strings: ■ "g ■ "g Return:

	● A Promise to a Chat object
	
	Example:
	
	`local room = Firecast.getRoomOf(sheet);`
	
	`local promise = room:asyncOpenPVT("AlyssonRPG", {autoFocus=true});`
	
	`local chat = await(promise);`
	
	`chat:write("Hello!");`

mesa:getChats()	Returns all chat objects that belong to this room. Arguments: None Return: ● An array of Chat object

Exemplos

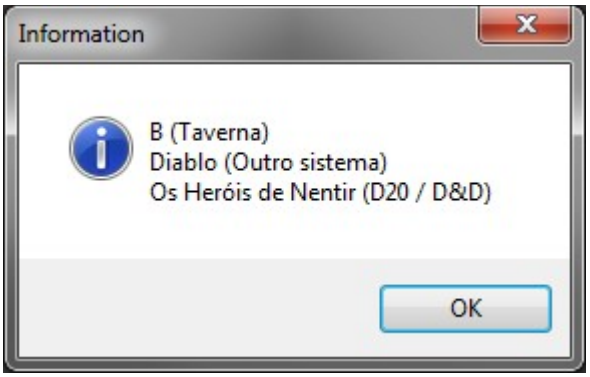
Exemplo 1 - Exibindo uma mensagem com o nome e sistema de todas as mesas que o usuário está dentro.

```
local mesas = rrpg.getMesas();
local msg = "";

for i = 1, #mesas, 1 do
	local objMesa = mesas[i];

	msg = msg .. objMesa.nome .. " (" .. objMesa.sistema .. ")\n";
end;

if msg ~= "" then
	showMessage(msg);
else
	showMessage("O usuário não está em nenhuma mesa");
end;
```



Objeto Jogador

Este objeto representa um usuário em uma mesa aberta no RRPG Firecast.

Observações:

- Se um mesmo usuário estiver em mais de 1 mesa ao mesmo tempo, existirá 1 Objeto Jogador distinto para cada mesa em que ele estiver.

Herança

O **Objeto Jogador** possui todas as características de um WrappedObject. Veja:

- Objeto WrappedObject

Características

Além das características herdadas, o Objeto Jogador também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
mesa	Objeto Mesa	Somente leitura, contém o Objeto Mesa que representa em qual mesa este usuário está.
id	Integer	Somente leitura, contém o identificador interno do usuário.
login	String	Somente leitura, contém o login do usuário.
nick	String	Somente leitura, contém o

		nick do usuário.
avatar	String	Somente leitura, contém a URL da imagem de seu avatar.
capa	String	Somente leitura, contém a URL da imagem de capa do usuário.
isGold	Boolean	Somente leitura, contém true se o usuário for um assinante RRPG Gold.
isGoldPlus	Boolean	Somente leitura, contém true se o usuário for um assinante RRPG Gold Plus.
isRuby	Boolean	Somente leitura, contém true se o usuário for um assinante RRPG Ruby.
isAusente	Boolean	Somente leitura, contém true se o usuário estiver aparentemente ausente do computador.
isCnxDormente	Boolean	Somente leitura, contém true se a conexão do usuário está dormente, isto é, o usuário perdeu a conexão com o RRPG e está tentando a recuperar de volta.
isSessaoMobile	Boolean	Somente leitura, contém true se o usuário estiver conectado usando um dispositivo móvel (celular/tablet)
isEspectador	Boolean	Somente leitura, contém true se o usuário está no modo Espectador na mesa.
isJogador	Boolean	Somente leitura, contém true se o usuário estiver

		com o modo +jogador na mesa.
isMestre	Boolean	Somente leitura, contém true se o usuário estiver com o modo +mestre na mesa.
isJuggernaut	Boolean	Somente leitura, contém true se o usuário estiver com o modo +juggernaut na mesa.
isMudo	Boolean	Somente leitura, contém true se o usuário estiver com o modo +mudo na mesa.
isCego	Boolean	Somente leitura, contém true se o usuário estiver com o modo +cego na mesa.
haveVoice ou haveVoz	Boolean	Somente leitura, contém true se o usuário estiver com o modo +voz na mesa.
codigoInterno	Integer	Somente leitura, contém um número que identifica o jogador naquela mesa.
personagemPrincipal	Integer	Somente leitura, o código interno do item do personagem principal do jogador na mesa. Este valor é o mesmo valor da propriedade "codigoInterno" dos objetos BibliotecaItem O valor -1 significa que o jogador não tem personagem.

Métodos

Método	Descrição

jogador:isType(typeName)	Retorna true se passado "jogador" como parâmetro.
jogador:requestSetCego(cego)	Requisita a alteração da propriedade isCego do jogador. Parâmetros: ○ cego - Booleano, onde true significa colocar o modo "+cego" e false colocar o modo "-cego". Observações: ○ A alteração do comportamento só ocorre após o servidor RRPG aprovar a mudança. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG.
jogador:requestSetMudo(mudo)	Requisita a alteração da propriedade isMudo do jogador. Parâmetros: ○ mudo - Booleano, onde true significa colocar o modo "+mudo" e false colocar o modo "-mudo". Observações: ○ A alteração do comportamento só ocorre após o servidor RRPG aprovar a

	mudança. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG.
jogador:requestSetVoz(voz)	Requisita a alteração da propriedade haveVoice do jogador. Parâmetros: ○ voz - Booleano, onde true significa colocar o modo "+voz" e false colocar o modo "-voz". Observações: ○ A alteração do comportamento só ocorre após o servidor RRPG aprovar a mudança. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG.
jogador:requestSetJogador(isJogador)	Requisita a alteração da propriedade isJogador do usuário. Parâmetros: ○ isJogador - Booleano, onde true significa colocar o modo "+jogador" e false colocar o modo "-jogador". Observações: ○ A alteração do

	comportamento só ocorre após o servidor RRPG aprovar a mudança. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG.
jogador:requestKick();	Requisita o kick (expulsão) do jogador da mesa. Observações: ○ O kick só ocorre após o servidor RRPG aprová-lo. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG.
jogador:requestSetBarValue(index, currentValue, maxValue)	Requisita a alteração de valores em uma das barrinhas de status do jogador. Parâmetros: ○ index - um número que varia de 1 a 4, indicando qual barrinha deseja-se alterar os valores. ○ currentValue - Um número representando o novo valor atual da barrinha ou **nil** se não quiser alterar este campo. ○ maxValue - Um número representando o novo valor

	máximo da barrinha ou **nil** se não quiser alterar este campo. Observações: ○ A alteração só ocorre após o servidor RRPG aprová-la. ○ Esta função não faz nada se o usuário do plug-in não for assinante do RRPG e o dono da mesa atual não for assinante do Gold Plus.
jogador:requestSetEditableLine(index, text)	Requisita a alteração de texto de uma das linhas digitáveis do jogador. Parâmetros: ○ index - um número que varia de 1 a 2, indicando de qual linha se deve alterar o texto. ○ text - Uma cadeia de caracteres/texto contendo o novo texto que será aplicao à linha digitável especificada.. Observações: ○ A alteração só ocorre após o servidor RRPG aprová-la. ○ Esta função não faz nada se o usuário do plug-

	in não for assinante do RRPG e o dono da mesa atual não for assinante do Gold Plus.
jogador:getBarValue(index)	Retorna os valores atuais de uma das barrinhas de status do jogador. Parâmetros: ○ index - um número que varia de 1 a 4, indicando de qual barrinha se deseja obter informações. Retorno: Esta função retorna 3 valores na seguinte ordem: ○ currentValue - Um número, valor atual da barrinha ○ maxValue - Um número, valor máximo da barrinha ○ percent - Um número, percentual da barrinha Observações: ○ Se index indicar uma barrinha que não está ativa na mesa, os três valores retornados (currentValue, maxValue e percent) serão **nil**. ○ Se o usuário atual não tiver

	permissões para ver os números da barinha (isto é, a visibilidade está protegida), currentValue e maxValue serão retornados como **nil** enquanto percent trará uma porcentagem válida. Exemplo: local atual, maximo, percent = player:getBarValue(1);
jogador:getEditableLine(index)	Retorna os o texto atual de uma das linhas digitáveis do jogador. Parâmetros: ○ index - um número que varia de 1 a 2, indicando de qual linha se deseja obter informações. Retorno: ○ Uma cadeia de caracteres/texto contendo o que está escrito na linha digitável especificada.

Exemplos

Exemplo 1 - Uma ficha que exibe uma mensagem com a lista de Login e Nick dos jogadores que estão na mesa

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<button>
		<event name="onClick">
			local msg = "";
			local mesaDoPersonagem = rrpg.getMesaDe(sheet);
			local jogadores = mesaDoPersonagem.jogadores;

			for i = 1, #jogadores, 1 do
				local objJogador = jogadores[i];
				msg = msg .. objJogador.nick .. " (" ..
objJogador.login .. ")\n";
			end;

			showMessage(msg);
		</event>
	</button>
</form>
```

Neste exemplo, também foram usados:

- função rrpg.GetMesaDe
- propriedade jogadores do Objeto Mesa

Objeto Rolagem

Este objeto contém informações sobre uma rolagem de dados que ocorreu ou que poderá ocorrer.

Você pode criar um objeto rolagem utilizando a função rrpg.interpretarRolagem

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
tipo	Enumerado: ● "direta"	Somente leitura, contém o tipo da rolagem.

	● "storyteller"	"direta" - Rolagem de dados padrão. "storyteller" - Rolagem de dados storyteller. Quantidade de dados cuja face seja maior ou igual a algum numero e que um resultado natural 1 anula um sucesso
possuiAlgumDado	Boolean	Somente leitura, contém true se a rolagem possuir pelo menos um dado de qualquer face. Exemplo: Na rolagem "+5" a propriedade conteria false, enquanto na rolagem "d6 + 5" conteria true.
asString	String	Somente leitura, contém a rolagem como uma cadeia de caracteres.
isRolado	Boolean	Somente leitura, contém true se a rolagem já foi executada e possui o resultado, ou false se ainda não.
resultado	Integer	Somente leitura, contém o valor resultado da rolagem se ela já tiver sido feita. Se a rolagem ainda não tiver executada, contém 0.
ops	Arranjo de Objetos OperacaoRolagem	Contém a lista de operações "aritméticas" da rolagem. É um arranjo de tabelas/objetos, isto é, uma tabela lua indexada de 1 a N, onde cada posição possui uma tabela

		conforme a operação... Veja Lista de Operações de Rolagens

Métodos

Método	Descrição
rolagem:interpretarString(str)	Dado uma cadeia de caracteres representando uma rolagem de dados (Exemplos: "1d20 + 5", "1d20 + 1d10 - 5", etc..), atualiza as informações da rolagem de dados. Observação: Se "str" não contiver um texto de uma rolagem válida, o objeto rolagem é resetado.
rolagem:concatenar(outraRolagem)	Adiciona ao final desta rolagem de dados as operações "aritméticas" de outra rolagem, juntando-as. Parâmetros: ○ outraRolagem - uma cadeia de caracteres ou um outro objeto rolagem que será concatenada. Retorno: ○ Retorna o próprio objeto rolagem.
rolagem:rolarLocalmente()	Executa a rolagem localmente. Esta função realiza os sorteios e demais operações sem haver necessidade do usuário estar em uma mesa. Observações: ● Após a chamada desta função, o resultado pode ser acessado através das propriedades "resultado" e "ops". Este resultado não será exibido em lugar algum automaticamente e depende de você tratá-lo conforme desejar. ● Se você executar este método em uma rolagem que já foi rolada, o

	resultado anterior será sobrescrito com novos resultados.

Operações de Rolagem (Cada item do arranjo Rolagem.ops)

Cada operação de rolagem é uma tabela lua com as seguintes propriedades:

Propriedade	Tipo	Descrição
tipo	Enumerado: ● "dado" ● "imediato" ● "soma" ● "subtracao" ● "comparacao" ● "ordenacao" ● "desconhecido"	Determina qual é a operação. "dado" - Um grupo de dados a serem rolados "imediato" - Um número constante especificado pelo usuário. "comparacao" - Uma comparação entre números/dados. "ordenacao" - Pedido para ordenar o resultado de forma decrescente. Os demais valores são autoexplicativos.
quantidade	Integer	Quando o tipo == "dado", determina quantos dados da face informada serão rolados.
face	Integer ou String	Quando tipo == "dado", contém qual é face dos dados.. Exemplos: ● 4 ● 6 ● 8 ● 10 ● "F" (dado fudge)
resultados	Arranjo de Integer	Quando tipo == "dado" e se a rolagem já tiver sido feita, contém um arranjo

		de números representando o resultado individual de cada dado na operação.
comparacao	String	Quando tipo == "comparacao", contém qual é o comparador.
valor	Integer	Quando tipo == "imediato" contém o valor do número constante literal informado.

Exemplos

Exemplo 1 - Uma ficha que rola dados na mesa

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFichaTeste">

	<button>
		<event name="onClick">
			local rolagem =
rrpg.interpretarRolagem(sheet.ataqueDoPersonagem);

			if not rolagem.possuiAlgumDado then
				-- se o usuario não tiver preenchido qual
dado rolar,
				-- vamos adicionar um 1d20 + na "fórmula
da rolagem"
				rolagem =
rrpg.interpretarRolagem("1d20"):concatenar(rolagem);
			end;

			local mesaDoPersonagem = rrpg.getMesaDe(sheet);
			mesaDoPersonagem.chat:rolarDados(rolagem, "Ataque
do personagem");
		</event>
	</button>
```

```
</form>
```

Exemplo 2 - Rolar localmente e exibir uma mensagem personalizada.

```
local msg = "";

-- Interpretar uma rolagem, digamos: 4d6 + 2d4 + d20 + 5
local rolagem = rrpg.interpretarRolagem("4d6 + 2d4 + d20 + 5");

-- Executar a rolagem localmente
rolagem:rolarLocalmente();

-- Loop para navegar em cada uma das operações da rolagem
for i = 1, #rolagem.ops, 1 do
        local operacao = rolagem.ops[i];

        -- Vamos verificar que tipo de operação é esta.

        if operacao.tipo == "dado" then
                -- Adicionar à mensagem informações sobre este conjunto de
dados.
                msg = msg .. operacao.quantidade .. "d" .. operacao.face
.. ": \n  ";

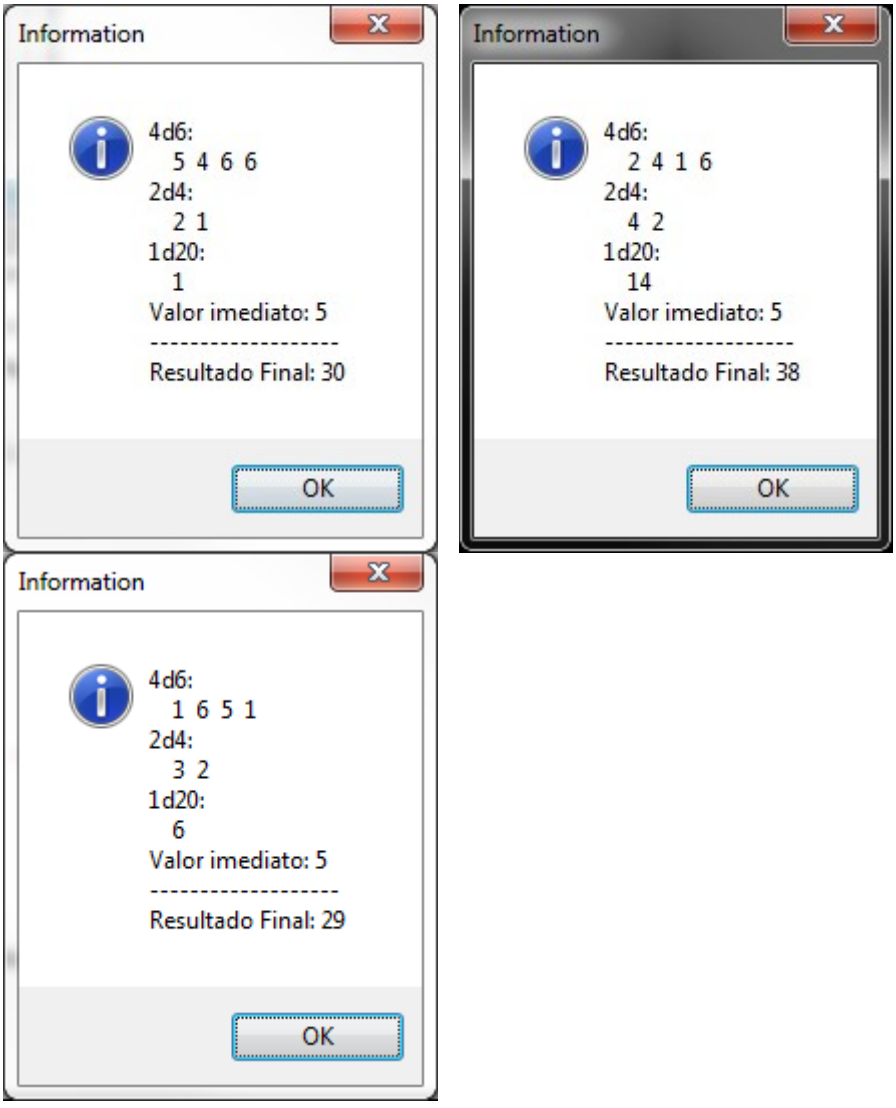
                -- Loop percorrendo cada um dos resultados individuais
deste conjunto de dados.
                for j = 1, #operacao.resultados, 1 do
                        msg = msg .. "  " .. operacao.resultados[j];
                end;

                msg = msg .. "\n";
        elseif operacao.tipo == "imediato" then
                msg = msg .. "Valor imediato: " .. operacao.valor .. "\n";
        end;
end;

-- Concatenar à mensagem o resultado final da rolagem
msg = msg .. "------------------- \nResultado Final: " ..
```

```
rolagem.resultado;

-- Exibir a mensagem que montamos.
showMessage(msg);
```



Exemplo 3 - Uma ficha fazendo um teste (de resistência, por exemplo) e postando o resultado na mesa.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmFTeste">
        <script>
                local function realizarTesteDeResistencia()
                        -- obter a mesa do personagem
                        local mesaDoPersonagem = rrpg.getMesaDe(sheet);


                        -- se o usuário não preencheu modificador, vamos
usar o valor 0
                        sheet.modificador = sheet.modificador or 0;

                        mesaDoPersonagem.chat:rolarDados("1d20 + " ..
sheet.modificador, "Teste de Resistência",
                                function (rolado)
                                        -- A dificuldade do teste é 15


                                        if rolado.resultado >= 15 then

mesaDoPersonagem.chat:enviarMensagem("SUCESSO =) você tirou " ..
rolado.resultado);
                                        else

mesaDoPersonagem.chat:enviarMensagem("FALHA =/ você tirou " ..
rolado.resultado);
                                        end;

                                end);
                end;
        </script>

        <button onClick="realizarTesteDeResistencia()"/>
</form>
```

Objeto Bibliotecaltem

Este objeto representa um item na biblioteca de uma mesa no RRPG Firecast.

Herança

O **Objeto BibliotecaItem** possui todas as características de um WrappedObject. Veja:

- Objeto WrappedObject

Características

Além das características herdadas, o Objeto BibliotecaItem também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
room **mesa**	Objeto Mesa	Somente leitura, contém o Objeto Mesa que representa em qual mesa este item está.
parent **pai**	Objeto BibliotecaItem	Somente leitura, contém o objeto BibliotecaItem que é pai deste item. Observação: Se o item for raiz da biblioteca, esta propriedade conterá **nil**.
children **filhos**	Arranjo de Objeto BibliotecaItem	Contém um arranjo de Objetos BibliotecaItem representando os sub-itens deste item. Um arranjo é uma tabela lua indexada de 1 a N.
tipo	Enumerado: ● "biblioteca" ● "diretorio" ● "personagem" ● "imagem" ● "sceneUnitClass" ● "scene2" ● "scene3"	Somente leitura, contém que tipo de item este é. "biblioteca" - diretório raiz da biblioteca. "diretorio" - uma pasta na biblioteca "personagem" - Um personagem "imagem" - Uma imagem (implementação futura) "sceneUnitClass" - Um item do

		Scene2 "scene2" - Um tabuleiro de Scene 2 "scene3" - Um tabuleiro/grid de Scene 3
codigoInterno	Integer	Somente Leitura, contém o código interno que identifica este item de biblioteca unicamente no RRPG Firecast.
name **nome**	String	Somente leitura, contém o nome do item da biblioteca. Para alterar esta propriedade, utilize o método asyncUpdate deste objeto.
firecastURI	String	(Read-only) A string containing the shareable Firecast URI for this library item. You can use this URI with the GUI.asyncOpenFirecastURI() function.
ownerLogin **loginDono**	String	Somente leitura, contém o login Firecast do usuário que é dono deste item. Para alterar esta propriedade, utilize o método asyncUpdate deste objeto.
owner **dono**	Objeto Jogador	Somente leitura, contém o o objeto Jogador do dono deste item. Observação: Se o dono não estiver na mesa no momento, esta propriedade conterá **nil.** Utilize a propriedade "loginDono" para descobrir o login do dono mesmo se ele não estiver na mesa.
creatorLogin	String	Somente leitura, contém o login do usuário que criou este item.

loginCriador		
creator **criador**	Objeto Jogador	Somente leitura, contém o objeto Jogador do usuário que criou este item. Observação: Se o criador não estiver na mesa no momento, esta propriedade conterá **nil.** Utilize a propriedade "loginCriador" se você quiser descobrir quem criou o item mesmo se ele não estiver na mesa.
visible **visivel**	Boolean	Somente leitura, contém true se o item estiver marcado para ser visível a todos da mesa. Observação: Esta propriedade pode conter true mas o item não estar de fato visível a todos. Isto ocorre quando o item está dentro de um item pai que está escondido. Para descobrir se o item está realmente visível a todos, utilize a propriedade "visivelRecursivamente". Para alterar esta propriedade, utilize o método asyncUpdate deste objeto.
recursiveVisible **visivelRecursivamente**	Boolean	Somente leitura, contém true se o item está realmente visível a todos da mesa.

Métodos

Método	Descrição
bibliotecaItem:asyncCreateChar(charParams)	Asynchronously creates a character in the room. Parameters: ● charParams - A Lua

	table with the following fields: ○ name - String containing the name of the new character to be created. ○ dataType - String identifying the sheet template the new character will use. Refer to forms dataType for details. ○ *(Optional)* visible - A boolean value; set to true if the

	new character should be visible to all, false otherwise. Default: true. ○ *(Optional)* owner Login - A string containing the user account name that will own the new character. **nil** designates an NPC. Default: **nil** Returns: ● A Promise object for the newly created Objeto Personagem Remarks: ● The character will be created within the 'BibliotecaItem'

	object that received the function call ● This function is only applicable if the 'BibliotecaItem' object receiving the function call has the attribute "tipo" set to "diretorio" or "biblioteca".
bibliotecaItem:asyncCreateDir(dirParams)	Asynchronously creates a directory in the room llibrary. Parameters: ● dirParams - A table with the following fields: ○ name - String containing the name of the new directory to be created. ○ *(Optional)* visible - A boolean value; set to true if the new directory should be visible to all,

	false otherwise. Default: true. Returns: ● A Promise object for the newly created Objeto BibliotecaItem Remarks: ● The directory will be created within the 'BibliotecaItem' object that received the function call ● This function is only applicable if the 'BibliotecaItem' object receiving the function call has the attribute "tipo" set to "diretorio" or "biblioteca".
bibliotecaItem:asyncCreateScene3(gridParams)	Asynchronously creates a grid in the room. Parameters: ● gridParams - A table with the following fields: ○ name - String containing the name of the new grid to be created. ○ *(Optional)*

	visible - A boole an value; set to true if the new grid should be visible to all, false other wise. Defaul t: true. Returns: ● A Promise object for the newly created Objeto BibliotecaItem Remarks: ● The grid will be created within the 'BibliotecaItem' object that received the function call ● This function is only applicable if the 'BibliotecaItem' object receiving the function call has the attribute "tipo" set to "diretorio" or "biblioteca".
bibliotecaItem:asyncDelete()	Asynchronously delete this bibliotecaItem. Parameters: None

	Returns: ● A promise that tracks the asynchronous process.
bibliotecaItem:asyncMoveTo(newParent)	Asynchronously move this BibliotecaItem to a new location in the library tree hierarchy. Parameters: ● newParent - Another BibliotecaItem object representing the new parent for this BibliotecaItem Returns: ● A promise that tracks the asynchronous process. Remarks ● This operation only works if the newParent parameter has the attribute "tipo" set to "diretorio" or "biblioteca"."
bibliotecaItem:asyncUpdate(changes)	Asynchronously update the attributes of this BibliotecaItem. Parameters: ● changes - A table representing the attributes you wish to modify. It may contain the following fields: ○ *(Optional)* name - String containing

	the new name for this BibliotecaItem. ○ *(Optional)* visible - A boolean value; set to true if the BibliotecaItem should be visible to all, false otherwise. ○ *(Optional)* editionBlocked - A boolean value; set to true if character edition should be blocked, false other

	wise. Only applicable to characters. ○ *(Optional)* ownerLogin - A string containing the user account name that will own the character. Use an empty string "" to designate an NPC. Only applicable to characters. ○ *(Optional) bar0Val (Optional) bar1Val (Optional)*

	bar2Val (Optional) bar3Val - A number, representing the new current value for the corresponding character bar. Only applicable to characters. ○ *(Optional) bar0Max (Optional) bar1Max (Optional) bar2Max (Optional) bar3Max* - A number, indicating the new maxim

	um value for the corresponding character bar. Only applicable to characters. ○ *(Optional) edtLine0 (Optional) edtLine1* - A string, indicating the new content for the corresponding character editable line. Only applicable to characters. ○ *(Optional) avatar* - A string,

	denoting the new image URL for the character avatar. Only applicable to characters. Returns ● A promise that tracks the asynchronous process. Remarks: ● All fields inside "changes" parameter are optional. Its presence means you want to change that attribute while its absence means to keep it unchanged.
bibliotecaItem:isType(typeName)	Retorna true se passado "bibliotecaItem" como parâmetro.

Exemplos

Exemplo 1 - Obter todos os nomes de todos os itens na biblioteca de uma mesa

```lua
local function obterNomesRecursivo(bibItem)
        local itensFilhos = bibItem.children;
        local nomes = bibItem.name;
        local i;

        for i = 1, #itensFilhos, 1 do
                local bibItemFilho = itensFilhos[i];
                local nomesDoFilho = obterNomesRecursivo(bibItemFilho) or
"";

                if nomesDoFilho ~= "" then
                        nomes = nomes .. "\n" .. nomesDoFilho;
                end;
        end;

        return nomes;
end;


local nomesDeTodosOsItens = obterNomesRecursivo(umObjetoMesa.biblioteca);
showMessage(nomesDeTodosOsItens);
```

Objeto Personagem

Este objeto representa um personagem na biblioteca de uma mesa no RRPG Firecast.

Herança

O **Objeto Personagem** possui todas as características de um Objeto BibliotecaItem. Veja:

- Objeto BibliotecaItem
- Objeto WrappedObject

Características

Além das características herdadas, o Objeto Personagem também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
dataType	String	Somente leitura, contém o dataType da ficha do personagem, isto é, em qual formato de dados está a ficha do personagem.
editionBlocked **escritaBloqueada**	Boolean	Somente leitura, contém true se a ficha do personagem estiver com marcada como edição bloqueada. Para alterar esta propriedade, utilize o método asyncUpdate deste objeto.
avatar	String	Somente leitura, contém a URL da imagem do personagem. Para alterar esta propriedade, utilize o método asyncUpdate deste objeto.
bar0Val **bar1Val** **bar2Val** **bar3Val**	Integer	Read-only, the current value for the corresponding character bar. To modify this attribute, utilize this object's asyncUpdate method.
bar0Max **bar1Max** **bar2Max** **bar3Max**	Integer	Read-only, the maximum value for the corresponding character bar. To modify this attribute, utilize this object's asyncUpdate method.
edtLine0	String	Read-only, content for the

edtLine1		corresponding character editable line. To modify this attribute, utilize this object's asyncUpdate method.

Métodos

Método	Descrição
personagem:isType(typeName)	Retorna true se passado "personagem" como parâmetro.

personagem:asyncOpenNDB([options])	Asynchronously opens the NodeDatabase of the character.
	
	Arguments:
	
	● *(Optional)* options - A Lua table describing additional settings that may contain:
	○ skipLoad - a boolean. If **true**, the returned Promise will be resolved before the load process is finished. If **true**, you will receive "Loading node" instead of a "Loaded node" Default: **false**
	
	Return:
	
	● a Promise for a Node Object. Unless *options.skipLoad* is tru the Promise resolves after all data is downloaded and available for use.
	
	Remarks:
	
	● Each character possesses only one NodeDatabase.
	● Changes made to the NodeDatabase opened by this function are automatically synchronized with other users who have also opened the same NodeDatabase.
	● If the user leaves the room, the NodeDatabase will have its connection dropped and will no longer synchronize change
	● By default, users with +GM mode and the character owner can read and write the NodeDatabase, while other users c only read.
	
	Example:
	
	`local promise = character:asyncOpenNDB();`
	`local node = await(promise);`
	`-- variable node is ready to be used here`

personagem:loadSheetNDB(callback)	**FUNÇÃO DESCONTINUADA!** Utilize a função personagem:asyncLoadNDB() ao invés desta. Abre um NodeDatabase do personagem que está fisicamente armazenado no servidor do RRPG. Parâmetros: ○ callback - Uma função que será invocada quando o NodeDatabase for carregado ou quando ocorrer um erro no carregamento. ■ Quando o carregamento fo bem sucedido, a função callback será chamada contendo o Objeto Nodo raiz do NodeDatabase no primeiro parâmetro. Este node é o mesmo equivalen ao "sheet" do lua form de fichas. ■ Quando ocorrer algum err no carregamento, a função callback será chamada contendo **nil** no primeiro parâmetro. Observações: ○ Esta função é assíncrona, isto é, o código LUA continua sua execução normal enquanto o NDB é carregado em segundo plano. É preciso informar o parâmetro "callback" para obter o NodeDatabase carregado. ○ Cada personagem possui apenas um NodeDatabase. ○ Alterações feitas no NodeDatabase aberto por esta função são automaticamente sincronizadas com os outros usuários que também abriram o mesmo NodeDatabase, inclusives com a ficha aberta localmente (s tiver aberta). ○ Se o usuário sair da mesa, o NodeDatabase terá sua conexão cortada e não mais sincronizará as alterações. ○ Por padrão, os usuários com o modo +mestre e o dono do personagem podem le e alterar o NodeDatabase enquanto os demais usuários podem apenas ler valores ○ A função "callback" pode demorar um pou para ser chamada, pois um download precisará ser feito se o nodedatabase do personagem ainda não foi carregado

Objeto SceneUnitClass

Este objeto representa um item de scene 2 na biblioteca de uma mesa no RRPG Firecast.

Herança

O **Objeto SceneUnitClass** possui todas as características de um Objeto BibliotecaItem. Veja:

- Objeto BibliotecaItem
- Objeto WrappedObject

Características

Além das características herdadas, o Objeto SceneUnitClass também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
tamanhoX	Double	(Somente leitura) Contém quantos metros o item ocupa em sua largura no tabuleiro de combate.
tamanhoY	Double	(Somente leitura) Contém quantos metros o item ocupa em sua altura no tabuleiro de combate.
camada	Enumerado: ● "tokens" ● "objects" ● "background"	(Somente leitura) Contém em qual camada o item pertence por padrão no tabuleiro de combate.
descricao	String	(Somente leitura) Contém um texto descritivo do item de scene.
urlImg	String	(Somente leitura) Contém a URL da imagem do item de scene.
facing	Enumerado:	(Somente leitura) Contém como o item deve se

	• "rotate" • "drawArrow"	comportar quando for rotacionado no tabuleiro de combate. "rotate" - Rotacionar também a imagem do item. "drawArrow" - Não rotacionar a imagem e desenhar uma seta indicando a direção do item.
snapToGrid	Boolean	(Somente leitura) Contém true se o item deve se alinhar ao grid do tabuleiro de combate.
direcaoImg	Double	(Somente leitura) Contém, em graus, a direção do conteúdo da imagem... Isto é, "para onde a imagem está apontando"
layoutCenterX	Double	(Somente leitura) Contém onde o CENTRO do token está na imagem no eixo X. Exemplos de valores: 0.5 - O centro do token está no centro da imagem. 1.0 - O centro do token está no canto da direita da imagem. 0.0 - O centro do token está no canto esquerdo da imagem
layoutCenterY	Double	(Somente leitura) Contém onde o CENTRO do token está na imagem no eixo Y. Exemplos de valores: 0.5 - O centro do token está no centro da imagem.

		1.0 - O centro do token está no canto inferior da imagem. 0.0 - O centro do token está no canto superior da imagem
layoutTamX	Double	(Somente leitura) Contém o tamanho da imagem em relação ao tamanho do token, no eixo X. Exemplos de valores: 1.0 - A largura da imagem é a mesma que a largura do token. 1.5 - A largura da imagem é 1.5x a largura do token 0.5 - A largura da imagem é 0.5x a largura do token.
layoutTamY	Double	(Somente leitura) Contém o tamanho da imagem em relação ao tamanho do token, no eixo Y. Exemplos de valores: 1.0 - A altura da imagem é a mesma que a altura do token. 1.5 - A altura da imagem é 1.5x a altura do token 0.5 - A altura da imagem é 0.5x a altura do token.

Métodos

Método	Descrição
sceneUnitClass:isType(typeName)	Retorna true se passado "sceneUnitClass" como parâmetro.

Objeto Chat

Este objeto representa uma interface de batepapo no RRPG Firecast.

Herança

O **Objeto Chat** possui todas as características de um WrappedObject. Veja:

- Objeto WrappedObject

Características

Além das características herdadas, o Objeto Chat também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
impersonation	Impersonation table	Represents the current UI impersonation selected by the user in the chat. Remarks: ● Not all chat types allow impersonations. ● Attempts to change the UI impersonation while the current user does not have permission to impersonate will be ignored. You can use the setImpersonation method if you need to identify if the change was accepted.
medium	Medium table	(Read-only) Contains the identification of the chat medium to which this object is related.
room	Objeto Mesa	(Somente leitura) Contém a mesa a qual este chat pertence. **Importante**: Pode ser **nil** se o chat não pertencer a uma mesa.
participants	Array of Entity table	(Read-only) Contains the identification of all participants in

		this chat
talemarkOptions	TalemarkOptions table	Contains the Talemark options used when the user sends messages to this chat. Remarks: ● Assigning **nil** to this property will reset the Talemark options to the user settings.

Métodos

Método	Descrição
chat:asyncInvite(logins)	Asynchronously invites users to this chat. Arguments: ● logins - An array of strings containing the usernames to be invited to the group private chat. Return: ● A Promise object to track the operation Remarks: ● This method works with group private chats only. It is possible to check what kind of chat this is by accessing the value of "medium" property.
chat:asyncQueryLogRecs([params])	Asynchronously queries the log for messages stored on the server Arguments: ○ *(Optional)* params - A QueryLogRecs Params table describing how to fetch the LogRecs. If missing, the

	latest messages from the chat will be fetched. Return: ○ A promise for a QueryLogRecs Response table. Remarks: ○ This function retrieves messages from the server's log. To read local messages, use the method chat:readLogRecs() instead.
chat:asyncRoll(expression[, message[, params]])	Asynchronously executes a dice roll in the chat. Arguments: ○ expression - A string containing the expression of the roll, such as "1d20 + 5," or a Roll Object. ○ *(Optional)* message - A string containing an additional message describing the roll. ○ *(Optional)* params - A SendMessage Parameters table

	containing parameters of the message.
	
	Return:
	
	○ A promise for three pieces of information in the following order:
	■ A
	■ A
	■ A
	
	Remarks:
	
	○ See also the Example 3
	
	Short example:
	
	``
	local promise = chat:asyncRoll("1d20 + 5");
	
	local n, rollObject, logRec = await(promise);
	
	-- the variables n, rollObject, and logRec contain the return values of the promise.
	``

chat:asyncSendAction(content[, params])	Asynchronously sends an action message to the chat. Arguments: ○ content - A string containing the action message to be sent. ○ *(Optional)* params - A SendMessage Parameters table

	containing parameters of the message. Return: ○ A promise for a LogRec table that describes the sent message.
chat:asyncSendLaugh([params])	Asynchronously sends a laugh message to the chat. Arguments: ○ *(Optional)* params - A SendMessage Parameters table containing parameters of the message. Return: ○ A promise for a LogRec table that describes the sent message.
chat:asyncSendStd(content[, params])	Asynchronously sends a standard message to the chat. Arguments: ○ content - A string containing the standard message to be sent. ○ *(Optional)* params - A SendMessage Parameters table containing parameters of

	the message. Return: ○ A promise for a LogRec table that describes the sent message.
chat:enviarMensagem(msg)	Envia uma mensagem comum para o chat, como se o usuário a tivesse digitado. Parâmetros: ○ msg - a mensagem que deseja enviar no chat. Observações: ○ Prefira usar a função chat:asyncSendStd() ao invés desta.
chat:enviarAcao(acao)	Enviar uma mensagem como uma ação para o chat. Parâmetros: ○ acao - a mensagem que deseja enviar no chat com formatação de ação. Observações: ○ Prefira usar a função chat:asyncSendAction() ao invés desta.
chat:enviarRisada()	Enviar uma típica risada do RRPG para o chat. Observações:

	○ Prefira usar a função chat:asyncSendLaugh() ao invés desta.
chat:enviarMensagemNPC(npc, msg)	Envia uma mensagem para o chat como se um NPC estivesse falando. Parâmetros: ○ npc - cadeia de caracteres contendo o nome do NPC ○ msg - a mensagem que deseja enviar no chat. Observação: ○ Apenas o mestre da mesa consegue enviar mensagens como NPC. ○ Prefira usar a função chat:asyncSendStd() ao invés desta.
chat:enviarNarracao(narracao)	Enviar uma mensagem como narrador para o chat. Parâmetros: ○ narracao - a mensagem que deseja enviar no chat com formatação de narração. Observação: ○ Apenas o mestre da

	mesa consegue enviar narrações. ○ Prefira usar a função chat:asyncSendStd() ao invés desta.
chat:readLogRecs()	Reads the messages of the chat. Arguments: None Return: ○ Array of LogRec table Remarks: ○ Only messages already available on the Firecast client are returned. This function does not query the server. To query the server, use the method chat:asyncQueryLogRecs() instead. ○ The returned array is already sorted in the same way the user sees the messages on the interface. ○ Messages that are in the process of being sent or have failed to send are not

	returned by this function. ○ This function also reads the generic system log messages written to the chat.
chat:rolarDados(rolagem [, msg, calllback])	Envia uma rolagem de dados para o chat. Parâmetros: ○ rolagem - um Objeto Rolagem ou uma cadeia de caracteres contendo a rolagem de dados. ○ (OPCIONAL) msg - Uma mensagem para acompanhar a rolagem de dados. ○ (OPCIONAL) callback - Uma função que será invocada após a rolagem ser executada no chat. A função recebe no primeiro parâmetro um **NOVO** Objeto Rolagem contendo o resultado da rolagem. Observações: ○ Esta função tem comportamento assíncrono. Isto é, a função

	retorna imediatamente e o resto do código continua sendo executado enquanto a rolagem é feita em segundo plano. Depois de algum tempo, quando o RRPG já souber o resultado, é que a função "callback" será invocada. ○ A função callback pode não ser invocada se a rolagem não for autorizada/realizada pelo RRPG. Exemplo de situações: A mesa está moderada e o usuário não possui o modo +voz para rolar dados, a conexão com o servidor caiu antes da rolagem ser realizada, etc.. ○ Prefira usar a função chat:asyncRoll() ao invés desta. Veja também o exemplo 2.
chat:setImpersonation(impersonation)	Attempts to change the UI impersonation

	selected by the user in the chat. Arguments: ○ impersonation - A Impersonation table describing the new impersonation to use. Return: ○ A Boolean; Indicates if the change was accepted, and the impersonation was incorporated into the UI.
chat:write(text [, breakLine, allowSmileys]) or **chat:escrever(texto [, quebrarLinha, permitirSmileys])**	Escreve/Imprime um texto no chat. O texto não é enviado para ninguém e só é exibido para o usuário que está usando o Firecast. Parâmetros: ○ text - cadeia de caracteres contendo o texto que deve ser impresso no chat. ○ (OPCIONAL) breakLine - Este parâmetro é ignorado no Firecast 8+ ○ (OPCIONAL) allowSmileys - Booleano indicando se o RRPG deve interpretar smileys neste texto. Se

	omitido, o valor padrão true é assumido. Observações: ○ Para mais opções, verifique o método chat:writeEx()
chat:writeEx(text[, talemarkOptions])	Write/print a text in the chat. The text is not sent; only the local user can see the text. Arguments: ○ text - A string containing the text to be printed. ○ *(Optional)* talemarkOptions - A TalemarkOptions table containing how Firecast should color and format the text. If missing, Firecast will use a default TalemarkOptions. Return: ○ A Boolean indicating if the operation was concluded successfully.

Exemplos

Exemplo 1 - Uma ficha enviando um Hello World para a sua mesa.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<form name="frmFichaTeste">

        <button>
                <event name="onClick">
                        local minhaMesa = Firecast.getRoomOf(sheet);
                        local chat = minhaMesa.chat;

                        chat:enviarMensagem("Olá mundo!!!!");
                </event>
        </button>
</form>
```

Exemplo 2 - Uma ficha fazendo um teste (de resistência, por exemplo) e postando o resultado na mesa.

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmTeste">
        <script>
                local function realizarTesteDeResistencia()
                        -- obter a mesa do personagem
                        local mesaDoPersonagem =
Firecast.getRoomOf(sheet);

                        -- se o usuário não preencheu modificador,
vamos usar o valor 0
                        sheet.modificador = sheet.modificador or 0;

                        mesaDoPersonagem.chat:rolarDados("1d20 + " ..
sheet.modificador, "Teste de Resistência",
                                function (rolado)
                                        -- A dificuldade do teste é
15

                                        if rolado.resultado >= 15
then

mesaDoPersonagem.chat:enviarMensagem("SUCESSO =) você tirou " ..
rolado.resultado);
                                        else
```

```
mesaDoPersonagem.chat:enviarMensagem("FALHA =/ você tirou " ..
rolado.resultado);
                                        end;

                                    end);
                end;
        </script>

        <button onClick="realizarTesteDeResistencia()"/>
</form>
```

Example 3 - Using asyncRoll

```
<?xml version="1.0" encoding="UTF-8"?>
<form name="frmTest">
    <script>
        local function mindCheck()
            -- Get the room
            local room = Firecast.getRoomOf(sheet);


            -- Start the roll
            local promise = room.chat:asyncRoll("1d20 + " ..
tostring(sheet.mind or 0));


            -- Await for the roll to conclude
            local resultNumber = await(promise);


            -- Compare the result
            if resultNumber >= 15 then
                room.chat:write("Success!");
            else
                room.chat:write("Failure!");
            end;
```

```
        end;
    </script>


    <button onClick="mindCheck()"/>
</form>
```

Objeto WrappedObject

Objeto que representa algo no RRPG Firecast, é como uma classe pai abstrata e não faz sentido sua existência por si só.

Exemplo de objetos que são/herdam de WrappedObject:

- O objeto Mesa é um WrappedObject
- O objeto BibliotecaItem é um WrappedObject
- O objeto Personagem é um WrappedObject
- etc...

Características

Todos objetos que são WrappedObject possuem as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectID	Integer	Propriedade somente leitura, contém um número único à qual objeto do RRPG Firecast este WrappedObject representa.
objectType	String	Propriedade somente leitura, contém qual tipo de objeto este WrappedObject representa. Exemplo de valores: ● "mesa" ● "jogador" ● "bibliotecaItem"

		● "personagem" ● "rolagem" ● "chatBase" ● "object"
isObjectAlive	Boolean	Propriedade somente leitura, contém "true" se o objeto ainda é válido no RRPG ou "false" se já não é mais válido.

Métodos

Método	Descrição
objeto:isType(typeName)	Retorna true se o objeto é do tipo "typeName", senão retorna false.

Biblioteca Firecast.Messaging

A biblioteca Firecast.Messaging contém funções que tratam as notificações que o Firecast dispara para todos os plug-ins através de mensagens.

Em resumo, existem 2 tipos de eventos que o SDK 3 permite o programador "tratar":

- Eventos de tags/controles - São eventos específicos de cada controle.
- Mensagens do Firecast - São acontecimentos que são compartilhado com todos os plugins instalados, pois podem ser de interesse de todos. Exemplos: Quando a conexão do RRPG cai, quando o usuário entra em uma nova mesa, etc..

Esta biblioteca auxilia na manipulação do segundo tipo de evento descrito acima.

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "firecast.lua"
require("firecast.lua");

-- Agora é possível acessar as funções da biblioteca
Firecast.Messaging.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Firecast.Messaging

```
function Firecast.Messaging.newReceiver(messageName, callbackFunction
```

```
[, filters])
```

Cria e retorna um objeto que representa a "escuta" de mensagens eventos.

Parâmetros:

- messageName - uma cadeia de caracteres definindo qual mensagem-evento a escuta deve ouvir. Para saber quais mensagens você pode escutar, veja Lista de Mensagens Eventos do Firecast.
- callbackFunction - uma função que será invocada sempre que a escuta captar uma mensagem. Quando invocada, a função recebe no primeiro parâmetro uma tabela/objeto lua contendo as propriedades da mensagem. Cada mensagem-evento possui sua própria lista de atributos e você deve, portanto, consultar Lista de Mensagens-Eventos do Firecast para mais detalhes..
- (OPCIONAL) filters - Uma tabela/objeto Lua contendo filtros que a escuta deve verificar para decidir se deve invocar "callbackFunction" ou não. Exemplo:
  - Filters= {from="Login"}. A escuta só disparará "callbackFunction" se a mensagem possuir um atributo de nome "from" com conteúdo igual a "Login".

Retorno:

- Um objeto representando a escuta instalada ou **nil** caso falhe

Observações:

- Mantenha referência ao objeto retornado enquanto a escuta for necessária. Se o objeto for liberado da memória pelo coletor de lixo, a escuta será desinstalada automaticamente.
- A função não falha se você informar, no parâmetro "messageName", uma identificação de mensagem não esteja na lista Lista de Mensagens-Eventos do Firecast, mas, provavelmente, a escuta nunca irá disparará a função "callbackFunction".
- É possível cancelar a escuta de 3 maneiras:
  - Invocar a função receiverObj:disable() do objeto retornado, conforme exemplo abaixo
  - Retornar explicitamente **false** na "callbackFunction"
  - Deixar o coletor de lixo do Lua coletar o objeto, eventualmente.
- A diferença básica entre `Firecast.Messaging.newReceiver` e `Firecast.Messaging.listen` é a forma de gerenciar o ciclo de vida da escuta.

Exemplo:

```
require("firecast.lua");


local receiverObj =
Firecast.Messaging.newReceiver("SessionLost",
                        function (msg)
                                    -- This
function will be called when the Firecast
connection is lost


showMessage("The connect was lost!");
                        end);


-- To disable the receiver:
receiverObj:disable(); -- disable right now
receiverObj = nil;  -- let the garbage
collector disable the receiver
```

```
function Firecast.listen(messageName, callbackFunction [, filters])
ou
function Firecast.Messaging.listen(messageName, callbackFunction [,
filters])
```

Instala uma "escuta" de mensagens eventos.

Parâmetros:

- messageName - uma cadeia de caracteres definindo qual mensagem-evento a escuta deve ouvir. Para saber quais mensagens você pode escutar, veja Lista de Mensagens Eventos do Firecast.
- callbackFunction - uma função que será invocada sempre que a escuta captar uma mensagem. Quando invocada, a função recebe no primeiro parâmetro uma tabela/objeto lua contendo as propriedades da mensagem. Cada mensagem-evento possui sua própria lista de atributos e você deve, portanto, consultar Lista de Mensagens-Eventos do Firecast para mais detalhes..
- (OPCIONAL) filters - Uma tabela/objeto Lua contendo filtros que a escuta deve verificar para decidir se deve invocar "callbackFunction" ou não. Exemplo:
    - Filters= {from="Login"}. A escuta só disparará "callbackFunction" se a mensagem possuir um atributo de nome "from" com conteúdo igual a "Login".

Retorno:

- Um número identificando a escuta instalada ou **nil** caso falhe. Você pode utilizar este valor na função Firecast.Messaging.unlisten para cancelar a escuta .

Observações:

- A função não falha se você informar, no parâmetro "messageName", uma identificação de mensagem não esteja na lista Lista de Mensagens-Eventos do Firecast, mas, provavelmente, a escuta nunca irá disparará a função "callbackFunction".
- É possível cancelar a escuta de 2 maneiras:
  - Invocar a função Firecast.Messaging.unlisten passando o valor retornado por esta função (rrpg.messaging.listen) como parâmetro;
  - Retornar explicitamente **false** na "callbackFunction"
- A diferença básica entre `Firecast.Messaging.newReceiver` e `Firecast.Messaging.listen` é a forma de gerenciar o ciclo de vida da escuta.

Exemplo:

```
require("firecast.lua");

Firecast.Messaging.listen("SessionLost",
        function (detalhesDaMensagem)
                -- Esta função será chamada
quando a conexão do RRPG for perdida
                showMessage("A conexão foi
perdida!");
        end);
```

```
function Firecast.listenOnce(messageName, callbackFunction [, filters])
ou
function Firecast.Messaging.listenOnce(messageName, callbackFunction [,
filters])
```

Instala uma "escuta" de mensagens eventos. O funcionamento é idêntico à da função Firecast.Messaging.listen, porém a escuta é automaticamente desinstalada após captar 1 mensagem.

```
function Firecast.unlisten(listenerId)
ou
function Firecast.Messaging.unlisten(listenerId)
```

Desinstala uma "escuta" que foi previamente instalada pela função Firecast.Messaging.listen ou pela função Firecast.Messaging.listenOnce.

Parâmetros:

- listenerId - Identificação da escuta a ser desinstalada. A identificação da escuta é retornada pela função que instala escutas (Firecast.Messaging.listen ou Firecast.Messaging.listenOnce)

```
function Firecast.groupOnceListeners(listenerId1 [, listenerId2 [, listenerId3 [, listenerIdN...]
ou
function Firecast.Messaging.groupOnceListeners(listenerId1 [, listenerId2 [, listenerId3 [,
listenerIdN...]]])
```

Agrupa "escuta" de mensagens-eventos do tipo once (àquela que capta somente 1 mensagem). Quando OnceListeners são agrupados, a característica de se desinstalar automaticamente após receber a primeira mensagem é "expandida" para o grupo. Isto é, quando o Listener1 captar uma mensagem e se desinstalar, o Listener2, Listener3 e ListenerN são automaticamente desinstalados juntos.

Esta função é especialmente útil em situações que possam resultar 2 ou mais mensagens-eventos distintas. Exemplo: O código requisitou a criação de 1 novo personagem em uma mesa... duas coisas podem acontecer: O servidor pode responder à requisição ou a conexão com o rrpg pode cair. Nesta situação, devemos instalar uma escuta "Quando o servidor responder", uma escuta "Quando a conexão cair" e agrupá-las. Note que se o servidor responder, não estaremos mais interessados se a conexão cair e vice-versa.

Parâmetros:

- listenerId1, listenerId2, listenerId3, listenerId4, listenerId5, listenerIdN... As identificações de escuta que queira agrupar. Este valor é retornado pela função Firecast.Messaging.listenOnce.
  - Este é um parâmetro "vararg", isto é, você pode passar quantos valores quiser. Exemplo de chamadas válidas: Firecast.Messaging.groupOnceListeners(id1, id2) ou rrpg.messaging.groupOnceListeners(id1, id2, id3, id4, id5, id7, id8).

Lista de Mensagens-Eventos do RRPG

- Mensagens de Conexão - Mensagens eventos que são disparadas quando o estado da conexão com o RRPG server é alterado (Ex: uma perda de conexão)
- Mensagens do Gerenciador de Plugins - Mensagens eventos que são disparadas pelo gerenciador de plug-ins do RRPG (Exemplo: Um plug-in foi instalado, um plug-in foi desinstalado, etc..).
- Mensagens de Chat - Mensagens eventos que são disparadas pelas interfaces de Chat do RRPG.
- Mensagens de Estado das Mesas - Mensagens eventos que são disparados quando ocorre a alteração de algum estado das Mesas dos usuários

Mensagens de Conexão

```
"SessionLost"
```

Mensagem enviada quando o RRPG perder definitivamente a conexão com o servidor.

```
"SessionStarted"
```

Mensagem enviada quando o RRPG conseguir estabelecer uma conexão com o servidor.

```
"SessionLogin"
```

Mensagem enviada quando o processo de Login do usuário for concluído com êxito. Quando esta mensagem-evento for enviada, significa que o usuário está logado e algumas informações sobre sua conta e lista de contatos também estão disponíveis.

```
"SessionStarting"
```

Mensagem enviada quando o RRPG iniciar o processo de estabelecimento de uma nova conexão com o servidor.

```
"SessionStartFailure"
```

Mensagem enviada quando o RRPG não conseguir estabelecer uma nova conexão com o servidor.

Propriedades da mensagem:

- reason - uma cadeia de caracteres contendo a razão da conexão ter falhado.

```
"SessionBroken"
```

O link da conexão foi quebrada, isto é, a conexão TCP/IP com o servidor foi desfeita. Mas não está tudo perdido ainda: Talvez seja possível reestabelecer a conexão, e o RRPG vai tentar isso!

```
"SessionWillTryToFix"
```

O RRPG tentará recuperar o link quebrado da conexão daqui a alguns milisegundos!

Parâmetros:

- waitTime - Um número contendo quantos milisegundos o RRPG vai esperar até a tentativa de recuperação.

```
"SessionTryingToFix"
```

O RRPG tentará recuperar o link quebrado da conexão AGORA!

```
"SessionFixed"
```

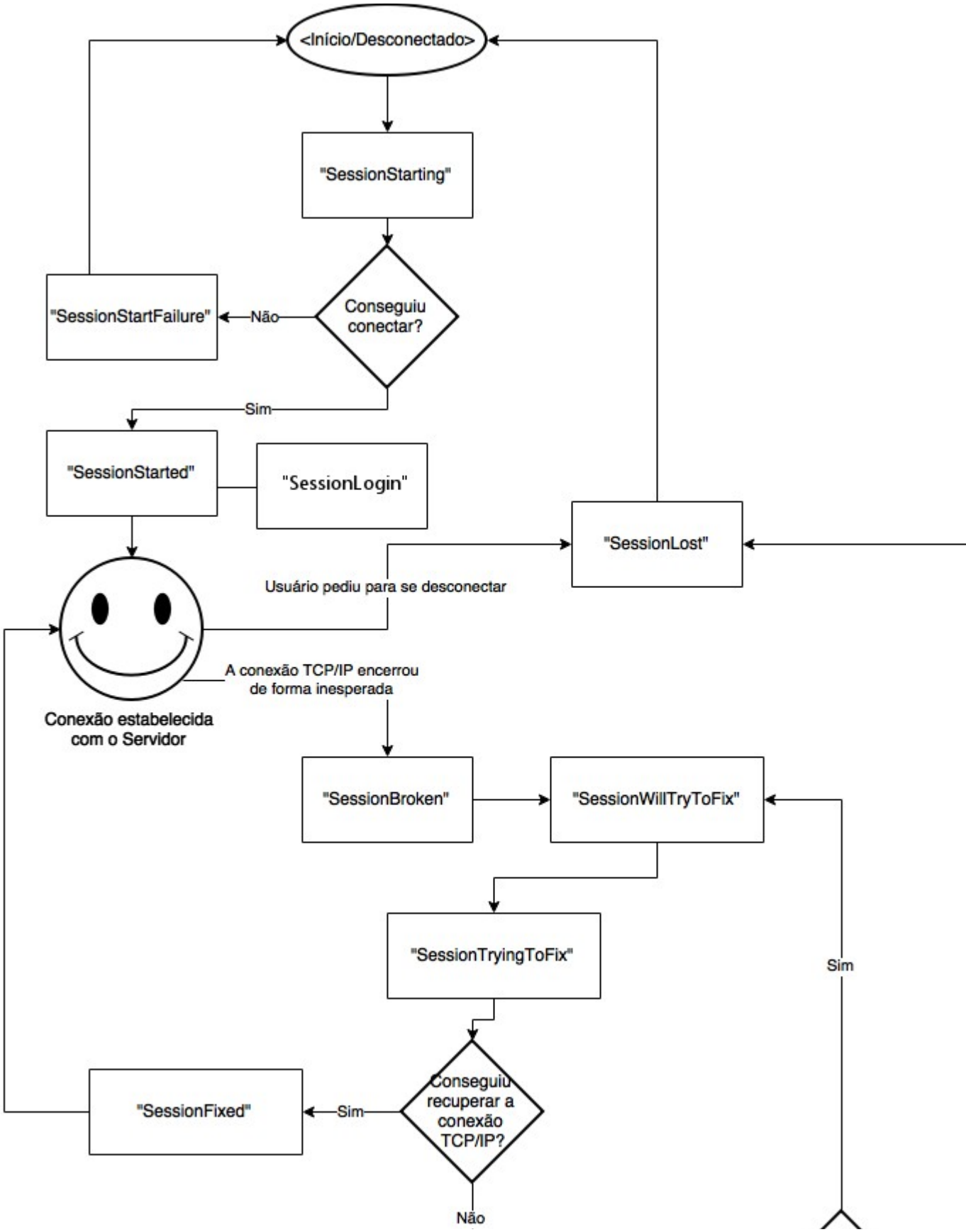
O RRPG conseguiu recuperar o link da conexão. A conexão está normal agora!

```
"SessionFixFailure"
```

Uma tentativa de recuperar o link da conexão falhou... Se a mensagem "SessionWillTryToFixIn" for disparada logo a seguir, então o RRPG continuará tentando consertar o link. Se a mensagem "SessionLost" for disparada logo a seguir, então o RRPG não vai mais tentar recuperar o link.

Estados de Conexão do RRPG

As mensagens-eventos sobre a conexão do RRPG são disparadas conforme o seguinte fluxo:


<Início/Desconectado>
"SessionStarting"
Conseguiu conectar?
Não
"SessionStartFailure"
Sim
"SessionStarted"
"SessionLogin"
"SessionLost"
Usuário pediu para se desconectar
A conexão TCP/IP encerrou de forma inesperada
Conexão estabelecida com o Servidor
"SessionBroken"
"SessionWillTryToFix"
"SessionTryingToFix"
Sim
Conseguiu recuperar a conexão TCP/IP?
Sim
"SessionFixed"
Não

Mensagens do Gerenciador de Plugins

```
"PluginInstalled"
```

Mensagem enviada quando um plug-in for instalado no RRPG Firecast.

Parâmetros:

- moduleId - Cadeia de caracteres contendo a identificação do plug-in

```
"PluginUninstalled"
```

Mensagem enviada quando um plug-in for desinstalado do RRPG Firecast

Parâmetros:

- moduleId - Cadeia de caracteres contendo a identificação do plug-in que foi desinstalado.

Mensagens de Chat

```
"ChatMessage"
```

Enviada quando uma mensagem de chat for recebida ou enviada.

Parâmetros da Mensagem:

- chat - Objeto Chat onde a mensagem foi recebida ou enviada.
- mine - Booleano, onde true significa que a mensagem foi enviada pelo usuário atual do rrpg e false significa uma mensagem enviada por outro usuário.
- type - Contém o tipo da mensagem e pode ser um dos seguintes valores:
  - "standard" - Uma mensagem de texto normal
  - "asNarrator" - Uma mensagem de texto como Narrador.
  - "asNPC" - Uma mensagem de texto como NPC
  - "action" - Uma mensagem de texto ação (/me)
  - "laugh" - Uma risada do RRPG (/rir)
  - "dice" - Uma rolagem de dados

- content - Cadeia de caracteres contendo o texto principal da mensagem. Se tipo for "dice", contém a mensagem adicional que acompanha a rolagem.
- (OPCIONAL) npc - Se o tipo da mensagem for "asNPC", contém o nome do NPC da mensagem.
- (OPCIONAL) roll - Se o tipo da mensagem for "dice", contém o Objeto Rolagem com detalhes da rolagem feita.
- (OPCIONAL) room - Se a mensagem foi enviada em um chat ligado a uma mesa, contém o Objeto Mesa da mesma.
- (OPCIONAL) player - Se a mensagem foi enviada em um chat ligado a uma mesa, contém o Objeto Jogador de quem enviou a mensagem.
- (OPCIONAL) pvtPlayer - Se a mensagem foi enviada em uma janela PVT, contém o Objeto Jogador do PVT aberto.

Observações:

- Se o usuário do Firecast for um usuário Silver, esta mensagem-evento não é enviada. Esta mensagem-evento só é enviada se o usuário for um assinante do Firecast.
- Prefira usar a mensagem evento ChatMessageEx ao invés desta.

```
"ChatMessageEx"
```

The "ChatMessageEx" event is triggered by Firecast when a message is either sent or received in a chat.

Fields:

- chat - Chat Object indicating the chat where the message was sent or received.
- mine - Boolean; Indicates whether the message was sent by the current user.
- logRec - LogRec table containing all details about the message.

Remarks:

- This event is dispatched exclusively for Premium users.
- Not all messages dispatched by this event are visible to the user, such as when someone is typing.

```
"HandleChatCommand"
```

Mensagem enviada quando o usuário digitar um comando no chat que não é conhecido pelo RRPG, dando a chance ao plug-in de tratá-lo.

Por, exemplo, quando o usuário digitar no chat: "/atacar orbe de fogo", onde "atacar" é o comando e "orbe de fogo" é o parâmetro.

Parâmetros da Mensagem:

- command - Cadeia de caracteres contendo o nome do comando que o usuário digitou.
- parameter - Cadeia de caracteres contendo o parâmetro do comando.
- chat - Objeto Chat onde o comando foi digitado.
- (OPCIONAL) room - Se o comando for digitado em um chat ligado a uma mesa, contém o Objeto Mesa da mesma, senão contém **nil**.
- (OPCIONAL) player - Se o comando for digitado em uma janela PVT, contém o Objeto Jogador com quem o usuário está conversando, senão contém **nil**.

Retorno:

- "response = {handled = true}" para sinalizar ao RRPG que o comando foi devidamente tratado e que não precisa mais continuar sua busca por alguém que trate este comando.

Observações:

- Se o usuário do RRPG for um usuário Silver, esta mensagem não é enviada. Esta mensagem-evento só é enviada se o usuário for um assinante do RRPG.
- Recomendamos que trate também a mensagem evento "ListChatCommands"

Exemplo: Tratando o comando /versao

```
rrpg.messaging.listen("HandleChatCommand",
        function (message)
                if message.command == "versao"
then
                        -- Tratar o comando
/versão

message.chat:enviarMensagem("A Minha versão é
1.0");

message.chat:escrever("Este texto é só para o
usuário");

                        -- Sinalizar ao Firecast
que o comando foi tratado com sucesso
```

```
                                message.response =
        {handled = true};
                            end;
                    end);
```

```
"ListChatCommands"
```

Mensagem enviada quando o usuário digitar "/help" ou "/?" no chat, permitindo o plug-in informar ao usuário a existência de comandos suportados.

Parâmetros:

- chat - Objeto Chat onde o help será exibido.
- (OPCIONAL) room - Se o help for ser exibido em um chat ligado a uma mesa, contém o Objeto Mesa da mesma, senão contém **nil**.
- (OPCIONAL) player - Se o help for ser exibido em uma janela PVT, contém o Objeto Jogador com quem o usuário está conversando, senão contém **nil**.

Retorno:

- response = Um arranjo de tabelas, onde cada tabela representa um comando e possui as seguintes propriedades:
  - command - Uma cadeia de caracteres contendo o comando , exemplo: "/meuComando"
  - description - Uma cadeia de caracteres contendo a descrição do comando, exemplo: "Faz isso, isso e aquilo"

Observações:

- Este evento ocorre apenas para informar ao usuário quais comandos o plug-in possui, tornando necessário também tratara mensagem-evento "HandleChatCommand" se quiser fazer o comando realmente funcionar.
- Se o usuário do RRPG for um usuário Silver, esta mensagem não é enviada. Esta mensagem-evento só é enviada se o usuário for um assinante do RRPG.

Exemplo 1: Informando, ao RRPG, a existência do comando /versao

```
    rrpg.messaging.listen("ListChatCommands",
            function(message)
```

```
                message.response =
{{command="/versao", description="Mostra a versão
do plug-in"}};
        end);
```

Exemplo 2: Informando, ao RRPG, vários comandos:

```
rrpg.messaging.listen("ListChatCommands",
        function(message)
                message.response =
{{command="/comando1", description="Este comando
faz uma coisa"},

{command="/comando2", description="Este comando
faz duas coisas"},

{command="/comando3", description="Este comando
faz tres coisas"}};
        end);
```

```
"HandleChatTextInput"
```

Evento enviado quando o usuário confirma o envio de um comando/mensagem para um chat, dando oportunidade para o plug-in alterar ou ajustar o texto de entrada antes que seja processado pelo RRPG.

Parâmetros da Mensagem:

- chat - Objeto Chat onde a mensagem será enviada.
- text - Cadeia de caracteres contendo o texto de entrada que o usuário digitou.
- isCommand - True se o usuário informou um texto para um comando do RRPG (aqueles que começam com o caracter "/").
- (OPCIONAL) room - Se a mensagem estiver prestes a ser enviada a um chat ligado a uma mesa, contém o Objeto Mesa da mesma.
- (OPCIONAL) pvtPlayer - Se a mensagem estiver prestes a ser enviada em uma janela PVT, contém o Objeto Jogador do PVT aberto.

Retorno:

- Se o plug-in quiser substituir o texto por outro, o programador deverá atribuir "message.response = {newText = <<novo texto que será utilizado>>}" para sinalizar ao RRPG que o texto de entrada deve ser substítuido pelo conteudo de newText.

Alerta:

- Tenha consciência do parametro "message.isCommand" desta mensagem! Se o usuário digitar um comando (por exemplo "/nick NovoNick"), nem toda substituíção de texto deve ser feita, ou deve ser feita de forma consciente para não atrapalhar a experiência do usuário com o programa!

Observações:

- O RRPG apenas envia esta mensagem-evento se uma das duas seguintes condições forem satisfeitas:
  - O usuário logado é um assinante pago do RRPG.
  - O usuário logado não é assinante pago do RRPG, a mensagem está prestes a ser enviada em uma mesa onde o criador é Gold Plus.

Exemplo: Um plug-in que substitui o texto "CA" por "Categoria de Armadura" nos textos digitados pelo usuário.

```
RRPG.listen('HandleChatTextInput',
    function(message)
        message.response =
            {newText = string.gsub(message.text,
"CA", "Categoria de Armadura") };
    end);
```

Mensagens de Estado das Mesas

```
"MesaJoined"
```

Enviada quando alguém entrar em alguma mesa no RRPG.

Parâmetros da Mensagem:

- room - Objeto Mesa representando a mesa onde o jogador entrou.
- me - Booleano, onde true significa que o usuário atual do RRPG foi quem entrou na mesa.

- player - Objeto Jogador representando quem entrou na mesa.

```
"MesaParted"
```

Enviada quando alguém sair de alguma mesa do RRPG.

Parâmetros da Mensagem:

- room - Objeto Mesa representando a mesa que o jogador saiu.
- me - Booleano, onde true significa que o usuário atual do RRPG foi quem saiu da mesa.
- player - Objeto Jogador representando quem saiu da mesa.
- (OPCIONAL) text - Cadeia de caracteres contendo, opcionalmente, um texto deixada pelo usuário no momento da saída.
- isKick - Booleano, onde true significa que o jogador saiu da mesa por causa de um kick.
- (OPCIONAL) responsible - Quando "isKick" for true, este campo contem o Objeto Jogador representando quem realizou o kick. Observação: Se este atributo vier **nil** durante um kick, significa que o sistema RRPG kickou o usuário.

Biblioteca Firecast.Plugins

A biblioteca Firecast.Plugins contém funções relacionadas aos plug-ins instalados no Firecast.

Todas as funções estão contidas na table/variável "Firecast.Plugins" da unidade "plugins.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "plugins.lua"
require("plugins.lua");

-- Agora é possível acessar as funções da biblioteca
Firecast.Plugins.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções de Gerenciamento de Plugins

```
function Firecast.Plugins.getInstalledDataTypes()
```

Retorna a lista de DataTypes instalados no Firecast.

Parâmetros:

Não há parâmetros

Retorno:

- Um array (uma tabela Lua indexada de 1 a N) de objetos, onde cada objeto representa um DataType instalado e possuem, cada um, os seguintes atributos:
  - moduleId - Cadeia de caracteres identificando qual plug-in instalou o dataType.
  - dataType - Cadeia de caracteres identificando o dataType
  - formType - Cadeia de caracteres identificando o formType. Veja atributo formType da tag form para saber quais são os possíveis valores deste atributo.
  - title - Cadeia de caracteres que identifica o dataType de forma intuitiva para o usuário.

```
function Firecast.Plugins.getInstalledPlugins()
```

Retorna a lista de plug-ins SDK 3 instalados no Firecast.

Parâmetros:

Não há parâmetros

Retorno:

- Um array (uma tabela Lua indexada de 1 a N) de objetos, onde cada objeto representa um Plug-in instalado e possuem, cada um, os seguintes atributos:
  - moduleId - Cadeia de caracteres com a identificação única do plug-in.
  - name - Cadeia de caracteres contendo o nome do plug-in.
  - version - Cadeia de caracteres contendo a versão do plug-in.
  - description - Cadeia de caracteres contendo a descrição do plug-in.
  - author - Cadeia de caracteres contendo informações sobre o autor do plug-in.
  - site - Cadeia de caracteres contendo informações do website relacionado ao plug-in.
  - contact - Cadeia de caracteres contendo informações sober como entra em contato com o autor do plug-in.

- fileName - Cadeia de caracteres contendo o nome do arquivo ".rpk" relacionado a este plug-in. Este atributo contém apenas o nome curto do arquivo e não descreve sua localização no HD. Exemplos: "DnD.rpk", "Vampiros.rpk", etc..
- compilationTimestamp - A data e hora (UTC) em que o plugin foi compilado. Este atributo é representado por um número inteiro, contendo quantos **segundos** se passaram desde UNIX Epoch (01 de janeiro de 1970)
- compilationTimestampStr - A data e hora (UTC) em que o plugin foi compilado. Este atributo é representado em uma cadeia de caracteres de tamanho fixo no formato "ddmmyyyyhhnnss" (onde "nn" são os minutos)
- compilationDigest - Uma cadeia de caracteres contento o hash SHA-1 do conteúdo do plugin. Um plugin pode ser compilado mais de uma vez, e, por isso, compilationTimestamp pode variar sem que o conteúdo do plugin mude. É seguro afirmar que se compilationDigest não mudar, então o conteúdo do plugin não mudou. Este campo está presente em plugins compilados com o SDK 3.5+.

```
function Firecast.Plugins.getInstalledTablesDock()
```

Retorna a lista de "Table's Dock"/janelas acopláveis instalados no Firecast.

Parâmetros:

Não há parâmetros

Retorno:

- Um array (uma tabela Lua indexada de 1 a N) de objetos, onde cada objeto representa um Table's Dock que o usuário pode utilizar em suas mesas, cada um, os seguintes atributos:
  - moduleId - Cadeia de caracteres identificando qual plug-in instalou o TablesDock.
  - name - Cadeia de caracteres contendo o nome do formulário
  - dataType - Cadeia de caracteres identificando o dataType do formulário TablesDock
  - title - Cadeia de caracteres que identifica o Table's Dock de forma intuitiva para o usuário.

```
function Firecast.Plugins.getRPKDetails(stream)
```

Retorna algumas informações sobre um stream contendo um plug-in SDK3 (arquivo de extensão .rpk).

Parâmetros:

- stream - Um objeto Stream que possui o conteúdo do arquivo ".rpk"

Retorno:

- Se o parâmetro "stream" contiver um arquivo .rpk válido, a função retorna uma tabela lua com os seguintes atributos:
  - moduleId - Cadeia de caracteres contendo o identificador único do plug-in
  - name - Cadeia de caracteres contendo o nome do plug-in
  - version - Cadeia de caracteres contendo a versão do plug-in
  - description - Cadeia de caracteres contendo a descrição do plug-in.
  - author - Cadeia de caracteres contendo a identificação do autor do plug-in.
  - site - Cadeia de caracteres contendo informações sobre o website do plug-in.
  - contact - Cadeia de caracteres contendo informações sobre como contatar o autor do plug-in.
  - minApiVersion - Número contendo qual a versão de API do SDK3 mínima para este plug-in rodar.
  - sdkVersion - Cadeia de caracteres contendo qual a versão do SDK3 o plug-in foi compilado.
  - compilationTimestamp - A data e hora (UTC) em que o plugin foi compilado. Este atributo é representado por um número inteiro, contendo quantos **segundos** se passaram desde UNIX Epoch (01 de janeiro de 1970)
  - compilationTimestampStr - A data e hora (UTC) em que o plugin foi compilado. Este atributo é representado em uma cadeia de caracteres de tamanho fixo no formato "ddmmyyyyhhnnss" (onde "nn" são os minutos)
  - compilationDigest - Uma cadeia de caracteres contento o hash SHA-1 do conteúdo do plugin. Um plugin pode ser compilado mais de uma vez, e, por isso, compilationTimestamp pode variar sem que o conteúdo do plugin mude. É seguro afirmar que se compilationDigest não mudar, então o conteúdo do plugin não mudou. Este campo está presente em plugins compilados com o SDK 3.5+.
- Se o parâmetro "stream" **não** contiver um arquivo rpk válido, a função retorna **nil**.

Observações:

- A função começa a ler o stream à partir da posição 0.

```
function Firecast.Plugins.installPlugin(stream [,
```

```
ignoreIfOlder])
```

Instala um plug-in SDK3 no Firecast.

Parâmetros:

- stream - Um objeto Stream que possui o conteúdo do arquivo ".rpk" que deseja instalar.
- (OPCIONAL) ignoreIfOlder - Booleano indicando se a função deve ignorar a instalação caso exista uma versão mais nova já instalada deste plug-in. Se omitido, o parâmetro possui valor false.

Retorno:

- Se a instalação for bem sucedida, a função retorna "true". Se a instalação não for bem sucedida, retorna "false" e uma cadeia de caracteres contendo o motivo da falha.

Observações:

- A função começa a ler o stream à partir da posição 0.
- Por motivo de segurança, apenas plug-ins autorizados conseguem gerenciar os plug-ins do Firecast.
- Se o plug-in já estiver instalado: o plug-in é descarregado, a atualização é instalada, e a nova versão é carregada novamente.

```
function Firecast.Plugins.uninstallPlugin(moduleId)
```

Desinstala um plug-in SDK3 que está atualmente instalado no Firecast.

Parâmetros:

- moduleId - Cadeia de caracteres identificando qual plug-in deseja desinstalar. Cada plug-in possui um moduleId, que é uma identificação única do mesmo.

Retorno:

- Se a instalação for bem sucedida, a função retorna "true". Se a instalação não for bem sucedida, retorna "false" e uma cadeia de caracteres contendo o motivo da falha.

Observações:

- Por motivo de segurança, apenas plug-ins autorizados conseguem gerenciar os plug-ins do Firecast.

Intercomunicação de plug-ins.

Os plug-ins do Firecast podem comunicar entre si afim de realizar pedidos, coordenar ações, informar algum ocorrido, etc..

Todas as mensagens de plugins são compostas por:

- identificação do plug-in destinatário.
- nome da mensagem.
- conteúdo da mensagem.

Antes de receber mensagens, o plug-in deve registrar uma "escuta" através da função Firecast.Plugins.listenPM informando o nome da mensagem que ouvirá e uma função que será chamada para tratá-la.. Durante o tratamento da mensagem recebida, o plug-in receptor pode enviar uma resposta para o plug-in emissor.

Para enviar uma mensagem a outro plug-in, este deve utilizar a função Firecast.Plugins.sendPM informando a identificação do plug-in destinatário, o nome da mensagem, conteúdo, uma função callback de sucesso e uma função callback de falha.

```
function Firecast.Plugins.sendPM(moduleId, pmName, data [, callback, failureCallback])
```

Envia uma mensagem a outro plug-in instalado no Firecast.

Parâmetros:

- moduleId - Uma cadeia de caracteres contendo a identificação única do módulo/plugin que receberá a mensagem. Este é o mesmo valor encontrado no arquivo module.xml de cada plug-in. Também é possível obter este valor através da função Firecast.Plugins.getInstalledPlugins.
- pmName - Uma cadeia de caracteres contendo o nome da mensagem.
- data - O corpo da mensagem, uma tabela lua que será transmitida ao plug-in destinatário.
- (OPCIONAL) callback - Uma função que será invocada quando o plug-in destinatário responder a esta mensagem. O primeiro parâmetro da função recebe os dados enviados pelo plug-in destinatário.
- (OPCIONAL) failureCallback - Uma função que será invocada quando ocorrer algum erro no envio e/ou processamento da mensagem. O primeiro parâmetro da função recebe os dados sobre a falha.

Observações:

- Para saber se o envio foi bem sucedido, utilize o parâmetro "callback".
- Para saber se o envio **não** foi bem sucedido, utilize o parâmetro "failureCallback". A função failureCallback é invocada se o plugin destinatário não estiver instalado no Firecsat, se o plug-in destinatário não souber tratar mensagens de nome "pmName" ou se o ocorrer alguma falha de execução no plug-in que recebeu a mensagem.

```
function Firecast.Plugins.listenPM(pmName, callback)
```

Instala uma escuta de mensagens de plugins.

Parâmetros:

- pmName - Uma cadeia de caracteres contendo o nome da mensagem que esta escuta trata.
- callback - Uma função que será disparada quando a escuta capturar uma mensagem de nome igual a "pmName" e recebe, no primeiro parâmetro, o corpo da mensagem. Esta informação é uma tabela LUA contendo os mesmos dados que foram enviados pela função Firecast.Plugins.sendPM acrescida da propriedade "moduleId" (a identificação de qual plug-in enviou a mensagem).

Observações:

- Um plug-in só pode ter uma escuta para cada "pmName". Não é possível instalar 2 escutas para a mensagem "TESTE", por exemplo.
- Para enviar uma resposta a quem enviou a mensagem, a função "callback" deve:
  - Se já tiver a resposta pronta, usar "return VALOR"; ... Este valor será transmitido para a função callback do plug-in emissor.
  - Se ainda não tiver a resposta pronta, deve, dentro da função "callback", invocar o método Firecast.Plugins.setLatePMReply() e, posteriormente, utilizar as funções Firecast.Plugins.replyPM ou Firecast.Plugins.replyPMFailure para enviar a resposta ao plug-in emissor.

```
function Firecast.Plugins.setLatePMReply(message)
```

Ao receber uma mensagem, a escuta, instalada previamente pela função Firecast.Plugins.listenPM, pode invocar este método para sinalizar que ainda não tem a resposta para a mensagem e que a enviará posteriormente.

Parâmetros:

- message - A mensagem cuja resposta deseja postergar. Esta informação é a mesma que foi recebida no primeiro parâmetro da função callback da escuta (ver Firecast.Plugins.listenPM).

Observações:

- Você deve, em algum momento, invocar Firecast.Plugins.replyPM ou Firecast.Plugins.replyPMFailure para enviar uma resposta ao plug-in remetente. Utilizar setLatePMReply e não invocar Firecast.Plugins.replyPM e nem Firecast.Plugins.replyPMFailure faz com que o Firecast não libere os recursos envolvidos na comunicação, tornando seu plug-in um "devorador de memória RAM" ao longo do tempo.

```
function Firecast.Plugins.replyPM(message, data)
```

Envia uma resposta ao remetente da mensagem inter-plugin.

Parâmetros:

- message - A mensagem que deseja responder. Esta informação é a mesma que foi recebida no primeiro parâmetro da função callback da escuta (ver Firecast.Plugins.listenPM).
- data - O conteúdo da resposta. Pode ser um número, uma cadeia de caracteres, um booleano ou uma tabela lua. Esta informação será repassada no primeiro parâmetro da função "callback" de quem enviou a mensagem (ver Firecast.Plugins.sendPM)

```
function Firecast.Plugins.replyPMFailure(message, data)
```

Envia uma resposta de FALHA ao remetente da mensagem inter-plugin.

Parâmetros:

- message - A mensagem que deseja responder. Esta informação é a mesma que foi recebida no primeiro parâmetro da função callback da escuta (ver Firecast.Plugins.listenPM).
- data - O conteúdo da resposta. Pode ser um número, uma cadeia de caracteres, um booleano ou uma tabela lua. Esta informação será repassada no primeiro parâmetro da função "failureCallback" de quem enviou a mensagem (ver Firecast.Plugins.sendPM

Biblioteca Firecast.Requests

A biblioteca rrpg.requests contém funções que solicitam determinadas ações ao RRPG, como, por exemplo, criar itens na biblioteca da mesa, obter informações de perfil de um usuário, etc..

```
-- Primeiro, é necessário usar a unidade "rrpgRequests.lua"
require("rrpgRequests.lua");

-- Agora é possível acessar as funções da biblioteca
Firecast.Requests.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca Firecast.Requests

```
function Firecast.Requests.criarPersonagem(itemPai, nome, dataType, visivelATodos, dono
```

```
                                    [, successCallback [, failureCallback]])
```

FUNÇÃO DESCONTINUADA! Utilize a função bibliotecaItem:asyncCreateChar() ao invés desta.

Requisita a criação de um novo personagem na mesa.

Parâmetros:

- itemPai - Um objeto BibliotecaItem identificando onde o personagem deverá ser criado.
- nome - Uma cadeia de caracteres identificando o nome do novo personagem.
- dataType - Uma cadeia de caracteres identificando o dataType do personagem. dataType é uma identificação única de um template de ficha.
- visivelATodos - true se o personagem deve ser visível a todos ou false se for ser visível apenas ao mestre e ao dono da ficha.
- dono - Um objeto Jogador identificando quem será o dono do personagem, ou **nil** se for um NPC.
- (OPCIONAL) successCallback - Uma função que será chamada quando a criação do personagem for bem sucedida.
- (OPCIONAL) failureCallback - Uma função que será chamada quando a criação do personagem **não** for bem sucedida. Uma cadeia de caracteres contendo o motivo da falha é passada como primeiro parâmetro da função.

Retorno:

A função não retorna nada. Para saber se a criação do personagem foi bem sucedida, utilize os parâmetros "successCallback' e "failureCallback".

Observações:

- Por motivo de segurança, apenas plug-ins autorizados podem alterar a estrutura da biblioteca das mesas.
- A criação de personagens é um processo assíncrono. Sendo assim, a função não aguarda uma resposta de sucesso/falha e as funções successCallback e failureCallback serão invocadas mais tarde com o resultado.
- Não é possível criar um personagem cujo dataType não está instalado no RRPG do usuário. Para obter uma lista de dataTypes que o usuário possui instalado, utilize a função Firecast.Plugins.getInstaledDataTypes

Biblioteca SceneLib

Biblioteca que contém funções relacionadas ao Scene 3/Tabuleiro de combate do RRPG

Todas as funções estão contidas na unidade "scene.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "scene.lua"
require("scene.lua");

-- Agora é possível acessar as funções da biblioteca
SceneLib.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca SceneLib

```
function SceneLib.getLoadedScenes()
```

Retorna um array de Scenes que estão rodando no RRPG no momento.

Retorno:

- Array (tabela lua indexada de 1 a #tabela) de objeto Scene

```
function SceneLib.registerPlugin(attachCallback)
```

Registra um plug-in de scene no RRPG.

Parâmetros:

- attachCallback - Uma função que será invocada para cada Scene rodando ou que vier a ser carregado. Esta função recebe dois parâmetros na seguinte ordem: scene (um objeto Scene) e attachment (um objeto ScenePluginAttachment) representando, respectivamente, o scene o qual o plug-in deve instalar e um objeto que identifica esta ligação entre o plug-in e o scene.

Retorno:

- Um identificador do plug-in registrado que poderá ser utilizado posteriormente como parâmetro para a função SceneLib.unregisterPlugin para desregistrá-lo.

Observações:

- Um pacote .rpk (plugin SDK 3) pode instalar mais de um Plug-in de Scene.
- Um plug-in de scene pode existir mesmo quando não há nenhum Scene rodando no RRPG, não tem problema.
- Um plug-in de scene pode se anexar a mais de 1 scene ao mesmo tempo! A função "attachCallback" será chamada uma vez para cada Scene existente e uma instância do objeto ScenePluginAttachment será criada para representar cada uma desta ligação "plugin x scene".

- Assim como um plug-in pode se anexar a vários scenes, um scene pode estar anexado a vários plug-ins. É uma relação N:N (leia-se: N para N).

Exemplo de um plug-in simples que apenas exibe uma mensagem quando se anexa a um Scene:

```
require("scene.lua");

require("dialogs.lua");



SceneLib.registerPlugin(

    function (scene, attachment)

        showMessage("Plugin anexado a um tabuleiro de
tamanho " ..

                    scene.worldWidth ..
scene.worldMetricName .. " x " ..

                    scene.worldHeight ..
scene.worldMetricName ..

                    " de cor de fundo "..
scene.bkgColor);

    end);
```

```
function SceneLib.unregisterPlugin(pluginID)
```

Desregistra um plug-in de scene do RRPG que fora previamente instalado pela função SceneLib.registerPlugin

Parâmetros:

- pluginID - O identificador do plugin que deseja desinstalar. Este valor é retornado pela função SceneLib.registerPlugin .

Observações:

- Normalmente não é preciso se preocupar em desregistrar plug-ins de scene pois o RRPG já faz este trabalho automaticamente quando o .rpk é descarregado da memória.

```
function SceneLib.Math.newRotationMatrix(angle, pivoX, pivoY)
```

Cria e retorna uma Matriz de rotação 2D

Parâmetros:

- angle - O ângulo da rotação em graus
- pivoX - A posição X do pivô da rotação
- pivoY - A posição Y do pivô da rotação

Retorno:

- A Matriz de rotação. (Um tipo de dados interno)

```
function SceneLib.Math.transformPoint(x, y, matrix)
```

Transforma a posição de um ponto ao multiplicá-lo por uma matriz.

Parâmetros:

- x - A posição X do ponto
- y - A posição Y do ponto
- matrix - Uma matriz de transformação.

Retorno:

- x - A nova posição X
- y - A nova posição Y

```
function SceneLib.Math.rotatePoint(x, y, angle, pivoX, pivoY)
```

Rotaciona um ponto.

Parâmetros:

- x - A posição X do ponto
- y - A posição Y do ponto
- angle - O ângulo da rotação em graus
- pivoX - A posição X do pivô da rotação
- pivoY - A posição Y do pivô da rotação

Retorno:

- x - A nova posição X
- y - A nova posição Y

Visão geral da SceneLib

(Conteúdo desta página não consta no documento original.)

Métrica do Mundo vs Métrica da Tela

(Conteúdo desta página não consta no documento original.)

As camadas do tabuleiro

(Conteúdo desta página não consta no documento original.)

Objeto Scene

O Objeto Scene representa um "tabuleiro de combate" aberto pelo usuário e é a instância principal de um Scene e, através deste objeto, o programador consegue acessar todas as propriedades do mesmo.

O programador **não** consegue instanciar um "Objeto Scene" através do SDK 3, mas:

- Consegue descobrir todos os Scenes abertos pelo usuário utilizando a função SceneLib.getScenes
- Consegue instalar um plug-in de Scene para todos os plug-ins abertos (e que vierem a ser abertos depois) utilizando a função SceneLib.registerPlugin.

Herança

O o**bjeto Scene** herda de SceneBaseObject e possui também todas as suas características.

Características

Além das características herdadas, o objeto Scene também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
worldWidth	Double	Define a LARGURA do tabuleiro de combate. O valor desta propriedade contem o valor em Métrica do Mundo Scene.
worldHeight	Double	Define a ALTURA do tabuleiro de combate. O valor desta propriedade contem o valor em Métrica do Mundo Scene.
worldMetricNam	String	Define o nome da métrica do

e		mundo que é usada por este scene. Exemplo: "m", "km", "cm", etc.. Exemplo2: supondo que a propriedade worldWidth = 60 e worldMetricName = "km", intepreta-se que tabuleiro possui uma largura de 60 quilômetros.
bkgColor	String de Color	Define a cor de fundo do tabuleiro. Exemplo: "white", "black", "#FFDE05"
bkgImageURL	String	Contém a URL da imagem/bitmap que será renderizada como fundo do tabuleiro. Este valor: ● Pode ser um endereço de internet (exemplo: "http://xxxx.com.br/a.png") ● Pode ser o caminho de um arquivo contido no pacote do plugin. ● Pode ser um arquivo que se encontra no HD Virtual do plugin instalado.
workingLayer	Enumerado: ● "background" ● "objects" ● "tokens"	Define qual em que camada do scene o usuário está trabalhando. "map" - O usuário está trabalhando na camada Background "objects" - O usuário está trabalhando na camada Objetos "tokens" - O usuário está trabalhando na camada Tokens. Este é um valor local e não é

		compartilhado com os outros usuários que estão visualizando o mesmo scene. Saiba mais sobre as camadas em: As camadas do tabuleiro
viewport	Objeto SceneViewport	(Somente Leitura) Objeto viewport do Scene. Objeto que contém propriedades sobre a "tela" do Scene, como "Qual região do scene o usuário está vendo", "Qual o zoom que o usuário aplicou no tabuleiro?", etc..
grid	Objeto SceneGridLayer	(Somente Leitura) Objeto que contém características, informações e utilitários a respeito da GRID do scene.
items	Objeto SceneGraphicItemList	(Somente Leitura) Objeto que reúne os itens visuais do tabuleiro como tokens.
fogOfWar	Objeto SceneFogOfWar	(Somente Leitura) Objeto que reúne informações e operações com o FogOfWar/Névoa de Guerra do Tabuleiro de combate.
isGM	Boolean	(Somente Leitura) True se o usuário atual é mestre deste scene ou false.
isViewingAsGM	Boolean	True se o usuário atual for mestre e estiver visualizado como mestre do scene. O usuário atual pode ser mestre mas desejar ver como os jogadores estão enxergando o scene, então isGM = true e isViewingAsGM = false.
isViewingWalls	Boolean	True se o usuário atual for mestre e estiver visualizado as paredes/áreas opacas do tabuleiro.

currentUserID	String	(Somente Leitura) Contém o login do usuário atual que está utilizando o Scene.
renderAsPlayer	Boolean	(Somente Leitura) Uma propriedade bem peculiar e mais difícil de explicar. Algumas vezes, mesmo quando "isGM" é true e "isViewingAsGM" é true, o tabuleiro deve ser renderizado como "visualizando como jogador". Esta propriedade foi feita inicialmente pensando na integração do Firecast com o programa "OBS Studio" para streaming. Enquanto o mestre está visualizando o tabuleiro como GM, o tabuleiro deve ser renderizado como jogador para o "OBS Studio".

Métodos

Método	Descrição
scene:newTransaction()	Cria um NodeTransaction que poderá ser utilizado para gerenciar as mudanças feitas localmente no Scene. Retorna: ● Um objeto NodeTransaction que poderá ser utilizado em scene:pushTransaction()
scene:pushTransaction(transaction)	Ativa uma transação para controlar mudanças no Scene. Parâmetros: ● transaction - um objeto NodeTransaction previamente criado pela função scene:newTransaction que controlará as próximas mudanças no Scene. Observações: ● Após ativar uma transação,

	todas as alterações que ocorrerem no Scene serão mantidas localmente e não serão compartilhadas automaticamente com os outros usuários. Você deverá controlar o que acontecerá com estas mudanças ao invocar "transaction:rollback()" se quiser desfazê-las ou "transaction:commit()" para confirmá-las e compartilhar-lhas com os outros usuários. ● Para cada chamada de scene:pushTransaction(), deve haver uma chamada de scene:popTransaction().
scene:popTransaction()	Desativa a última transação ativada no Scene pela função scene:pushTransaction() Observações: ● Para cada chamada de scene:pushTransaction(), deve haver uma chamada de scene:popTransaction(). ● Se antes de chamar pushTransaction o Scene já estava sendo controlado por alguma transação, esta transação é restaurada. ● Apenas invocar este método não é o suficiente para realizar o controle das mudanças. Você o invocar o método "rollback()" da transaction se quiser desfazer as mudanças capturadas ou invocar o método "commit()" da transaction para confirmar e compartilhar as mudanças capturadas.

scene:broadcastMessage(messageId, message [, loopBack])	Envia uma mensagem a todos os outros usuários que estão com o scene aberto. Parâmetros: ● messageId - Uma cadeia de caracteres contendo a identificação da mensagem. ● message - A mensagem a ser transmitida. Aqui são aceitos textos, tabelas lua, números e booleanos. ● (OPICIONAL) loopBack - Informa se quer que esta mensagem seja retransmitida para você mesmo. Valor padrão: false Observações: ● A mensagem enviada é entregue a todos os outros usuários conectados no scene. ● Para receber as mensagens recebidas, o usuário destinatário deve ter previamente invocado a função scene:newBroadcastListener
scene:newBroadcastListener(messageId, callback)	Cria uma escuta de mensagens para receber mensagens postadas pela função scene:broadcastMessage Parâmetros: ● messageId - Uma cadeia de caracteres contendo a identificação das mensagens que deseja receber. Deve ser igual ao messageId usado na função scene:broadcastMessage. ● callback - Uma função que será invocada ao receber mensagens. Esta função recebe três parâmetros na seguinte ordem: ○ sender - Cadeia de

	caracteres contendo o **login** do usuário que enviou a mensagem ○ messageId - Cadeia de caracteres contendo a identificação da mensagem recebida ○ message - A mensagem que foi postada. Retorno: ● Retorna um objeto representando o Listener. Observações: ● Importante: Mantenha referência ao objeto retornado por esta função enquanto quiser que a escuta esteja ativa. Se o coletor de lixo LUA coletar este objeto, a escuta é destruída e a função callback não será mais invocada. ● Se quiser destruir a escuta imediatamente, invoque a função destroy do objeto retornado.... obj:destroy();
scene:sendDelayedUpdates()	Force the scene to transmit all the pending delayed updates caused by the network transfer optimization

Eventos

Nome do evento	Descrição
onWorkingLayerChange	Evento que é chamado quando a propriedade workingLayer for alterada.

onGMStateChange	Evento que é chamado quando as propriedades isGM, isViewingAsGM ou isViewingWalls mudarem.

Exemplos

Exemplo 1 - Um Plugin de Scene que, ao se anexar a um tabuleiro, exibe uma mensagem com algumas características do Scene.

```
require("scene.lua");

require("dialogs.lua");



SceneLib.registerPlugin(

    function (scene, attachment)

        showMessage("Plugin anexado a um tabuleiro de tamanho " ..

                    scene.worldWidth .. scene.worldMetricName .. " x " ..

                    scene.worldHeight .. scene.worldMetricName ..

                    " de cor de fundo ".. scene.bkgColor);

    end);
```

Objeto SceneBaseObject

A classe SceneBaseObject é uma abstração e todos os outros objetos do SceneLib herdam desta classe. Isto significa que não é possível existir um objeto SceneBaseObject diretamente e que sua existência está condicionada à existência dos objetos que herdam dele.

Características

Todos os objetos que herdam de SceneBaseObject possuem as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String	(Somente Leitura) Cadeia de caracteres que informam o tipo

		do objeto. Exemplos de valores: "scene", "viewport", "token", etc..
isObjectAlive	Boolean	(Somente Leitura) Contém **true** se o objeto ainda estiver vivo no código fonte do RRPG ou **false** se ele já foi destruído
objectID	Integer	(Somente Leitura) Contém um número único que identifica o objeto no RRPG local.
userData	Objeto Nodo	(Somente Leitura) Contém um objeto nodo de um NodeDatabase que armazena dados extras que o programador possa querer armazenar. Cada Objeto do Scene 3 possui seu próprio userData. Saiba mais lendo sobre NodeDatabase e sobre Objeto Nodo.

Métodos

Método	Descrição
obj:isType(typeName)	Retorna true se o objeto é do tipo "typeName", senão retorna false.
obj:beginUpdate()	Coloca o objeto em estado de alteração. Se você for fazer várias mudanças em um objeto de uma única vez, talvez seja possível obter uma melhoria de performance usando beginUpdate. para cada chamada de beginUpdate, deve haver 1 chamada de endUpdate
obj:endUpdate()	Retira o objeto do estado de alteração que foi previamente configurado pela função beginUpdate. para cada chamada de beginUpdate, deve haver 1 chamada de endUpdate

obj:newUserDataLink([field])	Cria e retorna um novo objeto DataLink que monitora um determinado field do objeto nodo "userData". Parâmetros: ● (OPCIONAL) field - Uma cadeia de string contendo o nome/caminho do campo que será monitorado. Retorna: ● Um objeto dataLink que monitora o field solicitado do "userData" deste objeto. Observações: ● Esta é a melhor maneira de capturar mudança de valores nos valores de userData.

Eventos

Nome do evento	Descrição

Objeto SceneCanvas

O Objeto SceneCanvas representa como um item do Scene deve ser desenhado na tela do usuário e, no fundo, é uma lista de operações gráficas que devem ser executadas para compor a aparência de um item no tabuleiro.

O SceneCanvas de um item gráfico pode ser obtido através da propriedade "canvas" de um que herde de objeto SceneGraphicItem

A grande vantagem em utilizar o SceneCanvas é que as operações gráficas são persistidas no scene e serão executadas no tabuleiro de cada usuário, mesmo que algum não tenha instalado algum determinado plug-in.

Herança

O o**bjeto SceneCanvas** herda de SceneBaseObject e possui também todas as suas características.

Características

Além das características herdadas, o objeto SceneCanvas também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
opCount	Integer	(Somente Leitura) Contém a quantidade de operações gráficas que este Canvas possui.

Métodos

Método	Descrição
canvas:getOp(index)	Retorna a operação gráfica de índice passado por parâmetro. Parâmetros: ○ index - Um número inteiro informando qual operação gráfica do canvas você deseja obter. Este valor deve estar entre 0 e "opCount" - 1 Retorno: ○ Um objeto SceneDrawingOp representando a operação gráfica do índice informado. Observação: ○ O item retornado pode ser uma operação gráfica de bitmap/imagem (objeto SceneDrawingOpBitmap) ou uma operação gráfica de caminho/formas/polígonos (objeto SceneDrawingOpPath). Para saber qual é o tipo da operação gráfica, verifique a propriedade objectType do objeto

	retornado.
canvas:addBitmap()	Adiciona uma operação gráfica de desenho de bitmap/imagem ao canvas. Parâmetros: Nenhum Retorno: ○ Um objeto SceneDrawingOpBitmap representando a operação gráfica criada.
canvas:addPath()	Adiciona uma operação gráfica de desenho de caminho/formas/polígonos ao canvas. Parâmetros: Nenhum Retorno: ○ Um objeto SceneDrawingOpPath representando a operação gráfica criada.
canvas:addText()	Adiciona uma operação gráfica de desenho de texto ao canvas. Parâmetros: Nenhum Retorno: ○ Um objeto SceneDrawingOpText representando a operação gráfica criada.

canvas:findByName(name)	Localiza uma operação gráfica pelo seu nome. Parâmetros: ○ name - Um texto, o nome da operação gráfica que está procurando neste Canvas/Item do Scene. Retorno: ○ Se o Scene encontrar uma operação gráfica cuja propriedade name for igual ao texto informado, retorna um objeto SceneDrawingOp representando a operação gráfica localizada. ○ Se o Scene não encontrar, retorna **nil**. Observação: ○ O item retornado pode ser uma operação gráfica de bitmap/imagem (objeto SceneDrawingOpBitmap) ou uma operação gráfica de caminho/formas/polígonos (objeto SceneDrawingOpPath). Para saber qual é o tipo da operação gráfica, verifique a propriedade objectType do objeto retornado.
canvas:clear()	Limpa o canvas excluindo todas as operações gráficas contida nele.

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 - Um plug-in que adiciona/remove uma marca personalizada quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(

    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)

            -- Usuário clicou no Scene

            -- Transformar as coordenadas de tela em métrica de mundo

            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);


            -- Localizar o item que o usuário clicou

            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then

                -- Encontrou um item na posição clicada.

                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome

                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addBitmap();
                    opGrafica.name = NOME_MINHA_MARCA;
                   -- Abaixo, URL da imagem da nossa marca.
                    opGrafica.url =
"https://wiki.teamfortress.com/w/images/d/dd/Bleed_drop.png?
t=20110425044341";
                    opGrafica.x = 0.25;
                    opGrafica.width = 0.3;
                    opGrafica.y = -0.1;
                    opGrafica.height = 0.3;
                    opGrafica.z = 10;
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;
```

```
        end);
```

 Item sem a marca	 Item com a marca

Exemplo 2 -

```

```

Objeto SceneDrawingOp

O Objeto SceneDrawingOp representa uma operação gráfica de um SceneCanvas .

Herança

O o**bjeto SceneDrawingOp** herda de SceneBaseObject e possui também todas as suas características.

Abstração

Esta é uma **classe abstrata** e você nunca encontrará um objeto SceneDrawingOp vivo sem ser uma das seguintes classes:

- Objeto SceneDrawingOpBitmap
- Objeto SceneDrawingOpPath

Características

Além das características herdadas, o objeto SceneDrawingOp também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição

objectType	String: ● "opBitmap" ● "opPath" ● "opText"	(Somente Leitura) Contém um texto que identifica o tipo da operação gráfica e pode conter um dos seguintes valores: ● "opBitmap" - este operação gráfica é um Objeto SceneDrawingOpBitmap, uma imagem/bitmap. ● "opPath" - esta operação gráfica é um Objeto SceneDrawingOpPath, um caminho/forma/polígono. ● "opText" - Esta operação gráfica é um Objeto SceneDrawingOpText, um texto.
x	Double	Contém a posição X de onde esta operação gráfica deve ser renderizada. Por padrão, este valor está em uma métrica relativa ao SceneCanvas de um item no scene e é normalizado entre 0.0 e 1.0, onde 0.0 = canto esquerdo do item, 0.5 = centro do item e 1.0 = canto direito do item. (Item, neste caso, é o Token/UserDrawing/EffectArea desta operação gráfica)
y	Double	Contém a posição Y de onde esta operação gráfica deve ser renderizada. Por padrão, este valor está em uma métrica relativa ao SceneCanvas de um item no

		scene e é normalizado entre 0.0 e 1.0, onde 0.0 = canto superior do item, 0.5 = centro do item e 1.0 = canto inferior do item (Item, neste caso, é o Token/UserDrawing/EffectArea desta operação gráfica)
width	Double	Contém a largura que a operação gráfica deve ser renderizada. Por padrão, este valor está em uma métrica relativa ao SceneCanvas de um item no scene e é normalizado entre 0.0 e 1.0, onde 0.5 = metade da largura do item, 1.0 = largura do item e 2.0 = duas vezes a largura do item. (Item, neste caso, é o Token/UserDrawing/EffectArea desta operação gráfica)
height	Double	Contém a altura que a operação gráfica deve ser renderizada. Por padrão, este valor está em uma métrica relativa ao SceneCanvas de um item no scene e é normalizado entre 0.0 e 1.0, onde 0.5 = metade da altura do item, 1.0 = altura do item e 2.0 = duas vezes a altura do item. (Item, neste caso, é o Token/UserDrawing/EffectArea desta operação gráfica)
z	Double	Contém um número para definir a ordem Z da operação gráfica, isto é, a ordem em que deve ser renderizada. Exemplo: Se A e B forem duas operações gráficas de um mesmo Item de Scene e se A possuir ordem Z equivalente a 5 e B possuir ordem Z equivalente a 10, a operação gráfica A aparecerá abaixo da operação gráfica B na tela.

name	String	Um texto auxiliar qualquer para ajudar a identificar qual é esta operação gráfica. Pode ser utilizado pela função "canvas:findByName()" do objeto SceneCanvas para fácil localização.
opacity	Double	Define a opacidade da operação gráfica. Este valor varia de 0.0 a 1.0, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco.
rotMode	Enumerado: ● "default" ● "ignoreCanvasRot"	Define como a operação gráfica deve se comportar em relação à rotação do item/canvas dono da operação gráfica. "default": Padrão, a operação gráfica deve ser rotacionada conforme a rotação de seu canvas/item do scene. "ignoreCanvasRot": A renderização desta operação gráfica deve ignorar a rotação de seu canvas/item do scene.
visibility	Enumerado: "everyone" "gmOnly"	Define quem pode visualizar esta operação gráfica: "everyone": Padrão, todos podem visualizar esta operação gráfica. "gmOnly": Apenas o mestre da mesa pode ver esta operação gráfica.
rotation	Double	Define a rotação da operação gráfica em graus.
rotCenterX	Double	Define onde, no eixo X da operação gráfica, está o pivô de rotação. O valor varia entre 0.0 a 1.0:

		● 0.0 = lado esquerdo do desenho. ● 0.5 = centro do desenho (Padrão) ● 1.0 = lado direito do desenho.
rotCenterY	Double	Define onde, no eixo Y da operação gráfica, está o pivô de rotação. O valor varia entre 0.0 a 1.0: ● 0.0 = parte superior do desenho. ● 0.5 = centro do desenho (Padrão). ● 1.0 = parte inferior do desenho.
drawingTrigger	Conjunto: ● "hover" ● "selected"	Define gatilhos para esta operação ser desenhada. É um conjunto, uma tabela contendo um ou mais dos seguintes itens: "hover" - Quando o mouse estiver por cima do objeto/token. "selected" - Quando o objeto/token estiver selecionado. Exemplos: ● {} - Não há gatilhos, sempre é desenhada ● {"hover"} - Quando o mouse estiver por cima do item. ● {"hover", "selected"} - Quando o mouse estiver por cima do item ou quando o item estiver selecionado.

outOfOrderMode	Enumerado: ● "noOutOfOrder" ● "afterFogOfWarLayer" ● "beforeOwnerLayer" ● "afterOwnerLayer"	Define se a operação gráfica possui um comportamento fora da ordem padrão. "noOutOfOrder" - (Padrão) A operação não possui comportamento de desenho fora da ordem e é desenhado na ordem normal. "afterFogOfWarLayer" - A operação possui comportamento de desenho fora de ordem. É desenhada após a camada de Fog Of War ser desenhada no tabuleiro. "beforeOwnerLayer" - A operação possui comportamento de desenho fora de ordem. Ela é desenhada abaixo da camada do item que a possui. "afterOwnerLayer" - "beforeOwnerLayer" - A operação possui comportamento de desenho fora de ordem. Ela é desenhada acima da camada do item que a possui.
xMetric **yMetric** **widthMetric** **heightMetric**	Enumerado: ● "screenMetric" ● "worldMetric" ● "canvasMetric" ● "cellMetric"	Altera a medida em que as propriedades "x", "y", "width" e "height" foram informadas: "canvasMetric" - (Padrão) A propriedade está representado em métricas de canvas, onde, por exemplo, 0.5 = metade da largura/altura do token/desenho e 1.0 = largura/altura do token/desenho. "screenMetric" - A propriedade está representado em métricas de tela. "worldMetric" - A propriedade está representado em métricas de mundo.

		"cellMetric" - A propriedade está representado em métricas de células. (Exemplo, 0.5 = meia célula do grid, 2 = duas células do grid)

xOrigin
yOrigin
	Double	Contém a posição de origem no eixo X e no eixo Y de onde as propriedades "x" e "y" são relativas.
		Este valor está em uma métrica relativa ao SceneCanvas de um item no scene e é normalizado entre 0.0 e 1.0, onde 0.0 = canto esquerdo do item, 0.5 = centro do item e 1.0 = canto direito do item. (Item, neste caso, é o Token/UserDrawing/EffectArea desta operação gráfica)
		Observação: Estes valores são úteis apenas quando é usado xMetric/yMetric diferente de "canvasMetric".
		Exemplo:
		x = -1;
		xMetric = "cellMetric";
		xOrigin = 0.5;
		width=2
		widthMetric="cellMetric"
		Faz com que a operação gráfica tenha largura equivalente a 2 células do grid e sua posição X seja equivalente ao "centro X do token menos 1 célula do grid" (Horizontalmente alinhado)

Métodos

Método	Descrição
drawingOp:delete()	Remove/Apaga a operação gráfica do Canvas do Item no Scene.

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 - Um plug-in que adiciona/remove uma marca personalizada quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(

    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)

            -- Usuário clicou no Scene

            -- Transformar as coordenadas de tela em métrica de mundo

            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);


            -- Localizar o item que o usuário clicou

            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then

                -- Encontrou um item na posição clicada.

                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome

                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addBitmap();
                    opGrafica.name = NOME_MINHA_MARCA;
                   -- Abaixo, URL da imagem da nossa marca.
                    opGrafica.url =
"https://wiki.teamfortress.com/w/images/d/dd/Bleed_drop.png?
t=20110425044341";
                    opGrafica.x = 0.25;
                    opGrafica.width = 0.3;
                    opGrafica.y = -0.1;
                    opGrafica.height = 0.3;
                    opGrafica.z = 10;
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;
```

```
        end);
```

 Item sem a marca	 Item com a marca

Exemplo 2 -

```

```

Objeto SceneDrawingOpBitmap

O Objeto SceneDrawingOpBitmap representa uma operação gráfica de um Bitmap/Imagem em um SceneCanvas .

Herança

O o**bjeto SceneDrawingOpBitmap** herda de SceneDrawingOp e possui também **TODAS** todas as suas características.

Veja também:

- Objeto SceneDrawingOp
- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneDrawingOpBitmap também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição

objectType	String "opBitmap"	(Somente Leitura) Contém um texto que identifica o tipo da operação gráfica - "opBitmap".
url	String	Contém a URL da imagem/bitmap que será renderizada. Este valor: ● Pode ser um endereço de internet (exemplo: "http://xxxx.com.br/a.png") ● Pode ser o caminho de um arquivo contido no pacote do plugin. ● Pode ser um arquivo que se encontra no HD Virtual do plugin instalado.
flipHorz	Boolean	Determina se a imagem deve ser desenhada de forma espelhada horizontalmente.
flipVert	Boolean	Determina se a imagem deve ser desenhada de forma espelhada verticalmente.
grayscale	Boolean	Determina se imagem deve ser desenhada usando um filtro de cores que transforma a imagem em uma imagem de escala cinzenta.
colorBlend	String de Cor	Determina se a imagem deve ser desenhada utilizando um filtro que misturando as cores originais com uma cor definida pela propriedade "colorBlend". O valor padrão é "#00000000" e significa não realizar misturar de cores. Exemplo: "Blue" fará com que a imagem seja desenhada misturada

		com a cor azul.
colorMask	String de Cor	Determina se a imagem deve ser desenhada utilizando um filtro que seleciona as cores da imagem realizando uma multiplicação das cores da imagem pela cor definida em "colorMask". O valor padrão é "#FFFFFFFF" e significa que todas as coras originais da imagem deve ser desenhadas sem alteração. Exemplo: "Blue" fará com que a imagem seja desenhada selecionando apenas os tons azuis pois os componentes da cor azul são R = 0, G = 0 e B = 255, o que transformando em notação de ponto flutuante fica R = 0.0, G = 0.0, B = 1.0... Para cada pixel da imagem, o Red será multiplicado por 0, o Green será multiplicado por 0 e o Blue será multiplicado por 1.0.
filterEx	Enumerado: ● "none" (Padrão) ● "glow" ● "innerGlow" ● "blur" ● "sepia"	Determina se a imagem deve ser desenhada com mais algum filtro de imagem especial. "none" - Nenhum filtro especial. "glow" - Um filtro que adiciona uma incandescência à imagem. Não esqueça de preencher a propriedade "filterExColor" ao utilizar este filtro. "innerGlow" - Um filtro que adiciona um brilho interno à imagem. Não esqueça de preencher a propriedade "filterExColor" ao utilizar este filtro. "blur" - Um filtro que aplica um "borrão" na imagem. "sepia" - Um filtro que aplica um tom marrom na imagem.

filterExColor	String de Cor	Alguns filtros permitem escolher uma cor como parâmetro. Se a propriedade filterEx for: "glow" - Esta propriedade contém a cor da incandescência que será criada na imagem. "innerGlow" - Esta propriedade contém a cor do brilho interno que será criado na imagem. A cor padrão é "Red"
frame	Enumerado: ● "none" (Padrão) ● "boton"	Determina se a imagem deve ser desenhada com alguma borda. "none" - A imagem será desenhada sem bordas. "boton" - A imagem será desenhada com borda de um bóton arredondado.
sourceRect	Estrutura SceneRect	Determina, utilizando uma Estrutura SceneRect, qual porção da imagem original deve ser desenhada. Os valores de left, top, right e bottom vão de 0.0 a 1.0, onde 0.0 = representa o inicio e 1.0 o final do eixo. O valor padrão é: {left=0.0, top=0.0, right=1.0, bottom=1.0} ... isto é, desenhar todo contéudo da imagem original. Exemplo: ● {left=0.5, top=0, right=1.0, bottom=0.5} - Desenhar apenas o quadrante superior-direita da imagem.

Métodos

Método	Descrição

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 - Um plug-in que adiciona/remove uma marca personalizada quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(

    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)

            -- Usuário clicou no Scene

            -- Transformar as coordenadas de tela em métrica de mundo

            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);


            -- Localizar o item que o usuário clicou

            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then

                -- Encontrou um item na posição clicada.

                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome

                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addBitmap();
                    opGrafica.name = NOME_MINHA_MARCA;
                   -- Abaixo, URL da imagem da nossa marca.
                    opGrafica.url =
"https://wiki.teamfortress.com/w/images/d/dd/Bleed_drop.png?
t=20110425044341";
                    opGrafica.x = 0.25;
                    opGrafica.width = 0.3;
                    opGrafica.y = -0.1;
                    opGrafica.height = 0.3;
                    opGrafica.z = 10;
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;
```

```
        end);
```

 Item sem a marca	 Item com a marca

Exemplo 2 -

```

```

Objeto SceneDawingOpPath

O Objeto SceneDrawingOpPath representa uma operação gráfica de um caminho/forma/polígono em um SceneCanvas .

Herança

O o**bjeto SceneDrawingOpPath** herda de SceneDrawingOp e possui também **TODAS** todas as suas características.

Veja também:

- Objeto SceneDrawingOp
- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneDrawingOpPath também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String "opPath"	(Somente Leitura) Contém um texto que

		identifica o tipo da operação gráfica - "opBitmap".
data, pathData ou path	String	Define a forma vetorial do path. Este texto deve estar no mesmo padrão do path do SVG e do XAML. Consulte a documentação deles na internet! Outro link que pode ajudar: http://www.w3schools.com/svg/svg_path.asp Observação Importante: Path será automaticamente redimensionado para ficar com a mesma dimensão definida para esta operação gráfica.
color	String de Cor	Define a cor do conteúdo da forma. Veja String de cores para obter a completa de cores e outras formas de uso. Define a cor do conteúdo da forma. Valor padrão: White
strokeColor	String de Cor	Define a cor do contorno da forma. Veja String de cores para obter a completa de cores e outras formas de uso. Define a cor do conteúdo da forma. Valor padrão: Black
strokeSize	Double	Define a largura do contorno da forma. O significado deste número depende do valor da propriedade "strokeSizeMetric" Valor padrão: 0.0;

strokeSizeMetric	Enumerado: ● "screenMetric" ● "worldMetric" ● "canvasMetric" ● "cellMetric"	Define em que medida a propriedade "strokeSize" está medida. "screenMetric" - (Padrão) strokeSize está representado em métricas de tela. "worldMetric" - strokeSize está representado em métricas de mundo. "canvasMetric" - strokeSize está representado em métricas de canvas, onde, por exemplo, 0.5 = metade da largura/altura do token/desenho e 1.0 = largura/altura do token/desenho. "cellMetric" - strokeSize está representado em métricas de células. (Exemplo, 0.5 = meia célula do grid, 2 = duas células do grid)
strokeCap	Enumerado: ● "flat" ● "round"	Define o estilo gráfico usado ao desenhar o fim das linhas. "flat" - linhas com pontas retangulares "round" - linhas com pontas arredondadas. Valor padrão: "flat"
strokeJoin	Enumerado: ● "miter" ● "round" ● "bevel"	Define o estilo gráfico usado ao juntar segmentos de linha em uma forma. "miter" - As quinas/junções são "quadradas". "round" - As quinas/junções são arredondadas. "bevel" - As quintas/junções são ligadas de forma diagonal. Valor padrão: "miter"

strokeDash	Enumerado: ● "solid" ● "dash" ● "dot" ● "dashDot" ● "dashDotDot"	Define o estilo gráfico da linha. "solid" - A linha é sólida "dash" - A linha é tracejada "dot" - A linha é feita de pontos. "dashDot" - A linha é alternada entre traços e pontos. "dashDotDot" - A linha é alternada entre traço, ponto e ponto. Valor padrão: "solid"

Métodos

Método	Descrição

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 - Um plug-in que adiciona/remove um triângulo amarelo quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(
    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)
            -- Usuário clicou no Scene
            -- Transformar as coordenadas de tela em métrica de mundo
            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);

            -- Localizar o item que o usuário clicou
            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then
                -- Encontrou um item na posição clicada.
                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome
                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addPath();
                    opGrafica.name = NOME_MINHA_MARCA;

                    -- Começar a desenhar no X equivalente a 80% da
largura do item
                    -- Começar a desenhar no Y equivalente a 25% da
altura do item
                    opGrafica.x = 0.8;
                    opGrafica.y = 0.25;

                    opGrafica.width = 0.5;  -- Largura do Path: 50%
da largura do item
                    opGrafica.height = 0.4;  -- Altura do Path: 40%
da altura do item

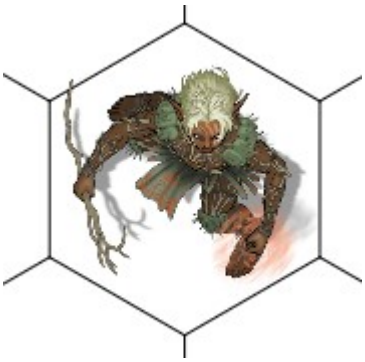
                    opGrafica.z = 10;

                    -- Cores e estilo do PATH
                    opGrafica.color = "yellow";
```

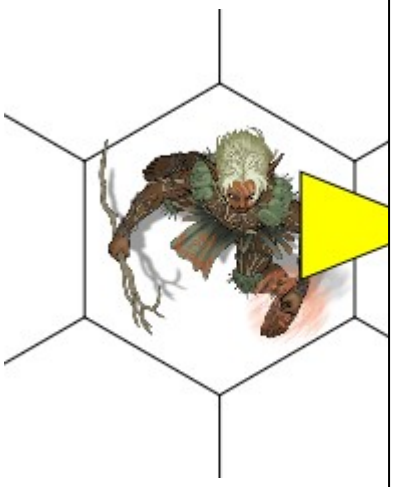
```
                    opGrafica.strokeSize = 1;


                    -- O path data (Um triângulo apontado para a
direita)
                    opGrafica.data = "M 0 0 L 1 0.5 L 0 1 Z";
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;


    end);
```



Item sem a marca



Item com a marca

Exemplo 2 -

Objeto SceneDrawingOpText

O Objeto SceneDrawingOpText representa uma operação gráfica de um texto em um SceneCanvas .

Herança

O o**bjeto SceneDrawingOpText** herda de SceneDrawingOp e possui também **TODAS** todas as suas características.

Veja também:

- Objeto SceneDrawingOp
- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneDrawingOpText também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String "opText"	(Somente Leitura) Contém um texto que identifica o tipo da operação gráfica - "opBitmap".
text	String	O texto a ser desenhado no canvas.
color	String de Cor	Define a cor do texto. Veja String de cores para obter a completa de cores e outras formas de uso. Valor padrão: Black.
fontStyle	Uma tabela contendo de um ou mais dos seguintes valores:	Define o estilo que será aplicado ao texto e pode ser a combinação de 0 ou mais dos seguintes valores:

	• "bold" • "italic" • "underline" • "strikeout"	"bold" - Texto negrito "italic" - Texto em itálico "underline" - Texto com underline. "strikeout" - Texto tachado. Exemplos: • {} - (Valor padrão) • {"bold", "italic"} - (negrito e itálico) • {"bold"} - (apenas negrito)
horzTextAlign	Enumerado: • "leading" • "center" • "trailing"	Define como será o alinhamento horizontal do texto. "leading" - Alinhado à esquerda "center" - Alinhado ao centro "trailing" - Alinhado à direita Valor padrão: "leading"
vertTextAlign	Enumerado: • "leading" • "center" • "trailing"	Define como será o alinhamento vertical do texto. "leading" - Alinhado ao topo. "center" - Alinhado ao centro "trailing" - Alinhado ao fundo. Valor padrão: "center".
aspect	Double	(Somente Leitura) Retorna o aspecto do conteúdo do texto, isto é, a divisão entre a largura do texto por sua altura.

Métodos

Método	Descrição

Eventos

Nome do evento	Descrição

Exemplos

Objeto SceneEffectArea

(Conteúdo desta página não consta no documento original.)

Objeto SceneFogOfWar

O Objeto SceneFogOfWar é responsável por lidar a névoa de guerra do scene.

Herança

O o**bjeto SceneFogOfWar** herda de SceneBaseObject e possui também **TODAS** todas as suas características.

Veja também:

- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneDrawingOpText também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String "fogOfWar"	(Somente Leitura) Contém um

		texto que identifica o tipo do objeto - "fogOfWar"
enabled	Boolean	Define se o Fog of War está ativo no tabuleiro. Apenas o mestre pode alterar esta propriedade.
ambientLight	Enumerado: ● "intense" ● "weak" ● "none"	Define qual é a iluminação ambiente do scene e pode ser: ● "intense" (Padrão) - Todo o tabuleiro recebe uma iluminação de ambiente Intensa. Exemplo: Ambiente à luz do sol. ● "weak" - Todo o tabuleiro recebe uma iluminação de ambiente fraca. Exemplo: Luz do luar. ● "none" - O tabuleiro não recebe nenhum tipo de iluminação do ambiente.
sharingMode	Enumerado: ● "individual" ● "group"	Define como ocorre o compartilhamento de visão entre os membros do grupo. ● "individual" (Padrão) - Cada membro do grupo enxerga apenas o que seus personagens enxergam. ● "group" - Cada membro do grupo enxerga o que todo o grupo enxerga.

Métodos

Método	Descrição
fogOfWar:setArea(polygon, areaType)	Configura uma área do tabuleiro para se comportar como um determinado tipo de área de Fog of War. Parâmetros: ● polygon - Um array de tabelas que contém propriedades x e y (em métrica de mundo), e cada cada item do array representando um vértice do polígono. Este é o polígono que será configurado. ● areaType - O tipo que esta área se comportará como. Pode ser um dos seguintes valores: ○ " ○ " ○ " ○ "

	Observações: ● Apenas o mestre pode configurar áreas do tabuleiro
fogOfWar:resetAreas();	Reseta todas as áreas definidas no tabuleiro. Isto é, todo o tabuleiro passará a ser uma área do tipo "explorable". Observações: ● Apenas o mestre pode configurar áreas do tabuleiro
fogOfWar:addOpaqueArea(polygon)	Adiciona uma área opaca/parede ao tabuleiro. Parâmetros: ● polygon - Um array de tabelas que contém propriedades x e y (em métrica de mundo), e cada cada item do array representando um vértice do polígono. Este é o polígono que será configurado.

	Observações: ● Apenas o mestre pode configurar áreas opacas do tabuleiro.
fogOfWar:removeOpaqueArea(polygon)	Remove uma área opaca/parede ao tabuleiro. Parâmetros: ● polygon - Um array de tabelas que contém propriedades x e y (em métrica de mundo), e cada cada item do array representando um vértice do polígono. Este é o polígono que será removido das áreas opacas. Observações: ● Apenas o mestre pode configurar áreas opacas do tabuleiro.
fogOfWar:resetOpaqueAreas();	Remove todas as áreas opacas do tabuleiro. Observações:

	● Apenas o mestre pode configurar áreas opacas do tabuleiro.
fogOfWar:testCircleMovementCollisionVsOpaqueAreas(centerX, centerY, r, offsetX, offsetY)	Verifica se um determinado círculo movendo colide com alguma área opaca/parede do tabuleiro. Parâmetros: ● centerX - Centro do círculo no eixo X (em métrica de mundo) antes de realizar o movimento ● centerX - Centro do círculo no eixo Y (em métrica de mundo) antes de realizar o movimento . ● r - Raio do círculo (em métrica de mundo) ● offsetX - Define o sentido e comprimento do movimento do círculo no eixo X (em métrica de mundo) ...

	Exemplo: 1.0 significa que o círculo andou 1.0 métrica mundo para a direita. ● offsetX - dirX - Define o sentido e comprimento do movimento do círculo no eixo Y (em métrica de mundo) ... Exemplo: 1.0 significa que o círculo andou 1.0 métrica mundo para baixo. Retorno: Esta função retorna 3 valores na seguinte ordem: ● true se o ocorre uma colisão durante o movimento ou false ● Centro do círculo no eixo X (em métrica de mundo) onde ocorreu a colisão. ● Centro do círculo no

	eixo Y (em métrica de mundo) onde ocorreu a colisão.

Eventos

Nome do evento	Descrição

Exemplos

Objeto SceneGraphicItem

O Objeto GraphicItem representa um único item gráfico no Scene, como, por exemplo, um token.

Herança

O o**bjeto SceneGraphicItem** herda de SceneBaseObject e possui também **todas** as suas características.

Abstração

Esta é uma **classe abstrata** e você nunca encontrará um objeto SceneGraphicItem vivo sem ser uma das seguintes classes:

- Objeto SceneToken
- Objeto SceneEffectArea
- Objeto SceneUserDrawing

Por ser uma abstração, um objeto SceneGraphicItem possui todas as características que são comuns aos objetos listados acima.

Características

Além das características herdadas, o objeto SceneGraphicItem também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
scene	Objeto Scene	(Somente Leitura) Contém o Objeto Scene a qual este item gráfico pertence.
objectType	String: ● "token" ● "effectArea" ● "userDrawing"	(Somente Leitura) Contém um texto que identifica o tipo do objeto gráfico e pode conter um dos seguintes valores: ● "token" - este item gráfico é um Objeto SceneToken ● "effectArea" - este item gráfico é um Objeto SceneEffectArea ● "userDrawing" - este item gráfico é um Objeto SceneUserDrawing
width	Double	Contem a LARGURA do item em métrica de mundo.
height	Double	Contem a ALTURA do item em métrica de mundo.
layer	Enumerado: ● "background" ● "objects" ● "tokens"	Define em qual camada do tabuleiro este item se encontra.
x	Double	Contém, em métrica de mundo, a posição do item no eixo X.

y	Double	Contém, em métrica de mundo, a posição do item no eixo Y.
z	Double	Contém um número para definir a ordem Z do item, isto é, a ordem em que o item é desenhado na tela. Exemplo: Se A e B forem dois items de uma mesma camada do scene e se A possuir ordem Z equivalente a 5 e B possuir ordem Z equivalente a 10, o item A aparecerá abaixo do item B na tela.
rotation	Double	Contém um ângulo em graus que define a rotação do item na interface.
visible	Boolean	Se for **true**, o item deve ser desenhado para todos. Se for **false**, o item deve ser desenhado apenas para o mestre.
snapToGrid	Boolean	Define como o item deve se posicionar em relação ao grid. Se for true, o item deve se ajeitar para se encaixar na grid do tabuleiro. Se for false, o item não deve tentar ajeitar sua posição de forma automática.
locked	Boolean	Se true, os usuários não conseguirão mover o item pelo scene. Ótimo para, após o tabuleiro tiver sido montado pelo mestre, evitar que os usuários movam peças sem querer e desmontarem o scene.
selected	Boolean	Indica se o item está contido na lista de itens selecionados do usuário atual.
isHover	Boolean	Indica se o mouse do usuário está em cima do item.

isMine	Boolean	(Somente Leitura) Contém true se o usuário atual do RRPG puder controlar/for dono do item.
ownerUserID	String	Contém o login do usuário do RRPG que é dono deste item.
canBeRendered	Boolean	(Somente Leitura) Contém true se o item pode ser renderizado na tela do usuário. O valor desta propriedade considera o Fog of War do tabuleiro.
canvas	Objeto SceneCanvas	(Somente Leitura) Contém o objeto SceneCanvas do item. Este objeto contém as operações gráficas que compõe a aparência do item; Se for um SceneToken, por exemplo, você encontrará um item dentro de canvas que é responsável por desenhar a imagem no tabuleiro. Se for um SceneUserDrawing, dentro de canvas você encontrará as informações do desenho que o usuário fez.

Métodos

Método	Descrição
item:getCenter([snapToGrid])	Retorna onde no mundo do tabuleiro fica o centro do item. Parâmetros: ● (OPCIONAL) snapToGrid - Booleano. Se True, o scene retornará a posição mais perto do centro do item que se encaixe na grid. False, o scene não tentará fazer nenhum tipo de

	arredondamento. Se este parâmetro não for informado, o scene usará o valor da propriedade "snapToGrid" do item como parâmetro. Retorno: A função retorna dois números na seguinte ordem: ○ worldX - A posição central no eixo X em relação ao tabuleiro, em métrica de mundo. ○ worldY - A posição central no eixo Y em relação ao tabuleiro, em métrica de mundo. Exemplo: local wX, wY = scene.items[1]:getCenter();
item:setCenter(worldX, worldY[, snapToGrid])	Move o item de forma que seu centro ocupe a posição passada como parâmetro. Parâmetros: ○ worldX - Posição no eixo X do tabuleiro, em métrica de mundo, onde o centro do item deve ficar. ○ worldY - Posição no eixo Y do tabuleiro, em métrica de mundo, onde o centro do item deve ficar. ○ (OPCIONAL) snapToGrid - Booleano. Caso seja True, o Scene tentará arredondar a posição de forma que o item se encaixe no grid do

	tabuleiro. Se não informado, o scene utiliza o valor da propriedade "snapToGrid" do próprio item.
item:getBounds()	Retorna as informações dos limites do item no scene. Esta função retorna 5 valores na seguinte ordem: ● left - Valor numérico em métrica de mundo contendo o valor da propriedade "x" do item. ● top - Valor numérico em métrica de mundo contendo o valor da propriedade "y" do item. ● right - Valor numérico em métrica de mundo contendo o valor da propriedade "x + width" do item. ● bottom - Valor numérico em métrica de mundo contendo o valor da propriedade "y + height" do item. ● rotation - Valor numérico contendo o valor da propriedade "rotation" do item.
item:setBounds(left, top, right, bottom[, rotation])	Altera os limites do item no scene. Parâmetros ● left - Valor numérico em métrica de mundo contendo o valor da propriedade "x" do item. ● top - Valor numérico em métrica de mundo contendo o valor da propriedade "y" do item. ● right - Valor numérico em métrica de mundo contendo o valor da propriedade "x + width" do item. ● bottom - Valor numérico em métrica de mundo contendo o valor da propriedade "y + height" do item. ● (OPCIONAL) rotation - Valor numérico contendo o valor da propriedade "rotation" do item.

item:getCellsOccupied()	Retorna as células do grid que o item está ocupando no momento. Parâmetros: Não há Retorno: ○ Um array (tabela lua indexada de 1 a #tabela) de tabelas luas, cada uma contendo as propriedades "i"e "j" das coordenadas Aij de cada célula ocupada pelo item.
item:compareZOrder(otherItem)	Realiza uma comparação de ordem Z entre dois graphics itens e retorna a ordem em relação aos dois. Parâmetros: ● otherItem - Outro objeto SceneGraphicItem com o qual deseja comparar Retorno: ● -1 se item fica abaixo de otherItem ● 0 se item fica exatamente na mesma ordem z ● 1 se item fica acima de otherItem
item:doSnapToGrid()	Ajusta a posição atual do item para que fique agarrado ao grid.
item:delete()	Remove o item do tabuleiro.

Eventos

Nome do evento	Descrição
onBeforeDraw	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento!

	Este evento é chamado antes do RRPG desenhar o item gráfico na tela.
onAfterDraw	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento! Este evento é chamado depois do RRPG desenhar o item gráfico na tela.
onBoundsChange	Evento que é chamado quando os limites do item for alterado (posição, largura, altura, rotação, etc..) Parâmetros: ● item - objeto SceneGraphicItem que sofreu a alteração nos limites.

Exemplos

Exemplo 1 - Um plugin que movimenta de forma simples os itens do Scene com o mouse

```lua
require("scene.lua");


SceneLib.registerPlugin(
    function (scene, attachment)


        -- Variáveis para armazenar informações sobre o "arrastar" dos itens
        local objMouseDown = nil;  -- Qual objeto está sendo arrastado
        local selecaoX, selecaoY;  --  Em que ponto do item o usuário clicou (em relação ao centro do item)




        -- Manipular o evento onMouseUp do Viewport afim de detectar o click do mouse
        scene.viewport.onMouseDown =
            function(event)
                -- Converter para métrica de mundo a posição do clique
                local wx, wy = scene.viewport:screenToWorld(event.x, event.y);


                -- Descobrir qual item se encontra neste ponto
                objMouseDown = scene.items:itemAtPoint(wx, wy);
```

```
                if objMouseDown ~= nil then
                    -- Existe um item nesta posição.

                    -- Vamos obter onde fica a posição central deste
item
                    local centerX, centerY =
objMouseDown:getCenter();

                    if objMouseDown.snapToGrid then
                        -- Movimento "agarrado" ao grid
                        -- Vamos pegar a posição do mundo e
arredondar para o centro
                        -- de uma célula mais próxima
                        local cellI, cellJ =
scene.grid:worldToCell(wx, wy);
                        wx, wy = scene.grid:cellToWorld(cellI,
cellJ);
                    end;

                    -- calcular onde no item o usuário clicou, em
relação ao seu centro.
                    selecaoX = wx - centerX;
                    selecaoY = wy - centerY;
                end;
            end;
```

```
        -- Manipular o evento onMouseMove para arrastar o item
        scene.viewport.onMouseMove =
            function(event)
                if objMouseDown ~= nil then
                    -- Mouse moveu e existe um item selecionado
                    -- Converter a posição clicada para métrica de
mundo
                    local wx, wy =
scene.viewport:screenToWorld(event.x, event.y);

                    if objMouseDown.snapToGrid then
                        -- Movimento agarrado ao grid
                        -- Arredondar a posição do mouse para o
centro de uma célula mais
                        -- próxima.
                        local newCellI, newCellJ =
scene.grid:worldToCell(wx, wy);
                        wx, wy = scene.grid:cellToWorld(newCellI,
newCellJ);
                    end;

                    -- Calcular o novo centro do item
                    local centerX, centerY =
```

```
    objMouseDown:getCenter();

                        centerX = wx - selecaoX;

                        centerY = wy - selecaoY;


                        -- Setar o novo centro do item.

                        objMouseDown:setCenter(centerX, centerY);

                    end;

                end;


            -- Manipular o evento onMouseUp para parar de arrastar o item

            scene.viewport.onMouseUp =

                function(event)

                    -- Mouse liberou o click, vamos

                    -- sinalizar para o onMouseMove que não estamos mais
    arrastando

                    -- ninguem


                    objMouseDown = nil;

                end;


        end);
```

Exemplo 2 - Um plug-in que adiciona/remove uma marca personalizada quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(

    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)

            -- Usuário clicou no Scene

            -- Transformar as coordenadas de tela em métrica de mundo

            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);


            -- Localizar o item que o usuário clicou

            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then

                -- Encontrou um item na posição clicada.

                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome

                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addBitmap();
                    opGrafica.name = NOME_MINHA_MARCA;
                   -- Abaixo, URL da imagem da nossa marca.
                    opGrafica.url =
"https://wiki.teamfortress.com/w/images/d/dd/Bleed_drop.png?
t=20110425044341";
                    opGrafica.x = 0.25;
                    opGrafica.width = 0.3;
                    opGrafica.y = -0.1;
                    opGrafica.height = 0.3;
                    opGrafica.z = 10;
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;
```

```
    end);
```

 Item sem a marca	 Item com a marca

Objeto SceneGraphicItemList

O Objeto SceneGraphicItem é um objeto que contém a coleção de todos os itens gráficos/visuais do tabuleiro, como os tokens.

O SceneGraphicItemList de um scene pode ser obtido através da propriedade "items" do objeto Scene

Herança

O o**bjeto SceneGraphicItemList** herda de SceneBaseObject e possui também todas as suas características.

Características

Além das características herdadas, o objeto SceneGraphicItemList também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
operador #	Integer	(Somente Leitura) Contem a quantidade de itens gráficos que existem no scene. Exemplo: local qtDeItens = #scene.items;
operador []	Objeto	(Somente Leitura) Contem um

	SceneGraphicItem	objeto SceneGraphicItem representando um item gráfico do scene, como um token.
		
		O valor passado dentro do colchete deve ser um número entre 1 a "count".
		
		Exemplo de uso:
		local item1 = scene.items[1]; -- Primeiro item
		local item2 = scene.items[2];
		
		-- Percorrendo todos os itens gráficos do scene
		
		for i = 1, #scene.items, 1 do
		
		local umItem = scene.items[i];
		
		end;

count	Integer	(Somente Leitura), equivale ao operador #. Contem a quantidade de itens gráficos que existem no scene.
selection	Array de SceneGraphicItem	(Somente Leitura) Retorna um array (uma tabela lua indexada de 1 a #array) contendo os itens do scene que estão selecionados no momento.

Métodos

Método	Descrição
list:get(index)	Equivalente ao operador [], retorna um SceneGraphicItem representando um item gráfico do scene, como um token. Parâmetros: ○ index - um número inteiro entre 1 e "count" Retorno: Um objeto SceneGraphicItem.
list:enumAtPoint(worldX,	Dada uma posição em métrica de mundo,

worldY [, layer])	retorna todos os SceneGraphicItem do scene que podem ser encontrados neste ponto (isto é, se o ponto for uma posição dentro do item). Parâmetros: ○ worldX - Número em métrica de mundo que contem a coordenada X da ponto. ○ worldY - Número em métrica de mundo que contem a coordenada Y do ponto. ○ (OPCIONAL) layer - String contendo uma camada. Se este parâmetro for informado, apenas itens que estiverem na camada informada serão enumeradas por esta função. Se este parâmetro não informado, todos os itens no ponto serão retornados, independente de qual camada pertencer. Retorno: Um array (tabela lua com índices de 1 a #tabela) de objetos SceneGraphicItem. Caso nenhum item seja encontrado na posição informada, o array vem vazio. Observação: O array retornado pela função vem ordenado pela posição Z dos itens quando há sobreposição. Isto é, quando um item ocupa o mesmo espaço que outro item ocupa, a função garante que o item que é desenhado mais abaixo apareça antes no array retornado do que o item que é desenhado acima dele.Se um item achado não sobrepor nenhum outro item, sua ordem no array

	retornado é indefinida (Ordenação Z apenas dos itens em sobreposição).
list:enumInRect(rect [, layer])	Dado um retângulo em métrica de mundo, retorna todos os SceneGraphicItem do scene que sobrepõe ao retângulo (que coincidem, justapõe, que intercedem, etc..) Parâmetros: ○ rect - Um retângulo em estrutura SceneRect com valores em métrica de mundo. ○ (OPCIONAL) layer - String contendo uma camada. Se este parâmetro for informado, apenas itens que estiverem na camada informada serão enumeradas por esta função. Se este parâmetro não informado, todos os itens no ponto serão retornados, independente de qual camada pertencerem. Retorno: Um array (tabela lua com índices de 1 a #tabela) de objetos SceneGraphicItem. Caso nenhum item seja encontrado no retângulo informado, o array vem vazio. Observação: O array retornado pela função vem ordenado pela posição Z dos itens quando há sobreposição. Isto é, quando um item ocupa o mesmo espaço que outro item ocupa, a função garante que o item que é desenhado mais abaixo apareça antes no array retornado do que o item que é desenhado acima dele.Se um item achado não sobrepor nenhum outro item, sua ordem no array retornado é indefinida (Ordenação Z apenas

	dos itens em sobreposição).
list:itemAtPoint(worldX, worldY [, layer])	Dada uma posição em métrica de mundo, retorna um SceneGraphicItem do scene que se encontra neste ponto. Parâmetros: ○ worldX - Número em métrica de mundo que contem a coordenada X da ponto. ○ worldY - Número em métrica de mundo que contem a coordenada Y do ponto. ○ (OPCIONAL) layer - String contendo uma camada. Se este parâmetro for informado, apenas itens que estiverem na camada informada serão retornados por esta função. Se este parâmetro não informado, todos os itens no ponto poderão ser retornados, independente de qual camada pertencerem. Retorno: ○ Se houver algum item na posição informada, retorna um SceneGraphicItem, aquele mais acima na ordem Z, isto é, o que é desenhado acima dos outros. ○ Se não houver item na posição informada, retorna

	nil.
list:clearSelection()	Deseleciona todos os itens que estavam na lista de seleção do usuário.
list:addToken([layer])	Adiciona um novo Token ao tabuleiro. Parâmetros: ● (OPCIONAL) layer - Define em qual camada o novo token deve aparecer. Pode conter "background", "objects" ou "tokens". Retorno: ● Objeto SceneToken representando o novo token.
list:addUserDrawing([layer])	Adiciona um novo UserDrawing ao tabuleiro. Parâmetros: ● (OPCIONAL) layer - Define em qual camada o novo user drawing deve aparecer. Pode conter "background", "objects" ou "tokens". Retorno: ● Objeto SceneUserDrawing representando o novo desenho de usuário.

Eventos

Nome do evento	Descrição
onItemAdded	Evento invocado quando um item for adicionado ao scene, seja porque acabou de ser criado ou porque estava invisível e se tornou visível agora. Parâmetros: ● Item - Um objeto SceneGraphicItem representando o item que foi adicionado.
onItemRemoved	Evento invocado quando um item for removido do scene, seja porque foi destruído ou porque se tornou invisível.

	Parâmetros: ● Item - Um objeto SceneGraphicItem representando o item que foi removido.
onItemSelected	Evento invocado quando um item for adicionado à seleção de itens do usuário. Parâmetros: ● Item - Um objeto SceneGraphicItem representando o item que foi selecionado.
onItemDeselected	Evento invocado quando um item for removido da seleção de itens do usuário. Parâmetros: ● Item - Um objeto SceneGraphicItem representando o item que foi removido da seleção.
onItemLayerChange	Evento invocado quando a camada de um item for alterada. Parâmetros: ● Item - Um objeto SceneGraphicItem representando o item que teve a propriedade "layer" alterada.

Exemplos

Exemplo 1 - Um plugin que descobre em qual item do scene o usuário clicou

```lua
require("scene.lua");


SceneLib.registerPlugin(
    function (scene, attachment)


        -- Manipular o evento onMouseUp do Viewport afim de detectar
o click do mouse
        scene.viewport.onMouseUp =
            function(event)
                -- Converter para métrica de mundo a posição do
clique
                local wx, wy = scene.viewport:screenToWorld(event.x,
event.y);


                -- Localizar o objeto mais acima da posição clicada
                local o = scene.items:itemAtPoint(wx, wy);


                -- Exibir uma mensagem
                if o ~= nil then
                    showMessage("Clique em um objeto");
                else
                    showMessage("Clique em nada");
                end;
            end;
```

```
end);
```

Objeto SceneGridLayer

O Objeto SceneGridLayer representa o grid do tabuleiro de combate e possui informações e utilitários a respeito do mesmo.

O SceneGridLayer de um scene pode ser obtido através da propriedade "grid" do objeto Scene

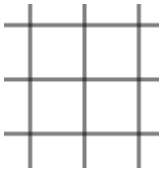
Herança

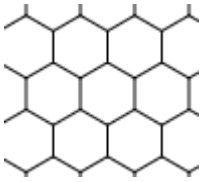
O o**bjeto SceneGridLayer** herda de SceneBaseObject e possui também todas as suas características.

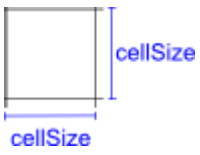
Características

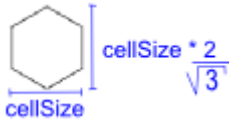
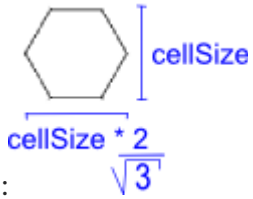
Além das características herdadas, o objeto também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
scene	Objeto Scene	(Somente Leitura) Contém o Objeto Scene a qual este SceneGridLayer pertence.
gridType	Enumerado: ● "square" ● "vertHex" ● "horzHex"	Define o tipo de grid que o tabuleiro de combate utiliza. "square": Grid com células quadradas  "vertHex": Grid com células hexagonais organizadas verticalmente.

		 "horzHex": Grid com células hexagonais organizadas horizontalmente. 
squareDiagonals	Enumerado: ● "simplified" ● "manhattan" ● "alternated" ● "euclidean"	Quando gridType for "square", esta propriedade define como as diagonais do grid quadriculado devem se comportar e pode ser um dos seguintes valores: ● "simplified" - (Valor padrão) Movimento pelas diagonais simplificado. Andar pelas diagonais possui o mesmo "custo" que andar pelas laterais. ● "manhattan" - O movimento pelas diagonais custam 2x o custo de andar pelas laterais. ● "alternated" - O primeiro movimento diagonal custa 1x, o segundo custa 2x, o terceiro custa 1x, o quarto 2x, e assim por diante. ● "euclidean" - O custo de andar pelas diagonais é igual à distância real.
offsetX	Double	Define o deslocamento em métrica de mundo que a primeira célula do grid (i:0, j:0) deve receber no eixo X.

		Em outras palavras, offsetX e offsetY são usadas para ajuste fino para posicionar corretamente o grid no tabuleiro. O RRPG faz um "WrapAround" no offset: Se você especificar um OffsetX que equivalha a 3,3 células por exemplo, o RRPG fará um deslocamento de apenas 0,3 células, pois deslocar 3,3 ou deslocar 0,3 células produz o mesmo resultado visual.
offsetY	Double	Define o deslocamento em métrica de mundo que a primeira célula do grid (i:0, j:0) deve receber no eixo Y. Em outras palavras, offsetX e offsetY são usadas para ajuste fino para posicionar corretamente o grid no tabuleiro. O RRPG faz um "WrapAround" no offset: Se você especificar um OffsetY que equivalha a 3,1 células por exemplo, o RRPG fará um deslocamento de apenas 0,1 células, pois deslocar 3,1 ou deslocar 0,1 células produz o mesmo resultado visual.
cellSize	Double	Define o tamanho de cada célula do grid em métrica de mundo. Exemplo: 1.5 significa que cada célula do grid ocupam 1.5 metros no espaço do mundo do tabuleiro (ou outra unidade de medida, conforme a propriedade scene.worldMetricName). Dimensões se o gridType for "square": 

		Dimensões se o gridType for "vertHex":  Dimensões se o gridType for "horzHex": 
gridColor	String de Cor	Define a cor da linha do grid.
gridThickness	Double	Define, em métrica de tela, a espessura das linhas do grid.
drawGrid	Boolean	True se as linhas do grid devem ser desenhadas, ou false se as linhas do grid devem ser invisíveis.
rowCount	Integer	(Somente Leitura) Com base no tamanho do tabuleiro (propriedade scene.worldWidth) e no tamanho de cada célula da grid (cellSize), esta propriedade contém quantas linhas a grid/matriz tem.
colCount	Integer	(Somente Leitura) Com base no tamanho do tabuleiro (scene.worldHeight) e no tamanho de cada célula da grid (cellSize), esta propriedade contém quantas colunas a grid/matriz tem.

Métodos

Método	Descrição
grid:cellToWorld(i, j)	Dada uma coordenada de matriz/grid Aij, retorna

	qual é a posição **central** desta célula em métrica de mundo. Parâmetros: ○ i - Número inteiro contendo qual linha a célula se encontra, o valor "i" da coordenada Aij. A primeira linha da grid começa no índice 0. ○ j - Número inteiro contendo qual coluna a célula se encontra, o valor "j" da coordenada Aij. A primeira coluna da grid começa no índice 0. Retorno: Esta função retorna dois valores na seguinte ordem: ○ worldX - Um número em métrica de mundo contendo onde, no eixo X, fica o centro da célula informada. ○ worldY - Um número em métrica de mundo contendo onde, no eixo Y, fica o centro da célula informada. Observações: ● Se a célula não existir de fato na grid, a função retornará a posição de onde seria o centro desta célula se ela existisse. Exemplo: local wX, wY = scene.grid:cellToWorld(10, 5); Neste exemplo, o centro da célula [Linha 10, Coluna 5] fica na coordenada (wX, wY) em métrica de mundo.

grid:worldToCell(worldX, worldY)	Dada uma posição em métrica de mundo, retorna qual célula da grid que este ponto pertence. Parâmetros: ○ worldX - Número em métrica de mundo contendo a posição no eixo X ○ worldY - Número em métrica de mundo contendo a posição no eixo Y Retorno: Esta função retorna dois valores na seguinte ordem: ○ i - Número inteiro contendo qual linha a célula se encontra ○ j - Número inteiro contendo qual coluna a célula se encontra Observações: ● Se o ponto (worldX, worldY) não estiver dentro das dimensões do tabuleiro, a função retornará qual célula a posição se refere se o tabuleiro fosse maior. Exemplo: local i, j = scene.grid:worldToCell(16, 6); Meste exemplo, a posição do mundo (x: 16, y: 6) pertence à célula Aij [Linha i, Coluna j].

grid:shortestPath(srcI, srcJ, destI, destJ[, options])	Dada duas coordenadas de matriz/grid Aij, calcula e retorna o caminho mais curto entres as duas células da grid. Parâmetros: ○ srcI - Número inteiro contendo qual linha a célula origem se encontra, o valor "i" da coordenada Aij. A primeira linha da grid começa no índice 0. ○ srcJ - Número inteiro contendo qual coluna a célula origem se encontra, o valor "j" da coordenada Aij. A primeira coluna da grid começa no índice 0. ○ destI - Número inteiro contendo qual linha a célula destino se encontra, o valor "i" da coordenada Aij. ○ destJ - Número inteiro contendo qual coluna a célula destino se encontra, o valor "j" da coordenada Aij. ○ (OPCIONAL) options - Uma tabela lua contendo opções de como o caminho mais curto deve ser calculado e pode possuir os seguintes atributos: ■ diagonals - Válido apenas para grid do tipo "square", Define como o cálculo deve se

	comportar em relação às diagonais do grid e pode conter um dos seguintes valores: ● "si ● " ● "a ● "e ■ endOfGrid - O que acontece quando a célula destino estiver fora dos limites do tabuleiro/grid? Pode conter um dos seguintes valores: ● "f ● "c

	"t ■ continuationData - Se quiser emendar caminhos de forma adequada, passe aqui o continuationData retornado pela última chamada de shortestPath. ■ trackCollisions - True se quiser que o sistema rastreie colisões durante o movimento. O padrão é false. ■ collisionRadius - Comprimento do raio de um círculo

	(em métrica de mundo) que será utilizado durante o rastreio de colisões de movimento. Retorno: Esta função retorna 4 valores na seguinte ordem: ○ success - True se conseguiu encontrar um menor caminho ou false se não foi possível. ○ distance - Quando success for true, contém um número contendo a distância do caminho encontrado em métrica de mundo . ○ steps - Quando success for true, contém os movimentos necessários no grid para chegar ao destino. É um array (tabela lua indexada de 1 a #tabela) de tabelas luas, cada uma contendo as propriedades "i"e "j" das coordenadas Aij de cada célula no caminho. ○ continuationData - Um valor que você pode passar para a próxima chamada de "grid:shortestPath" se quiser emendar caminhos de forma

	adequada.

Eventos

Nome do evento	Descrição
onChange	Evento que é disparado quando o Grid do scene mudar. As seguintes situações disparam este evento: ● Alteração no tamanho do Scene (propriedades scene.worldWidth e scene.worldHeight) ● Alteração em algumas das seguintes propriedades do grid: gridType, cellSize, offsetX, offsetY, gridColor, gridThickness e drawGrid

Exemplos

Exemplo 1 - Um plug-in que descobre em qual célula do grid o usuário clicou

```
require("scene.lua");


SceneLib.registerPlugin(
    function (scene, attachment)
        -- Manipular o evento onMouseUp do Viewport afim de detectar
o click do mouse


        scene.viewport.onMouseUp =
            function (event)
                -- Converter coordenada de tela do click para
coordenada de mundo
                local wx, wy = scene.viewport:screenToWorld(event.x,
                                                            event.y);
```

```
                -- Converter coordenada de mundo para coordenada do
grid
                local i, j = scene.grid:worldToCell(wx, wy);


                -- Exibir uma Mensagem sobre a célula clicada
                showMessage("Click na célula [Linha " .. i ..
                            ", Coluna " .. j .. "]");
            end;
    end);
```

Objeto ScenePluginAttachment

(Conteúdo desta página não consta no documento original.)

Objeto SceneUserDrawing

O Objeto SceneUserDrawing representa um único item gráfico no Scene do tipo "desenho do usuário".

Herança

O o**bjeto SceneUserDrawing** herda de SceneGraphicItem e possui também **TODAS** todas as suas características.

Veja também:

- Objeto SceneGraphicItem
- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneUserDrawing também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String "userDrawing"	(Somente Leitura) Contém um texto que identifica o tipo do objeto gráfico - "userDrawing".

Métodos

Método	Descrição
userDrawing:markAsMyDrawing();	Marca o UserDrawing como pertencente ao usuário atual. A marca de dono do userDrawing permite usuários com +jogadores desenhar e controlar seus desenhos no scene. Se o usuário atual for MESTRE, esta função transforma o desenho em um desenho que faz parte do scene e não pertence a ninguém. Se o usuário atual for +JOGADOR e se o item tiver acabado de ser criado, esta função marca o desenho como "desenho de jogador". Se o usuário atual for +JOGADOR mas o item já existir, a função não faz nada.

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 -

```
```

Exemplo 2 -

```
```

Objeto SceneToken

O Objeto SceneToken representa um único item gráfico no Scene do tipo token, isto é, personagens, objetos e outros símbolos gráficos.

Herança

O o**bjeto SceneToken** herda de SceneGraphicItem e possui também **TODAS** todas as suas características.

Veja também:

- Objeto SceneGraphicItem
- Objecto SceneBaseObject

Características

Além das características herdadas, o objeto SceneToken também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
objectType	String "token"	(Somente Leitura) Contém um texto que identifica o tipo do objeto gráfico - "token".
image	Objeto SceneDrawingOpBitma	(Somente Leitura) Contém um Objeto

	p	SceneDrawingOpBitmap com os detalhes da imagem principal do token.
name	String	Define o nome do token. Este nome é exibido visualmente no Scene.
facingMode	Enumerado: ● "rotate" ● "drawArrow" 	Define como o Scene deve indicar visualmente para que lado o token está virado. "rotate": Rotacionar a imagem para a direção. "drawArrow": Não rotacionar a imagem e desenhar uma seta para a direção.
friendOrFoe	Enumerado: ● "unknown" ● "friend" ● "neutral" ● "foe"	Define se este token é hostil ou amigável ao grupo do RRPG. "unknown" - Desconhecido, o mestre não definiu ou é um objeto. "friend" - Este token representa alguém/algo amigável aos personagens do grupo. "neutral" - Este token representa alguém/algo neutro ao grupo. "foe" - Este token representa alguém/algo hostil aos personagens do grupo.
ownerUserID	String	Contém o login do usuário do RRPG que é dono deste token e pode movimentá-lo.
ownerCharacter	Inteiro	Contém o ID interno do personagem deste token. Este valor é equivalente ao valor da propriedade "codigoIntero" do objeto

		"BibliotecaItem" 0 = Não foi definido o personagem.
visionIntenseLightRange	Double	Define, em métrica de mundo, o alcance da visão do token sob luz intensa.
visionWeakLightRange	Double	Define, em métrica de mundo, o alcance da visão do token sob luz fraca (exemplo: Luz do luar, uma vela fraca, etc..)
visionDarknessRange	Double	Define, em métrica de mundo, o alcance da visão do token na escuridão total.
visionHaveVision	Boolean	Define se o token possui visão/enxerga coisas no tabuleiro. Padrão: False
visionAngle	Double	Define, em graus, o ângulo de visão do token. Padrão: 360
lightIntenseRange	Double	Define, em métrica de mundo, o alcance da luz intensa emitida por este token. Se for igual a 0, este token não emite luz intensa. Padrão: 0
lightWeakRange	Double	Define, em métrica de mundo, o alcance da luz fraca emitida por este token. Se for igual a 0, este token não emite luz fraca. Padrão: 0
lightAngle	Double	Define, em graus, o ângulo da

		luz emitida por este token. Padrão: 360
barValue1 **barValue2** **barValue3**	String	Define o valor atual de uma das barrinhas do token.
barMax1 **barMax2** **barMax3**	String	Define o valor máximo de uma das barrinhas do token.
barColor1 **barColor2** **barColor3**	String de Cor	Define a cor de uma das barrinhas do token.
isMine	Boolean	(Somente Leitura) Contém true se o token for controlado pelo usuário atual do RRPG.

Métodos

Método	Descrição
token:clearMemoryAreas();	No FogOfWar, limpa a memória do Token das áreas que ele já viu em algum momento do passado.

Eventos

Nome do evento	Descrição

Exemplos

Exemplo 1 -

```
```

Exemplo 2 -

```
```

Objeto SceneViewport

O Objeto Viewport representa a tela do usuário, a janela de exibição, onde o scene está sendo renderizado.

O Viewport de um scene pode ser obtido através da propriedade "viewport" do objeto Scene

Herança

O o**bjeto SceneViewport** herda de SceneBaseObject e possui também todas as suas características.

Características

Além das características herdadas, o objeto SceneViewport também possui as seguintes características:

Propriedades e atributos

Propriedade	Tipo	Descrição
scene	Objeto Scene	(Somente Leitura) Contém o Objeto Scene que este Viewport renderiza.
width	Double	(Somente Leitura) Contem a LARGURA em Métrica de Tela do viewport, isto é, da janela onde o scene está sendo renderizado.

height	Double	(Somente Leitura) Contem a ALTURA em Métrica de Tela do viewport, isto é, da janela onde o scene está sendo renderizado.
worldX	Double	Define, em Métrica de Mundo, em qual posição do eixo X do mundo o ponto (X: 0, Y: 0) da tela aponta. As propriedades "worldX" e "worldY" servem para fazer "scroll" no scene. Se quiser fazer um scroll para a direita, por exemplo, incrementa-se o valor de worldX. Se quiser fazer um scroll para cima, decrementa-se o valor de worldY; Este é um valor local e não é compartilhado com os outros usuários que estão visualizando o mesmo scene.
worldY	Double	Define, em Métrica de Mundo, em qual posição do eixo Y do mundo o ponto (X: 0, Y: 0) da tela aponta. As propriedades "worldX" e "worldY" servem para fazer "scroll" no scene. Se quiser fazer um scroll para a direita, por exemplo, incrementa-se o valor de worldX. Se quiser fazer um scroll para cima, decrementa-se o valor de worldY; Este é um valor local e não é compartilhado com os outros usuários que estão visualizando o mesmo scene.

scale	Double	Define a escala entre a Métrica do Mundo e a Métrica da Tela do Scene. Em outras palavras, esta propriedade controla o "zoom" do Scene. Exemplo: Um valor 32.0 significa que cada unidade métrica do mundo ocupam 32 unidades métricas de tela. Se quiser, por exemplo, efetuar um zoom+ no scene, incrementa-se "scale". Se quiser reduzir o zoom pela metade, divide-se por dois o valor de "scale". Este é um valor local e não é compartilhado com os outros usuários que estão visualizando o mesmo scene.

cursor	Enumerado: ● "default" ● "arrow" ● "handPoint" ● "hourGlass" ● "IBeam" ● "size" ● "sizeNESW" ● "sizeNS" ● "sizeNWSE" ● "sizeWE" ● "upArrow" ● "drag" ● "noDrop" ● "hSplit" ● "vSplit" ● "multiDrag" ● "sqlWait" ● "no" ● "appStart" ● "help" ● "cross"	Define qual cursor de mouse deve ser apresentado quando o mouse estiver em em cima do viewport do scene. ● "default" - Cursor padrão, uma seta padrão. ● "arrow" - Uma seta padrão. ● "handPoint" - Uma mão com um dedo apontando para cima. Normalmente é usado para identificar um item selecionável/clicável ● "hourGlass" - Uma ampulheta para indicar um processo em andamento. ● "IBeam" - Cursor que sinaliza a possibilidade de entrar com texto no controle. ● "size" - Indicação de possibilidade de redimensionamento. ● "sizeNESW" - Indicação de possibilidade de redimensionamento no sentido Nordeste-Sudoeste ● "sizeNS" - Indicação de possibilidade de redimensionamento no sentido Norte-Sul ● "sizeNWSE" - Indicação de possibilidade de redimensionamento no sentido Noroeste-Sudeste ● "sizeWE" - Indicação de possibilidade de redimensionamento no sentido Leste-Oeste ● "upArrow" - Uma

		seta apontando para cima. ● "drag" - Um cursor indicando que itens estão sendo arrastados. ● "noDrop" - Indicação de que não é possível arrastar para este alvo. ● "hSplit" - Indicação de splitter horizontal ● "vSplit" - Indicação de splitter vertical. ● "multiDrag" - Indicação de que vários itens estão sendo arrastados. ● "sqlWait" - Ampulheta indicando um processo de banco de dados em andamento. ● "no" - Uma indicação de negação, de que não é possível fazer algo. ● "appStart" - Combinação de "default" e "hourGlass", normalmente usado para indicar que um aplicativo está sendo aberto. ● "help" - Uma seta com um ponto de interrogação. ● "cross" - Um cursor em formato de cruz, usado para indicar um posicionamento preciso.

Métodos

Método	Descrição
viewport:screenToWorld(screenX,	Converte uma posição definida em métricas de tela em uma posição definida em

screenY)	métricas de mundo. Em outras palavras, esta função responde a seguinte pergunta: O ponto (screenX, screenY) da tela do usuário representa qual ponto (worldX, worldY) do tabuleiro? Esta conversão leva em consideração o "scroll" e o "zoom" (propriedades worldX, worldY e scale) do Viewport. Parâmetros: ○ screenX - a posição do eixo X do ponto da tela ○ screenY - a posição do eixo Y do ponto da tela Retorno: ○ Esta função retorna 2 valores na seguinte ordem: worldX e worldY contendo o ponto convertido em métricas de mundo. Exemplo de uso: local worldX, worldY = scene.viewport:screenToWorld(10, 50);
viewport:screenToWorld(screenRect)	Converte um retângulo definido em métricas de tela em um retângulo definido em métricas de mundo. Em outras palavras, esta função responde a seguinte pergunta: O retângulo "screenRect" da tela do usuário representa qual retângulo do tabuleiro? Esta conversão leva em consideração o "scroll" e o "zoom" (propriedades worldX, worldY e scale) do Viewport.

	Parâmetros: ○ screenRect - Uma tabela SceneRect contendo o retângulo que será convertido. Retorno: ○ Uma tabela SceneRect contendo o retângulo convertido em métricas de mundo.
viewport:worldToScreen(worldX, worldY)	Converte uma posição definida em métricas de mundo em uma posição definida em métricas de tela. Em outras palavras, esta função responde a seguinte pergunta: O ponto (worldX, worldY) do mundo do scene fica em qual posição (screenX, screenY) na tela do usuário? Esta conversão leva em consideração o "scroll" e o "zoom" (propriedades worldX, worldY e scale) do Viewport. Parâmetros: ○ worldX - a posição do eixo X do mundo do scene. ○ worldY - a posição do eixo Y do mundo do scene. Retorno: ○ Esta função retorna 2 valores na seguinte ordem: screenX e screenY contendo o

	ponto convertido em métricas de tela. Exemplo de uso: local screenX, screenY = scene.viewport:worldToScreen(3, 4.5);
viewport:worldToScreen(worldRect)	Converte um retângulo definido em métricas de mundo em um retângulo definido em métricas de tela. Em outras palavras, esta função responde a seguinte pergunta: O retângulo "worldRect" do mundo do scene ocupa qual retângulo da tela do usuário? Esta conversão leva em consideração o "scroll" e o "zoom" (propriedades worldX, worldY e scale) do Viewport. Parâmetros: ○ worldRect - Uma tabela SceneRect contendo o retângulo que será convertido. Retorno: ○ Uma tabela SceneRect contendo o retângulo convertido em métricas de tela.
viewport:getScrollBounds()	Retorna os limites de scroll para os valores worldX e worldY. Retorno: Esta função retorna 4 valores numéricos na seguinte ordem e definição: ○ minX - Um número. O scene não deixa a propriedade worldX ficar menor que este

	valor. ○ minY - Um número. O scene não deixa a propriedade worldY ficar menor que este valor; ○ maxX - Um número. O scene não deixa a propriedade worldX ficar maior que este valor. ○ maxY - Um número. O scene não deixa a propriedade worldY ficar maior que este valor. Exemplo de uso: local minX, minY, maxX, maxY = viewport:getScrollBounds();
viewport:setupToolCategory(name, title, order)	Atualiza ou cria uma categoria de ferramentas no Scene. Parâmetros: ○ name - Uma cadeia de caracteres contendo o nome da categoria. Este nome pode ser referenciado depois pela função addToolButton ○ title - Uma cadeia de caracteres contendo a descrição/título da categoria. ○ order - Um

	número definindo a ordem da categoria. Esta categoria será exibida depois/abaixo de outra categoria de ordem menor. (A grosso modo, quem tiver número de ordem menor fica na frente)
viewport:addToolButton(categoryName, title, iconURL, order, options, callback [, deactivationCallback])	Adiciona um botão-ferramenta ao Scene. Parâmetros: ○ categoryName - O nome da categoria em que o botão deve ser inserido. Este é o mesmo nome usado na função setupToolCategory. Se for especificado uma categoria inexistente, ela será criada. ○ title - Cadeia de caracteres contendo o título do botão. ○ iconURL - Cadeia de caracteres contendo o endereço/caminho de uma imagem que será utilizada como ícone da ferramenta. ■ Pod

	e ser um endereço de internet (exemplo: “http://xxxx.

	com.br/a.png”) ■ Pode ser o caminho de um arquivo cont

	ido no pacote do plugin. Pode ser um arquivo que se

	encontra no HD Virtual do plugin instalado. ○ order - Um número contendo a ordem do botão em sua categoria. A grosso modo, quem tiver um número de

	ordem menor fica na frente. ○ options - **nil** ou uma tabela LUA podendo conter os seguintes atributos: ■ "selectable" - Booleano, onde true significa q

	ue a ferramenta é uma ferramenta selecionável. Uma f

	erramenta selecionável permanece ativa até que fique

	"não selecionada". Por padrão, este valor é false e

	significa que a ferramenta executa algo que não pre

	cisa mudar seleção da ferramenta ativa. ■ defaultOfCat

	egory - Booleano onde true significa que esta ferram

	enta é a padrão da categoria que está e que ela ser

	á u t i l i z a d a q u a n d o o u s u á r i o c l i c a r n a c a t e g o r i a . O

	■ padrão é false. dontFocus - Booleano onde true significa

	ica que a ferramenta não deve ser focada como ferra

	mentapadrãodacategoriaaposousuárioclicarnel

	a. O padrão é false. DontFocus não funciona se select

	ableestiverativo. ○ callback - Uma função que será chamada quando o usuário ativar/clicar esta ferramenta. ○ (OPCIONAL) deactivationCallback - Uma função que será chamada quando esta ferramenta estava selecionada e o usuário selecionou outra. Este callback é chamado apenas se a ferramenta for do tipo "selecionável" e o Scene espera que sua ferramenta pare de fazer o que ela faz quando selecionada.

	Retorno: ○ Esta função retorna um número identificador do ToolButton no scene. Este mesmo valor pode ser utilizado posteriormente na função removeToolButton.
viewport:removeToolButton(toolID)	Remove um botão ferramenta que foi previamente adicionado pela função addToolButton. Parâmetros: ○ toolID - Um número identificador do ToolButton. Este valor é retornado pela função addToolButton.
viewport:checkToolButton(toolID[, checked])	Marca ou desmarca um ToolButton. A marcação é uma mera indicação visual para o usuário entender que o Tool está ativo de alguma forma. Podem existir mais de 1 ToolButton marcados ao mesmo tempo. Parâmetros: ○ toolID - toolID - Um número identificador do ToolButton. Este valor é retornado pela função addToolButton. ○ (OPCIONAL) checked - Um booleano, onde

	true significa marcar o tool button e **false** significa desmarcar o tool button. Se não informado, true será utilizado como padrão.
viewport:isToolButtonChecked(toolID)	Retorna se um determinado ToolButto está marcado. A marcação é uma mera indicação visual para o usuário entender que o Tool está ativo de alguma forma. Podem existir mais de 1 ToolButton marcados ao mesmo tempo. Parâmetros: ○ toolID - toolID - Um número identificador do ToolButton. Este valor é retornado pela função addToolButton. Retorno: ○ Um Boolean, onde **true** significa que o toolbutton está marcado.
viewport:selectDefaultToolButton()	Seleciona a Ferramenta/Tool Button padrão. Pode existir apenas uma ferramenta selecionada ao mesmo tempo e representa a ferramenta que o usuário está trabalhando no momento. A ferramenta padrão normalmente é a de "selecionar/movimentar" itens do scene.
viewport:selectToolButton(toolID)	Seleciona a Ferramenta/Tool Button especificada pelo padrão. Pode existir apenas uma ferramenta selecionada ao mesmo tempo e representa a ferramenta que o usuário está trabalhando no

	momento. Parâmetros: ○ toolID - toolID - Um número identificador do ToolButton. Este valor é retornado pela função addToolButton.
viewport:showForm(form[, options])	Exibe uma janela na interface do usuário na forma de um Popup flutuante acima do viewport do scene. Parâmetros: ● form - O formulário a ser exibido na interface. Deve ser um objeto Lua Form já instanciado. ● (OPCIONAL) options - Uma tabela/objeto LUA contendo as opções da exibição do popup. As opções são: ○ onClose - [function, padrão = nil] - Uma função lua que será invocada quando o popup for fechado. ○ placement - [string enumerado] - Onde o popup será exibido.. Os possíveis valores são: ■ "center"

	- no centro da tela "bottom" - na parte de baixo da te

	■ la "top" - na parte de cima da tela ■ "left" - à esquerda

	na tela "right" - à direita na tela "topLeft" - no can

	to superior esquerdo da tela "topRight-no canto superior

	■ rior direito da tela "bottomLeft" - no canto inferior

	esquerdo da tela "bottomRight" - no canto inferior d

	■ direito da tela "mouse" - onde o mouse está atualmente

	■ "mouseCenter" - centralizado onde o mouse está atual

	■ mente. nilou não informado - Posição personalizada.

	Retorno: Se conseguir exibir o form como um popup no viewport, retorna **true**; Senão retorna **false**. Observações: ● A função não espera o popup fechar para retornar. Após esta chamada, o código LUA continuará sua execução normal enquanto o form é mostrado.
viewport:closeForm(form)	Fecha uma janela popup que foi previamente aberta pela da função viewport:showPopup. Parâmetros: ● form - O formulário a ser fechado/tirado da interface. Deve ser um objeto Lua Form já instanciado Retorno: Se conseguir fechar o form, retorna **true**; Senão retorna **false**.
viewport:needRepaint()	Informa ao RRPG que o Scene precisa ser redesenhado.

Métodos para renderização

Todos os métodos listados abaixos são válidos APENAS dentro de eventos "onBeforeDraw"s e "OnAfterDraw"s

Método	Descrição
viewport:prepareFill(color)	Prepara o viewport para renderizar o preenchimento de polígonos com uma cor sólida. Parâmetros: ● color - uma cadeia de caracteres contendo uma cor. Observações:

	● Após chamar este método, os métodos viewport:fill*** utilizarão esta cor no desenho.
viewport:prepareDraw(color [, thickness, strokeCap, strokeJoin, strokeDash])	Prepara o viewport para renderizar o contorno de polígonos com uma cor sólida. Parâmetros: ● color - uma cadeia de caracteres contendo uma cor. ● (OPCIONAL) thickness - A espessura do contorno métrica de tela. Se omitido, o valor padrão é 1.0; ● (OPCIONAL) strokeCap - Define o estilo gráfico usado ao desenhar o fim das linhas. Pode ser "flat" (linhas com pontas retangulares) ou "round" (linhas com pontas arredondadas). Se omitido, o valor padrão é "flat". ● (OPCIONAL) strokeJoin - Define o estilo gráfico usado ao juntar segmentos de linha em uma forma. Pode ser "miter" (As quinas/junções são "quadradas"), "round" (As quinas/junções são arredondadas) ou "bevel" (As quintas/junções são ligadas de forma diagonal). Se omitido, o valor padrão é "miter" ● (OPCIONAL) strokeDash - Define o estilo gráfico da linha. Pode ser "solid" (A linha é sólida), "dash" (A linha é tracejada), "dot" (A linha é feita de pontos), "dashDot" (A linha é alternada entre traços e pontos), ou "dashDotDot" (A linha é alternada entre traço, ponto e ponto). Se

	omitido, o valor padrão é "solid" Observações: ● Após chamar este método, os métodos viewport:draw*** utilizarão estas configurações.
viewport:prepareFont(size [,fontName, bold, italic, underline])	Prepara o viewport para preencher textos com uma determinada fonte. Parâmetros: ● size - Um número contendo o tamanho da fonte. Ex: 12 ● (OPCIONAL) fontName - O nome da fonte a ser usada. Se **nil**, uma fonte padrão será utilizada. ● (OPCIONAL) bold - True se desejar que a fonte esteja em negrito. ● (OPCIONAL) italic - True se desejar que a fonte esteja em itálico. ● (OPCIONAL) underline - True se desejar que a fonte esteja sublinhada.
viewport:fillRect(left, top, right, bottom[, opacity])	Desenha o preenchimento de um retângulo na tela do scene. Parâmetros: ● left - Posição esquerda (no eixo X) em métrica de tela de onde o retângulo deve ser preenchido. ● top - Posição superior (no eixo Y) em métrica de tela de onde o retângulo deve ser preenchido. ● right - Posição direita (no eixo X) em métrica de tela de onde o retângulo deve ser preenchido. ● bottom - Posição inferior (no eixo Y) em métrica de tela de onde o retângulo deve ser preenchido.

	● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor do retângulo, invoque primeiro o método "prepareFill"
viewport:fillEllipse(left, top, right, bottom[, opacity])	Desenha o preenchimento de uma elipse/figura oval na tela do scene. Parâmetros: ● left - Posição esquerda (no eixo X) em métrica de tela de onde a elipse deve ser preenchida. ● top - Posição superior (no eixo Y) em métrica de tela de onde a elipse deve ser preenchida. ● right - Posição direita (no eixo X) em métrica de tela de onde a elipse deve ser preenchida. ● bottom - Posição inferior (no eixo Y) em métrica de tela de onde a elipse deve ser preenchida. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor da elipse, invoque primeiro

	o método "prepareFill"
viewport:fillPolygon(polygon[, opacity])	Desenha o preenchimento de um polígono na tela do scene. Parâmetros: ● polygon - Um array de tabelas que contém propriedades x e y, e cada cada item do array representando um vértice do polígono. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor do polígono, invoque primeiro o método "prepareFill"
viewport:fillACell(i, j [, opacity])	Desenha o preenchimento de uma célula do grid do scene. Parâmetros: ● i - a posição i da célula "Aij" ● j - a posição j da célula "Aij" ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor do retângulo, invoque primeiro o método

	"prepareFill"
viewport:drawRect(left, top, right, bottom[, opacity])	Desenha o contorno de um retângulo na tela do scene. Parâmetros: ● left - Posição esquerda (no eixo X) em métrica de tela de onde o retângulo deve ser preenchido. ● top - Posição superior (no eixo Y) em métrica de tela de onde o retângulo deve ser preenchido. ● right - Posição direita (no eixo X) em métrica de tela de onde o retângulo deve ser preenchido. ● bottom - Posição inferior (no eixo Y) em métrica de tela de onde o retângulo deve ser preenchido. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor do retângulo, invoque primeiro o método "prepareDraw"

viewport:drawEllipse(left, top, right, bottom[, opacity])	Desenha o contorno de uma elipse/figura oval na tela do scene. Parâmetros: ● left - Posição esquerda (no eixo X) em métrica de tela de onde a elipse deve ser preenchida. ● top - Posição superior (no eixo Y) em métrica de tela de onde a elipse deve ser preenchida. ● right - Posição direita (no eixo X) em métrica de tela de onde a elipse deve ser preenchida. ● bottom - Posição inferior (no eixo Y) em métrica de tela de onde a elipse deve ser preenchida. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir o material/cor da elipse, invoque primeiro o método "prepareDraw"
viewport:drawLine(startX, startY, endX, endY [,opacity])	Desenha uma linha na tela do scene. Parâmetros: ● startX - Posição no eixo X em métrica de tela de onde a linha deve se originar. ● startY - Posição no eixo Y em métrica de tela de onde a linha deve se originar. ● endX - Posição no eixo X em métrica de tela de onde a linha deve finalizar. ● endY - Posição no eixo X em métrica de tela de onde a

	linha deve finalizar. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0.
viewport:textSize(txt)	Calcula o tamanho do texto usando a fonte configurada no viewport. Parâmetros: ● txt - A cadeia de caracteres que será medida. Retorno: ● x - A largura do texto em métrica de tela ● y - A altura do texto em métrica de tela Observações: ● Para definir a fonte, invoque primeiro o método "prepareFont"
viewport:fillText(x, y, txt [, opacity])	Desenha um texto no scene. Parâmetros: ● x - Posição no eixo X em métrica de tela de onde o texto deve começar a ser desenhado. ● y - Posição no eixo Y em métrica de tela de onde o texto deve começar a ser desenhado. ● txt - A cadeia de caracteres que será desenhada. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for

	informado, será utilizado 1.0. Observações: ● Para definir a fonte, invoque primeiro o método "prepareFont" ● Para definir o material/cor do texto, invoque primeiro o método "prepareFill"
viewport:fillTextFit(x, y, right, bottom, txt[, horzTextAlign, vertTextAlign, , opacity])	Desenha um texto no scene de forma a fazer cabe-lo da melhor forma em um retângulo definido. Parâmetros: ● x - Posição no eixo X em métrica de tela de onde o texto deve começar a ser desenhado. ● y - Posição no eixo Y em métrica de tela de onde o texto deve começar a ser desenhado. ● right - Posição no eixo X em métrica de tela de onde o texto deve começar a ser desenhado. ● txt - A cadeia de caracteres que será desenhada. ● (OPCIONAL) opacity - Um número entre 0.0 e 1.0 definindo a opacidade do desenho, onde 0.0 = totalmente transparente e 1.0 = totalmente opaco. Se este parâmetro não for informado, será utilizado 1.0. Observações: ● Para definir a fonte, invoque primeiro o método "prepareFont" ● Para definir o material/cor do texto, invoque primeiro o método "prepareFill"
viewport:pushRenderState()	Salva as configurações atuais de

	renderização para que possam ser restauradas mais tardes usando popRenderState. Observação: Para cada chamada de pushRenderState, deve haver uma chamada de popRenderState.
viewport:popRenderState()	Restaura as configurações de renderização previamente salvadas pelo método pushRenderState. Observação: Para cada chamada de pushRenderState, deve haver uma chamada de popRenderState.
viewport:pushRotationTransform(angle, centerX, centerY)	Empurra uma transformação de rotação no viewport. Parâmetros: ● angle - Angulo em graus da rotação que deve ser feita. ● centerX - Posição do pivô de rotação no eixo X em métrica de tela. ● centerY - Posição do pivô de rotação no eixo Y em métrica de tela. Observações: ● As próximas renderizações serão rotacionadas com esta configuração até a chamada de popTransform. ● Para cada chamada de pushRotationTransform deve haver uma chamada de popTransform.
viewport:pushWorldToScreenTransform( );	Empurra uma transformação de coordenadas em métrica de mundo para métrica de tela no viewport. Após a chamada desta função, todos os métodos de desenho entenderão que as coordenadas passadas estão em métrica de mundo. Observações:

	• Para cada chamada de pushWorldToScreenTransform deve haver uma chamada de popTransform.
viewport:popTransform();	Remove do viewport a última transformação que havia sido empurrada.

Eventos

Nome do evento	Descrição
onChange	Evento que é disparado quando o Viewport do scene mudar. As seguintes situações disparam este evento: • Alteração no tamanho da tela (propriedades width e height) • Alteração no scroll do viewport (propriedades worldX e worldY) • Alteração no zoom do viewport (propriedade scale)
onMouseDown	Este evento é invocado quando um botão do mouse é pressionado sobre o Viewport ou quando o dedo toca a tela (touchscreen). Parâmetros: • event - uma tabela Lua que contém os seguintes atributos: ○ button – indica qual botão do mouse foi pressionado, e pode conter um dos seguintes valores: “left”, “middle” ou “right”... Quando o dedo é pressionado (touchscreen), este valor é “left” e isTouch = true. ○ x – posição X em métrica de tela indicando onde o mouse foi

	pressionado. ○ y – posição Y em métrica de tela indicando onde o mouse foi pressionado. ○ shiftKey - true se a tecla shift estiver pressionada. ○ ctrlKey - true se a tecla control estiver pressionada. ○ altKey - true se a tecla alt estiver pressionada. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen.
onMouseMove	Este evento é invocado quando o mouse se movimenta pelo viewport ou quando o dedo (touchscreen) é arrastado pelo viewport. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X em métrica de tela indicando para onde o mouse moveu. ○ y – posição Y em

	métrica de tela indicando para onde o mouse moveu. ○ shiftKey - true se a tecla shift estiver pressionada também ○ ctrlKey - true se a tecla control estiver pressionada também ○ altKey - true se a tecla alt estiver pressionada também. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen.
onMouseUp	Este evento é invocado quando um botão do mouse é liberado sobre o Viewport ou quando o dedo solta da tela (touchscreen). Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ button – indica qual botão do mouse foi liberado, e pode conter um dos seguintes

	valores: “left”, “middle” ou “right”... Quando o dedo é levantado (touchscreen), este valor é “left” e isTouch = true. ○ x – posição X em métrica de tela indicando onde o mouse foi liberado. ○ y – posição Y em métrica de tela indicando onde o mouse foi liberado. ○ shiftKey - true se a tecla shift estiver pressionada também ○ ctrlKey - true se a tecla control estiver pressionada também ○ altKey - true se a tecla alt estiver pressionada também. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen.
onMouseDblClick	Este evento é invocado quando o usuário realiza a

	ação duplo clique com o mouse sobre o Viewport. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ x – posição X em métrica de tela indicando onde ocorreu o duplo clique. ○ y – posição Y em métrica de tela indicando onde ocorreu o duplo clique. ○ shiftKey - true se a tecla shift estiver pressionada. ○ ctrlKey - true se a tecla control estiver pressionada. ○ altKey - true se a tecla alt estiver pressionada. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen. Observação: Este evento é disparado junto com onMouseDown e onMouseUp na seguinte ordem: Mouse Down -> Mouse Up -> Mouse Down -> Dbl Click -> Mouse Up

onMouseWheel	Este evento é invocado quando a "rodinha" do mouse é é rolada sobre o Viewport. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos: ○ delta - Um número que indica a distancia que a roda foi rodada. Se for positivo, significa que a rodinha foi rolada para cima e se for negativo significa que ela foi rolada para baixo. ○ shiftKey - true se a tecla shift estiver pressionada ○ ctrlKey - true se a tecla control estiver pressionada ○ altKey - true se a tecla alt estiver pressionada ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. Se seu evento controlou adequadamente este evento e não quer que nenhum outro plug-in tente manipular mais este evento, retorne explicitamente true usando "return true;"
onFallbackMouseWheel	Idêntico ao evento onMouseWheel, porém é chamado apenas quando nenhum código tratar o evento onMouseWheel.

onGestureBeginZoom	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, começa a realizar um gesto de zoom. Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi iniciado. Esta posição é o pivô do zoom. ● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi iniciado. Esta posição é o pivô do zoom. Observações: Neste momento, o usuário ainda não realizou zoom, apenas destacou sua intenção de o fazer. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureZoom	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, realiza um gesto de zoom. Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi realizado. Esta posição é o pivô do zoom. ● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi realizado. Esta posição é o pivô do zoom. ● delta - Um número contendo a distância em métrica de tela do zoom desejado. Se negativo, é um gesto de zoom out. Este valor é a variação da distância deste o último evento onGestureZoom. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureEndZoom	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, finaliza um gesto de zoom.

	Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi finalizado. Esta posição é o pivô do zoom. ● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi finalizado. Esta posição é o pivô do zoom. Observações: Neste momento, o usuário já realizou todo zoom que queria fazer. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureBeginRotation	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, começa a realizar um gesto de rotação. Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi iniciado. Esta posição é o pivô da rotação. ● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi iniciado. Esta posição é o pivô da rotação. Observações: Neste momento, o usuário ainda não realizou rotação, apenas destacou sua intenção de o fazer. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureRotation	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, realiza um gesto de rotação. Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi realizado. Esta posição é o pivô da rotação.

	● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi realizado. Esta posição é o pivô da rotação. ● angle - Número contendo o ângulo da rotação em graus. Valor positivo significa rotação em sentido anti-horário, valor negativo em sentido horário. Este valor é a variação do ângulo deste o último evento onGestureRotation. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureEndRotation	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, finaliza um gesto de rotação. Parâmetros: ● x - Número, a posição CENTRAL no eixo X em métrica de tela de onde o gesto foi finalizado. Esta posição é o pivô da rotação. ● y - Número, a posição CENTRAL no eixo Y em métrica de tela de onde o gesto foi finalizado. Esta posição é o pivô da rotação. Observações: Neste momento, o usuário já realizou toda rotação que queria fazer. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureBeginPan	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, começa a realizar um gesto de deslize/scroll. Parâmetros: ● x - Número, a posição no eixo X em métrica de tela de onde o gesto foi iniciado. ● y - Número, a posição no eixo Y em métrica de tela de onde o gesto foi iniciado. Observações: Neste momento, o usuário ainda não realizou deslizamento, apenas destacou sua

	intenção de o fazer. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGesturePan	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, realiza um gesto de deslize/scroll. Parâmetros: ● x - Número, a posição no eixo X em métrica de tela de onde o gesto foi realizado. Este representa onde, no viewport, está o ponteiro/dedo. ● y - Número, a posição no eixo Y em métrica de tela de onde o gesto foi realizado. Este representa onde, no viewport, está o ponteiro/dedo. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureEndPan	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, finaliza um gesto de deslize/scroll. Parâmetros: ● x - Número, a posição no eixo X em métrica de tela de onde o gesto foi finalizado. ● y - Número, a posição no eixo Y em métrica de tela de onde o gesto foi finalizado. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onGestureLongTap	Este evento é invocado quando o usuário, utilizando o TouchScreen e seus dedos, realiza um gesto de longTap (pressionar e segurar por algum tempo) Parâmetros: ● x - Número, a posição no eixo X em métrica de tela de onde o gesto foi realizado.

	● y - Número, a posição no eixo Y em métrica de tela de onde o gesto foi realizado. Retorne explicitamente **true** para indicar ao Scene que você já lidou com este evento e nenhum outro script precisa se preocupar com ele.
onKeyDown	Este evento é invocado quando uma tecla (teclado) é pressionada no Scene. Parâmetros: ● event - uma tabela Lua que contém os seguintes atributos ○ key - Cadeia de caracteres contendo a letra do teclado que foi pressionada ○ keyCode – Código numérico da tecla que foi pressionada (algumas teclas, como F7, por exemplo, não representam uma letra, daí a necessidade do keyCode) ○ shiftKey - true se a tecla shift estiver pressionada. ○ ctrlKey - true se a tecla control estiver pressionada. ○ altKey - true se a tecla alt estiver pressionada. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true

	se a tecla control estiver pressionada. Observações: ● Se você alterar o valor de key e/ou keyCode, a mudança é incorporada. Caso queira suprimir/ignorar a tecla que acabou de ser pressionada, atribua event.key = "" e event.keyCode = 0. ● Se a combinação de teclas pressionada puder ser um atalho, então event.key virá com valor "" (string vazio) e event.keyCode com valor diferente de 0. ● Se a combinação de teclas pressionada puder ser tratada como um caractere exibível, event.key vira com uma string contendo apenas este caracter e event.keyCode virá com valor igual a 0.
onKeyUp	Este evento é invocado quando uma tecla (teclado) é liberada. Parâmetros: ● event - uma tabela Lua que contém as mesmas propriedades descritas no evento onKeyDown mas com informações sobre a tecla liberada.
onBeforeDrawBkgLayer	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento! Este evento é invocado antes do RRPG pintar a camada background do scene.
onBeforeDrawGridLayer	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento! Este evento é invocado depois do RRPG pintar a camada background e antes de pintar o grid do scene.
onBeforeDrawObjsLayer	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário.

	Não realize tarefas pesadas neste evento! Este evento é invocado depois do RRPG pintar o grid e antes de pintar a camada objetos do scene.
onBeforeDrawTokensLayer	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento! Este evento é invocado depois do RRPG pintar a camada objetos e antes de pintar a camada tokens do scene.
onAfterDrawLayers	Evento de renderização. Quando estiver neste evento, utilize os métodos de renderização do viewport para realizar pinturas na tela do usuário. Não realize tarefas pesadas neste evento! Este evento é invocado depois do RRPG pintar a camada tokens.
onStartDrop	Evento que é chamado quando alguém arrasta uma informação (drag) em cima do scene e você deseja informar quais ações ou dados existem em um determinado ponto do tabuleiro. Parâmetros: ● drop - Objeto Drop o qual você deve preencher as informações para o RRPG acasalar o drag com o drop. ● x - Numero contendo a posição do eixo X, em métrica de tela, de onde o drop está tentando ser feito. ● y - Numero contendo a posição do eixo y, em métrica de tela, de onde o drop está tentando ser feito. ● drag - Objeto Drag representando a informação que está sendo arrastada. Você não deve alterar as propriedades deste objeto. ● event - uma tabela Lua que contém os seguintes atributos: ○ x – Numero contendo a posição do eixo X, em métrica de tela, de onde o drop está tentando ser feito. ○ y – Numero contendo a posição do eixo y, em

	métrica de tela, de onde o drop está tentando ser feito. ○ shiftKey – true se a tecla shift estiver pressionada. ○ ctrlKey – true se a tecla control estiver pressionada. ○ altKey – true se a tecla alt estiver pressionada. ○ cmdKey - true se a tecla command (macOS) estiver pressionada. ○ ctrlOrCmdKey - No macOS: true se a tecla command estiver pressionada. Outros dispositivos: true se a tecla control estiver pressionada. ○ isTouch - true se este evento foi disparado por causa de um toque em um display touchscreen. Por favor, consulte Arrastando-e-Soltando Informações / Drag and Drop para conhecer melhor sobre o processo drag and drop.

Exemplos

Exemplo 1 - Um plugin que detecta quando o Viewport de um scene muda

```
require("scene.lua");


SceneLib.registerPlugin(
    function (scene, attachment)
        scene.viewport.onChange =
            function()
                local r = scene.viewport:screenToWorld({left=0,
top=0,

right=scene.viewport.width,

bottom=scene.viewport.height});


                local texto = "O Viewport mudou..\n" ..
                              "Dimensões da tela: "    ..
scene.viewport.width .. " por " ..
                              scene.viewport.height .. "\n" ..
                              "Posicao do Scroll: " ..
scene.viewport.worldX .. " por " ..
                              scene.viewport.worldY .. "\n" ..
                              "Região do Scene que é exibido neste
viewport: " ..
                              string.format("[(left %f, top %f)(right
%f, bottom %f)]",
                                            r.left, r.top, r.right,
```

```
    r.bottom);


                showMessage(texto);
            end;
        end);
```

Exemplo 2 - Um plug-in que descobre em qual célula do grid o usuário clicou

```
    require("scene.lua");


    SceneLib.registerPlugin(
        function (scene, attachment)
            -- Manipular o evento onMouseUp do Viewport afim de detectar
    o click do mouse


            scene.viewport.onMouseUp =
                function (event)
                    -- Converter coordenada de tela do click para
    coordenada de mundo
                    local wx, wy = scene.viewport:screenToWorld(event.x,
                                                                event.y);




                    -- Converter coordenada de mundo para coordenada do
    grid
```

```
                local i, j = scene.grid:worldToCell(wx, wy);



                -- Exibir uma Mensagem sobre a célula clicada
                showMessage("Click na célula [Linha " .. i ..
                            ", Coluna " .. j .. "]");
            end;
    end);
```

Exemplo 3 - Um plug-in que adiciona/remove uma marca personalizada quando o usuário clica nos itens do scene.

```lua
require("scene.lua");


SceneLib.registerPlugin(

    function (scene, attachment)


        -- Manipular o evento OnMouseDown do Viewport


        scene.viewport.onMouseDown = function(event)

            -- Usuário clicou no Scene

            -- Transformar as coordenadas de tela em métrica de mundo

            local worldX, worldY =
scene.viewport:screenToWorld(event.x, event.y);


            -- Localizar o item que o usuário clicou

            local item = scene.items:itemAtPoint(worldX, worldY);


            if item ~= nil then

                -- Encontrou um item na posição clicada.

                local NOME_MINHA_MARCA = "MarcaExemplo";


                -- Localizar uma operação gráfica neste item com o
nosso nome

                local opGrafica =
item.canvas:findByName(NOME_MINHA_MARCA);
```

```
                if opGrafica == nil then
                    --[[ A operação gráfica da minha marca ainda não
                        existe neste token/item. Vamos criar ]]

                    opGrafica = item.canvas:addBitmap();
                    opGrafica.name = NOME_MINHA_MARCA;
                   -- Abaixo, URL da imagem da nossa marca.
                    opGrafica.url =
"https://wiki.teamfortress.com/w/images/d/dd/Bleed_drop.png?
t=20110425044341";
                    opGrafica.x = 0.25;
                    opGrafica.width = 0.3;
                    opGrafica.y = -0.1;
                    opGrafica.height = 0.3;
                    opGrafica.z = 10;
                else
                    -- Vamos apagar nossa operação gráfica
previamente criada
                    opGrafica:delete();
                end;
            end;
        end;
```

```
    end);
```



Item sem a marca



Item com a marca

Estrutura SceneRect

(Conteúdo desta página não consta no documento original.)

Biblioteca System

Biblioteca que contém funções para lidar com o dispositivo do usuário e outras funcionalidades básicas.

Todas as funções estão contidas na unidade "system.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "system.lua"
require("system.lua");

-- Agora é possível acessar as funções da biblioteca
System.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca System

```
function System.setClipboardText(text)
```

Altera o conteúdo do Clipboard/Área de Transferência do sistema operacional (ctrl+c).

Parâmetros:

- text - Cadeia de caracteres contendo o novo conteúdo que será colocado no Clipboard do usuário.

Retorno:

- Caso consiga alterar o conteúdo do clipboard, retorna **true**, senão retorna **false.**

```
function System.getClipboardText()
```

Retorna o conteúdo do Clibpboard/Área de Transferência do sistema operacional (ctrl+c)

Retorno:

- Uma cadeia de caracteres caso haja um texto no clipboard ou **nil**.

```
function System.readClipboardAsDragNDrop(callback, [options])
```

Read the device clipboard as a drag-and-drop operation. Helpful to read multiple clipboard formats (such as files and images).

Parameters:

- callback - a function that will be called to handle the drag-and-drop operation. The callback will receive the following parameters:
  - drop - The Drop Object that you must fill in the information so Firecast can match the drag to the drop.
  - x - The number passed in "options.x" or 0
  - y - The number passed in "options.y" or 0
  - drag - The Drag Object representing the clipboard data. You must not change the properties of this object.
- (Optional) options - a table that may contain the following attributes:
  - (Optional) x -The "x" position that the callback function will receive
  - (Optional) y -The "y" position that the callback function will receive.

Returns:

- **true** if the operation had a match or **false** instead.

Remarks:

- The callback function is called immediately. The drag object is valid only inside the callback execution.
- The drag and drop operation works exactly in the same way the onStartDrop event that all visual tags have.
- Please read Dragging and Dropping Information to learn more about the drag-and-drop process.

```
function System.getScreenSize()
```

Retorna as dimensões da tela do dispositivo do usuário.

Retorno:

- Retorna dois números: largura e altura da tela em pontos lógicos.

Exemplo:

```
local largura, altura =
system.getScreenSize();
```

```
function System.isMobile()
```

Retorna se o plugin está rodando no Firecast Mobile.

Retorno:

- Retorna **true** se o plugin está rodando no App do FireCastMobile ou **false** se está rodando no RRPG de computador

```
function System.getTickCount()
```

Retorna um número contendo há quantos milisegundos o dispositivo do usuário está ligado. Ótimo para calcular o tempo decorrido entre operações.

SDK3 API Version Functions

Version history

Major	Minor	Description
87	0	SDK 3.6 (Firecast 8+)
87	1	SDK 3.6b (Firecast 8.5+)
87	2	SDK 3.6c (Firecast 8.6+)
87	3	SDK 3.6d (Firecast 8.8+)

87	4	SDK 3.6e (Firecast 8.11+)
88	0	SDK 3.7 (Firecast 8.12+)

Identical major versions possess compatibility with each other.

For example, you can compile a plugin using SDK3.6b (87.1), and it will still run fine on an SDK 3.6 (87.0) running API because the two have the same major version.

```
function System.checkAPIVersion(major, [minor])
```

Checks if the current running SDK3 API version is greater or equal to a specific version.

Parameters:

- major - The major version integer to check.
- (Optional) minor - The minor version integer to check. When omitted, 0 is used by default

Returns:

- **true** if the current running SDK3 API version is greater or equal to the specified version; **false** otherwise.

Remarks:

- You may want to check the SDK3 API version history.

```
function System.getAPIVersion()
```

Gets the current running SDK3 API version.

Returns:

- An integer number in the same format returned by the function System.makeAPIVersionNumber

Remarks:

- You may want to check the SDK3 API version history.

```
function System.makeAPIVersionNumber(major, [minor])
```

Make an API version integer number that you can use to compare versions.

Parameters:

- major - The major version integer

- (Optional) minor - The minor version integer. When omitted, 0 is used by default

Returns:

- An integer number. It's a 32 bits where the upper 16 bits contain the major version, and the lower 16 bits hold the minor version.

Remarks:

- You may want to check the SDK3 API version history.

Biblioteca Utils

A biblioteca gui provê funções relacionadas às interfaces Lua Form.

Todas as funções estão contidas na unidade "utils.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "utils.lua"
require("utils.lua");

-- Agora é possível acessar as funções da biblioteca
Utils.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);

FUNCAO_DA_BIBLIOTECA2(Parametro1, Parametro2, ...);
```

Funções da biblioteca utils

```
function clearInterval(intervalId)
ou
function Utils.clearInterval(intervalId)
```

Cancela o temporizador que setInterval havia previamente configurado.

Parâmetros:

- intervalId - O identificador retornado pela função setInterval.

```
function clearTimeout(timeoutId)
```

```
ou
function Utils.clearTimeout(timeoutId)
```

Cancela o temporizador que setTimeout havia previamente configurado.

Parâmetros:

- timeoutId - O identificador retornado pela função setTimeout.

```
function Utils.compareStringPtBr(strLeft, strRight)
```

Realiza comparação entre duas cadeias de caracteres de forma case-insensitive e ignorando acentuações

Retorna:

- Número **menor** que 0 se strLeft for lexicograficamente **menor** que strRight
- 0 se strLeft for lexicograficamente **igual** a strRight
- Número **maior** que 0 se strLeft for lexicograficamente **maior** que strRight

```
function lowercase(str)
```

Converte e retorna um string em caixa baixa (lower case)

Parâmetros:

- str - a cadeia de caracteres que deseja converter para caixa baixa.

Retorno:

- uma cadeia de caracteres

```
function uppercase(str)
```

Converte e retorna um string em caixa alta (upper case)

Parâmetros:

- str - a cadeia de caracteres que deseja converter para caixa alta.

Retorno:

- uma cadeia de caracteres

```
function Utils.generateUniqueString()
```

Gera e retorna uma cadeia de caracteres globalmente única, isto é, um texto que nunca mais será gerado novamente por esta função

Retorno:

- Uma cadeia de caracteres única, ótima para ser usada como identificador.

```
function isnumber(value)
ou
function isNumber(value)
```

Retorna true se o parâmetro "value" for um número, senão retorna false.

```
function round(value)
```

Arredonda "value" para o numero inteiro mais próximo e o retorna.

```
function removerAcentos(str)
ou
function Utils.removerAcentos(str)
```

Retorna o texto passado em "str" sem sua acentuação.

```
function Utils.removerFmtChat(str[, removerSmileys])
```

Retorna o texto passado em "str" sem os caracteres de formatação de cores/negrito/itálico que o RRPG aceita no chat.

Parâmetros:

- str - Cadeia de caracteres contendo o texto a ser processado
- (OPCIONAL) removerSmileys - Booleano indicando se deve remover os smileys/memes do texto também. Se omitido, o valor padrão é false.

Retorno:

- Cadeia de caracteres sem as tags de formatação.

```
function setInterval(callbackFunction, interval [, parametersToCallbackFunction])
ou
function Utils.setInterval(callbackFunction, interval [, parametersToCallbackFunction])
```

setInterval chama uma função qualquer após em intervalos de tempo especificado em milisegundos.

Parâmetros:

- callbackFunction - A função que será chamada no intervalo definido.
- interval - O intervalo de tempo em milisegundos que callbackFunction será chamada.
- (OPCIONAL) parametersToCallbackFunction - Uma lista de parâmetros que serão repassados para a callbackFunction quando ela for chamada.

Retorno:

- Retorna um valor que poderá ser usado na função clearInterval para cancelar o setInterval.

Observações:

- A função "callbackFunction" é executada inúmeras vezes até que algum dos seguintes eventos aconteça:
  - O plug-in seja descarregado.
  - A função clearInterval seja executada passando em seu parâmetro o valor retornado por setInterval.
  - A função informada em "callbackFunction" retorne explicitamente false.
- Se você deseja executar "callbackFunction" apenas uma vez após um determinado tempo, use a função setTimeout.

```
function setTimeout(callbackFunction, interval [, parametersToCallbackFunction])
ou
function Utils.setTimeout(callbackFunction, interval [,
parametersToCallbackFunction])
```

setTimeout chama uma função qualquer após uma quantidade específica de milisegundos se passarem.

Parâmetros:

- callbackFunction - A função que será chamada.
- interval - O tempo de espera até que a função seja chamada, em milisegundos.

- (OPCIONAL) parametersToCallbackFunction - Uma lista de parâmetros que serão repassados para a callbackFunction quando ela for chamada.

Retorno:

- Retorna um valor que poderá ser usado na função clearTimeout para prevenir a função "callbackFunction" de ser chamada.

Observações:

- A função "callbackFunction" é executada somente 1 vez. Se você precisar que ela seja chamada constantemente, use a função setInterval

```
function Utils.sortPtBrArray(array)
```

Dado um array de strings (uma tabela lua indexada de 1 a n), ordena os valores lexicograficamente de forma case insensitive e ignorando acentuações.

Retorno:

- Não há retorno. O próprio array estará ordenado no fim desta função.

```
function totable(str)
ou
function strToTable(str)
ou
function Utils.strToTable(str)
```

Converte o texto passado em "str" para uma tabela lua. O texto deve seguir o mesmo padrão que você usa ao declarar uma tabela lua quando está programando.

Exemplo: "{nome='joão', idade=10, filhos={}}"

```
function tableToStr(table [, pretty])
ou
function Utils.tableToStr(table [, pretty])
```

Converte uma tabela LUA para string.

Parâmetros:

- table - a tabela lua que deseja converter para texto
- (OPCIONAL) pretty - se true, converte para um texto legível... Se omitido ou se false, converte para um texto que ocupa o menor espaço o possível.

Retorno:

- Uma cadeia de caracteres que corresponde à tabela.

```
function Utils.binaryEncode(destArray, format, value [, startIndex])
```

Codifica um determinado valor em um formato binário.

Parâmetros:

- destArray - Uma tabela lua que receberá os BYTES da codificação binária.
- format - Uma cadeia de caracteres identificando qual o formato binário deve ser utilizado. Veja Formatos Binários para saber quais são os formatos disponíveis.
- value - O valor que será codificado.
- (OPCIONAL) startIndex - Um número identificando em qual posição de "destArray" os bytes devem ser escritos. Se omitido, o valor 1 será utilizado.

Retorno:

- Um número contendo quantos bytes foram inseridos em destArray.

Observações:

- Cada BYTE da representação binária é colocado em uma posição de "destArray". Se um determinado formato binário ocupar 4 bytes, após a chamada da função destArray[1] conterá o valor do primeiro byte, destArray[2] o valor do segundo byte, destArray[3] o valor do terceiro byte e destArray[4] o valor do quarto byte.

Formatos Binários:

Formato	Qtd. de Bytes	Descrição
"u8"	1	Um número entre 0 e 255 codificado como Unsigned Integer de 8 Bits (também conhecido como Byte)
"s8"	1	Um número entre -128 a 127 codificado como Signed Integer de 8 Bits.
"u16"	2	Um número entre 0 e 65535 codificado como Unsigned Integer de

		16 bits (Também conhecido como Word) no padrão LittleEndian.
"s16"	2	Um número entre -32768 e 32767 codificado como Signed Integer de 16 bits no padrão LittleEndian.
"u32"	4	Um número entre 0 e 4294967295 codificado como Unsigned Integer de 32 bits (Também conhecido como DoubleWord) no padrão LittleEndian.
"s32"	4	Um número entre -2147483648 e 2147483647 codificado como Signed Integer de 32 bits no padrão LittleEndian
"u64"	8	Um número entre 0 e (2^64) - 1 codificado como Unsigned Integer de 64 bits (Também conhecido como QuadWord) no padrão LittleEndian
"s64"	8	Um número entre -2^63 a (2^63) - 1 codificado como Signed Integer de 64 bits no padrão LittleEndian.
"float"	4	Um número de ponto flutuante de 4 bytes no padrão IEEE 754.
"double"	8	Um número de ponto flutuante de 8 bytes no padrão IEEE 754.
"ansi"	Varia	Uma cadeia de caracteres codificada em ANSI sem o terminador nulo.
"utf8"	Varia	Uma cadeia de caracteres codificada em UTF-8 sem o terminador nulo.
"utf16"	Varia	Uma cadeia de caracteres codificada em UTF-16 sem o terminador nulo.

```
function Utils.binaryDecode(sourceArray, format [, startIndex [, length]])
```

Interpreta um valor que está codificado em um formato binário.

Parâmetros:

- sourceArray - Uma tabela lua que contém os BYTES da codificação binária.
- format - Uma cadeia de caracteres identificando em qual formato binário o valor está codificado. Veja Formatos Binários para saber quais são.
- (OPCIONAL) startIndex - Um número identificando a partir de qual posição de "sourceArray" os bytes devem ser lidos para a decodificação. Se omitido, o valor 1 será utilizado.
- (OPCIONAL) length - Quantos bytes à partir de "startIndex" devem ser usados para a decodificação. Se omitido, o valor (#sourceArray - startIndex + 1) será utilizado.

Retorno:

- A função retorna 2 valores:
  - O valor decodificado ou **nil** se não foi possível decodificar
  - A quantidade de bytes usados para a decodificação do valor.

Observações:

- Cada BYTE da representação binária deve estar em uma posição de "sourceArray". Se um determinado formato binário ocupar 4 bytes, sourceArray[1] deve conter o valor do primeiro byte, sourceArray[2] o valor do segundo byte, sourceArray[3] o valor do terceiro byte e sourceArray[4] o valor do quarto byte.

```
function Utils.getBinarySize(format)
```

Retorna quantos bytes um determinado formato binário ocupa.

Parâmetros:

- format - Uma cadeia de caracteres identificando o formato binário que deseja consultar. Veja Formatos Binários para saber quais são.

Retorno:

- Um número identificando a quantidade de bytes um determinado formato binário ocupa. Se um formato binário ocupar uma quantidade variada de bytes, o valor -1 é retornado.

```
function Utils.zlibCompress(sourceStream, destStream, [, level [, qtBytesInSource]])
```

Lê dados de um objeto stream fonte e, usando a biblioteca zLib, o compacta escrevendo o resultado em outro objeto stream.

Parâmetros:

- sourceStream - Um objeto stream representando os dados a serem compactados.
- destStream - Um objeto stream que receberá o conteúdo compactado.
- (OPCIONAL) level - Define o nível da compactação. Pode ser
  - "none" - Não haverá compactação no final das contas
  - "fastest" - Compactação rápida
  - "default" - Modo em que balanceia velocidade e taxa de compressão. Este é o modo padrão quando não for definido o nível.
  - "max" - Modo com maior taxa de compressão.
- (OPCIONAL) qtBytesInSource - Quantos bytes serão lidos de "sourceStream" à partir de sua posição atual para a compactação. Quando não definido, todo o restante do sourceStream será lido para a compactação.

Retorno:

- Um número identificando quantos bytes foram escritos em destStream

```
function Utils.zlibDecompress(sourceStream, destStream)
```

Lê dados de um objeto stream fonte e, usando o a biblioteca zLib, o descompacta escrevendo o resultado em outro objeto stream.

Parâmetros:

- sourceStream - Um objeto stream contendo os dados a serem descompactados.
- destStream - Um objeto stream que receberá o conteúdo descompactado.

Observações:

- Esta função descompacta um "bloco compactado" por completo. Não tem como antecipar quantos bytes serão lidos de "sourceStream" nem quantos bytes serão escritos em "destStream" utilizando esta função. No fim da chamada, destStream conterá todo o dado descompactado e a posição de "sourceStream" apontará para o primeiro byte subsequente do conteúdo que foi processado.

Retorno:

- Um número identificando quantos bytes foram escritos em destStream

```
function Utils.zlibCompressAsync(sourceStream, destStream, level, qtBytesInSource, onFinishCallba
onErrorCallback)
```

Lê dados de um objeto stream fonte e, usando a biblioteca zLib, o compacta escrevendo o resultado em outro objeto stream.

A compactação se dá de forma assíncrona, isto é, em "background". Ótimo para compactar streams grandes e não "congelar" o programa durante o processo.

Parâmetros:

- sourceStream - Um objeto stream representando os dados a serem compactados.
- destStream - Um objeto stream que receberá o conteúdo compactado.
- level - Define o nível da compactação. Pode ser
  - "none" - Não haverá compactação no final das contas
  - "fastest" - Compactação rápida
  - "default" - Modo em que balanceia velocidade e taxa de compressão. Este é o modo padrão quando não for definido o nível.
  - "max" - Modo com maior taxa de compressão.
- qtBytesInSource - Quantos bytes serão lidos de "sourceStream" à partir de sua posição atual para a compactação.
- onFinishCallback - Uma função que será invocada quando a compactação finalizar.
- onErrorCallback - Uma função que será invocada quando a compactação retornar um erro. A mensagem de erro é passada no primeiro parâmetro desta função.

Observações:

- IMPORTANTE: Durante a compactação assíncrona, seu código não deve fazer nenhuma operação em sourceStream e nem em destStream, senão o planeta explodirá e 2d6 erros estranhos surgirão.

```
function Utils.zlibDecompressAsync(sourceStream, destStream, onFinishCallback, onErrorCallback)
```

Lê dados de um objeto stream fonte e, usando o a biblioteca zLib, o descompacta escrevendo o resultado em outro objeto stream.

A descompactação se dá de forma assíncrona, isto é, em "background". Ótimo para descompactar streams grandes e não "congelar" o programa durante o processo.

Parâmetros:

- sourceStream - Um objeto stream contendo os dados a serem descompactados.
- destStream - Um objeto stream que receberá o conteúdo descompactado.

- onFinishCallback - Uma função que será invocada quando a descompactação finalizar.
- onErrorCallback - Uma função que será invocada quando a descompactação retornar um erro. A mensagem de erro é passada no primeiro parâmetro desta função.

Observações:

- Esta função descompacta um "bloco compactado" por completo. Não tem como antecipar quantos bytes serão lidos de "sourceStream" nem quantos bytes serão escritos em "destStream" utilizando esta função. No fim da chamada, destStream conterá todo o dado descompactado e a posição de "sourceStream" apontará para o primeiro byte subsequente do conteúdo que foi processado.
- IMPORTANTE: Durante a descompactação assíncrona, seu código não deve fazer nenhuma operação em sourceStream e nem em destStream, senão o planeta explodirá e 2d6 erros estranhos surgirão.

```
function Utils.newMemoryStream()
```

Cria e retorna um objeto stream que armazena os dados na memória RAM do dispositivo.

Observações:

- Os dados armazenados no Memory Stream não são salvos em nenhum lugar e são perdidos quando o código não mais estiver usando o Stream ou quando o RRPG fechar.
- Ótimo para fazer processamento de dados temporários.
- Enquanto estiver em uso, os dados ocupam memória RAM do dispositivo.

Retorno:

- Um objeto stream

```
function Utils.newTempFileStream()
```

Cria e retorna um objeto stream que armazena os dados em um arquivo temporário no dispositivo do usuário.

Observações:

- Um arquivo temporário é criado no dispositivo do usuário e é automaticamente apagado quando não for mais utilizado.
- Ótimo para fazer processamento de dados temporários.
- Enquanto estiver em uso, os dados ocupam espaço de armazenamento do dispositivo (HD, SSD, Memory Card, etc..).

Retorno:

- Um objeto stream

```
function tryFinally(tryCallback, finallyCallback)
ou
function Utils.tryFinally(tryCallback, finallyCallback)
```

LUA não possui um operador try-finally para tratamento de exceções. Esta função é uma alternativa para o comportamento try-finally que o LUA não provê nativamente.

Executa a função "tryCallback", e:

- Se ocorrer algum erro na execução, a função "finallyCallback" é invocada e o erro é levantado logo após.
- Se não ocorrer erro na execução, a função "finallyCallback" é invocada e a execução continua normalmente.

Parâmetros:

- tryCallback - Uma função que será invocada.
- finallyCallback - Uma função que sempre será invocada após a execução de "tryCallback", mesmo havendo erros de execução.

```
function colorToRGBA(color)
ou
function Utils.colorToRGBA(color)
```

Dado uma cor no formato string de cor, retorna os tons de Red, Green, Blue e Alpha da cor

Parâmetros:

- Color - um string de cor contendo a cor que deseja decompor

Retorno:

- 4 números representando, respectivamente os tons de Red, Green, Blue e Alpha da cor. Os tons são números entre 0.0 e 1.0

```
function RGBAToColor(r, g, b, a)
ou
function Utils.RGBAToColor(r, g, b, a)
```

Dado tons de cores Red, Green, Blue e Alpha, retorna um string de cor que representa a cor.

Parâmetros:

- r - Tom de Red, número entre 0.0 e 1.0
- g - Tom de Green, número entre 0.0 e 1.0
- b - Tom de Blue, número entre 0.0 e 1.0

- a - Tom de Alpha, número entre 0.0 e 1.0. Se omitido, o valor é considerado 1.0

Retorno:

- Um string de cor

Objeto Stream

Um objeto stream contém uma sequência de dados genéricos, possui um início e um fim e você estará sempre em algum lugar entre estes dois pontos.

Usando stream, você pode ler e/ou escrever em vários tipos de armazenamentos, como um arquivo, uma memória dinâmica, etc..

Um stream representa uma sequência de BYTES que podem ser acessadas de forma sequencial ou aleatória (isto é, você pode ler o inicio de um stream, pular para seu final para ler mais dados, voltar ao meio do stream, etc..)

Exemplos de operações típicas em um stream:

- Ler 1 byte. Da próxima vez que você ler, você obterá o byte posterior, e assim por diante.
- Ler vários bytes
- Mover a posição atual do cursor no stream para, da próxima vez que você ler, obter os bytes à partir desta nova posição.
- Escrever 1 byte.
- Escrever vários bytes.

Formas de instanciar um objeto stream

Existem várias maneiras de obter um objeto stream. Algumas delas são:

- Criar um stream temporário que reside na memória RAM através da função utils.newMemoryStream
- Criar um stream temporário que reside no HD do dispositivo através da função utils.newTempFileStream
- Abrir ou criar um arquivo no HD virtual usando a função vhd.openFile
- Baixar um conteúdo da internet através da função internet.download
- Realizar uma requisição HTTP e obter a resposta através da propriedade responseStream do objeto HTTPRequest

Características

Propriedades e atributos

Propriedade	Tipo	Descrição
size	Integer	Contém o tamanho atual do stream em quantidade

		de bytes.
position	Integer	A posição do cursor do stream. onde o valor 0 representa o primeiro byte do stream. Todas as operações de leitura e escrita são feitas nesta posição.
remainingBytes	Integer	Quantos bytes ainda podem ser lidos no stream à partir da posição atual.
md5	String	Uma cadeia de caracteres contendo o Hash MD5 de todo o conteúdo do stream. O Hash é calculado na hora de obter o valor da propriedade.
sha1	String	Uma cadeia de caracteres contendo o Hash SHA1 de todo o conteúdo do stream. O Hash é calculado na hora de obter o valor da propriedade.

Métodos

Método	Descrição
stream:read(destArray, qtBytes)	Efetua a leitura de bytes à partir da position atual do stream. Parâmetros: ○ destArray - Uma tabela lua que receberá os bytes lidos do stream. Os valores são armazenados em índices de 1 a N, onde N = quantidade de bytes lidos.

	○ qtBytes - Um número contendo a quantidade de bytes que devem ser lidos à partir desta posição. Retorno: ○ Um número contendo quantos bytes realmente foram lidos do stream e preenchidos em destArray Importante: ○ A position do stream é avançada automaticamente após a chamada desta função conforme a quantidade de bytes lidos. ○ Nem sempre é possível ler a quantidade de bytes informada no parâmetro qtBytes. Você DEVE analisar o número retornado pela função a fim de saber quantos bytes realmente foram lidos.
stream:write(sourceArray [, qtBytes])	Efetua a escrita de bytes na position atual do stream. Parâmetros: ○ sourceArray - Uma tabela lua que contém os bytes que devem ser escritos no stream. Os valores devem estar armazenados em índices de 1 a

	N, onde N = quantidade de bytes que serão escritos. ○ (OPCIONAL) qtBytes - Quantos bytes do sourceArray devem ser escritos no stream. Se omitido, o length do array (#sourceArray) será usado. Retorno: ○ Um número contendo quantos bytes realmente foram escritos no stream. Importante: ○ A position do stream é avançada automaticamente após a chamada desta função conforme a quantidade de bytes escritos. ○ Os valores em sourceArray devem ser números! Para cada valor em sourceArray, se for menor que 0, o byte 0 será gravado e se for maior que 255, o byte 255 será gravado. Se em algum dos índices de sourceArray não contiver um número válido, o byte 0 é escrito no stream.

stream:close()	Fecha o stream imediatamente e libera os recursos que foram previamente alocados. Após esta chamada, nenhuma operação do stream estará mais disponível.
stream:writeBinary(format, valor1 [, valor2 [, valor3 [, valorN...]]])	Escreve um valor codificado em binário na posição atual do stream. Parâmetros: ○ format - Cadeia de caracteres identificando qual formatação binária deve ser usada. Veja Formatos Binários para saber quais são os possíveis formatos. ○ valor1 - O valor a ser codificado e escrito ○ (OPCIONAL) valor2, valor3, valorN, etc... - Outros valores que também devem ser codificados e escritos no stream. Retorno: ○ Um número contendo quantos bytes foram escritos no stream. Importante: ○ A position do stream é avançada automaticamente após a chamada desta função conforme a quantidade de bytes escritos.
stream:readBinary(format [, qt])	Efetua a leitura de valores codificados em

	binário na posição atual do stream. Parâmetros: ○ format - Cadeia de caracteres identificando em qual formatação binária o valor está codificado. Veja Formatos Binários para saber quais são os possíveis formatos. ○ (OPCIONAL) qt - Um número identificando QUANTOS valores codificados devem ser lidos. Se omitido, o valor 1 será utilizado. Retorno: ○ Retorna os valores decodificados ou nil se não foi possível decodificar. Importante: ○ A position do stream é avançada automaticamente após a chamada desta função conforme a quantidade de bytes lidos ○ Se especificado os formatos "ansi', "utf8" ou "utf16", todo o restando do stream é lido como uma cadeia de caracteres e o parâmetro "qt" é ignorado como se o valor 1 tivesse sido passado para

	ele.
stream:readAsBase64([qtBytes])	Lê bytes do stream e retorna uma cadeia de caracteres representando os dados codificados em Base64. Parâmetros: ○ (OPCIONAL) qtBytes - Um número definindo quantos bytes do stream devem ser lidos à partir da posição atual e codificados. Se omitido, o restante dos dados do stream será lido. Retorno: ○ Uma cadeia de caracteres representando os dados codificados em Base64
stream:writeBase64(str)	Decodifica dados previamente codificados em Base64 e os escreve na posição atual do stream. Parâmetros: ○ str - Uma cadeia de caracteres contendo os dados codificados em Base64. Retorno: ○ Um número contendo quantos bytes foram escritos no Stream.
stream:copyFrom(stream, qtBytes)	Copia o conteúdo de um outro objeto stream dentro deste stream. Parâmetros:

	○ stream - um objeto stream de onde os dados serão copiados. ○ qtBytes - um número representando quantos bytes serão copiados à partir da posição atual do parâmetro "stream". Retorno: ○ Um número contendo quantos bytes realmente foram copiados para este stream.

Exemplos

Exemplo 1 - .....

```

```

Biblioteca VHD

A biblioteca vhd provê funções para manipular o HD Virtual do plug-in, como ler um arquivo, excluir, etc...

Todas as funções estão contidas na table/variável "VHD" da unidade "vhd.lua".

Exemplo de uso:

```
-- Primeiro, é necessário usar a unidade "vhd.lua"
require("vhd.lua");
```

```
-- Agora é possível acessar as funções da biblioteca
VHD.FUNCAO_DA_BIBLIOTECA(Parametro1, Parametro2, ...);
```

Funções da biblioteca vhd

```
function VHD.copyFile(srcFileName, dstFileName)
```

Copy the file in the virtual HD

Arguments:

- `srcFileName` - String containing the path to the existing source file
- dstFileName - String containing the path of the new file

Remarks:

- The paths are expanded using "vhd.expandFileName" before copying.
- If the file "dstFileName" already exists, it will be replaced by the copied file.

```
function VHD.expandFileName(shortFileName)
```

Expande o nome curto de um arquivo a fim de ter sua localização em relação à raiz do HD virtual.

Parâmetros:

- shortFileName – o caminho encurtado de um arquivo a ser expandido.

Retorno:

Uma cadeia de caracteres, onde:

- Se a cadeia de caracteres passada pelo parâmetro shortFileName iniciar com o caractere "/", a função simplesmente retorna o valor passado no parâmetro (pois o nome do arquivo já está expandido, isto é, já está em relação à raiz do HD virtual).
- Caso contrário, a função procura pelo arquivo dentro de cada um dos diretórios de pesquisa (SEARCH PATHs) que foram previamente registrados (ver descrição da função "vhd.addSearchPath").
  - Caso encontre dentro de algum dos diretórios de pesquisa, a função retorna o caminho completo (expandido, com relação à raiz do HD virtual) do arquivo.
  - Caso não encontre o arquivo, a função retorna o mesmo valor passado pelo parâmetro shortFileName.

- Esta função leva em consideração os aliases registrados pela função VHD.registerAlias, traduzindo o apelido e retornando o nome real do arquivo.

```
function VHD.addSearchPath(directory)
```

Adiciona um diretório à lista de "diretórios de pesquisa" (SEARCH PATH) do HD Virtual.

Parâmetros:

- directory - cadeia de caracteres contendo o caminho **expandido** (isto é, em relação à raiz do HD Virtual) do diretório.

A função "vhd.expandFileName" utiliza estes diretórios de pesquisas na hora de expandir o nome curto de um arquivo.

```
function VHD.removeSearchPath(directory)
```

Remove um diretório da lista de "diretórios de pesquisa" (SEARCH PATH) que havia sido previamente registrado através a função "vhd.addSearchPath".

Parâmetros:

- directory - cadeia de caracteres contendo o caminho **expandido** (isto é, em relação à raiz do HD Virtual) do diretório.

```
function VHD.registerAlias(aliases)
```

Registra aliases/apelidos para arquivos no VHD.

Parâmetros:

- aliases - uma tabela lua contendo os aliases, onde cada par de chave-valor da tabela representa:
  - chave - uma cadeia de caracteres representando o aliases/apelido a ser criado
  - valor - o nome do arquivo no VHD que o alias deve traduzir.

Observações:

- O alias funciona com a função VHD.expandFileName e com a função **require** do Lua.
- O alias é case-sensitive, isto é, "myFile" é diferente de "MyFile".
- O alias não pode começar com o caracter "/". O nome traduzido pode começar com "/"

Exemplo:

```
local aliases = {};
aliases["mylua"] = "init.lua";
aliases["myFile2"] = "secondFile.lua";

VHD.registerAlias(aliases);
require("myFile2");
```

```
function VHD.fileExists(fileName)
```

Verifica se um arquivo existe no HD virtual.

Parâmetros:

- fileName - cadeia de caracteres contendo o caminho a verificar.

Retorno:

Retorna **true** caso o arquivo exista ou **false** se não existir.

Observação:

Esta função expande o nome do arquivo (através de "vhd.expandFileName") antes de realizar a verificação.

```
function VHD.directoryExists(path)
```

Verifica se um diretório existe no HD virtual.

Parâmetros:

- `path` - cadeia de caracteres contendo o caminho a verificar.

Retorno:

Retorna **true** caso o diretório exista ou **false** se não existir.

Observação:

Esta função expande o caminho (através de "vhd.expandFileName") antes de realizar a verificação.

```
function VHD.openFile(fileName [, mode])
```

Esta função abre um arquivo do HD virtual no modo especificado.

Parâmetros:

- fileName - cadeia de caracteres contendo o nome do arquivo a abrir.
- (OPCIONAL) mode - cadeia de caracteres definindo o modo em que o arquivo será aberto.Pode ser um dos seguintes valores:
  - "r" - modo somente leitura (padrão se omitido). A função falhará se o arquivo não existir.
  - "w" - modo escrita/leitura preservando o contendo do arquivo (se existir). Se o arquivo não existir, a função cria o arquivo.
  - "a" - modo append. Equivalente ao modo "w", porém o objeto stream retornado é posicionado no fim do arquivo.
  - "w+" - modo escrita/leitura, o conteúdo do arquivo é apagado se ele existir. Se o arquivo não existir, a função cria o arquivo.

Retorno:

- Caso a função obtenha êxito, retorna um Objeto Stream representando o conteúdo do arquivo. Qualquer alteração feita no objeto stream é refletida no arquivo.
- Caso a função falhe, retorna **nil** e a mensagem de erro.

Observações:

- Esta função expande o nome do arquivo (através de "vhd.expandFileName") antes de realizar a abertura.
- Para fechar o arquivo que foi aberto, você deve invocar o método close do objeto stream retornado. Se o objeto stream for coletado pelo coletor de lixo, o arquivo também é fechado automaticamente.
- Exceto para o modo "a", o objeto stream retornado aponta para o inicio do arquivo.
- Apenas o modo "r" é aceito para abrir arquivos que originalmente vieram no plug-in. Não é possível alterar o conteúdo destes arquivos

```
function VHD.forceDirectory(directoryPath)
```

Esta função cria, se necessário, um ou mais diretórios especificado por directoryPath no HD virtual.

Parâmetros:

- `directoryPath` - cadeia de caracteres contendo o caminho do diretório a ser "forçado".

Retorno:

- Caso o diretório já exista e não foi preciso fazer nada, a função retorna **false**.
- Caso o diretório não existia e a função conseguiu criá-lo, retorna **true**.
- Caso o diretório não exista mas a função também não consiga criá-lo, um erro é levantado.

Observações:

- O valor de "directoryPath" deve ser um caminho absoluto no VHD (isto é, deve começar com o caracter "/")
- A criação de diretório ocorre de forma recursiva. Isto é, ao especificar "/diretorioNaoExistente1/diretorioA/diretorioB", a função irá criar "/diretorioNaoExistente1" primeiro, depois "diretorioA" dentro de "/diretorioNaoExistente1/", e, por fim, "diretorioB" dentro de "/diretorioNaoExistente1/diretorioA/".

```
function VHD.deleteFile(fileName)
```

Esta função remove um arquivo do HD virtual.

Parâmetros:

- `fileName` - cadeia de caracteres contendo o caminho do arquivo a ser excluído.

Observações:

- Esta função expande o nome do arquivo (através de "vhd.expandFileName") antes de realizar a remoção.
- Esta função não pode ser usada para remover diretórios.
- Não é possível apagar um arquivo que originalmente veio no plug-in. Não é possível alterar o conteúdo destes arquivos

```
function VHD.deleteDirectory(directoryPath)
```

Esta função remove um diretório e todo seu conteúdo do HD virtual.

Parâmetros:

- `directoryPath` - cadeia de caracteres contendo o caminho do diretório que será excluído.

Observações:

- Esta função expande o caminho do diretório (através de "vhd.expandFileName") antes de realizar a remoção.
- Não é possível apagar um arquivo que originalmente veio no plug-in. Não é possível alterar o conteúdo destes arquivos.
- Esta função apaga o diretório e todo seu conteúdo recursivamente.

```
function VHD.enumerateContent(directoryPath)
```

Esta função retorna quais arquivos e diretórios estão dentro de um diretório no HD Virtual.

Parâmetros:

- `directoryPath` - cadeia de caracteres contendo o caminho do diretório que terá seu conteúdo enumerado.

Retorno:

- Se "directoryPath" não conter um caminho válido de diretório, um erro será lançado.
- Senão, um array de strings será retornado, onde cada posição do arranjo contém o nome do diretório/arquivo.

Observações:

- Esta função expande o caminho de directoryPath (através de "vhd.expandFileName") antes de realizar a enumeração..
- Esta função **não** enumera o conteúdo de forma recursiva. Isto é, se existirem sub-diretórios, seus conteúdos não serão enumerados neste retorno, apenas os itens que estão diretamente dentro de "directoryPath".
- O texto retornado no arranjo contém o nome do arquivo/diretório sem incluir "barras" ou outra forma de expressar o diretório que se encontra.
    - Exemplo: Se existirem dois arquivos (a.txt e b.txt) e um subdiretório ("/textos/histórias/") na pasta "/textos/", o retorno será:
        - {'a.txt', 'b.txt', 'histórias'}
- Para saber se um item do arranjo retornado é diretório ou é arquivo, utilize as funções VHD.fileExists e VHD.directoryExists

MACROS

Macros são atalhos criados pelo usuário para comandos/textos que são repetidos frequentemente. Por exemplo: Se o personagem de D&D do usuário possui um ataque padrão, é conveniente criar uma macro de nome, digamos, "ataque" que faz:

```
/r 1d20 + 5: Rolagem do
Ataque
/r 1d8 + 5: Rolagem do Dano
```

Quando o usuário digitar "/ataque" no chat do RRPG, estes comandos serão executados.

Criando e editando suas macros

digite "/macros" no chat do RRPG para abrir o gerenciador de macros!

Tipos de macros

Existem dois tipos de macros no RRPG:

- Macros Simples
- Macros em Lua

A macros simples contém comandos que são enviados para o chat, linha a linha. São extremamente fáceis e intuitivas de serem feitas, porém são limitadas.

As macros em Lua contém código de programação Lua e requerem um pouco de conhecimento de programação, porem são flexíveis!

Macros em Lua

As macros em Lua possuem um conjunto de funções utilitárias pré-definidas para serem utilizadas!

Conheça as funções disponíveis para as macros em lua.

Exemplo 1: Fazendo um teste com dificuldade igual a 15

```lua
local resultado = rolar("1d20 +
5");

if resultado >= 15 then
```

```
        enviar("Acertou!");
else
        enviar("Errou!");
end;
```

Biblioteca para Macros em Lua

Variáveis

```
variável parametro ou variável parameter
e
variável arg[indice]
```

(Não é uma função) Possui os parâmetros passado para a macro, onde a variável "parametro" contém a cadeia de caracteres completa e a variável "arg" contém um arranjo, onde cada item representa um argumento separado por espaço.

Se o usuário digitar "/minhaMacro Texto de Teste":

- A variável "parametro" conterá "Texto de Teste"
- A variável arg[1] conterá "Texto", arg[2] conterá "de", arg[3] conterá "Teste" e #arg contém a quantidade de argumentos.

Exemplo: Fazer um teste de resistência parametrizado.

```
local dificuldade = tonumber(parametro);

if dificuldade == nil then
   escrever("Utilize: /macro <NÚMERO>");
   return;
end;

local resultado = rolar("1d20 + 5", "Dificuldade " ..
dificuldade);

if resultado >= dificuldade then
  enviar("Passou no teste!");
else
```

```
    enviar("Não passou no teste");
  end;
```

```
variável chat
```

(Não é uma função) Objeto Chat representando o chat em que a macro foi disparada.

```
variable elapsedMS
```

(Not a function) Contains how many milliseconds have passed since the macro started executing

```
variável meuJogador ou variável myPlayer
```

(Não é uma função) objeto Jogador representando o usuário da macro naquela mesa.

Exemplo 1: Escrever na tela o seu próprio login.

```
    escrever(meuJogador.login
    );
```

```
variável myCharacter
```

(Não é uma função) Objeto Personagem representando o personagem principal do usuário naquela mesa.

Observações:

- Se o jogador não possuir nenhum personagem, acessar esta variável irá causar um erro com uma mensagem informativa de que "o jogador não possui nenhum personagem" e a macro pára de executar.

Exemplo 1: Escrever na tela o nome do seu personagem.

```
    write(myCharacter.nome)
```

```
variável sheet
```

(Não é uma função) O Objeto Nodo raiz representando o NodeDatabase/Sheet do personagem principal do usuário logado.

Observações:

- Se o jogador não possuir nenhum personagem, acessar esta variável irá causar um erro com uma mensagem informativa de que "o jogador não possui nenhum personagem" e a macro pára de executar.
- A execução da macro pode ser pausada até que o NodeDatabase/Ficha seja carregada pela internet.
- O Objeto Nodo retornado é o mesmo daquela variável "sheet" usada para programar as fichas/lua forms.
- As alterações feitas no objeto nodo retornado são sincronizadas normalmente, confome as regras de permissões do Node Database.
- Se houver algum erro ao carregar o NodeDatabase, a macro pára de executar.

Exemplo:

```
-- Escrever no chat a estrutura dos dados armazenados da
ficha

write(utils.tableToStr(sheet, true));


-- Se for a ficha D&D 5, a linha abaixo aumenta a CA em 1

sheet.CA = (tonumber(sheet.CA) or 0) + 1;


-- Se for a ficha D&D 5, a linha abaixo esreve no chat
Quantos PV o personagem tem

write(sheet.PV)
```

```
variável mesa ou variável room
```

(Não é uma função) objeto Mesa representando a mesa em que a macro está sendo executada.

Exemplo 1: Escrever algumas informações da mesa na tela.

```
escrever(mesa.nome);

escrever(mesa.sistema);

escrever(mesa.msgStatus);
```

```
variável jogadores ou variável players
```

(Não é uma função) Arranjo de objeto Jogador, representando os usuários que estão na mesa no momento.
Forma de uso: jogadores[1] contém o primeiro jogador, jogadores[2] contém o segundo jogador, jogadores[3] contém o terceiro, jogadores[#jogadores] possui o último jogador.

Exemplo 1: Escrever na tela o login de todos os jogadores da mesa.

```
for i = 1, #jogadores, 1
do

escrever(jogadores[i].log
in);
end;
```

Exemplo 2: Dar +voz para todos os espectadores da mesa.

```
for i = 1, #jogadores, 1 do
  if
jogadores[i].isEspectador
then

jogadores[i]:requestSetVoz(
true);
  end;
end;
```

```
variável inRoom
```

(Não é uma função) Booleano (true ou false) indicando se a macro está sendo executada em algum chat associado a alguma mesa.

Funções

```
function await(promise)
```

Pauses the macro execution until a Promise object is resolved

Arguments:

- promise - The Promise object to wait for.

Return:

- The resolved value of the Promise

Remarks:

- If the Promise is resolved with an error, an error will be raised with the corresponding error message.

```
function enviar(text[, params])
ou
function send(text[, params])
```

Sends a standard message to the chat.

Arguments:

- text - A string containing the standard message to be sent.
- *(Optional) params* - A SendMessageParameters table containing optional settings for the message.

Return:

- A LogRec table describing the sent message.

Examples:

```
send("Hello!");
send("Hello!", {impersonation = {mode="character",
name="Turok"}});
send("Hello!", {impersonation = {mode="narrator"}});
```

```
function agir(text[, params])
ou
function act(text[, params])
```

Sends an action message to the chat.

Arguments:

- text - A string containing the action message to be sent.

- *(Optional) params* - A SendMessageParameters table containing optional settings for the message.

Return:

- A LogRec table describing the sent message.

Examples:

```
act("wields his long sword");
act("wields his long sword", {impersonation =
{mode="character", name="Turok"}});
act("wields his long sword", {impersonation =
{mode="narrator"}});
```

```
function narrar(narracao)
ou
function narrate(narracao)
```

Envia uma mensagem de narração para o chat.

Parâmetros:

- narracao - Cadeia de caracteres contendo a narração que deseja enviar.

Observação: Apenas o mestre consegue enviar mensagens de narração.

Exemplo:

```
narrar("Após uma péssima noite de descanso, Gruumsh
acorda com as mãos algemadas");
```

```
function enviarNPC(npc, mensagem)
ou
function sendNPC(npc, mensagem)
```

Envia uma mensagem para o chat como se um NPC estivesse falando

Parâmetros:

- npc - Cadeia de caracteres contendo o nome do NPC.

- mensagem - Cadeia de caracteres contendo a mensagem que deseja enviar.

Observação: Apenas o mestre consegue enviar mensagens como NPC.

Exemplo:

```
enviarNPC("Rei Darik", "Guardas, prendam estes infelizes!");
```

```
function rolar(expression[, message[, params]])
ou
function roll(expression[, message[, params]])
```

Executes a dice roll in the chat.

Arguments:

- expression - A string containing the expression of the roll, such as "1d20 + 5," or a Roll Object.
- *(Optional) message* - A string containing an additional message describing the roll.
- *(Optional) params* - A SendMessageParameters table containing optional settings for the message.

Return:

- This function returns 3 pieces of information in the following order:
  - A number containing the roll result
  - A Roll Object describing the roll result.
  - A LogRec table describing the sent message.

Example 1:

```
local r = roll("1d20 + 5");

if r >= 15 then
        send("Hit!");
else
        send("Miss!");
end;
```

Example 2:

```
local r = roll("1d20 + 5", "Attack", {impersonation =
{mode="character", name="Turok"}});

if r >= 15 then
        roll("1d8 + 5", "Damage", {impersonation = {mode="character",
name="Turok"}});
end;
```

Exemplo 3: Rolar 4d6 e selecionar os 3 maiores números da rolagem.

```
local resultado, rolagem = roll("4d6");
local dados = {};

-- coletar os resultados individuais de cada dado
for i = 1, #rolagem.ops, 1 do
        local op = rolagem.ops[i];

        if op.tipo == "dado" then
                for j = 1, #op.resultados, 1 do
                        dados[#dados + 1] = op.resultados[j];
                end;
        end;
end;

-- Ordenar os resultados, menores primeiro
table.sort(dados);

local texto = "";

-- Pegar os ultimos 3, isto é, os 3 maiores.. de traz para frente

for i = #dados, #dados - 2, -1 do
                texto = texto .. " " .. dados[i];
end;

enviar("Os 3 maiores números foram:" .. texto);
```

```
function rolarLocal(rolagem)
```

```
ou
function rollLocal(rolagem)
```

Executa uma rolagem de dados local, isto é, uma rolagem que não será exibida em lugar nenhum.

Parâmetros:

- rolagem - Cadeia de caracteres contendo a rolagem a ser feita. Exemplo: "1d20 + 5"

Retorno:

- Esta função retorna um número contendo o resultado e um objeto rolagem com informações detalhadas do resultado.

Exemplos:

O comportamento da função "rolarLocal" é igual a da função "rolar", portanto, consulte os exemplos de "rolar".

```
function esperar(milisegundos)
ou
function wait(milisegundos)
```

Realiza uma pausa cronometrada na execução da Macro.

Parâmetros:

- milisegundos - Um número em milisegundos representando o intervalo de tempo que a execução da Macro ficará pausada. 1000 = 1 segundo, 500 = 0,5 segundos.

```
function load(luaCode)
```

Interpreta um código de macro escrita na linguagem LUA

Parâmetros:

- luaCode - Cadeia de caracteres contendo um código na linguagem LUA (seguindo o mesmo padrão de uma macro).

Retorno:

- Se luaCode possui um código Lua válido, retorna uma função que poderá ser invocada
- Se lucaCode **não** possui um código Lua válido, retorna dois valores: **nil** e uma mensagem de erro.

Exemplo:

```
local func, errorMsg = load("write(1 + 5 + 9 + 10)");

if func ~= nil then
  -- O codigo foi carregado. Vamos invocá-lo agora
  func();
else
  write("could not load lua code: " .. errorMsg);
end
```

```
function Log.e(tag, msg)
function Log.w(tag, msg)
function Log.i(tag, msg)
function Log.d(tag, msg)
function Log.v(tag, msg)
```

Emmit log message to the Firecast's log file and to the Firecast's console window. Use the /console command to reveal the console

Arguments:

- tag - Used to identify the source of a log message. It usually identifies the location or activity where the log call occurs.
- msg - The message you would like to be logged

Log severity:

Log.e	**Error message** A situation that represents an unhandled error that may interrupt some operation
Log.w	**Warning message** A situation that can be handled but may cause some unintended behavior
Log.i	**Informational/Notice message** A normal but significant condition that may require special handling.

Log.d

Debug message

A developer debug message

Log.v

Verbose message

Algorithm steps message, very low priority messages.

Remarks:

- Messages of this severity are not emitted in the Firecast's log files
- In the Firecast's console, you need to execute the command "filter add *" to be able to see these messages.

```
function showMessage(msg)
```

Exibe uma janela para o usuário contendo uma mensagem.

Parâmetros:

- msg - Cadeia de caracteres contendo a mensagem que deverá ser exibida para o usuário.

Observação:

- A execução da macro é pausada até que o usuário pressione OK na janela.

```
function alert(msg)
```

Exibe uma janela para o usuário contendo uma mensagem de alerta.

Parâmetros:

- msg - Cadeia de caracteres contendo a mensagem que deverá ser exibida para o usuário.

Observação:

- A execução da macro é pausada até que o usuário pressione OK na janela.

```
function toast(message)
```

Show a quick informative message to the user that does not require user interaction and hides automatically after a short time

Parameters

- message - The message string to be shown

```
function confirmOkCancel(question)
```

Exibe uma janela para o usuário onde ele deve escolher responder ou "Ok" ou "Cancelar"

Parâmetros:

- question - Cadeia de caracteres contendo a mensagem que deverá ser exibida para o usuário.

Retorno:

- **true** se o usuário responder "Ok" ou **false** se o usuário responder "Cancelar"

Observação:

- A execução da macro é pausada até que o usuário responda a janela.

Exemplo:

```
if confirmOkCancel("Será adicionado 6 à rolagem") then
  write("O usuario respondeu OK");
else
  write("O usuario respondeu Cancelar");
end
```

```
function confirmYesNo(question)
```

Exibe uma janela para o usuário onde ele deve escolher responder ou "Sim" ou "Não"

Parâmetros:

- question - Cadeia de caracteres contendo a pergunta que deverá ser exibida para o usuário.

Retorno:

- **true** se o usuário responder "Sim" ou **false** se o usuário responder "Não"

Observação:

- A execução da macro é pausada até que o usuário responda a janela.

Exemplo:

```
if confirmYesNo("Deseja continuar?") then
  write("O usuario respondeu Sim");
else
  write("O usuario respondeu Não");
end
```

```
function inputQuery(prompt[, initialValue, allowEmptyString])
```

Exibe uma janela para o usuário onde ele deve digitar um valor.

Parâmetros:

- prompt - Cadeia de caracteres orientando o usuário o que ele deve preencher.
- (OPCIONAL) initialValue - Cadeia de caracteres contendo uma sugestão de valor inicial para o usuário.
- (OPCIONAL) allowEmptyString - true se o usuário pode informar um texto em branco. O padrão é false.

Retorno:

- Se o usuário confirmar o valor, esta função retorna o valor escrito pelo usuário
- Se o usuário escolher cancelar na janela exibida, a macro pára de executar.

Observação:

- A execução da macro é pausada até que o usuário responda a janela.

Exemplo:

```
local bonus = inputQuery("Bônus de Ataque");
local defesa = inputQuery("Defesa", "10");

if roll("1d20 + " .. bonus) >= tonumber(defesa) then
  write("Acertou");
else
  write("Errou");
end;
```

```
function selectImageURL([defaultURL])
```

Esta função exibe uma janela para o usuário escolher uma URL de uma imagem.

Parâmetros:

- (OPCIONAL) defaultURL - Texto contendo uma URL sugerida para o usuário

Retorno:

- Retorna a URL de uma imagem escolhida pelo usuário

Observação:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.

Exemplo:

```
local url =
selectImageURL();

write("Imagem escolhida: "
.. url);
```

```
function choose(prompt, options, [defaultOptionIndex, shortCircuit])
```

Exibe um diálogo para que o usuário possa escolher uma opção dentre uma lista de opções.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- options - Um array/arranjo/lista de cadeias de caracteres/textos, contendo as opções que devem ser apresentadas ao usuário.
- (OPCIONAL) defaultOptionIndex - Um número contendo um índice informado qual dos itens de "options" deve ser apresentado como a escolha padrão. Se não informado, nenhuma opção será mostrada como padrão.
- (OPCIONAL) shortCircuit - Avaliação Curto-Circuito: Se for "true" e se houver apenas uma opção a ser mostrada ao usuario, a interface não é exibida e esta opçao é automaticamente selecionada. Se não informado, este parâmetro assume o valor false.

Retorno:

Esta função retorna dois valores na seguinte ordem:

- index - Um número, o índice de "options" que foi escolhido pelo usuário
- text - O texto equivalente ao índice escolhido pelo usuário, conforme "options"

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Lembre-se que indices de arrays em lua começa do número 1 ao invés de 0
- Se "options" não for válido ou se for um arranjo com 0 itens, a macro pára de executar como se o usuário tivesse cancelado a janela.

Exemplo:

```
local indice, texto = choose("O que deseja fazer?",
{"Atacar", "Defender"}, 1);


escrever("O usuário escolheu o índice: " .. indice);

escrever("O usuário escolheu: " .. texto);
```

```
function chooseMultiple(prompt, options)
```

Exibe um diálogo para que o usuário possa escolher **uma ou mais** opções dentre uma lista de opções.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- options - Um array/lista/arranjo de textos/cadeias de caracteres, contendo as opções que devem ser apresentadas ao usuário.

Retorno:

Esta função retorna dois valores na seguinte ordem:

- indexes - Um array/arranjo/lista de números, contendo os índices de "options" que foram escolhidos pelo usuário
- texts - Um array/arranjo/lista de textos equivalentes aos índices escolhidos pelo usuário, conforme "options"

Observação:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Lembre-se que indices de arrays em lua começa do número 1 ao invés de 0

- Se "options" não for válido ou se for um arranjo com 0 itens, a macro pára de executar como se o usuário tivesse cancelado a janela.

Exemplo:

```
local indices, textos = chooseMultiple("Bônus ativos",
{"Cobertura", "Defesa Total", "Escondido"});


for i = 1, #indices, 1 do

   escrever("O usuário escolheu o índice: " .. indices[i]);

   escrever("O usuário escolheu: " .. textos[i]);

end;
```

```
function chooseCharacter(prompt[, filter, shortCircuit])
```

Exibe um diálogo para que o usuário possa escolher um personagem da lista de personagens que estão na biblioteca da mesa.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- (OPCIONAL) filter - Uma cadeia de caracteres informando quais persnagens devem ser apresentados na interface de seleção e pode ser um dos seguintes valores:
  - "all" - Todos os personagens da mesa deverão ser apresentados na janela (Opção padrão caso não seja informado filter)
  - "pc" - Apenas os personagens de jogadores deverão ser apresentados na janela.
  - "pcOnline" - Apenas os personagens de jogadores que estão online deverão ser apresentados na janela.
  - "npc" - Apenas os personagens do mestre deverão ser apresentados.
  - "mine" - Apenas os personagens do jogador que executar a macro deverão ser apresentados.
- (OPCIONAL) shortCircuit - Avaliação Curto-Circuito: Se for "true" e se houver apenas um personagem a ser mostrado ao usuario, a interface não é exibida e este personagem é automaticamente selecionado. Se não informado, este parâmetro assume o valor false .

Retorno:

- Um Objeto Personagem representando o personagem que o usuário escolheu.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Se o filtro especificado por "filter" não for capaz de apresentar nenhum personagem para escolha, um erro será exibido para o usuário.

Exemplo:

```
local personagem = chooseCharacter("Qual personagem deseja
testar a força?", "pc");

escrever("Você selecionou: " .. personagem.nome);
```

```
function chooseCharacterOfPlayer(prompt, player [, shortCircuit])
```

Exibe um diálogo para que o usuário possa escolher um personagens da lista de personagens de um jogador na mesa.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- player - Uma cadeia de caracteres contendo o login do usuário **ou** um Objeto Jogador representando de qual jogador os personagens deverão ser apresentados na tela.
- (OPCIONAL) shortCircuit - Avaliação Curto-Circuito: Se for "true" e se houver apenas um personagem a ser mostrado ao usuario, a interface não é exibida e este personagem é automaticamente selecionado. Se não informado, este parâmetro assume o valor false .

Retorno:

- Um Objeto Personagem representando o personagem que o usuário escolheu.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Se o jogador não possuir nenhum personagem, um erro é lançado.

Exemplo:

```
local player = choosePlayer("Selecione um jogador");


local personagem = chooseCharacterOfPlayer("Selecione um
personagem", player);

escrever(personagem.nome);


local personagem2 = chooseCharacterOfPlayer("Selecione um
```

```
personagem", "AlyssonRPG");
escrever(personagem2.nome);
```

```
function chooseCharacters(prompt[, filter])
```

Exibe um diálogo para que o usuário possa escolher **um ou mais** personagens da lista de personagens que estão na biblioteca da mesa.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- (OPCIONAL) filter - Uma cadeia de caracteres informando quais persnagens devem ser apresentados na interface de seleção e pode ser um dos seguintes valores:
  - "all" - Todos os personagens da mesa deverão ser apresentados na janela (Opção padrão caso não seja informado filter)
  - "pc" - Apenas os personagens de jogadores deverão ser apresentados na janela.
  - "pcOnline" - Apenas os personagens de jogadores que estão online deverão ser apresentados na janela.
  - "npc" - Apenas os personagens do mestre deverão ser apresentados.
  - "mine" - Apenas os personagens do jogador que executar a macro deverão ser apresentados.

Retorno:

- Um array/arranjo/lista de Objeto Personagem representando os personagens que o usuário escolheu.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Se o filtro especificado por "filter" não for capaz de apresentar nenhum personagem para escolha, um erro será exibido para o usuário.

Exemplo:

```
local personagens = chooseCharacters("Quais personagens
deseja testar a força?", "pcOnline");


for i = 1, #personagens, 1 do
  escrever("Você selecionou: " .. personagens[i].nome);
end;
```

```
function choosePlayer(prompt[, shortCircuit])
```

Exibe um diálogo para que o usuário possa escolher um jogador da lista de jogadores que estão na mesa.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.
- (OPCIONAL) shortCircuit - Avaliação Curto-Circuito: Se for "true" e se houver apenas um jogador a ser mostrado ao usuario, a interface não é exibida e este jogador é automaticamente selecionado. Se não informado, este parâmetro assume o valor false .

Retorno:

- Um Objeto Jogador representando o jogador que o usuário escolheu.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.

Exemplo:

```
local jogador = choosePlayer("De qual jogador voce quer
obter informações?")

escrever("Você selecionou: " .. jogador.login);


if jogador.isMestre then

   escrever("O jogador é mestre na mesa");

end;
```

```
function choosePlayers(prompt)
```

Exibe um diálogo para que o usuário possa escolher **um ou mais** jogadores da lista de jogadores que estão na mesa.

Parâmetros

- prompt - Uma cadeia de caracteres contendo uma orientação ao usuário sobre o que ele deve selecionar.

Retorno:

- Um array/lista de Objeto Jogador representando os jogadores que o usuário escolheu.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.

Exemplo:

```
local jogadores = choosePlayers("Selecione quais jogadores
devem rolar iniciativa");


for i = 1, #jogadores, 1 do

   escrever("Rolar iniciativa de " .. jogadores[i].login);

end;
```

```
function getCharacterOfPlayer(player)
```

Retorna o personagem principal de um jogador da mesa.

Parâmetros

- player - Uma cadeia de caracteres contendo o login do usuário **ou** um Objeto Jogador representando de qual jogador se deve obter o personagem principal.

Retorno:

- Um Objeto Personagem representando o personagem principal do jogador.

Observações:

- A execução da macro é pausada até que o usuário responda a janela.
- Se o usuário cancelar a janela, a macro pára de executar.
- Se o jogador não possuir nenhum personagem, um erro é lançado.
- Se o jogador possuir dois ou mais personagens, o personagem principal dele será retornado.

Exemplo:

```
local player = choosePlayer("Escolha um jogador");

local personagemPrincipal = getCharacterOfPlayer(player);

escrever(personagemPrincipal.nome);
```

```
function getCharacterSheet(character)
```

Retorna o Node Database/sheet de um personagem.

Parâmetros

- character - Um Objeto Personagem representando o personagem de qual se deve obter o NodeDatabase/sheet

Retorno:

- O Objeto Nodo raiz representando o NodeDatabase/Sheet do personagem

Observações:

- A execução da macro pode ser pausada até que o NodeDatabase/Ficha seja carregada pela internet.
- O Objeto Nodo retornado é o mesmo daquela variável "sheet" usada para programar as fichas/lua forms.
- As alterações feitas no objeto nodo retornado são sincronizadas normalmente, confome as regras de permissões do Node Database.
- Se houver algum erro ao carregar o NodeDatabase, a macro pára de executar.

Exemplo:

```
local personagem = chooseCharacter("Escolha um personagem",
"pc")

local node = getCharacterSheet(personagem);


-- Escrever no chat a estrutura dos dados armazenados da
ficha

write(utils.tableToStr(node, true));


-- Se for a ficha D&D 5, a linha abaixo aumenta a CA em 1

node.CA = (tonumber(node.CA) or 0) + 1;
```

```
function getUserNDB(name[, options])
```

Open a per-user NodeDatabase that is physically stored in the Firecast server.

Arguments:

- name - A string identifying which remote user NodeDatabase should be opened

- (OPTIONAL) options - A Lua table describing additional settings that may contain:
  - create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false**

Return:

- Node Object

Remarks:

- The content of NodeDatabases opened by this function is stored separately per-user. Each user will have their own copy of the NodeDatabase identified by the “name” argument

```
function getUserRoomNDB(name[, options])
```

Open a per user NodeDatabase in the current room that is physically stored in the Firecast server.

Arguments:

- name - A string identifying which remote user NodeDatabase in the current room should be opened
- (OPTIONAL) options - A Lua table describing additional settings that may contain:
  - create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false**

Return:

- Node Object

Remarks:

- The content of NodeDatabases opened by this function is stored separately per-user and per-room. Each user in the current room will have their own copy of the NodeDatabase identified by the “name”.

```
function getRoomNDB(name[, options])
```

Open a per room NodeDatabase that is physically stored in the Firecast server.

Arguments:

- name - A string identifying which remote NodeDatabase of the current room should be opened
- (OPTIONAL) options - A Lua table describing additional settings that may contain:
  - create - a boolean. If **true**, the NodeDatabase can be created if it does not exist. Default: **false**

Return:

- Node Object

Remarks:

- The content of NodeDatabases opened by this function is stored separately per-room. All users of the current room will access the same content of the NodeDatabase identified by the “name”.
- By default, users with the +GM mode can read and modify the NodeDatabase of the room while other users can only read values.
- To open a NodeDatabase of the room, one of the following statements must be true:
  - The current user is a Gold subscriber; or
  - The creator of the room is a Platinum subscriber.

```
function invoke(macroName [, parameter])
```

Esta função invoca outra macro e aguarda sua execução.

Parâmetros:

- macroName - Cadeia de caracteres identificando qual macro deve ser invocada.
  - Exemplo: Se deseja invocar a macro /obterBonusAtaque, o valor "obterBonusAtaque" deverá se passado aqui, sem a barra.
- (OPCIONAL) parameter - Cadeia de caracteres contendo os parâmetros que devem ser passados para a macro quando ela for invocada.

Retorno:

- Se a macro invocada tiver sido executada sem erros, o retorno desta função é idêntico ao "return" da macro invocada (**nil** se ela não especificou valor de return).

Observação:

- A execução desta macro é pausada até a outra macro seja executada.
- Se a macro invocada parar de executar (exemplo: Pois o usuário cancelou um inputQuery), esta macro também pára de executar.

Exemplo (duas macros, uma invocando a outra):

Macro: ObterBonusAtaque

```
local attack = inputQuery("Informe o Bônus de
Ataque");

local damageBonus = inputQuery("Informe o Bônus
de Dano");
```

```
return attack, tonumber(damageBonus);
```

Macro que invoca outra macro

```
local bonusAttack, damage =
invoke("ObterBonusAtaque")

write("Ataque informado: " .. bonusAttack)
write("Dano informado: " .. damage);
```

```
function playLocal(address[, volume])
```

Plays an audio file using the user's default audio device.

Arguments:

- address - A string containing the external URL of the audio. Example: "https://www.domain.com/my/audioFile.mp3"
- *(Optional)* volume - A number representing the volume at which the audio should be played. This number functions as a multiplication factor, where 0.0 means total silence, and 2.0 means double the original audio volume. The default value is 1.0.

Return:

- AudioTrack object representing the audio playback

Remarks:

- This function will play the audio from start to end only once.
- This function plays audio to the user only. If you want to transmit it to other players, consider using the playRoom function.

```
function playRoom(address[, volume])
```

Plays an audio file to all players in the room.

Arguments:

- address - A string containing the external URL of the audio. Example: "https://www.domain.com/my/audioFile.mp3"
- *(Optional)* volume - A number representing the volume at which the audio should be played. This number functions as a multiplication factor, where 0.0 means total silence, and 2.0 means double the original audio volume. The default value is 1.0.

Return:

- AudioTrack object representing the audio playback

Remarks:

- This function will play the audio from start to end only once.
- This function transmits audio to everyone in the room. If you want to play only to the local user, consider using the playLocal function.

```
function waitMessage(timeoutMS)
```

Waits for the next message received in the current chat.

Arguments:

- timeoutMS - The duration, in milliseconds, for which the function will wait for a message. This value should be greater than 0.

The function returns two values:

- A boolean indicating if a message has been received within the specified timeout.
- A LogRec table containing details of the received message.This value is valid only if the first return value is **true**.

Example 1:

```
local success, logRec = waitMessage(30000);


if success then
   write("Message received from " .. logRec.entity.login);
else
   write("No message received in time");
end;
```

Example 2 - Waiting for a roll:

```
TIMEOUT = 60000;


while elapsedMS < TIMEOUT do
    local received, logRec = waitMessage(TIMEOUT -
elapsedMS);


    if received and (logRec.msg.msgType == "dice") then
        write(string.format("%s rolled and the result is %d",
```

```
                                logRec.entity.login,
                                logRec.msg.roll.resultado));


            return;
        end;
    end;

    write("No roll in time");
```

```
function write(texto[, quebrarLinha, permitirSmileys])
ou
function escrever(texto[, quebrarLinha, permitirSmileys])
```

Escreve um texto na tela do chat. Este texto não é enviado para ninguém e aparecerá somente no chat do próprio usuário.

Parâmetros:

- texto - Cadeia de caracteres contendo o texto que deseja escrever.
- (OPCIONAL) quebrarLinha - Booleano. O Firecast8 ignora este parâmetro.
- (OPCIONAL) permitirSmileys - Booleano (true ou false) indicando se o Firecast deve interpretar smileys/memes neste texto. Se omitido, o valor padrão true é assumido.

Exemplo:

```
escrever("Olá
mundo!!!");
```

```
function writeEx(text[, talemarkOptions])
```

Write/print a text in the chat. The text is not sent; only the local user can see the text.

Arguments:

- text - A string containing the text to be printed.
- *(Optional)* talemarkOptions - A TalemarkOptions table containing how Firecast should color and format the text. If missing, Firecast will use a default TalemarkOptions.

Examples:

```
writeEx("Hello World!");
writeEx("Hello World!", {defaultTextStyle = {color = "red"}});
```

APÊNDICE

A linguagem de programação LUA

LUA é uma linguagem de programação interpretada/script poderosa, rápida e leve, projetada por **brasileiros** para estender aplicações e é usada vastamente pelo mundo (inclusive em projetos famosos como o jogo World of Warcraft, por exemplo).

A versão utilizada da linguagem LUA no SDK 3 é a 5.3

Segue abaixo uma lista de links úteis para o aprendizado da linguagem LUA:

- Site oficial da linguagem LUA: http://www.lua.org/
- Manual de referência da linguagem: http://www.lua.org/manual/5.3/
- LUA na Wikipedia: http://pt.wikipedia.org/wiki/Lua_%28linguagem_de_programa%C3%A7%C3%A3o%29
- Livros sobre Lua: http://www.lua.org/pil/

Observação importante:

Visando a proteção dos usuários contra plug-ins maliciosos, os códigos LUA rodam em uma sandbox e nem todas as funções nativas do LUA estão disponíveis para o plug-in SDK 3.

Saiba mais sobre sandboxes e sobre as funções nativas LUA disponíveis lendo o tópico: Sandboxes de Plug-ins

Veja também:

O SDK3 já provê algumas bibliotecas e funções LUA para você utilizar em seus plug-ins! Saiba mais em Bibliotecas LUA.

Documentos XML

XML são documentos usados para guardar e catalogar informações de forma intuitiva e flexível, e servem para diversas finalidades!

Importante: Os documentos XMLs possuem "um rosto" diferente para cada finalidade. Exemplo: Enquanto um XML de interface possui um conjunto de informações e marcações específicas, um XML de informações do plugin possui **outro** conjunto **distinto** de informações e marcações específicas. Haverá uma documentação própria para guiar seus passos em cada uma dessas finalidades!

Links úteis:

- XML na wikipedia: http://pt.wikipedia.org/wiki/XML
- Introdução ao XML: http://www.macoratti.net/xml.htm

Se você está tendo problemas com os documentos XML, eis aqui algumas dicas:

- Certifique que as marcações/tags abertas estão sendo fechadas corretamente. Nomes de marcações são case-sensitives e precisam ser exatamente iguais à marcação de abertura.
- Certifique que as marcações usadas são válidas para a finalidade desejada.
- Alguns caracteres são reservados para controle do XML. Se você precisar deles mesmo assim, é preciso fazer substituição por um código especial. Veja Lidando com caracteres reservados do XML.
- Em caso de erro relacionado à codificação do texto, cerifique-se que o documento possui um cabeçalho XML identificando corretamente a codificação do documento. O cabeçalho é a primeira linha do documento, onde:
  - Se o documento estiver codificado em UTF-8, o cabeçalho deve ser:
    - <?xml version="1.0" encoding="UTF-8"?>
  - Se você não sabe a codificação do documento, talvez o cabeçalho correto seja:
    - <?xml version="1.0" encoding="ISO-8859-1"?>

Lidando com caracteres reservados do XML

Alguns caracteres são reservados para controle do XML. Se você precisar deles mesmo assim, é preciso fazer substituição por um código especial:

Tabela de caracteres reservados:

Caractere reservado	Substituição
<	<
>	&gt

&	&
‘	'
“	"

Exemplos de substituição:

- <label text=”D&D”/>
  Substituição de "&" por "&" ... Contém "D&D"

- <button text=”20 > 10”/>
  Substituição de ">" por ">" .... Contém "20 > 10"

CDATA

Você também pode utilizar blocos CDATA para evitar fazer substituições, o que é útil em códigos LUA dentro de XML.
Saiba mais em http://en.wikipedia.org/wiki/CDATA

Exemplo de CDATA

```
<event name="onResize">
        <![CDATA[
                local teste = "D&D";

                if (x < y) or (x > z) then
                        local teste = 'D&D 4.0';
                end;
        ]]>
</event>
```

Não é preciso fazer nenhuma substituição de caracteres reservados dentro de um bloco CDATA.

Observação: Só é possível utilizar CDATA no corpo de um nodo do XML. Não é possível usar CDATA no valor de um atributo, por exemplo.

FAQ - Perguntas Frequentes

1. **O SDK 3 precisa do Delphi para funcionar?**

Não. O SDK3 é uma nova ferramenta que utiliza uma nova tecnologia de plug-ins para o RRPG. Ela não exige ter o Delphi instalado.

1. **Tenho uma ficha feita no Delphi. O SDK3 é compatível com esta minha ficha?**

Não. As fichas feitas em Delphi usam o SDK 2, que é incompatível com o SDK 3.

1. **Quais as vantagem do SDK 3 sobre o SDK 2 + Delphi?**
    - Os plug-ins do SDK 2 rodam apenas no Windows, enquanto os feitos no SDK 3 podem rodar em outros dispositivos.
    - Você não precisa baixar o Delphi (um aplicativo grande e pago)
1. **Existe alguma ferramenta que converte os plug-ins feitos no SDK2 para a versão SDK3?**

Até o momento, não. Você precisará portar o plug-in "do zero". Se você pretende criar um modelo de ficha baseado em um PDF editável, você pode utilizar a ferramenta de conversão PDF -> Ficha SDK3. Saiba mais lendo Criar um modelo de ficha à partir de um PDF.

1. **Desenvolvi um plug-in usando o SDK 3. Quem for usar meu plug-in precisa ter o SDK3 instalado também?**

Não. O SDK 3 é uma ferramenta para quem desenvolve plug-ins. Os usuários do plug-in precisam apenas ter o RRPG Firecast mais atual instalado.

1. **A ferramenta que converte um PDF editável em uma ficha também converte os cálculos que existem no PDF?**

Não. Você precisa recriar os cálculos e scripts manualmente. Veja o tutorial Criar campos calculados em fichas caso tenha dúvidas sobre como fazer isso.